University
Politehnica
of Bucharest

PL/SQL (I)

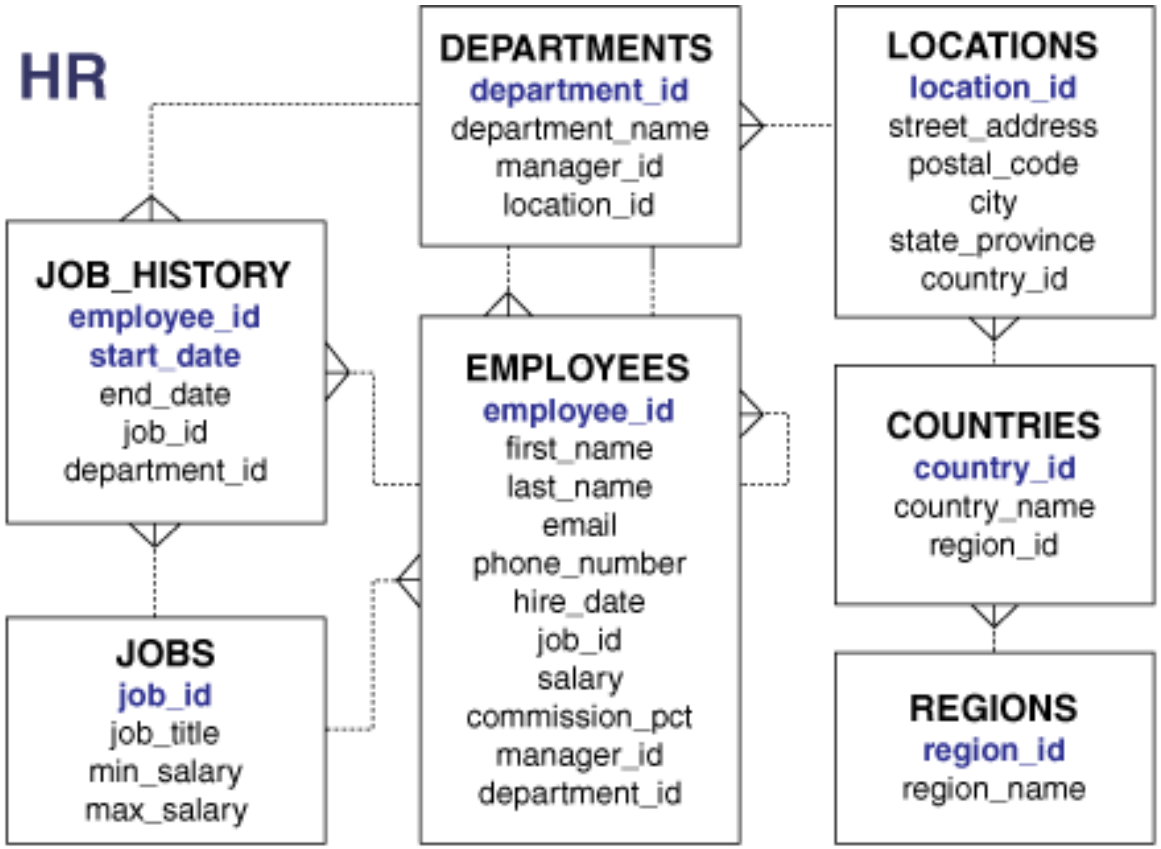
Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@upb.ro

Overview

- PL/SQL
- PL/SQL Main Features
- PL/SQL Engine

Examples

- Database



Overview

- PL/SQL
- PL/SQL Main Features
- PL/SQL Engine

PL/SQL

- PL/SQL (Procedural Language/SQL)
  - Is the Oracle procedural extension of SQL
  - Is a portable, high-performance transaction-processing language
  - Is a proprietary language

PL/SQL

- Advantages of PL/SQL:
  - Tight Integration with SQL
  - High Performance
  - High Productivity
  - Portability
  - Scalability
  - Manageability
  - Support for Object-Oriented Programming

PL/SQL
Tight Integration with SQL

- PL/SQL supports
  - SQL data manipulation statements DML
  - Cursor control statements (OPEN, LOOP, CLOSE)
  - Transaction control statements TCL

PL/SQL
Tight Integration with SQL

- DML (Data Manipulation Language)
  - SELECT
  - INSERT
  - UPDATE
  - DELETE
- TCL (Transaction Control Language)
  - COMMIT
  - ROLLBACK
  - SAVEPOINT

PL/SQL
Tight Integration with SQL

- PL/SQL supports
  - All Oracle SQL Functions
  - All SQL Operators
  - Pseudocolumns
    - A **pseudocolumn** behaves like a table column but is not actually stored in the table
    - E.g., COLUMN_VALUE, **ROWID**, **ROWNUM**, XMLDATA

PL/SQL
Tight Integration with SQL

- PL/SQL supports all SQL data types
  - No need for conversions between PL/SQL and SQL data types
  - Attributes for data type correlation
    - %ROWTYPE attribute to declare a record that represents either a full or partial row of a database table or view
    - %TYPE attribute to declare a data item of the same data type as a previously declared variable or column

PL/SQL
Tight Integration with SQL

- PL/SQL supports
  - Running SQL query
  - Processing the rows of the result set one at a time

- PL/SQL functions can be declared and defined in the WITH clauses of SQL SELECT statements (CTE – Common Table Expressions)

Examples

```sql
with no_emps as (
    select department_id, count(*) c
    from employees
    group by department_id
)
select d.department_name, q.c  emps
from departments d
    inner join no_emps q
        on q.department_id = d.department_id
order by 1;

-- Equivalent

select
    d.department_name,
    count(e.employee_id) emps
from departments d
    inner join employees e
        on d.department_id=e.department_id
group by d.department_name
order by 1;
```

PL/SQL
Tight Integration with SQL

- PL/SQL supports
  - Static SQL
    - SQL whose full text is known at compile time
  - Dynamic SQL
    - SQL whose full text is not known until run time
    - Make applications more
      - Flexible
      - Versatile

PL/SQL
Tight Integration with SQL

- Dynamic SQL Drawbacks
  - Performance loss
    - The execution plan for dynamic queries cannot be cached
  - Hard to debug
    - The error management becomes more unreliable.
    - Maintenance is difficult because the schema is hard coded in the dynamic code.
  - Security can be compromised with SQL injection

PL/SQL
High Performance

- PL/SQL
  - Statements can be organized into blocks (BEGIN … END)
  - Blocks significantly reduce traffic between the application and the database.
- High performance is achieved through:
  - Bind Variables
  - Subprograms
  - Optimizer

PL/SQL
High Performance

- Bind Variables
  - Are placeholders for actual values in SQL statements
  - Allow the database server to prepare the statement once and execute it multiple times without reparsing or reanalyzing it.
  - The PL/SQL compiler turns the variables in the WHERE and VALUES clauses into bind variables when embedding a SQL data manipulation statements directly in a block
  - Oracle Database can reuse these SQL statements each time the same code runs
  - PL/SQL does not create bind variables automatically when you use dynamic SQL
    - They can be used with dynamic SQL by specifying them explicitly

PL/SQL
High Performance

- PL/SQL supports subprograms:
  - Functions
  - Procedures
- Subprograms
  - Are stored in executable form
  - Can be invoked repeatedly
  - Are cached lowering memory requirements
  - Are shared among users lowering invocation overhead
  - A single invocation over the network can start a large job
    - Reducing network traffic
    - Improving response times
- Optimizer
  - The PL/SQL compiler has an optimizer that can rearrange code for better performance.

PL/SQL
High Productivity

- PL/SQL
  - Is the same in all environments
  - Can be used to write compact code for manipulating data
  - Is a scripting language
  - Can query, transform, and update data in a database
  - Supported by many other Oracle tools (Forms Builder, Apex, etc.)

PL/SQL Portability

- PL/SQL is a portable and standard language for Oracle development
- PL/SQL applications can be run on any operating system and platform where Oracle Database runs
- PL/SQL applications can be run on different environments (e.g., server, client application, etc.)

PL/SQL Scalability

- PL/SQL stored subprograms increase scalability by centralizing application processing on the database server
- The shared memory facilities of the shared server let Oracle Database support thousands of concurrent users on a single node
- Oracle Connection Manager can be used to multiplex network connections

PL/SQL Manageability

- PL/SQL stored subprograms increase manageability:
  - Only one copy of a subprogram can be maintained on the database server
  - Rather than one copy on each client system
- Any number of applications can use the subprograms
- Subprograms can be changed without affecting the applications that invoke them

PL/SQL
Support for Object-Oriented Programming

- PL/SQL supports OOP
  - Allows defining object types that can be used in object-oriented designs
  - Abstract Data Types (ADT):
    - Consists of a data structure and subprograms that manipulate the data
    - The variables that form the data structure are called attributes.
    - The subprograms that manipulate the attributes are called methods.

Overview

- PL/SQL
- PL/SQL Main Features
- PL/SQL Engine

PL/SQL Main Features

- PL/SQL combines the data-manipulating power of SQL with the processing power of procedural languages
- SQL statements can be issued from a PL/SQL program
- Complex problems can be broken into easily understandable subprograms
- Subprograms can be reused in multiple applications

PL/SQL Main Features

- PL/SQL is a procedural language
- PL/SQL supports the declaration of
  - Constants
  - Variables
- Using PL/SQL, a developer can
  - Control program flow
  - Define subprograms
  - Handle runtime errors

PL/SQL Main Features

- PL/SQL provides:
  - Blocks
  - Variables and Constants
  - Error Handling
  - Subprograms
  - Packages
  - Triggers
  - Input and Output
  - Data Abstraction
  - Control Statements
  - Conditional Compilation
  - Processing a Query Result Set One Row at a Time

PL/SQL Main Features
Blocks

- The basic unit of a PL/SQL source program is the block
- The block groups related declarations and statements.
- A PL/SQL block is defined by the keywords:
  - DECLARE - declarative part
  - BEGIN - an executable part
  - EXCEPTION - an exception-handling part
  - END.
- Only the executable part is required (BEGIN ... END)
- A block can have a label.

PL/SQL Main Features
Blocks

```
<< label >> (optional)
DECLARE    -- Declarative part (optional)
  -- Declarations of local types, variables, & subprograms

BEGIN      -- Executable part (required)
  -- Statements (which can use items declared in declarative part)

[EXCEPTION -- Exception-handling part (optional)
  -- Exception handlers for exceptions (errors) raised in executable part]
END;
```

PL/SQL Main Features
Blocks

- Between DECLARE and BEGIN
  - Declaration part of the block
  - A developer can declare here:
    - Variables
    - Cursors
    - User-defined data types
    - User-defined exceptions
    - Subprograms (Functions, Procedures)
  - This part is optional

PL/SQL Main Features
Blocks

- Between BEGIN and EXCEPTION
  - Executable part of the block
  - Contains
    - Executable statements
    - SQL queries
    - Control structures
    - Loop structures
  - The EXECEPTION part of the block is optional
    - In this case, the block ends with the key-word END.

PL/SQL Main Features
Blocks

- Between EXCEPTION and END
  - Exception handler part of the block
  - This part is optional

PL/SQL Main Features
Blocks

- Block example:

```
BEGIN
    null;
END;
/
```

PL/SQL Main Features
Blocks

- There should be at least on instruction in the execution part of the block (BEGIN … END)
- Each instruction in a PL/SQL block ends with a semicolon (**;**)
- There is no semicolon after DECLARE, BEGIN and EXCEPTION
- There is a semicolon after END
- To run the block in SQL*Plus use:
  - Slash (/)
  - Run (r)

Examples

```
SET AUTOCOMMIT OFF

BEGIN
  INSERT INTO countries
    VALUES('RO', 'Romania', 1);
  INSERT INTO locations(location_id, city, country_id)
    VALUES(4000, 'Bucuresti', 'RO');
  INSERT INTO departments(department_id, department_name, location_id)
    VALUES(500, 'IT Bucuresti', 4000);
  COMMIT;
END;
/
```

Examples

```sql
SELECT * FROM countries
  WHERE country_id='RO';

SELECT * FROM locations
  WHERE location_id=4000;

SELECT * FROM departments
  WHERE department_id=500;
```

PL/SQL Main Features
Blocks

- Declarations are local to the block
- Declarations cease to exist when the block completes execution
  - This helps to avoid cluttered namespaces for variables and subprograms
- Blocks can be nested:
  - A block is an executable statement
  - A block can appear in another block wherever an executable statement is allowed.

PL/SQL Main Features
PL/SQL Nested Blocks

```
 1 [<<label>>] -- block label (optional)
 2 [DECLARE] -- optional
 3   -- variables for all the blocks
 4 BEGIN -- start of parent block
 5     [DECLARE] -- optional
 6       -- variables for child_1 block
 7     BEGIN -- start of child_1 block
 8       -- can access variables from parent and child_1 blocks
 9     [EXEPTION] -- optional
10       -- exception handler for child_1 block
11     END; -- end of child_1 block
12
13     [DECLARE] -- optional
14       -- variables for child_2 block
15     BEGIN -- start of child_2 block
16       -- can access variables from parent and child_2 blocks
17     [EXCEPTION] -- optional
18       -- exception handler for child_2 block
19     END; -- end of child_2 block
20 [EXCEPTION]  -- optional
21   -- exception handler for parent block and
22   -- any unhandled exception from the child_1 or child_2 blocks
23 END;
24 /
```

PL/SQL Main Features
PL/SQL Nested Blocks

```
[<<label>>] -- block label (optional)
[DECLARE] -- optional
  -- variables for all the blocks
BEGIN -- start of parent block

    [DECLARE] -- optional
        -- variables for child_1 block
    BEGIN -- start of child_1 block
        -- can access variables from parent and child_1 blocks

        [DECLARE] -- optional
           -- variables for child_2 block
        BEGIN -- start of child_2 block
           -- can access variables from parent, child_1 and child_2 blocks

           [DECLARE] -- optional
               -- variables for child_3 block
           BEGIN -- start of child_3 block
               -- can access variables from parent and child_3 blocks
           [EXCEPTION] -- optional
               -- exception handler for child_3 block
           END; -- end of child_3 block


        [EXCEPTION] -- optional
          -- exception handler for child_2 block
          -- handler unhandler exception from child_3 block
        END; -- end of child_2 block


    [EXEPTION] -- optional
      -- exception handler for child_1 block
      -- handler unhandler exception from child_2 and child_3 blocks
    END; -- end of child_1 block

[EXCEPTION]  -- optional
  -- exception handler for parent block and
  -- handler unhandled exception from child_1, child_2 and child_3 blocks
END;
/
```

PL/SQL Main Features
Blocks

- A block can be
  - Submitted to an interactive tool (such as SQL*Plus or Enterprise Manager)
  - Embedded in an Oracle Precompile
  - Embedded in an OCI (Oracle Call Interface) program
- The interactive tool or program runs the block one time

PL/SQL Main Features
Blocks

- The block is not stored in the database
- It is called an anonymous block (even if it has a label)
- An anonymous block is compiled each time it is loaded into memory
- An anonymous block compilation has three stages:
  1. Syntax checking: PL/SQL syntax is checked, and a parse tree is generated.
  2. Semantic checking: Type checking and further processing on the parse tree.
  3. Code generation

Examples

```
1 BEGIN
2   UPDATE locations
3     SET postal_code = '252525',
4     state_province = 'Sector 7'
5     WHERE location_id = 4000;
6   COMMIT;
7 END;
8 /
```

PL/SQL Main Features
Variables and Constants

- In the DECLARATION part of a PL/SQL block there can be declared
  - Variables
  - Constants
- They can be use them wherever you can use an expression
- As the program runs
  - The values of variables can change
  - The values of constants cannot

PL/SQL Main Features
Variables and Constants

- Example

```
DECLARE
    A NUMBER(5,2);
    PI CONSTANT NUMBER(3,2) := 3.14;
BEGIN
    A := PI * &r ** 2;
END;
/
```

PL/SQL Main Features
Error Handling

- PL/SQL supports error handlers to efficiently detect errors

- When an error occurs:
  - An exception is raised
  - Normal execution stops
  - The control transfers to the exception-handling part of the PL/SQL block.

PL/SQL Main Features Subprograms

- A PL/SQL subprogram is a named PL/SQL block that can be invoked repeatedly
- If the subprogram has parameters, their values can differ for each invocation
- PL/SQL has two types of subprograms
  1. Procedures – does not return a result
  2. Functions – returns a result
- PL/SQL also lets you invoke external programs written in other languages

PL/SQL Main Features
Packages

- A PL/SQL subprogram is a named PL/SQL block that can be invoked repeatedly

- A package is a schema object that groups logically related PL/SQL
  - Types
  - Variables
  - Constants
  - Subprograms
  - Cursors
  - Exceptions

PL/SQL Main Features
Packages

- A package is compiled and stored in the database
- Applications can share package
- Developers can write their own packages
- Oracle provides many product-specific packages

PL/SQL Main Features
Triggers

- A trigger
  - Is a named PL/SQL unit
  - Is stored in the database
  - Runs in response to an event that occurs in the database

PL/SQL Main Features
Triggers

- A developer can specify
  - The event to be handled by the trigger
  - Whether the trigger fires
    - Before the event
    - After the event
  - Whether the trigger runs for each:
    - Event
    - Row affected by the event

PL/SQL Main Features
Input and Output

- Most PL/SQL input and output (I/O) is done with SQL statements
- These statements
  - Store data in database tables
  - Query those tables
- PL/SQL I/O is also done with PL/SQL packages that Oracle Database supplies

PL/SQL Main Features
Input and Output

Package	Description	More Information
DBMS_OUTPUT	Lets PL/SQL blocks, subprograms, packages, and triggers display output. Especially useful for displaying PL/SQL debugging information.	*Oracle Database PL/SQL Packages and Types Reference*
HTF	Has hypertext functions that generate HTML tags (for example, the HTF.ANCHOR function generates the HTML anchor tag <A>).	*Oracle Database PL/SQL Packages and Types Reference*
HTP	Has hypertext procedures that generate HTML tags.	*Oracle Database PL/SQL Packages and Types Reference*
DBMS_PIPE	Lets two or more sessions in the same instance communicate.	*Oracle Database PL/SQL Packages and Types Reference*
UTL_FILE	Lets PL/SQL programs read and write operating system files.	*Oracle Database PL/SQL Packages and Types Reference*
UTL_HTTP	Lets PL/SQL programs make Hypertext Transfer Protocol (HTTP) callouts, and access data on the Internet over HTTP.	*Oracle Database PL/SQL Packages and Types Reference*
UTL_SMTP	Sends electronic mails (emails) over Simple Mail Transfer Protocol (SMTP) as specified by RFC821.	*Oracle Database PL/SQL Packages and Types Reference*

PL/SQL Main Features
Data Abstraction

- A developer can specify their own data types
- Data structures can be used to achieve data abstraction
- Developers can use:
  - Cursors
  - Composite Variables
  - The %ROWTYPE Attribute
  - The %TYPE Attribute
  - Abstract Data Types

PL/SQL Main Features
Data Abstraction

- A cursor
  - Is a pointer to a private SQL area that stores information about processing
    - a specific SQL statement
    - PL/SQL SELECT INTO statement
  - Is used to retrieve the rows of the result set one at a time.
- Cursor attributes can be used to get information about the state of the cursor (e.g., how many rows the statement has affected so far)

PL/SQL Main Features
Data Abstraction

- A composite variable has internal components
- An internal component can be accessed individually
- Entire composite variables can be passed to subprograms as parameters

PL/SQL Main Features
Data Abstraction

- PL/SQL has two kinds of composite variables:
  - Collections
    - The internal components (elements) are always of the same data type
    - Each element is accessed by its unique index.
    - E.g., lists and arrays
  - Records
    - The internal components (fields) can be of different data types
    - Each field can be accessed by its name
    - A record variable can hold
      - A table row
      - Some columns from a table row.

PL/SQL Main Features
Data Abstraction

- The %ROWTYPE attribute
  - Is used to declare a record that represents either
    - A full row of a database table or view
    - Partial row of a database table or view

- The %TYPE attribute
  - Is used to declare a data item of the same data type as a previously declared variable or column

Examples

```
SET SERVEROUTPUT ON

DECLARE
  dn departments.department_name%TYPE;
  idd departments.department_id%TYPE := 500;
BEGIN
  SELECT department_name INTO dn
    FROM departments
    WHERE department_id = idd;
  DBMS_OUTPUT.PUT_LINE('Department id:   ' || idd);
  DBMS_OUTPUT.PUT_LINE('Department name: ' || dn);
END;
/
```

Examples

```
SET SERVEROUTPUT ON

DECLARE
  dn departments.department_name%TYPE;
  idd departments.department_id%TYPE := &id;
BEGIN
  SELECT department_name INTO dn
    FROM departments
    WHERE department_id = idd;
  DBMS_OUTPUT.PUT_LINE('Department id:   ' || idd);
  DBMS_OUTPUT.PUT_LINE('Department name: ' || dn);
END;
/
```

Examples

```
SET SERVEROUTPUT ON

DECLARE
  dn departments.department_name%TYPE;
BEGIN
  SELECT department_name INTO dn
    FROM departments
    WHERE department_id = &&idd;
  DBMS_OUTPUT.PUT_LINE('Department id:   '||&idd);
  DBMS_OUTPUT.PUT_LINE('Department name: '||dn);
END;
/

define
undef idd
```

Examples

```
SET SERVEROUTPUT ON

DECLARE
  country countries%ROWTYPE;
BEGIN
  SELECT * INTO country
    FROM countries
    WHERE country_id = &id;
  DBMS_OUTPUT.PUT_LINE('Country id:   '|| country.country_id);
  DBMS_OUTPUT.PUT_LINE('Country name: '|| country.country_name);
END;
/
```

PL/SQL Main Features
Data Abstraction

- An Abstract Data Type (ADT) consists of a data structure and subprograms that manipulate the data
- The variables that form the data structure are called attributes
- The subprograms that manipulate the attributes are called methods
- ADTs are stored in the database
- Instances of ADTs can be
  - Stored in tables
  - Used as PL/SQL variables.

PL/SQL Main Features
Data Abstraction

- ADTs reduce complexity by separating a large system into logical reusable components
- ADTs are created using CREATE TYPE Statement.
- ADTs are also called user-defined types and object types

PL/SQL Main Features
Conditional Compilation

- Conditional compilation enables developers to customize the functionality in a PL/SQL application without removing source text

- For example:
  - Use new features with the latest database release
  - Disable new features when running the application in an older database release
  - Activate debugging or tracing statements in the development environment
  - Hide debugging or tracing statements when running the application at a production site

PL/SQL Main Features

Processing a Query Result Set One Row at a Time

- Inside PL/SQL, the results of a SQL query can be processed one row a time
- This can be achieved by using:
  - A basic loop
  - An individual statements to
    - Run the query
    - Retrieve the results
    - Finish processing

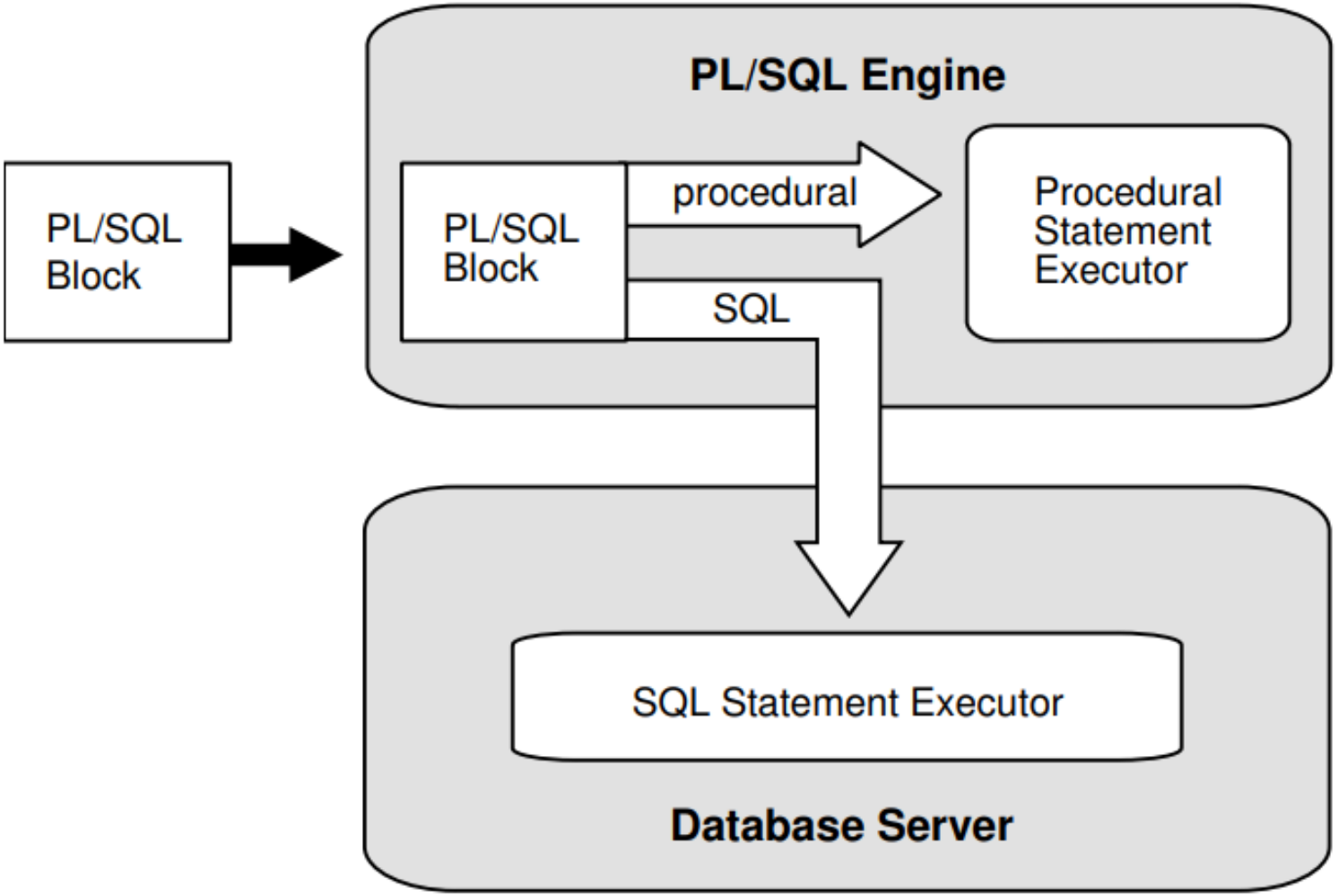
Overview

- PL/SQL
- PL/SQL Main Features
- PL/SQL Engine

PL/SQL Engine

- The PL/SQL compilation and runtime system is an engine
- The PL/SQL engine compiles and runs PL/SQL units
- The PL/SQL engine can be installed:
  - In the database
  - In an application development tool (e.g., Oracle Forms)
- Regardless of the environment, the PL/SQL engine accepts as input any valid PL/SQL unit

PL/SQL Engine



PL/SQL Engine

- The engine
  - Runs procedural statements
  - Sends SQL statements to the SQL engine in the database
  - PL/SQL units are processed by:
    - The database if the database PL/SQL engine is used
    - The local PL/SQL engine if an application development tool is used
  - If a PL/SQL unit contains no SQL statements
    - The local engine processes the entire PL/SQL unit
    - This is useful if the application development tool can benefit from conditional and iterative control

PL/SQL Engine

- Why using PL/SQL Engine is useful:
  - Oracle Forms applications frequently use SQL statements to
    - Test the values of field entries
    - Do simple computations
  - By using PL/SQL instead of SQL
    - Avoid calls to the database

Examples

```
 1 DECLARE
 2   country countries%ROWTYPE;
 3   region regions%ROWTYPE;
 4 BEGIN
 5   SELECT
 6     c.country_id,
 7     c.country_name,
 8     r.region_id,
 9     r.region_name
10   INTO
11     country.country_id,
12     country.country_name,
13     region.region_id,
14     region.region_name
15   FROM countries c
16     INNER JOIN regions r
17       on c.region_id = r.region_id
18     WHERE country_id = &id;
19   DBMS_OUTPUT.PUT_LINE('Country id:   '|| country.country_id);
20   DBMS_OUTPUT.PUT_LINE('Country name: '|| country.country_name);
21   DBMS_OUTPUT.PUT_LINE('Region id:    '|| region.region_id);
22   DBMS_OUTPUT.PUT_LINE('Region name:  '|| region.region_name);
23 END;
24 /
```

Examples

```
1 BEGIN
2   DELETE FROM departments
3     WHERE department_id = 500;
4   DELETE FROM locations
5     WHERE location_id = 4000;
6   DELETE FROM countries
7     WHERE country_id = 'RO';
8   COMMIT;
9 END;
10 /
11
```

Bibliography

- Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 19c*, Oracle Corporation, 2022 [pdf]
- Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 21c*, Oracle Corporation, 2022 [pdf]
- Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 23ai*. Oracle Corporation 2024 [pdf]
- Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 19c*, Oracle Corporation, 2020 [pdf]
- Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 21c*, Oracle Corporation, 2021 [pdf]
- Sarah Hirschfeld et al. *Oracle® Database: Database PL/SQL Language Reference 23ai*. Oracle Corporation 2024 [pdf]

PL/SQL (II)

Ș.L. Dr. Ing. Ciprian-Octavian Truică

ciprian.truica@upb.ro

As. Drd. Ing. Alexandru Petrescu

Alex.petrescu@upb.ro

Overview

Variables and constants

Data types

Operators

Control statements

Variables

```
v_valoare NUMBER(15) NOT NULL := 0;
v_data_achizitie DATE DEFAULT SYSDATE;
v_material VARCHAR2(15) := 'Matase';
c_valoare CONSTANT NUMBER := 100000;
v_stare VARCHAR2(20) DEFAULT 'Buna';
v_clasificare BOOLEAN DEFAULT FALSE;
int_an_luna INTERVAL YEAR TO MONTH := INTERVAL '3-2' YEAR TO
MONTH;
```

variable_name [CONSTANT] datatype [NOT NULL] [:= | DEFAULT initial_value]

- When declaring a **variable**, **storage space** is allocated for a value of a specified **data type**
- The **storage location** is **marked** with a name so it can be **referenced**
- Objects must be declared **before** referencing them
- Declarations can appear in the **declarative** part of any Block / Subprogram / Package

Datatypes:

- **Scalar**: Contain a **single** value (No internal components)
- **Composit**: **Collections** / **Records**

variable_name [CONSTANT] datatype [NOT NULL] [:= | DEFAULT initial_value]

- **variable_name** - Name of the variable that you are declaring
- **CONSTANT –** Imposes that the value cannot be changed after assignment
- **datatype** - Name of a scalar data type, including any qualifiers for size, precision, and character or byte semantics
- **NOT NULL** - Imposes the NOT NULL constraint on the variable
- **:= / DEFAULT –** Assigning the initial value
- **initial_value** – Immediate value or expression

Variables

```
v_valoare NUMBER(15) NOT NULL := 0;
v_data_achizitie DATE DEFAULT SYSDATE;
v_material VARCHAR2(15) := 'Matase';
c_valoare CONSTANT NUMBER := 100000;
v_stare VARCHAR2(20) DEFAULT 'Buna';
v_clasificare BOOLEAN DEFAULT FALSE;
int_an_luna INTERVAL YEAR TO MONTH := INTERVAL '3-2' YEAR TO
MONTH;
```

NOT NULL Constraint

An item can acquire this **constraint** either **Implicitly** (from its data type) or **Explicitly**

The **default initial value** for a scalar variable is NULL.

A scalar variable declaration that specifies **NOT NULL** (either implicitly or explicitly) **must** assign **an initial value** to the variable.

PL/SQL treats any **zero-length string** as a **NULL** value, this includes:

- Values returned by character functions
- BOOLEAN expressions.

Initial Values of Variables and Constants

If the declaration is

- **A variable**: the initial value is optional unless the NOT NULL constraint is specified
- **A constant**: the initial value is required

If the declaration is in a **block** or **subprogram**:

- The initial value is assigned to the variable or constant **every time** control passes to the block or subprogram

If the declaration is in a **package specification**

- The initial value is assigned to the variable or constant for **each session** (whether the variable or constant is public or private)

Initial Values of Variables and Constants

To specify the **initial value** use

- The assignment operator (**:=**) followed by an expression
- The keyword **DEFAULT** followed by an expression

The expression can include **previously declared**

- **Constants**
- **Initialized variables**
- **Functions results**

If **no initial value** is specified for a variable, then a value **must be assigned before using it** in any other context.

Example

```
DECLARE
    no_months constant BINARY_INTEGER:= 12;
    emp_name varchar2(45);
    emp_salary employees.salary%TYPE;
    anual_salary float(10);
BEGIN
    SELECT first_name || ' ' || last_name, salary
    INTO emp_name, emp_salary
    FROM employees WHERE employee_id = &id;

    anual_salary := emp_salary * no_months;
    dbms_output.put_line(rpad(emp_name, 20) || '-' || lpad(anual_salary, 10));
END;
/
```

Data Types

Initial Values of Variables and Constants

Every PL/SQL **constant**, **variable**, **parameter**, and **function** (block that returns a value) has a data type.

The data type determines

- The storage format
- Valid values
- Operations

The data type can be assiged

- **Explicit** – by naming the data type used (e.g. **NUMBER**, **VARCHAR2**, etc.)
- **Implicit** – by using attributes (**%TYPE**, **%ROWTYPE**)

Find YOUR Type
(using %TYPE)

Declaring **Variables** using **%TYPE**

The **%TYPE** attribute permits the declaration of a data item of the same data type as a previously declared **variable** or **column** (**without** knowing what that type is).

If the declaration of the **referenced** item changes, then the declaration of the **referencing** item changes accordingly.

The syntax of the declaration is: referencing_item referenced_item**%TYPE**;

Declaring **Variables** using **%TYPE**

The referencing item **inherits** the following from the **referenced** item:

- **Data type** and **size**
- **Constraints** (unless the referenced item is a column)

The referencing item **does not inherit** the initial **value** of the referenced item: If the referencing item specifies or inherits the **NOT NULL** constraint, an initial **value** for it **must be specified**.

The **%TYPE** attribute is particularly useful when declaring variables to hold **database values.**

Example

```
DECLARE
    a INT NOT NULL := 4;
    b a%TYPE := 3;

    c employees.employee_id%TYPE := 4;
    d c%TYPE default 3;
BEGIN
    dbms_output.put_line(a);
    dbms_output.put_line(b);

    dbms_output.put_line(c);
    dbms_output.put_line(d);
END;
/
```

Scalar data types

- Scalar data types store values with **no internal components**
- A scalar data type can have **subtypes**
- A **subtype** is a data type that is a **subset** of another data type (its base type)
- A **subtype** has the same valid **operations** as its base type
- A data type and its subtypes comprise a **data type family**
- PL/SQL **predefines** many types and subtypes
- The **predefined** data types are available in the **STANDARD package**
- Developers can also define **their own subtypes** (**UDST** – User Defined Subtypes)

Scalar data types

The PL/SQL **scalar** data types are:

- The SQL data types
- BOOLEAN
- PLS_INTEGER
- BINARY_INTEGER
- REF CURSOR
- User-defined subtypes

SQL data types

- **Character** Data Types
- **Number** Data Types
- **Datetime** and **Interval** Data Types
- **RAW** and **LONG RAW** Data Types
- Large Object (**LOB**) Data Types
- **Rowid** Data Types

E

Character Data Types

CHAR : specifies a **fixed**-length character string in the database character set

VARCHAR2 : specifies a **variable**-length character string in the database character set.

ASCII VS **Unicode**

NCHAR : specifies a **fixed**-length character string in the national character set

NVARCHAR2 : specifies a **variable**-length character string in the national character set.

6
24
4
4
6
6
4
24

Number Data Types

The NUMBER data type stores zero as well as positive and negative fixed **numbers**.

NUMBER(p, s)

- p is the **precision**: the **maximum** number of **significant decimal digits**
- s is the **scale**: the number of digits from the **decimal point** to the least significant digit.

Number Data Types

The **FLOAT** data type is a **subtype** of **NUMBER**. It can be specified with or without **precision**. Scale cannot be specified, is interpreted from the data.

FLOAT(p) - p is the precision: the maximum number of **significant decimal digits**

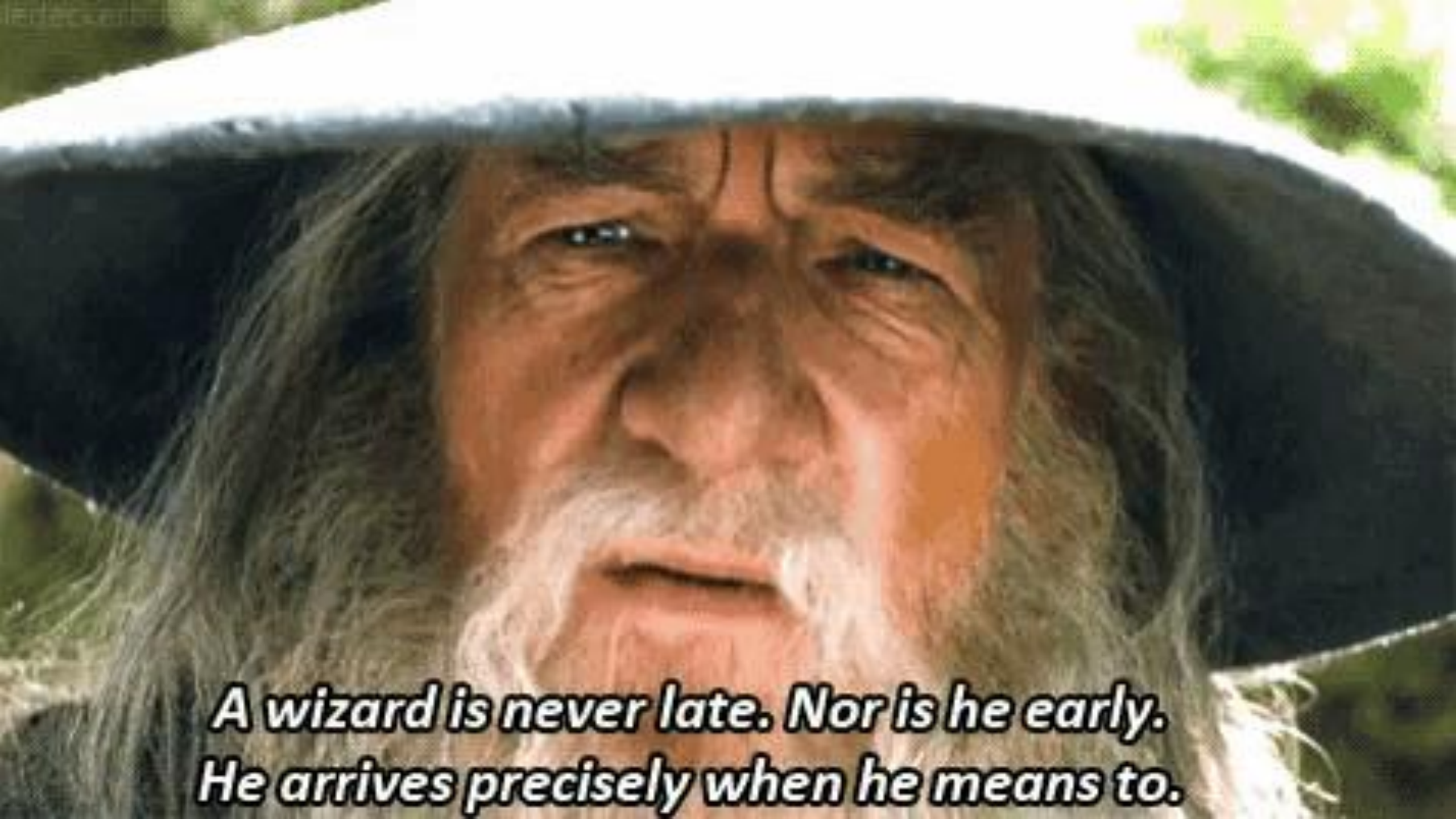
Floating-point numbers can have a decimal point anywhere from the first to the last digit or can have no decimal point at all.

Oracle Database provides two numeric data types exclusively for floating-point numbers:

- **BINARY_FLOAT** is a **32-bit**, single-precision floating-point number data type. Each **BINARY_FLOAT** value requires **4 bytes**.
- **BINARY_DOUBLE** is a **64-bit**, double-precision floating-point number data type. Each **BINARY_DOUBLE** value requires **8 bytes**.

FLOAT vs **BINARY_FLOAT** vs **BINARY_DOUBLE**

- **BINARY_FLOAT** & **BINARY_DOUBLE** data types take advantage of **hardware acceleration**, therefore, they have better performance for numerical computations.
- **BINARY_FLOAT** & **BINARY_DOUBLE** data types can store smaller / larger numbers than **FLOAT** type.
- **BINARY_FLOAT** & **BINARY_DOUBLE** data types store only **approximate** values, while the **FLOAT** data type stores **exact** values.
- **BINARY_FLOAT** & **BINARY_DOUBLE** data types are suitable for the **scientific calculations** but **not** suitable for **financial calculations**.


A wizard is never late. Nor is he early.
He arrives precisely when he means to.

Datetime and **Interval** Data Types

The **DATE** data type stores **date** and **time** information

- Oracle stores the following information for a **DATE**: year, month, day, hour, minute, and second

The **TIMESTAMP** data type is an **extension** of the **DATE** data type. It stores all information from of the **DATE** data type plus **fraction of seconds**. This data type is useful for

- Storing **precise** time values
- **Collecting** and **Evaluating** date information across **geographic** regions

timestamp_var **TIMESTAMP** [(fractional_seconds_precision)]

timestamp_with_local_var **TIMESTAMP** [(fractional_seconds_precision)] **WITH LOCAL TIME ZONE**

- **TIMESTAMP WITH LOCAL TIME ZONE** is another variant of **TIMESTAMP** that is sensitive to time zone information.
- **fractional_seconds_precision** optionally specifies the **number of digits** Oracle stores in the fractional part of the SECOND datetime field.

Datetime and **Interval** Data Types

INTERVAL YEAR TO MONTH stores a period of time using the **YEAR** and **MONTH** datetime fields. This data type is useful for representing the difference between two datetime values when only the year and month values are significant.

INTERVAL YEAR [(year_precision)] **TO MONTH**

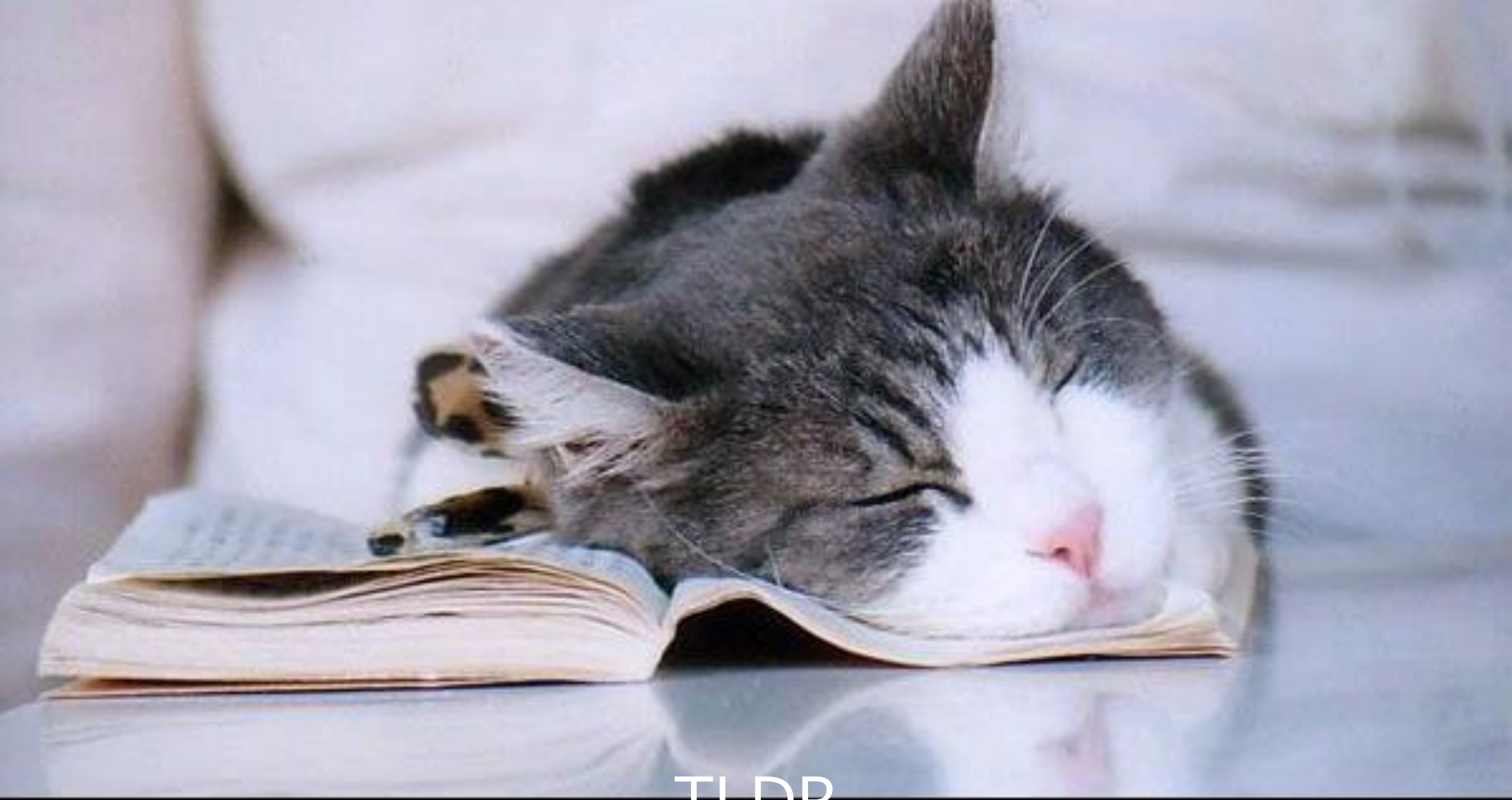
- **year_precision** is the number of **digits** in the **YEAR** datetime field

Datetime and **Interval** Data Types

INTERVAL DAY TO SECOND stores a period of time in terms of days and seconds. This data type is useful for representing the precise difference between two datetime values

INTERVAL DAY [(day_precision)] **TO SECOND** [(fractional_seconds_precision)]

- **day_precision** is the number of digits in the **DAY** datetime field. Accepted values are 0 to 9. The default is **2**.
- **fractional_seconds_precision** is the number of digits in the fractional part of the **SECOND** datetime field. Accepted values are 0 to 9. The default is **6**.


TLDR

RAW and **LONG RAW** Data Types

The **RAW** and **LONG RAW** data types store data that is **not to be explicitly converted** by Oracle Database when moving data between different systems. These data types are intended for **binary data** or **byte strings**. Oracle **strongly recommends** that you convert **LONG RAW** columns to binary **LOB** (BLOB) columns. **LOB** columns are subject to far **fewer restrictions** than **RAW** columns. The built-in LOB data types:

- BLOB
- CLOB
- NCLOB (stored internally)
- BFILE (stored externally)

LOBs can store **large** and **unstructured** data such as text, image, video, and spatial data.

BLOB vs **CLOB** vs **NCLOB** Data Types

The **BLOB** data type stores **unstructured binary large objects**. The **CLOB** data type stores **single-byte** (ASCII) character data or **multibyte** (Unicode) character data. The **NCLOB** data type stores **Unicode** data.

BLOB objects can be thought of as bitstreams with no character set semantics.

CLOB and **NCLOB** support:

- **Fixed-width** character sets (uses the database character set)
- **Variable-width** character sets (uses the database character set)

ALL of Them store binary data up to (4 GB - 1B) * LOB_CHUNK_SIZE

- **LOB_CHUNK_SIZE** is **equal** to the database **block size**

BFILE Data Types

The **BFILE** data type enables access to binary file **LOBs** that are stored in file systems **outside** Oracle Database. A **BFILE column** or **attribute** stores a **BFILE locator**.

BFILE locator serves as a **pointer** to a **binary file** on the **server file system**. The **locator** maintains the **directory name** and the **filename**.

Binary file **LOBs** do not participate in transactions and are **not recoverable**. The **operating system** provides file **integrity** and **durability**.

BFILE data can be up to 2^64-1 bytes (the operating system may impose restrictions on this maximum). The **database administrator** must **ensure** that the **external** file exists. **Oracle processes** have operating system **read** permissions on the file.

The **BFILE** data type enables **read-only** support of **large binary files**, such a file cannot be modified or replicated.

int *
int
int **
@programmer.meme

ROWID Data Types

Each row in the database has an **address**. The rows in **heap-organized tables** that are native to Oracle Database have **row addresses** called **rowids**.

A **rowid** row address can be examined by querying the **pseudocolumn ROWID**. Values of this **pseudocolumn** are **strings** representing the **address of each row**. These strings have the data type **ROWID**. **Rowids** contain the following information:

- The data block of the data file containing the row.
- The row in the data block.
- The database file containing the row
- The data object number which is an identification number assigned to every database segment.

PERHAPS

BOOLEAN Data Type

The PL/SQL data type **BOOLEAN** stores logical values: **TRUE** / **FALSE** / **NULL** (representing an unknown value)

Because SQL has **no** data type **equivalent** to **BOOLEAN**, it is **not** possible to:

- Assign a **BOOLEAN** value to a database **table column**
- **Select** or **fetch** the value of a database table column **into a BOOLEAN** variable
- Use a **BOOLEAN** value in a **SQL function**
- Use a **BOOLEAN** expression in a **SQL statement**, except
  - As an **argument** to a PL/SQL **function** invoked in a SQL query
  - In a PL/SQL **anonymous block**

Unlimited power!

PLS_INTEGER vs **BINARY_INTEGER** Data Types

The PL/SQL data types **PLS_INTEGER** and **BINARY_INTEGER** are identical. Both data types:

- Store **signed** integers in the range -2,147,483,648 through 2,147,483,647
- **Integers** are represented in **32 bits**

They have the following advantages over the **NUMBER** data type and **NUMBER subtypes**:

- Values require **less storage**
- Operations use **hardware arithmetic**
- Are faster than NUMBER operations which use **library arithmetic**

I MADE THIS

User-Defined PL/SQL Subtypes

Developers can define their **own subtypes** for restrincting: **Size** / **Precision** / **Scale**

```
SUBTYPE subtype_name IS base_type
  { precision [, scale ] | RANGE low_value .. high_value } [ NOT NULL ]
```

Basically **syntactic sugar** for some types. (see **INT** vs **NUMBER**)

Operators

Supported Operators in PL/SQL

- Arithmetic operators
- Logic operators
- Comparison operators
- SQL operators

Operator	Operation	Example
**	***Exponential***	val2 ** val1
+	***Identity***	+ val
-	***Negation***	- val
+	***Addition***	val1 + val2
-	***Subtraction***	val1 - val2
*	***Multiplication***	val1 * val2
/	***Division***	val1 / val2
||	***Concatenation***	val1 || val2

Arithmetic operators

Logical operators

Operator	Operation	Example
NOT	***Negation***	NOT val
AND	***Logical AND***	val1 AND val2
OR	***Logical OR***	val1 OR val2

OPERATOR	OPERATION	EXAMPLE
=	***Equal***	val2 = val1
<	***Less then***	val2 < val1
>	***Greater than***	val2 > val1
<=	***Less or equal than***	val1 <= val2
>=	***Greater or equal than***	val1 >= val2
<>	***Different than***	val1 <> val2
!=	***Different than***	val1 != val2
~=	***Different than***	val1 ~= val2
^=	***Different than***	val1 ^= val2

Comparison operators

OPERATOR	OPERATION	EXAMPLE
BETWEEN	***Verifies if a value is in a range***	val BETWEEN val1 AND val2
IN	***Verifies if a value is in a list***	val IN(val1, val2, …, val3)
LIKE	***Verifies a specified patter n***	val LIKE pattern
IS NULL	***Verifies if a value is NULL***	val IS NULL

SQL operators

PRECEDENCE	OPERATORS	OPERATION
1	**	Exponential
2	+, -	Identity, negation
3	*, /	Multiplication, division
4	+, -, ||	Addition, subtraction, concatenation
5	=, <, >, <=, >=, <>, !=, ~=, ^=, IS NULL, LIKE, BETWEEN, IN.	Comparison and SQL operators
6	NOT	Logical negation
7	AND	Conjunction
8	OR	Inclusion

Operators precedence

Symbol	Meaning
:=	Assignment operator
=>	Association operator
%	Attribute indicator
'	String delimiter
.	Component selector
( )	Expression or list delimiter
:	Host variable indicator
,	Item separator
<< >>	Label delimiter
/* */	Multiline comment delimiter
"	Quoted identifier delimiter
..	Range operator
@	Remote access indicator
--	Single-line comment indicator
;	Statement terminator

Other symbols

Control statements

Control Statements

- Conditional selection statements
- Loop statements
- Sequential control statements

Conditional selection statements

IF Statement

The condition can contain:

- Comparison operators
- Logical operators
- SQL operators

```
-- IF THEN Statement
IF condition THEN statements
END IF;

-- IF THEN ELSE Statement
IF condition THEN statements
ELSE else_statements
END IF;

-- IF THEN ELSIF Statement
IF condition_1 THEN statements_1
ELSIF condition_2 THEN statements_2
[ ELSIF condition_3 THEN statements_3 ]
...
[ ELSE else_statements ]
END IF;

-- Logically equivalent nested
-- IF THEN ELSE statements
IF condition_1 THEN statements_1;
ELSE
  IF condition_2 THEN statements_2;
  ELSE
    IF condition_3 THEN statements_3;
    ...
    END IF;
  END IF;
END IF;
```

```
DECLARE
    emp_name varchar2(45);
    emp_salary employees.salary%TYPE;
    emp_iddept employees.department_id%TYPE;
    avg_sal number(6,2);
    avg_sal_dept float(8);
BEGIN
    SELECT first_name ||' '|| last_name, salary, department_id
    INTO emp_name, emp_salary, emp_iddept
    FROM employees WHERE employee_id = &id;

    SELECT AVG(salary) INTO avg_sal FROM employees;

    SELECT AVG(salary) INTO avg_sal_dept FROM employees
    WHERE department_id= emp_iddept;

    IF emp_salary > avg_sal THEN
        dbms_output.put_line(emp_name || ' are sal > decat sal mediu din firma');
    END IF;

    IF emp_salary > avg_sal_dept THEN
        dbms_output.put_line(emp_name || ' are sal > sal mediu din departamentul in care lucreaza');
    ELSE
        dbms_output.put_line(emp_name || ' are sal <= sal mediu din departamentul in care lucreaza');
    END IF;
END;
/
```

```
DECLARE
    year_of_experience INTERVAL YEAR TO MONTH;
    expert INTERVAL YEAR TO MONTH;
    emp_name varchar2(45);
    emp_hiredate employees.hire_date%TYPE;
BEGIN
    expert := INTERVAL '15-1' YEAR TO MONTH;

    dbms_output.put_line(expert);
    dbms_output.put_line(sysdate());
    dbms_output.put_line(sysdate - expert);
    SELECT first_name ||' '|| last_name, hire_date
    INTO emp_name, emp_hiredate
    FROM employees WHERE employee_id = &id;

    IF emp_hiredate < sysdate - expert THEN
        dbms_output.put_line(emp_name ||' e expert');
    ELSE
        dbms_output.put_line(emp_name || ' nu e expert');
    END IF;
END;
/
```

Conditional selection statements

CASE Statement

Simple CASE

- The selector is an expression (typically a single variable)
- Each selector_value can be either a literal or an expression (cannot use NULL)
- The simple CASE statement runs the first statements for which selector_value equals selector
- Remaining conditions are not evaluated
- If no selector_value equals selector the CASE statement runs else_statements if they exist
- Raises the predefined exception CASE_NOT_FOUND otherwise

Searched CASE

- The searched CASE statement runs the first statements for which condition is true
- Remaining conditions are not evaluated

```
-- Simple CASE
CASE selector
WHEN selector_value_1 THEN statements_1
WHEN selector_value_2 THEN statements_2
...
WHEN selector_value_n THEN statements_n
[ ELSE else_statements ]
END CASE;

-- Searched CASE
CASE
WHEN condition_1 THEN statements_1
WHEN condition_2 THEN statements_2
...
WHEN condition_n THEN statements_n
[ ELSE else_statements ]
END CASE;
```

```
DECLARE
    emp_name varchar2(45);
    emp_iddept employees.department_id%TYPE;
    emp_reg regions.region_name%TYPE;
    emp_city locations.city%TYPE;
BEGIN
    SELECT first_name || ' ' || last_name, department_id
    INTO emp_name, emp_iddept
    FROM employees WHERE employee_id = &id;

    SELECT r.region_name, l.city INTO emp_reg, emp_city FROM regions r
        INNER JOIN countries c ON c.region_id = r.region_id
        INNER JOIN locations l ON l.country_id = c.country_id
        INNER JOIN departments d ON d.location_id = l.location_id
    WHERE d.department_id = emp_iddept;

    CASE emp_reg
        WHEN 'Europe' THEN
            IF emp_city = 'London' THEN
                dbms_output.put_line(emp_name || ' works in European HQ');
            ELSE
                dbms_output.put_line(emp_name || ' doesn''t works in HQ');
            END IF;
        WHEN 'Americas' THEN
            IF emp_city = 'Seattle' THEN
                dbms_output.put_line(emp_name || ' works in American HQ');
            ELSE
                dbms_output.put_line(emp_name || ' doesn''t works in HQ');
            END IF;
        ELSE
            dbms_output.put_line(emp_name || ' works outside Europe and America');
    END CASE;
END;
/
```

```
DECLARE
    emp_name varchar2(45);
    comm employees.commission_pct%TYPE;
BEGIN
    SELECT first_name ||' '|| last_name, commission_pct
    INTO emp_name, comm
    FROM employees WHERE employee_id = &id;

    CASE
        WHEN comm IS NULL THEN
            dbms_output.put_line(emp_name || ' nu are commision');
        WHEN comm >= 0 and comm < 0.2 THEN
            dbms_output.put_line(emp_name || ' are commision mic');
        WHEN comm >= 0.2 and comm < 0.3 THEN
            dbms_output.put_line(emp_name || ' are commision mediu');
        ELSE
            dbms_output.put_line(emp_name || ' are commision mare');
    END CASE;
END;
/
```

```
DECLARE
    emp_name varchar2(45);
    comm employees.commission_pct%TYPE;
BEGIN
    SELECT first_name || ' ' || last_name, commission_pct
    INTO emp_name, comm
    FROM employees WHERE employee_id = &id;

    IF comm IS NULL THEN
        dbms_output.put_line(emp_name || ' nu are commision');
    ELSIF comm >= 0 and comm < 0.2 THEN
        dbms_output.put_line(emp_name || ' are commision mic');
    ELSIF comm >= 0.2 and comm < 0.3 THEN
        dbms_output.put_line(emp_name || ' are commision mediu');
    ELSE
        dbms_output.put_line(emp_name || ' are commision mare');
    END IF;
END;
/
```

Loop Statements

Basic LOOP Statement

With **each iteration** of the loop:

- The statements are run
- Control returns to the top of the loop

To prevent an **infinite loop** and exit the loop:

- Use a statement
- Raised exception

```
-- Basic LOOP
[label] LOOP
  statements
  [ EXIT [ WHEN condition ] ]
  [ CONTINUE [ WHEN condition ] ]
END LOOP [label];
```

EXIT statements

The **EXIT** statement **ends** the **current iteration** of a loop **unconditionally**. It transfers control to the **end** of either the **current loop** or an **enclosing labeled** loop.

The **EXIT WHEN** statement ends the current iteration of a loop when the condition in its **WHEN** clause is **true**. The condition in the **WHEN** clause is evaluated **each time** control reaches the **EXIT WHEN** statement. The **EXIT WHEN** statement does nothing if the condition is not true.

CONTINUE statement

The **CONTINUE** statement **continues** the **current iteration** of a loop **unconditionally**. It transfers control to the **next iteration** of either the **current loop** or an **enclosing labeled** loop.

The **CONTINUE WHEN** statement continues the current iteration of a loop when the condition in its **WHEN** clause is true. The condition in the **WHEN** clause is evaluated each time control reaches the **CONTINUE WHEN** statement. The **CONTINUE WHEN** statement does nothing if the condition is not true.

```
DECLARE
    idx number NOT NULL := 1;
    deptname departments.department_name%TYPE;
    avg_sal_dept number(7,2);
BEGIN
    LOOP
        SELECT d.department_name, avg(e.salary)
        INTO deptname, avg_sal_dept
        FROM departments d
            INNER JOIN employees e
                ON d.department_id = e.department_id
        WHERE d.department_id = idx * 10
        GROUP BY d.department_name;

        dbms_output.put_line(deptname || ' ' || avg_sal_dept);
        idx := idx + 1;

        CONTINUE WHEN idx < 11;

        -- EXIT WHEN idx = 12;

    END LOOP;
END;
/
```

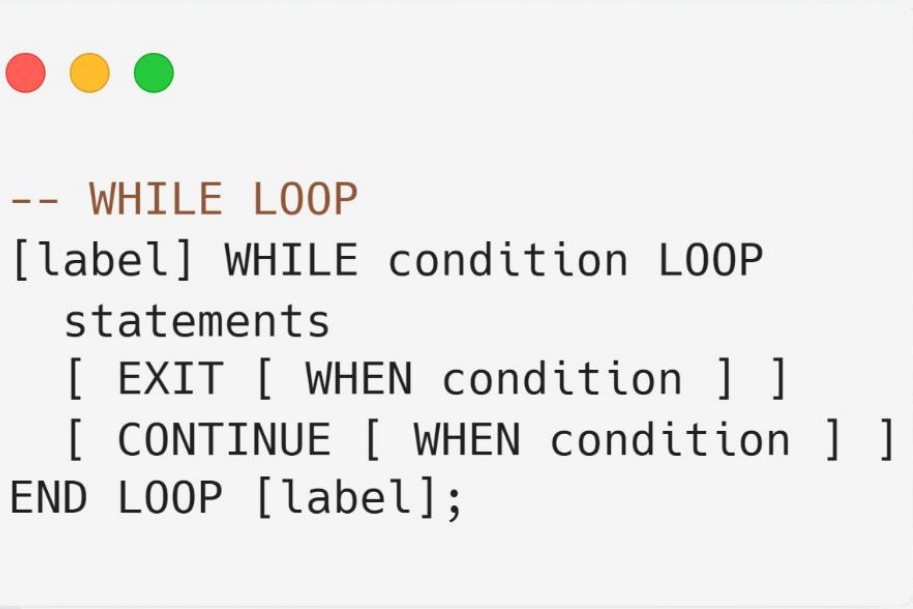
Loop Statements
WHILE LOOP Statement

The **WHILE** LOOP statement runs one or more statements **while** a condition is true.

If the condition is true the statements are run, then **control returns** to the **top of the loop** where condition is **evaluated** again.

The **WHILE** LOOP **stops** when the condition is false. A statement inside the loop must make the condition false or null to prevent an **infinite loop**.

An **EXIT**, **EXIT WHEN**, **CONTINUE**, or **CONTINUE WHEN** in the statements can cause the loop or the current iteration of the loop to end **early**.



```
-- WHILE LOOP
[label] WHILE condition LOOP
  statements
  [ EXIT [ WHEN condition ] ]
  [ CONTINUE [ WHEN condition ] ]
END LOOP [label];
```

```
DECLARE
    idx number := 1;
    deptname departments.department_name%TYPE;
    avg_sal_dept number(7,2);
BEGIN
    WHILE idx < 12
    LOOP
        SELECT d.department_name, avg(e.salary)
        INTO deptname, avg_sal_dept
        FROM departments d
        INNER JOIN employees e
        ON d.department_id = e.department_id
        WHERE d.department_id = idx * 10
        GROUP BY d.department_name;

        dbms_output.put_line(deptname || ' ' || avg_sal_dept);
        idx := idx + 1;
    END LOOP;
END;
/
```

Loop Statements
FOR LOOP Statement

The **FOR** LOOP statement runs one or more statements while the **loop index** is in a **specified range**.

Without **REVERSE**:
- The value of index starts at **lower_bound** and **increases by one** with each iteration of the loop until it reaches **upper_bound**
- The statements **never run if** lower_bound is **greater than** upper_bound

With **REVERSE**:
- The value of index starts at **upper_bound** and **decreases by one** with each iteration of the loop until it reaches **lower_bound**
- If upper_bound is **less than** lower_bound, then the statements **never run**

An **EXIT**, **EXIT WHEN**, **CONTINUE**, or **CONTINUE WHEN** in the statements can cause the loop or the current iteration of the loop to end **early**.

```
-- FOR LOOP
[label] FOR IN [ REVERSE ] LOWER_BOUND .. UPPER_BOUND
LOOP
  statements
  [ EXIT [ WHEN condition ] ]
  [ CONTINUE [ WHEN condition ] ]
END LOOP [label];
```

```
DECLARE
    deptname departments.department_name%TYPE;
    avg_sal_dept number(7,2);
BEGIN
    FOR idx IN 1 .. 12
    LOOP
        dbms_output.put_line('Index =  ' || idx);
        IF idx = 12 THEN
            GOTO exit_for;
        END IF;

        SELECT d.department_name, avg(e.salary)
        INTO deptname, avg_sal_dept
        FROM departments d
            INNER JOIN employees e
                ON d.department_id = e.department_id
        WHERE d.department_id = idx * 10
        GROUP BY d.department_name;

        dbms_output.put_line(deptname || ' ' || avg_sal_dept);
    END LOOP;

    <<exit_for>>dbms_output.put_line('Done!');
END;
/
```

Conditional selection statements: **GOTO**

The sequential control statement **GOTO** transfers control to a label **unconditionally**.

Can be used to **exit** a LOOP. NULL statement only passes control to the next statement

```
GOTO label_example;

<<label_example>>
--Code
```

```
DECLARE
    deptname departments.department_name%TYPE;
    avg_sal_dept number(7,2);
BEGIN
    FOR idx IN 1 .. 12
    LOOP
        dbms_output.put_line('Index =  ' || idx);
        IF idx = 12 THEN
            GOTO exit_for;
        END IF;

        SELECT d.department_name, avg(e.salary)
        INTO deptname, avg_sal_dept
        FROM departments d
            INNER JOIN employees e
                ON d.department_id = e.department_id
        WHERE d.department_id = idx * 10
        GROUP BY d.department_name;

        dbms_output.put_line(deptname || ' ' || avg_sal_dept);
    END LOOP;

    <<exit_for>>dbms_output.put_line('Done!');
END;
/
```

Bibliography

Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 19c*, Oracle Corporation, 2022 [pdf]

Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 21c*, Oracle Corporation, 2022 [pdf]

Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 19c*, Oracle Corporation, 2020 [pdf]

Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 21c*, Oracle Corporation, 2021 [pdf]

Memes

- E: https://www.dailydot.com/unclick/lord-farquaad-e-meme/
- "What do the Number Mean" : COD
- Gandalf A Wizard Is Never Late : Lord of the Rings
- TLDR : https://www.seoreseller.com/wp-content/uploads/2014/01/tldr-cat.jpg
- Anya Pointer : https://www.instagram.com/programmer.meme/p/CysHW-4LgLL/
- Perhaps: https://en.meming.world/wiki/Perhaps
- Unlimited Power: Star Wars
- I made this : https://knowyourmeme.com/memes/i-made-this

PL/SQL (III)

Ș.L. Dr. Ing. Ciprian-Octavian Truică

ciprian.truica@upb.ro

As. Drd. Ing. Alexandru Petrescu

Alex.petrescu@upb.ro

Overview - Composite data types

Records

Collection Types: Associative Arrays, VARRAYS, Nested Tables

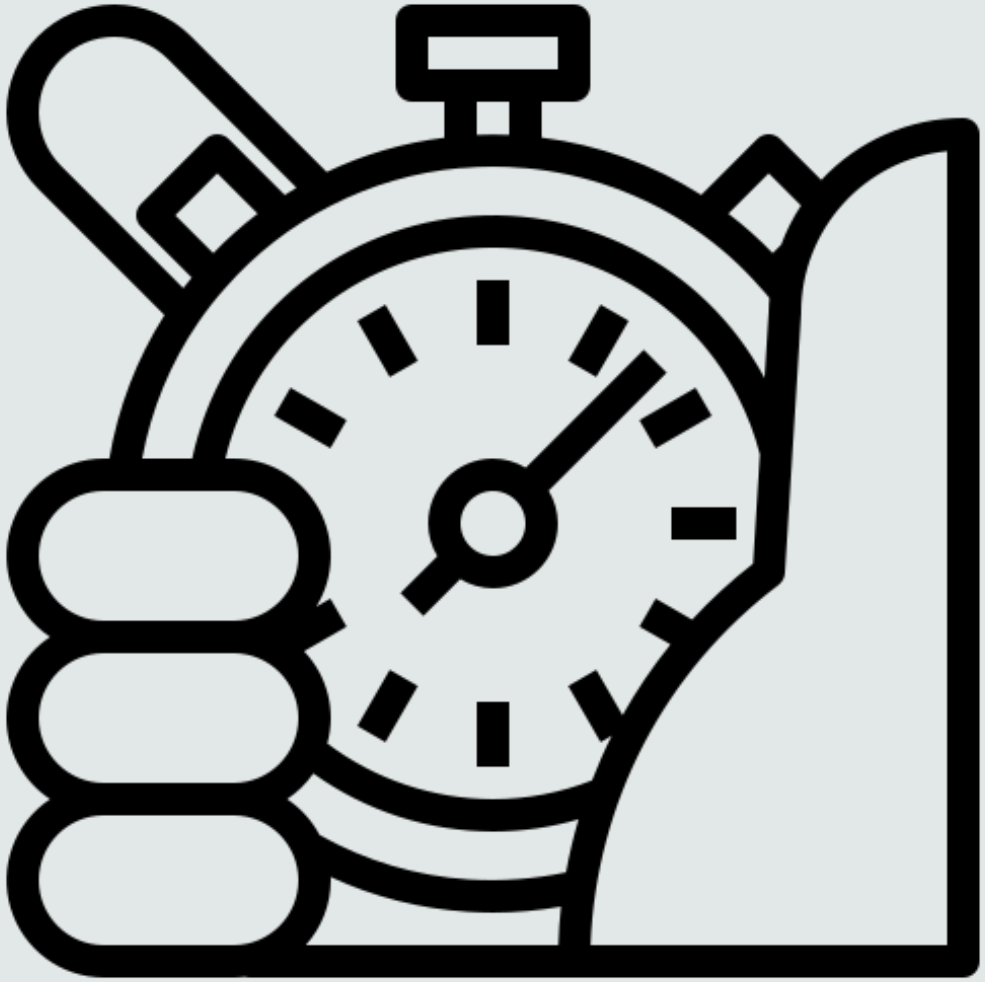
Collection Features

Composite data types

A **composite** data type stores values that have **internal components**. PL/SQL lets you define two kinds of composite data types:

- **Record**
- **Collection**

A **composite variable** can be passed to **subprograms** as **parameters** and its **internal components** can be accessed **individually**. Internal components can be either **Scalar** or **Composite**.



RECORD

Type

Records

The **internal components** of a **record** can have **different** data types and are called **fields**.

Each **field** of a record variable can be accessed by its name, e.g., variable_name.field_name.

Record variables can be:

- **Copies** of existing record variables
  - Created using the attribute **%ROWTYPE** or **%TYPE**
  - Inherits the only **structure**, **not** the **values**
- **User defined**
  - Define a **RECORD** type
  - Create a **variable** of that type

```
DECLARE
    TYPE rec1 IS RECORD(
        id countries.country_id%TYPE,
        name varchar2(20),
        reg regions%ROWTYPE
    );
    r1 rec1;
    r2 r1%type;
BEGIN
    r2.id := 1;
    r2.name = 'Hi, My Name is';
    dbms_output.put_line(r2.id);
END;
/
```

Records

A **RECORD** type defined in a PL/SQL **block** is a **local** type available **only** in the **block.** It is **stored** in the **database** only if the block is in a **subprogram** or **package**.

A **RECORD** type defined in a **package specification** is a **public** item and can be referenced from **outside** the **package** by qualifying it with the package name (package_name.type_name). It is **stored** in the database until you **drop** the **package** with the **DROP PACKAGE** statement.

A **RECORD** type cannot be created at **schema level** (a RECORD type cannot be an ADT attribute data type).

Records

To define a **RECORD** type, specify the record's **name** and define its **fields**. To define a **field**, specify the field's **name** and a its data **type**. The **initial** value of a **field** is **NULL**. The **NOT NULL** constraint can be specified for a field (a non-NULL **initial value** must also be **specified**).

A **non-NULL** initial value is **optional without** the **NOT NULL** constraint.

A **RECORD** type defined in a **package specification** is **incompatible** with an identically defined **local RECORD** type.

A **field** declared with **%ROWTYPE** or **%TYPE** still has an **initial NULL** value. (the variable **does not inherit** the initial **value** of the referenced item)

DO'S

DON'TS

Records

Records **cannot** be tested natively for **nullity**, **equality**, or **inequality**.

These BOOLEAN expressions are **illegal**:

- My_Record **IS NULL**
- My_Record_1 **=** My_Record_2
- My_Record_1 **>** My_Record_2

Records

Record variables are **allowed** only in these places:

- On the **right side** of the **SET** clause in an **UPDATE** statement (**Only 1** SET clause is allowed in this case). This can be combined with the "**ROW**" keyword on the **left side**.
- In the **VALUES** clause of an **INSERT** statement
- In the **INTO** subclause of a **RETURNING** clause

Record variables are **not allowed** in a **SELECT** list, a **WHERE** clause, a **GROUP BY** clause or in an **ORDER BY** clause. The "**ROW**" keyword **cannot** be used with a **subquery**.

Records

The following **restrictions** apply to **record inserts** and **updates**:

- If the **VALUES** clause of an **INSERT** statement contains a **RECORD** variable, **no other variable** or **value** is allowed in the **clause**
- If the **INTO** subclause of a **RETURNING** clause contains a **RECORD** variable, **no other variable** or **value** is allowed in the **subclause**

These are **not supported**:

- **Nested RECORD** types
- **Functions** that **return** a **RECORD** type
- Record inserts and updates using the **EXECUTE IMMEDIATE** statement (Dynamic SQL)

PL/SQL engine cares about type compatibility so as long as our instructions leverage the proper types, all is good.

```
set serveroutput on
DECLARE
  TYPE rec1 IS RECORD(
    id countries.country_id%TYPE,
    name varchar2(20),
    reg regions%ROWTYPE
  );
  r1 rec1;
  r2 countries%ROWTYPE;
  r3 countries%ROWTYPE;
BEGIN
  SELECT * INTO r1.reg FROM regions
  WHERE lower(region_name) LIKE '%rop_';

  r1.id := 'UA';
  r1.name := 'Ukraina';

  INSERT INTO countries
  VALUES(r1.id, r1.name, r1.reg.region_id);


  SELECT * INTO r2 FROM countries
  WHERE country_id = r1.id;

  r2.country_id := 'ME';
  r2.country_name := 'Moldova';
  r2.region_id := r1.reg.region_id;

  INSERT INTO countries VALUES r2
  RETURNING country_id, country_name, region_id into r3;

  dbms_output.put_line('====INSERT====');
  dbms_output.put_line(r2.country_id || ' ' || r2.country_name || ' ' || r2.region_id);
  dbms_output.put_line(r3.country_id || ' ' || r3.country_name || ' ' || r3.region_id);


  r2.country_id := 'MD';
  r2.country_name := 'Moldova';
  r2.region_id := r1.reg.region_id;

  UPDATE countries SET ROW = r2 WHERE country_id = 'MD'
  RETURNING country_id, country_name, region_id into r3;

  dbms_output.put_line('====UPDATE====');
  dbms_output.put_line(r2.country_id || ' ' || r2.country_name || ' ' || r2.region_id);
  dbms_output.put_line(r3.country_id || ' ' || r3.country_name || ' ' || r3.region_id);

  DELETE FROM countries WHERE country_id = 'MD'
  RETURNING country_id, country_name, region_id into r3;

  dbms_output.put_line('====DELETE====');
  dbms_output.put_line(r2.country_id || ' ' || r2.country_name || ' ' || r2.region_id);
  dbms_output.put_line(r3.country_id || ' x ' || r3.country_name || ' ' || r3.region_id);

END;
/
```

Collection Types

Collection types

The **internal components** of a **collection** have the **same** data **type** and are called **elements**. Elements of a **collection** variable can be accessed by its **unique index**, e.g., variable_name(**index**)

Collections variables can be:

- **Copies** of **existing** composite variables (created using the attribute %TYPE - Inherits the only structure, not the values)
- **User defined** – define a collection type and then create a variable of that type

Collection types

Collections have **one dimension**. A **multidimensional** collection can be modeled by using a **collection** whose **elements** are **collections**. Developers can create:

- A **collection** of **records**: a **collection** that has **elements records** and accessing an **item** "variable_name(**index**).field_name"
- A **record** that contains **collections**: a **record** that has **items collections** and accessing a **field** "variable_name.field_name(**index**)"
- A **collection** of **collections**: a **collection** that has items **a collection** and accessing an item "variable_name(**index_1**)(**index_2**)⋯(**index_n**)"

Collection types

Collection Type	Number of Elements	Index Type	Dense or Sparse	Uninitialized Status	Defined in	ADT Type Attr?
Associative array (or index-by table)	Unspecified	**String** **PLS_INTEGER** (BINARY_INTEGER)	Either	Empty	In PL/SQL block or package	No
VARRAY (variable-size array)	Specified	Integer	Always dense	Null	In PL/SQL block or package or at schema level	Only defined at schema level
Nested table	Unspecified	Integer	Starts dense, can become sparse	Null	In PL/SQL block or package or at schema level	Only defined at schema level

Number of **Elements**, **Index**, **Dense** or **Sparse**

Number of **Elements**?

- **Specified**: the **maximum** number of elements in the **collection**
- **Unspecified**: the **maximum** number of elements in the collection is the **upper limit of the index type**

Dense or **Sparse**?

- A **dense** collection has **no gaps** between **elements**, every element between the **first** and **last** element is **defined** (meaning it has a value, the value can be **NULL** unless the element has a **NOT NULL** constraint).
- A **sparse** collection **has gaps** between elements.

Uninitialized Status

An **empty** collection **exists** but has **no elements**. The **EXTEND** method must be invoked to **add elements** to an **empty** collection.

A **null** collection (also called an **atomically null** collection) **does not exist**. The collection must be initialized to change it from a null collection to an existing collection. The **EXTEND** method cannot be used to initialize a null collection. Initialization:

- Making it empty
- Assigning a non-NULL value to it

Defined in

Same as for **RECORD**: if it's in a **block**, it is **local** / if it's in a **program specification**, it is **public**. Can be **stored** as part of a **subprogram** else it is gone with the session.

A **collection** type **defined** at **schema level** is a standalone type. It is **stored** in the database until you **drop** it with the **DROP TYPE** Statement.

- Can be **created** with the **CREATE** [**OR REPLACE**] **TYPE**
- Can be **modified** using the **ALTER TYPE**

A collection type defined in a package specification is incompatible with an identically defined local or standalone collection type.

ADT Attribute Data Type

To be an **ADT**(Abstract Data Type) **Attribute** Data Type, a collection type must be a **standalone** collection type. Translating Non-PL/SQL Composite Types to PL/SQL Composite Types:

Non-PL/SQL Composite Type	Equivalent PL/SQL Composite Type
Hash table	Associative array
Unordered table	Associative array
Set	Nested table
Bag	Nested table
Array	VARRAY

Associative Arrays

Associative Arrays

An **associative array** (formerly called **PL/SQL table** or **index-by table**) is a **set** of **key-value** pairs where each **key** is a **unique index**. The **index** is used to locate the associated **value** with the syntax "variable_name(**index**)".

An **associative array** (like a database table) is **empty** (but not null) until it is **populated**. It can **hold** an **unspecified** number of elements (accessible without knowing their positions). Unlike a database table, it **does not** need **disk space** and **cannot** be manipulated with **DML** statements .

Associative Arrays - Indexes

The data type of **index** can be:

- A **string** type (**VARCHAR2**, **VARCHAR**, **STRING**, or **LONG**)
- **PLS_INTEGER**/**BINARY_INTEGER**

Indexes are **stored** in **sort order**, but are **not created** in **order**. The **sort order** is determined by the initialization parameter **NLS_SORT** and **NLS_COMP** for string types. National Language Support (**NLS**) parameters such as **NLS_SORT**, **NLS_COMP**, and **NLS_DATE_FORMAT** affect associative arrays indexed by **string**.

Associative Arrays

An **associative array** is **appropriate for**:

- A **relatively small** lookup table constructed in **memory** each time the subprogram is invoked or the package that declares it is initialized.
- To pass **collections** to and from the **database server**: Declare formal **subprogram parameters** of **associative array** types > Bind the **host arrays** to the corresponding actual **parameters** (use Oracle Call Interface (**OCI**) or an Oracle Precompiler) > PL/SQL **automatically converts** between host arrays and associative arrays indexed by **PLS_INTEGER** (You **cannot bind** an associative array **indexed** by **VARCHAR**)

Associative Arrays

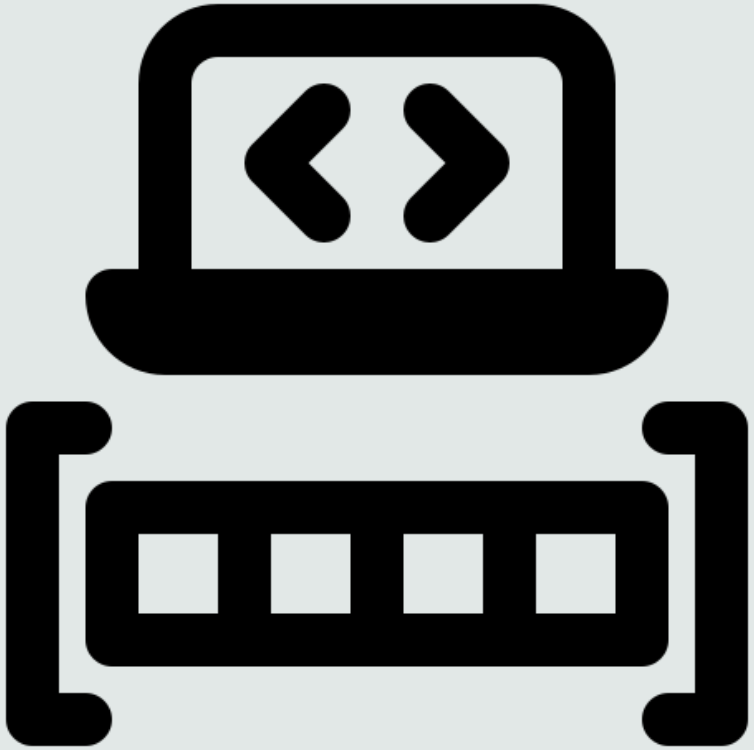
The **associative array** type **cannot** be **declared** at **schema level** as it is intended for **temporary data storage**. To make an **associative array persistent** declare it in a **package specification** or populate it in the **package body**.

To pass an **associative array** variable as a **parameter** to a standalone **subprogram** the type of that variable must be declared in a **package specification**. Doing so makes the type **available** to the **invoked subprogram** (which declares a formal parameter of that type) and the **invoking subprogram** or anonymous block (which declares and passes the variable of that type).

The **most efficient** way to pass **collections** to and from the database server is to use **associative arrays** with **FORALL** statement and **BULK COLLECT** clause.

```
set serveroutput on
DECLARE
  TYPE rec IS RECORD(
    country_id countries.country_id%TYPE,
    country_name countries.country_name%TYPE,
    region_id countries.region_id%TYPE
  );
  TYPE aa IS TABLE OF rec INDEX BY PLS_INTEGER;

  v aa; -- create a null associative array
  r rec;

  idx PLS_INTEGER;

  TYPE idxs_aa IS TABLE OF PLS_INTEGER INDEX BY PLS_INTEGER;
  idxs idxs_aa;

  i PLS_INTEGER := 1;
BEGIN
  r.country_id := 'UA';
  r.country_name := 'Ukraina';
  r.region_id := 1;

  v := aa(1 => r, 2 => rec('MD', 'Moldova', 1));

  v(5).country_id := 'BG';
  v(5).country_name := 'Bulgaria';
  v(5).region_id := 1;

  idx := v.FIRST;
  WHILE idx IS NOT NULL
  LOOP
    dbms_output.put_line(idx||' '||v(idx).country_id||' '
                         ||v(idx).country_name||' '||v(idx).region_id);
    idxs(i) := idx;
    idx := v.next(idx);
    i := i + 1;
  END LOOP;

  FORALL j IN VALUES OF idxs INSERT INTO COUNTRIES VALUES v(j);

END;
/
```

```
DECLARE
  TYPE rec IS RECORD(
    country_id countries.country_id%TYPE,
    country_name countries.country_name%TYPE,
    region_id countries.region_id%TYPE
  );

  TYPE aa IS TABLE OF rec INDEX BY PLS_INTEGER;
  v aa := aa(); -- create an empty associative array;
  r rec;
  idx PLS_INTEGER;
BEGIN
  r.country_id := 'RO';
  r.country_name := 'Romania';
  r.region_id := 1;

  v(1) := r;
  v(2) := rec('MD', 'Moldova', 1);

  v(3).country_id := 'BG';
  v(3).country_name := 'Bulgaria';
  v(3).region_id := 1;

  SELECT * BULK COLLECT INTO v from countries order by country_id desc;

  idx := v.FIRST;
  WHILE idx IS NOT NULL
  LOOP
    dbms_output.put_line(idx||' '||v(idx).country_id||' '||v(idx).country_name||' '||v(idx).region_id);
    idx := v.next(idx);
  END LOOP;
END;
/
```



VARRARAYS

VARRARYS

A **varray** (**variable-size** array) is an array whose **number** of **elements** can vary from zero (empty) to the **declared** maximum size. An **uninitialized** varray variable is a **null collection**. A **varray** variable must be **initialized** by either making it **empty** or assigning a **non-NULL** value to it.

A **varray** is **appropriate** when the maximum number of needed elements is **known** and the elements are **accessed sequentially**. A **varray** might be **impractical** for storing **many** elements because all elements must be stored or retrieved at the same time.

VARRARYS - Index

Use the syntax "variable_name(index)" to **access** an **element** of a **varray** variable, where the **lower bound** of **index** is **1** and the **upper bound** is the **current number** of **elements**.

The **upper bound changes** as you add or delete **elements but** cannot exceed the maximum size.

```
DECLARE
  TYPE vec1 IS VARRAY(150) OF countries.country_id%type;
  v1 vec1 := vec1('RO', 'MD');
  TYPE vec2 IS VARRAY(150) OF countries.country_name%type;
  v2 vec2 := vec2('Romania', 'Moldova');
BEGIN
  FORALL i IN v1.FIRST .. v1.LAST
    INSERT INTO countries(country_id) VALUES(v1(i));

  FORALL i IN v2.FIRST .. v2.LAST
    UPDATE countries SET country_name=v2(i), region_id=1 WHERE country_id = v1(i);

  SELECT country_id, country_name
  BULK COLLECT INTO v1, v2
  FROM countries
  WHERE country_id NOT IN (SELECT DISTINCT country_id FROM locations WHERE country_id IS NOT
NULL);
  FOR i IN v1.FIRST .. v1.LAST
  LOOP
    dbms_output.put_line(v1(i)||' '||v2(i));
  END LOOP;

  FORALL i IN v1.FIRST .. v1.LAST
    DELETE countries WHERE country_id = v1(i);

END;
/
```

Nested Tables

Nested Tables

A **nested table** is a collection type that stores an **unspecified number** of rows in **no particular order**. When retrieving a **nested table** value from the **database** PL/SQL gives the rows **consecutive indexes** starting at **1**.

The syntax is "variable_name(**index**)".

A **nested table**'s **indexes** and **row order** might **not remain stable** when **storing** or **retrieving** the **nested table** to and from the **database**.

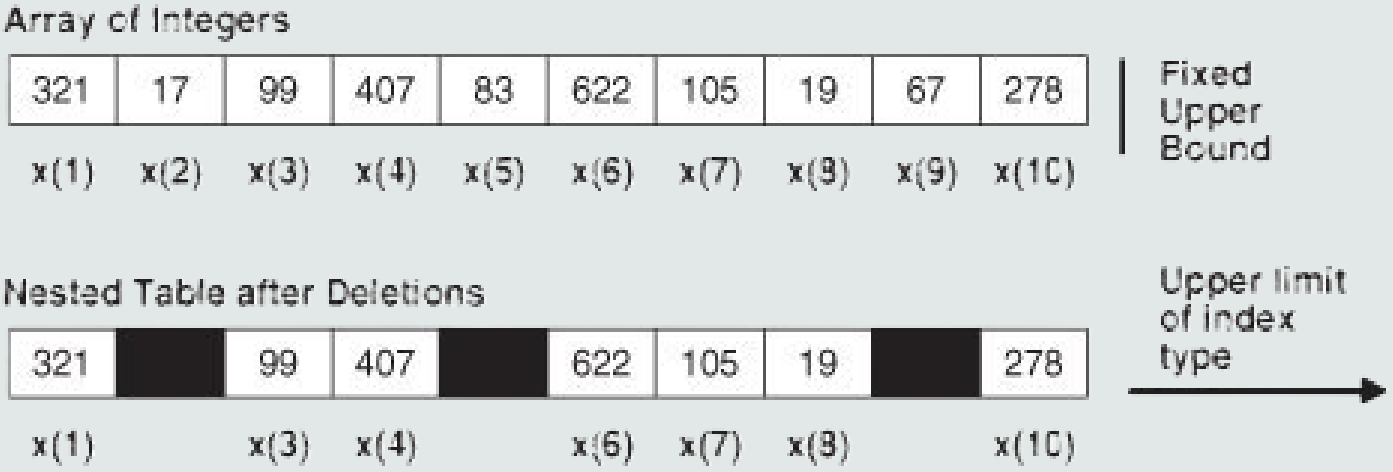
The amount of **memory** that a nested table variable occupies can **increase** or **decrease dynamically** as elements are added or deleted.

An **uninitialized** nested table variable is a **null collection**. A **nested table** is initialized by either making it **empty** or assigning a **non-NULL** value to it.

Nested Tables

Conceptually a **nested table** is like a one-dimensional **array** with an **arbitrary** number of elements, however, there are **differences**:

- The **size** of a nested table can **increase dynamically**: an **array** has a **declared** (**maximum**) number of elements, a nested table **does not**
- A **nested table** is **dense initially** but can become **sparse** as elements are **deleted** (at arbitrary indexes), whereas the **array** remains **dense**



Nested Tables

A **nested table** is **appropriate** when:

- The (maximum)number of elements is **not set**
- **Index** values are **not consecutive**
- To **delete** or **update some elements**, but **not all** elements **simultaneously**

```
DECLARE
  TYPE tbl1 IS TABLE OF countries.country_id%type;
  t1 tbl1 := tbl1('RO', 'MD');
  TYPE tbl2 IS TABLE OF countries.country_name%type;
  t2 tbl2 := tbl2('Romania', 'Moldova');
BEGIN
  FORALL i IN INDICES OF t1 INSERT INTO countries(country_id) VALUES(t1(i));

  FORALL i IN INDICES OF t2 UPDATE countries SET country_name=t2(i), region_id=1 WHERE country_id =
t1(i);

  SELECT country_id, country_name
  BULK COLLECT INTO t1, t2 FROM countries
  WHERE country_id NOT IN (SELECT DISTINCT country_id FROM locations WHERE country_id IS NOT NULL);

  FOR i IN t1.FIRST .. t1.LAST
  LOOP
    dbms_output.put_line(t1(i)||' '||t2(i));
  END LOOP;

  FORALL i IN INDICES OF t1 DELETE countries WHERE country_id = t1(i);

END;
/
```

Collection Features

Collection Constructors

A **collection constructor** (constructor) is a system-defined **function** that has the **same name** as a collection type and **returns** a collection of that type.

```
-- declare the type

TYPE vec IS VARRAY(10) OF INTEGER;

-- initialize an empty variable

var vec := vec();
```

IT'S DANGEROUS TO GO ALONE! TAKE THIS.

Assigning Values to Collection Variables

- **Constructor**: Invoke a **constructor** to **create** a collection and **assign values** to the collection **elements**
- **Assignment**: To assign the value of **another** existing **collection** variable
- **Subprogram**: **Pass** the collection variable to a **subprogram** / Use the **OUT** or **IN OUT** parameter / **Assign** the value **inside** the **subprogram**

A collection can be assigned to a collection variable if they have the same data type. Having the same element type is not enough for an assignment.

Assigning Values to Collection Variables

To make a **collection** variable **null** assign either a **NULL value** or a **NULL collection** of the same data type.

The **SQL MULTISET** operators (MULTISET **EXCEPT**, MULTISET **INTERSECT**, MULTISET **UNION**) **combine** two **nested tables** into a **single** nested table. The elements of the two **nested table**s must have **comparable** data types.

The result of the **SQL SET** function invocation takes a **nested table** argument and returns a nested table of the **same data type** whose elements are **distinct**. The function **eliminates duplicate** elements.

```
DECLARE
    TYPE t IS TABLE OF INTEGER;
    s1 t := t(1,1,2,2,3,4);
    s2 t := t(2,4,4,6,6, 8, 8);
    s3 t;
    s4 t := t(3, 4);
    PROCEDURE print(x t) IS
      s string(100) := '';
    BEGIN
        FOR i IN x.first .. x.last
        LOOP
            s := s||x(i)||' ';
        END LOOP;
        dbms_output.put_line(s);
    END print;
BEGIN
    IF s3 IS NULL THEN
        dbms_output.put_line('EMPTY AND NULL');
    ELSIF s3 IS NOT NULL AND s3 IS EMPTY THEN
        dbms_output.put_line('EMPTY AND NOT NULL');
    ELSIF s3 IS NOT NULL AND s3 IS NOT EMPTY THEN
        dbms_output.put_line('NOT EMPTY AND NOT NULL');
    END IF;

    dbms_output.put_line('UNION');
    s3 := s2 MULTISET UNION S1;
    print(s3);

    dbms_output.put_line('SET');
    print(SET(s3));

    dbms_output.put_line('INTERSECT');
    s3 := s1 MULTISET INTERSECT S2;
    print(s3);

    dbms_output.put_line('EXCEPT');
    s3 := s1 MULTISET EXCEPT S2;
    print(s3);

    dbms_output.put_line('EXCEPT DISTINCT');
    s3 := s1 MULTISET EXCEPT DISTINCT S2;
    print(s3);

    dbms_output.put_line('EXCEPT');
    s3 := s2 MULTISET EXCEPT S1;
    print(s3);

    dbms_output.put_line('EXCEPT DISTINCT');
    s3 := s2 MULTISET EXCEPT DISTINCT S1;
    print(s3);

    IF s4 SUBMULTISET OF s1 THEN
        dbms_output.put_line('TRUE');
    ELSE
        dbms_output.put_line('FALSE');
    END IF;

    dbms_output.put_line(CARDINALITY(s1));
    dbms_output.put_line(CARDINALITY(SET(s1)));

    IF &VAL MEMBER OF S1 THEN
        dbms_output.put_line('TRUE');
    END IF;
END;
/
```

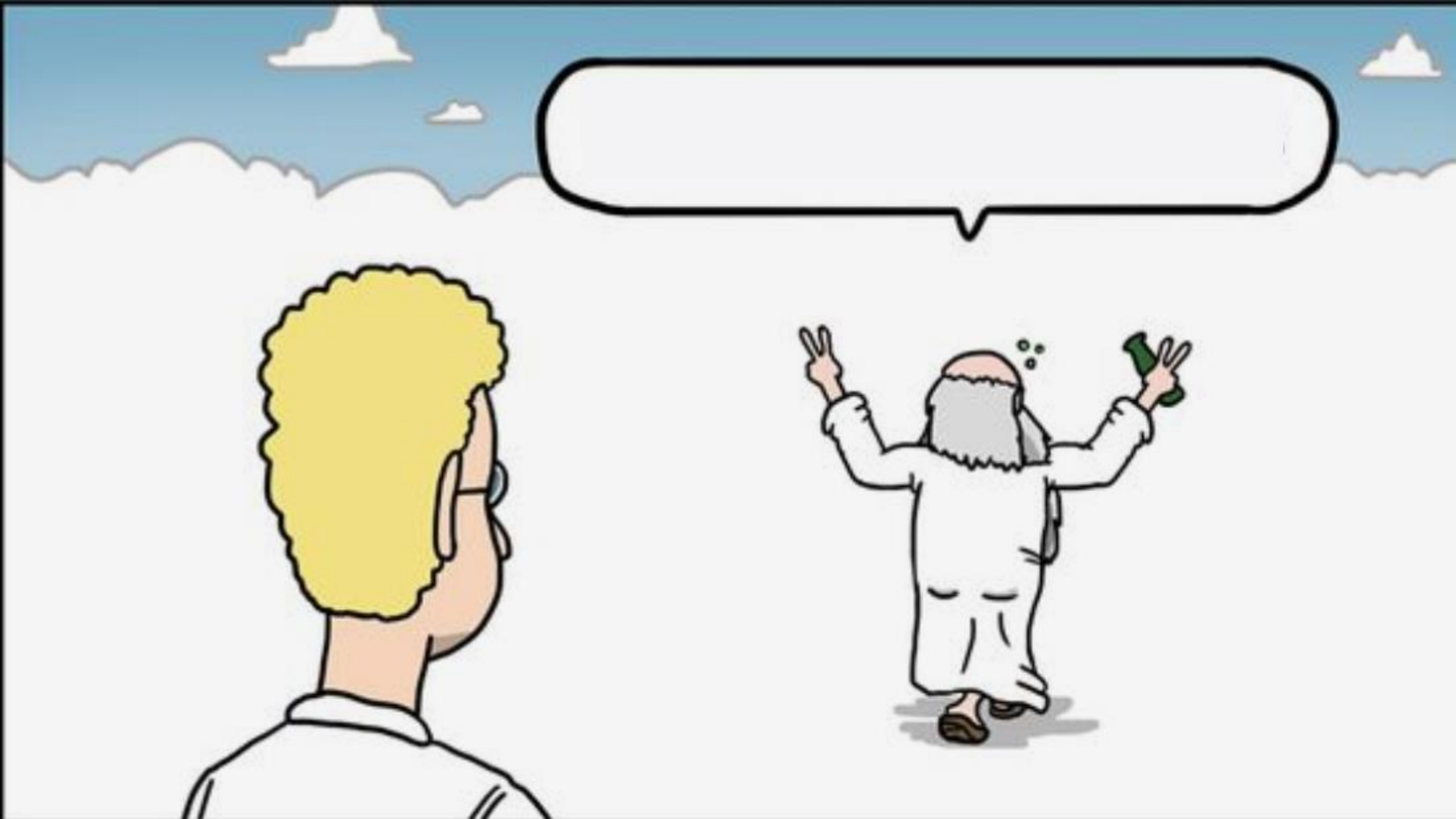
```
DECLARE
    TYPE t IS TABLE OF INTEGER;
    s1 t := t(1,1,2,2,3,4);
    s2 t := t(2,4,4,6,6, 8, 8);
    s3 t;
    s4 t := t(3, 4);
    card number(4);
BEGIN
    IF s3 IS NULL THEN
        dbms_output.put_line('EMPTY AND NULL');
    ELSIF s3 IS NOT NULL AND s3 IS EMPTY THEN
        dbms_output.put_line('EMPTY AND NOT NULL');
    ELSIF s3 IS NOT NULL AND s3 IS NOT EMPTY THEN
        dbms_output.put_line('NOT EMPTY AND NOT NULL');
    END IF;

    s3 := s2 MULTISET UNION S1; -- 2 4 4 6 6 8 8 1 1 2 2 3
4   s3 := SET(s3); -- 2 4 6 8 1 3
    s3 := s1 MULTISET INTERSECT S2; -- 2 4
    s3 := s1 MULTISET EXCEPT S2; -- 1 1 2 3
    s3 := s1 MULTISET EXCEPT DISTINCT S2; -- 1 3
    s3 := s2 MULTISET EXCEPT S1; -- 4 6 6 8 8
    s3 := s2 MULTISET EXCEPT DISTINCT S1; -- 6 8

     IF s4 SUBMULTISET OF s1 THEN
        dbms_output.put_line('TRUE');
    ELSE
        dbms_output.put_line('FALSE');
    END IF;

    card := CARDINALITY(s1); -- 6
    card := CARDINALITY(SET(s1)); -- 4

    IF &VAL MEMBER OF S1 THEN
        dbms_output.put_line('TRUE');
    END IF;
END;
/
```

Collection Methods

A **collection method** is a PL/SQL **subprogram**, either a **function** that **returns information** about a collection or a **procedure** that **operates** on a collection.

Collection methods make collections easier to use and applications easier to maintain. The syntax for a method invocation is "collection_name.method".

Method	Type	Description
DELETE	Procedure	Deletes elements from collection.
TRIM	Procedure	Deletes elements from end of varray or nested table.
EXTEND	Procedure	Adds elements to end of varray or nested table.
EXISTS	Function	Returns TRUE if and only if specified element of varray or nested table exists.
FIRST	Function	Returns first index in collection.
LAST	Function	Returns last index in collection.
COUNT	Function	Returns number of elements in collection.
LIMIT	Function	Returns maximum number of elements that collection can have.
PRIOR	Function	Returns index that precedes specified index.
NEXT	Function	Returns index that succeeds specified index.



Collection Methods - **DELETE**

DELETE is a **procedure** that removes elements from a **collection**. "**DELETE**" **removes all** elements from a collection of **any type**. This operation **immediately** frees the memory allocated by the deleted elements.

For an **associative array** or **nested table** (but **not** a **varray**):

- **DELETE**(n) **removes the** element whose **index** is **n** if that element exists, otherwise it does nothing
- **DELETE**(m,n) **removes all** elements whose **indexes** are in the **range** m..n if both m and n exist and m <= n, otherwise it does nothing

For the above forms of **DELETE** PL/SQL keeps **placeholders** for the deleted elements. The deleted elements are included in the internal size of the collection, and they can be "restored" by assigning a valid value to it. (the cell, not the value of the element)

```
DECLARE
  TYPE nt is TABLE OF NUMBER(4);
  t nt := nt(11, 22, 33, 44, 55, 66);
  i  NUMBER;
BEGIN
  t.DELETE(2);      -- Delete second element
  t(2) := 2222;     -- Restore second element
  t.DELETE(2, 4);   -- Delete range of elements
  i := t.FIRST;
  WHILE i IS NOT NULL LOOP
    dbms_output.put_line(t(i));
    i := t.NEXT(i);
  END LOOP;
  t(3) := 3333;     -- Restore third element
  t.DELETE;         -- Delete all elements
END;
/
```

```
DECLARE
  TYPE nt IS TABLE OF NUMBER(4) INDEX BY VARCHAR2(10);
  t nt := nt('a'=>1, 'b'=>2, 'c'=>30, 'd'=>4, 'e'=>5, 'f'=>6);
  i VARCHAR2(10);
BEGIN
  t('c') := 300;  -- Replace element
  t('g') := 7; -- Add a new element
  t.DELETE('c');
  t.DELETE('g');
  t('c') := 3; -- Replace deleted element with new value
  t('g') := 7; -- Replace deleted element with same value
  t('h') := 7; -- Add a new element
  t('i') := 7; -- Add a new element
  t('j') := 7; -- Add a new element
  t.DELETE('c','e');  -- Delete range of elements
  t.DELETE('j','f');  -- Does nothing

  i := t.FIRST;
  WHILE i IS NOT NULL LOOP
    dbms_output.put_line(i||' '||t(i));
    i := t.NEXT(i);
  END LOOP;

  t.DELETE; -- Delete all elements
END;
/
```

BEFORE

AFTER

Collection Methods - **TRIM**

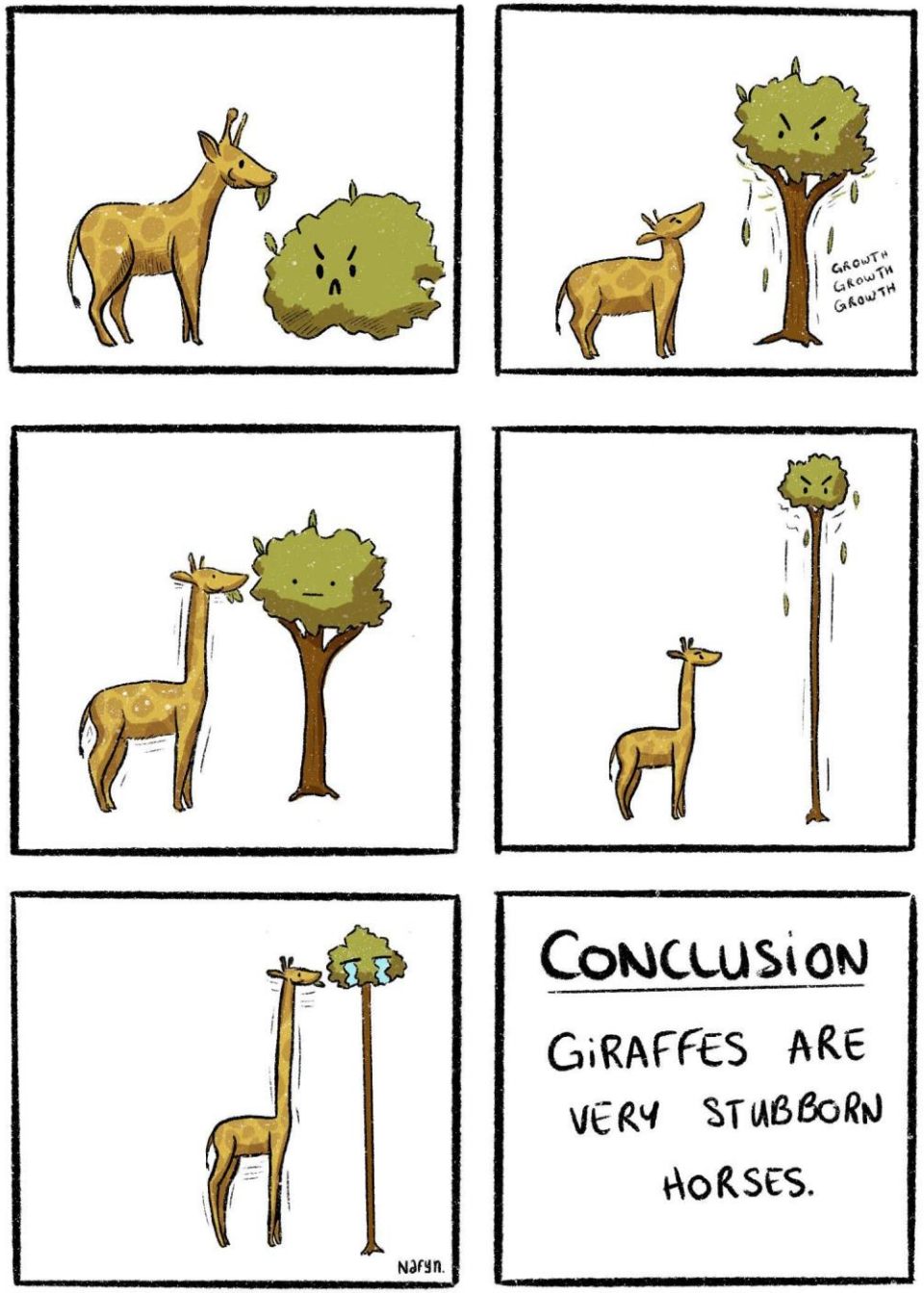
TRIM is a **procedure** that **deletes** elements from the **end** of a **varray** or **nested table**. This method has these forms:

- **TRIM** removes **one element** from the **end** of the collection if the collection has at least one element, otherwise it **raises** the predefined exception **SUBSCRIPT_BEYOND_COUNT**
- **TRIM**(n) removes **n elements** from the **end** of the collection if there are at least n elements at the end, otherwise it **raises** the predefined exception **SUBSCRIPT_BEYOND_COUNT**

TRIM operates on the **internal size** of a collection. It considers the element to exist: if **DELETE** removed an element but **kept** a **placeholder** for it, **TRIM** does a **clean wipe**. PL/SQL **does not** keep **placeholders** for **trimmed** elements. (Trimmed elements are not included in the internal size of the collection; a trimmed element cannot be restored by assigning a valid value to it)

```
DECLARE
  TYPE nt is TABLE OF NUMBER(4);
  t nt := nt(1, 2, 3, 4, 5, 6);
  i  NUMBER;
BEGIN
  t.TRIM; -- Trim last element
  t.DELETE(4); -- Delete fourth element
  t.TRIM(2); -- Trim last two element
  i := t.FIRST;
  WHILE i IS NOT NULL LOOP
    dbms_output.put_line(t(i));
    i := t.NEXT(i);
  END LOOP;
END;
/
```

GIRAFFE: THE GENESIS



Collection Methods - **EXTEND**

EXTEND is a **procedure** that **adds elements** to the **end** of a **varray** or **nested table**. The collection **can be empty**, but **not null** (Use a constructor to make a collection empty or add elements to a null collection). The **EXTEND** method has these forms:

- **EXTEND**: appends **one null** element to the end of the collection
- **EXTEND**(n): appends **n null** elements to the end of the collection
- **EXTEND**(n,i): appends **n copies** of the **i-th elemen**t to the collection. Is the **only** form that you can use for a collection whose elements have the **NOT NULL** constraint.

EXTEND operates on the **internal size** of a collection, it considers the element **to exist**, like **TRIM**, unlike **DELETE**.

```
DECLARE
  TYPE nt is TABLE OF NUMBER(4);
  t nt := nt(1, 2, 3, 4, 5, 6);
  i  NUMBER;
BEGIN
  t.EXTEND(2, 3); -- Append two copies of first element
  t.DELETE(8); -- Delete eighth element
  t.EXTEND; -- Append one null element
  i := t.FIRST;
  WHILE i IS NOT NULL LOOP
    dbms_output.put_line(i||' '||t(i));
    i := t.NEXT(i);
  END LOOP;
END;
/
```

It's Free Real Estate

Collection Methods - **EXISTS**

EXISTS is a **function** that tells you whether the specified element of a **varray** or **nested table** exists.

EXISTS(n): returns **TRUE** if the **n-th element** of the collection exists and **FALSE** otherwise.

If **n** is **out of range EXISTS** returns **FALSE** instead of raising the predefined exception **SUBSCRIPT_OUTSIDE_LIMIT**.

EXISTS(n) returns **FALSE** for a **deleted element** (even if **DELETE** kept a **placeholder** for it).

```
DECLARE
  TYPE rec IS RECORD(
      country_name countries.country_name%TYPE,
      region_name regions.region_name%TYPE
    );
  TYPE tbl IS TABLE OF rec;
  t tbl;
BEGIN

  SELECT country_name, region_name
  BULK COLLECT INTO t
  FROM countries natural join regions;

  dbms_output.put_line(t.FIRST); -- 1
  dbms_output.put_line(t.LAST); -- 25
  dbms_output.put_line(t.COUNT); -- 25

  t.DELETE(5, 15); -- Delete range

  FOR i IN t.FIRST .. t.LAST
  LOOP
    IF t.EXISTS(i) THEN -- test if exists
      dbms_output.put_line(t(i).country_name||' '||t(i).region_name);
    END IF;
  END LOOP;

  dbms_output.put_line(t.FIRST); -- 1
  dbms_output.put_line(t.LAST); -- 25
  dbms_output.put_line(t.COUNT); -- 14

END;
/
```

i am at my limit.

Collection Methods – **COUNT** and **LIMIT**

COUNT is a **function** that returns the **number of elements** in the **collection**. **COUNT ignores** deleted elements, even if **DELETE** kept **placeholders** for them.

For a **nested table COUNT** is smaller than **LAST** (the **index** of the **last** element) if elements from the **middle** of the nested table are **deleted**.

https://pt.memedroid.com/memes/detail/3965907/i-am-at-my-limit-too

LIMIT is a function that returns the **maximum number** of elements that the collection can have.

LIMIT returns **NULL** if the collection has no maximum number of elements. (Only a varray has a maximum size)

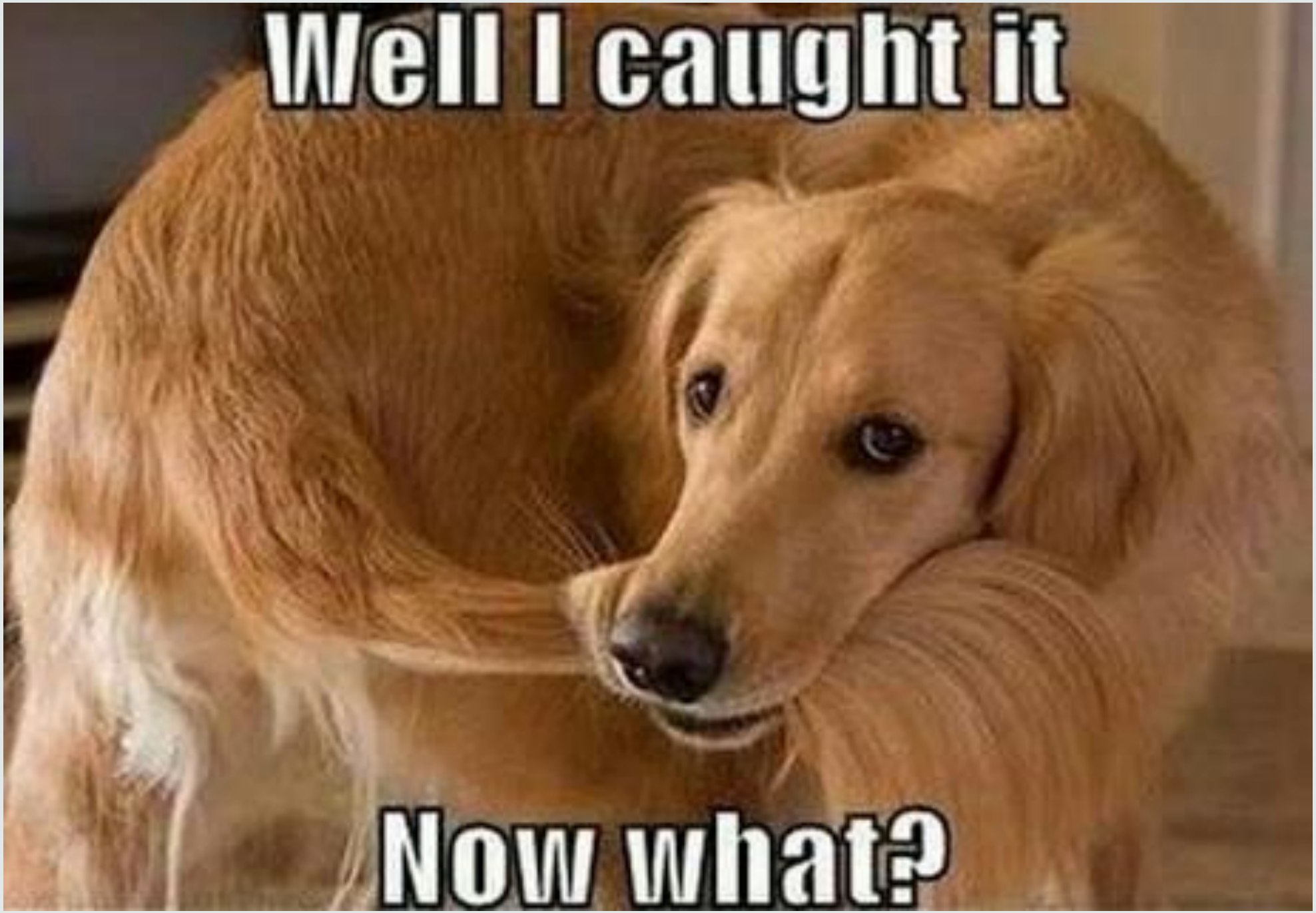
Collection Methods – **FIRST** and **LAST**

FIRST and **LAST** are functions. If the **collection** has at least one element **FIRST** return the **index** of the first element and **LAST** returns the **index** of the last element. They **ignore deleted** elements even if **DELETE** kept **placeholders**. **FIRST** and **LAST** return the same **index** if the collection has only one element. **FIRST** and **LAST** return **NULL** if the collection is **empty**.

- For an **associative array**, indexed by **PLS_INTEGER**: The **FIRST** element is the **smallest index** and the **LAST** element is the **largest index**
- For an **associative array**, indexed by **string**: The **FIRST** element is the **lowest key value** and the **LAST** element is the **highest key value**
- For a **varray** that is **not empty**: **FIRST** always returns **1** and **LAST** always equals **COUNT**
- For a **nested table**: **LAST** equals **COUNT** if elements from the **middle** of the nested table are **NOT deleted** and **LAST** is larger than **COUNT** if elements from its middle are **deleted**.

```
DECLARE
  TYPE rec IS RECORD(
      country_name countries.country_name%TYPE,
      region_name regions.region_name%TYPE
    );
  TYPE tbl IS TABLE OF rec;
  t tbl;
BEGIN

  SELECT country_name, region_name
  BULK COLLECT INTO t
  FROM countries natural join regions;

  dbms_output.put_line(t.FIRST); -- 1
  dbms_output.put_line(t.LAST); -- 25
  dbms_output.put_line(t.COUNT); -- 25

  t.DELETE(5, 15); -- Delete range

  FOR i IN t.FIRST .. t.LAST
  LOOP
    IF t.EXISTS(i) THEN -- test if exists
      dbms_output.put_line(t(i).country_name||' '||t(i).region_name);
    END IF;
  END LOOP;

  dbms_output.put_line(t.FIRST); -- 1
  dbms_output.put_line(t.LAST); -- 25
  dbms_output.put_line(t.COUNT); -- 14

END;
/
```

```
DECLARE
  TYPE tbl IS TABLE OF countries.country_name%TYPE;
  TYPE vec IS VARRAY(100) OF regions.region_name%TYPE;
  TYPE rec IS RECORD(
      country_names tbl,
      region_names vec
    );
  r rec;
BEGIN

  SELECT country_name
  BULK COLLECT INTO r.country_names
  FROM countries;

  SELECT region_name
  BULK COLLECT INTO r.region_names
  FROM regions;

  dbms_output.put_line('country_names.FIRST '||r.country_names.FIRST); -- 1
  dbms_output.put_line('country_names.LAST '||r.country_names.LAST); -- 25
  dbms_output.put_line('country_names.COUNT '||r.country_names.COUNT); -- 25
  dbms_output.put_line('country_names.LIMIT '||r.country_names.LIMIT); -- NULL

  dbms_output.put_line('region_names.FIRST '||r.region_names.FIRST); -- 1
  dbms_output.put_line('region_names.LAST '||r.region_names.LAST); -- 4
  dbms_output.put_line('region_names.COUNT '||r.region_names.COUNT); -- 4
  dbms_output.put_line('region_names.LIMIT '||r.region_names.LIMIT); -- 100

  FOR i IN r.country_names.FIRST .. r.country_names.LAST
  LOOP
    IF r.country_names.EXISTS(i) THEN -- test if exists
      dbms_output.put_line(r.country_names(i));
    END IF;
  END LOOP;
END;
/
```


Well I caught it
Now what?

Collection Methods – **PRIOR** and **NEXT**

PRIOR and **NEXT** are **functions** that let you **move backward** and **forward** in the collection. They **ignore** deleted elements, even if **DELETE** kept **placeholders** for them and are useful for **traversing sparse** collections. Given an **index** (**must exist**):

- **PRIOR**: If it **exists**, then **PRIOR** returns the index of the **preceding existing** element else **PRIOR** returns **NULL**
- **NEXT**: If it **exists**, then **NEXT** returns the index of the **succeeding existing** element else **NEXT** returns **NULL**

For an **associative array indexed** by **string** **PRIOR** and **NEXT** indexes are determined by key values (always sorted order).

If the collection c is a varray and the index exceeds c.LIMIT then c.PRIOR(index) returns c.LAST and c.NEXT(index) returns NULL.

c.PRIOR(c.FIRST) returns NULL for any collection c

c.NEXT(c.LAST) returns NULL for any collection c

```
DECLARE
    TYPE column IS TABLE OF INTEGER INDEX BY
PLS_INTEGER;
    TYPE row IS TABLE OF column INDEX BY PLS_INTEGER;
    mat row;
    tmat row;
    l STRING(100);
    i PLS_INTEGER;
    j PLS_INTEGER;
BEGIN
    mat := row( 1 => column(1=>1, 2=> 2, 3=> 3, 4=> 4),
                2 => column(1=>5, 2=> 6, 3=> 7, 4=> 8),
                3 => column(1=>9, 2=>10, 3=>11, 4=>12));

    dbms_output.put_line('Original Matrix');
    i := mat.LAST;
    WHILE i IS NOT NULL
    LOOP
        j := mat(i).LAST;
        WHILE j IS NOT NULL
        LOOP
            tmat(j)(i) := mat(i)(j);
            j := mat(i).PRIOR(j);
        END LOOP;
        i := mat.PRIOR(i);
    END LOOP;

    dbms_output.put_line('Transpose Matrix');
    i := tmat.FIRST;
    WHILE i IS NOT NULL
    LOOP
        l := '';
        j := tmat(i).FIRST;
        WHILE j IS NOT NULL
        LOOP
            l := l || tmat(i)(j) || ' ';
            j := tmat(i).NEXT(j);
        END LOOP;
        dbms_output.put_line(l);
        i := tmat.NEXT(i);
    END LOOP;

END;
/
```

```sql
CREATE OR REPLACE TYPE employees_vec AS VARRAY(20) OF varchar2(20);
/

CREATE TABLE EmployeesDeparments(
  numedept varchar2(30),
  angajati Employees_vec
);

DECLARE
  emps employees_vec;
  idd departments.department_id%TYPE := 110;
  dname departments.department_name%TYPE;
BEGIN
  SELECT department_name INTO dname FROM departments WHERE department_id =
idd;
  SELECT first_name||' '||last_name
  BULK COLLECT INTO emps
  FROM employees
  WHERE department_id = idd;

  INSERT INTO EmployeesDeparments values(dname, emps);
END;
/

SELECT * FROM EmployeesDeparments;

DROP TABLE EmployeesDeparments;
DROP TYPE employees_vec;
```

```
CREATE OR REPLACE TYPE jobs_nt AS TABLE OF varchar2(30);
/

CREATE TABLE EmployeesJobs(
  ename varchar2(40),
  ejobs jobs_nt
)
NESTED TABLE ejobs STORE AS ejobs_store;

set serveroutput on;
DECLARE
  ejobs jobs_nt;
  ide employees.employee_id%TYPE := 101;
  ename varchar2(40);
BEGIN
  SELECT first_name||' '||last_name INTO ename
  FROM employees WHERE employee_id = ide;

  WITH
    q1 as (SELECT e.employee_id, j.job_title
            FROM employees e
              INNER JOIN job_history jh ON jh.employee_id=e.employee_id
              INNER JOIN jobs j ON j.job_id = jh.job_id
            UNION
            SELECT e.employee_id, j.job_title
            FROM employees e
              INNER JOIN jobs j ON j.job_id = e.job_id)
  SELECT job_title BULK COLLECT INTO ejobs
  FROM q1
  WHERE q1.employee_id = ide;

  INSERT INTO EmployeesJobs values(ename, ejobs);

END;
/

SELECT * FROM EmployeesJobs;

DROP TABLE EmployeesJobs;

DROP TYPE jobs_nt;
```

Bibliography

Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference* 23ai, Oracle Corporation, 2024 [pdf]

Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 19c*, Oracle Corporation, 2022 [pdf]

Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 21c*, Oracle Corporation, 2022 [pdf]

Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 19c*, Oracle Corporation, 2020 [pdf]

Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 21c*, Oracle Corporation, 2021 [pdf]

Resources

- Flaticon:
  - https://www.flaticon.com/free-icon/time-record_1576640
  - https://www.flaticon.com/free-icon/dictionary_5402323
  - https://www.flaticon.com/free-icon/array_8093284
  - IStock Photo: https://www.istockphoto.com/ro/vector/verifica%C8%9Bi-marcajele-butonul-ui-cu-dos-%C8%99i-donts-plat-simplu-stil-tendin%C8%9B%C4%83-modern%C4%83-gm1163207144-319328344
- Memes:
  - https://i.pinimg.com/736x/f9/c6/24/f9c624561595995676ffec4d360257e7.jpg
  - https://ro.pinterest.com/pin/225186854727330 6/
  - https://www.dailydot.com/unclick/tim-eric-free-real-estate-meme-reddit/
  - https://www.reddit.com/r/funny/comments/hd51wn/giraffes_the_genesis/
  - https://pt.memedroid.com/memes/detail/3965907/i-am-at-my-limit-too
  - https://upload.wikimedia.org/wikipedia/commons/a/a5/Glazed-Donut.jpg
  - https://pt.memedroid.com/memes/detail/2104382

University
Politehnica
of Bucharest

PL/SQL (IV)

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@cs.pub.ro

Overview

- PL/SQL Error Handling
- Cursors

Overview

- PL/SQL Error Handling
- Cursors

PL/SQL Error Handling

- Exceptions (PL/SQL runtime errors) can arise from
  - Design faults
  - Coding mistakes
  - Hardware failures
  - Many other sources

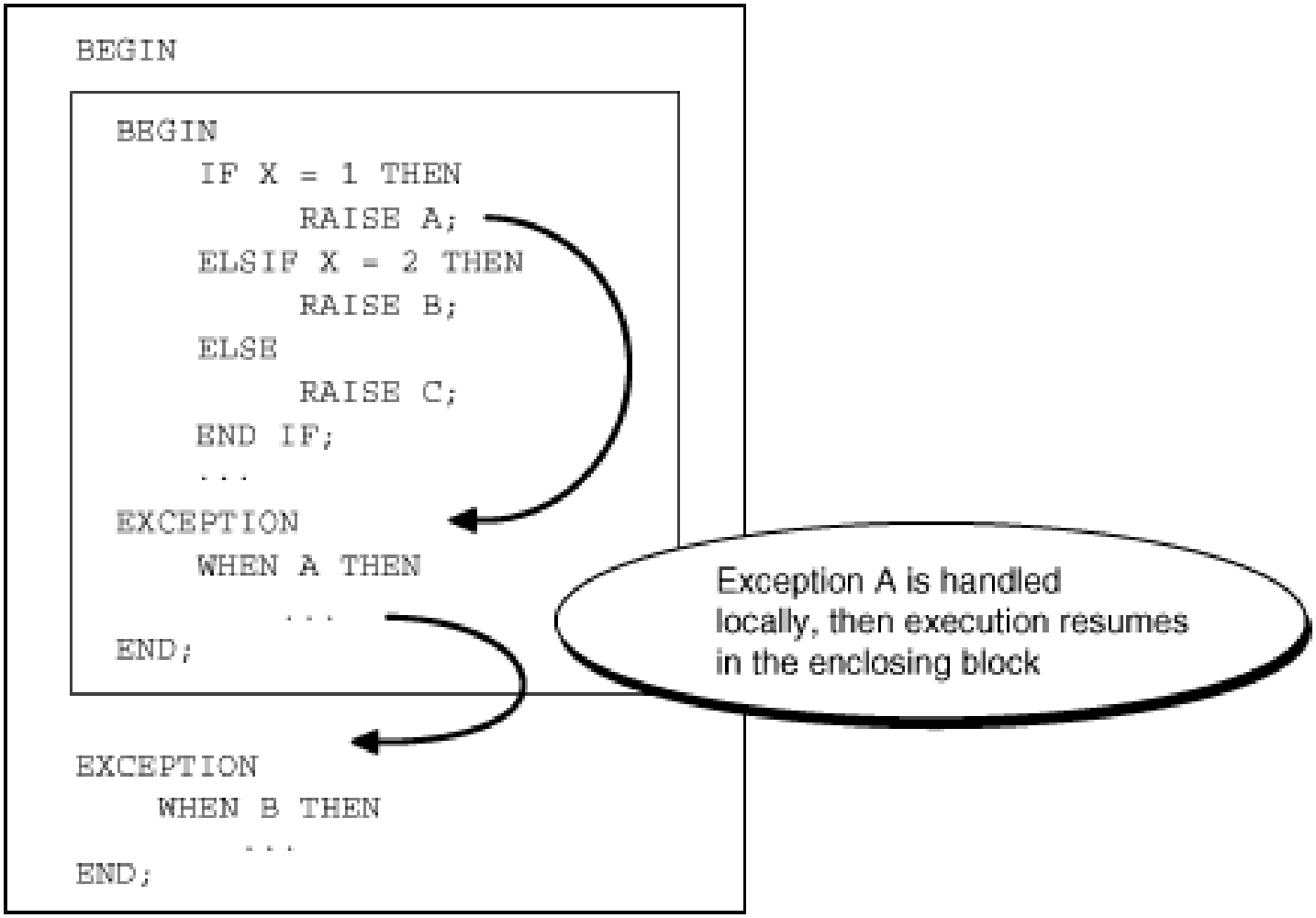
- A developer
  - Cannot anticipate all possible exceptions
  - Can write exception handlers that lets the program continue to operate in their presence

- Any PL/SQL block can have an exception-handling part:

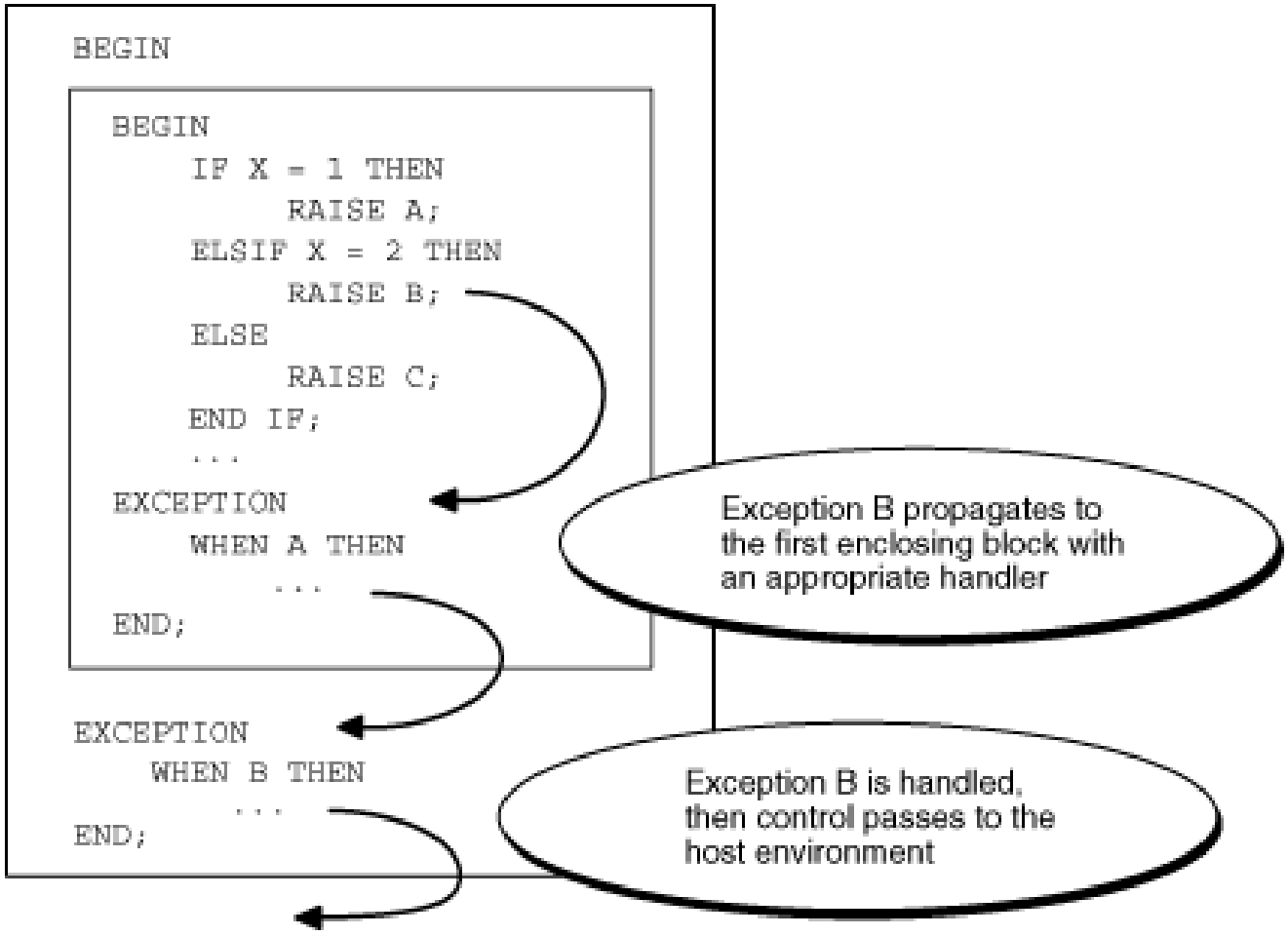
EXCEPTION

- The exception-handling part can have one or more exception handlers

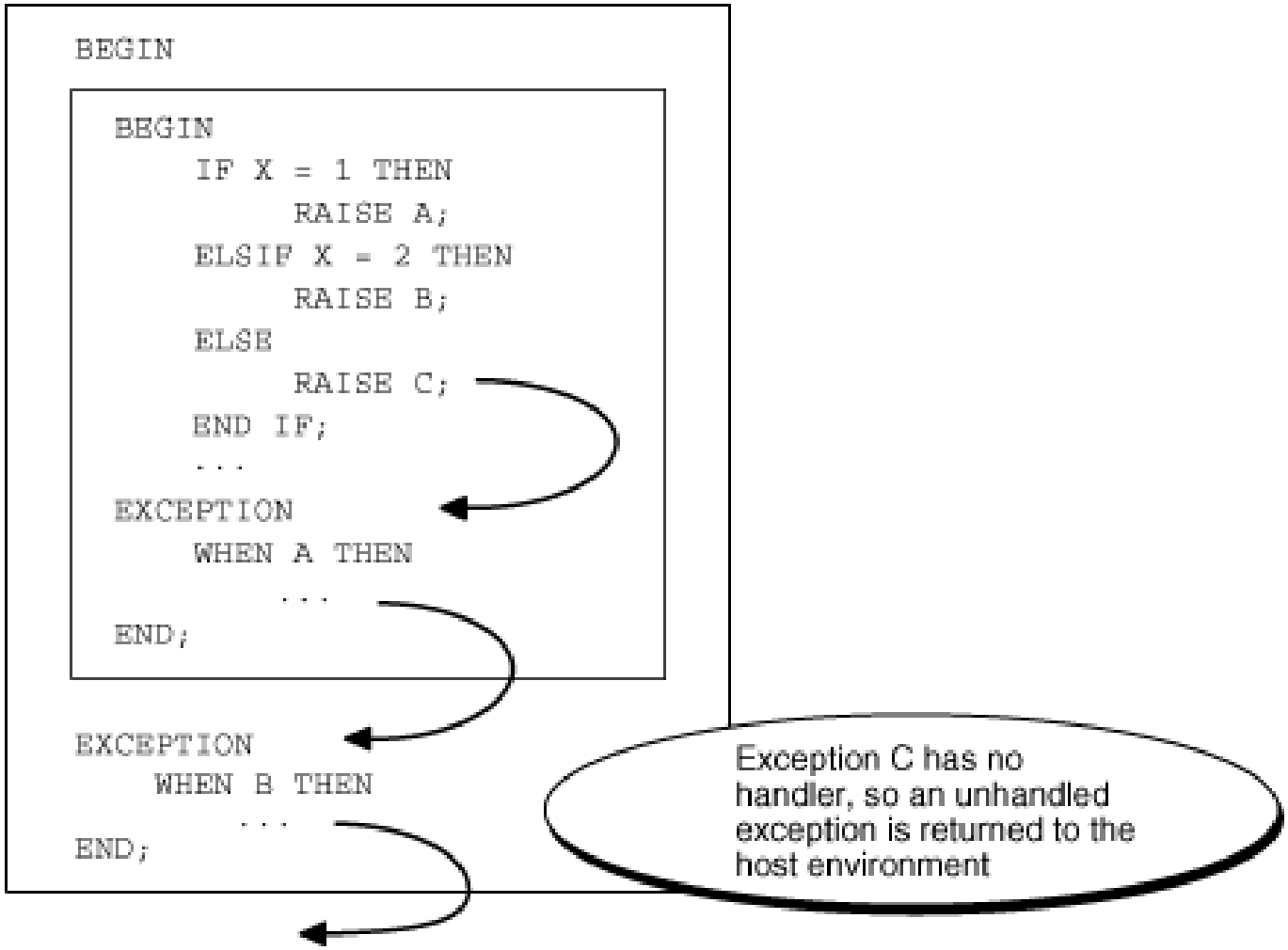
PL/SQL Error Propagation



PL/SQL Error Propagation



PL/SQL Error Propagation



PL/SQL Error Propagation

- When an exception is raised in the executable part of the block
  - The executable part stops
  - Control transfers to the exception-handling part

- If there is no enclosing block:
  - If the exception handler is in a subprogram, then control returns to the invoker, at the statement after the invocation.
  - If the exception handler is in an anonymous block, then control transfers to the host environment (for example, SQL*Plus)

- If there is an enclosing block:
  - Control transfers to the next statement of the enclosing block after an exception handler runs
  - The exception propagates if an exception is raised in a block that has no exception handler
  - The exception reproduces itself in successive enclosing blocks until a block has a handler for it or there is no enclosing block
  - If there is no handler for the exception, then PL/SQL returns an unhandled exception error to the invoker or host environment, which determines the outcome

PL/SQL Error Handling

- Exception categories:
  1. Internally defined
     - The runtime system raises internally defined exceptions implicitly (automatically).
     - Always has an error code
     - Does not have a name unless PL/SQL gives it one or you give it one
     - E.g.
       - ORA-00060 (deadlock detected while waiting for resource)
       - ORA-27102 (out of memory)

PL/SQL Error Handling

- The exception categories are:
  2. Predefined
     - A predefined exception is an internally defined exception that PL/SQL has given a name.
     - E.g.: ORA-06500 (PL/SQL: storage error) has the predefined name STORAGE_ERROR

  3. User-defined
     - Developers can declare their own exceptions in the declarative part of any PL/SQL
       - Anonymous block
       - Subprogram
       - Package
     - User-defined exceptions are raised explicitly
     - E.g.: Declare an exception named insufficient_funds to flag overdrawn bank accounts

PL/SQL Error Handling

Category	Definer	Has Error Code	Has Name	Raised Implicitly	Raised Explicitly
Internally defined	Runtime system	Always	Only if you assign one	Yes	Optionally
Predefined	Runtime system	Always	Always	Yes	Optionally
User-defined	User	Only if you assign one	Always	No	Always

- SQLCODE – function to get the error code
- SQLERRM – function to get the error message

PL/SQL Error Handling
Internally defined exception

- Internally defined exceptions (ORA-n errors)
  - Are described in Oracle Database Error Messages Reference
  - The runtime system raises them implicitly (automatically).
  - Does not have a name unless either PL/SQL gives it one or a developer gives it one
  - Can be handled only with **OTHERS** exception handlers.

- To give a name to an internally defined exception, do the following in the declarative part of the appropriate anonymous block, subprogram, or package:
  - Declare the name
  - Associate the name with the error code of the internally defined exception

PL/SQL Error Handling
Internally defined exception example

```
INSERT INTO employees(first_name, last_name)
VALUES('Ion', 'Ionescu');
```

PL/SQL Error Handling
Internally defined exception example

```
 1 DECLARE
 2   insert_null_exp exception;
 3   PRAGMA exception_init(insert_null_exp, -1400);
 4 BEGIN
 5   INSERT INTO employees(first_name, last_name)
 6   VALUES('Ion', 'Ionescu');
 7 EXCEPTION
 8   WHEN insert_null_exp THEN
 9     dbms_output.put_line('Error msg  '||SQLERRM);
10     dbms_output.put_line('Error code '||SQLCODE);
11   WHEN others THEN
12     dbms_output.put_line('Others error');
13 END;
14 /
```

PL/SQL Error Handling Pragma

- A pragma is an instruction to the compiler that is processed at compilation time
- A pragma begins with the reserved word PRAGMA followed by the name of the pragma
- Some pragmas have arguments
- A pragma may appear before a declaration or a statement
- Additional restrictions may apply for specific pragmas
- The extent of a pragma's effect depends on the pragma
- A pragma whose name or argument is not recognized by the compiler has no effect.

PL/SQL Error Handling
Predefined exceptions

- Predefined exceptions are internally defined exceptions that have predefined names
- PL/SQL declares the names of the predefined exceptions globally in the package STANDARD
- The runtime system raises predefined exceptions implicitly (automatically)
- Developers can write exception handlers specifically for predefined exceptions because they have names
- To explicitly raise an exception use RAISE

PL/SQL Error Handling
Predefined exceptions example

```
 1 DECLARE
 2   n number := &n;
 3 BEGIN
 4   IF n < 0 THEN
 5     RAISE INVALID_NUMBER;   -- raise explicitly
 6   END IF;
 7 EXCEPTION
 8   WHEN INVALID_NUMBER THEN
 9     dbms_output.put_line('n < 0');
10     dbms_output.put_line('Error msg  '||SQLERRM);
11     dbms_output.put_line('Error code '||SQLCODE);
12   WHEN others THEN
13     dbms_output.put_line('Others error');
14 END;
15 /
```

PL/SQL Error Handling
User-defined exceptions

- Developers can declare their own exceptions in the declarative part of any PL/SQL anonymous block, Subprogram, or Package

- An exception name declaration has this syntax:

exception_name exception;

- Developers can attach to a user defined exception
  - An exception code (-20999 și -20000) using

PRAGMA exception_init(exception_name, exception_code);

  - An exception message using the procedure

RAISE_APPLICATION_ERROR(exception_code, exception_message [, TRUE | FALSE]);

  - **TRUE** – PL/SQL puts **exception_code** on top of the error stack
  - **FALSE (default) –** PL/SQL replaces the error stack with **exception_code** (i.e., all previous errors are replaced with the current one)

PL/SQL Error Handling
User-defined exceptions Example

```
DECLARE
  n number := &n;
  my_exp1 exception;
  my_exp2 exception;
  PRAGMA exception_init(my_exp1, -20100);
  PRAGMA exception_init(my_exp2, -20200);
BEGIN
  IF n >= 200  THEN
    RAISE my_exp1;
  ELSIF n >= 100 THEN
    raise_application_error(-20200, 'Numar > 200');
  END IF;
EXCEPTION
  WHEN my_exp1 THEN
    dbms_output.put_line('n >100');
    dbms_output.put_line('my_exp1 error msg  '||SQLERRM);
    dbms_output.put_line('my_exp1 error code '||SQLCODE);
   WHEN my_exp2 THEN
    dbms_output.put_line('n > 200');
    dbms_output.put_line('my_exp2 error msg  '||SQLERRM);
    dbms_output.put_line('my_exp2 error code '||SQLCODE);
  WHEN others THEN
    dbms_output.put_line('Others error');
END;
/
```

Overview

- PL/SQL Error Handling
- Cursors

PL/SQL Static SQL

- Static SQL is a PL/SQL feature that allows SQL syntax to be used directly in a block

- PL/SQL static SQL statements (some have a modified syntax):
  - SELECT
  - INSERT
  - UPDATE
  - DELETE
  - MERGE
  - COMMIT
  - ROLLBACK
  - SAVEPOINT
  - SET TRANSACTION
  - LOCK TABLE

PL/SQL Static SQL

- A PL/SQL static SQL statement can have a PL/SQL identifier (names given to a PL/SQL object) wherever its SQL counterpart can have a placeholder for a bind variable

- The PL/SQL identifier must identify either a variable or a formal parameter

- Bind variables
  - Are used to reuse the query parse tree and execution plan when new values are given
  - Can be used in dynamic PL/SQL code, but that's not good practice!
  - Are a placeholder for values within the query.
  - Unlike substitution variables, these are not prompted for when you come to compile the code.

PL/SQL Dynamic SQL

```
DECLARE
    v_sql varchar2(200);
    v_dname departments.department_name%type;
    id_dept departments.department_id%type := &id;
BEGIN
    v_sql := 'SELECT department_name FROM departments WHERE department_id = :1';
    EXECUTE IMMEDIATE v_sql INTO v_dname USING id_dept;
    dbms_output.put_line(v_dname);
END;
/
```

PL/SQL Static SQL

```
 1 DECLARE
 2     v_sql varchar2(200);
 3     v_dname departments.department_name%type;
 4     id_dept departments.department_id%type := &id;
 5 BEGIN
 6     SELECT department_name
 7     INTO v_dname
 8     FROM departments
 9     WHERE department_id = id_dept;
10
11     dbms_output.put_line(v_dname);
12 END;
13 /
14
15 -- SQL Engine:
16 SELECT department_name FROM departments WHERE department_id = :1;
```

PL/SQL Static SQL

- The values of some variables are undefined after PL/SQL code runs a DML statement:
  - After a FETCH or SELECT statement raises an exception
    - The values of the define variables after that statement are undefined.
  - After a DML statement that affects zero rows
    - The values of the OUT bind variables are undefined unless the DML statement is a BULK or multiple-row operation

PL/SQL Static SQL

- Static SQL includes these SQL pseudocolumns (for different types of objects):
  - CURRVAL and NEXTVAL
  - LEVEL
    - Hierarchical Query Pseudocolumns (SELECT queries with START WITH ... CONNECT BY)
    - Returns 1 for a root row, 2 for a child of a root, and so on.
  - OBJECT_VALUE
    - Returns system-generated names for the columns of an object table, XMLType table, object view, or XMLType view
    - Useful for
      - Identifying the value of a substitutable row in an object table
      - Creating object views with the WITH OBJECT IDENTIFIER clause.
  - ROWID - returns the physical address of the row
  - ROWNUM - returns a number indicating the order in which Oracle selects the row from a table or set of joined rows for each row returned by a query

PL/SQL Static SQL

- To access the values of a sequence in SQL statements:
  - CURRVAL pseudocolumn
    - Returns the current value of the sequence
  - NEXTVAL pseudocolumn
    - Increments the sequence
    - Returns the new value

- Use dot notation to reference these pseudocolumns :

  sequence_name.CURRVAL

  sequence_name.NEXTVAL

- Each time sequence_name.NEXTVAL is reference
  - The sequence is incremented immediately and permanently
  - Commit or roll back on the transaction will not decrement the sequence

PL/SQL Static SQL

- Use sequence_name.CURRVAL and sequence_name.NEXTVAL in a PL/SQL expression wherever NUMBER expression can be used.

- Note:
  - Using sequence_name.CURRVAL or sequence_name.NEXTVAL to provide a **default value for an ADT method** parameter causes a **compilation error**
  - **PL/SQL evaluates every occurrence** of sequence_name.CURRVAL and sequence_name.NEXTVAL
  - **SQL evaluates a sequence expression for every row** in which it appears.

Cursors

- A cursor is a pointer to a private SQL area that stores information about processing a specific SELECT or DML statement
- A session cursor lives in session memory until the session ends
- When the session ends the cursor ceases to exist
- Cursor types:
  - Implicit
    - Constructed and managed by PL/SQL
  - Explicit
    - Construct and manage by the developer

Cursors

- Cursor attributes
  - Are used to get information about any session cursor
  - Can be referenced in procedural statements, but not in SQL statements
- Query the dynamic performance view V$OPEN_CURSOR to list the session cursors that each user session currently has opened and parsed
- The number of cursors that a session can have open simultaneously is determined by:
  - The amount of memory available to the session
  - The value of the initialization parameter OPEN_CURSORS

Cursors

- Generally, PL/SQL parses
  - An explicit cursor only the first time the session opens it
  - A SQL statement (creating an implicit cursor) only the first time the statement runs
- All parsed SQL statements are cached
- A SQL statement is reparsed only if it is aged out of the cache by a new SQL statement
- PL/SQL does not reparse the associated query if an explicit cursor is closed and immediately reopened

Implicit Cursors

- An implicit cursor is a session cursor that is constructed and managed by PL/SQL
- PL/SQL opens an implicit cursor every time SELECT or DML statement is executed
- A developer:
  - Cannot control an implicit cursor
  - Can get information from its attributes.

Implicit Cursors

- The syntax of an implicit cursor attribute value is SQL%attribute
- An implicit cursor is also called a SQL cursor
- SQL%attribute always refers to the most recently run SELECT or DML statement
- If no such statement has run, the value of SQL%attribute is NULL
- An implicit cursor closes after its associated statement runs
- Its attribute values remain available until another SELECT or DML statement runs

Implicit Cursors

- The most recently run SELECT or DML statement might be in a different scope:
  - To save an attribute value for later use, assign it to a local variable immediately
  - Otherwise, other operations, such as subprogram invocations, might change the value of the attribute before testing it
- The implicit cursor attributes are:
  - SQL%ISOPEN Attribute: Is the Cursor Open?
  - SQL%FOUND Attribute: Were Any Rows Affected?
  - SQL%NOTFOUND Attribute: Were No Rows Affected?
  - SQL%ROWCOUNT Attribute: How Many Rows Were Affected?
  - SQL%BULK_ROWCOUNT
  - SQL%BULK_EXCEPTIONS

Implicit Cursors

- SQL%ISOPEN always returns FALSE
  - An implicit cursor always closes after its associated statement runs
- SQL%FOUND returns:
  - NULL if no SELECT or DML statement has run
  - TRUE if a SELECT statement returned one or more rows or a DML statement affected one or more rows
  - FALSE otherwise
- SQL%NOTFOUND returns:
  - NULL if no SELECT or DML statement has run
  - FALSE if a SELECT statement returned one or more rows or a DML statement affected one or more rows
  - TRUE otherwise

Implicit Cursors

- The SQL%NOTFOUND attribute is not useful with the PL/SQL SELECT INTO statement:
  - If the SELECT INTO statement returns no rows, PL/SQL raises the predefined exception NO_DATA_FOUND immediately, before you can check SQL%NOTFOUND.
  - A SELECT INTO statement that invokes a SQL aggregate function always returns a value (possibly NULL), thus the SQL%NOTFOUND attribute is always FALSE, so checking it is unnecessary.

Implicit Cursors

- SQL%ROWCOUNT returns:
  - NULL if no SELECT or DML statement has run
  - Otherwise, the number of rows returned by a SELECT statement or affected by a DML statement (an INTEGER)
- If a SELECT INTO statement without a BULK COLLECT clause returns multiple rows
  - PL/SQL raises the predefined exception TOO_MANY_ROWS and SQL%ROWCOUNT returns 1
  - This is not the actual number of rows that satisfy the query
- The value of SQL%ROWCOUNT attribute is unrelated to the state of a transaction
- The value of SQL%ROWCOUNT is not restored
  - To the value it had before the savepoint when a transaction rolls back to a savepoint
  - To the original value in the parent transaction when an autonomous transaction ends

Implicit Cursors

- To allow a FORALL statement to continue even if some of its DML statements fail, include the SAVE EXCEPTIONS clause
- Use SQL%BULK_EXCEPTIONS in the FORALL clause to find the element that caused an error
- Use SQL%BULK_ROWCOUNT to get the total number of rows affected by the FORALL statement

Implicit Cursors

```
set serveroutput on
DECLARE
  TYPE rec IS RECORD(
    country_id countries.country_id%TYPE,
    country_name countries.country_name%TYPE,
    region_id countries.region_id%TYPE
  );
  TYPE aa IS TABLE OF rec INDEX BY PLS_INTEGER;
  v aa; -- create a null associative array
  err_idx PLS_INTEGER;
  err_count PLS_INTEGER;
BEGIN

  v := aa(1 => rec('RO', 'Romania', 1), 2 => rec('MD', 'Moldova', 1),
          5 => rec('BG', 'Bulgaria', 1));

  FORALL j IN v.FIRST .. v.LAST SAVE EXCEPTIONS
  INSERT INTO COUNTRIES VALUES v(j);

EXCEPTION
  WHEN others THEN
    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT);
    err_count := SQL%BULK_EXCEPTIONS.COUNT;
    FOR i IN 1..err_count LOOP
      DBMS_OUTPUT.PUT_LINE ('SQL Code '||SQL%BULK_EXCEPTIONS(i).ERROR_CODE);
      err_idx := SQL%BULK_EXCEPTIONS(i).ERROR_INDEX;
      DBMS_OUTPUT.PUT_LINE('Bad statement #: ' || err_idx);
    END LOOP;

    FOR i IN v.FIRST..v.LAST LOOP
      DBMS_OUTPUT.PUT_LINE('INSERT '|| i ||' EXECUTETD? '||SQL%BULK_ROWCOUNT(i));
    END LOOP;

    ROLLBACK;
END;
/
```

Explicit Cursors

- An explicit cursor is a session cursor that the developer constructs and manages

- The developer
  - Declares and defines an explicit cursor
  - Gives the cursor name associating a query with it (typically, the query returns multiple rows).
  - Can process the query result set in either of these ways:
    - Using **OPEN-FETCH-CLOSE**
      - Open the explicit cursor (with the OPEN statement)
      - Fetch rows from the result set (with the FETCH statement)
      - Close the explicit cursor (with the CLOSE statement)
    - Use the explicit cursor in a cursor **FOR LOOP** statement

Explicit Cursors

- The developer cannot
  - Assign a value to an explicit cursor
  - Use a cursor in an expression
  - Use a cursor as a formal subprogram parameter or a host variable
- A developer can reference an explicit cursor or cursor variable by its name unlike an implicit cursor
- An explicit cursor or cursor variable is called a named cursor

Declaring and Defining Explicit Cursors

- The developer can either
  - Declare an explicit cursor first and then define it later in the same block, subprogram, or package,
  - Declare and define it at the same time.

- An explicit cursor declaration, which only declares a cursor, has this syntax:

CURSOR cursor_name [parameter_list] RETURN return_type;

- An explicit cursor definition has this syntax:

CURSOR cursor_name [parameter_list] [RETURN return_type] IS select_statement;

Opening an Explicit Cursors

- After declaring and defining an explicit cursor, the developer can open it using the OPEN statement

- The OPEN statement does the following:

1. Allocates database resources to process the query
2. Processes the query:
   - Identifies the result set
     - If the query references variables or cursor parameters, their values affect the result set
   - If the query has a FOR UPDATE clause, locks the rows of the result set
3. Positions the cursor before the first row of the result set

Fetching Data with Explicit Cursors

- After opening an explicit cursor, the developer can fetch the rows of the query result set with the FETCH statement
- The basic syntax of a FETCH statement that returns one row is:

FETCH cursor_name INTO into_clause;

- The **into_clause** is either a list of variables or a single record variable
- The variable list or record must have a corresponding type-compatible variable or field for each column that the query returns
- The %TYPE and %ROWTYPE attributes are useful for declaring variables and records for use in FETCH statements

Fetching Data with Explicit Cursors

- The FETCH statement
  - Retrieves the current row of the result set
  - Stores the column values of that row into the variables or record
  - Advances the cursor to the next row
- The developer can
  - Use the FETCH statement inside a LOOP statement
  - Exit the LOOP when the FETCH statement runs out of rows (use %NOTFOUND)
- PL/SQL does not raise an exception when a FETCH statement returns no rows

Closing an Explicit Cursors

- The developer can close an open explicit cursor with the CLOSE statement
- The CLOSE statement allows the allocated resources for the cursor to be reused.
- After closing a cursor, t**he developer cannot**
  - Fetch records from the cursor's result set
  - Reference the cursor's attributes.
  - PL/SQL raises the predefined exception INVALID_CURSOR otherwise
- The developer can reopen a closed cursor
- The explicit cursor must be closed before trying to reopen it.
  - PL/SQL raises the predefined exception CURSOR_ALREADY_OPEN otherwise

Explicit Cursors
%ISOPEN and %ROWCOUNT Attributes

- %ISOPEN returns
  - TRUE if its explicit cursor is open
  - FALSE otherwise.

- %ISOPEN is useful for checking that an explicit cursor is not already open before you try to open it

- If a developer tries to open an explicit cursor that is already open PL/SQL raises the predefined exception CURSOR_ALREADY_OPEN

- %ROWCOUNT returns:
  - Zero after the explicit cursor is opened but before the first fetch
  - Otherwise, the number of rows fetched (an INTEGER)

Explicit Cursors
%FOUND and %NOTFOUND Attributes

- %FOUND returns:
  - NULL after the explicit cursor is opened but before the first fetch
  - TRUE if the most recent fetch from the explicit cursor returned a row
  - FALSE otherwise

- %FOUND is useful for determining whether there is a fetched row to process.

- %NOTFOUND returns:
  - NULL after the explicit cursor is opened but before the first fetch
  - FALSE if the most recent fetch from the explicit cursor returned a row
  - TRUE otherwise

- %NOTFOUND is useful for exiting a loop when FETCH fails to return a row

Explicit Cursors Example

```
set serveroutput on

DECLARE
  CURSOR c1 IS
    SELECT region_name, country_name cn, country_id
    FROM regions NATURAL JOIN countries ORDER BY 1, 2;
  CURSOR c2 RETURN locations%ROWTYPE IS
    SELECT  *
    FROM locations
    ORDER BY city;
  r1 c1%ROWTYPE;
  r2 locations%ROWTYPE;
BEGIN
  OPEN c1;
  LOOP
    FETCH c1 INTO r1;
    EXIT WHEN c1%NOTFOUND;
    OPEN c2;
    LOOP
      FETCH c2 INTO r2;
      EXIT WHEN c2%NOTFOUND;
      IF r2.country_id = r1.country_id THEN
        DBMS_OUTPUT.PUT_LINE(r1.region_name
                    || '::' || r1.cn ||'::'||r2.city);
      END IF;
    END LOOP;
    IF c2%ISOPEN THEN
      CLOSE c2;
    END IF;
  END LOOP;
  IF c1%ISOPEN THEN
    CLOSE c1;
  END IF;
END;
/
```

Variables in Explicit Cursors Queries

- An explicit cursor query can reference any variable in its scope
- When opening an explicit cursor
  - PL/SQL evaluates any variables in the query
  - Uses those values when identifying the result set
- Changing the values of the variables later does not change the result set

Explicit Cursors Example

```
 1 set serveroutput on
 2 DECLARE
 3   idr regions.region_id%TYPE := &id;
 4   CURSOR c1 IS
 5     SELECT region_name rn, country_name cn, city
 6     FROM regions NATURAL JOIN countries NATURAL JOIN LOCATIONS
 7     WHERE region_id = idr
 8     ORDER BY 1, 2;
 9   r1 c1%ROWTYPE;
10 BEGIN
11   OPEN c1;
12   LOOP
13     FETCH c1 INTO r1;
14     EXIT WHEN c1%NOTFOUND;
15     DBMS_OUTPUT.PUT_LINE(r1.rn || '::' || r1.cn ||'::'||r1.city);
16   END LOOP;
17   IF c1%ISOPEN THEN
18     CLOSE c1;
19   END IF;
20 END;
21 /
```

Explicit Cursors Parameters

- Developers can
  - Create an explicit cursor that has formal parameters
  - Pass different actual parameters to the cursor each time you open it
- A developer can use a formal cursor parameter anywhere a constant can be used in the cursor query
- A developer cannot reference formal cursor parameters outside the cursor query
- Use different names for formal and actual cursor parameters to avoid confusion
- A developer can specify a default value for formal parameters of a cursor

Explicit Cursors Parameters Example

```
set serveroutput on
DECLARE
  CURSOR c1(idr regions.region_id%TYPE DEFAULT 1, idc varchar) IS
    SELECT region_name rn, country_name cn, city
    FROM regions NATURAL JOIN countries NATURAL JOIN LOCATIONS
    WHERE region_id = idr AND country_id = idc
    ORDER BY 1, 2;
  r1 c1%ROWTYPE;
BEGIN
  OPEN c1(idc => 'UK');
  LOOP
    FETCH c1 INTO r1;
    EXIT WHEN c1%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(r1.rn || '::' || r1.cn ||'::'||r1.city);
  END LOOP;
  IF c1%ISOPEN THEN
    CLOSE c1;
  END IF;

  OPEN c1(idr=> 2, idc => 'US');
  LOOP
    FETCH c1 INTO r1;
    EXIT WHEN c1%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(r1.rn || '::' || r1.cn ||'::'||r1.city);
  END LOOP;
  IF c1%ISOPEN THEN
    CLOSE c1;
  END IF;
END;
/
```

Cursors FOR LOOP Statement

- The cursor FOR LOOP statement lets developers run a SELECT statement and then immediately loop through the rows of the result set

- This statement can use either an implicit or explicit cursor (but not a cursor variable)

- If only a SELECT statement is specified inside the FOR LOOP statement
  - This form of the cursor FOR LOOP statement uses an implicit cursor
  - Is called an implicit cursor FOR LOOP statement
  - The implicit cursor is internal to the statement
  - **The statement cannot be referenced with the name SQL**

- If an explicit cursor is specified inside the FOR LOOP statement
  - Is called an explicit cursor FOR LOOP statement

Cursors FOR LOOP Statement

- **The cursor FOR LOOP statement implicitly declares its loop index**
- The loop index is %ROWTYPE record variable of the type that the cursor returns
- This **index record is local to the loop and exists only during loop execution.**
- Statements inside the loop can reference the record and its fields
- They can reference virtual columns only by aliases

Cursors FOR LOOP Statement

- After declaring the loop index record variable, the FOR LOOP statement
  - Opens the specified cursor
  - Fetches a row from the result set and stores it in the record with each iteration of the loop
  - Closes the cursor when there are no more rows to fetch

- The cursor also closes if a statement inside the loop
  - Transfers control outside the loop (i.e., using GOTO)
  - PL/SQL raises an exception

Cursors FOR LOOP Statement Example

```
 1 set serveroutput on
 2 DECLARE
 3   CURSOR c1 IS
 4     SELECT region_name rn, country_name cn, country_id cid
 5     FROM regions NATURAL JOIN countries ORDER BY 1, 2;
 6 BEGIN
 7   FOR r1 in c1
 8   LOOP
 9     FOR r2 IN (SELECT city c FROM locations
10                WHERE country_id=r1.cid ORDER BY city)
11     LOOP
12         DBMS_OUTPUT.PUT_LINE(r1.rn || '::' || r1.cn ||'::'||r2.c);
13     END LOOP;
14   END LOOP;
15 END;
16 /
```

Cursor Variables

- A cursor variable is like an explicit cursor, except that:
  - It is **not limited to one query**
  - It can be used to open a cursor variable for a query, process the result set, and then use the cursor variable for another query.
  - **A value can be assigned to it**
  - It can be used in an **expression**
  - It can be **a subprogram parameter**
  - It can be used to **pass query result sets between subprograms**
  - It can be a **host variable**
  - It can be used to **pass query result sets between PL/SQL stored subprograms and their clients**
  - It **cannot accept parameters**
  - Whole queries can be passed to it (the queries can include variables)

Cursor Variables

- A cursor variable offers flexibility because it is a pointer
  - Its value is the address of an item, not the item itself

- Before a cursor variable can be referenced it must point to a SQL work area
  - By opening the SQL work area
  - By assigning it the value of an opened
    - PL/SQL cursor variable
    - Host cursor variable

- Cursor variables and explicit cursors are not interchangeable, one cannot be used where the other is expected
  - E.g., a cursor variable cannot be used in a cursor FOR LOOP statement.

Cursor Variable

- To create a cursor variable
  - Declare a variable of the predefined type SYS_REFCURSOR
  - Define a REF CURSOR type and then declare a variable of that type.
- Informally, a cursor variable is sometimes called a REF CURSOR
- The basic syntax of a REF CURSOR type definition is:
  TYPE type_name IS REF CURSOR [RETURN return_type]
- The REF CURSOR type and cursor variables of that type are strong if **return_type** is specified, otherwise, they are weak.
- SYS_REFCURSOR and cursor variables of that type are weak

Cursor Variable

- Only queries that return the specified type can be associated with a strong cursor variable
- Any query can be associated with a weak cursor variable
- Weak cursor variables are more error-prone than strong ones, but they are also more flexible.
- Weak REF CURSOR types are interchangeable with each other and with the predefined type SYS_REFCURSOR.
- The value of a weak cursor variable can be assigned to any other weak cursor variable
- The value of a strong cursor variable can be assigned to another strong cursor variable only if both cursor variables have the same type (not merely the same return type).

Assigning Values to a Cursor Variable

- PL/SQL cursor variable can be initialized with
  - Another PL/SQL cursor variable
  - A host cursor variable

- The syntax is:

  target_cursor_variable := source_cursor_variable;

- If source_cursor_variable is open, then after the assignment, target_cursor_variable is also open

- The two cursor variables point to the same SQL work area

- If source_cursor_variable is not open, opening target_cursor_variable after the assignment does not open source_cursor_variable

Variables in a Cursor Variable

- The query associated with a cursor variable can reference any variable in its scope
- When a cursor variable is opened with the OPEN FOR statement, PL/SQL
  - Evaluates any variables in the query
  - Uses those values when identifying the result set
- Changing the values of the variables later does not change the result set

Opening a Cursor Variable

- A cursor variable can be opened with the OPEN FOR statement

1. Associates the cursor variable with a query (typically, the query returns multiple rows)
   - The query can include placeholders for bind variables
   - The bind variable values can be specified in the USING clause of the OPEN FOR statement
2. Allocates database resources to process the query
3. Processes the query:
   - Identifies the result set
   - If the query references variables, their values affect the result set
   - If the query has a FOR UPDATE clause, locks the rows of the result set
4. Positions the cursor before the first row of the result set

Fetching Data with a Cursor Variable

- After opening a cursor variable the rows of the query result set can be fetched with the FETCH statement
- The return type of the cursor variable must be compatible with the INTO clause of the FETCH statement
- PL/SQL catches incompatibility
  - At compilation time if the cursor variable is strong
  - At run time if the cursor variable is weak
- The predefined exception ROWTYPE_MISMATCH is raised before the first fetch in case of incompatibility for a weak cursor variable

Closing a Cursor Variable

- A cursor variable needs to be closed before reopening it (that is, using it in another OPEN FOR statement)
- The query previously associated with a cursor variable is lost after it is reopened
- A cursor variable is closed with the CLOSE statement
- Using the CLOSE statement allows the resources associated with the cursor variable to be reused
- After closing a cursor variable
  - Its records cannot be fetched
  - Its result set cannot be accessed
  - Its attributes cannot be referenced
- Trying any of these raises the predefined exception INVALID_CURSOR

Cursor Variable Example

```
DECLARE
  TYPE tbl IS TABLE OF VARCHAR2(50);
  TYPE rc IS REF CURSOR RETURN departments%ROWTYPE;
  c1 rc;
  c2 SYS_REFCURSOR;
  r1 departments%ROWTYPE;
  names tbl;
BEGIN
  OPEN c1 FOR SELECT * FROM departments;
  LOOP
    FETCH c1 INTO r1;
    EXIT WHEN c1%NOTFOUND;
    OPEN c2 FOR SELECT first_name || ' ' || last_name ename
    FROM employees WHERE department_id=r1.department_id;
    FETCH c2 BULK COLLECT INTO names;
    IF names.COUNT > 0 THEN
      FOR i IN names.FIRST .. names.LAST LOOP
        DBMS_OUTPUT.PUT_LINE(r1.department_name || '::' || names(i));
      END LOOP;
    END IF;
    IF c2%ISOPEN THEN
      CLOSE c2;
    END IF;
  END LOOP;
  IF c1%ISOPEN THEN
    CLOSE c1;
  END IF;
END;
/
```

Cursor Expressions

- A CURSOR expression returns a nested cursor
- It has this syntax:

  CURSOR (subquery)

- CURSOR expression can be
  - Used in a SELECT statement that is not a subquery
  - Passed to a function that accepts a cursor variable parameter
- A cursor expression cannot be used with an implicit cursor

Cursor Expressions Example

```
DECLARE
  CURSOR c1 IS
    SELECT department_name,
      CURSOR ( SELECT e.last_name || ' ' || e.first_name name
                 FROM employees e
                 WHERE e.department_id = d.department_id
                 ORDER BY e.last_name
               ) employees
    FROM departments d
    ORDER BY department_name;
  c2 SYS_REFCURSOR;
  dname departments.department_name%TYPE;
  ename varchar2(45);
BEGIN
  OPEN c1;
  LOOP  -- Process each row of query result set
    FETCH c1 INTO dname, c2;
    EXIT WHEN c1%NOTFOUND;
    LOOP -- Process each row of subquery result set
      FETCH c2 INTO ename;
      EXIT WHEN c2%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE(dname||'::'||ename);
    END LOOP;
  END LOOP;
  CLOSE c1;
END;
/
```

FOR UPDATE

- The SELECT statement with the FOR UPDATE clause (SELECT FOR UPDATE statement) selects the rows of the result set and locks them

- SELECT FOR UPDATE
  - Locks rows for update
  - It ensures that no other user can change those values before an update

- By default, the SELECT FOR UPDATE statement waits until the requested row lock is acquired

- To change this behavior, use the NOWAIT, WAIT, or SKIP LOCKED clause of the SELECT FOR UPDATE statement.

FOR UPDATE

```
DECLARE
  CURSOR c1 IS
    SELECT employee_id, first_name, last_name, salary, department_id
    FROM employees
    WHERE employee_id NOT IN
        (SELECT DISTINCT manager_id FROM employees WHERE manager_id IS NOT NULL)
    FOR UPDATE OF salary
    ORDER BY department_id;
  asd number(10,2);
  r1 c1%ROWTYPE;
BEGIN
  OPEN c1;
  LOOP
    FETCH c1 INTO r1;
    EXIT WHEN c1%NOTFOUND;
    SELECT avg(salary) INTO asd FROM employees
    WHERE department_id = r1.department_id;
    IF r1.salary > asd THEN
      UPDATE employees SET salary = salary * 1.10
      WHERE
      CURRENT OF c1;
    END IF;
  END LOOP;
  IF c1%ISOPEN THEN
    CLOSE c1;
  END IF;
  ROLLBACK;
END;
/

select salary from employees;
```

FOR UPDATE

- When SELECT FOR UPDATE is associated with an explicit cursor, the cursor is called a FOR UPDATE cursor
- Only a FOR UPDATE cursor can appear in the CURRENT OF clause of an UPDATE or DELETE statement.
- The CURRENT OF clause
  - Is a PL/SQL extension to the WHERE clause of the SQL statements UPDATE and DELETE
  - Restricts the statement to the current row of the cursor
- When SELECT FOR UPDATE queries multiple tables, it locks only rows whose columns appear in the FOR UPDATE clause.

Bibliography

- Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 19c*, Oracle Corporation, 2022 [pdf]
- Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 21c*, Oracle Corporation, 2022 [pdf]
- Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 19c*, Oracle Corporation, 2020 [pdf]
- Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 21c*, Oracle Corporation, 2021 [pdf]

University
Politehnica
of Bucharest

PL/SQL (V)

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@upb.ro

Overview

- Subprograms

Subprograms

- A PL/SQL subprogram is a named PL/SQL block that can be invoked repeatedly
- If the subprogram has parameters, their values can differ for each invocation
- A subprogram is either a
  - **Procedure** used to perform an action
  - **Function** used to compute and return a value
- Subprograms support the development and maintenance of reliable and reusable code

Subprograms
Features

- Modularity
  - Subprograms let you break a program into manageable, well-defined modules
- Easier Application Design
  - Implementation details of the subprograms can be deferred until the main program is tested
  - The programs can be refined one step at a time
  - Easier to test modules independently
- Maintainability
  - The implementation details of a subprogram can be changed without changing its invokers
- Packageability
  - Subprograms can be grouped into packages

Subprograms Features

- Reusability
  - Any number of applications, in many different environments, can use the same package subprogram or standalone subprogram
- Better Performance
  - Each subprogram is compiled and stored in executable form, which can be invoked repeatedly
  - Because stored subprograms run in the database server, a single invocation over the network can start a large job
  - This division of work
    - Reduces network traffic
    - Improves response times
  - Stored subprograms are cached and shared among users
    - Lowers memory requirements
    - Lowers invocation overhead

Subprograms Types

- Subprogram can be
  - Nested subprograms:
    - Created inside the WITH clause
    - Created inside a PL/SQL block (which can be another subprogram)
  - Package subprograms: created inside a package
  - Standalone subprograms: are created at schema level

Nested Subprograms

- A subprogram created inside a PL/SQL block or WITH clause is a nested subprogram
- In the WITH clause
  - Declare **functions** that will be used in the SQL statement
  - Declare **procedures** that can only be used in subprograms
- Nested subprograms can be
  - Declared and defined at the same time
  - First declared and then defined in the same block
- A nested subprogram is stored in the database only if it is nested in
  - A standalone subprogram
  - Package subprogram

Nested Subprograms

```
 1 WITH
 2   FUNCTION calc_venit(eid employees.employee_id%type)
 3   RETURN FLOAT
 4   IS
 5     sal employees.salary%type;
 6     com employees.commission_pct%type;
 7   BEGIN
 8     SELECT salary, commission_pct
 9     INTO sal, com
10     FROM employees
11     WHERE employee_id = eid;
12     RETURN sal * (1 + nvl(com, 0));
13   END;
14 SELECT
15   first_name||' '||last_name nume,
16   calc_venit(employee_id) venit
17 FROM employees;
18 /
```

Nested Subprograms

```
WITH
  PROCEDURE procent_comm(eid employees.employee_id%type, procent NUMBER)
  IS
    name varchar2(50);
  BEGIN
    SELECT first_name||' '||last_name INTO name
    FROM employees WHERE employee_id=eid;
    DBMS_OUTPUT.put_line(name || ' procent=' || procent);
  END;

  FUNCTION calc_venit(eid employees.employee_id%type)
  RETURN FLOAT
  IS
    sal employees.salary%type;
    com employees.commission_pct%type;
  BEGIN
    SELECT salary, commission_pct INTO sal, com
    FROM employees WHERE employee_id = eid;
    procent_comm(eid, 1 + nvl(com, 0));
    RETURN sal * (1 + nvl(com, 0))
  END;
SELECT
  first_name||' '||last_name nume,
  calc_venit(employee_id) venit
FROM employees;
/
```

Package Subprograms

- A subprogram created inside a package is a **package subprogram**

- Developers can
  - Declare it in the package specification
  - Define it in the package body

- It is stored in the database until the package is dropped

Standalone Subprograms

- **A subprogram created at schema level is a standalone subprogram**
- A developer can create it with the statements:

CREATE [OR REPLACE] FUNCTION function_name
CREATE [OR REPLACE] PROCEDURE procedure_name

- It is stored in the database until it is dropped with the statements:

DROP FUNCTION function_name;
DROP PROCEDURE procedure_name;

Stored Subprograms

- A stored subprogram is either
  - A package subprogram
  - A standalone subprogram
- A stored subprogram is affected by the AUTHID and ACCESSIBLE BY clauses
- These clauses can appear in the CREATE FUNCTION, CREATE PROCEDURE, and CREATE PACKAGE statements
- The AUTHID clause affects the name resolution and privilege checking of SQL statements that the subprogram issues at run time
- The ACCESSIBLE BY clause specifies a white list of PL/SQL units that can access the subprogram

Subprogram Invocations

- A subprogram invocation has this form:

subprogram_name [([parameter [, parameter]...])]

- If the subprogram has no parameters, or specifies a default value for every parameter
  - Omit the parameter list
  - Specify an empty parameter list

- A procedure invocation is a PL/SQL statement:

procedure_name(val_parameter_1, val_parameter_2, ...);

- A function invocation is an expression:

value := function_name(val_parameter_1, val_parameter_2, ...);

IF function_name(val_parameter_1, val_parameter_2, ...) THEN ...

Subprogram Properties

- Each subprogram property (e.g., AUTHID, ACCESSIBLE BY, etc.) can appear only once in the subprogram declaration
- The properties can appear in any order
- Properties appear before the IS or AS keyword in the subprogram heading
- The properties cannot appear in nested subprograms
- Only the ACCESSIBLE BY property can appear in package subprograms

Subprogram Parts

- A subprogram begins with a subprogram heading, which specifies
  - The subprogram name
  - The subprogram parameter list (optional)

- Like an anonymous block, a subprogram has these parts:
  - Declarative part (optional)
    - This part declares and defines local types, cursors, constants, variables, exceptions, and nested subprograms
    - These items cease to exist when the subprogram completes execution
    - This part can also specify pragmas
    - The declarative part of a subprogram does not begin with the keyword DECLARE, as the declarative part of an anonymous block does
  - Executable part (required)
    - This part contains one or more statements that assign values, control execution, and manipulate data
  - Exception-handling part (optional)
    - This part contains code that handles runtime errors

Additional Subprogram Parts For Functions

- A function has the same structure as a procedure, except that:

1. A function heading must include a RETURN clause
   - The RETURN clause specifies the data type of the value that the function returns
   - A procedure heading cannot have a RETURN clause

2. In the executable part of a function, every execution path must lead to a RETURN statement
   - The PL/SQL compiler issues a compile-time warning otherwise
   - The RETURN statement is optional and not recommended in a procedure

Additional Subprogram Parts For Functions

- A function declaration can include these options:

Option	Description
DETERMINISTIC option	Helps the optimizer avoid redundant function invocations.
PARALLEL_ENABLE option	Enables the function for parallel execution, making it safe for use in slave sessions of parallel DML evaluations.
PIPELINED option	Makes a table function pipelined, for use as a row source.
RESULT_CACHE option	Stores function results in the PL/SQL function result cache.

Additional Subprogram Parts For Functions

```
 1 CREATE OR REPLACE
 2 FUNCTION f_pow(x IN NUMBER, y IN NUMBER)
 3     RETURN NUMBER
 4     DETERMINISTIC
 5     PARALLEL_ENABLE
 6     RESULT_CACHE
 7 IS
 8 BEGIN
 9   RETURN x**y;
10 END;
11 /
12
13 SELECT f_pow(2, 10) from dual;
14
15 DROP FUNCTION f_pow;
```

Additional Subprogram Parts For Functions

```
 1 CREATE TYPE info_row AS OBJECT (
 2   id   NUMBER,
 3   info VARCHAR2(50)
 4 );
 5 /
 6
 7 CREATE TYPE info_tab IS TABLE OF info_row;
 8 /
 9
10 CREATE OR REPLACE FUNCTION get_info(n IN NUMBER)
11   RETURN info_tab PIPELINED
12 AS
13 BEGIN
14   FOR i IN 1 .. n LOOP
15     PIPE ROW(info_row(i, 'Description for ' || i));
16   END LOOP;
17 END;
18 /
19
20 SELECT * FROM  TABLE(get_info(10));
21
22 DROP FUNCTION get_info;
23 DROP TYPE info_tab;
24 DROP TYPE info_row;
```

Subprogram Parts
Return

- RETURN Statement
  - The RETURN statement immediately ends the execution of the subprogram or anonymous block that contains it
  - A subprogram or anonymous block can contain multiple RETURN statements

- RETURN Statement in Function
  - Every execution path must lead to a RETURN statement and every RETURN statement must specify an expression
  - The RETURN statement assigns the value of the expression to the function identifier and returns control to the invoker, where execution resumes immediately after the invocation
  - In a pipelined table function, a RETURN statement need not specify an expression

Subprogram Parts
Return

```
 1 DECLARE
 2   x INTEGER;
 3
 4   FUNCTION f (n INTEGER) RETURN INTEGER
 5   IS
 6   BEGIN
 7     RETURN (n*n);
 8   END;
 9
10 BEGIN
11   DBMS_OUTPUT.PUT_LINE ('f returns ' || f(2) || '. Execution returns here (1).' );
12   x := f(3);
13   DBMS_OUTPUT.PUT_LINE('Execution returns here (2).');
14   x := &x;
15   IF f(x) < 10 THEN
16     DBMS_OUTPUT.PUT_LINE('Execution returns here (3).');
17   ELSE
18     DBMS_OUTPUT.PUT_LINE('Execution returns here (4).');
19   END IF;
20 END;
21 /
```

Subprogram Parts
Return

- RETURN Statement in Procedure
  - The RETURN statement returns control to the invoker, where execution resumes immediately after the invocation
  - The RETURN statement cannot specify an expression

- RETURN Statement in Anonymous Block
  - The RETURN statement exits its own block and all enclosing blocks
  - The RETURN statement cannot specify an expression

Subprogram Parts
Return

```
 1 DECLARE
 2   PROCEDURE p IS
 3   BEGIN
 4     DBMS_OUTPUT.PUT_LINE('Inside p');
 5     RETURN;
 6     DBMS_OUTPUT.PUT_LINE('Unreachable statement.');
 7   END;
 8 BEGIN
 9   p;
10   DBMS_OUTPUT.PUT_LINE('Control returns here.');
11 END;
12 /
```

Subprogram Parts
Return

```
 1 BEGIN
 2   BEGIN
 3     DBMS_OUTPUT.PUT_LINE('Inside inner block.');
 4     RETURN;
 5     DBMS_OUTPUT.PUT_LINE('Unreachable statement.');
 6   END;
 7   DBMS_OUTPUT.PUT_LINE('Inside outer block.
 8                         Unreachable statement.');
 9 END;
10 /
```

Forward Declarations

- If nested subprograms in the same PL/SQL block invoke each other, then one requires a forward declaration, because a subprogram must be declared before it can be invoked.
- A forward declaration declares a nested subprogram but does not define it
- A developer must define it later in the same block
- The forward declaration and the definition must have the same subprogram heading

Forward Declarations

```
 1 DECLARE
 2   -- Declare proc1 (forward declaration)
 3   PROCEDURE proc1;
 4
 5   -- Declarea and define proc2
 6   PROCEDURE proc2
 7   IS
 8   BEGIN
 9     DBMS_OUTPUT.PUT_LINE('Call proc1');
10     proc1;
11   END;
12
13   -- Define proc1
14   PROCEDURE proc1
15   IS
16   BEGIN
17     DBMS_OUTPUT.PUT_LINE('Inside proc1');
18   END;
19 BEGIN
20   proc2;
21 END;
22 /
```

Subprogram Parameters

- If a subprogram has parameters, their values can differ for each invocation
- Formal parameters must be declared in the subprogram heading to have subprogram with parameters
- In each formal parameter declaration specify
  - Mandatory: the name and data type of the parameter
  - Optional: the mode and default value
- Reference the formal parameters by their names in the execution part of the subprogram

Subprogram Parameters

- When invoking the subprogram, specify the actual parameters whose values are to be assigned to the formal parameters
- **Corresponding actual and formal parameters** must have compatible **data types**
- To avoid confusion, use different names for formal and actual parameters
- Actual parameters (including default values of formal parameters) can be evaluated in any order
- LOB parameters cannot be used in a server-to-server remote procedure call (RPC)

Subprogram Parameters

```
 1 DECLARE
 2   emp_num NUMBER(6) := 120;
 3   bonus   NUMBER(6) := 100;
 4   merit   NUMBER(4) := 50;
 5
 6   PROCEDURE raise_salary (
 7     emp_id NUMBER,  -- formal parameter
 8     amount NUMBER   -- formal parameter
 9   ) IS
10   BEGIN
11     UPDATE employees
12     SET salary = salary + amount  -- reference to formal parameter
13     WHERE employee_id = emp_id;   -- reference to formal parameter
14   END raise_salary;
15
16 BEGIN
17   raise_salary(emp_num, bonus);           -- actual parameters
18   raise_salary(emp_num, merit + bonus);   -- actual parameters
19 END;
20 /
```

Subprogram Parameters

- If the data type of a formal parameter is a constrained subtype, then:
  - If the subtype has the NOT NULL constraint, then the actual parameter inherits it
  - If the subtype has the base type VARCHAR2, then the actual parameter does not inherit the size of the subtype
  - If the subtype has a numeric base type, then the actual parameter inherits the range of the subtype, but not the precision or scale

- In a function
  - The clause RETURN datatype declares a hidden formal parameter
  - The statement RETURN value specifies the corresponding actual parameter
  - The same rules apply for subtype data types in RETURN

Subprogram Parameters

```
 1 DECLARE
 2   -- Declare subtype
 3   SUBTYPE license IS VARCHAR2(7) NOT NULL;
 4   n license := 'DLLLDDD';
 5
 6   PROCEDURE p (x License) IS
 7   BEGIN
 8     DBMS_OUTPUT.PUT_LINE(x);
 9   END;
10
11 BEGIN
12   p('1ABC123456789');   -- Succeeds; size is not inherited
13   p(NULL);              -- Raises error; NOT NULL is inherited
14 END;
15 /
```

Subprogram Parameters

```
 1 DECLARE
 2   FUNCTION test (p INTEGER) RETURN INTEGER IS
 3   BEGIN
 4     DBMS_OUTPUT.PUT_LINE('p = ' || p);
 5     RETURN p;
 6   END test;
 7
 8 BEGIN
 9   -- actual parameter and return value inherit
10   -- only range from subtype
11   DBMS_OUTPUT.PUT_LINE('test(p) = ' || test(0.66));
12 END;
13 /
```

Subprogram Parameters

```
 1 DECLARE
 2   FUNCTION test (p NUMBER) RETURN NUMBER IS
 3     -- Implicitly converts p to INTEGER
 4     q INTEGER := p;
 5   BEGIN
 6     -- Display q, not p
 7     DBMS_OUTPUT.PUT_LINE('p = ' || q);
 8     -- Return q, not p
 9     RETURN q;
10   END test;
11 BEGIN
12   DBMS_OUTPUT.PUT_LINE('test(p) = ' || test(0.66));
13 END;
14 /
```

Subprogram Parameters Passing Method

- The PL/SQL compiler has two ways of passing an actual parameter to a subprogram:
  - By reference
    - The compiler passes the subprogram a pointer to the actual parameter
    - The actual and formal parameters refer to the same memory location
  - By value
    - The compiler assigns the value of the actual parameter to the corresponding formal parameter
    - The actual and formal parameters refer to different memory locations
- The method by which the compiler passes a specific actual parameter depends on its mode

Subprogram Parameters Passing Method

- If necessary, the compiler implicitly converts the data type of the actual parameter to the data type of the formal parameter

- Avoid implicit data conversion, in either of these ways:
  - Declare the variables that you intend to use as actual parameters with the same data types as their corresponding formal parameters
  - Explicitly convert actual parameters to the data types of their corresponding formal parameters, using the SQL conversion functions described in Oracle Database SQL Language Reference

Subprogram Parameters Passing Method

```
 1 CREATE OR REPLACE PROCEDURE p (n NUMBER)
 2 AUTHID DEFINER
 3 IS
 4 BEGIN
 5   DBMS_OUTPUT.PUT_LINE(n);
 6 END;
 7 /
 8
 9 DECLARE
10   x NUMBER       :=  1;
11   y VARCHAR2(1) := '1';
12 BEGIN
13   -- No conversion needed
14   p(x);
15   -- y implicitly converted from VARCHAR2 to NUMBER
16   p(y);
17   -- y explicitly converted from VARCHAR2 to NUMBER
18   p(TO_NUMBER(y));
19 END;
20 /
```

Subprogram Parameters Mode

- The **mode** of a formal parameter determines its behavior

Parameter Mode	Is Default?	Role
IN	Default mode	Passes a value to the subprogram.
OUT	Must be specified.	Returns a value to the invoker.
IN OUT	Must be specified.	Passes an initial value to the subprogram and returns an updated value to the invoker.

Subprogram Parameters Mode Characteristics

Mode	Formal Parameter	Actual Parameter	Passed by Reference?
IN	**Formal parameter acts like a constant**: When the subprogram begins, its value is that of either its actual parameter or default value, and the subprogram cannot change this value.	Actual parameter can be a constant, initialized variable, literal, or expression.	Actual parameter is passed by reference.
OUT	**Formal parameter is initialized to the default value of its type.** The default value of the type is NULL except for a record type with a non-NULL default value. When the subprogram begins, the formal parameter has its initial value regardless of the value of its actual parameter. Oracle recommends that the subprogram assign a value to the formal parameter.	If the default value of the formal parameter type is NULL, then the actual parameter must be a variable whose data type is not defined as NOT NULL.	By default, actual parameter is passed by value. If you specify NOCOPY, it might be passed by reference.
IN OUT	**Formal parameter acts like an initialized variable**: When the subprogram begins, its value is that of its actual parameter. Oracle recommends that the subprogram update its value.	Actual parameter must be a variable (typically, it is a string buffer or numeric accumulator).	By default, actual parameter is passed by value (in both directions). If you specify NOCOPY, it might be passed by reference.

Subprogram Parameters Mode

- Do not use OUT and IN OUT for function parameters
  - Ideally, a function takes zero or more parameters and returns a single value
  - A function with IN OUT parameters returns multiple values and has side effects

- Regardless of how an OUT or IN OUT parameter is passed:
  - If the subprogram exits successfully, then the value of the actual parameter is the final value assigned to the formal parameter
  - The formal parameter is assigned at least one value (the initial value)
  - If the subprogram ends with an exception, then the value of the actual parameter is undefined
  - Formal OUT and IN OUT parameters can be returned in any order

Subprogram Parameters Mode

- When an OUT or IN OUT parameter is passed by reference:
  - The actual and formal parameters refer to the same memory location
  - If the subprogram changes the value of the formal parameter, the change shows immediately in the actual parameter

Subprogram Parameters Mode

```
 1 CREATE OR REPLACE PROCEDURE p (
 2   a         PLS_INTEGER,  -- IN by default
 3   b      IN PLS_INTEGER,
 4   c     OUT PLS_INTEGER,
 5   d IN OUT PLS_INTEGER
 6 )
 7 IS
 8 BEGIN
 9   DBMS_OUTPUT.PUT_LINE('Inside procedure p:');
10   -- Print values of parameters:
11   DBMS_OUTPUT.PUT_LINE('IN a = '     || NVL(TO_CHAR(a), 'NULL'));
12   DBMS_OUTPUT.PUT_LINE('IN b = '     || NVL(TO_CHAR(b), 'NULL'));
13   DBMS_OUTPUT.PUT_LINE('OUT c = '    || NVL(TO_CHAR(c), 'NULL'));
14   DBMS_OUTPUT.PUT_LINE('IN OUT d = ' || TO_CHAR(d));
15   -- Can reference IN parameters a and b,
16   -- but cannot assign values to them.
17   c := a + 10;  -- Assign value to OUT parameter
18   d := b * 10;  -- Assign value to IN OUT parameter
19 END;
20 /
```

Subprogram Parameters Mode

```
DECLARE
  aa  CONSTANT PLS_INTEGER := 1;
  bb  PLS_INTEGER := 2;
  cc  PLS_INTEGER := 3;
  dd  PLS_INTEGER := 4;
  ee  PLS_INTEGER;
  ff  PLS_INTEGER := 5;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Before invoking procedure p:');
  DBMS_OUTPUT.PUT_LINE('aa = ' || NVL(TO_CHAR(aa), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('bb = ' || NVL(TO_CHAR(bb), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('cc = ' || NVL(TO_CHAR(cc), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('dd = ' || TO_CHAR(dd));

  p (aa, -- constant
     bb, -- initialized variable
     cc, -- initialized variable
     dd  -- initialized variable
  );

  DBMS_OUTPUT.PUT_LINE('After invoking procedure p:');
  DBMS_OUTPUT.PUT_LINE('aa = ' || NVL(TO_CHAR(aa), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('bb = ' || NVL(TO_CHAR(bb), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('cc = ' || NVL(TO_CHAR(cc), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('dd = ' || TO_CHAR(dd));

  DBMS_OUTPUT.PUT_LINE('Before invoking procedure p:');
  DBMS_OUTPUT.PUT_LINE('ee = ' || NVL(TO_CHAR(ee), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('ff = ' || TO_CHAR(ff));

  p (1,         -- literal
     (bb+3)*4, -- expression
     ee,        -- uninitialized variable
     ff         -- initialized variable
   );

  DBMS_OUTPUT.PUT_LINE('After invoking procedure p:');
  DBMS_OUTPUT.PUT_LINE('ee = ' || NVL(TO_CHAR(ee), 'NULL'));
  DBMS_OUTPUT.PUT_LINE('ff = ' || TO_CHAR(ff));
END;
/
```

Subprogram Parameters Aliasing

- Aliasing is having two different names for the same memory location
- If a stored item is visible by more than one path and if the item is changed by one path, then the change propagates to all paths
- Subprogram parameter aliasing always occurs when
  - The compiler passes an actual parameter by reference
  - A subprogram has cursor variable parameters
- When the compiler passes an actual parameter by reference:
  - The actual and formal parameters refer to the same memory location
  - The change shows immediately in the actual parameter if the subprogram changes the value of the formal parameter

Subprogram Parameters Aliasing

- The compiler always passes IN parameters by reference:
  - The resulting aliasing cannot cause problems
  - Subprograms cannot assign values to IN parameters

- If you specify NOCOPY for an OUT or IN OUT parameter
  - The compiler might pass an OUT or IN OUT parameter by reference
  - NOCOPY is only a hint: each time the subprogram is invoked, the compiler decides, silently, whether to obey or ignore NOCOPY.
  - Aliasing can occur for one invocation but not another, making subprogram results indeterminate

Subprogram Parameters Aliasing

- If the actual parameter is a global variable, then an assignment to the formal parameter might show in the global parameter
- If the same variable is the actual parameter for two formal parameters, then an assignment to either formal parameter might show immediately in both formal parameters
- If the actual parameter is a package variable, then an assignment to either the formal parameter or the package variable might show immediately in both the formal parameter and the package variable
- If the subprogram is exited with an unhandled exception, then an assignment to the formal parameter might show in the actual parameter

Subprogram Parameters Aliasing

```
 1 DECLARE
 2   n NUMBER := 10;
 3
 4   PROCEDURE p (
 5     n1 IN NUMBER,
 6     n2 IN OUT NUMBER,
 7     n3 IN OUT NOCOPY NUMBER
 8   ) IS
 9   BEGIN
10     -- actual parameter is 20 only after procedure succeeds
11     n2 := 20;
12     -- actual parameter value is still 10
13     DBMS_OUTPUT.put_line(n1);
14     -- might change actual parameter immediately
15     n3 := 30;
16     -- actual parameter value is either
17     -- 10 (ignore NOCOPY)
18     -- 30 (obey NOCOPY)
19     DBMS_OUTPUT.put_line(n1);
20   END;
21
22 BEGIN
23   p(n, n, n);
24   DBMS_OUTPUT.put_line(n);
25 END;
26 /
```

Subprogram Parameters
Aliasing with Cursor Variable Parameters

- Cursor variable parameters are pointers
- If a subprogram assigns one cursor variable parameter to another, they refer to the same memory location
- This aliasing can have unintended results

Subprogram Parameters Aliasing with Cursor Variable Parameters

```
DECLARE
  c1 SYS_REFCURSOR;
  c2 SYS_REFCURSOR;

  PROCEDURE get_emp_data (
    emp_cv1 IN OUT SYS_REFCURSOR,
    emp_cv2 IN OUT SYS_REFCURSOR
  )
  IS
    emp_rec employees%ROWTYPE;
  BEGIN
    OPEN emp_cv1 FOR SELECT * FROM employees;
    -- now both variables refer to same location
    emp_cv2 := emp_cv1;
    -- fetches first row of employees
    FETCH emp_cv1 INTO emp_rec;
    -- fetches second row of employees
    FETCH emp_cv1 INTO emp_rec;
    -- fetches third row of employees
    FETCH emp_cv2 INTO emp_rec;
    -- closes both variables
    CLOSE emp_cv1;
    -- causes error when get_emp_data is invoked
    -- FETCH emp_cv2 INTO emp_rec;
  END;
BEGIN
  get_emp_data(c1, c2);
END;
/
```

Subprogram Parameters Default Values

- A formal IN parameter can have a default
- A formal parameter with a default value is called an optional parameter (its corresponding actual parameter is optional in a subprogram invocation)
- If the actual parameter is omitted, then the invocation assigns the default value to the formal parameter
- A formal parameter with no default value is called a required parameter (its corresponding actual parameter is required in a subprogram invocation)
- Omitting an actual parameter does not make the value of the corresponding formal parameter NULL
- To make the value of a formal parameter NULL, specify NULL as either the default value or the actual parameter

Subprogram Parameters Positional notation

- When invoking a subprogram, a user can specify the actual parameters using either

- Positional notation
  - Specify the actual parameters in the same order as the formal parameters are declared
  - You can omit trailing optional parameters
  - Specifying actual parameters in the wrong order can cause problems that are hard to detect, especially if the actual parameters are literals.
  - Subprogram invocations must change if the formal parameter list changes, unless the list only acquires new trailing optional parameters
  - Reduced code clarity and maintainability
  - Not recommended if the subprogram has a large number of parameters

Subprogram Parameters
Named notation

- Specify the actual parameters in any order, using this syntax:
  formal => actual

- Where
  - **formal** is the name of the formal parameter
  - **actual** is the actual parameter

- You can omit any optional parameters

- There is no wrong order for specifying actual parameters

- Subprogram invocations must change only if the formal parameter list acquires new required parameters

- Recommended when you invoke a subprogram defined or maintained by someone else

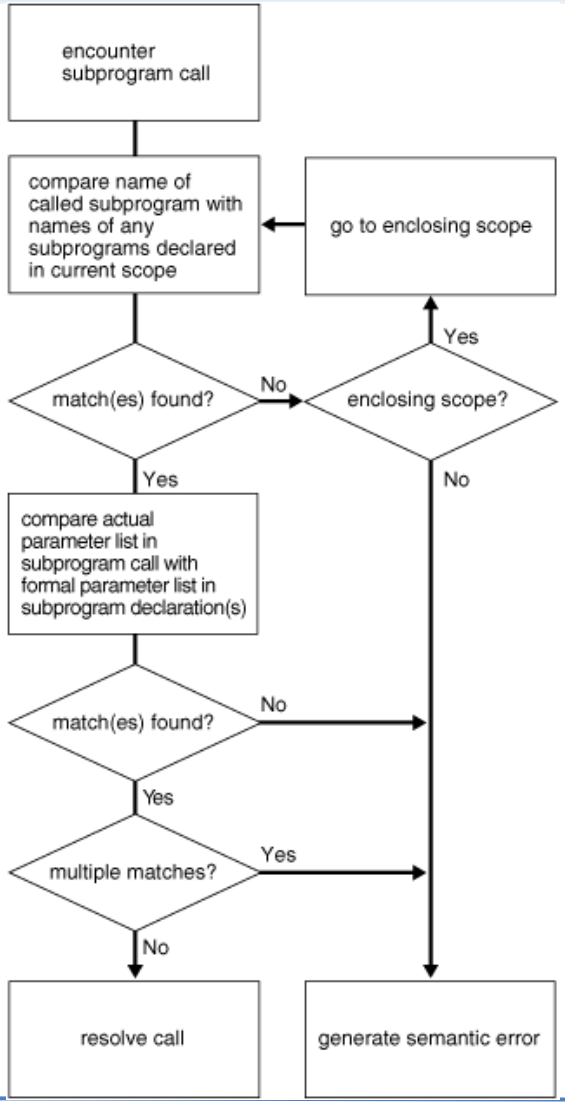
Subprogram Parameters
Mixed notation

- Start with positional notation, then use named notation for the remaining parameters
- In the positional notation: you can omit trailing optional parameters
- In the named notation: you can omit any optional parameters
- Convenient when you invoke a subprogram that has required parameters followed by optional parameters, and you must specify only a few of the optional parameters.
- In the positional notation, the wrong order can cause problems that are hard to detect, especially if the actual parameters are literals
- Changes to the formal parameter list might require changes in the positional notation

Subprogram Parameters
Default Values and Notations

```
 1 DECLARE
 2   FUNCTION f_pow (
 3     x IN number,
 4     y IN number := 2,
 5     z IN number default 1
 6   ) RETURN NUMBER IS
 7   BEGIN
 8     RETURN (x ** y) ** z;
 9   END f_pow;
10 BEGIN
11   DBMS_OUTPUT.PUT_LINE(f_pow(2));
12   DBMS_OUTPUT.PUT_LINE(f_pow(2, 4));
13   DBMS_OUTPUT.PUT_LINE(f_pow(2, 4, 4));
14   DBMS_OUTPUT.PUT_LINE(f_pow(z => 2, x => 4));
15   DBMS_OUTPUT.PUT_LINE(f_pow(3, 3, z => 4));
16   DBMS_OUTPUT.PUT_LINE(f_pow(5, z => 4));
17   DBMS_OUTPUT.PUT_LINE(f_pow(z => 3, x => 3, y => 4));
18 END;
19 /
```

Subprogram Invocation Resolution



Subprogram Invocation Resolution

- When the PL/SQL compiler encounters a subprogram invocation, it searches for a matching subprogram declaration
  - First in the current scope
  - If necessary, in successive enclosing scopes
- A declaration and invocation match if their subprogram names and parameter lists match
- The parameter lists match if each required formal parameter in the declaration has a corresponding actual parameter in the invocation
- If the compiler finds no matching declaration for an invocation, then it generates a semantic error

Overloaded Subprograms

- PL/SQL permits to overload nested subprograms, package subprograms, and type methods
- The same name can be used for several different subprograms if their formal parameters differ in name, number, order, or data type family
- A data type family is a data type and its subtypes
- If formal parameters differ only in name, then named notation must be used to specify the corresponding actual parameters

Overloaded Subprograms

- Subprograms can be overloaded if their formal parameters differ only in numeric data type

- This technique is useful in writing mathematical application programming interfaces (APIs) (i.e., several versions of a function can use the same name, and each can accept a different numeric type)

- To avoid problems or unexpected results when passing parameters to such overloaded subprograms:
  - Ensure that the expected version of a subprogram is invoked for each set of expected parameters
  - Qualify numeric literals and use conversion functions to make clear what the intended parameter types are

Overloaded Subprograms

- PL/SQL looks for matching numeric parameters in this order:
  - PLS_INTEGER (or BINARY_INTEGER, an identical data type)
  - NUMBER
  - BINARY_FLOAT
  - BINARY_DOUBLE
  - A VARCHAR2 value can match a NUMBER, BINARY_FLOAT, or BINARY_DOUBLE parameter.

- PL/SQL uses the first overloaded subprogram that matches the supplied parameters

- If PL/SQL must convert a parameter to another data type, it first tries to convert it to a higher data type

Overloaded Subprograms DO NOT DO

- Cannot overload these subprograms:
  - Standalone subprograms
  - Subprograms whose formal parameters differ only in mode; e.g.:

    PROCEDURE s (p IN VARCHAR2) IS …
    PROCEDURE s (p OUT VARCHAR2) IS …

  - Subprograms whose formal parameters differ only in subtype; e.g.:

    PROCEDURE s (p INTEGER) IS ...
    PROCEDURE s (p REAL) IS …

    - INTEGER and REAL are subtypes of NUMBER, so they belong to the same data type family.

  - Functions that differ only in return value data type, even if the data types are in different families; e.g.:

    FUNCTION f (p INTEGER) RETURN BOOLEAN IS ...
    FUNCTION f (p INTEGER) RETURN INTEGER IS …

Overloaded Subprograms
Overload Errors

- The PL/SQL compiler catches overload errors as soon as it determines that it cannot tell which subprogram was invoked

- When subprograms have identical headings, the compiler catches the overload error at the compilation time of
  - The subprograms themselves (if they are nested)
  - The package specification that declares them
  - An ambiguous invocation of a subprogram

Overloaded Subprograms
Overload

```
 1 DECLARE
 2   TYPE date_tab_typ IS TABLE OF DATE   INDEX BY PLS_INTEGER;
 3   TYPE num_tab_typ  IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
 4
 5   hiredate_tab  date_tab_typ;
 6   sal_tab       num_tab_typ;
 7
 8   PROCEDURE initialize (tab OUT date_tab_typ, n INTEGER) IS
 9   BEGIN
10     DBMS_OUTPUT.PUT_LINE('Invoked first version');
11     FOR i IN 1..n LOOP
12       tab(i) := SYSDATE;
13     END LOOP;
14   END initialize;
15
16   PROCEDURE initialize (tab OUT num_tab_typ, n INTEGER) IS
17   BEGIN
18     DBMS_OUTPUT.PUT_LINE('Invoked second version');
19     FOR i IN 1..n LOOP
20       tab(i) := 0.0;
21     END LOOP;
22   END initialize;
23
24 BEGIN
25   initialize(hiredate_tab, 50);
26   initialize(sal_tab, 100);
27 END;
28 /
```

Recursive Subprograms

- A recursive subprogram invokes itself
- Recursion is a powerful technique for simplifying an algorithm
- A recursive subprogram must have at least two execution paths
  1. One leading to the recursive invocation
  2. One leading to a terminating condition
  - Without the latter, recursion continues until PL/SQL runs out of memory and raises the predefined exception STORAGE_ERROR
- Each recursive invocation of a subprogram creates an instance of
  - Each item that the subprogram declares
  - Each SQL statement that the subprogram executes
- A recursive invocation inside a cursor FOR LOOP statement, or between an OPEN or OPEN FOR statement and a CLOSE statement
  - Opens another cursor at each invocation
  - Might cause the number of open cursors to exceed the limit set by the database initialization parameter OPEN_CURSORS

Recursive Subprograms

```
 1 CREATE OR REPLACE FUNCTION factorial(n POSITIVE)
 2 RETURN POSITIVE AUTHID DEFINER
 3 IS
 4 BEGIN
 5   IF n = 1 THEN
 6     -- terminating condition
 7     RETURN n;
 8   ELSE
 9     -- recursive invocation
10     RETURN n * factorial(n-1);
11   END IF;
12 END;
13 /
14
15 BEGIN
16   FOR i IN 1..5 LOOP
17     DBMS_OUTPUT.PUT_LINE(i || '! = ' || factorial(i));
18   END LOOP;
19 END;
20 /
```

Subprograms Side Effects

- A subprogram has side effects if it changes anything except the values of its own local variables.

- A subprogram that changes any of the following has side effects:
  - Its own OUT or IN OUT parameter
  - A global variable
  - A public variable in a package
  - A database table
  - The database
  - The external state

- Side effects can
  - Prevent the parallelization of a query
  - Yield order-dependent (and therefore, indeterminate) results
  - Require that package state be maintained across user sessions

- Minimizing side effects is especially important when defining a result-cached function or a stored function for SQL statements to invoke.

PL/SQL Function Result Cache

- When a PL/SQL function has the RESULT_CACHE option, its results are cached in the shared global area (SGA) so sessions connected to the same instance can reuse these results when available

- Oracle Database
  - Automatically detects all data sources (tables and views) that are queried while a result-cached function is running
  - Changes to any of these data sources are committed, the cached result becomes invalid across all instances

- The best candidates for result-caching are functions that are invoked frequently but depend on information that changes infrequently or never

- To make a function result-cached, include the RESULT_CACHE clause in the function declaration and definition

PL/SQL Function Result Cache

```
 1 CREATE OR REPLACE FUNCTION f
 2 RETURN VARCHAR2
 3 RESULT_CACHE
 4 IS
 5   name VARCHAR2(50);
 6 BEGIN
 7   SELECT first_name || ' ' || last_name
 8   INTO name
 9   FROM employees
10   WHERE manager_id IS NULL;
11   RETURN name;
12 END;
13 /
```

PL/SQL Functions Invoked by SQL

- To be invokable from SQL statements, a stored function (and any subprograms that it invokes) must obey the following purity rules, which are meant to control side effects:
  - The subprogram **cannot** modify any database tables when invoked from a SELECT statement or a parallelized INSERT, UPDATE, DELETE, or MERGE statement
  - The subprogram **cannot** query or modify any database tables modified by that statement when invoked from an INSERT, UPDATE, DELETE, or MERGE statement
  - The **subprogram cannot execute** any of the following SQL statements (unless PRAGMA AUTONOMOUS_TRANSACTION was specified) when invoked from a SELECT, INSERT, UPDATE, DELETE, or MERGE statement:
    - Transaction control statements (such as COMMIT)
    - Session control statements (such as SET ROLE)
    - System control statements (such as ALTER SYSTEM)
    - Database definition language (DDL) statements (such as CREATE), which are committed automatically

PL/SQL Functions Invoked by SQL

- If any SQL statement in the execution part of the function violates a rule, then a runtime error occurs when that statement is parsed
- The fewer side effects a function has, the better it can be optimized in a SELECT statement, especially if the function is declared with the option DETERMINISTIC or PARALLEL_ENABLE

```
1 SELECT f CEO_NAME FROM dual;
```

External Subprograms

- If a **C procedure** or **Java method** is **stored** in the database, you can **publish** it **as an external** subprogram
- To **publish** an external subprogram, **define** a stored PL/SQL **subprogram** with a **call specification**
- **Invoke** the published external subprogram by its PL/SQL name

External Subprograms

```
CREATE OR REPLACE JAVA SOURCE NAMED "Hello" AS
  /* Pure Java code */
  public class Hello {
    static public String Msg(String tail) {
      return "Hello " + tail;
    }
  }

CREATE OR REPLACE FUNCTION hello (str varchar2)
RETURN varchar
AS
  LANGUAGE JAVA NAME 'Hello.Msg(java.lang.String) return java.lang.String';
/

SELECT hello(first_name|| ' '||last_name) FROM employees;
```

External Subprograms

```
CREATE OR REPLACE JAVA SOURCE NAMED "Adjuster" AS
  import java.sql.*;
  import oracle.jdbc.driver.*;
  public class Adjuster {
    public static void raiseSalary (int empNo, float percent)
    throws SQLException {
      Connection conn = new OracleDriver().defaultConnection();
      String sql = "UPDATE employees SET salary = salary * ? WHERE employee_id = ?";
      try {
        PreparedStatement pstmt = conn.prepareStatement(sql);
        pstmt.setFloat(1, (1 + percent / 100));
        pstmt.setInt(2, empNo);
        pstmt.executeUpdate();
        pstmt.close();
      } catch (SQLException e)
           {System.err.println(e.getMessage());}
      }
  }
/
```

External Subprograms

```
 1 CREATE OR REPLACE PROCEDURE raise_salary (
 2   empid NUMBER,
 3   pct   NUMBER
 4 )
 5 AS -- call specification
 6   LANGUAGE JAVA NAME 'Adjuster.raiseSalary (int, float)';
 7 /
 8
 9 SELECT salary FROM employees WHERE employee_id=120;
10
11 BEGIN
12   -- invoke Adjuster.raiseSalary by PL/SQL name
13   raise_salary(120, 10);
14 END;
15 /
16
17 SELECT salary FROM employees WHERE employee_id=120;
```

Bibliography

- Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 19c*, Oracle Corporation, 2022 [pdf]
- Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 21c*, Oracle Corporation, 2022 [pdf]
- Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 19c*, Oracle Corporation, 2020 [pdf]
- Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 21c*, Oracle Corporation, 2021 [pdf]

University
Politehnica
of Bucharest

PL/SQL (VI)

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@upb.ro

Overview

- Packages
- Triggers

Overview

- Packages
- Triggers

Packages

- A package is a schema object that groups logically related PL/SQL:
  - Types
  - Variables and constants
  - Subprograms
  - Cursors
  - Exceptions
- A package is compiled and stored in the database
- Applications can share the contents of a package
- A package contains two parts
  - A package specification (mandatory)
  - A package body (not mandatory)

Packages

- A package always has a specification where public items are declared
- Public items can be referenced from outside the package
- The package must also have a body if the public items include
  - Cursors (the cursors' queries should be defined in the body)
  - Subprograms (the code for public subprograms are defined in the body)
- The body can also declare and define private items that cannot be referenced from outside the package
- The body can have
  - An initialization part containing statements to initialize variables and do other one-time setup steps
  - An exception-handling part
- The body can be changed without changing
  - The specification
  - The references to the public items

Packages

- Package subprogram can be mapped to an external Java or C subprogram in either the package specification or package body
- The external subprogram name, parameter types, and return type are mapped to their SQL counterparts using a **call specification**
- A call specification
  - Declares a Java method or a C language subprogram so that it can be invoked from PL/SQL
  - Tells the database which Java method, or which named subprogram in which shared library, to invoke when an invocation is made
  - Tells the database what type conversions to make for the arguments and return value
  - Use the SQL CALL statement to invoke such a method or subprogram

Packages

- The AUTHID clause of the package specification determines whether the subprograms and cursors in the package run with the privileges of their definer (the default) invoker
  - Their unqualified references to schema objects are resolved in the schema of the definer or invoker

CREATE OR REPLACE PACKAGE pck_name AUTHID {CURRENT_USER | DEFINER}

  - CURRENT_USER – executes with the roles and privileges of the current user (invoker)
  - DEFINER – executes with the roles and privileges of the definer

- The ACCESSIBLE BY clause of the package specification lets developers to specify a white list of PL/SQL units that can access the package

CREATE OR REPLACE PACKAGE pck_name ACCESSIBLE BY (PROCEDURE proc_name, FUNCTION func_name, etc)

Packages

- Packages support the development and maintenance of **reliable** and **reusable code** with the following features:

1. Modularity
2. Easier Application Design
3. Hidden Implementation Details
4. Added Functionality
5. Better Performance
6. Easier to Grant Roles

Packages
Modularity

- Packages enables the encapsulation in named PL/SQL modules of logically related
  - Types
  - Variables and constants
  - Subprograms
  - Cursors
  - Exceptions

- Developers can make each package
  - Easy to understand
  - Make the interfaces between packages
    - Simple
    - Clear
    - Well defined

Packages
Easier Application Design

- When designing an application, developers need initially the interface information in the package specifications
- Developers
  - Can code and compile specifications without their bodies
  - Can compile standalone subprograms that reference the packages
  - Are not required to fully define the package bodies until the applications is ready for completion

Packages
Hidden Implementation Details

- Packages let developers
  - Share interface information in the package specification
  - Hide the implementation details in the package body
- Hiding the implementation details in the body has these advantages:
  - Implementation details can be changed without affecting the application interface
  - Application users cannot develop code that depends on implementation details that might change

Packages
Added Functionality

- Package public variables and cursors
  - **Can persist for the life of a session**
  - Can be shared by all subprograms that run in the environment
  - Permits the maintenance of data across transactions without storing them in the database

Packages
Better Performance

- Oracle Database loads the whole package into memory the first time a package subprogram is invoked
- Subsequent invocations of other subprograms in same the package require no disk I/O
- Packages prevent
  - Cascading dependencies
  - Unnecessary recompiling
- If the body of a package function is changed:
  - Oracle Database does not recompile other subprograms that invoke the function
  - Reason: these subprograms depend only on the parameters and return value that are declared in the specification

Packages
Easier to Grant Roles

- Developers and administrators can grant roles on the package, instead of granting roles on each object in the package

Package Specifications

- To create a package specification, use the CREATE PACKAGE statement

```
CREATE [OR REPLACE] PACKAGE package_name
[IS|AS]
    public declarations;
end [package_name];
```

Package Specifications

- A package specification declares public items
  - The scope of a public item is the schema of the package
  - A public item is visible everywhere in the schema
  - To reference a public item that is in scope but not visible, qualify it with the package name **pck_name.public_item_name**
  - Each public item declaration has all the information needed to use the item

- The ACCESSIBLE BY clause must be included in the package specification to restrict the use of a package to specified PL/SQL units

Package Specifications
Public Declarations

- Types, variables, constants, subprograms, cursors, and exceptions used by multiple subprograms
  - A type defined in a package specification is either a PL/SQL user-defined subtype or a PL/SQL composite type
  - A PL/SQL composite type defined in a package specification is incompatible with an identically defined local or standalone type

- Associative array types of standalone subprogram parameters
  - An associative array type cannot be declared at the schema level
  - The type of that variable must be declared in a package specification to pass an associative array variable as a parameter to a standalone subprogram
  - Doing so makes the type available to both
    - The invoked subprogram (which declares a formal parameter of that type)
    - To the invoking subprogram or anonymous block (which declares a variable of that type)

Package Specifications
Public Declarations

- Variables that must remain available between subprogram invocations in the same session
- Subprograms that read and write public/private variables
  - Write "get" and "set" subprograms (i.e., setters and getters)
  - Provide these subprograms to discourage package users from reading and writing public variables directly
- Subprograms that invoke each other
- Overloaded subprograms
  - Are variations of the same subprogram
  - Have the same name but different formal parameters (and data type for the parameters)

Package Body

- To create a package body, use the CREATE PACKAGE BODY statement

```
CREATE [OR REPLACE] PACKAGE
BODY package_name
[IS|AS]
    public definitions;
    private declarations;
    private definitions;
BEGIN
EXCEPTION
END [package_name];
```

Package Body

- A package body is required if a package specification declares cursors or subprograms (otherwise, it is optional)
- The package body and package specification must be in the same schema
- Every cursor or subprogram declaration in the package specification must have a corresponding definition in the package body
- The headings of corresponding subprogram declarations and definitions must match word for word (except for white space)
- The cursors and subprograms declared in the package specification and defined in the package body are public items that can be referenced from outside the package

Package Body

- The package body can also declare and define private items
  - That cannot be referenced from outside the package
  - Are necessary for the internal workings of the package

- The body can have an initialization part with statements initialize public variables and do other one-time setup steps
  - The initialization part runs only the first time the package is referenced
  - The initialization part can include an exception handler

- The package body can be changed without changing
  - The specification
  - The references to the public items.

Package Example

```
DROP TABLE people;

CREATE TABLE people (name, salary, deptno) AS
SELECT first_name || ' ' || last_name, salary, department_id
FROM employees;


CREATE OR REPLACE PACKAGE pkg_people IS
  CURSOR c RETURN people%ROWTYPE;
  PROCEDURE bonus;
END pkg_people;
/
```

```
BEGIN
  pkg_people.bonus;
END;
/
```

Package Example

```
CREATE OR REPLACE PACKAGE BODY pkg_people IS
  CURSOR c RETURN people%ROWTYPE IS SELECT * FROM people;

  FUNCTION cbonus(sal people.salary%type,
                  idd people.deptno%type,
                  avgsal OUT float)
  RETURN number
  IS
  BEGIN
    SELECT avg(salary) INTO avgsal
    FROM people WHERE deptno=idd;
    IF avgsal > sal THEN RETURN sal*0.2;
    ELSE RETURN sal*0.1;
    END IF;
  END cbonus;

  PROCEDURE bonus
  IS
    avgsal float(10);
    bonus float(10);
  BEGIN
    FOR p IN c LOOP
      bonus := cbonus(p.salary, p.deptno, avgsal);
      DBMS_OUTPUT.PUT_LINE(p.name||' ia un bonus de '
                           || bonus || ' (' || p.salary
                           || ' ~ ' || avgsal || ')');
    END LOOP;
    IF c%ISOPEN THEN
      CLOSE c;
    END IF;
  END bonus;
END pkg_people;
/
```

Package Example

```
CREATE OR REPLACE PACKAGE test IS
  FUNCTION power(x float) RETURN float;
  FUNCTION power(x float, y float) RETURN float;
END test;
/

CREATE OR REPLACE PACKAGE BODY test IS
  PROCEDURE power(x float, y float, z IN OUT float) IS
  BEGIN
    z := x ** y;
  END;

  FUNCTION power(x float) RETURN float IS
    rez float;
  BEGIN
    test.power(x, 2, rez);
    RETURN rez;
  END;

  FUNCTION power(x float, y float) RETURN float IS
    rez float;
  BEGIN
    test.power(x, y, rez);
    RETURN rez;
  END;
END test;
/

BEGIN
  dbms_output.put_line(test.power(2)); -- 4
  dbms_output.put_line(test.power(2, 5)); -- 32
END;
/
```

Package Instantiation and Initialization

- Oracle Database instantiates the package for that session when a session references a package item
- Every session that references a package has its own instantiation of that package
- When Oracle Database instantiates a package, it also initializes it
- Initialization includes whichever of the following are applicable:
  - Assigning initial values to public constants
  - Assigning initial values to public variables whose declarations specify them
  - Executing the initialization part of the package body

Package State

- The values of the variables, constants, and cursors that a package declares (in either its specification or body) comprise its **package state**
- If a PL/SQL package declares at least one variable, constant, or cursor, then the package is stateful (otherwise, it is stateless)
- Each session that references a package item has its own instantiation of that package.
- If the package is stateful, the instantiation includes its state

Package State

- The package state persists for the life of a session, except in these situations:
  - The package is SERIALLY_REUSABLE
  - The package body is recompiled.
  - Any of the session's instantiated packages are invalidated and revalidated

- Oracle Database treats a package as stateless if its state is constant for the life of a session (or longer).

- This is the case for a package whose items are all compile-time constants.

Packages SERIALLY_REUSABLE

- SERIALLY_REUSABLE packages let developers design applications that manage memory better for scalability
- If the package is **NOT SERIALLY_REUSABLE**
  - The package state is stored in the user global area (UGA) for each user
  - The amount of UGA memory needed increases linearly with the number of users, limiting scalability
  - The package state can persist for the life of a session, locking UGA memory until the session ends

Packages SERIALLY_REUSABLE

- If a package is SERIALLY_REUSABLE
  - The package state is stored in the work area in a small pool in the system global area (SGA)
  - The package state persists only for the life of a server call
  - After the server call, the work area returns to the pool
  - If a subsequent server call references the package, then Oracle Database reuses an instantiation from the pool
  - By reusing an instantiation
    - The package re-initializes the state
    - The changes made to the package state in previous server calls are invisible

Packages SERIALLY_REUSABLE

- Include the SERIALLY_REUSABLE pragma in the package specification and, if it exists, the package body

```
CREATE OR REPLACE PACKAGE pkg IS
  PRAGMA SERIALLY_REUSABLE;

CREATE OR REPLACE PACKAGE BODY pkg IS
  PRAGMA SERIALLY_REUSABLE;
```

Packages SERIALLY_REUSABLE

- The work unit is a server call for a SERIALLY_REUSABLE package
- Public variables can be used only within the work unit
- After the work unit (server call) of a SERIALLY_REUSABLE package completes, Oracle Database does the following:
  - Closes any open cursors
  - Frees some non-reusable memory (for example, memory for collection and long VARCHAR2 variables)
  - Returns the package instantiation to the pool of reusable instantiations kept for this package

Packages SERIALLY_REUSABLE

- An explicit cursor in a SERIALLY_REUSABLE package
  - Remains open until either
    - It is close explicitly
    - Its work unit (server call) ends
  - A new server call must be made to re-open the cursor
  - A server call can be different from a subprogram invocation

Package Example

```
CREATE OR REPLACE PACKAGE test IS
    FUNCTION getN RETURN NUMBER;
    PROCEDURE setN(val NUMBER := 10);
END test;
/

CREATE OR REPLACE PACKAGE BODY test IS
    n NUMBER;
    FUNCTION getN RETURN NUMBER
    IS
    BEGIN
        RETURN n;
    END getN;

    PROCEDURE setN(val NUMBER := 10)
    IS
    BEGIN
        n := val;
    END;
BEGIN
    n := 20;
END test;
/

set serveroutput on
BEGIN
    dbms_output.put_line(test.getN);
    test.setN;
    dbms_output.put_line(test.getN);
    test.setN(40);
    dbms_output.put_line(test.getN);
END;
/
```

```
CREATE OR REPLACE PACKAGE sr_test IS
    PRAGMA SERIALLY_REUSABLE;
    FUNCTION getN RETURN NUMBER;
    PROCEDURE setN(val NUMBER := 10);
END sr_test;
/

CREATE OR REPLACE PACKAGE BODY sr_test IS
    PRAGMA SERIALLY_REUSABLE;
    n NUMBER;
    FUNCTION getN RETURN NUMBER
    IS
    BEGIN
        RETURN n;
    END getN;

    PROCEDURE setN(val NUMBER := 10)
    IS
    BEGIN
        n := val;
    END;
BEGIN
    n := 20;
END sr_test;
/

set serveroutput on
BEGIN
    dbms_output.put_line(sr_test.getN);
    sr_test.setN;
    dbms_output.put_line(sr_test.getN);
    sr_test.setN(40);
    dbms_output.put_line(sr_test.getN);
END;
/
```

Package Example

```sql
CREATE OR REPLACE PACKAGE pkg IS
  n NUMBER;
END pkg;
/

CREATE OR REPLACE PACKAGE BODY pkg IS
BEGIN
  n := 5;
END pkg;
/

CREATE OR REPLACE PACKAGE sr_pkg IS
  PRAGMA SERIALLY_REUSABLE;
  n NUMBER;
END sr_pkg;
/

CREATE OR REPLACE PACKAGE BODY sr_pkg IS
  PRAGMA SERIALLY_REUSABLE;
BEGIN
  n := 5;
END sr_pkg;
/

BEGIN
  DBMS_OUTPUT.PUT_LINE('pkg.n: ' || pkg.n); -- pkg.n: 5
  DBMS_OUTPUT.PUT_LINE('sr_pkg.n: ' || sr_pkg.n); -- sr_pkg.n: 5
END;
/

BEGIN
  pkg.n := 10;
  sr_pkg.n := 10;
END;
/

BEGIN
  DBMS_OUTPUT.PUT_LINE('pkg.n: ' || pkg.n); -- pkg.n: 10
  DBMS_OUTPUT.PUT_LINE('sr_pkg.n: ' || sr_pkg.n); -- sr_pkg.n: 5
END;
/
```

Package Writing Guidelines

- Become familiar with the packages that Oracle Database supplies, and avoid writing packages that duplicate their features
- Keep your packages general so that future applications can reuse them
- Design and define the package specifications before the package bodies
- In package specifications, declare only items that must be visible to invoking programs to
  - Prevent other developers from building unsafe dependencies on your implementation details
  - Reduce the need for recompilation
  - Prevent the need to recompile any subprograms that invoke the public subprograms of the package if the package specification changes
  - Remove the need for recompiling the subprograms if only the package body changes

Package Writing Guidelines

- Declare public cursors in package specifications and define them in package bodies
  - Hides cursors' queries from package users
  - Developers can change cursors' queries without changing cursor declarations

- Assign initial values in the initialization part of the package body instead of initializing them when declared
  - The code for computing the initial values can be more complex and better documented
  - If computing an initial value raises an exception, the initialization part can handle it with its own exception handler.
  - Make helper packages available only to the API if the database applications have several PL/SQL packages

Overview

- Packages
- Triggers

Triggers

- A trigger is a named PL/SQL unit that is stored in the database and can be invoked repeatedly
- A developer
  - Can enable and disable a trigger
  - Cannot explicitly invoke a trigger
- If the trigger is enabled, the database automatically invokes the trigger whenever its triggering event occurs
- If the trigger is disabled, the database does not invoke it
- To create a trigger, use the CREATE TRIGGER statement

Triggers

- At creation time, specify
  - The **triggering event in terms of triggering statements and the item on which they act**
  - The **timing point**, which determines whether the **trigger fires before or after** the triggering statement runs
  - Whether it **fires for the entire event or each row** that the triggering statement affects
- By default, a trigger is created in the enabled state
- The trigger is said to be "created on" or "defined on" the item, which is either a table, a view, a schema, or the database

Triggers

- A **DML trigger**
  - Is created on a table or view
  - The triggering event is composed of DML statements
- A **cross edition trigger**
  - Is a DML trigger for use for edition-based redefinition
- A **system trigger**
  - Is created on a schema or the database
  - The triggering event is composed of either DDL or database operation statements

Triggers

- A **conditional trigger**
  - Is a DML or system trigger
  - Has a WHEN clause that specifies a SQL condition that the database evaluates for each row that the triggering statement affects

Triggers

- An **INSTEAD OF trigger** is either:
  - A DML trigger created on either
    - A non-editioning view
    - A nested table column of a non-editioning view
  - A system trigger defined on a CREATE statement
  - The database fires the INSTEAD OF trigger instead of running the triggering statement

Triggers

- NOTE: When a trigger fires, tables that the trigger references might be undergoing changes made by SQL statements in other users' transactions
- SQL statements running in triggers follow the same rules that standalone SQL statements do
- Specifically:
  - Queries in the trigger see the current read-consistent materialized view of referenced tables and any data changed in the same transaction
  - Updates in the trigger wait for existing data locks to be released before proceeding

Triggers

- Triggers enable the customization of the database management system

- Triggers can be used to:
  - Automatically generate virtual column values
  - Log events
  - Gather statistics on table access
  - Modify table data when DML statements are issued against views
  - Enforce referential integrity when child and parent tables are on different nodes of a distributed database
  - Publish information about
    - Database events
    - User events
    - SQL statements to subscribing applications

Triggers

- Triggers can be used to:
  - Prevent DML operations on a table after regular business hours
  - Prevent invalid transactions
  - Enforce complex business rules that cannot be defined with constraints
  - Enforce complex referential integrity rules that cannot be defined with constraints
- Caution:
  - Triggers are not reliable security mechanisms, because they are programmatic and easy to disable
  - For high-assurance security, use Oracle Database Vault

How Triggers and Constraints Differ

- Although triggers and constraints can constrain data input, they differ significantly
- A trigger always applies to new data only
- Example: Enforce NULL constraint
  - A table column does not have a null constraint
  - A trigger can prevent a DML statement from inserting NULL values
  - But the table can contain NULL values that were inserted
    - Before the trigger was defined
    - While the trigger was disabled

How Triggers and Constraints Differ

- A constraint can apply either to new data only (like a trigger) or to both new and existing data
- Constraints behavior depend on the constraint state
- Constraints are easier to write and less error-prone than triggers that enforce the same rules
- Triggers can enforce some complex business rules that constraints cannot

Triggers

- Oracle strongly recommends to use triggers to constrain data input only in these situations:
  - To enforce referential integrity when child and parent tables are on different nodes of a distributed database
  - To enforce complex business or referential integrity rules that you cannot define with constraints

DML Triggers

- A DML trigger
  - Is created on either a table or view
  - Its triggering event is composed of the DML statements DELETE, INSERT, and UPDATE
- To create a trigger that fires in response to a MERGE statement create triggers on the INSERT and UPDATE statements to which the MERGE operation decomposes
- A DML trigger is either
  - Simple
  - Compound

DML Triggers

- A **simple DML trigger** fires at exactly one of these timing points:
  - Before the triggering statement runs
    - The trigger is called
      - A *BEFORE statement trigger*
      - *Statement-level BEFORE trigger*
  - After the triggering statement runs
    - The trigger is called
      - An *AFTER statement trigger*
      - *Statement-level AFTER trigger*

DML Triggers

- A **simple DML trigger** fires at exactly one of these timing points:
  - Before each row that the triggering statement affects
    - The trigger is called
      - A *BEFORE each row trigger*
      - *Row-level BEFORE trigger*
  - After each row that the triggering statement affects
    - The trigger is called
      - An *AFTER each row trigger*
      - *Row-level AFTER trigger*

DML Triggers

- A **compound DML trigger** created on a table or editioning view can fire at one, some, or all of the preceding timing points
- Compound DML triggers help define an approach where the actions implemented for the various timing points manage to share common data
- A simple or compound DML trigger that fires at row level can access the data in the row that it is processing
- An **INSTEAD OF DML trigger** is a DML trigger created on either a non-editioning view or a nested table column of a non-editioning view

DML Triggers

- Syntax:

```
CREATE [OR REPLACE] trigger_name
  {BEFORE | AFTER | INSTEAD OF}
  {INSERT | UPDATE [OF column_list] | DELETE
     [OR {INSERT | UPDATE [OF column_list] | DELETE} ...]}
  ON [table_name | view]
  [referencing clauses]
  [FOR EACH ROW]
  [WHEN (conditions)]
DECLARE
   trigger_variables
BEGIN
    statements
END;
/
```

DML Trigger Example

```
1 CREATE OR REPLACE TRIGGER t1
2   AFTER DELETE
3   ON employees
4   FOR EACH ROW
5 BEGIN
6   DBMS_OUTPUT.PUT_LINE('Deleting');
7 END;
8 /
```

DML Triggers
Conditional Predicates

- The triggering event of a DML trigger can be composed of multiple triggering statements
- When one of them fires the trigger, the trigger can determine which one by using conditional predicates
- A conditional predicate can appear wherever a BOOLEAN expression can appear

Conditional Predicate	TRUE if and only if:
INSERTING	An INSERT statement fired the trigger.
UPDATING	An UPDATE statement fired the trigger.
UPDATING ('column')	An UPDATE statement that affected the specified column fired the trigger.
DELETING	A DELETE statement fired the trigger.

DML Trigger
Conditional Predicates Example

```
 1 CREATE OR REPLACE TRIGGER t2
 2   BEFORE INSERT OR UPDATE OF salary, department_id OR DELETE
 3   ON employees
 4 BEGIN
 5   CASE
 6     WHEN INSERTING THEN
 7       DBMS_OUTPUT.PUT_LINE('Inserting');
 8     WHEN UPDATING('salary') THEN
 9       DBMS_OUTPUT.PUT_LINE('Updating salary');
10     WHEN UPDATING('department_id') THEN
11       DBMS_OUTPUT.PUT_LINE('Updating department ID');
12     WHEN DELETING THEN
13       DBMS_OUTPUT.PUT_LINE('Deleting');
14   END CASE;
15 END;
16 /
```

Correlation Names and Pseudorecords

- A trigger that fires at row level can access the data in the row that it is processing by using correlation names
- The default correlation names are OLD, NEW, and PARENT
- To change the correlation names, use the REFERENCING clause of the CREATE TRIGGER statement

Correlation Names and Pseudorecords

- If the trigger is created on a nested table
  - OLD and NEW refer to the current row of the nested table
  - PARENT refers to the current row of the parent table
- If the trigger is created on a table or view
  - OLD and NEW refer to the current row of the table or view
  - PARENT is undefined
- OLD, NEW, and PARENT are also called pseudorecords
  - They have record structure
  - Are allowed in fewer contexts than records are

Correlation Names and Pseudorecords

- The structure of a pseudorecord is table_name%ROWTYPE, where table_name is
  - The name of the table on which the trigger is created (for OLD and NEW)
  - The name of the parent table (for PARENT)

- Syntax of a pseudorecord (in the trigger body)

  :*pseudorecord_name.field_name*

- In the WHEN clause of a conditional trigger
  - A correlation name is not a placeholder for a bind variable
  - Omit the colon

Correlation Names and Pseudorecords

Triggering Statement	OLD.field Value	NEW.field Value
INSERT	NULL	Post-insert value
UPDATE	Pre-update value	Post-update value
DELETE	Pre-delete value	NULL

Correlation Names and Pseudorecords

- The restrictions on pseudorecords are:
  - A pseudorecord cannot appear in a record-level operation
    - For example, the trigger cannot include this statement: **:NEW := NULL;**
  - A pseudorecord cannot be an actual subprogram parameter
    - A pseudorecord field can be an actual subprogram parameter
  - The trigger cannot change OLD field values
    - Trying to do so raises ORA-04085
  - If the triggering statement is DELETE, then the trigger cannot change NEW field values.
    - Trying to do so raises ORA-04084
  - An AFTER trigger cannot change NEW field values, because the triggering statement runs before the trigger fires.
    - Trying to do so raises ORA-04084

Correlation Names and Pseudorecords

- A BEFORE trigger can change NEW field values before a triggering INSERT or UPDATE statement puts them in the table
- If a statement triggers both a BEFORE trigger and an AFTER trigger and the BEFORE trigger changes a NEW field value, then the AFTER trigger "sees" that change

Correlation Names and Pseudorecords

```
 1 CREATE OR REPLACE TRIGGER t3
 2   BEFORE DELETE OR INSERT OR UPDATE ON employees
 3   REFERENCING NEW AS n
 4   FOR EACH ROW
 5   WHEN (n.job_id = 'ST_CLERK' and OLD.salary < 4000 )
 6 DECLARE
 7   sal_diff  NUMBER;
 8 BEGIN
 9   sal_diff  := :n.salary  - :OLD.salary;
10   DBMS_OUTPUT.PUT(:n.last_name || ': ');
11   DBMS_OUTPUT.PUT('Old salary = ' || :OLD.salary || ', ');
12   DBMS_OUTPUT.PUT('New salary = ' || :n.salary || ', ');
13   DBMS_OUTPUT.PUT_LINE('Difference: ' || sal_diff);
14 END;
15 /
16
17 update employees set salary = salary * 1.3;
```

INSTEAD OF DML Triggers

- An INSTEAD OF DML trigger is a DML trigger created on a non-editioning view, or on a nested table column of a non-editioning view.
- The database fires the INSTEAD OF trigger instead of running the triggering DML statement
- An INSTEAD OF trigger
  - Cannot be conditional (no WHEN clause)
  - Is the only way to update a view that is not inherently updatable
    - Design the INSTEAD OF trigger to determine what operation was intended and do the appropriate DML operations on the underlying tables
  - Is always a row-level trigger
  - Can read OLD and NEW values, but cannot change them

INSTEAD OF DML Triggers

```
CREATE OR REPLACE VIEW cr AS
  SELECT region_id, region_name, country_id, country_name
  FROM regions natural join countries;

CREATE OR REPLACE TRIGGER insert_cr INSTEAD OF INSERT ON cr
DECLARE
    ok number;
    duplicate_info EXCEPTION;
    PRAGMA EXCEPTION_INIT (duplicate_info, -00001);
BEGIN
  SELECT count(*) INTO ok FROM regions WHERE region_id = :new.region_id;
  IF ok=1 THEN
    IF :new.country_id IS NOT NULL THEN
      INSERT INTO countries VALUES(:new.country_id, :new.country_name, :new.region_id);
      DBMS_OUTPUT.PUT_LINE('Country added!');
    END IF;
  ELSE
    DBMS_OUTPUT.PUT_LINE('Region does not exist!');
    INSERT INTO regions VALUES(:new.region_id, :new.region_name);
    DBMS_OUTPUT.PUT_LINE('Region added!');
    IF :new.country_id IS NOT NULL THEN
      INSERT INTO countries VALUES(:new.country_id, :new.country_name, :new.region_id);
      DBMS_OUTPUT.PUT_LINE('Country added!');
    END IF;
  END IF;
EXCEPTION
  WHEN duplicate_info THEN
    RAISE_APPLICATION_ERROR(num=> -20107, msg=> 'Duplicate country');
END insert_cr;
/

INSERT INTO cr(region_id, country_id, country_name) VALUES(1, 'RO', 'Romania');

INSERT INTO cr(region_id, region_name, country_id, country_name)
VALUES(5, 'Oceania', 'AU', 'Australia');
```

INSTEAD OF DML Triggers

- An INSTEAD OF trigger with the NESTED TABLE clause
  - Fires only if the triggering statement operates on the elements of the specified nested table column of the view
  - Fires for each modified nested table element

INSTEAD OF DML Triggers

```sql
CREATE OR REPLACE TYPE job_line AS OBJECT(
  job_id VARCHAR2(10),
  job_title VARCHAR2(35),
  min_salary NUMBER(6),
  max_salary NUMBER(6)
);
/

CREATE OR REPLACE TYPE job_list IS TABLE OF job_line;
/

CREATE OR REPLACE VIEW dept_view AS
SELECT d.department_id, d.department_name,
  CAST (MULTISET (SELECT DISTINCT j.job_id, j.job_title, j.min_salary, j.max_salary
            FROM jobs j INNER JOIN employees e ON j.job_id = e.job_id
            WHERE e.department_id = d.department_id) AS job_list) dept_jobs
FROM departments d
WHERE EXISTS(SELECT employee_id FROM employees e WHERE e.department_id = d.department_id);

CREATE OR REPLACE TRIGGER dept_emplist_tr
  INSTEAD OF INSERT ON NESTED TABLE dept_jobs OF dept_view
  FOR EACH ROW
BEGIN
  DBMS_OUTPUT.PUT_LINE(:PARENT.department_name);
  DBMS_OUTPUT.PUT_LINE(:NEW.job_id||' '||:NEW.job_title||' '||:NEW.min_salary||' '||:NEW.max_salary);
  INSERT INTO jobs (job_id, job_title, min_salary, max_salary)
  VALUES (:NEW.job_id, :NEW.job_title, :NEW.min_salary, :NEW.max_salary);
END;
/

INSERT INTO TABLE(SELECT d.dept_jobs FROM dept_view d WHERE department_id = 10)
VALUES ('Tester', 'Super Tester', 2000, 5000);
```

Compound DML Triggers

- A compound DML trigger created on a table or editioning view can fire at multiple timing points

- Each timing point section
  - Has its own executable part
  - Has its own optional exception-handling part
  - Can access a common PL/SQL state

- The common state
  - Is established when the triggering statement starts
  - Is destroyed when the triggering statement completes, even when the triggering statement causes an error

- A compound DML trigger created on a non-editioning view is not really compound, because it has only one timing point section

Compound DML Triggers

- A compound trigger can be conditional, but not autonomous:
  - An autonomous subprogram
    - Can do SQL operations and commit or roll back those operations, without committing or rolling back the data in the main transaction
    - Uses **PRAGMA AUTONOMOUS_TRANSACTION**;

- Two common uses of compound triggers are:
  - To accumulate rows destined for a second table so that you can periodically bulk-insert them
  - To avoid the mutating-table error (ORA-04091):
    - A *mutating*-table is a table that
      - Is currently being modified by an UPDATE, DELETE, or INSERT statement
      - Might need to be updated by the effects of a declarative DELETE CASCADE referential integrity constraint

Compound DML Triggers

- A compound DML trigger created on a table or editioning view has at least one timing-point section

- If the trigger has multiple timing-point sections
  - They can be in any order
  - No timing-point section can be repeated
  - If a timing-point section is absent, then nothing happens at its timing point

Timing Point	Section
Before the triggering statement runs	BEFORE STATEMENT
After the triggering statement runs	AFTER STATEMENT
Before each row that the triggering statement affects	BEFORE EACH ROW
After each row that the triggering statement affects	AFTER EACH ROW

Compound DML Triggers

- Compound DML triggers have these restrictions:
  - OLD, NEW, and PARENT cannot appear
    - In the declarative part
    - The BEFORE STATEMENT section
    - The AFTER STATEMENT section
  - Only the BEFORE EACH ROW section can change the value of NEW
  - A timing-point section cannot handle exceptions raised in another timing-point section
  - If a timing-point section includes a GOTO statement, the target of the GOTO statement must be in the same timing-point section

Compound DML Triggers

- Syntax

```
CREATE OR REPLACE TRIGGER compound_trigger_name
FOR [INSERT|DELETE]UPDATE [OF column]
ON table COMPOUND TRIGGER
   -- Declarative Section (optional)
   -- Variables declared here have firing-statement duration.

   -- Executed before DML statement
   BEFORE STATEMENT IS
   BEGIN
     NULL;
   END BEFORE STATEMENT;

   -- Executed before each row change
   -- :NEW, :OLD are available
   BEFORE EACH ROW IS
   BEGIN
     NULL;
   END BEFORE EACH ROW;

   -- Executed aftereach row change
   -- :NEW, :OLD are available
   AFTER EACH ROW IS
   BEGIN
     NULL;
   END AFTER EACH ROW;

   -- Executed after DML statement
   AFTER STATEMENT IS
   BEGIN
     NULL;
   END AFTER STATEMENT;
END compound_trigger_name;
```

Compound DML Trigger Example

```
1 CREATE TABLE employee_salaries (
2   employee_id NUMBER NOT NULL,
3   change_date DATE   NOT NULL,
4   salary NUMBER(8,2) NOT NULL,
5   CONSTRAINT pk_employee_salaries PRIMARY KEY (employee_id, change_date),
6   CONSTRAINT fk_employee_salaries FOREIGN KEY (employee_id)
7     REFERENCES employees (employee_id) ON DELETE CASCADE
8 )
9 /
```

Compound DML Trigger Example

```
CREATE OR REPLACE TRIGGER maintain_employee_salaries
  FOR UPDATE OF salary ON employees COMPOUND TRIGGER
  -- Declarative Part:
  threshhold CONSTANT SIMPLE_INTEGER := 7;

  TYPE salaries_t IS TABLE OF employee_salaries%ROWTYPE
    INDEX BY SIMPLE_INTEGER;
  salaries  salaries_t;
  idx       SIMPLE_INTEGER := 0;

  PROCEDURE flush_array IS
    n CONSTANT SIMPLE_INTEGER := salaries.count();
  BEGIN
    FORALL j IN 1..n
      INSERT INTO employee_salaries VALUES salaries(j);
    salaries.delete();
    idx := 0;
    DBMS_OUTPUT.PUT_LINE('Flushed ' || n || ' rows');
  END flush_array;

  -- AFTER EACH ROW Section:
  AFTER EACH ROW IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('AFTER EACH ROW!');
    idx := idx + 1;
    salaries(idx).employee_id := :NEW.employee_id;
    salaries(idx).change_date := SYSTIMESTAMP;
    salaries(idx).salary := :NEW.salary;
    IF idx >= threshhold THEN
      flush_array();
    END IF;
  END AFTER EACH ROW;

  -- AFTER STATEMENT Section:
  AFTER STATEMENT IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('AFTER STATEMENT!');
    flush_array();
  END AFTER STATEMENT;
END maintain_employee_salaries;
/
```

Compound DML Trigger Example

```
1 UPDATE employees
2   SET salary = salary * 1.05
3   WHERE department_id = 50
4 /
5
6 SELECT employee_id, count(*) c
7   FROM employee_salaries
8   GROUP BY employee_id
9 /
```

System Triggers

- A system trigger
  - Is created on either
    - A schema
    - The database
  - Its triggering event is composed of either
    - DDL statements
    - Database operation statements
  - Fires at exactly one of these timing points:
    - Before the triggering statement runs
    - After the triggering statement runs
    - Instead of the triggering CREATE statement

Schema Triggers

- A **SCHEMA trigger**
  - Is created on a schema
  - Fires whenever the user who owns it
    - Is the current user
    - Initiates the triggering event

```
1 CREATE OR REPLACE TRIGGER drop_trigger
2   BEFORE DROP ON hr.SCHEMA
3   BEGIN
4     RAISE_APPLICATION_ERROR (
5       num => -20000,
6       msg => 'Cannot drop object');
7   END;
8 /
```

Database Triggers

- A **DATABASE trigger**
  - Is created on the database
  - Fires whenever any database user initiates the triggering event

```
 1 CREATE TRIGGER log_errors
 2   AFTER SERVERERROR ON DATABASE
 3   BEGIN
 4     IF (IS_SERVERERROR (1017)) THEN
 5       NULL;  -- (substitute code that processes logon error)
 6     ELSE
 7       NULL;  -- (substitute code that logs error code)
 8     END IF;
 9   END;
10 /
```

INSTEAD OF CREATE Triggers

- An **INSTEAD OF CREATE trigger**
  - Is a SCHEMA trigger
  - The triggering event is a CREATE statement
  - The database fires the trigger instead of executing its triggering statement

```
1 CREATE OR REPLACE TRIGGER t
2   INSTEAD OF CREATE ON SCHEMA
3   BEGIN
4     EXECUTE IMMEDIATE 'CREATE TABLE T (n NUMBER, m NUMBER)';
5   END;
6 /
```

Bibliography

- Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 19c*, Oracle Corporation, 2022 [pdf]
- Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 21c*, Oracle Corporation, 2022 [pdf]
- Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 19c*, Oracle Corporation, 2020 [pdf]
- Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 21c*, Oracle Corporation, 2021 [pdf]

University
Politehnica
of Bucharest

Microsoft SQL Server (I)

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@upb.ro

Overview

- Microsoft SQL Server Overview
- Microsoft SQL Server Architecture
- Microsoft SQL Server Big Data Cluster

Overview

- Microsoft SQL Server Overview
- Microsoft SQL Server Architecture
- Microsoft SQL Server Big Data Cluster

Microsoft SQL Server Overview

- Microsoft SQL Server (MSSQL) is a Relational Database Management System (RDBMS) capable of processing large amounts of data preparing for the big-data era
- MSSQL2022 stable release
- MSSQL2019 is **closed-source**
- MSSQL2019 is written in C/C++ like most of the Microsoft (core) products

Rank			DBMS	Database Model	Score		
Nov 2023	Oct 2023	Nov 2022			Nov 2023	Oct 2023	Nov 2022
1.	1.	1.	Oracle	Relational, Multi-model	1277.03	+15.61	+35.34
2.	2.	2.	MySQL	Relational, Multi-model	1115.24	-18.07	-90.30
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	911.42	+14.54	-1.09
4.	4.	4.	PostgreSQL	Relational, Multi-model	636.86	-1.96	+13.70
5.	5.	5.	MongoDB	Document, Multi-model	428.55	-2.87	-49.35
6.	6.	6.	Redis	Key-value, Multi-model	160.02	-2.95	-22.03
7.	7.	7.	Elasticsearch	Search engine, Multi-model	139.62	+2.48	-10.70
8.	8.	8.	IBM Db2	Relational, Multi-model	136.00	+1.13	-13.56
9.	9.	↑ 10.	SQLite	Relational	124.58	-0.56	-10.05
10.	10.	↓ 9.	Microsoft Access	Relational	124.49	+0.18	-10.53

Microsoft SQL Server Overview

- 1989 – Microsoft SQL Server started as a project to port Sybase SQL Server onto OS/2
- 1993 – SQL Server 4.2 is released for Windows NT
- 1995 – SQL Server 6.0 is released: end of collaboration with Sybase
- 1998 – SQL Server 7.0 is released: the source code is converted from C to C++
- 2005 – SQL Server 2005 is released: complete revision of the old Sybase code into Microsoft code
- 2017 – SQL Server 2017 adds Linux support for
  - Red Hat Enterprise Edition Linux
  - SUSE Linux Enterprise Edition
  - **Ubuntu**
  - **Docker Engine**

Microsoft SQL Server Overview

- Current versions still supported by Microsoft (Nov 2023):
  - SQL Server 2014
  - SQL Server 2016
  - SQL Server 2017
  - SQL Server 2019
  - SQL Server 2022 (current)

- From SQL Server 2016 support of x64 processors only

Microsoft SQL Server Overview

- SQL Server 2022 (like 2019) has 5 different editions
- The differences between them address
  - Scalability
  - Transparency
  - Performance
- Editions:
  - **Enterprise:** best for business (unlimited scaling)
  - **Standard:** ideal for small business (moderate scaling)
  - **Express:** portable version (lowest scaling)
  - **Web:** a low total-cost-of-ownership option for Web hosting
  - **Developer:** non-commercial (scale as much as the machine)

Microsoft SQL Server Overview

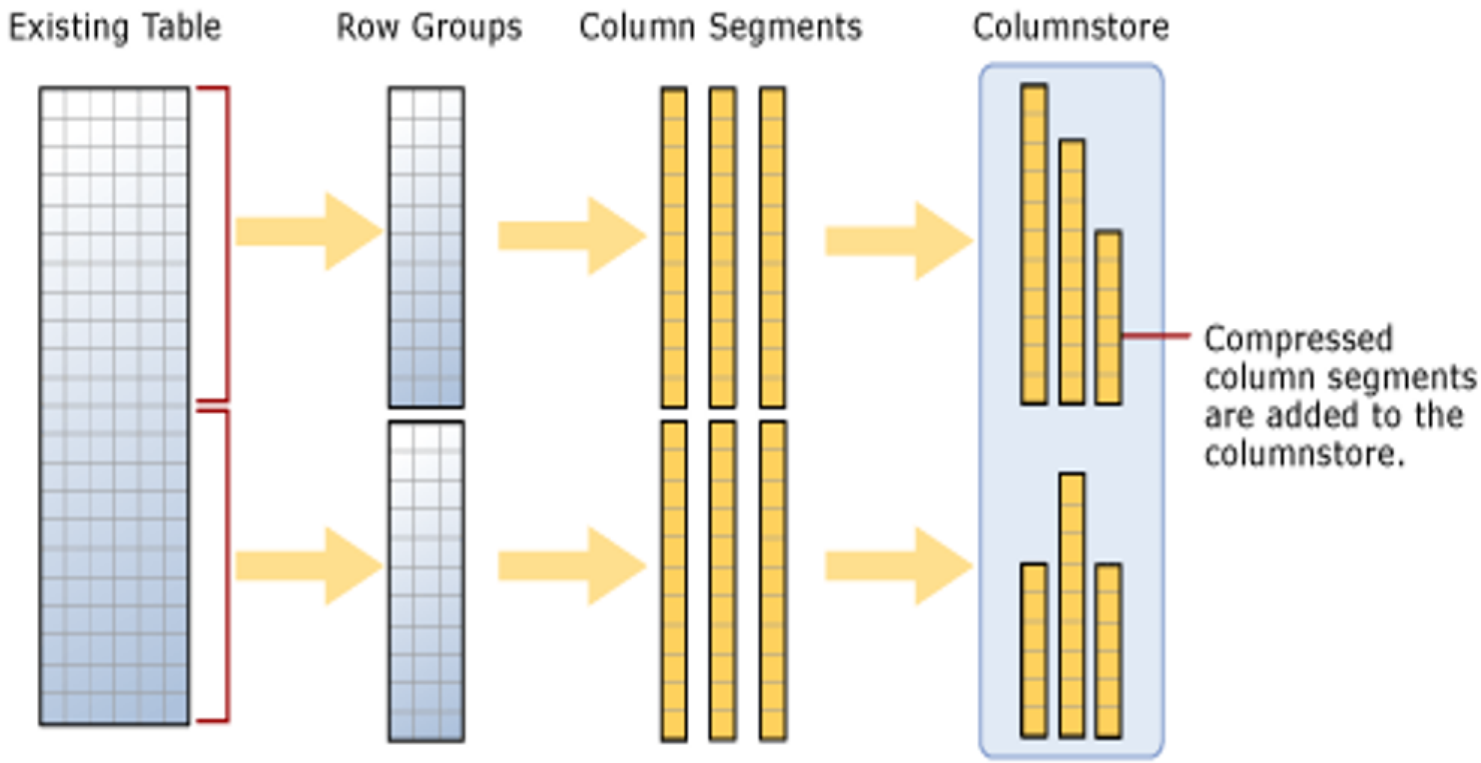
- Microsoft SQL Server is **cloud-ready**
  - Offered as Database-As-A-Service (**DaaS**) on multiple cloud providers
  - In Azure (Microsoft's Cloud Platform) it has a special flavour called Azure SQL Database
- SQL Server Big Data Cluster enabled with the release of Microsoft SQL Server 2019 (will be retired in Feb 2025)

Microsoft SQL Server Overview

- Microsoft SQL Server Features
  - Increase Performance
  - High-Availability
  - Security
  - Replication

Microsoft SQL Server Overview
Increase Peformance

- Columnstore indexes accelerate analytics performance by redefining data storage and queries



Microsoft SQL Server Overview
Increase Peformance

- **In-Memory OLTP** achieves greater speed through memory-optimized tables
- **Adaptive Query Processing** shapes optimization strategies based on application runtime conditions and characteristics
- **Automatic Database Tuning** corrects query execution plans causing performance issues
- **Query Store** gives insights on query plan choice and performance

Microsoft SQL Server Overview
High-Availability

- Always On Failover Cluster Instances and Availability Groups enable HADR (High-Availability Disaster Recovery) scenarios on both Linux and Windows
  - Always on Availability Groups feature is a HADR solution that provides an enterprise-level alternative to database mirroring.
  - Always on Failover Cluster Instances leverages Windows Server Failover Clustering (WSFC) functionality to provide local high availability through redundancy at the server-instance level-a *failover cluster instance* (FCI)
  - Up to eight secondary replicas are supported, including one primary replica and four synchronous-commit secondary replicas

- Read-Scale Availability Groups provide clusterless support for read-only workloads

- Database mirroring maintains two copies of a single database that must reside on different server instances

Microsoft SQL Server Overview
Security

- Database snapshot is a read-only, static view of a SQL Server database
- Dynamic Data Masking limits data exposure while simplifying security design and coding
- Row-Level Security controls access to rows in a database table based on user characteristics
- Always Encrypted provides separation between those who own and those who manage data
- Backup encryption gives extra security for backup files
- Backup compression used to compress backups which minimizes the I/O to increase backup speed significantly

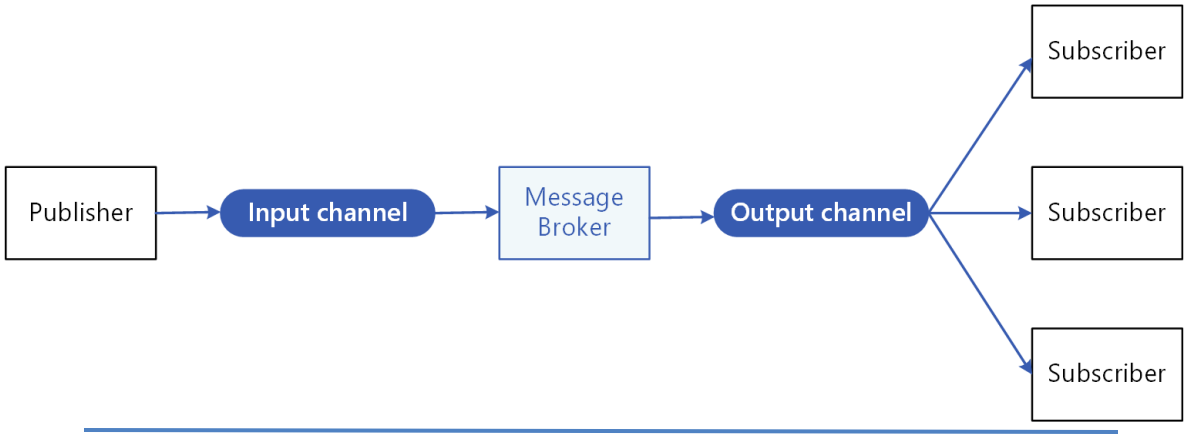
Microsoft SQL Server Overview
Replication

- Replication is a set of technologies for copying and distributing data and database objects from one database to another and then synchronizing to maintain consistency

- SQL Server replication types
  - Snapshot replication
  - Transactional replication
  - Merge replication

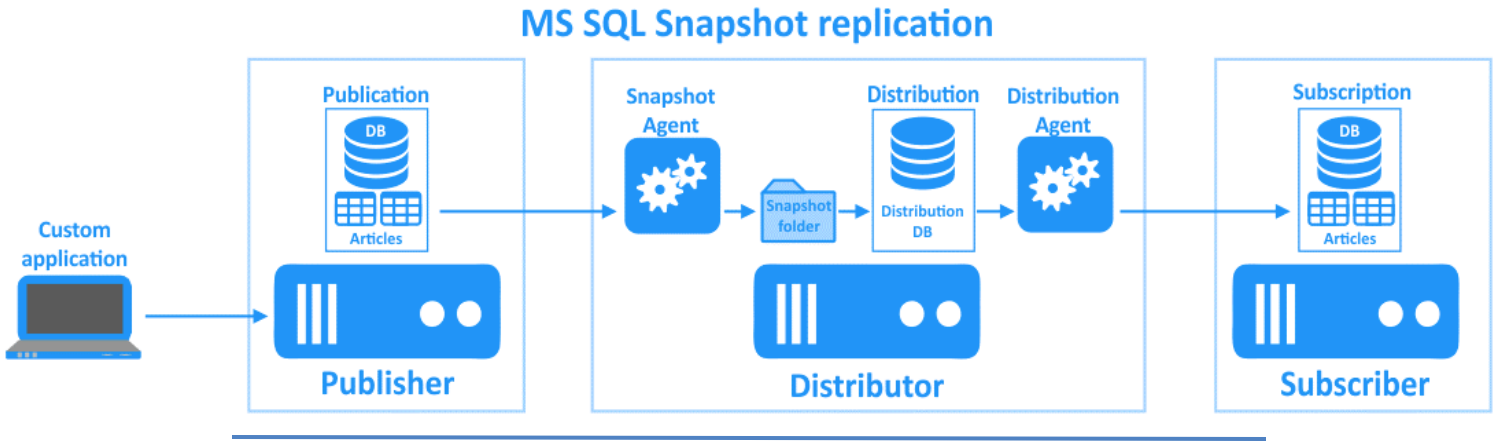
Microsoft SQL Server Overview
Publisher – Subscriber pattern

- Publisher-Subscriber is a design pattern that enables an application to announce events to many interested consumers asynchronously, where:
  - The publisher creates and sends messages/events
  - Subscribers register their interest in receiving some or all messages/events
  - The message broker (distributor) controls the interaction between the publishers and subscribers
    - The publisher post messages/events to the message broker
    - The subscribers register with the broker the messages/events they are interested in receiving
    - The broker stores and forwards messages/events from publishers to subscribers by filtering them by the subscribers' interests



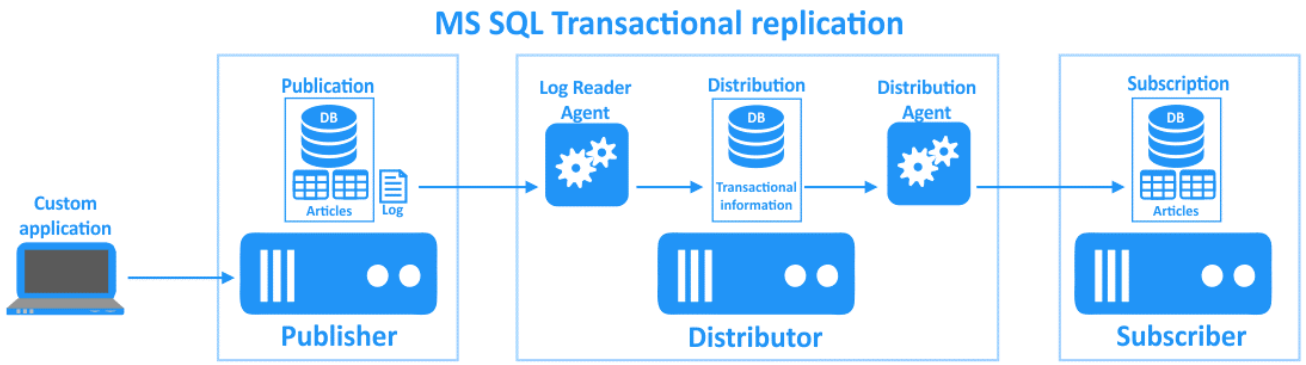
Microsoft SQL Server Overview
Replication

- Snapshot replication
  - Distributes data exactly as it appears at a specific moment in time
  - Does not monitor for updates to the data
  - When synchronization occurs, the entire snapshot is generated and sent to Subscribers



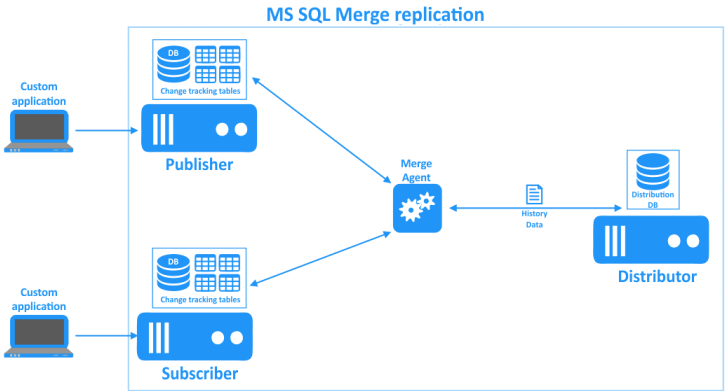
Microsoft SQL Server Overview
Replication

- Transactional replication
  - Starts with a snapshot of the publication database objects and data
  - After the initial snapshot subsequent data changes and schema modifications made at the Publisher are usually delivered to the Subscriber as they occur (in near real time)
  - The data changes are applied to the Subscriber in the same order and within the same transaction boundaries as they occurred at the Publisher



Microsoft SQL Server Overview
Replication

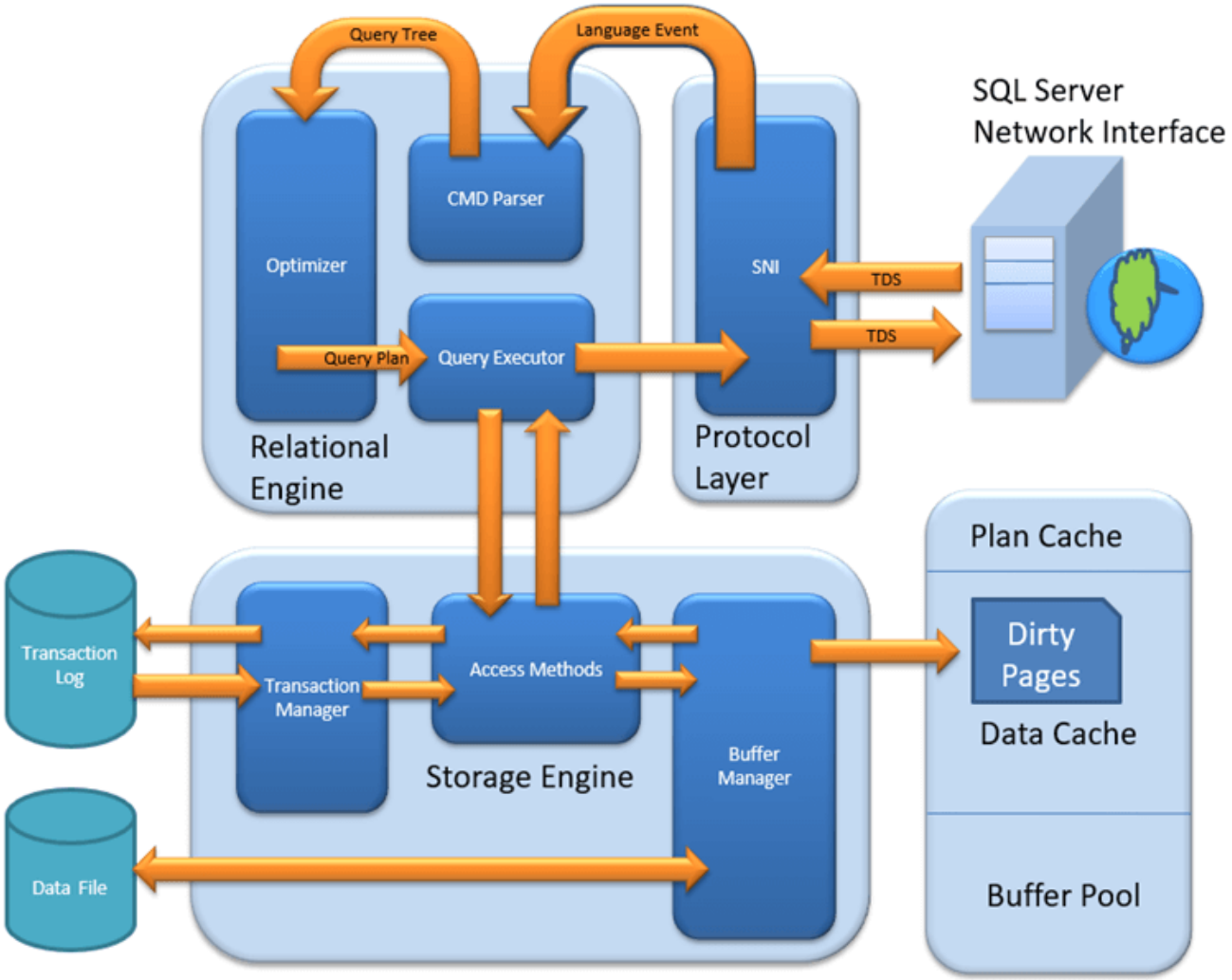
- Merge replication
  - Starts with a snapshot of the publication database objects and data
  - Subsequent data changes and schema modifications made at the Publisher and Subscribers are tracked with triggers
  - The Subscriber
    - Synchronizes with the Publisher when connected to the network
    - Exchanges all rows that have changed between the Publisher and Subscriber since the last time synchronization occurred



Overview

- Microsoft SQL Server Overview
- Microsoft SQL Server Architecture
- Microsoft SQL Server Big Data Cluster

Microsoft SQL Server Architecture



Microsoft SQL Server Architecture

- Microsoft SQL Server Main components:
  - Protocol Layer
  - Relational Engine
  - Storage Engine

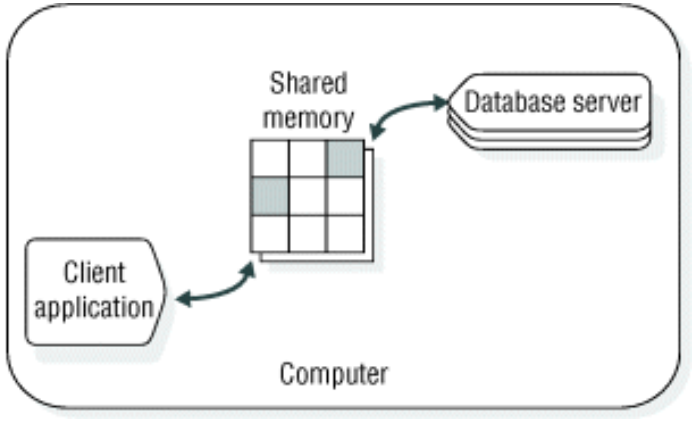
Microsoft SQL Server Architecture
Protocol Layer

- Microsoft SQL Server Network Interface (SNI)
  - Is a protocol layer that establishes the network connection between a client and the server
  - Consists of a set of APIs that are used by both the Database Engine and the SQL Server Native Client (SNAC)

- Supports 3 Client-Server Architectures types that employ one of the following protocols:
  - **Shared memory**
  - **TCP/IP**
  - **Named Pipes**

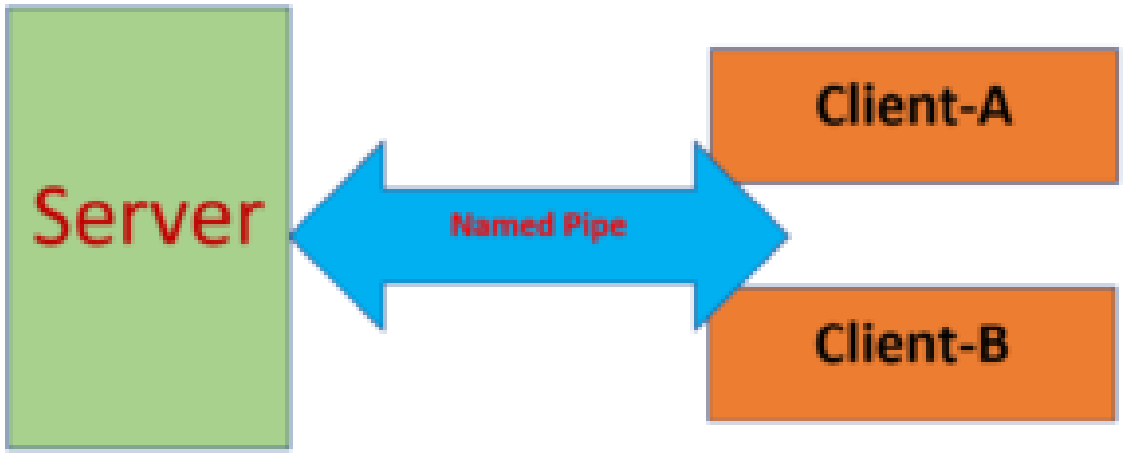
Microsoft SQL Server Architecture
Protocol Layer

- Shared memory
  - Is the default protocol used to connect from a client running on the same computer as Microsoft SQL Server.
  - Features:
    - Can only be used locally
    - Has no configurable properties
    - Is always tried first when connecting from the local machine



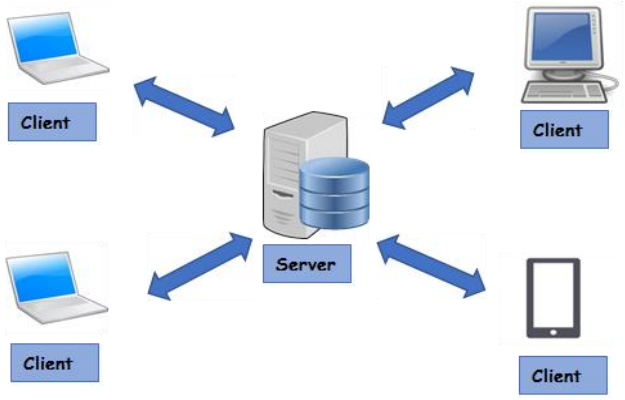
Microsoft SQL Server Architecture
Protocol Layer

- Named Pipes
  - TCP/IP and Named Pipes are comparable protocols in the architectures in which they can be used
  - Are used for local area networks (LANs)

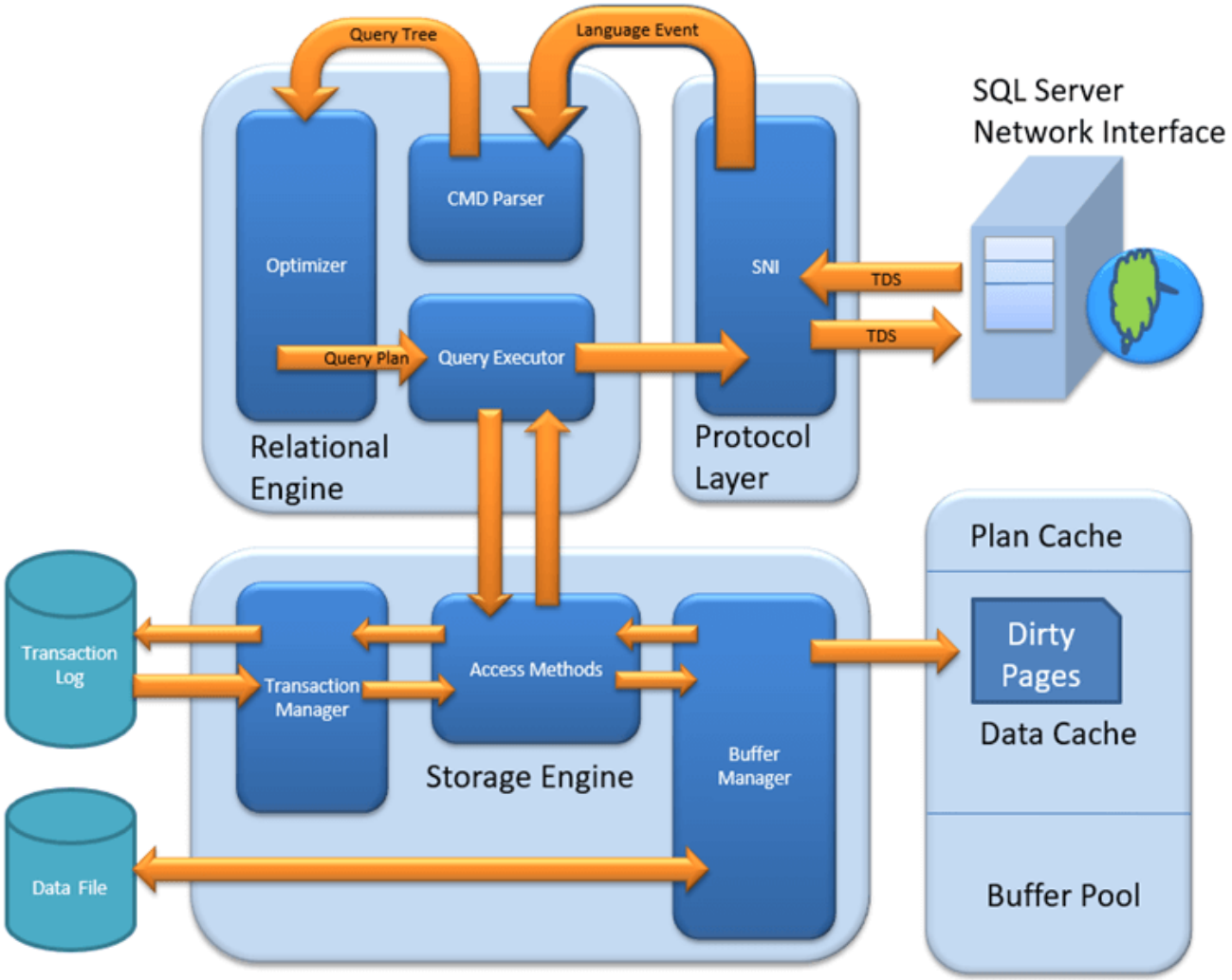


Microsoft SQL Server Architecture
Protocol Layer

- TCP/IP
  - The most used access protocol for MS SQL Server
  - Enables connection to MS SQL Server by specifying an IP address and a port number
  - The connection happens automatically when the instance to connect to is specify
  - The internal name resolution system resolves the hostname part of the instance name and gets the associated IP address
  - The connect either uses
    - The default TCP port number 1433 for default instances or
    - The SQL Browser service will find the right port for a named instance using UDP port 1434.

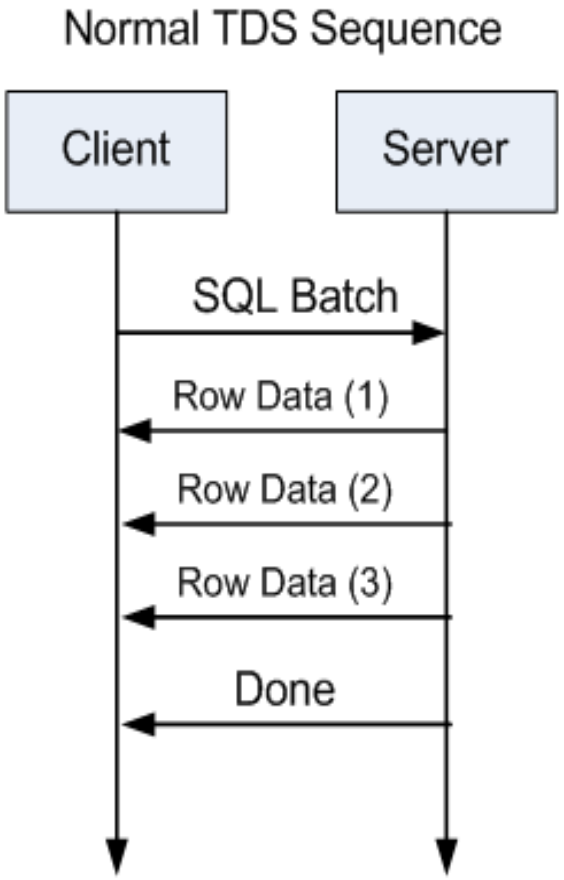


Microsoft SQL Server Architecture

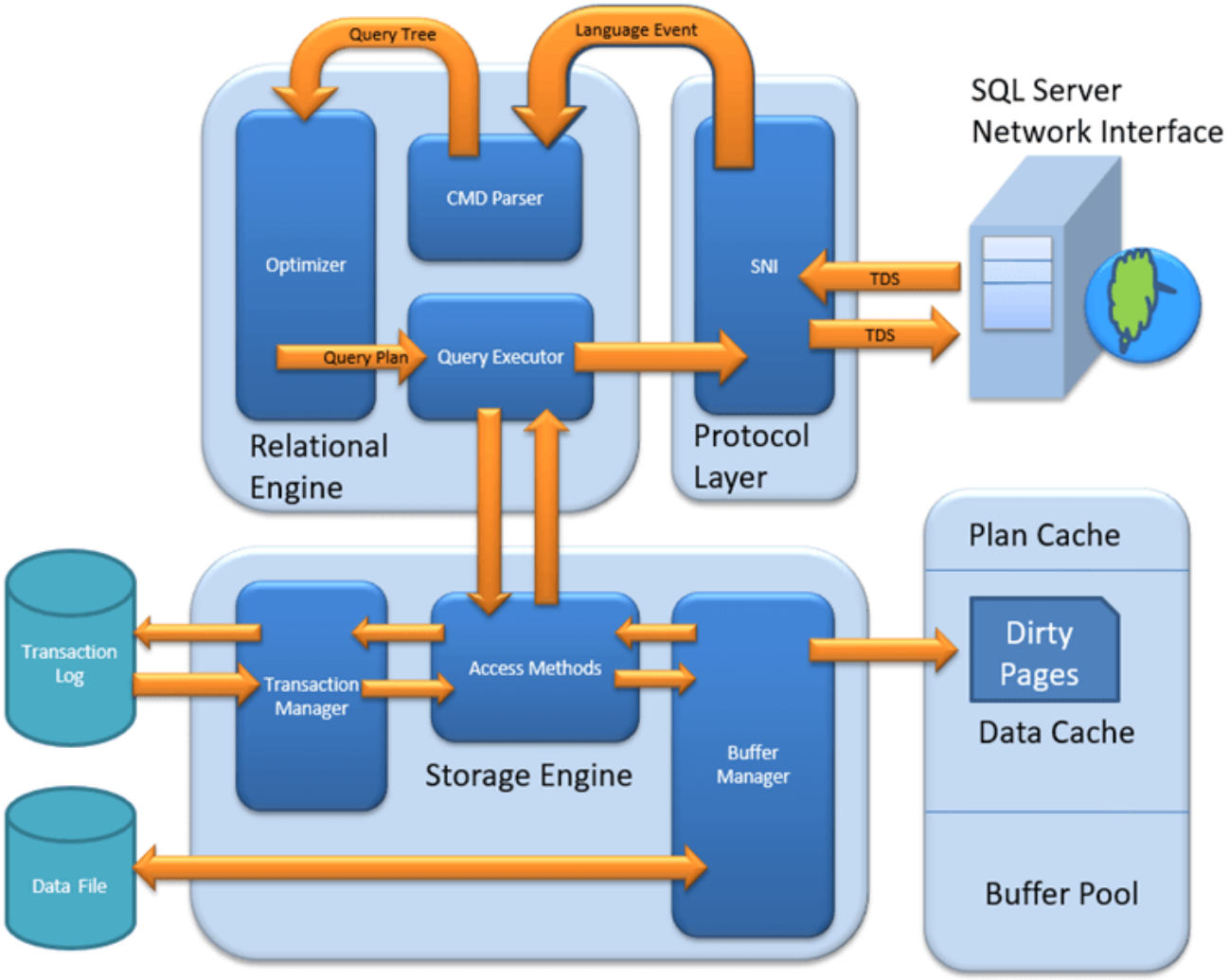


Microsoft SQL Server Architecture
Protocol Layer

- Tabular Data Stream (TDS) is a Microsoft proprietary protocol
- TDS were originally designed by Sybase
- TDS is used to interact with a database server
- All 3 protocols use TDS packets
- A TDS is encapsulated in Network packets
  - This enables data transfer from the client machine to the server machine
- Once a connection has been made using a network protocol (e.g., TCP/IP)
  - A link is established to the relevant TDS endpoint
  - TDS endpoint acts as the communication point between the client and the server



Microsoft SQL Server Architecture



Microsoft SQL Server Architecture
Relational Engine

- The Relational Engine
  - Is also sometimes called the query processor
  - Its primary function is query optimization and execution
  - It is responsible for the execution of user queries by
    - Requesting data from the storage engine
    - Processing the results that are returned

Microsoft SQL Server Architecture
Relational Engine

The main responsibilities of the relational engine are:

1. Parsing the SQL statements

- The parser scans an SQL statement
- Breaks down the SQL statement into
  - Logical units (i.e., keywords, parameters, operators, and identifiers)
  - A series of smaller logical operations

2. Optimizing the execution plans

- There are many ways that the server could use data from the source tables to build the result set
- The query optimizer determines
  - What these various series of steps are
  - Estimates the cost of each series (primarily in terms of file I/O)
  - Chooses the series of steps that has the lowest cost
  - Combines the specific steps with the **query tree** to produce an optimized execution plan

Microsoft SQL Server Architecture
Relational Engine

The main responsibilities of the relational engine are:

3. Executing the series of logical operations defined in the execution plan

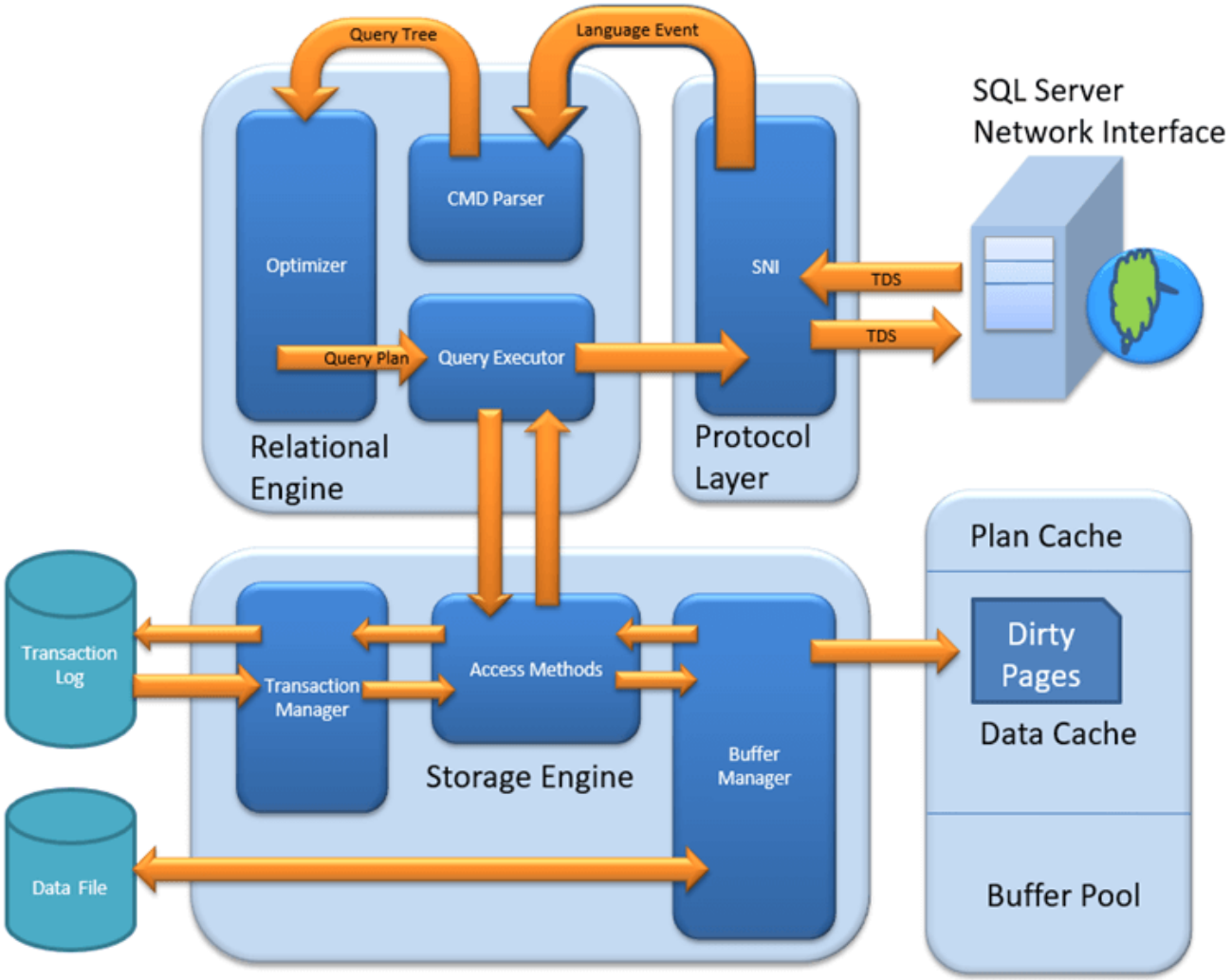
- After the query optimizer has defined the logical operations required to complete a statement, the relational engine steps through these operations in the sequence specified in the optimized execution plan

4. Processing Data Definition Language (DDL) and other statements.

5. Formatting results.

- The relational engine formats the results returned to the client
- The results are formatted as either
  - a traditional tabular result set
  - an XML document
- The results are then encapsulated in one or more TDS packets and returned to the application

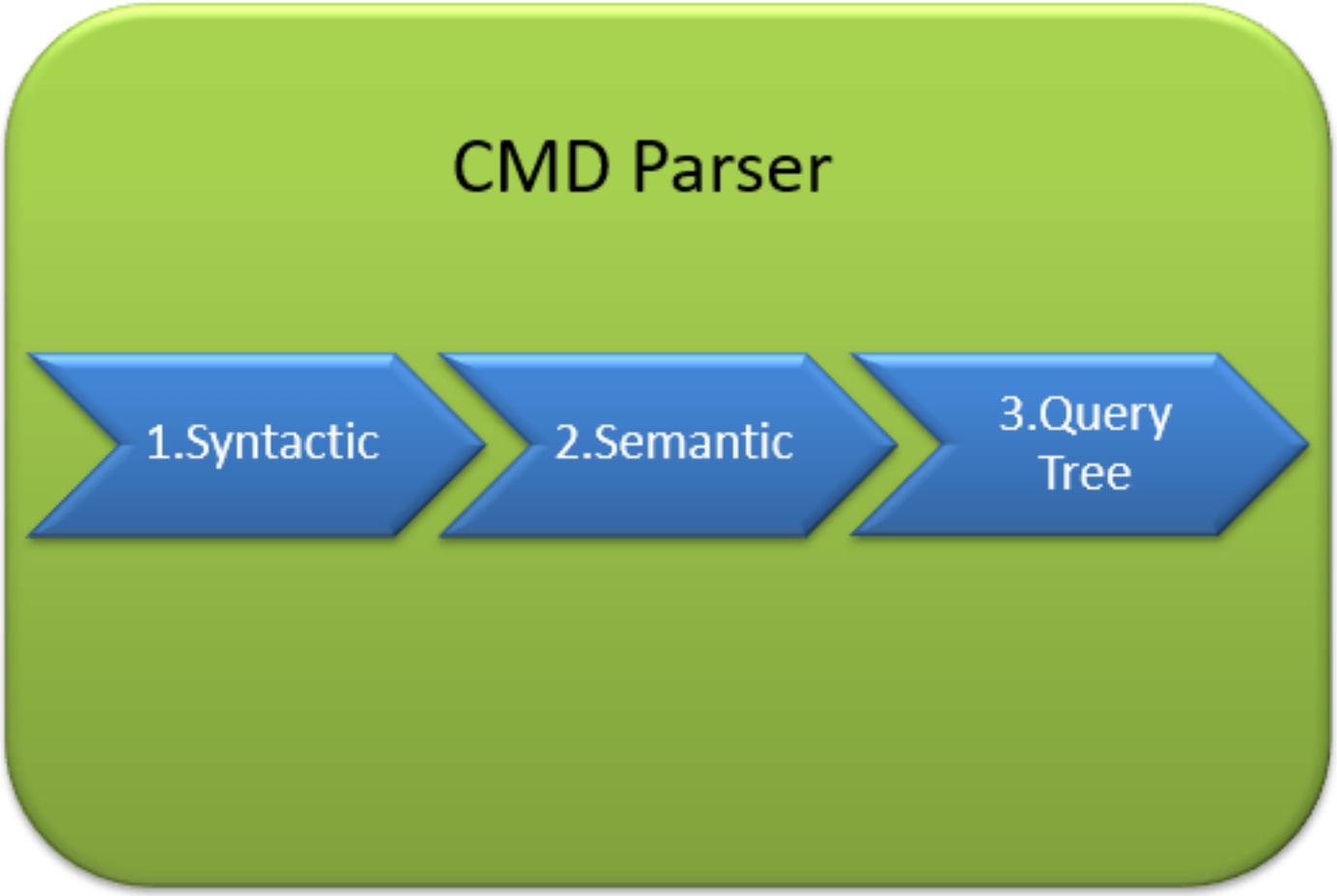
Microsoft SQL Server Architecture



Microsoft SQL Server Architecture
Relational Engine

- The Relational Engine contains below components:
  - **CMD Parser**
  - **Optimizer**
  - **Query Executor**

Microsoft SQL Server Architecture
Relational Engine

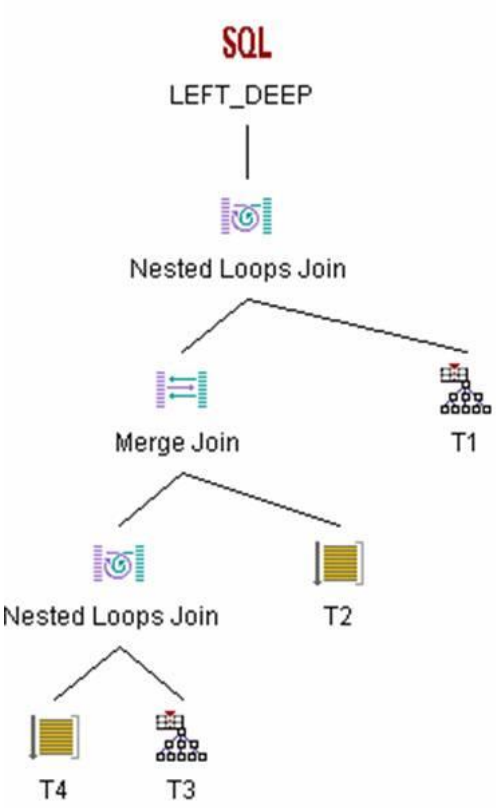


Microsoft SQL Server Architecture
Relational Engine

- **CMD Parser** (The Command Parser)
  - Handles SQL and Transact-SQL language
  - Steps:
    1. Checks the **syntactics and semantics of a query** and returns any errors back to the protocol layer to send to the client
    2. If the **syntax and semantics are valid**, then the next step is to generate a **query plan (query tree) or find an existing plan**.
    3. A **query plan** contains the details about how SQL Server is going to execute a piece of code (**execution plan**)
       - To check for a query plan, the Command Parser generates a **hash** of the SQL or Transact-SQL query
       - Checks the hash against the **plan cache** to determine whether a suitable plan already exists
    4. The **plan cache** is an area in the buffer pool used to cache query plans
       - If it finds a match, then the plan is read from cache and passed on to the Query Executor for execution
       - Otherwise, an execution plan is created by the optimizer

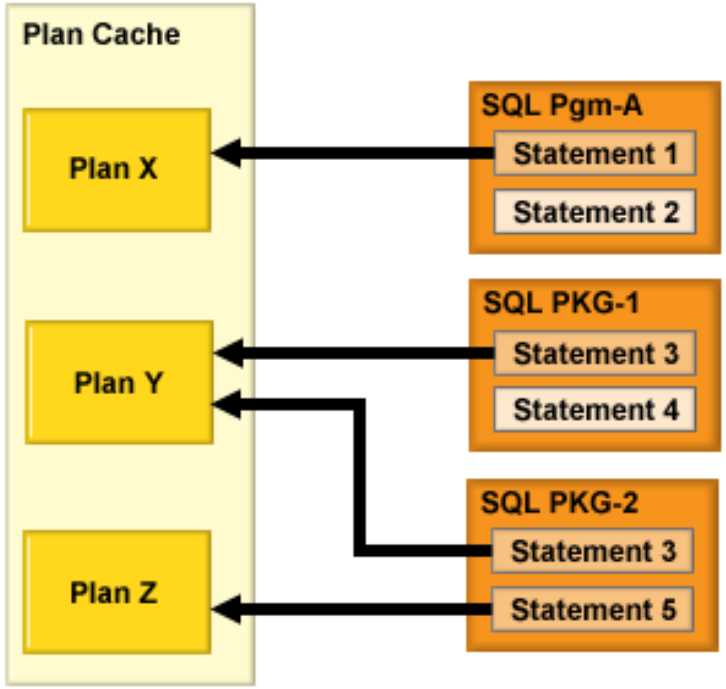
Microsoft SQL Server Architecture
Relational Engine

- **Execution Plan**
  - Is composed of primitive operations, some examples:
    - Reading a table completely
    - Using an index
    - Performing a nested loop or a hash join
  - All primitive operations
    - Have an output, i.e., their result set.
    - Some have one input, e.g., the nested loop
    - Other has two inputs, e.g., the hash join
  - Each input should be connected to the output of another primitive operation
  - **The execution plan can be sketched as a tree, i.e., information flows from leaves to the root.**



Microsoft SQL Server Architecture
Relational Engine

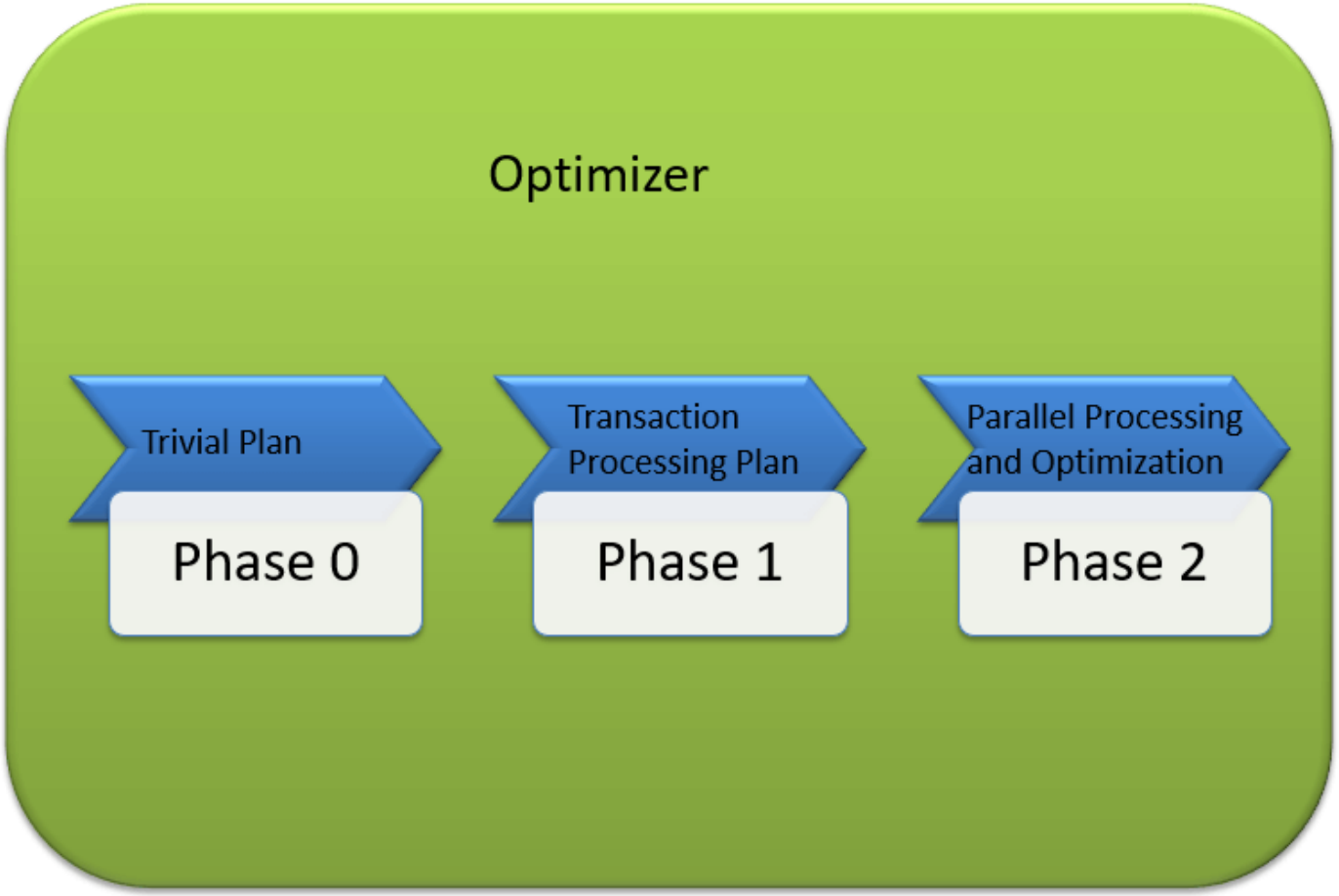
- **Plan Cache**
  - Is the part of Microsoft SQL Server's buffer pool
  - Is used to store execution plans in case they are needed later when the same type of scripts are submitted by the users.



Microsoft SQL Server Architecture
Relational Engine

- **Optimizer**
  - Is one of the important assets of a database engine
  - Is the component on which a particular RDBMS stands off
  - Its primary function is to generate execution plan
  - Note that not all queries are optimized
  - Optimization is done for DML commands
    - DDL commands are not optimized, but they are instead compiled into an internal form
  - The query cost is calculated based on factors like CPU usage, Memory usage, and I/O needs
  - Its role is to find the **cheapest, not the best, cost-effective execution plan that minimizes the query run time**
  - **Works on inbuilt exhaustive/heuristic algorithms (Microsoft propriety)**

Microsoft SQL Server Architecture
Relational Engine



Microsoft SQL Server Architecture
Relational Engine

- **Optimizer Phase 0: Search for Trivial Plan**
  - This is also known as Pre-optimization stage
  - For some cases, there could be only one practical, workable plan, known as a trivial plan
  - In this case there is no need for creating an optimized plan
    - Searching for more plans would result
      - In finding the same run time execution plan
      - Lowers performance
  - If no Trivial plan found, then Phase 1 starts.

Microsoft SQL Server Architecture
Relational Engine

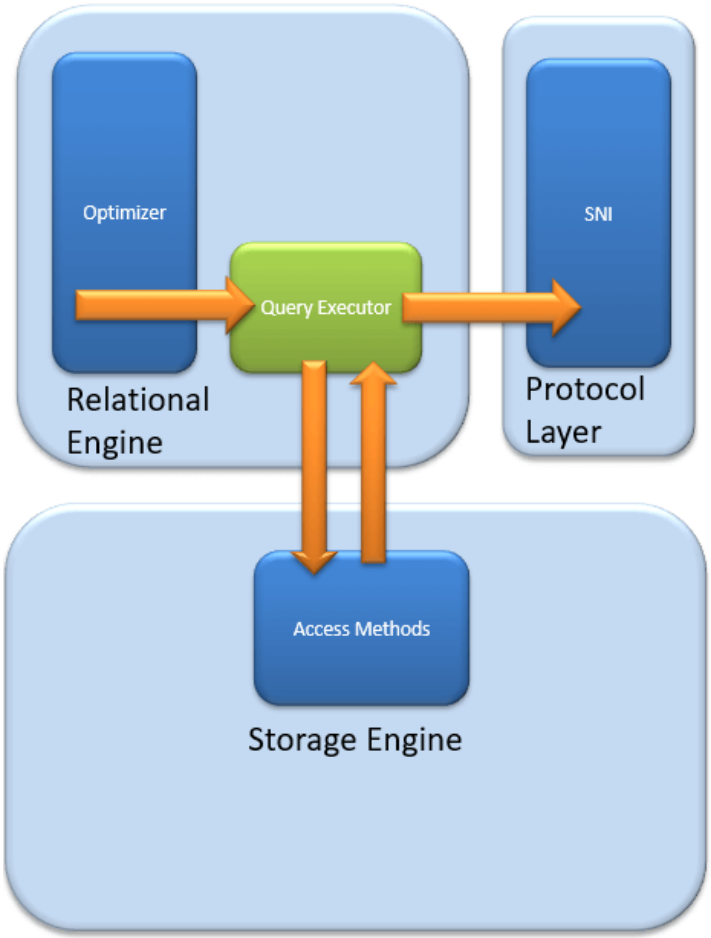
- **Optimizer Phase 1: Search for Transaction processing plans**
  - This includes the search for Simple and Complex Plans
  - Simple Plan
    - Analyses the past data of columns and indexes used in the query
    - Consists but is not restricted to analyze one index per table
  - Complex Plan involves analyzing multiple indexes per table, besides the use of past data

Microsoft SQL Server Architecture
Relational Engine

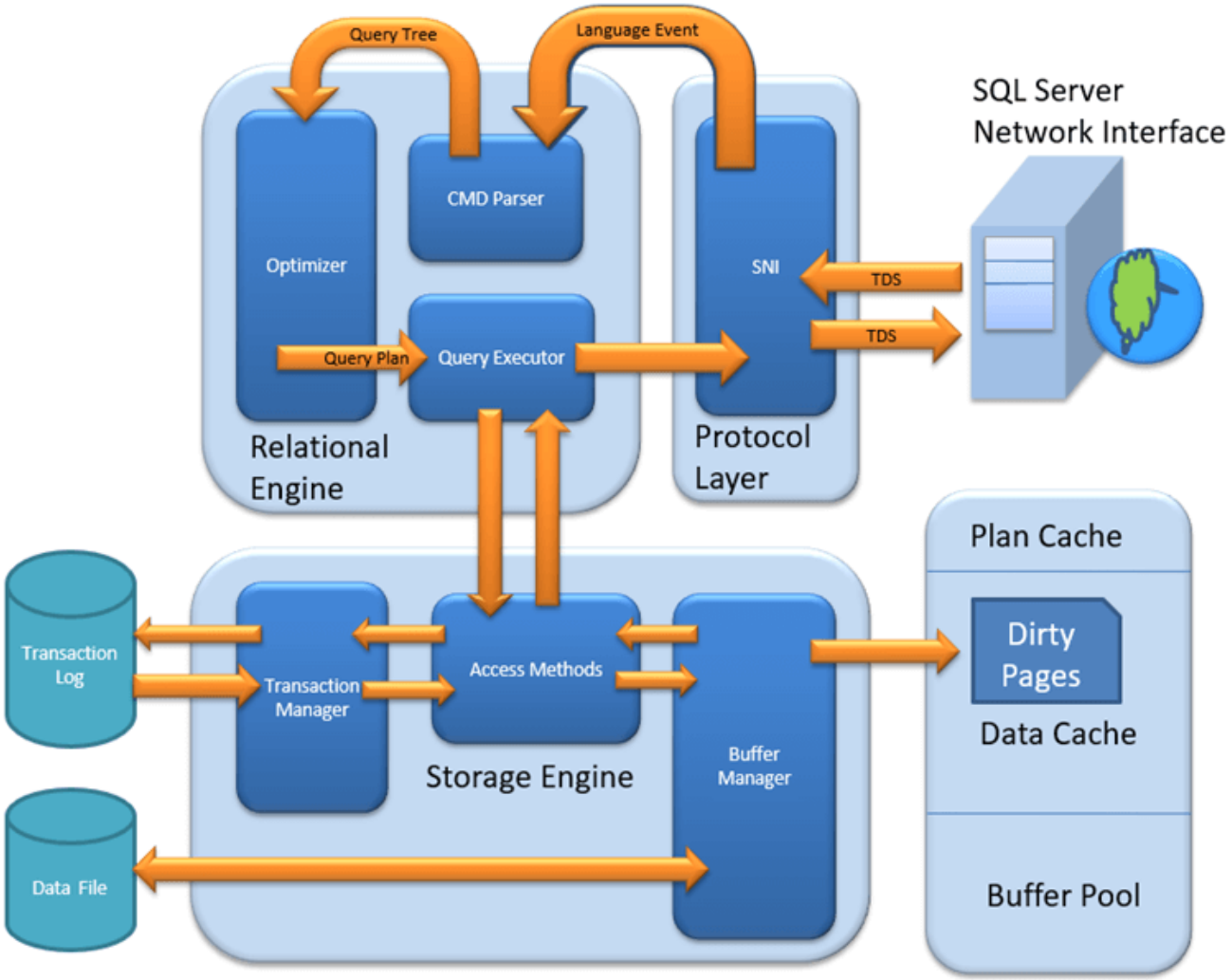
- **Optimizer Phase 2: Parallel Processing and Optimization**
  - If none of the above strategies work, the Optimizer searches for Parallel Processing possibilities
  - This depends on the Machine's processing capabilities and configuration
  - If that is still not possible, then the final optimization phase starts:
    - Finding all other possible options for executing the query in the best way

Microsoft SQL Server Architecture
Relational Engine

- **Query Executor**
  - Executes the query
    - More specific, it executes the query plan by working through each step it contains and interacting with the Storage Engine to retrieve or modify data
  - Calls the **Access Method**
  - Provides the execution plan for the logical requirement of fetching data
    - Once data is received from Storage Engine, the result gets published to the Protocol layer
    - Finally, data is sent to the end user using TDS



Microsoft SQL Server Architecture



Microsoft SQL Server Architecture
Storage Engine

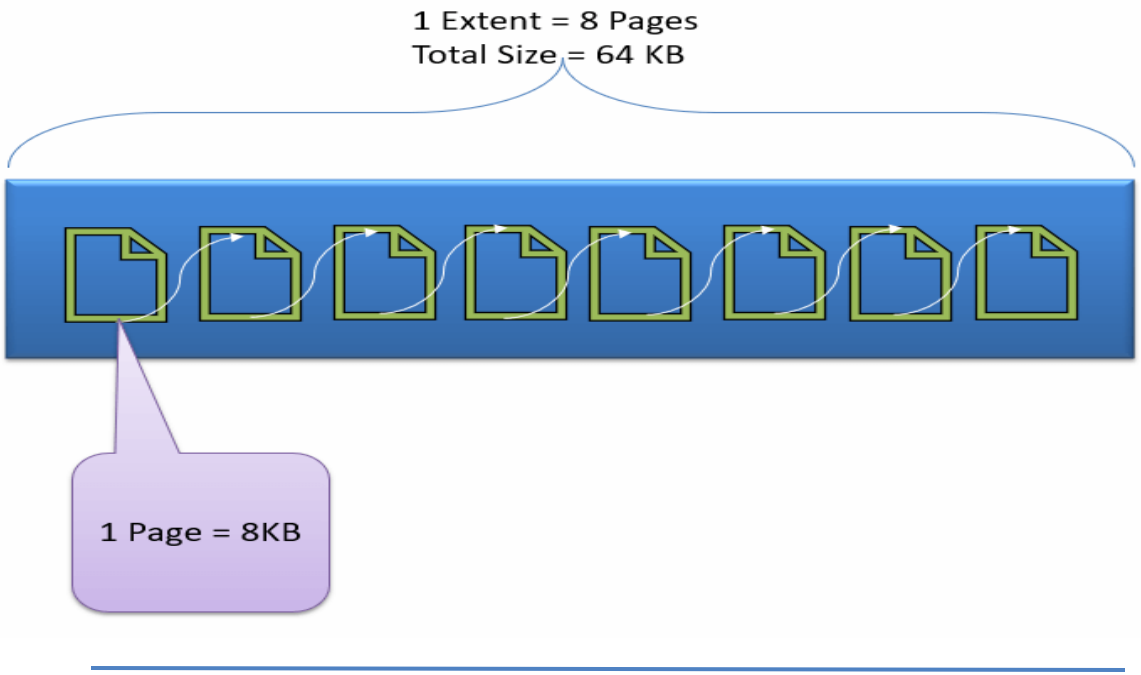
- The **Storage Engine** is responsible for managing all I/O to the data

- The main responsibilities of the storage engine include:
  - **Managing the files** on which the database is stored and managing the use of space in the files
  - **Building and reading the physical pages** used to store data
  - **Managing the data buffers** and **all I/O to the physical files**.
  - **Controlling concurrency**, i.e., managing transactions and using locking to control concurrent user access to rows in the database
  - **Logging and recovery**
  - Implementing utility **functions** such as the BACKUP, RESTORE, and DBCC statements, and bulk copy

Microsoft SQL Server Architecture
Storage Engine

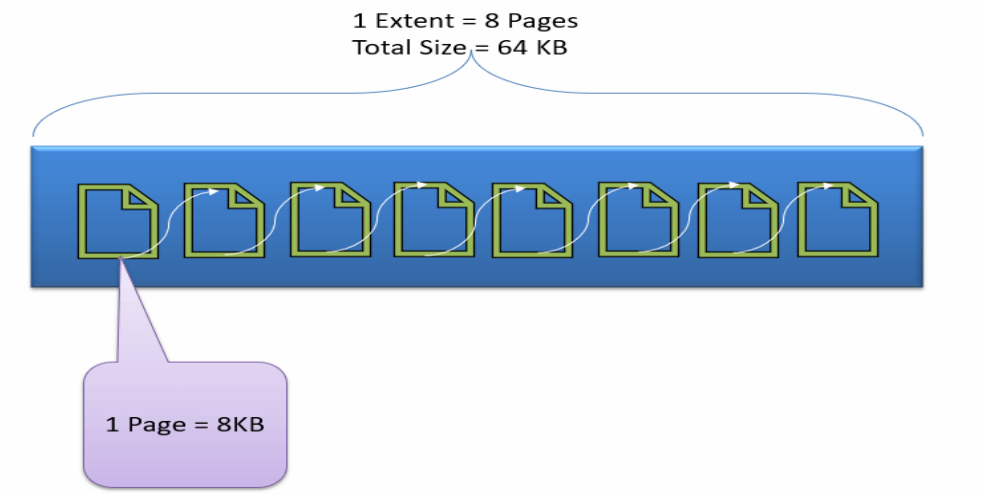
- The files in the storage engine are stored on Disk or SAN (Storage Area Network)

- The data is stored inside the database using Data Files



Microsoft SQL Server Architecture
Storage Engine

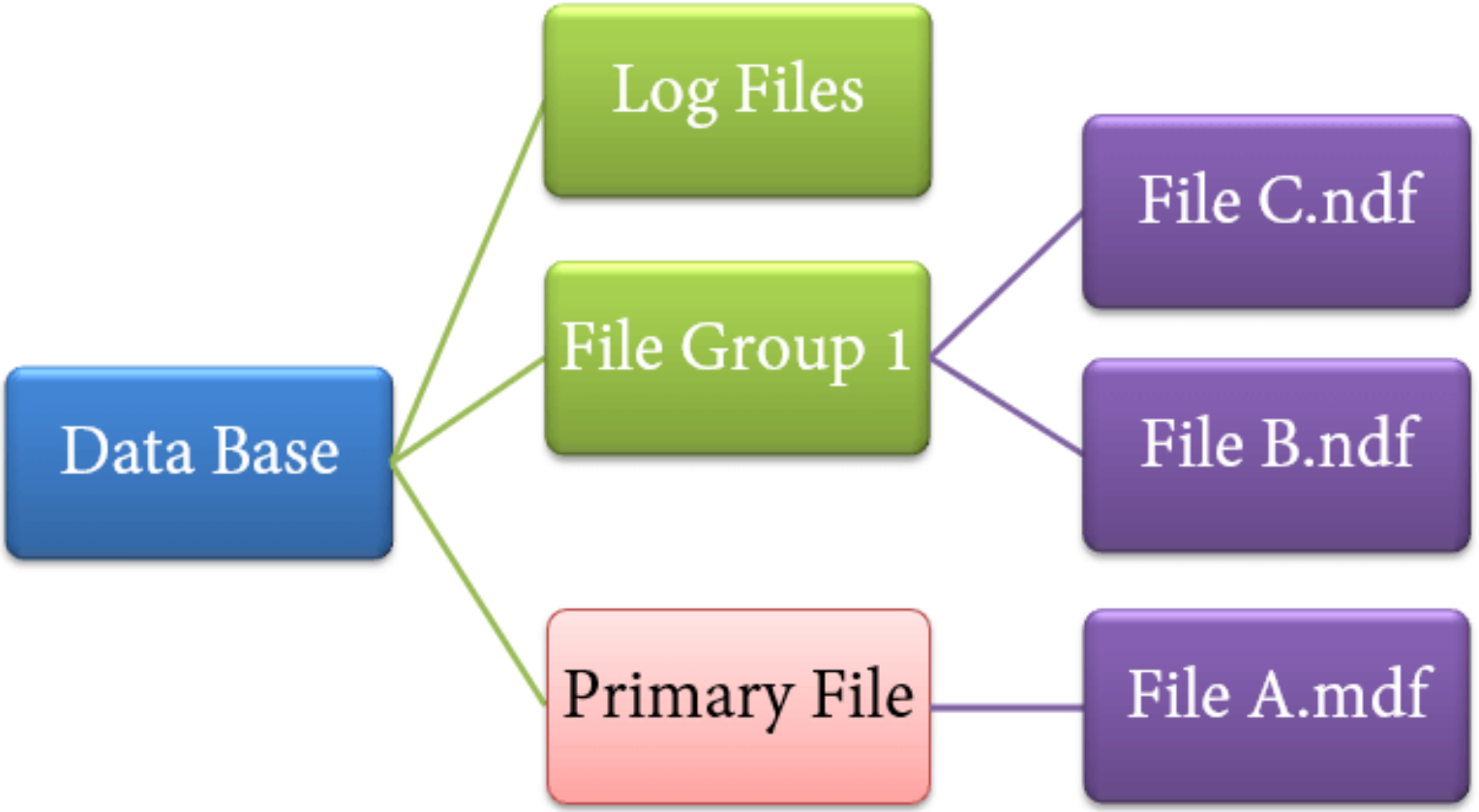
- Data File physically stores data in the form of data pages
- Each data page having a size of 8KB (forming the smallest storage unit in SQL Server)
- Consecutive data pages are logically grouped to form extents
- An extent is equal to 8 physically contiguous pages
- No object is assigned a page in SQL Server



Microsoft SQL Server Architecture
Storage Engine

- The maintenance of the object is done via extents
- The page has a section called the Page Header with a size of 96 bytes
- The Page Header stores metadata information about the page:
  - Page Type
  - Page Number
  - Size of Used Space
  - Size of Free Space
  - Pointer to the next page and previous page

Microsoft SQL Server Architecture
Storage Engine



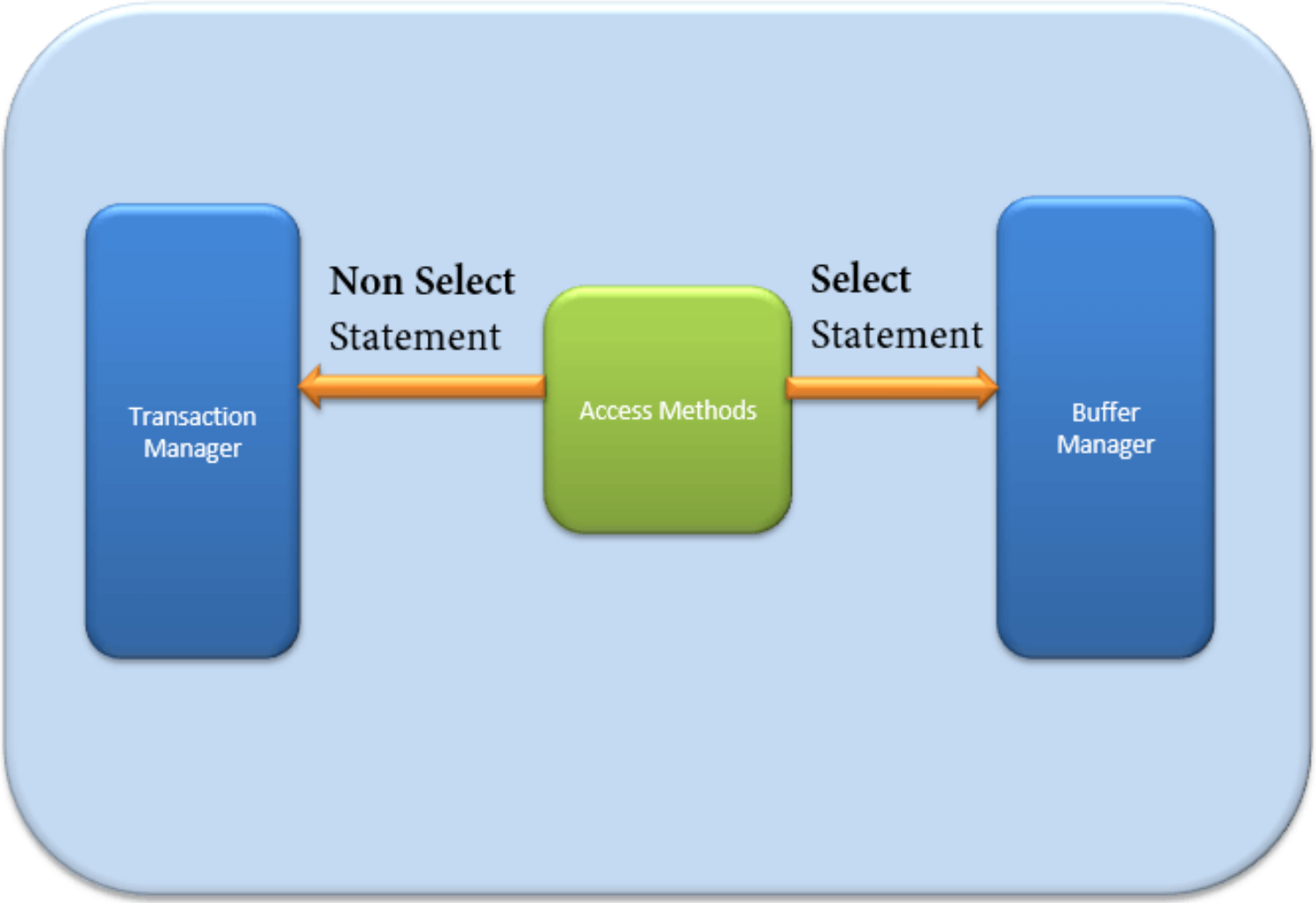
Microsoft SQL Server Architecture
Storage Engine

- Primary file
  - Every database contains one Primary file
  - This store all important data related to tables, views, Triggers, etc.
  - Extension is .mdf usually but can be of any extension
- Secondary file
  - A Database may or may not contains multiple Secondary files
  - This is optional and contains user-specific data
  - Extension is .ndf usually but can be of any extension
- Log file
  - Also known as Write Ahead Logs (WAL)
  - Extension is .ldf
  - Used for Transaction Management
    - To recover from any unwanted instances
    - To perform Rollback to uncommitted transactions

Microsoft SQL Server Architecture
Storage Engine

- The Storage Engine contains below components:
  - **Access Methods module**
  - **Buffer Manager**
  - **Transaction Manager**

Microsoft SQL Server Architecture
Storage Engine



Microsoft SQL Server Architecture
Storage Engine

- Access Methods act as an interface between query executor and Buffer Manager/Transaction Logs

- Access Methods provide
  - The storage structures for your data and indexes
  - The interface through which data is retrieved and modified

- Access Methods
  - Contain all the mechanisms and techniques to retrieve data
  - Does not perform the operation itself
  - The operation is passed as a request to
    - The Buffer Manager if the operation is a Select Statement
    - The Transaction Manager if the operation is not a Select Statement

Microsoft SQL Server Architecture
Storage Engine

- Buffer Manager consists of two mechanisms:
  - The Buffer Manager to access and update database pages
  - The Buffer Cache (also called the Buffer Pool), to reduce database file I/O

- The Buffer Manager manages the Buffer Pool (the majority of SQL Server's memory usage)
  - The Buffer Pool consist of various type of cache (e.g., data cache, plan cache, log cache, etc.)

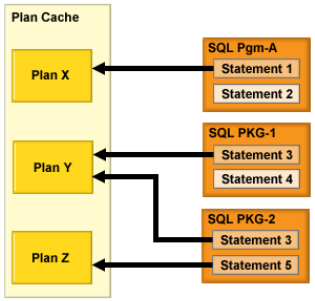
- Buffer Manager manages the core functions for:
  - Plan Cache
  - Data Cache
  - Dirty Page

Microsoft SQL Server Architecture
Storage Engine

- **Plan Cache** is used to store the query plan for a SQL query

- **Existing Query plan:**
  - The buffer manager checks if the execution plan is stored Plan Cache
  - If **Yes**, then query plan cache and its associated data cache is used

- **First time Cache plan:**
  - The buffer manager checks if the execution plan is stored Plan Cache
  - If **No,** then the first-time query execution plan is being run
  - After the run, the query plan store in the Plane Cache to ensure faster availability when the next time SQL server gets the same query
  - Therefore, it's nothing else but the query itself which plan execution is being stored if it is being run for the first time



Microsoft SQL Server Architecture
Storage Engine

- The **data cache** is the largest part of the Buffer Pool, thus, the largest memory consumer within SQL Server

- All the information read from disk is written in the data cache before use

- **Soft Parsing:**
  - When a query is issued, the Buffer Manager looks for Data in Buffer in Data Cache
  - If present, then this Data is used by the Query Executor
  - This improves the performance by reducing the number of I/O operation, i.e., faster to retrieve data from a cache as compared to fetching data from physical storage

- **Hard Parsing:**
  - When a query is issued and the data is not present in cache, then required Data is searched in Data Storage
  - After it is retrieved from the physical storage, the data is stored in the data cache for future use

Microsoft SQL Server Architecture
Storage Engine

- **Dirty pages**
  - When a page is read from disk into memory it is regarded as a clean page because it's the same as its counterpart on the disk
  - However, once the page has been modified in memory it is marked as a dirty page
  - A dirty page is simply a page that has changed in memory since it was loaded from disk and is now different from the on-disk page

Microsoft SQL Server Architecture
Storage Engine

- **Transaction Manager** interacts with the Access Methods and has two components
  - Lock Manager:
    - Responsible for providing concurrency to the data
    - Delivers the configured level of isolation by using locks
    - This process ensures **data consistency and isolation**
  - Log Manager writes the changes to the transaction log:
    - Keeps a track of all updates done in the system via logs in Transaction Logs
    - Logs have **Logs Sequence Number with the Transaction ID and Data Modification Record**.
    - This is used for keeping track of **Transaction Committed and Transaction Rollback**

Microsoft SQL Server Architecture
Storage Engine

- Execution Process
  - Log Manager start logging and Lock Manager locks the associated data.
  - A copy of the data is maintained in the Buffer cache
  - A copy of data supposed to be updated is maintained in Log buffer
  - Modification are applied on the data stored in Data buffer resulting in Dirty Pages.
  - Checkpoint and Write-Ahead Logging:
    - This process writes all the modified page (Dirty Pages) from memory to Disk, while keeping them in the cache
    - A page is first pushed to the data page of the log file from Buffer log before becoming persistent on the disk
    - This is known as Write Ahead Logging
    - Frequency is approximately 1 run per minute
  - Lazy Writer:
    - The Dirty page can remain in memory
    - When SQL server observes a huge load and Buffer memory is needed for a new transaction, it frees up Dirty Pages from the cache
    - It operates on LRU (Least Recently Used) Algorithm for cleaning page from buffer pool to disk

Overview

- Microsoft SQL Server Overview
- Microsoft SQL Server Architecture
- Microsoft SQL Server Big Data Cluster

SQL Server Big Data Cluster

- SQL Server Big Data Cluster enabled with the release of Microsoft SQL Server 2019

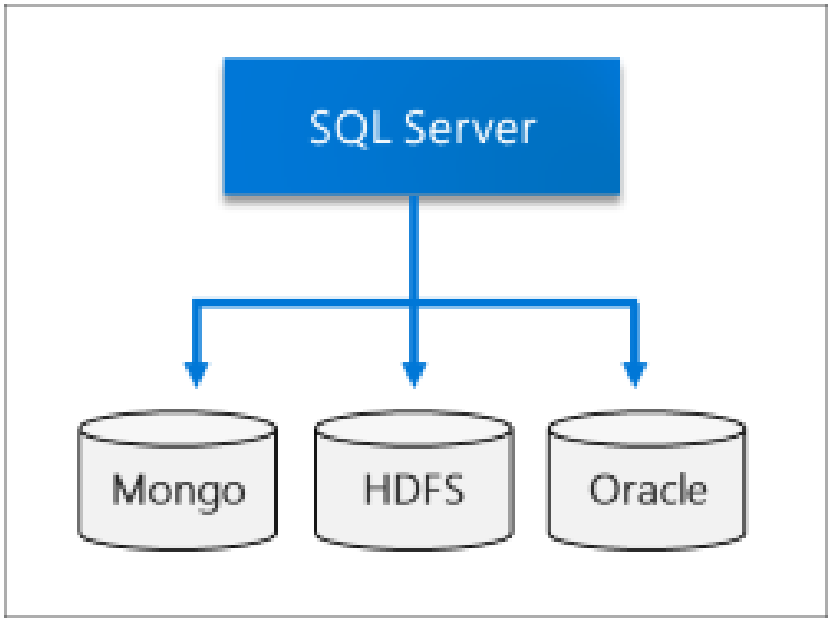
- SQL Server Big Data Cluster offers:
  - Data Virtualization
  - Data Lake
  - Scale out Data Mart
  - Integrated Artificial Intelligence and Machine Learning Tools
  - Management and Monitoring

- A SQL Server big data cluster is a cluster of Linux containers orchestrated by Kubernetes.

SQL Server Big Data Cluster
Data Virtualization

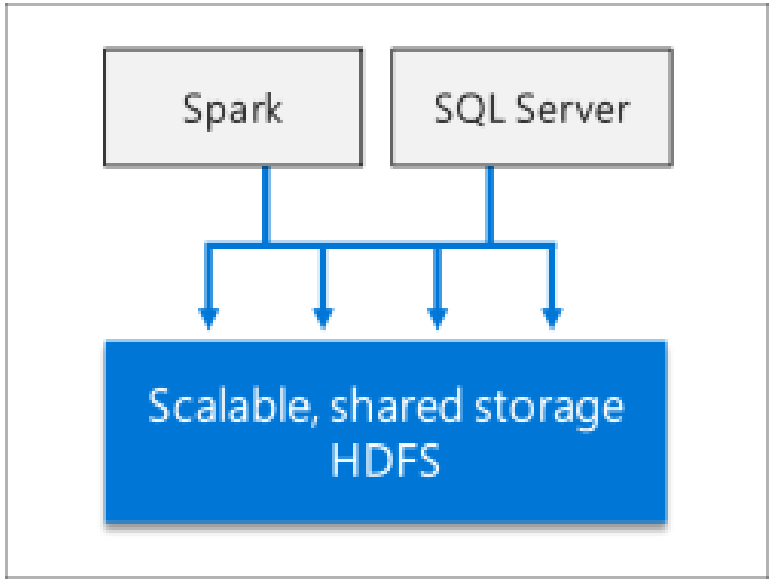
- SQL Server Big Data Clusters can query external data sources without moving or copying the data

- SQL Server 2019 (15.x) introduces new connectors to data sources



SQL Server Big Data Cluster
Data Lake

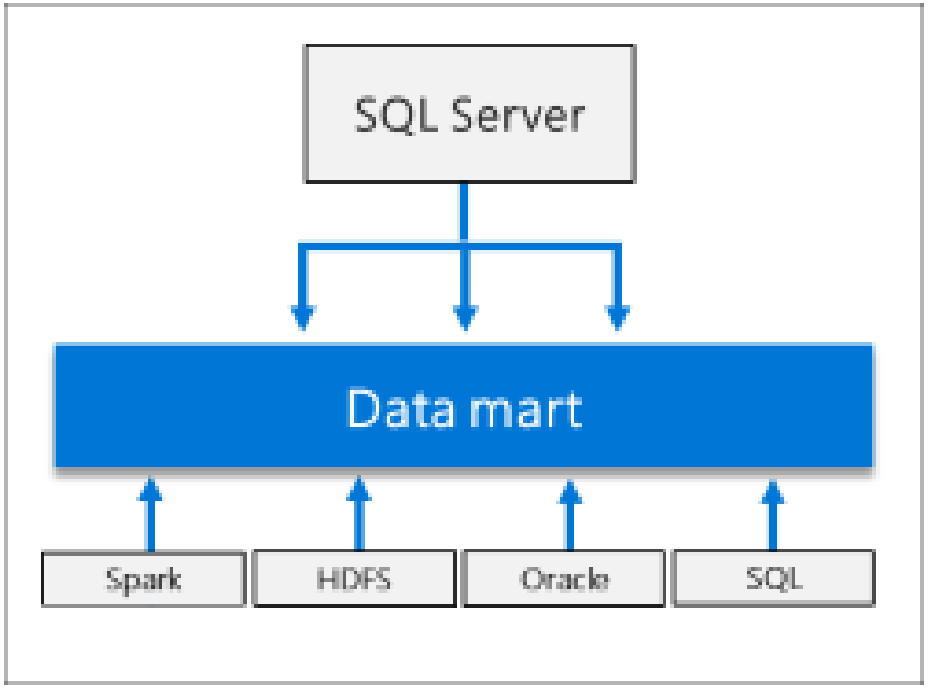
- SQL Server Big Data Cluster includes a scalable HDFS *storage pool*
- The HDFS *storage pool* can be used to store big data (potentially ingested from multiple external sources)
- Once the data is stored in HDFS in the big data cluster, it can be
  - Analyzed
  - Queried
  - Combined with relational data.



SQL Server Big Data Cluster
Scale-out Data Mart

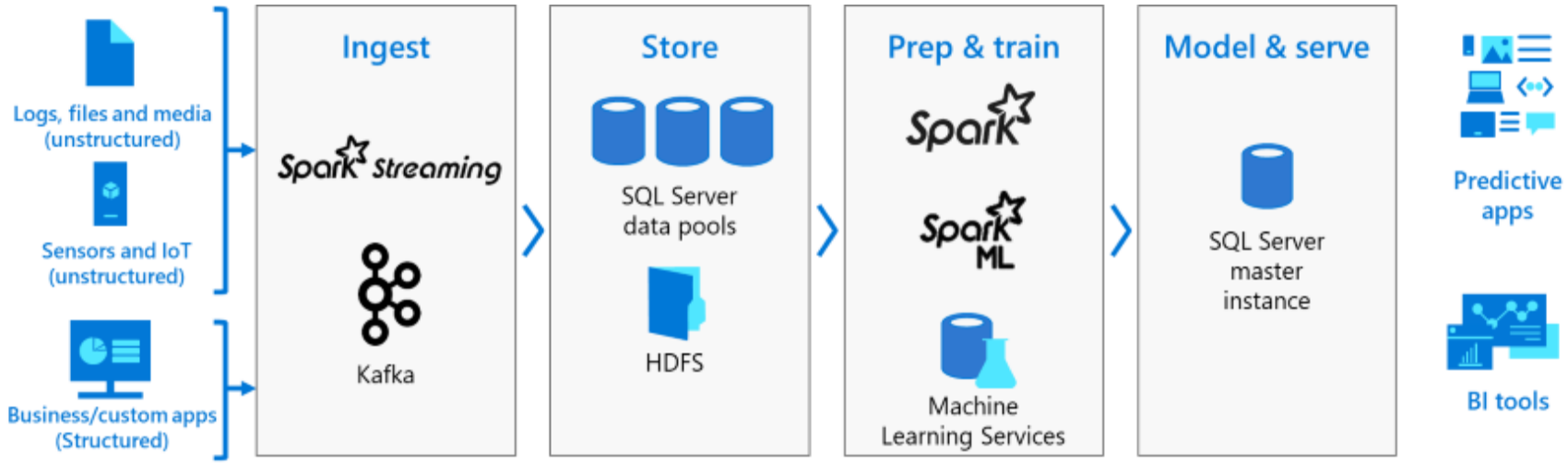
- SQL Server Big Data Clusters provide scale-out compute and storage to improve the performance of analyzing any data

- Data from a variety of sources can be ingested and distributed across *data pool* nodes as a cache for further analysis



Big Data Cluster
Integrated AI and ML Tools

- SQL Server Big Data Clusters enable AI and ML tasks on the data stored in HDFS storage pools and the data pools:
  - Spark
  - Built-in AI tools in SQL Server, using R, Python, Scala, or Java.

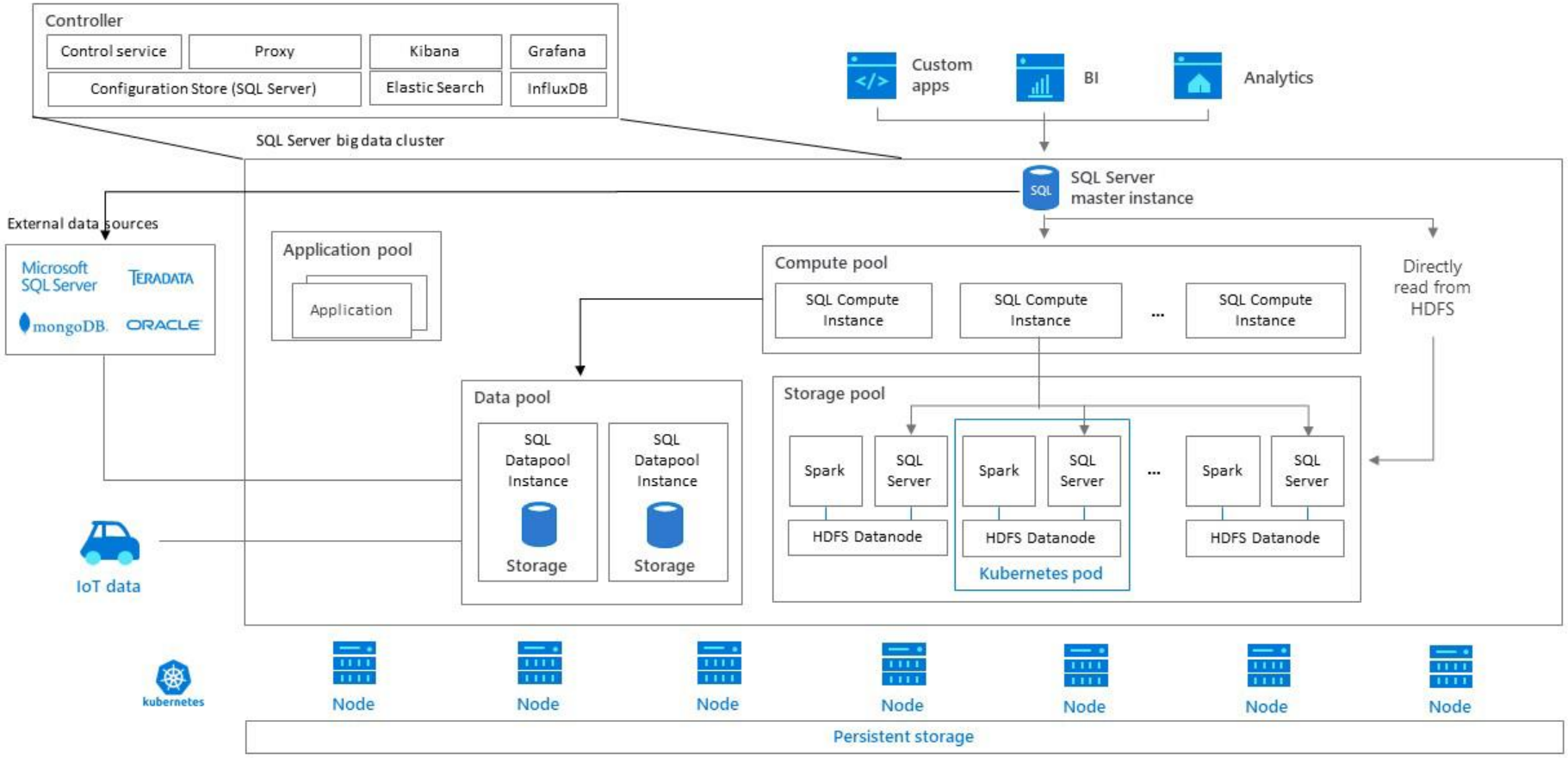


SQL Server Big Data Cluster Management and Monitoring

- Management and monitoring are provided through:
  - Command line tools
  - APIs
  - Portals
  - Dynamic management views

- Azure Data Studio can be used to perform a variety of tasks on the cluster:
  - Built-in tools for common management tasks.
  - Ability to browse HDFS, upload files, preview files, and create directories
  - Ability to create, open, and run Jupyter-compatible notebooks
  - Data virtualization wizard to simplify the creation of external data sources (enabled by the Data Virtualization Extension)

SQL Server Big Data Cluster



SQL Server Big Data Cluster

- Controller
  - Provides management and security for the cluster
  - Contains
    - The control service
    - The configuration store
    - Other cluster-level services such as Kibana, Grafana, and Elastic Search.

- Compute pool
  - Provides computational resources to the cluster
  - Contains nodes running SQL Server on Linux pods
  - The pods in the compute pool are divided into SQL Compute instances for specific processing tasks

SQL Server Big Data Cluster

- Data pool
  - Is used for data persistence and caching
  - Consists of one or more pods running SQL Server on Linux
  - Is used to ingest data from SQL queries or Spark jobs
  - SQL Server big data cluster data marts are persisted in the data pool
- Storage pool
  - Consists of storage pool pods comprised of SQL Server on Linux, Spark, and HDFS
  - All the storage nodes in a SQL Server big data cluster are members of an HDFS cluster
  - Kubernetes can be used to create Storage pods

SQL Server Big Data Cluster

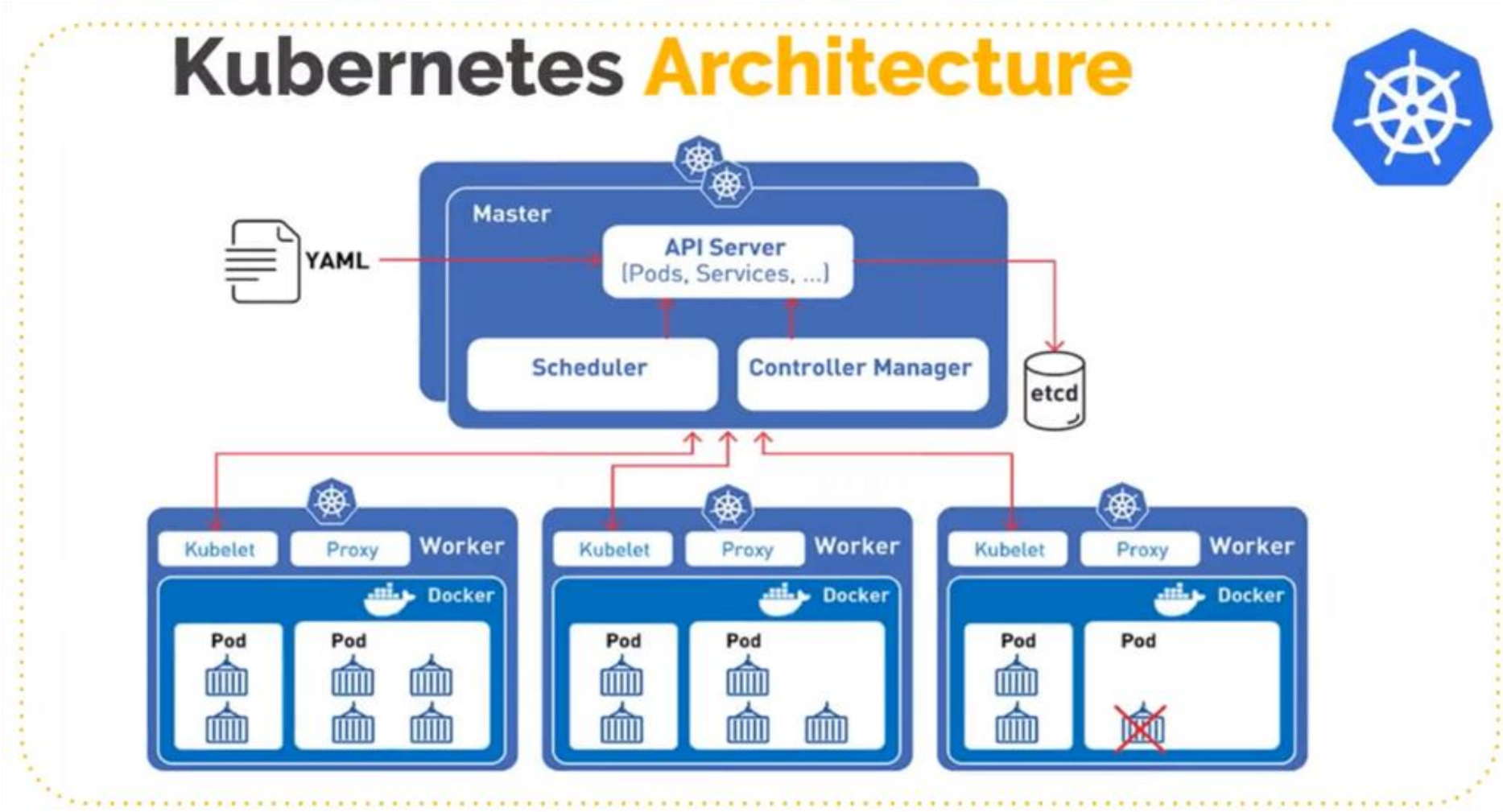
- Kubernetes is an open-source container orchestrator
- Kubernetes scales container deployments according to needs
- Kubernetes Cluster is a set of machines (nodes)
  - 1 node is the Master node
    - Controls the cluster
    - Distributes the work between the workers
    - Monitors the health of the cluster
  - The remaining nodes are the workers

SQL Server Big Data Cluster

- Kubernetes Node
  - Runs containerized applications
  - Can be either a physical machine (physical node) or a virtual machine (virtual node)
  - A Kubernetes cluster contains a mixture between physical and virtual nodes

- Kubernetes Pod
  - Is the atomic deployment unit of Kubernetes
  - Is a logical group of one or more containers and associated resources needed to run an application
  - Each pod runs on a node
  - A node can run one or more pods
  - The Kubernetes master automatically assigns pods to nodes in the cluster

SQL Server Big Data Cluster



References

- Microsoft SQL Server 2019 Technical Documentation
https://learn.microsoft.com/en-us/sql/sql-server/?view=sql-server-ver15

- Microsoft SQL Server 2022 Technical Documentation
https://learn.microsoft.com/en-us/sql/sql-server/?view=sql-server-ver16

University
Politehnica
of Bucharest

Microsoft SQL Server (II)

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@upb.ro

Overview

- SQL and Transact-SQL Language

Transactions

- A transaction is a single unit of work
- If a transaction is successful, all the data modifications made during the transaction are
  - Committed
  - Become a permanent part of the database
- If a transaction encounters errors, all the data modifications made during the transaction are
  - Erased
  - Must be cancelled or rolled back

Transactions

- Autocommit transactions
  - Are the default transaction management mode of the SQL Server Database Engine
  - In this case, each individual statement is a transaction
  - Every Transact-SQL statement is committed or rolled back when it completes
    - A transaction is committed if a statement completes successfully
    - A transaction is it is rolled back if it encounters any error

Transactions

- Explicit transactions
  - Are explicitly started with the BEGIN TRANSACTION statement
  - Are explicitly ended with a COMMIT TRANSACTION or ROLLBACK TRANSACTION statement
- Implicit transactions
  - A new transaction is implicitly started when the prior transaction completes
  - Are explicitly completed with a COMMIT or ROLLBACK statement

Transactions

```sql
BEGIN TRANSACTION
SELECT region_name AS [Before Update] FROM regions WHERE region_id = 3;
UPDATE regions SET region_name = 'Oceania' WHERE region_id = 3;
SELECT region_name AS [After Update] FROM regions WHERE region_id = 3;
COMMIT TRANSACTION
SELECT region_name AS [After Commit] FROM regions WHERE region_id = 3;
GO

BEGIN TRANSACTION
SELECT region_name AS [Before Update] FROM regions WHERE region_id = 3;
UPDATE regions SET region_name = 'Oceania' WHERE region_id = 3;
SELECT region_name AS [After Update] FROM regions WHERE region_id = 3;
ROLLBACK TRANSACTION
SELECT region_name AS [After Rollback] FROM regions WHERE region_id = 3;
GO
```

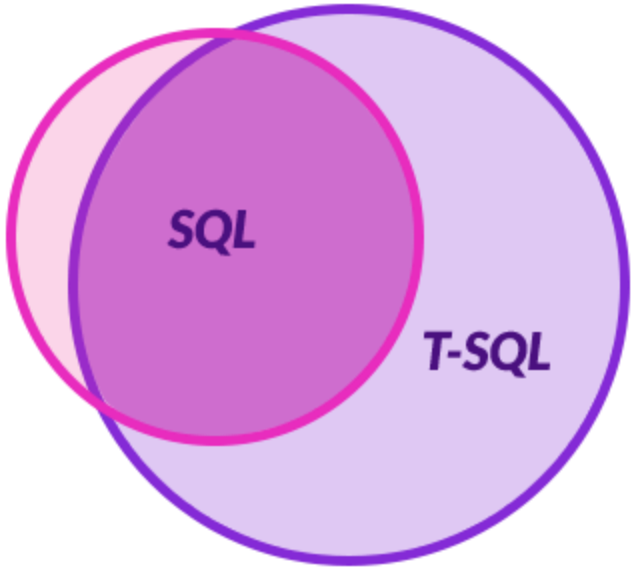
Transactions

- Batch-scoped transactions
  - Are applicable only to Multiple Active Result Sets (MARS)
    - MARS is a feature that allows the execution of multiple batches on a single connection
  - Are explicit or implicit transactions that start under a MARS session
  - If they are not committed or rolled back when a batch completes then they are automatically rolled back by SQL Server

Transact-SQL (T-SQL)

- Transact-SQL is a Microsoft SQL Server extension of SQL

- Transact-SQL contains:
  - Basic SQL clauses
  - Procedural programming abilities, (e.g., functions, procedures, triggers, exception handling, etc.)
  - Local variables



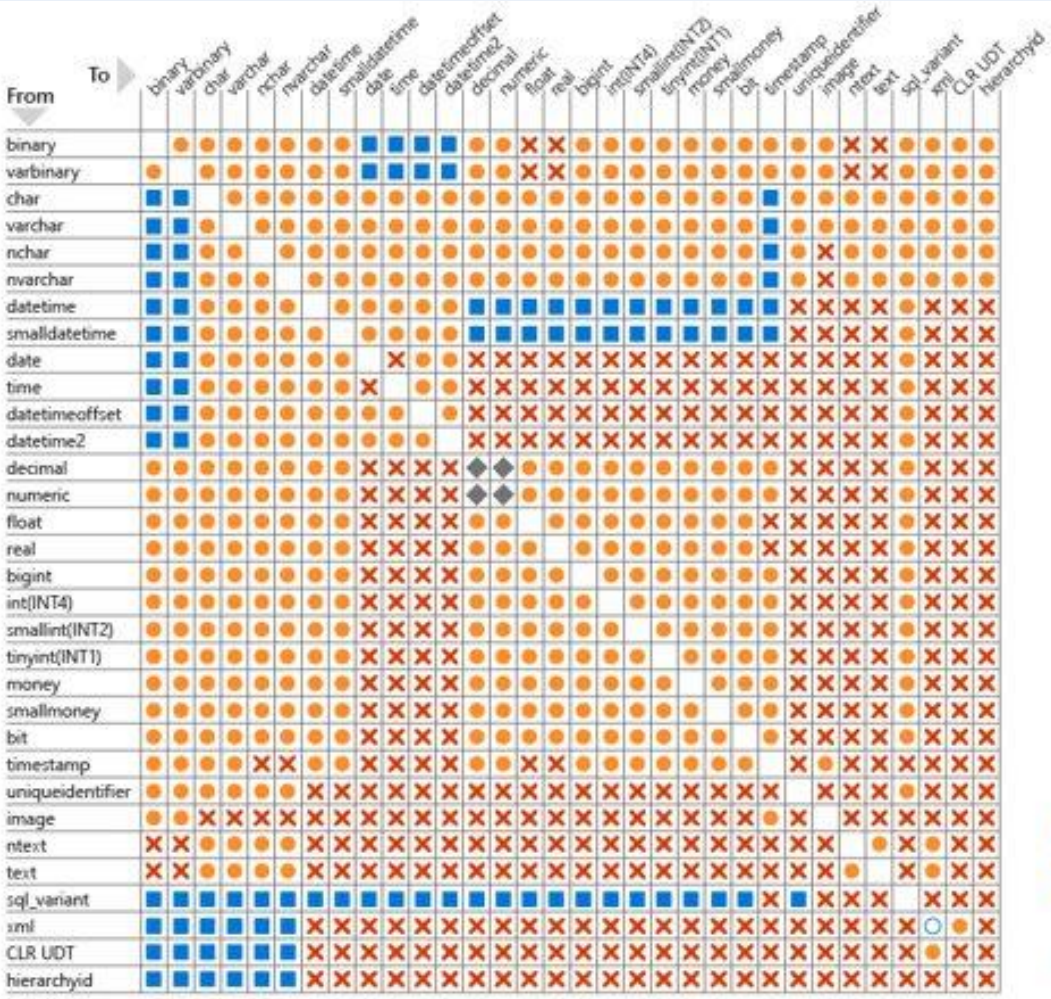
Data Types

- SQL Server supplies multiple system data types
  - Exact numeric: SMALLINT, INT, BIGINT, NUMERIC(p,s), and DECIMAL(p,s), etc.
  - Approximate numeric: FLOAT(p), REAL
  - Character strings: CHAR(p), VARCHAR(p), TEXT
  - Unicode character strings NCHAR(p), NVARCHAR(p), NTEXT
  - Binary strings: BINARY(p), VARBINARY(p), IMAGE
  - Date and time: TIME, DATE, DATETIME2

Data Types

- User defined types
  - Can be created using either T-SQL or the Microsoft .NET Framework
  - Obtain their characteristics from the class' methods and operators
    - The class is created using one of the programming languages support by the .NET Framework

Data Types


From
To
binary
varbinary
char
varchar
nchar
nvarchar
datetime
smalldatetime
date
time
datetimeoffset
datetime2
decimal
numeric
float
real
bigint
int(INT4)
smallint(INT2)
tinyint(INT1)
money
smallmoney
bit
timestamp
uniqueidentifier
image
ntext
text
sql_variant
xml
CLR UDT
hierarchyid
Explicit conversion
Implicit conversion
Conversion not allowed
Requires explicit CAST to prevent the loss of precision or scale that might occur in an implicit conversion.
Implicit conversions between xml data types are supported only if the source or target is untyped xml. Otherwise, the conversion must be explicit.

Variables

- Variables can be either:
  - Standard
  - Table (temporal)

- Syntax: DECLARE @var_name data_type [= initial_value]

- Use SET/SELECT to assign a value

```
DECLARE @var int = 4
SELECT region_name FROM regions WHERE region_id=@var
GO

DECLARE @var int
SET @var = 4
SELECT region_name FROM regions WHERE region_id=@var
GO

DECLARE @var int
SELECT @var = region_id FROM countries WHERE country_id='ZW'
SELECT region_name FROM regions WHERE region_id=@var
GO
```

Data Definition Language

- Data Definition Language (DDL)
  - CREATE
  - ALTER
  - DROP

Data Definition Language

CREATE +

AGGREGATE
APPLICATION ROLE
ASSEMBLY
ASYMMETRIC KEY
AVAILABILITY GROUP
BROKER PRIORITY
CERTIFICATE
COLUMNSTORE INDEX
COLUMN ENCRYPTION KEY
COLUMN MASTER KEY
CONTRACT
CREDENTIAL
CRYPTOGRAPHIC PROVIDER
DATABASE
DATABASE AUDIT SPECIFICATION
DATABASE ENCRYPTION KEY
DATABASE SCOPED CREDENTIAL
DEFAULT
ENDPOINT
EVENT NOTIFICATION
EVENT SESSION
EXTERNAL DATA SOURCE
EXTERNAL LIBRARY
EXTERNAL FILE FORMAT
EXTERNAL RESOURCE POOL
EXTERNAL TABLE

FULLTEXT CATALOG
FULLTEXT INDEX
FULLTEXT STOPLIST
FUNCTION
INDEX
LOGIN
MASTER KEY
MESSAGE TYPE
PARTITION FUNCTION
PARTITION SCHEME
PROCEDURE
QUEUE
REMOTE SERVICE BINDING
RESOURCE POOL
ROLE
ROUTE
RULE

SCHEMA
SEARCH PROPERTY LIST
SECURITY POLICY
SELECTIVE XML INDEX
SEQUENCE
SERVER AUDIT
SERVER AUDIT SPECIFICATION
SERVER ROLE
SERVICE
SPATIAL INDEX
STATISTICS
SYMMETRIC KEY
SYNONYM
TABLE
TABLE (SQL Graph)
TABLE IDENTITY (Property)
TRIGGER
TYPE
USER
VIEW
WORKLOAD GROUP
XML INDEX
XML INDEX (Selective XML Indexes)
XML SCHEMA COLLECTION

+ OPTIONS

Data Definition Language

ALTER +

AGGREGATE
APPLICATION ROLE
ASSEMBLY
ASYMMETRIC KEY
AVAILABILITY GROUP
BROKER PRIORITY
CERTIFICATE
COLUMNSTORE INDEX
COLUMN ENCRYPTION KEY
COLUMN MASTER KEY
CONTRACT
CREDENTIAL
CRYPTOGRAPHIC PROVIDER
DATABASE
DATABASE AUDIT SPECIFICATION
DATABASE ENCRYPTION KEY
DATABASE SCOPED CREDENTIAL
DEFAULT
ENDPOINT
EVENT NOTIFICATION
EVENT SESSION
EXTERNAL DATA SOURCE
EXTERNAL LIBRARY
EXTERNAL FILE FORMAT
EXTERNAL RESOURCE POOL
EXTERNAL TABLE

FULLTEXT CATALOG
FULLTEXT INDEX
FULLTEXT STOPLIST
FUNCTION
INDEX
LOGIN
MASTER KEY
MESSAGE TYPE
PARTITION FUNCTION
PARTITION SCHEME
PROCEDURE
QUEUE
REMOTE SERVICE BINDING
RESOURCE POOL
ROLE
ROUTE
RULE

SCHEMA
SEARCH PROPERTY LIST
SECURITY POLICY
SELECTIVE XML INDEX
SEQUENCE
SERVER AUDIT
SERVER AUDIT SPECIFICATION
SERVER ROLE
SERVICE
SPATIAL INDEX
STATISTICS
SYMMETRIC KEY
SYNONYM
TABLE
TABLE (SQL Graph)
TABLE IDENTITY (Property)
TRIGGER
TYPE
USER
VIEW
WORKLOAD GROUP
XML INDEX
XML INDEX (Selective XML Indexes)
XML SCHEMA COLLECTION

+ OPTIONS

Data Definition Language

DROP [IF EXISTS] +

- AGGREGATE
- APPLICATION ROLE
- ASSEMBLY
- ASYMMETRIC KEY
- AVAILABILITY GROUP
- BROKER PRIORITY
- CERTIFICATE
- COLUMNSTORE INDEX
- COLUMN ENCRYPTION KEY
- COLUMN MASTER KEY
- CONTRACT
- CREDENTIAL
- CRYPTOGRAPHIC PROVIDER
- DATABASE
- DATABASE AUDIT SPECIFICATION
- DATABASE ENCRYPTION KEY
- DATABASE SCOPED CREDENTIAL
- DEFAULT
- ENDPOINT
- EVENT NOTIFICATION
- EVENT SESSION
- EXTERNAL DATA SOURCE
- EXTERNAL LIBRARY
- EXTERNAL FILE FORMAT
- EXTERNAL RESOURCE POOL
- EXTERNAL TABLE
- FULLTEXT CATALOG
- FULLTEXT INDEX
- FULLTEXT STOPLIST
- FUNCTION
- INDEX
- LOGIN
- MASTER KEY
- MESSAGE TYPE
- PARTITION FUNCTION
- PARTITION SCHEME
- PROCEDURE
- QUEUE
- REMOTE SERVICE BINDING
- RESOURCE POOL
- ROLE
- ROUTE
- RULE
- SCHEMA
- SEARCH PROPERTY LIST
- SECURITY POLICY
- SELECTIVE XML INDEX
- SEQUENCE
- SERVER AUDIT
- SERVER AUDIT SPECIFICATION
- SERVER ROLE
- SERVICE
- SPATIAL INDEX
- STATISTICS
- SYMMETRIC KEY
- SYNONYM
- TABLE
- TABLE (SQL Graph)
- TABLE IDENTITY (Property)
- TRIGGER
- TYPE
- USER
- VIEW
- WORKLOAD GROUP
- XML INDEX
- XML INDEX (Selective XML Indexes)
- XML SCHEMA COLLECTION

+ Object Name

Data Definition Language

- CREATE OR ALTER
- Command introduced since SQL Server 2016 for Functions, Stored Procedures, Triggers and Views
- In case the object exists, it will be modified and if not, it will be created

Data Control Language

- Data Control Language DCL:
  - GRANT
  - DENY
  - REVOKE

- DDL commands are used to grant, deny or revoke access to resources to principals

- Principals are **entities that can request SQL Server resources**

Data Control Language

- Grants permissions on an object to a principal
- The general concept is to GRANT <some permission> ON <some object> TO <some user, login, or group/role>

```
GRANT { ALL [ PRIVILEGES ] }
      | permission [ ( column [ ,...n ] ) ] [ ,...n ]
      [ ON [ class :: ] securable ] TO principal [ ,...n ]
      [ WITH GRANT OPTION ] [ AS principal ]
```

Data Control Language

- Denies a permission to a principal
- Prevents that principal from inheriting the permission through its group or role memberships
- DENY takes precedence over all permissions,
- Exception: DENY does not apply to object owners or members of the sysadmin fixed server role.

```
DENY   { ALL [ PRIVILEGES ] }
     | <permission>  [ ( column [ ,...n ] ) ] [ ,...n ]
    [ ON [ <class> :: ] securable ]
    TO principal [ ,...n ]
    [ CASCADE] [ AS principal ]
[;]
```

Data Control Language

- Removes a previously granted or denied permission.

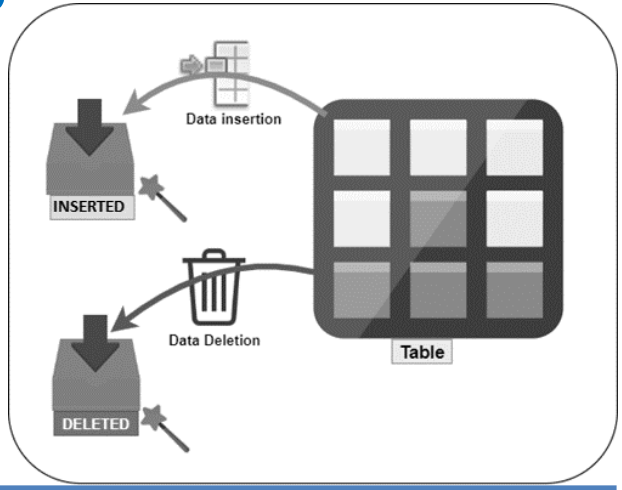
```
REVOKE [ GRANT OPTION FOR ]
      {
        [ ALL [ PRIVILEGES ] ]
        |
                permission [ ( column [ ,...n ] ) ] [ ,...n ]
      }
      [ ON [ class :: ] securable ]
      { TO | FROM } principal [ ,...n ]
      [ CASCADE] [ AS principal ]
```

Data Manipulation Language

- Data Manipulation Language (DML)
  - INSERT
  - BULK INSERT
  - UPDATE
  - DELETE
  - SELECT (considered DQL – Data Query Language)

Data Manipulation Language

- INSERT, UPDATE, DELETE, and DML TRIGGERS have access to 2 Virtual Tables: INSERTED and DELETED
- INSERTED: stores newly inserted rows
- DELETED: stores recently deleted rows



	INSERTED	DELETED
INSERT	New Values	NULL
UPDATE	Values after UPDATE	Value before UPDATE
DELETE	NULL	Deleted Values
DML TRIGER	New Values (Depended on DML on which it was triggered)	Old Values(Depended on DML on which it was triggered)

Data Manipulation Language
INSERT

- INSERT adds one or more rows to a table or a view in SQL Server

```
-- Syntax for SQL Server and Azure SQL Database

[ WITH <common_table_expression> [ ,...n ] ]
INSERT
{
        [ TOP ( expression ) [ PERCENT ] ]
        [ INTO ]
        { <object> | rowset_function_limited
          [ WITH ( <Table_Hint_Limited> [ ...n ] ) ]
        }
    {
        [ ( column_list ) ]
        [ <OUTPUT Clause> ]
        { VALUES ( { DEFAULT | NULL | expression } [ ,...n ] ) [ ,...n     ]
        | derived_table
        | execute_statement
        | <dml_table_source>
        | DEFAULT VALUES
        }
    }
}
[;]

<object> ::=
{
    [ server_name . database_name . schema_name .
      | database_name .[ schema_name ] .
      | schema_name .
    ]
  table_or_view_name
}

<dml_table_source> ::=
    SELECT <select_list>
    FROM ( <dml_statement_with_output_clause> )
      [AS] table_alias [ ( column_alias [ ,...n ] ) ]
    [ WHERE <search_condition> ]
        [ OPTION ( <query_hint> [ ,...n ] ) ]
```

Data Manipulation Language
INSERT

```sql
CREATE TABLE countries_copy(
    country_id   char(2),
    country_name varchar(30),
    region_id    int
);

INSERT INTO countries_copy VALUES (N'RO', N'Romania', 1);

INSERT INTO countries_copy VALUES
(N'MD', N'Moldova', 1),
(N'BG', N'Bulgaria', 1);

INSERT INTO countries_copy
SELECT country_id, country_name, region_id
FROM countries
WHERE region_id = 1;

WITH CountriesReg2(cid, cname, rid)
AS (
    SELECT country_id, country_name, region_id
    FROM countries
    WHERE region_id = 2
)
INSERT INTO countries_copy
OUTPUT inserted.country_id, inserted.country_name, inserted.region_id
SELECT cid, cname, rid
FROM CountriesReg2
ORDER BY cid desc;
```

Data Manipulation Language
INSERT

```sql
-- insert top 3 at random
WITH CountriesReg2(cid, cname, rid)
AS (
    SELECT country_id, country_name, region_id
    FROM countries
    WHERE region_id = 2
)
INSERT TOP(3) INTO countries_copy
OUTPUT inserted.country_id, inserted.country_name, inserted.region_id
SELECT cid, cname, rid
FROM CountriesReg2
ORDER BY cid DESC;

-- insert top 3 at ordered by ORDER BY
WITH CountriesReg2(cid, cname, rid)
AS (
    SELECT country_id, country_name, region_id
    FROM countries
    WHERE region_id = 2
)
INSERT INTO countries_copy
OUTPUT inserted.country_id, inserted.country_name, inserted.region_id
SELECT TOP(3) cid, cname, rid
FROM CountriesReg2
ORDER BY cid DESC;
```

Data Manipulation Language
INSERT

```
 1 DECLARE @CountriesTmp table(
 2     cid   char(2),
 3     cname varchar(30),
 4     rid   int
 5 );
 6
 7 INSERT countries_copy
 8     OUTPUT
 9         inserted.country_id,
10         inserted.country_name,
11         inserted.region_id
12     INTO @CountriesTmp
13 VALUES (N'SB', N'Serbia', 1),
14 (N'ME', N'Montenegro', 1);
15
16 SELECT * FROM @CountriesTmp;
17 GO
```

Data Manipulation Language
BULK INSERT

- BULK INSERT Imports a data file into a database table or view in a user-specified format in SQL Server

```
BULK INSERT
   { database_name.schema_name.table_or_view_name | schema_name.table_or_view_name | table_or_view_name }
      FROM 'data_file'
     [ WITH
    (
   [ [ , ] BATCHSIZE = batch_size ]
   [ [ , ] CHECK_CONSTRAINTS ]
   [ [ , ] CODEPAGE = { 'ACP' | 'OEM' | 'RAW' | 'code_page' } ]
   [ [ , ] DATAFILETYPE =
      { 'char' | 'native'| 'widechar' | 'widenative' } ]
   [ [ , ] DATA_SOURCE = 'data_source_name' ]
   [ [ , ] ERRORFILE = 'file_name' ]
   [ [ , ] ERRORFILE_DATA_SOURCE = 'data_source_name' ]
   [ [ , ] FIRSTROW = first_row ]
   [ [ , ] FIRE_TRIGGERS ]
   [ [ , ] FORMATFILE_DATA_SOURCE = 'data_source_name' ]
   [ [ , ] KEEPIDENTITY ]
   [ [ , ] KEEPNULLS ]
   [ [ , ] KILOBYTES_PER_BATCH = kilobytes_per_batch ]
   [ [ , ] LASTROW = last_row ]
   [ [ , ] MAXERRORS = max_errors ]
   [ [ , ] ORDER ( { column [ ASC | DESC ] } [ ,...n ] ) ]
   [ [ , ] ROWS_PER_BATCH = rows_per_batch ]
   [ [ , ] ROWTERMINATOR = 'row_terminator' ]
   [ [ , ] TABLOCK ]

   -- input file format options
   [ [ , ] FORMAT = 'CSV' ]
   [ [ , ] FIELDQUOTE = 'quote_characters']
   [ [ , ] FORMATFILE = 'format_file_path' ]
   [ [ , ] FIELDTERMINATOR = 'field_terminator' ]
   [ [ , ] ROWTERMINATOR = 'row_terminator' ]
    )]
```

Data Manipulation Language
UPDATE

- UPDATE Changes existing data in a table or view

```
-- Syntax for SQL Server and Azure SQL Database

[ WITH <common_table_expression> [...n] ]
UPDATE
    [ TOP ( expression ) [ PERCENT ] ]
    { { table_alias | <object> | rowset_function_limited
         [ WITH ( <Table_Hint_Limited> [ ...n ] ) ]
      }
      | @table_variable
    }
    SET
        { column_name = { expression | DEFAULT | NULL }
          | { udt_column_name.{ { property_name = expression
                                | field_name = expression }
                                | method_name ( argument [ ,...n ] )
                              }
          }
          | column_name { .WRITE ( expression , @Offset , @Length ) }
          | @variable = expression
          | @variable = column = expression
          | column_name { += | -= | *= | /= | %= | &= | ^= | |= } expression
          | @variable { += | -= | *= | /= | %= | &= | ^= | |= } expression
          | @variable = column { += | -= | *= | /= | %= | &= | ^= | |= } expression
        } [ ,...n ]

    [ <OUTPUT Clause> ]
    [ FROM{ <table_source> } [ ,...n ] ]
    [ WHERE { <search_condition>
            | { [ CURRENT OF
                  { { [ GLOBAL ] cursor_name }
                      | cursor_variable_name
                  }
                ]
              }
            }
    ]
    [ OPTION ( <query_hint> [ ,...n ] ) ]
[ ; ]

<object> ::=
{
    [ server_name . database_name . schema_name .
    | database_name .[ schema_name ] .
    | schema_name .
    ]
    table_or_view_name}
```

Data Manipulation Language
UPDATE

```
 1 DECLARE @EmpValues TABLE (
 2     SalaryBefore float,
 3     SalaryAfter float
 4 );
 5
 6 WITH BossTbl AS (
 7     SELECT DISTINCT manager_id id
 8     FROM employees WHERE manager_id IS NOT NULL
 9 )
10 UPDATE employees
11 SET salary = salary * 1.15
12     OUTPUT deleted.salary, inserted.salary
13     INTO @EmpValues
14 WHERE employee_id IN (SELECT id FROM BossTbl)
15
16 SELECT SalaryBefore, SalaryAfter
17 FROM @EmpValues
18 GO
```

Data Manipulation Language
DELETE

- DELETE removes one or more rows from a table or view in SQL Server

```
-- Syntax for SQL Server and Azure SQL Database

[ WITH <common_table_expression> [ ,...n ] ]
DELETE
    [ TOP ( expression ) [ PERCENT ] ]
    [ FROM ]
    { { table_alias
      | <object>
      | rowset_function_limited
      [ WITH ( table_hint_limited [ ...n ] ) ] }
      | @table_variable
    }
    [ <OUTPUT Clause> ]
    [ FROM table_source [ ,...n ] ]
    [ WHERE { <search_condition>
            | { [ CURRENT OF
                  { { [ GLOBAL ] cursor_name }
                      | cursor_variable_name
                  }
                ]
              }
            }
    ]
    [ OPTION ( <Query Hint> [ ,...n ] ) ]
[; ]

<object> ::=
{
    [ server_name.database_name.schema_name.
      | database_name. [ schema_name ] .
      | schema_name.
    ]
    table_or_view_name
}
```

Data Manipulation Language
DELETE

```
DECLARE @CountriesTmp TABLE(
  cid varchar(2),
  cname varchar(20)
);

WITH NoOffice AS (
  SELECT DISTINCT country_id AS cid FROM countries
  EXCEPT
  SELECT DISTINCT country_id AS cid FROM locations
)
DELETE FROM countries
OUTPUT deleted.country_id, deleted.country_name
INTO @CountriesTmp
WHERE country_id in (SELECT cid FROM NoOffice);

SELECT * FROM @CountriesTmp;
GO
```

Data Manipulation Language
SELECT

- SELECT retrieves rows from the database and enable

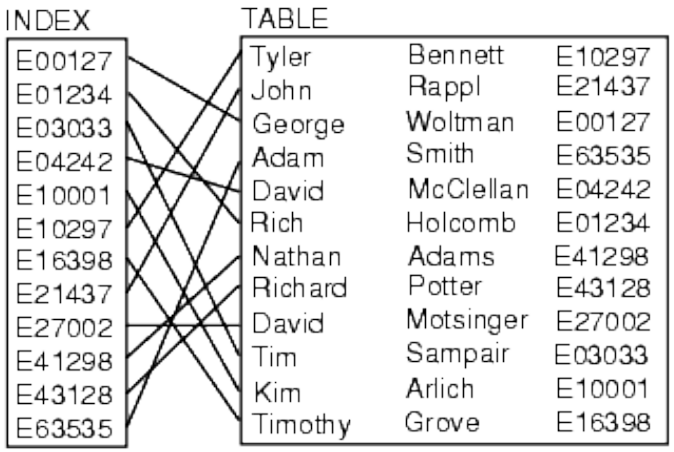
```
-- Syntax for SQL Server and Azure SQL Database

<SELECT statement> ::=
    [ WITH { [ XMLNAMESPACES ,] [ <common_table_expression> [,...n] ] } ]
    <query_expression>
    [ ORDER BY <order_by_expression> ]
    [ <FOR Clause>]
    [ OPTION ( <query_hint> [ ,...n ] ) ]
<query_expression> ::=
    { <query_specification> | ( <query_expression> ) }
    [  { UNION [ ALL ] | EXCEPT | INTERSECT }
        <query_specification> | ( <query_expression> ) [...n ] ]
<query_specification> ::=
SELECT [ ALL | DISTINCT ]
    [TOP ( expression ) [PERCENT] [ WITH TIES ] ]
    < select_list >
    [ INTO new_table ]
    [ FROM { <table_source> } [ ,...n ] ]
    [ WHERE <search_condition> ]
    [ <GROUP BY> ]
    [ HAVING < search_condition > ]
```

Data Manipulation Language
SELECT

```sql
SELECT TOP 5 first_name + ' ' + last_name
FROM EMPLOYEES
ORDER BY 1;
GO
```

Indexes

- A database index is a data structure that improves the speed of data retrieval operations on a database table at the cost of additional writes and storage space to maintain the index data structure
- Indexes are used to quickly locate data without having to search every row in a database table every time a database table is accessed
- Indexes can be created using one or more columns of a database table, providing the basis for both rapid random lookups and efficient access of ordered records



Indexes

Index type	Description
Hash	• Data is accessed through an in-memory hash table • Hash indexes consume a fixed amount of memory
Clustered	• Cluster Index sorts and stores the data rows of the table or view in order based on the clustered index key • The clustered index is implemented as a B-tree index structure that supports fast retrieval of the rows, based on their clustered index key values
Nonclustered	• A nonclusterd index can be defined on a table or view with a clustered index or on a heap (a table without a clustered index) • Each index row in the nonclustered index contains the nonclustered key value and a row locator • This locator points to the data row in the clustered index or heap having the key value. • The rows in the index are stored in the order of the index key values, but the data rows are not guaranteed to be in any particular order unless a clustered index is created on the table.
Memory-optimized Nonclustered	• For this index, the memory consumption is a function of the row count and the size of the index key columns

Indexes

Index type	Description
Unique	• A unique index ensures that the index key contains no duplicate values and therefore every row in the table or view is in some way unique. • Uniqueness can be a property of both clustered and nonclustered indexes.
Columnstore	• An in-memory columnstore index stores and manages data by using column-based data storage and column-based query processing • Columnstore indexes work well for data warehousing workloads that primarily perform bulk loads and read-only queries.
Index with included columns	• A nonclustered index that is extended to include non-key columns in addition to the key columns.
Index on computed columns	• An index on a column that is derived from the value of one or more other columns, or certain deterministic inputs.

Indexes

Index type	Description
Filtered	• An optimized nonclustered index, especially suited to cover queries that select from a well-defined subset of data. • It uses a filter predicate to index a portion of rows in the table. • A well-designed filtered index can improve query performance, reduce index maintenance costs, and reduce index storage costs compared with full-table indexes.
Spatial	• A spatial index provides the ability to perform certain operations more efficiently on spatial objects (spatial data) in a column of the geometry data type. • The spatial index reduces the number of objects on which relatively costly spatial operations need to be applied.
XML	• A shredded, and persisted, representation of the XML binary large objects (BLOBs) in the xml data type column.
Full-text	• A special type of token-based functional index that is built and maintained by the Microsoft Full-Text Engine for SQL Server • It provides efficient support for sophisticated word searches in character string data

Views

- A view
  - Is a virtual table whose contents are defined by a query
  - Consists of a set of named columns and rows of data (like a table)
  - Does not exist as a stored set of data values in a database (unless indexed)

- In a view, the rows and columns of data
  - Come from tables referenced in the query defining the view
  - Are produced dynamically when the view is referenced

Views

- Standard View
- Indexed View
  - A view that has been materialized, i.e., the view definition has been computed and the resulting data stored just like a table
- Partitioned view:
  - A partitioned view joins horizontally partitioned data from a set of member tables across one or more servers
  - This makes the data appear as if it is from one table
  - A view that joins member tables on the same instance of SQL Server is a local partitioned view
- System View views expose catalogue metadata

Views

```sql
-- standard view
CREATE VIEW ViewEmployeeName
AS
SELECT employee_id, first_name, last_name
FROM employees
GO

-- use the standard view
SELECT first_name, last_name FROM ViewEmployeeName
GO

--Set the options to support indexed views.
SET NUMERIC_ROUNDABORT OFF
SET ANSI_PADDING, ANSI_WARNINGS, CONCAT_NULL_YIELDS_NULL, ARITHABORT,
    QUOTED_IDENTIFIER, ANSI_NULLS ON
GO

--Create view with schemabinding.
CREATE VIEW ViewEmployeeName
WITH SCHEMABINDING
AS
SELECT employee_id, first_name, last_name
FROM [dbo].employees
GO

-- Create an index on the view to make the view indexed
CREATE UNIQUE CLUSTERED INDEX IDX_ViewEmployeeName ON ViewEmployeeName (employee_id)
GO

-- Use the indexed view
SELECT first_name, last_name FROM ViewEmployeeName
GO
```

Dynamic SQL

- Dynamic SQL
  - Not so fast, but can be anything
  - Usually used with Cursors
  - Syntax: EXEC @some_sql

EXEC ('SELECT * FROM employees')

Cursors in Transact-SQL

- A cursor is a data type for variables or stored procedure OUTPUT parameters that contain a reference to a cursor.
- The operations that can reference variables and parameters having a cursor data type are:
  - DECLARE, OPEN, FETCH, CLOSE, and DEALLOCATE cursor statements
  - Stored procedure output parameters
  - CURSOR_STATUS function

Cursors in Transact-SQL

```
DECLARE emp_cursor CURSOR FOR
  SELECT
    concat(e.first_name, ' ', e.last_name) AS nume_ang,
    d.department_name AS dept_ang
  FROM employees AS e
    LEFT JOIN  departments AS d ON e.department_id = d.department_id;
OPEN emp_cursor;
DECLARE @an varchar(50);
DECLARE @dn varchar(25);
FETCH NEXT FROM emp_cursor INTO @an, @dn;
WHILE (@@FETCH_STATUS <> -1)
BEGIN
  PRINT @an + ' works in ' + @dn;
  FETCH NEXT FROM emp_cursor INTO @an, @dn;
END;
PRINT 'Finished';
CLOSE emp_cursor;
DEALLOCATE emp_cursor;
GO
```

XQuery

- Transact-SQL supports a subset of the XQuery language
- XQuery is a language that can query structured or semi-structured XML data
- With the XML data type support provided in the Database Engine, documents can be stored in a database and then queried by using XQuery

XML DML

- The XML Data Modification Language is an extension of the XQuery language
- The XML DML provides a fully functional query and data-modification language that can be used for XML data
- It adds the following case-sensitive keywords to XQuery:
  - Insert
  - Delete
  - Replace value of

XML DML

```
 1 DROP TABLE IF EXISTS employees_xml;
 2
 3 CREATE TABLE employees_xml (xml_value XML);
 4
 5 DECLARE @XML_STRING NVARCHAR(MAX);
 6
 7 SET @XML_STRING = '<ROOT> ' + (
 8         SELECT e.name AS employee_name
 9             ,de.department_name AS employee_department
10             ,m.name AS manager_name
11             ,dm.department_name AS manager_department
12         FROM (SELECT employee_id, manager_id, department_id,
13                 concat(first_name, ' ', last_name) name
14               FROM employees) e
15         LEFT OUTER JOIN departments de
16             ON e.department_id = de.department_id
17         LEFT OUTER JOIN (SELECT employee_id, manager_id, department_id,
18                                 concat(first_name, ' ', last_name)
   name
19                          FROM employees) m
20             ON e.manager_id = m.employee_id
21         LEFT OUTER JOIN departments dm
22             ON m.department_id = dm.department_id
23         FOR XML AUTO
24         ) + '</ROOT>';
25
26 INSERT INTO employees_xml (xml_value) VALUES (@XML_STRING);
27 GO
```

XML DML

```
SET QUOTED_IDENTIFIER ON

DECLARE @doc XML, @iddoc INT

SELECT top(1) @doc = XML_VALUE FROM employees_xml

EXEC sp_xml_preparedocument @iddoc OUTPUT, @doc

SELECT
    e.x.value('@employee_name', 'varchar(max)')
        AS employee_name,
    de.x.value('@employee_department', 'varchar(max)')
        AS employee_department,
    m.x.value('@manager_name', 'varchar(max)')
        AS manager_name,
    dm.x.value('@manager_department', 'varchar(max)')
        AS manager_department
FROM
    @doc.nodes('/ROOT/e') AS e(x)
    OUTER APPLY e.x.nodes('de') AS de(x)
    OUTER APPLY de.x.nodes('m') AS m(x)
    OUTER APPLY m.x.nodes('dm') AS dm(x);
GO
```

Graph Query

- Using a graph, data can be represented by
  - Relationships (edges)
  - Entities (nodes)

- Graph structures
  - Make finding and expressing relationships easier
  - Help to generate better insights from interconnected data

Graph Query

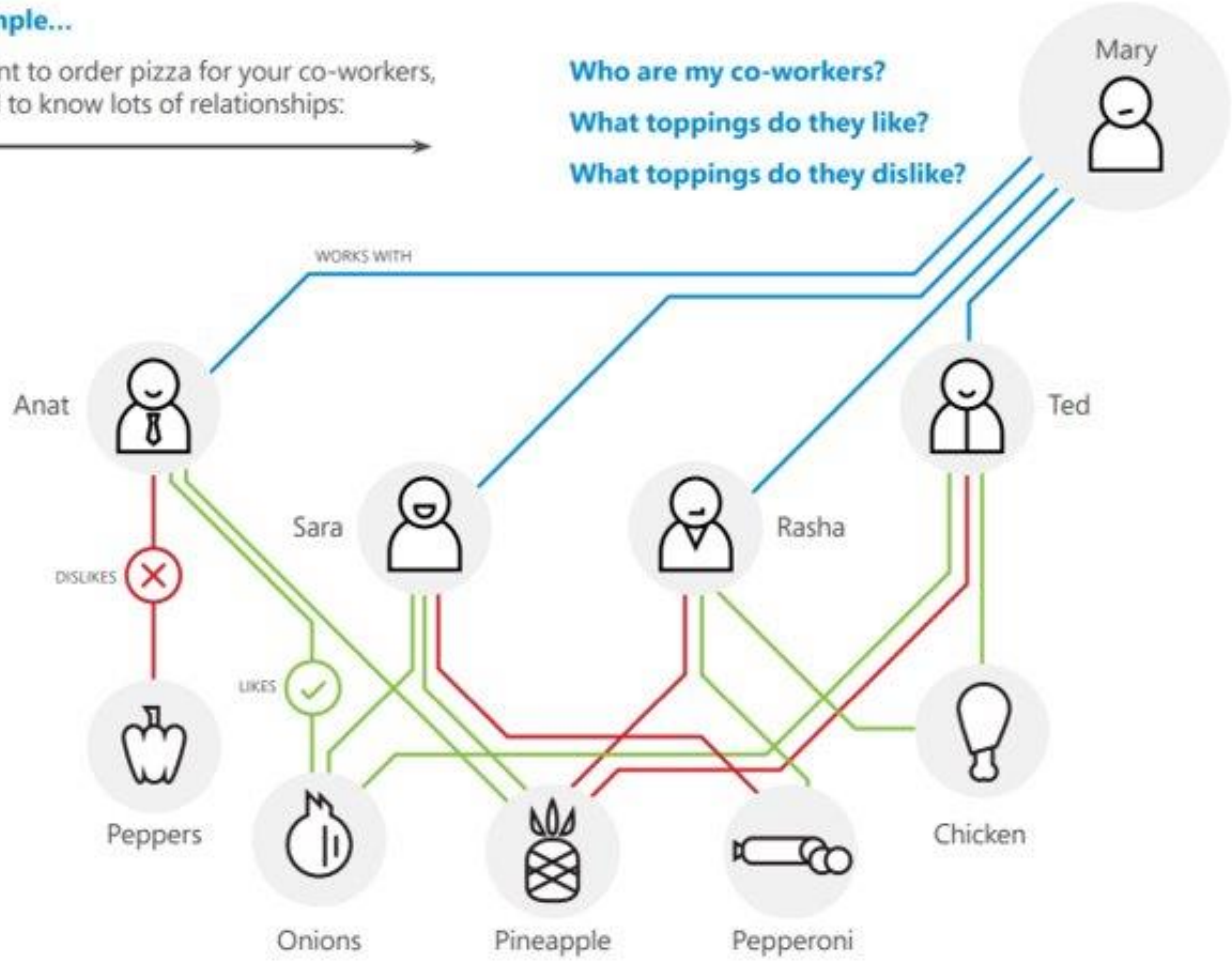
For example...

If you want to order pizza for your co-workers, you need to know lots of relationships:

Who are my co-workers?

What toppings do they like?

What toppings do they dislike?



Graph Query

TRADITIONAL SQL QUERY: VS **GRAPH QUERY:**

```
SELECT DISTINCT Toppings.topping
FROM dbo.toppings AS Toppings
WHERE Toppings.id IN
      (SELECT Likes.toppingID
      FROM dbo.relToppingLikes AS Likes
      WHERE Likes.peopleID IN
            (SELECT Coworkers.coworkerID
            FROM dbo.relCoworkers AS Coworkers
            WHERE Coworkers.personID IN
                  (SELECT Person.id
                  FROM dbo.people AS Person
                  WHERE Person.firstName = 'Mary')))
      AND Toppings.id NOT IN
      (SELECT DisLikes.toppingID
      FROM dbo.relToppingDisLikes AS DisLikes
      WHERE DisLikes.peopleID IN
            (SELECT Coworkers.coworkerID
            FROM dbo.relCoworkers AS Coworkers
            WHERE Coworkers.personID IN
                  (SELECT Person.id
                  FROM dbo.people AS Person
                  WHERE Person.firstName = 'Mary')))
```

```
SELECT DISTINCT Topping.topping
FROM gPeople AS Person, gPeople AS Coworker,
eWorkswith, eLikes, eDisLikes, gToppings AS Topping
WHERE MATCH(Person-(eWorkswith)->Coworker-(eLikes)-
>Topping)
AND Topping.ID NOT IN
      (SELECT DISTINCT Topping.ID
      FROM gPeople AS Coworker, eWorkswith,
      eDisLikes, gToppings AS Topping
      WHERE MATCH(Person-(eWorkswith)->Coworker-
      (eDisLikes)->Topping))
AND Person.firstName='Mary'
```

User-Defined Functions

- SQL Server supports user-defined functions (UDFs)
- UDFs are routines that
  - Accept parameters
  - Perform an action, such as a complex calculation
  - Return the result of that action as a value
- The return value can either be a single scalar value or a result set.
- Function features:
  - READ-ONLY
  - Faster execution
  - Can reduce network traffic

User-Defined Functions

```sql
CREATE OR ALTER FUNCTION ufnIsBoss(@eid int)
RETURNS varchar(3)
AS
BEGIN
  DECLARE @ok int;
  DECLARE @isBoss varchar(3) = 'Yes';
  SELECT @ok = count(*)
  FROM employees
  WHERE manager_id = @eid;
  IF (@ok = 0)
    SET @isBoss = 'No';
  RETURN @isBoss;
END;
GO

SELECT
    first_name,
    last_name,
    [dbo].ufnIsBoss(employee_id)
FROM employees;
GO
```

User-Defined Functions

```sql
CREATE FUNCTION ufnGetSubordinate(@eid int)
RETURNS TABLE
AS
RETURN
(
    SELECT concat(first_name, ' ', last_name) AS subordinate
    FROM employees
    WHERE manager_id = @eid;
);
GO

SELECT * FROM ufnGetSubordinate(100)
GO
```

Stored Procedures

- A stored procedure in SQL Server is
  - A group of one or more Transact-SQL statements
  - A reference to a Microsoft .NET Framework common runtime language (CLR) method
- Procedures:
  - Accept input parameters and return multiple values
  - Contain programming statements that perform operations in the database
  - Return a status value to a calling program to indicate success or failure (and the reason for failure)
- Features:
  - Reduced server/client network traffic
  - Stronger security
  - Reuse of code
  - Improved performance

Stored Procedures

```
CREATE OR ALTER PROCEDURE usp_divide(
  @a decimal,
  @b decimal,
  @c decimal OUT
)
AS
BEGIN
  BEGIN TRY
    SET @c = @a / @b
  END TRY
  BEGIN CATCH
    PRINT 'Error Number    ' + STR(ERROR_NUMBER())
    PRINT 'Error Severity  ' + STR(ERROR_SEVERITY())
    PRINT 'Error State     ' + STR(ERROR_STATE())
    PRINT 'Error Procedure ' + ERROR_PROCEDURE()
    PRINT 'Error Line      ' + STR(ERROR_LINE())
    PRINT 'Error Message   ' + ERROR_MESSAGE()
  END CATCH
END
GO

DECLARE @r decimal;
EXEC usp_divide 10, 2, @r output;
PRINT @r;
GO

DECLARE @r decimal;
EXEC usp_divide 10, 0, @r output;
PRINT @r;
GO
```

Triggers

- Event-driven types of instructions that are used to ensure the integrity of the database
- DDL Triggers fire in response to a DDL event
- DML Triggers
  - Is a special type of stored procedure
  - Automatically takes effect when a DML event that affects the table or view defined in the trigger takes place (i.e., INSERT, UPDATE, or DELETE) => AFTER triggers
- DML triggers can be used to
  - Enforce business rules
  - Data integrity
  - Query other tables
  - Include complex Transact-SQL statements

DML Triggers

```
CREATE OR ALTER TRIGGER UpdateSalaryOrCommision
ON employees
FOR UPDATE
AS
   DECLARE @sal float
   DECLARE @comm float
   DECLARE @emp varchar(50)
   DECLARE @message varchar(255)
   SELECT
      @emp = ins.first_name + ' ' + ins.last_name,
      @sal = ins.salary,
      @comm = ins.commission_pct
   FROM INSERTED ins

   IF UPDATE(salary) AND UPDATE(commission_pct)
      SET @message = concat('Updated ', @emp, ' salary to ', @sal, ' and comm to ',
@comm)
   ELSE
   BEGIN
      IF UPDATE(salary)
         SET @message = concat('Updated ', @emp, ' salary to ', @sal)
      ELSE
         SET @message = concat('Updated ', @emp, ' comm to ', @comm)
   END
   PRINT @message
GO

UPDATE employees SET salary = 5000 WHERE employee_id = 100;
GO

UPDATE employees SET commission_pct = 0.5 WHERE employee_id = 100;
GO

UPDATE employees SET salary = 5000, commission_pct = 0.5 WHERE employee_id = 100;
GO
```

References

- Microsoft SQL Server 2019 Technical Documentation
https://learn.microsoft.com/en-us/sql/sql-server/?view=sql-server-ver15

- Microsoft SQL Server 2022 Technical Documentation
https://learn.microsoft.com/en-us/sql/sql-server/?view=sql-server-ver16

University
Politehnica
of Bucharest

FACULTATEA DE
AUTOMATICĂ ȘI
CALCULATOARE
Universitatea POLITEHNICA din București

Computer
Science and
Engineering
Department

Data Warehouses and OLAP

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@upb.ro

Overview

- Data Warehouses
- OLAP

Overview

- Data Warehouses
- OLAP

Data warehousing

Data Scientist

also known as Data Managers, statisticians.

A data scientist will be able to take data science projects from end to end. They can help store large amounts of data, create predictive modelling processes and present the findings.

Skills: Mathematics, Programming, Communication

Will use programmes such as:

SQL, Python, R

Data Engineers

also known as database administrators and data architects.

They are versatile generalists who use computer science to help process large datasets. They typically focus on coding, cleaning up data sets, and implementing requests that come from data scientists.

Skills: Programming, Mathematics, Big data

Will use programmes such as:

Hadoop, NoSQL, and Python

Data Analysts

also known as business Analysts.

They typically help people from across the company understand specific queries with charts.

Skills: Statistics, Communication, Business knowledge

Will use programmes such as:

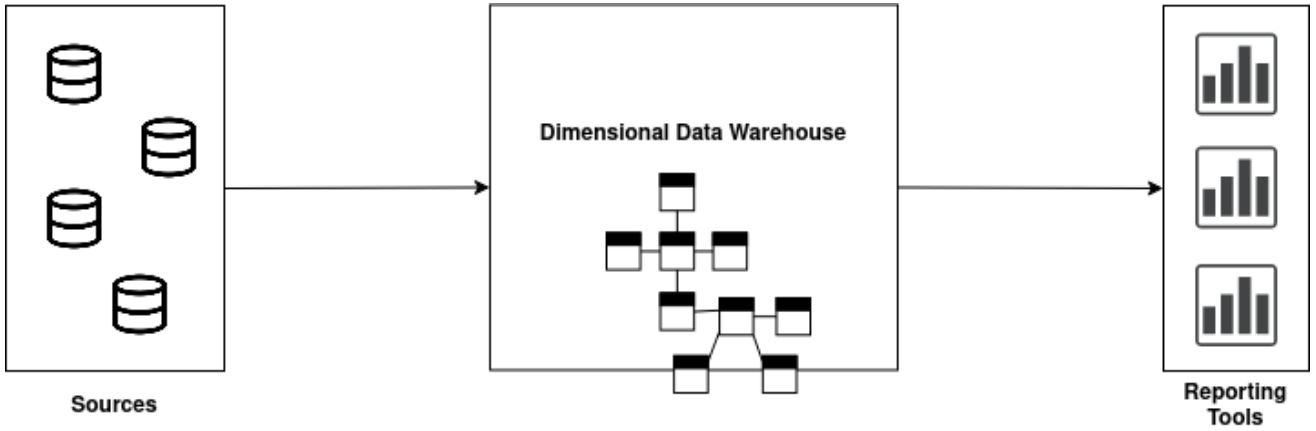
Excel, Tableau, SQL

Data warehousing

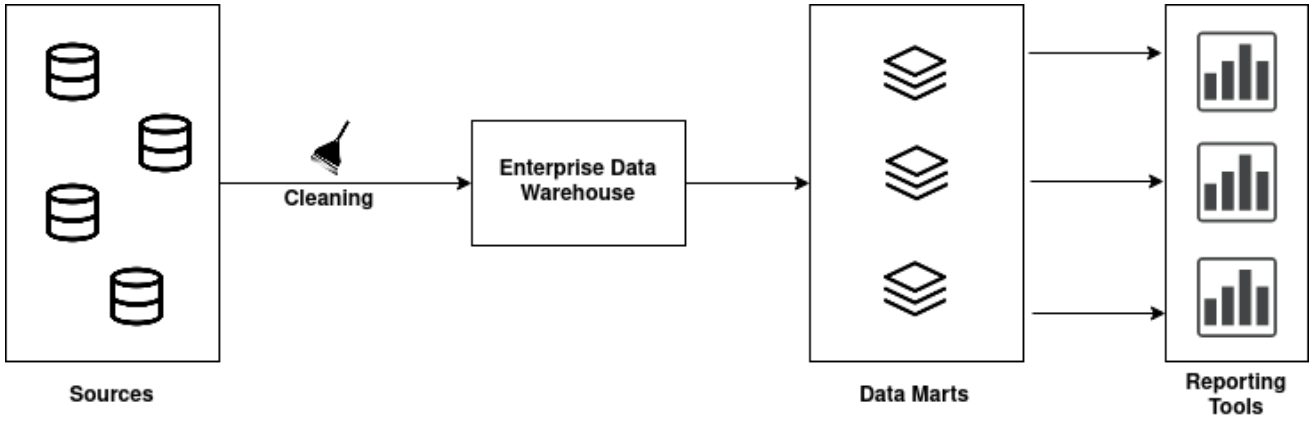
- Data warehouse is a repository of an organization's electronically stored data
- Data warehouses are designed to facilitate reporting and analysis
- A data warehouse stores a standardized, consistent, clean, and integrated form of data sourced from various operational systems in use in the organization
- Data sourced are structured in a way to specifically address the reporting and analytic requirements.

Data warehousing

- Bottom-up approach [Kimball 2002]:



- Top-down approach [Inmon 2002]:



Data warehousing
Kimball design

- Ralph Kimball design approach [Kimball 2002]:
  - The data model follows a bottom-up approach
  - Business process-oriented approach
    - First define data marts based on the business requirements
    - Build the data warehouse out of data marts
  - Each data mart uses a multi-dimensional modeling approach

Data warehousing
Kimball design advantages (I)

- R. Kimball design approach advantages [Kimball 2002]:
  - The dimensional modeling is fast to construct as no normalization is involved, which means swift execution of the initial phase of the data warehousing design process
  - A multi-dimensional modeling can easily be comprehended by data operators (users) because the denormalized structure also simplifies querying and analysis
  - Data warehouse focuses on individual business areas and processes rather than the whole enterprise

Data warehousing
Kimball design advantages (I)

- R. Kimball design approach advantages [Kimball 2002]:
  - It enables fast data retrieval from the data warehouse, as data is segregated into fact tables and dimensions.
  - A smaller team of designers and planners is sufficient for data warehouse management because data source systems are stable, and the data warehouse is process-oriented.
  - Query optimization is straightforward, predictable, and controllable.
  - Conformed dimensional structure for data quality framework, allowing business intelligence tools to deeper across several star schemas and generates reliable insights

Data warehousing
Kimball design disadvantages (I)

- R. Kimball design approach disadvantages [Kimball 2002]:
  - Data is not entirely integrated before reporting
  - Irregularities can occur when data is updated:
    - Denormalization adds redundant data
  - Performance issues may occur due to the addition of columns in the fact table, as these tables are quite in-depth.
    - The addition of new columns can expand the fact table dimensions, affecting its performance
    - Also, the dimensional data warehouse model becomes difficult to alter with any change in the business needs

Data warehousing
Kimball design disadvantages (II)

- R. Kimball design approach disadvantages [Kimball 2002]:
  - The model is business process-oriented
    - Does not focus on the enterprise as a whole
    - Cannot handle all the BI reporting requirements
  - The process of incorporating large amounts of legacy data into the data warehouse is complex

Data warehousing
Inmon design

- Bill Inmon modeling approach [Inmon 2002]:
  - The data model follows a top-down approach
    - Define the data warehouse
    - Afterwards define the data marts
  - A data warehouse is a: subject-oriented, integrated, nonvolatile, time-variant collection of data in support of management's decisions.
  - The Data Warehouse
    - Uses the normalized form for building entity structure, avoiding data redundancy as much as possible
    - Follows the normalized structure
    - Contains multiple data marts

Data warehousing
Inmon design advantages

- B. Inmon design approach advantages [Inmon 2002]:
  - Data warehouse acts as a unified source of truth for the entire business, where all data is integrated
  - This approach has very low data redundancy.
    - Less irregular data updates
    - The process of adding new data to the data warehouse is more straightforward and less susceptible to failure
  - It simplifies business processes, as the logical model represents detailed business objects.
  - This approach offers greater flexibility
    - It's easier to update the data warehouse in case there's any change in the business requirements or source data
  - It can handle diverse enterprise-wide reporting requirements

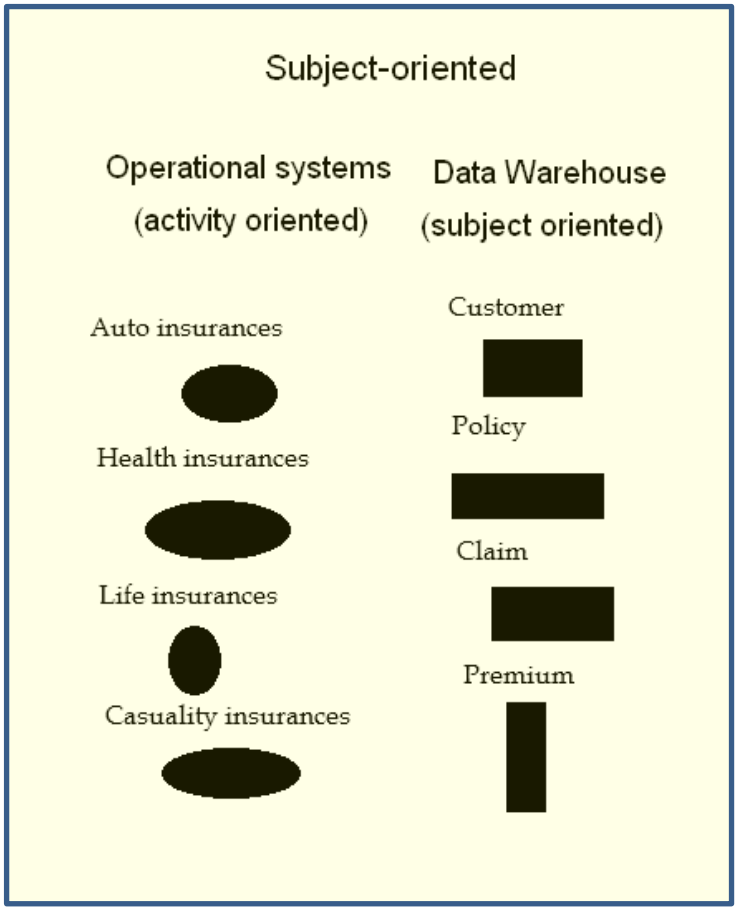
Data warehousing
Inmon design disadvantages

- B. Inmon design approach disadvantages [Inmon 2002]:
  - Complexity increases as multiple tables are added to the data model with time
  - Skilled data operators (users) are required
    - Users need to understand the data warehouse data modeling
    - Expensive and challenging to find
  - The preliminary setup and delivery are time-consuming
  - Additional ETL process operation is required since data marts are created after the creation of the data warehouse
  - This approach requires experts to manage a data warehouse effectively

Data warehousing
Subject-Oriented

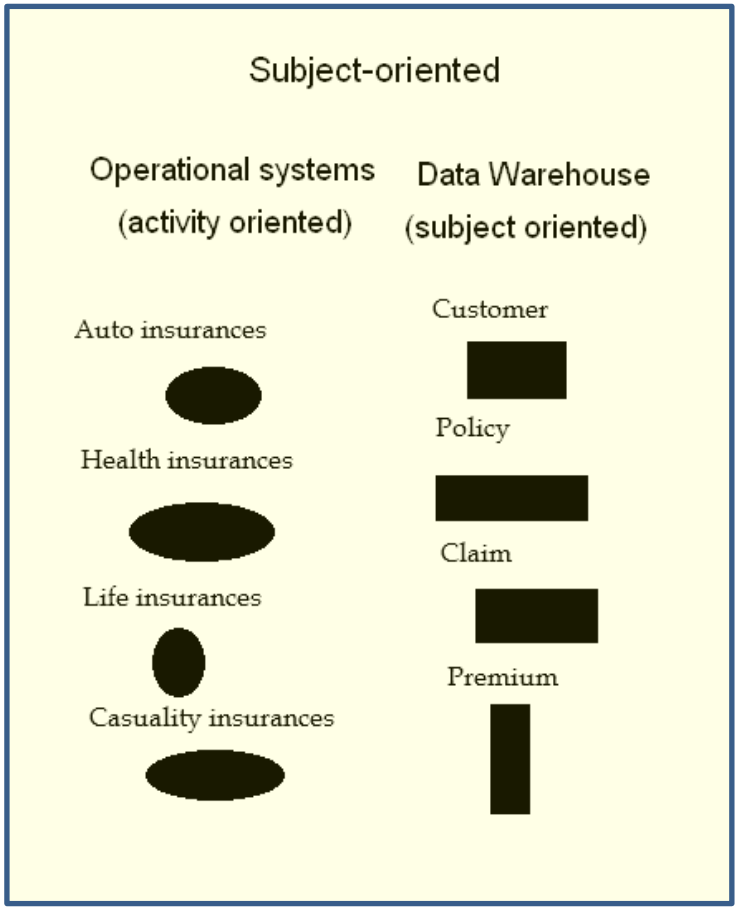
- Operational data systems of a company
  - Are organized considering the main activities
  - Are activity-oriented and not subject oriented

- A classical example in the literature is an insurance company where the ***main activities*** are:
  - Auto insurances
  - Health insurances
  - Life insurances
  - Casualty insurances



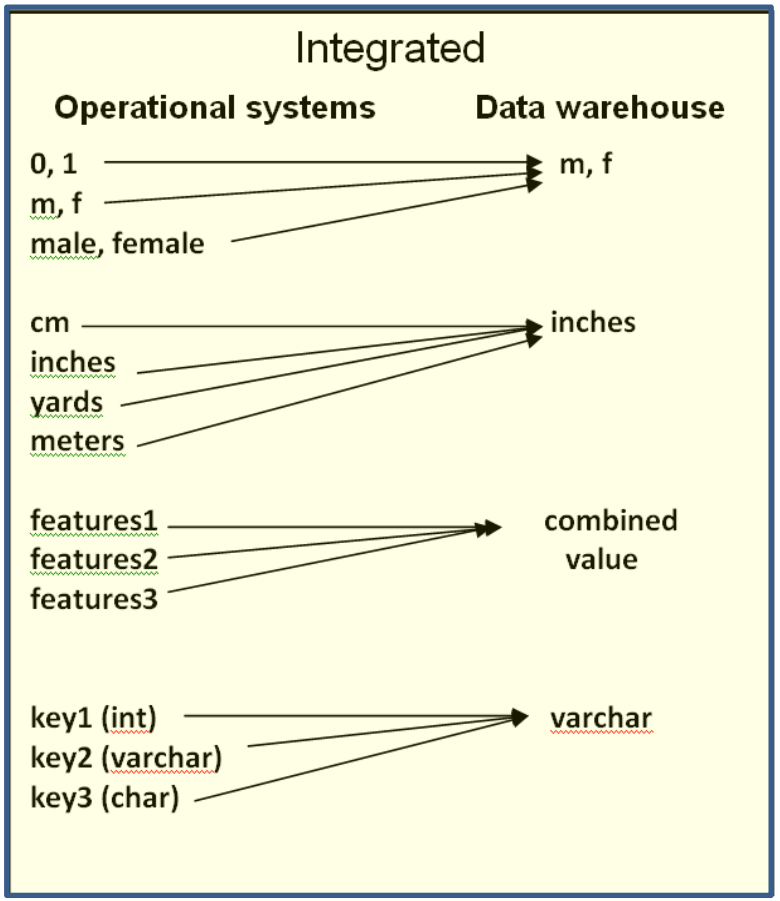
Data warehousing
Subject-Oriented

- For each activity there is possibly another software system managing data on the main ***subject areas***:
  - Policies
  - Customers
  - Claims
  - Premiums in the area
- There are possible separate databases, one for each activity, with similar but not identical structures
- When uploading data in the company data warehouse the data must first
  - Be ***restructured*** on these major subject areas
  - Integrates data on customers, policies, claims and premiums from each activity



Data warehousing
Integrated

- When preparing data for uploading in the data warehouse, one of the most important activities is the integration
- Data is loaded from operational sources and must be converted, summarized, re-keyed, etc., before loading it in the data warehouse
- Combine multiple encodings into a single one.
- For example
  - The gender may be encoded as (0, 1), (m, f), (male, female) in separate operational systems
  - If (m, f) is chosen as the data warehouse encoding, all data encoded using other convention must be converted



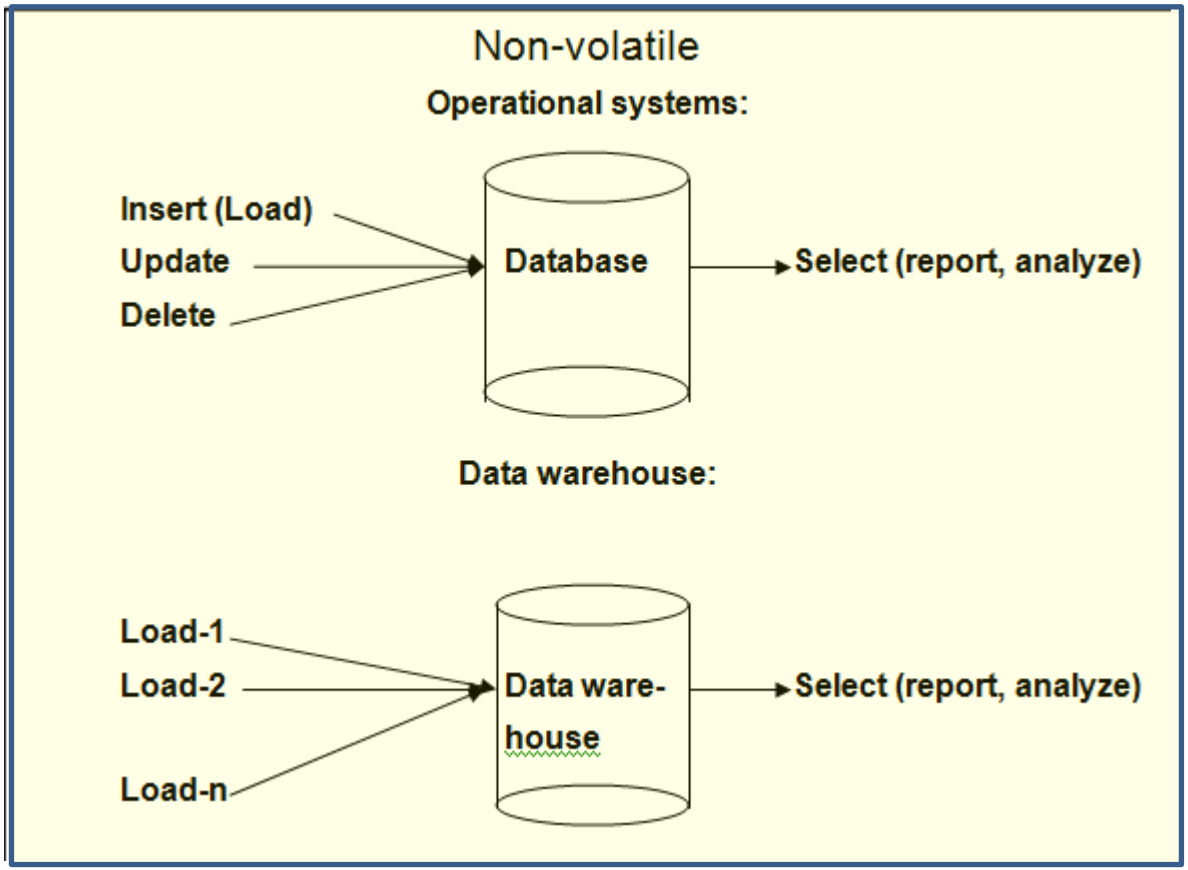
Data warehousing
Integrated

- Actions performed for data integration:
  - Chose a unique measure unit for each piece of information
    - For example, if length is measured in cm, inches, yards and meters in different operational systems, one unit must be chosen for the data warehouse and all other values must be converted
  - If the same object has in some data sources different values for the same attribute (e.g. description, name, features, etc.), these must be combined in a single one
  - If the same object has different keys in the source systems it must be re-keyed to have a single key in the data warehouse

Data warehousing
Non-volatile

- In usual operational systems data is updated or deleted to reflect the actual values
- In a data warehouse data is never updated and deleted:
  - After data is loaded, it stays there for future reporting
  - Similar to a snapshot it reflects the situation at a certain moment
- The next load operation
  - Data are added as new snapshots
  - The old snapshots are not changed
- The data warehouse is a sequence of snapshots that coexist
- In this way the data warehouse contains all the history of operational data (not only the operational data at a given moment)
- Because of this lack of change, once loaded, data in a data warehouse may be considered as read-only

Data warehousing
Non-volatile



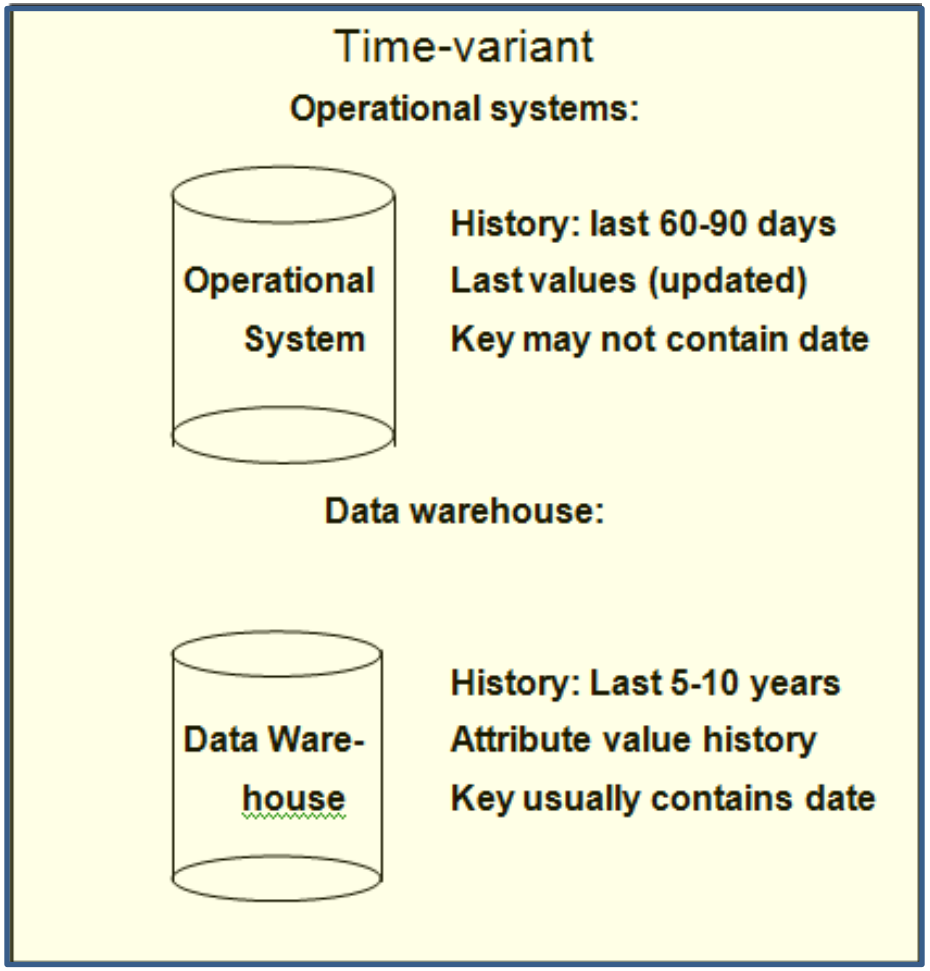
Data warehousing
Time variant

- As described above
  - A data warehouse contains a sequence of snapshots
  - Each snapshot being actual at a given moment of time

- Data Warehouse contains the whole history of a company thus it is possible to retrieve information from a time window (slice)

- Each unit of information is stamped or linked with the moment during which that information was accurate

Data warehousing



Data warehousing
Time variant

- In an operational system only the current data is kept
  - e.g., if a customer changes address, in the operational system old address is replaced (update) with the new one
- In the data warehouse all the successive information is kept
  - E.g., all successive addresses of a customer are kept
- The key (primary key) structure contains usually the date and sometimes the time
  - Date and time are very important for analyzing data and reporting

Data warehousing

- Requirements for a Data Warehouse [Kimball 2002]:
  - Information must be easy accessible
  - Information must be consistent
  - Flexibility
  - Security
  - Decision Support
  - User Acceptance

Data warehousing

- Why build a DW? [Kimball 2002]
  - “We have mountains of data in this company, but we can’t access it.”
  - “We need to slice and dice the data every which way.”
  - “You’ve got to make it easy for business people to get at the data directly.”
  - “Just show me what is important.”
  - “It drives me crazy to have two people present the same business metrics at a meeting, but with different numbers.”
  - “We want people to use information to support more fact-based decision making.”

Data warehousing
Information must be easy accessible

- DW content must be understandable
- DW content must be intuitive or obvious to the non-database specialists, because they are the key users of the system
- Names must be meaningful (for data categories, features, attributes and so on, so the structure of the DW must be understandable for a non-specialist user)
- The DW must provide options for combining data, the process being known and referred to as *slicing and dicing.*
- The methods and tools for accessing data in the data warehouse must
  - Be simple
  - Be easy to use
  - Return the answers in a short time.

Data warehousing
Information must be consistent

- The process of fueling a data warehousing with data contains a step of preprocessing, where
  - Data is assembled from many sources, cleansed, quality assured
  - Data is released (published) to the users only when it is fit for usage

- An integration step is performed when data is load from operational sources to unify encodings, units of measure, keys, names and common values/features, etc.

- Common definitions for the contents of the data warehouse must be available for DW users

Data warehousing Flexibility

- A data warehouse must be designed to be flexible considering the inevitable changes in computer science and engineering
- Its content must be structured in such a way that changes in the software and hardware platform must be possible
- Adding new data, reports, queries must be possible and must not interfere with existing ones

Data warehousing Security

- Because of its confidential content
  - The data warehouse must have the means for rejecting unauthorized access
  - Potential leaks of content may be harmful for the company if competitors have access to the data in the DW.

Data warehousing
Decision support

- The primary goal of implementing a data warehouse in an organization is the decision support

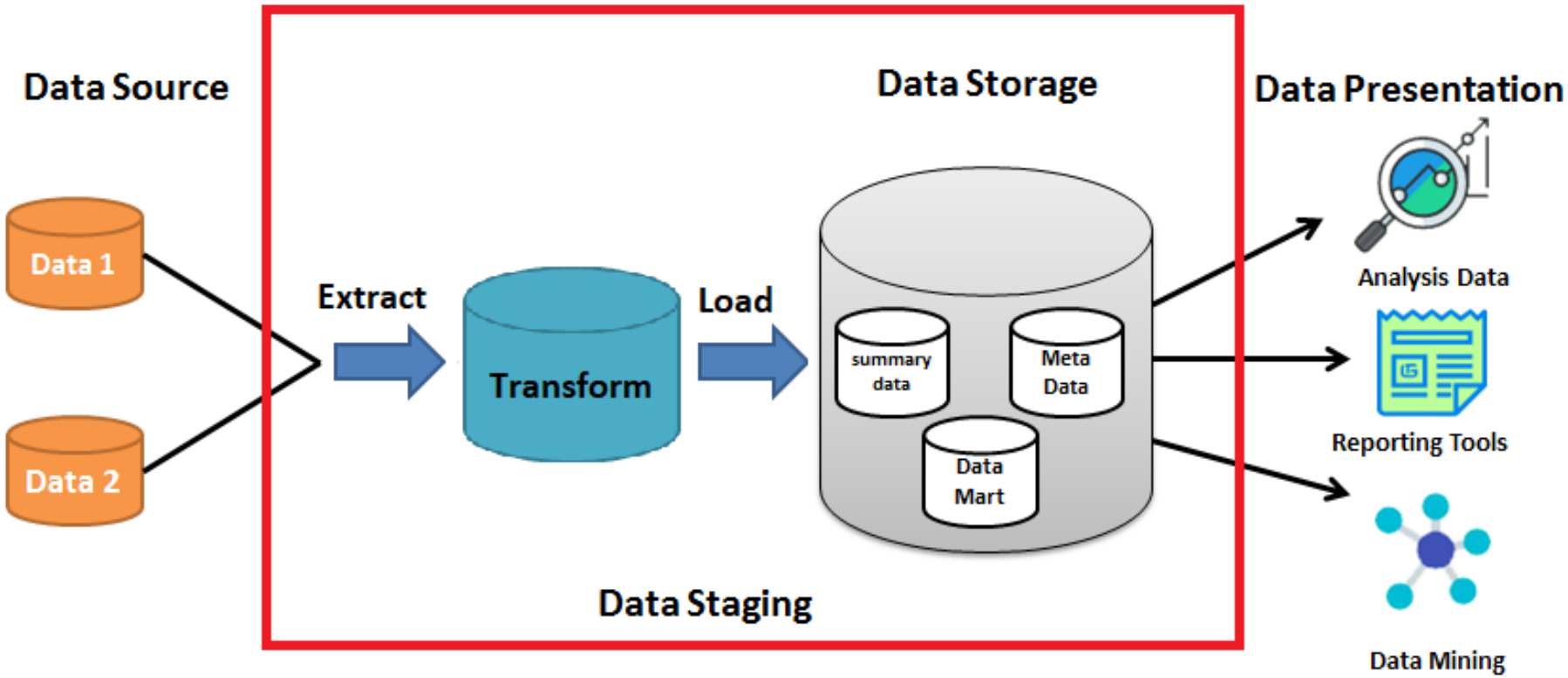
- The ultimate output from a DW is the set of decisions based on its content, analyzed and presented in different ways to the decision makers

- The original label for a data warehouse and the tools around it was ‘decision support system’

Data warehousing Acceptance

- The ultimate test for the success in implementing a data warehouse is the acceptance test
- If the business community does not continue to use it in the first six months after training, then the system has failed the acceptance test, no mater how bright is the technical solution
- It is possible to ignore using it because decisions may be adopted without a decision support system
- Key point in user acceptance is simplicity and user friendliness

DW architecture



DW architecture

- Operational Source Systems
  - These are the source of the data in the DW
  - Are placed outside of the data warehouse
- Data Staging Area
  - Here data is prepared (transformed) for loading in the data storage
  - This area is not accessible to the regular user
- Data Presentation
  - This part is what regular users see and consider to be a DW
  - Different Data Access Tools are available to the users
  - These tool used for analyzing and reporting
  - They provide the interface between the user and the DW

DW architecture

- The **data staging area** (DSA) of a data warehouse is :
  - Contains the two parts:
    - A data preprocessing pipeline (Extract-Transform-Load)
    - A storage area
  - A set of processes performing the so-called Extract-Transform-Load (ETL) operation:
    - Extract – Extracting data from Operational Source Systems
    - Transform – Integrating data from all sources
    - Load – Loading the data in the storage area and publishes the data to be accessed by the Data presentation area

DW architecture

- DSA contains everything between the operational source systems and the data presentation area
- This area is not accessible to the regular users of the data warehouse.
- Storing data in a DW (so also in DSA) may be done following two main approaches:
  1. The normalized approach
  2. The dimensional approach
- These approaches are not mutually exclusive, and there are other approaches.
- Dimensional approaches can involve normalizing data to a degree.

DW architecture
Normalized approach

- In the ***normalized approach (extensively used in OLTP)***
  - Data are stored following database normalization rules
  - Tables are grouped by subject areas (data on customers, policies, claims and premiums for example)

- The main ***advantage*** of this approach is that loading data is straightforward because the philosophy of structuring data is the same for operational source systems and the data warehouse

- The main ***disadvantage*** of this approach is the number of joins needed to obtain meaningful information.

- A regular user needs also to have a good knowledge about the data in the DW and also a training period in obtaining de-normalized tables from normalized ones.

- Missing a join condition when performing a query may lead to Cartesian products instead of joins. In other words, regular user may need assistance from a database specialist to perform usual operations.

DW architecture
Dimensional approach

- Data is partitioned in two main categories:
  - Facts (numeric transaction data). In a retail example, the fact table contains quantity sold, total price, total cost, total gross profit
  - Dimensions (standardized contexts for facts). In a retail example, dimensions may be: product, date, time, location, customer, salesperson, etc.
  - Main approach OLAP

- ***Advantages*** of the dimensional approach are:
  - Data is easy to understand, easy to use, no need for assistance from a database specialist, speed in solving queries.
  - Data being de-normalized (or partially de-normalized) the number of joins needed for performing a query is lower than in the normalized approach
  - Joins between the fact table and its dimensions is easy to perform because the fact table contains surrogate keys for all involved dimension tables

- ***Disadvantages*** of dimensional approach:
  - The ETL process is harder to be performed because of the different philosophy in structuring data in the operational systems and the data warehouse: transform and load steps are more complicated than in the normalized approach
  - It is more difficult to modify the data warehouse scheme when the company changes its way to do business.

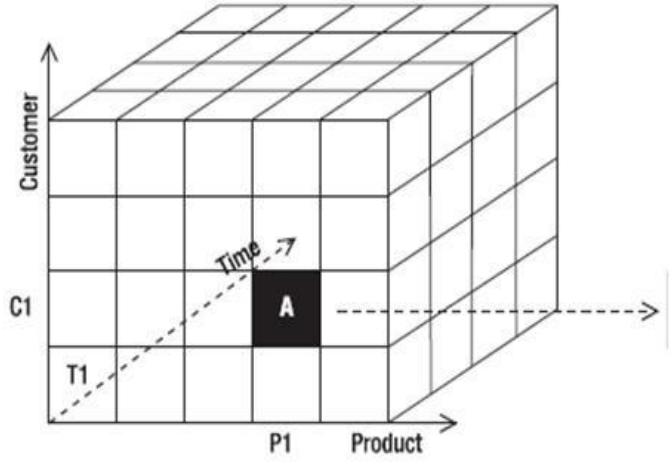
DW architecture
Data presentation area

- At the end of the ETL process prepared data is loaded in storage and published in the Data Presentation Area (DPA)
- After that moment, data is available for users for querying, reporting and other analytical applications
- Because regular users have access only to that area, they may consider the presentation area as being the data warehouse
- This area is structured as a series of integrated **data marts**, each presenting the data from a single business process

DW architecture
Data presentation area

- In the DPA data is presented and accessed in dimensional schemas

- We can imagine a hypercube with edges labeled with the dimensions, e.g., customer, product and time



DW architecture
Data access tools

- Almost all DW regular users (80% to 90%) will access the data via some prebuilt parameter-driven analytic applications.

- Generally, a user has four channels to interact with a DW:
  - Ah-hoc query tools
  - Report writers
  - Analytic applications
  - Modeling tools

DW architecture
Ah-hoc query tools

- By this channel the user obtains raw data verifying the conditions specified in the ad-hoc query
- For using this channel the user must have a good knowledge on the DW structure and on query language used
- This channel is for specialists and experienced users

DW architecture
Report writers

- This channel is at the same level as the first one
- Raw data is presented as a report
- Usually there are several pre-built reports that user may run without knowing the DW structure and query language
- Building new reports may need extra abilities

DW architecture
Analytic applications

- Interactive reports
- Dashboards
- Scorecards
- Other reporting tools allowing users to access and analyze on data in the DW

DW architecture
Modeling tools

- In this category can be mentioned
  - Data mining products
  - Forecasting and scoring tools
  - Machine Learning tools

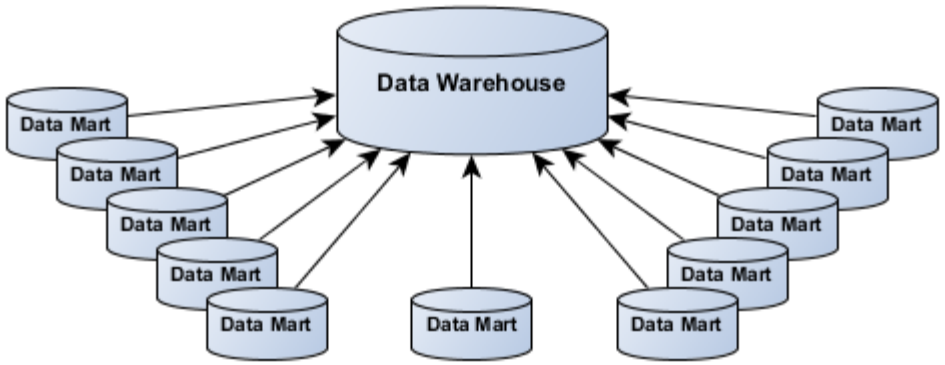
- At this level the result is not only a sophisticated report on existing data but also
  - Extracted new knowledge, patterns, and insights
  - Models for forecasting, classification, and clustering

DW architecture

- A ***data mart*** is defined as a repository of data gathered from operational data and other sources that is designed to serve a particular community of knowledge workers

- Data marts are analytical data stores designed to focus on specific business functions for a specific community within an organization.

- ***The data warehouse is created from the union of organizational data marts***.



Overview

- Data Warehouses
- OLAP

OLAP

- OLAP (On-Line Analytical Processing) is a technology used to organize information in Data Warehouses which offers support for knowledge discovery useful in the decision-making process

OLAP

- Concepts:
  - **Hierarchy** is a logical tree-type structure used to organize the members of a dimension
  - **Level** is a data level of the hierarchy: the information from a hierarchy can be organized using different inferior or superior levels
  - **Member** is an element in a hierarchy
  - **Calculated Member** is a member whose value is computed during execution
  - **Dimension** is a set that contain one or more hierarchies
  - **Measure** (measurement or metric) is a set of values used when creating reports
  - **Cube** is a data structure that aggregates the measures by the levels and hierarchies of each analyzed dimension

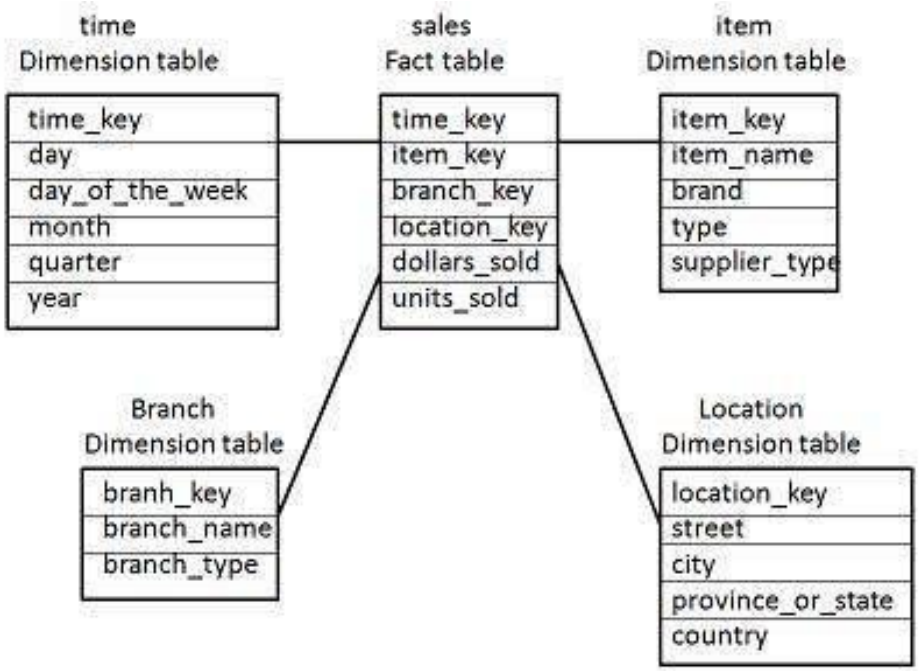
OLAP

OLAP

- In practice these concepts are used to create different schemas:
  - Star schema
  - Snowflake schema
  - Constellation schema

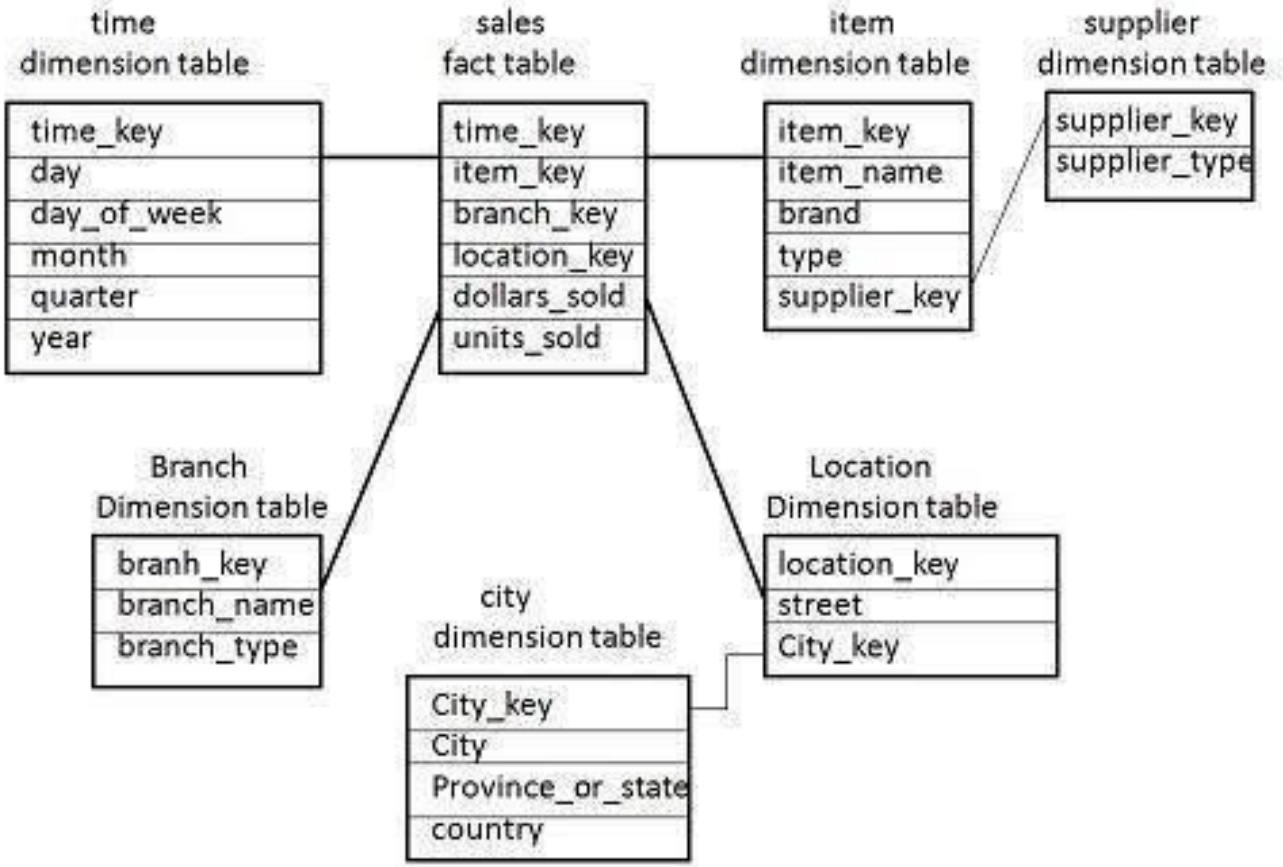
OLAP
Star Schema

- Each dimension is modeled as a table
- The dimension tables contain the set of attributes
- The Facts table constrains:
  - Foreign keys that reference the primary keys in the dimension tables
  - Measurements or metrics based on the dimension hierarchies



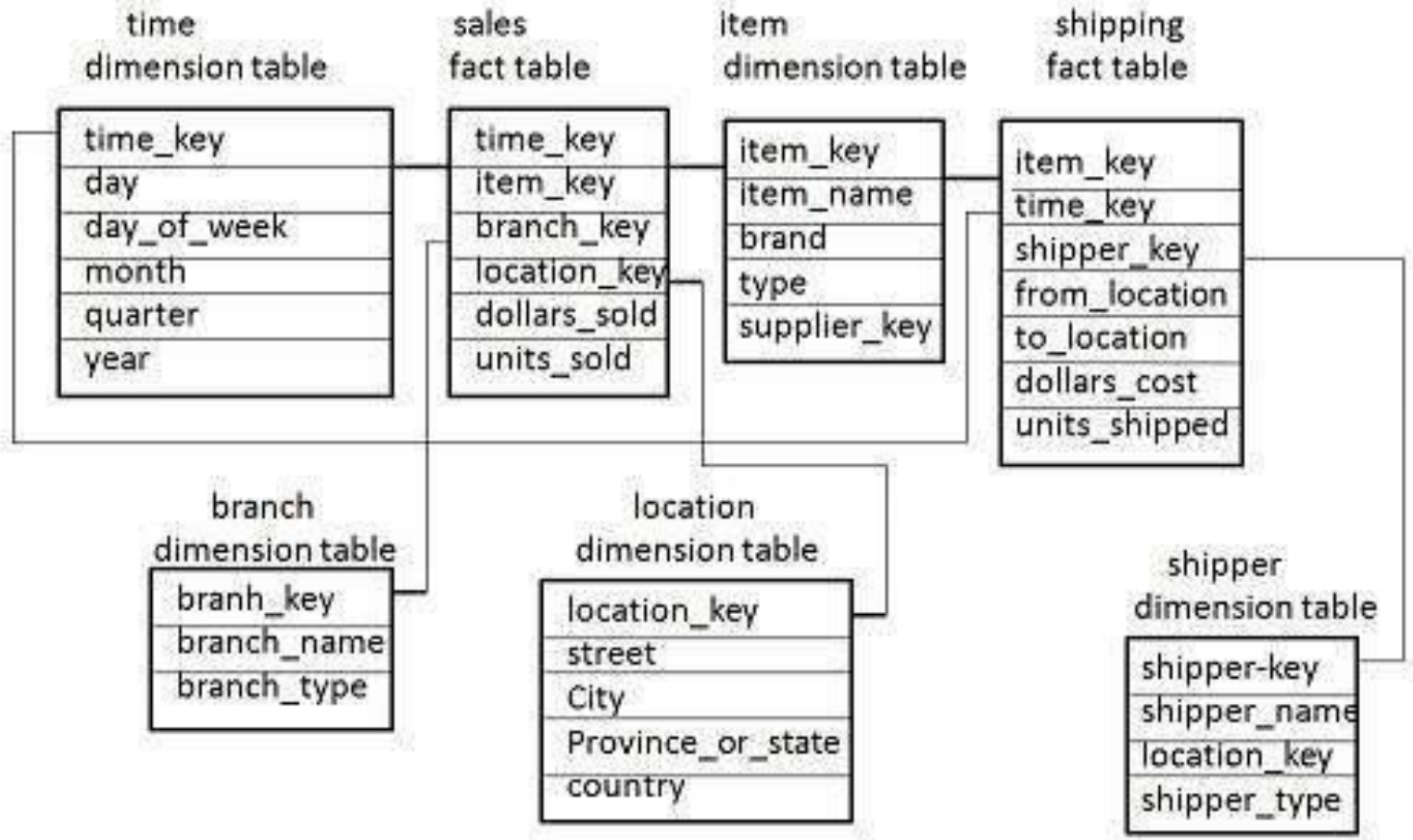
OLAP
Snowflake Schema

- The dimension are normalized
- The data from dimensions and hierarchies are split intro multiple tables



OLAP
Constellation Schema

- Contains multiple Fact Tables



OLAP Operations

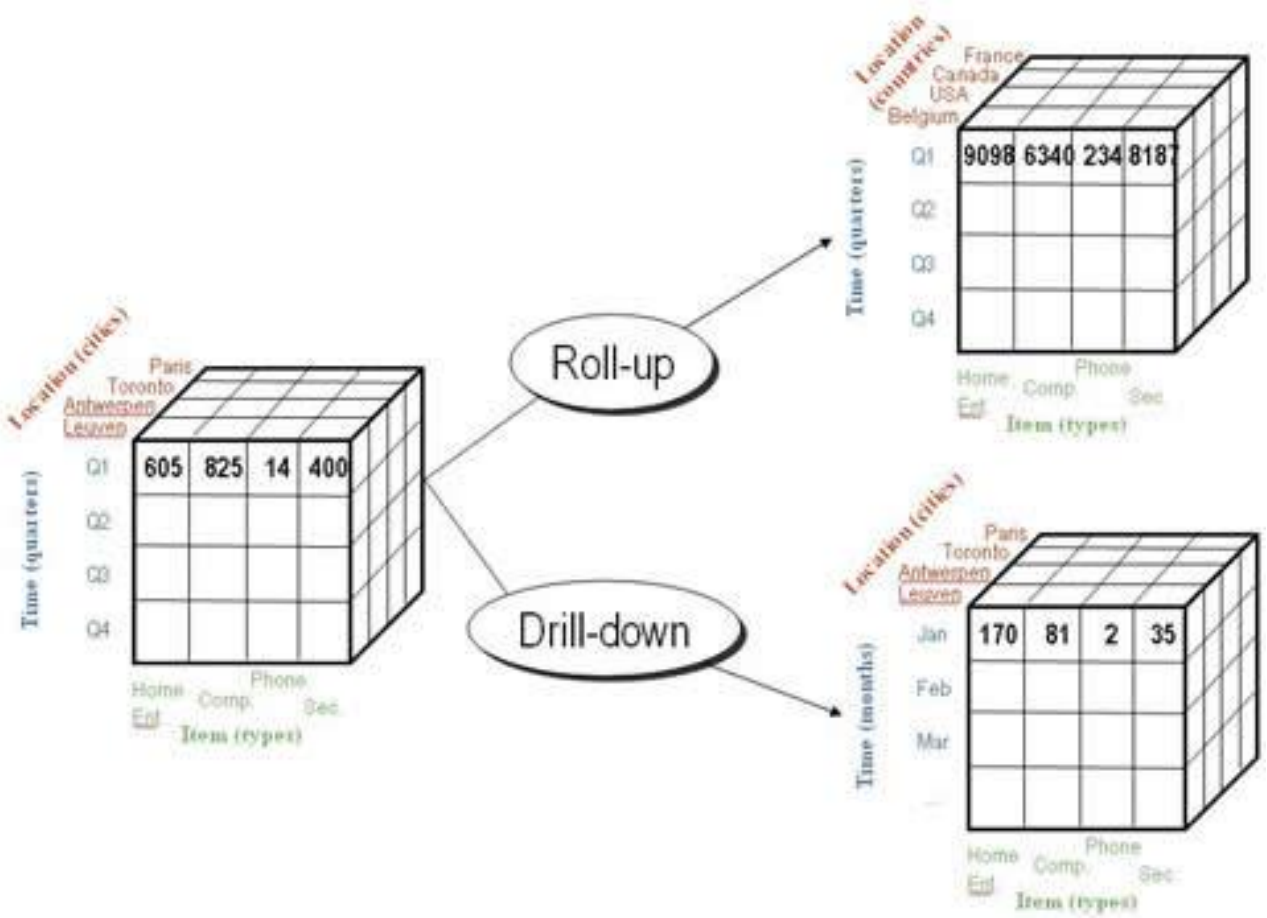
1. Roll-up is used to climb up a concept hierarchy for a dimension or by reducing dimensions

2. Drill-down is used o step down a concept hierarchy or by introducing a new dimension

- Roll-up is the reverse operation of Drill-down

OLAP Operations

- Roll-up and Drill-down



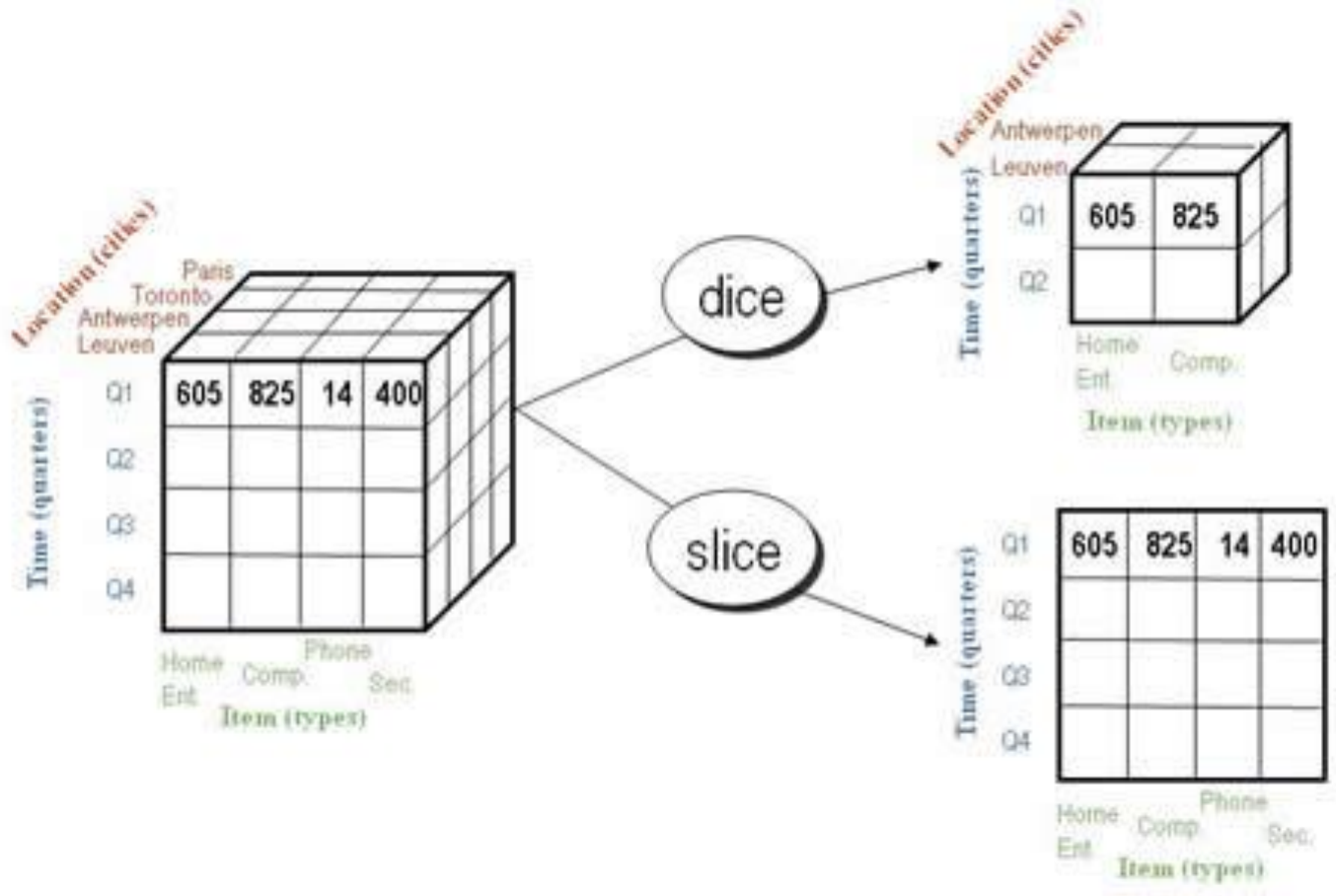
OLAP Operations

3. Slice operation selects one particular dimension from a given cube and provides a new sub-cube

4. Dice selects two or more dimensions from a given cube and provides a new sub-cube

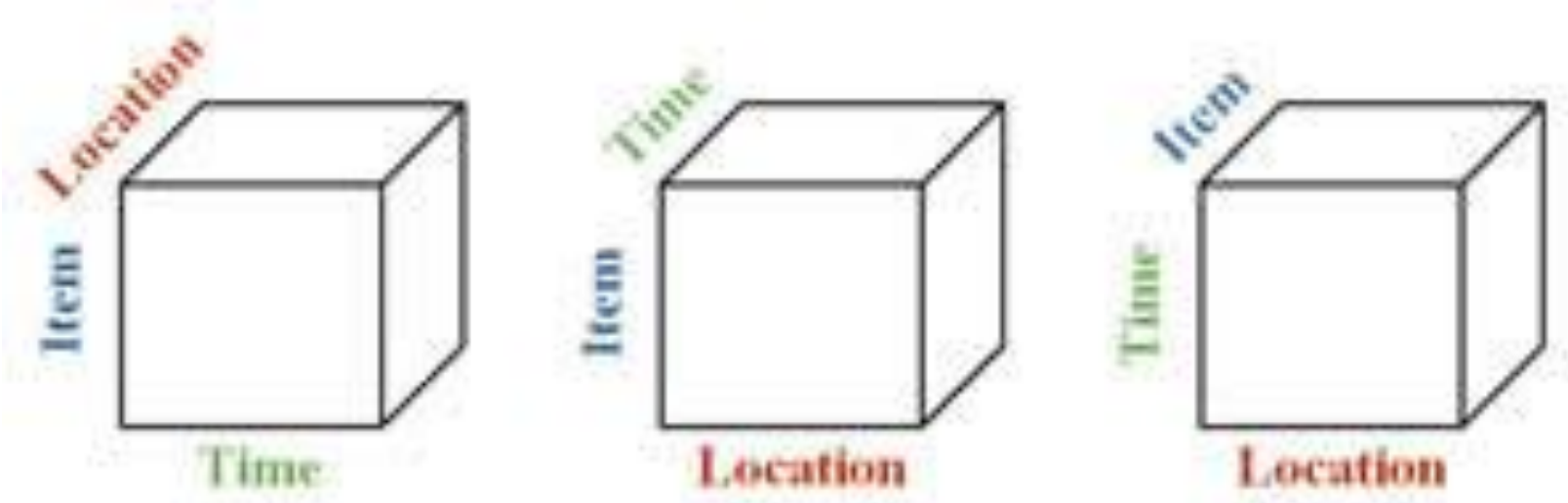
OLAP Operations

- Slice and Dice



OLAP Operations

5. Pivot (or rotate) is used to rotate the cube on one of the axes to visualize the date from different perspective

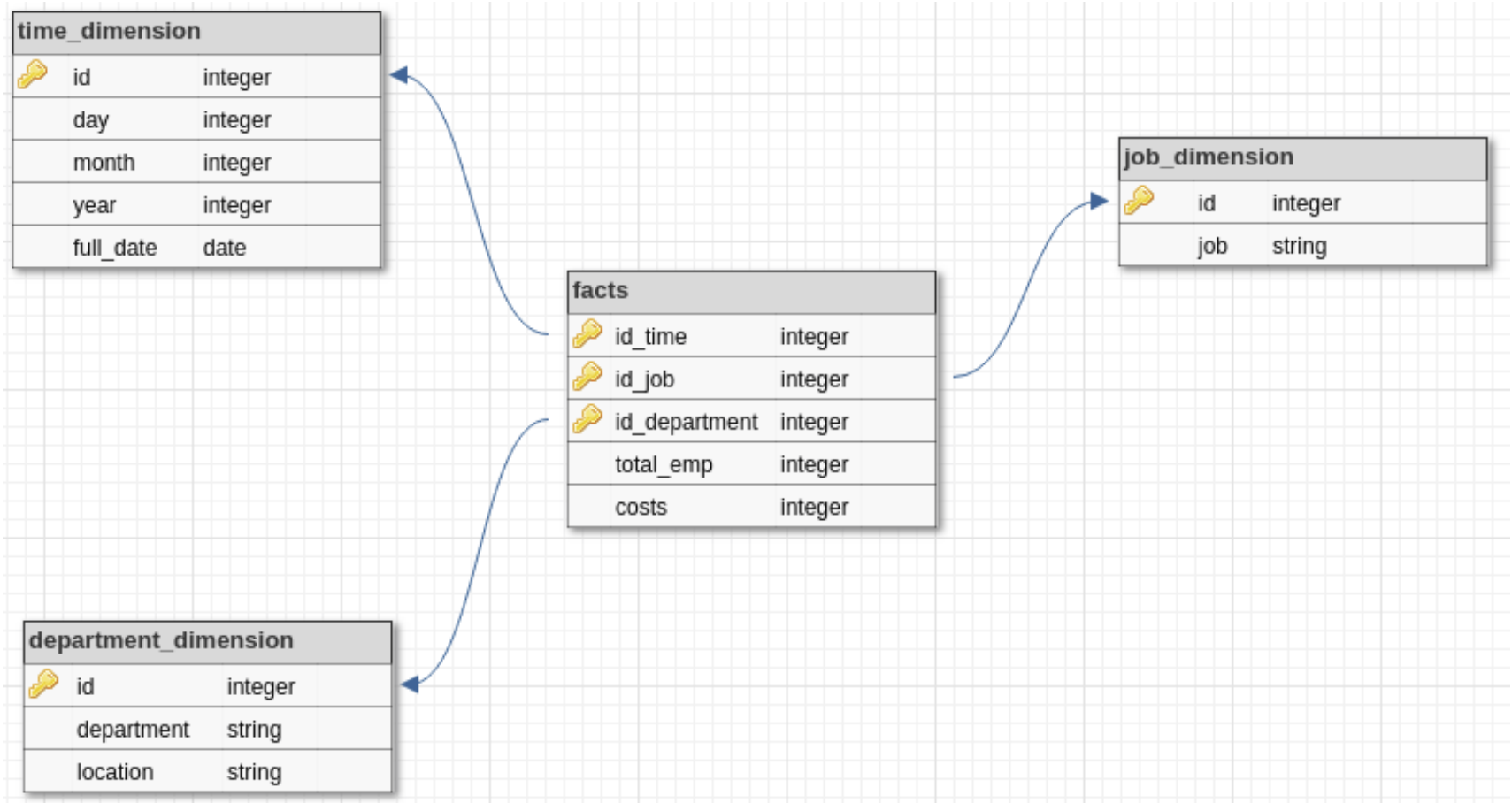


Example

- The following code is written for Oracle version >= 12c
- If you use an older version, when creating the tables you need to use sequences for autoincrementing the primary key
- The code in the slides is executable

Example

- Create multidimensional schema from the HR schema



Example

- Creating and populating the time_dimension

```sql
CREATE TABLE time_dimension(
  id INT generated as identity primary key,
  day INT,
  month INT,
  year INT,
  fulldate DATE
);

INSERT INTO time_dimension(fulldate, day, month, year)
SELECT
    DISTINCT hire_date,
    extract(day FROM hire_date),
    extract(month FROM hire_date),
    extract(year FROM hire_date)
FROM employees
ORDER BY hire_date;
```

Example

- Creating and populating the job_dimension:

```sql
CREATE TABLE job_dimension(
  id VARCHAR2(10) PRIMARY KEY,
  job varchar2(35)
);

INSERT INTO job_dimension(id, job)
SELECT DISTINCT job_id, job_title
FROM jobs
ORDER BY job_title;
```

Example

- Creating and populating the department_dimension

```sql
CREATE TABLE department_dimension(
  id INT generated AS identity PRIMARY KEY,
  department VARCHAR2(30),
  location VARCHAR2(30)
);

INSERT INTO department_dimension(department, location)
SELECT DISTINCT d.department_name, l.city
FROM departments d INNER JOIN locations l
    ON d.location_id = l.location_id
ORDER BY department_name;
```

Example

- Creating the facts table:
  - Add foreign keys to the primary keys from the dimension tables
  - Create a compound primary key from the foreign keys

```
CREATE TABLE facts(
  id_time INT,
  id_job VARCHAR2(10),
  id_departament INT,
  total_employees INT,
  costs float,
  CONSTRAINT fk_1 FOREIGN KEY(id_time) REFERENCES time_dimension(id),
  CONSTRAINT fk_2 FOREIGN KEY(id_job) REFERENCES job_dimension(id),
  CONSTRAINT fk_3 FOREIGN KEY(id_departament) REFERENCES department_dimension(id),
  CONSTRAINT pk_facts PRIMARY KEY(id_time, id_job, id_departament)
);
```

Example

- Populating the facts table (we need to get the keys from the dimension tables to keep the integrity)

```sql
INSERT INTO facts
SELECT
    td.id id_time,
    jd.id id_job,
    dd.id id_departament,
    count(employee_id) total_employees,
    sum(e.salary * (1 + NVL(e.commission_pct, 0))) costs
FROM employees e
  INNER JOIN departments d ON e.department_id = d.department_id
  INNER JOIN department_dimension dd ON dd.department = d.department_name
  INNER JOIN time_dimension td ON td.fulldate = e.hire_date
  INNER JOIN job_dimension jd ON jd.id = e.job_id
GROUP BY td.id, jd.id, dd.id;
```

Example

- Rollup to count the number of employees and the salary costs grouping by year, job and department

```sql
SELECT
  td.year,
  jd.job job,
  dd.department department,
  sum(total_employees) no_emp,
  sum(costs) sal_costs
FROM facts f
  INNER JOIN department_dimension dd ON f.id_departament = dd.id
  INNER JOIN time_dimension td ON f.id_time = td.id
  INNER JOIN job_dimension jd ON f.id_job = jd.id
GROUP BY
  rollup(td.year, jd.job, dd.department)
ORDER BY 1, 2, 3;
```

Example

- Drilldown to count the number of employees and the salary costs grouping by year, month, job and department

```sql
SELECT
  td.year,
  td.month,
  jd.job job,
  dd.department department,
  sum(total_employees) no_emp,
  sum(costs) sal_costs
FROM facts f
  INNER JOIN department_dimension dd ON f.id_departament = dd.id
  INNER JOIN time_dimension td ON f.id_time = td.id
  INNER JOIN job_dimension jd ON f.id_job = jd.id
GROUP BY
  rollup(td.year, td.month, jd.job, dd.department)
ORDER BY 1, 2, 3, 4;
```

Example

- Cube to count the number of employees and the salary costs grouping by year, month, job and department

```sql
SELECT
    td.year,
    td.month,
    jd.job job,
    dd.department department,
    sum(total_employees) no_emp,
    sum(costs) sal_costs
FROM facts f
    INNER JOIN department_dimension dd ON f.id_departament = dd.id
    INNER JOIN time_dimension td ON f.id_time = td.id
    INNER JOIN job_dimension jd ON f.id_job = jd.id
GROUP BY
    cube(td.year, td.month, jd.job, dd.department)
ORDER BY 1, 2, 3, 4;
```

Grouping sets

- If you want to group the data by multiple combinations of group by expressions, you may take one of two approaches:
  - The first approach is to create one grouped query per combination of expressions and merge the results using the UNION ALL operator.
  - The other approach is to use the GROUPING SETS operator along with the GROUP BY clause and define each grouping set within a single query.

Example

GROUPING SETS Equivalent of UNION ALL	
SELECT Col1, Col2, SUM(Col3) FROM Table1 GROUP BY GROUPING SETS ((Col1), (Col2))	SELECT Col1, NULL as Col2, SUM(Col3) FROM Table1 GROUP BY Col1 UNION ALL SELECT NULL as Col1, Col2, SUM(Col3) FROM Table1 GROUP BY Col2
GROUPING SETS ROLLUP Equivalents	
GROUP BY ROLLUP (Col1, Col2, Col3)	GROUP BY GROUPING SETS ((Col1, Col2, Col3) ,(Col1, Col2) ,(Col1) ,())
GROUPING SETS CUBE Equivalents	
GROUP BY CUBE (Col1, Col2, Col3)	GROUP BY GROUPING SETS ((Col1, Col2, Col3) ,(Col1, Col2) ,(Col1, Col3) ,(Col2, Col3) ,(Col1) ,(Col2) ,(Col3) ,())

Example

- Grouping Sets to count the number of employees and the salary costs grouping by the set (year, month) and (job, department)

```sql
SELECT
    td.year,
    td.month,
    jd.job job,
    dd.department department,
    sum(total_employees) no_emp,
    sum(costs) sal_costs
FROM facts f
    INNER JOIN department_dimension dd ON f.id_departament = dd.id
    INNER JOIN time_dimension td ON f.id_time = td.id
    INNER JOIN job_dimension jd ON f.id_job = jd.id
GROUP BY
    grouping sets( (td.year, td.month), (jd.job, dd.department));
```

Example

- Slice to get a facet of the cube (year = 2003)

```sql
SELECT
    td.year,
    td.month,
    dd.department department,
    sum(total_employees) no_emp,
    sum(costs) sal_costs
FROM facts f
    INNER JOIN department_dimension dd ON f.id_departament = dd.id
    INNER JOIN time_dimension td ON f.id_time = td.id
    INNER JOIN job_dimension jd ON f.id_job = jd.id
WHERE
    td.year = 2003
GROUP BY
    cube(td.year, td.month, jd.job, dd.department);
```

Example

- Dice to get a subcube (given by the condition in the where clause)

```sql
SELECT
    td.year,
    td.month,
    dd.department department,
    sum(total_employees) no_emp,
    sum(costs) sal_costs
FROM facts f
    INNER JOIN department_dimension dd ON f.id_departament = dd.id
    INNER JOIN time_dimension td ON f.id_time = td.id
    INNER JOIN job_dimension jd ON f.id_job = jd.id
WHERE
    td.year between 2002 AND 2003 AND
    td.month between 4 AND 6 AND
    lower(dd.department) IN ('accounting', 'shipping')
GROUP BY
    cube(td.year, td.month, dd.department);
```

Example

- Pivot change the column order in the projection, i.e., SELECT clause

```sql
SELECT
    dd.department department,
    td.year,
    td.month,
    sum(total_employees) no_emp,
    sum(costs) sal_costs
FROM facts f
    INNER JOIN department_dimension dd ON f.id_departament = dd.id
    INNER JOIN time_dimension td ON f.id_time = td.id
    INNER JOIN job_dimension jd ON f.id_job = jd.id
GROUP BY
    cube(td.year, td.month, jd.job, dd.department);
```

References

- [Inmon 2002] W.H. Inmon - Building The Data Warehouse. Third Edition, Wiley & Sons, 2002

- [Kimball 2002] Ralph Kimball, Margy Ross - The Data Warehouse Toolkit, Second Edition, Wiley & Sons, 2002

University
Politehnica
of Bucharest

Computer
Science and
Engineering
Department

Big Data
- An Overview -

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@upb.ro

Overview

- What is Big Data?
- Why Big Data?
- Types of Big Data
- Techniques
- Distributed architecture
- Cloud Computing
- Storage and tools

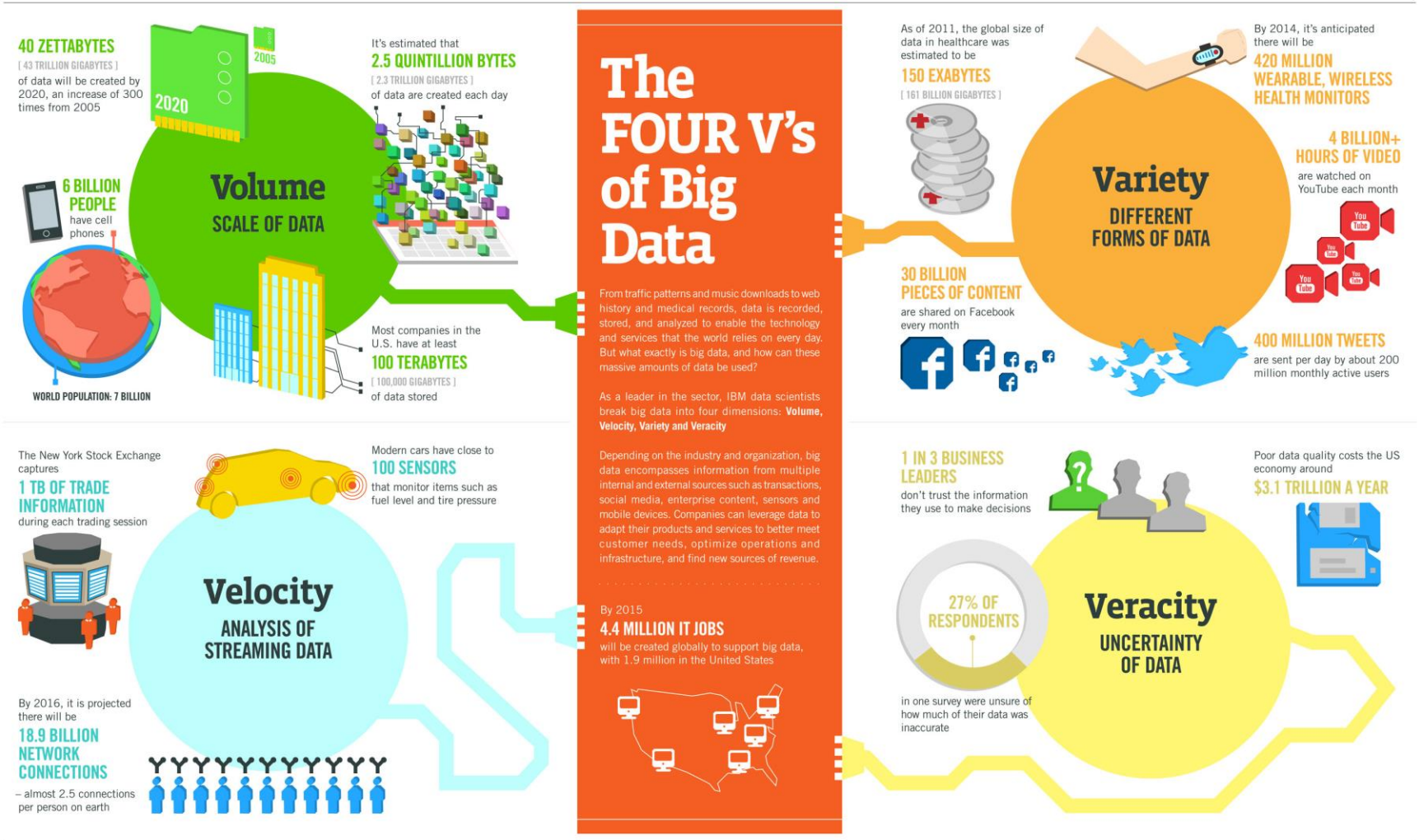
Overview

- What is Big Data?
- Why Big Data?
- Types of Big Data
- Techniques
- Distributed architecture
- Cloud Computing
- Storage and tools

What is Big Data?

- Big Data is high volume, high velocity and high variety of data that require new forms of processing to enable knowledge extraction, insight discovery, decision making, and process optimization

What is Big Data?



What is Big Data?

- The 4 V's of Big Data:

1. **Volume**
   - The main characteristic of Big Data is the volume
   - The volume of data impacts its analysis
   - Historical data is important especially for Business Intelligence
   - Data generated from different sources are stored together to create correlations and extract knowledge.

What is Big Data?

- The 4 V's of Big Data:

2. Variety

  - Variety refers to the types of data available for analysis
  - Multiple types of data: numbers, dates, text, images, video, etc.
  - Multiple sources for data: companies databases, social media, blogs, etc.
  - Structured, unstructured and hybrid types of data.

What is Big Data?

- The 4 V's of Big Data:

3. Veracity

  - Veracity refers to the trustworthiness of the data
  - The quality of the data can affect the analysis process
  - The data must be representative, relevant, consistent, accurate and current to discover patterns
  - The data must be preprocessed to extract relevant knowledge

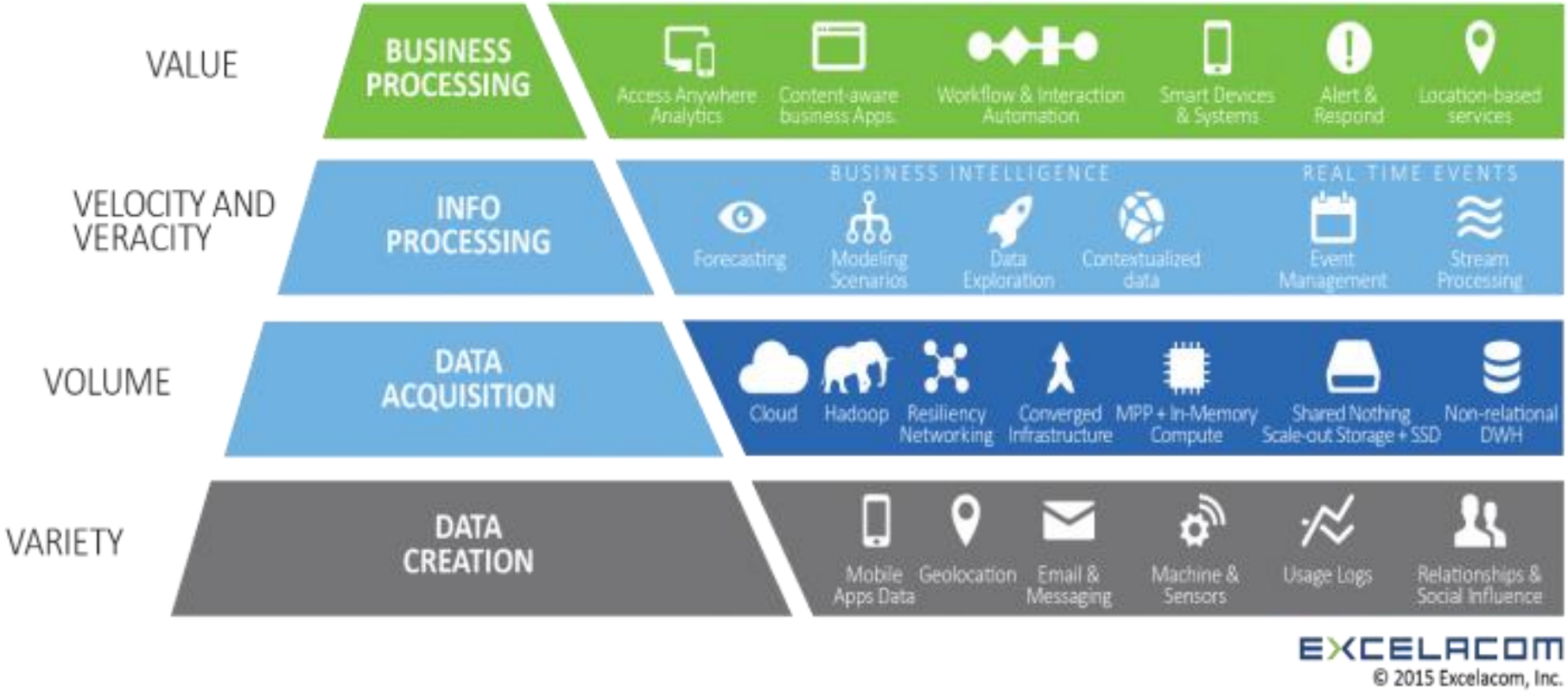
What is Big Data?

- The 4 V's of Big Data:

4. Velocity

  - Velocity refers to how fast the data is processed
  - The faster the date is processed, the faster it can be queried and interpreted to extract knowledge
  - A rapid data analysis helps to correctly and rapidly make and take decision

What is Big Data?



What is Big Data?

- The 8 V's of Big Data:

5. **Value**:

  - A critical objective of Big Data is to use the 4 V to create value

  - Through data analysis, Value is achieved by making the data profitable and help to increase revenue and decrease costs

What is Big Data?

- The 8 V's of Big Data:

6. **Variability** of the data must be also taken into account

   – The meaning of data is constantly changing

   – The context where the data appears must be understood

What is Big Data?

- The 8 V's of Big Data:

7. **Visualization** refers to presentation of data in a pictorial or graphical form

  - Analytics preserved visually
    - To understand difficult concept
    - To identify new patterns
    - Explore the data
    - Detect inconsistencies in the data structure

  - Interactive visualization: charts, graphs, etc.

  - Involves developing new techniques for data visualization

What is Big Data?

- The 8 V's of Big Data:

8. **Vicissitude** refers to the challenges of scaling Big Data complex workflows.

   – MapReduce frameworks

   – Hadoop Ecosystem

   – Spark Environment

   – Blockchain

Overview

- What is Big Data?
- Why Big Data?
- Types of Big Data
- Techniques
- Distributed architecture
- Cloud Computing
- Storage and tools

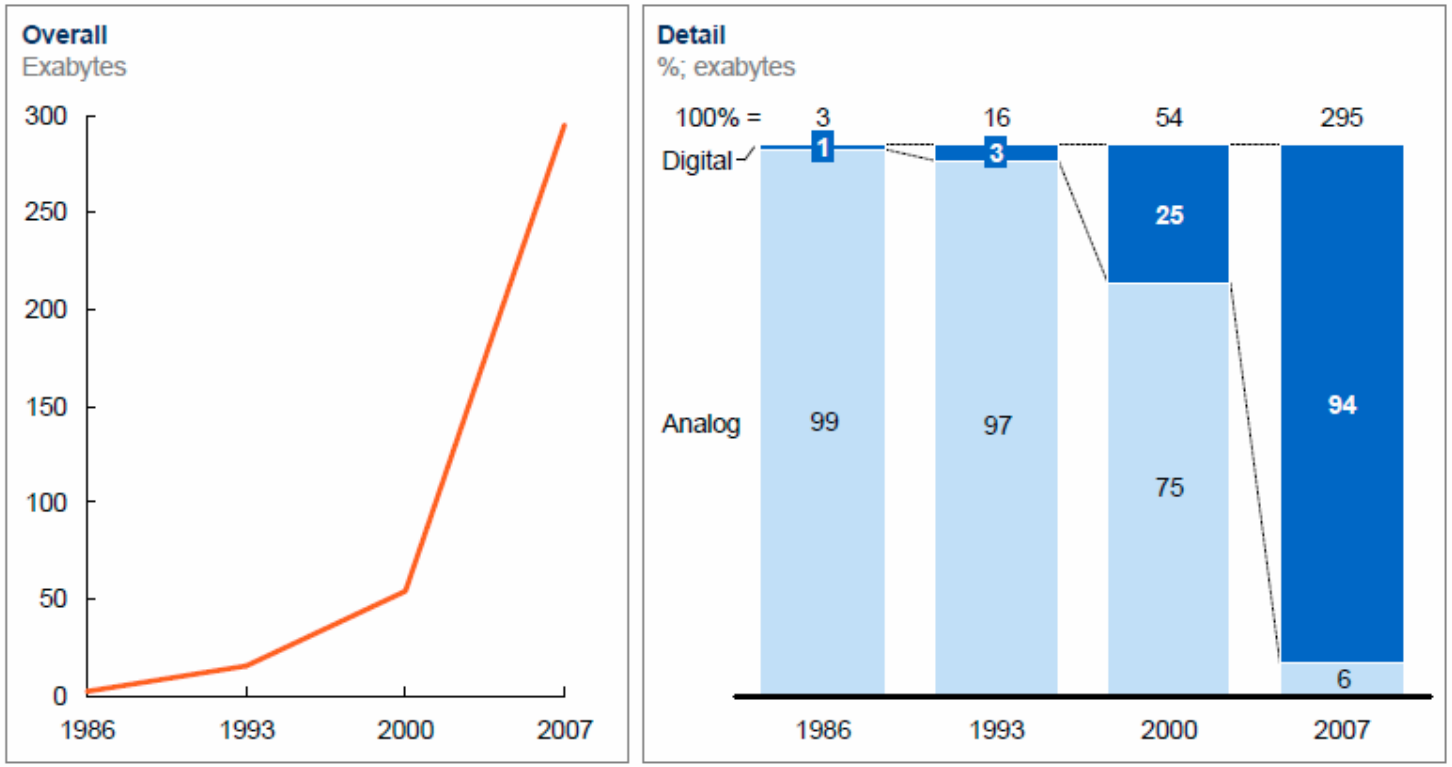
Why Big Data?

- Increased storage capacities
  - Storage capacities have grown
  - Storage (now) is cheap – cost of storage/GB decreased

Why Big Data?

Data storage has grown significantly, shifting markedly from analog to digital after 2000

Global installed, optimally compressed, storage



NOTE: Numbers may not sum due to rounding.

SOURCE: Hilbert and López, "The world's technological capacity to store, communicate, and compute information," *Science*, 2011

Why Big Data?

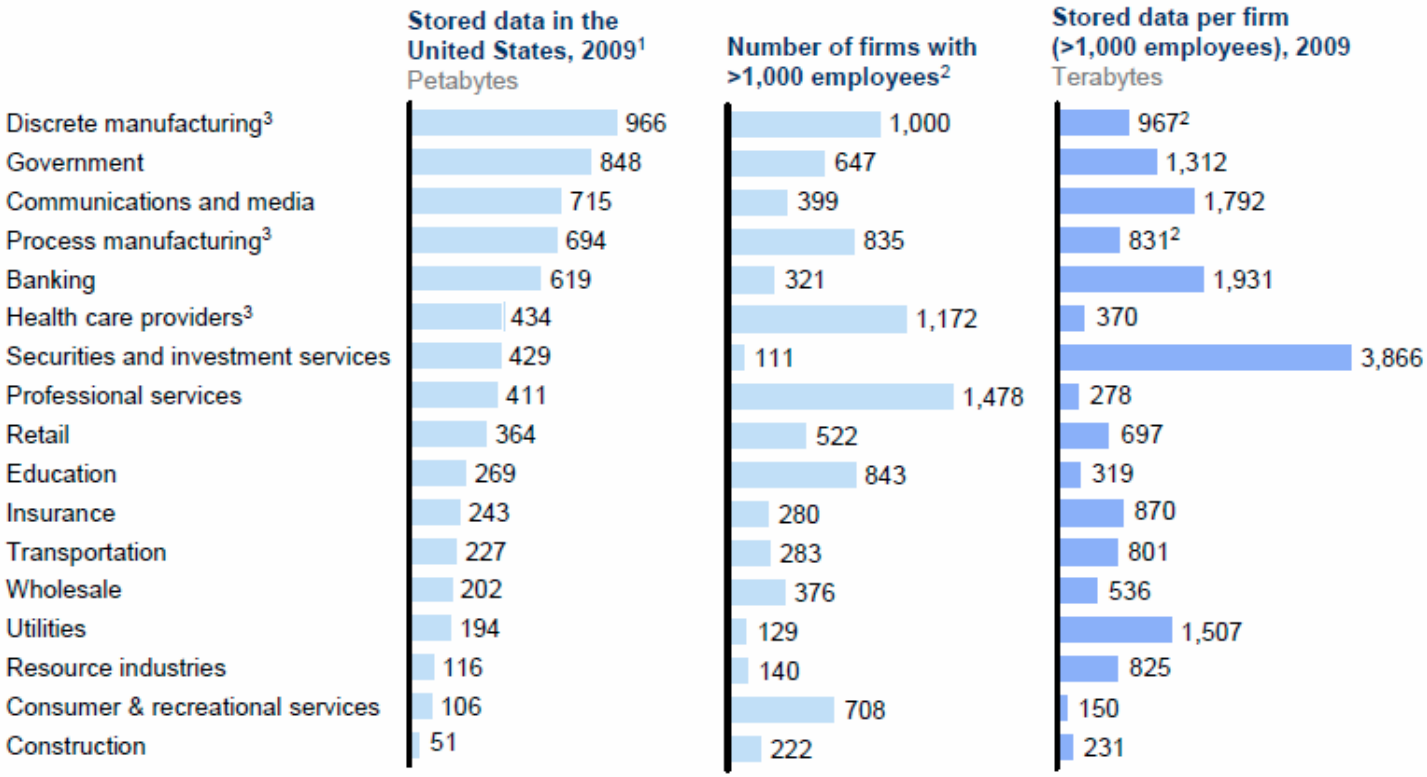
- Increase of processing power
  - Computation capacity has also risen sharply
  - New means of scaling computations:
    - TPUs (Tensor Processing Units)
    - GPUs (Graphics Processing Units)
    - Grid Computing
    - Cloud Computing
    - Hadoop Ecosystem
    - Spark Environment, etc.

Why Big Data?

- Availability of data
  - More data is available
  - Different types of data are available:
    - Private sector (financial corporations, banks, etc.)
    - Social media
    - Internet of Things

Why Big Data?

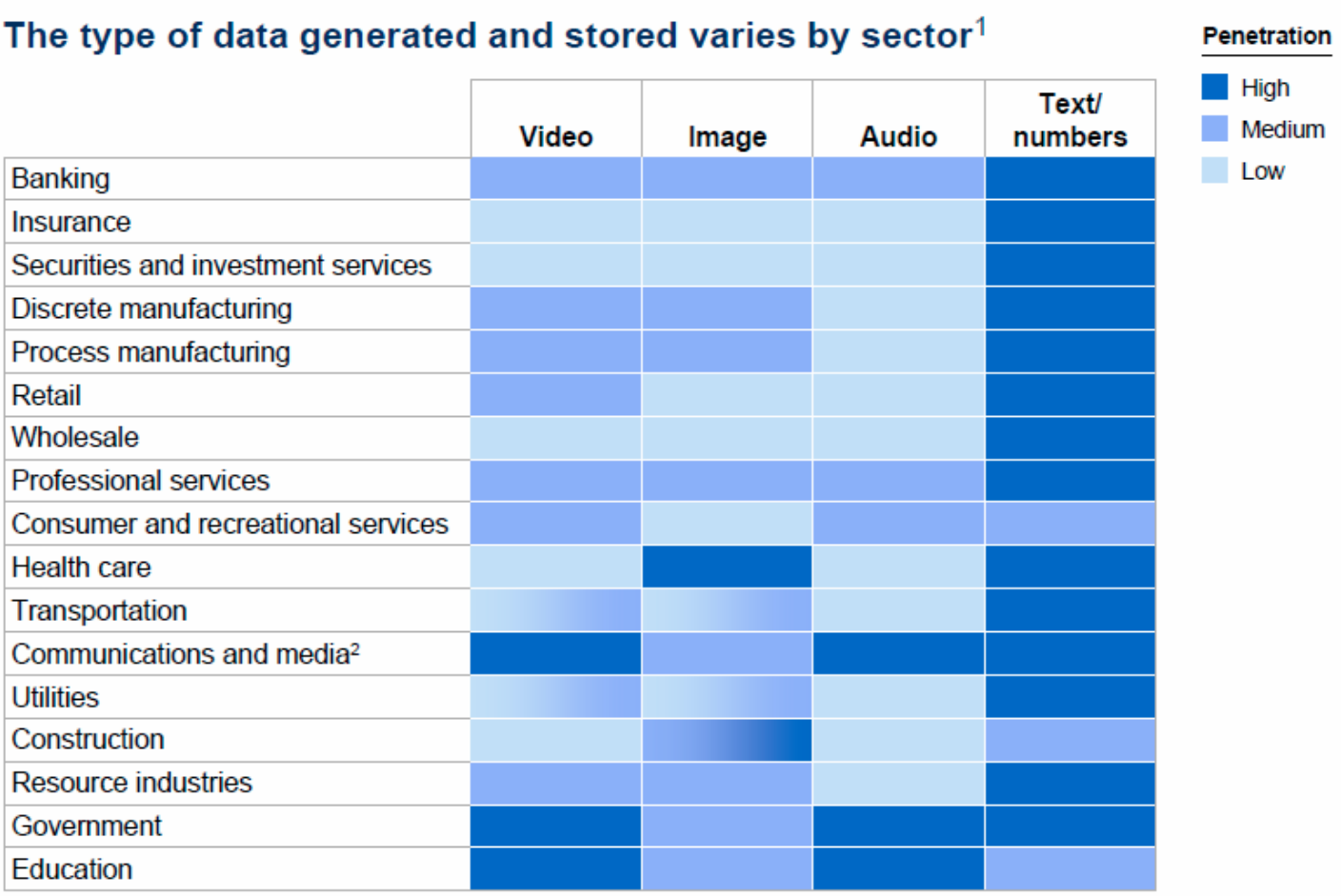
Companies in all sectors have at least 100 terabytes of stored data in the United States; many have more than 1 petabyte



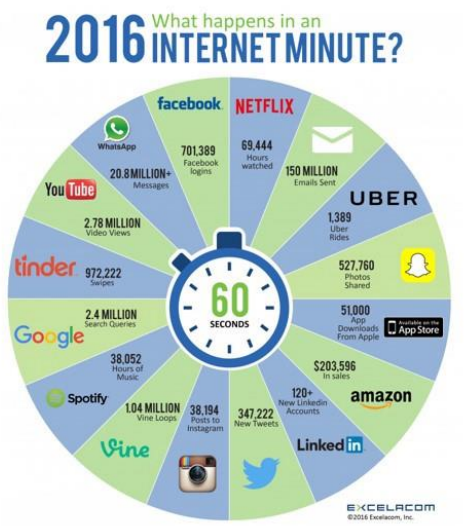
1 Storage data by sector derived from IDC.
2 Firm data split into sectors, when needed, using employment
3 The particularly large number of firms in manufacturing and health care provider sectors make the available storage per company much smaller.

SOURCE: IDC; US Bureau of Labor Statistics; McKinsey Global Institute analysis

Why Big Data?


The type of data generated and stored varies by sector[1]
Penetration
High
Medium
Low
Video
Image
Audio
Text/ numbers
Banking
Insurance
Securities and investment services
Discrete manufacturing
Process manufacturing
Retail
Wholesale
Professional services
Consumer and recreational services
Health care
Transportation
Communications and media[2]
Utilities
Construction
Resource industries
Government
Education
1 We compiled this heat map using units of data (in files or minutes of video) rather than bytes.
2 Video and audio are high in some subsectors.
SOURCE: McKinsey Global Institute analysis

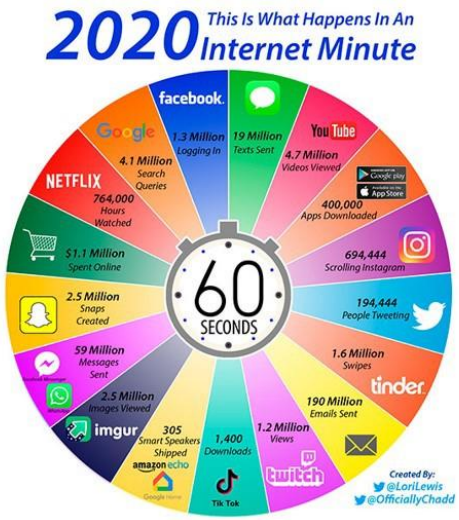
Why Big Data?

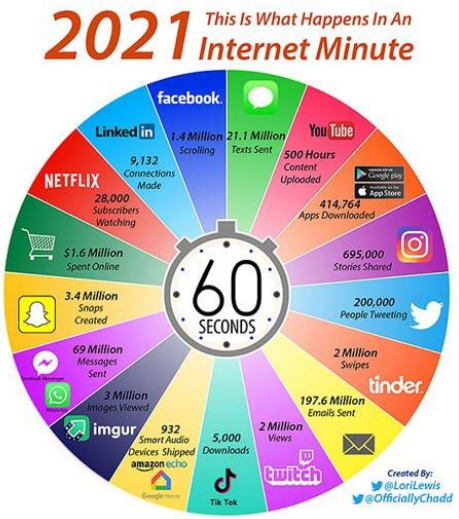

2016 What happens in an INTERNET MINUTE?
60 SECONDS
facebook
NETFLIX
UBER
Google
tinder
Spotify
Vine
Linked in
amazon
YouTube
WhatsApp
Instagram
Twitter
Snapchat
EXCELACOM

2017 This Is What Happens In An Internet Minute
60 SECONDS
facebook
Google
NETFLIX
YouTube
tinder
Linked in
amazon echo
Spotify
Created By:
@LoriLewis
@OfficiallyChadd

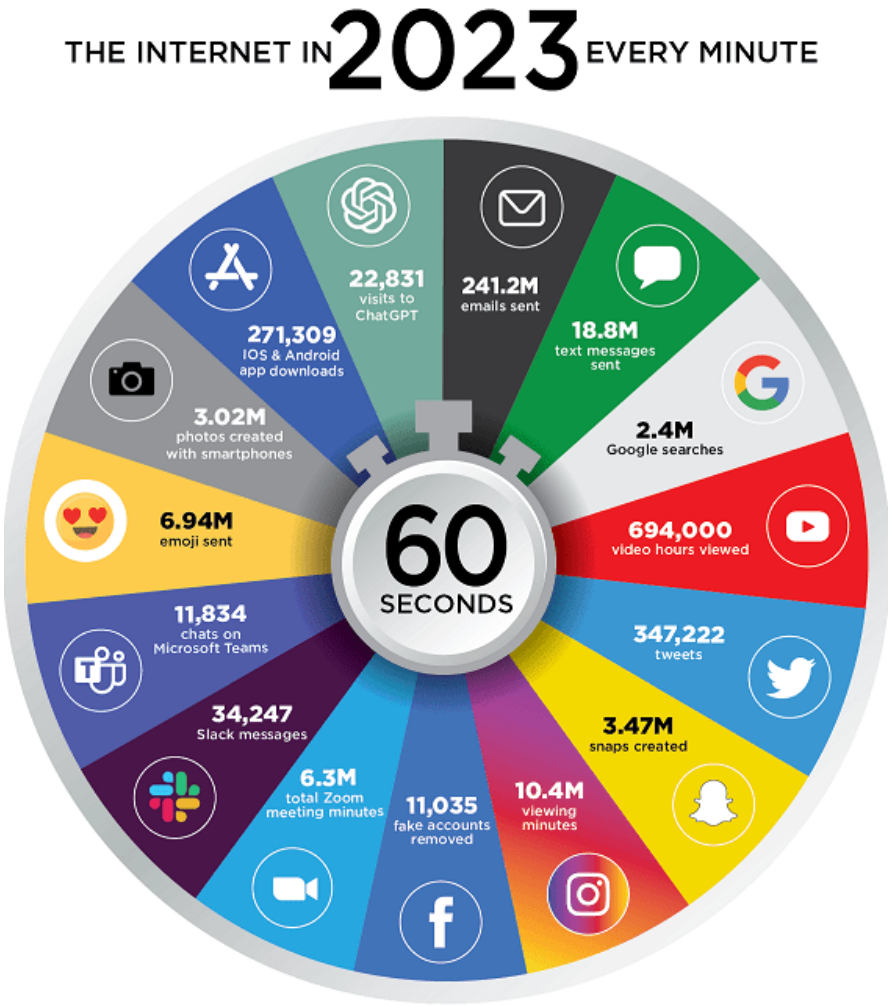
2018 This Is What Happens In An Internet Minute
60 SECONDS
facebook
Google
NETFLIX
YouTube
tinder
Instagram
Twitter
Snapchat
amazon echo
twitch
Created By:
@LoriLewis
@OfficiallyChadd

2019 This Is What Happens In An Internet Minute
60 SECONDS
facebook
Google
NETFLIX
YouTube
tinder
Twitter
Snapchat
GIPHY
amazon echo
twitch
Created By:
@LoriLewis
@OfficiallyChadd

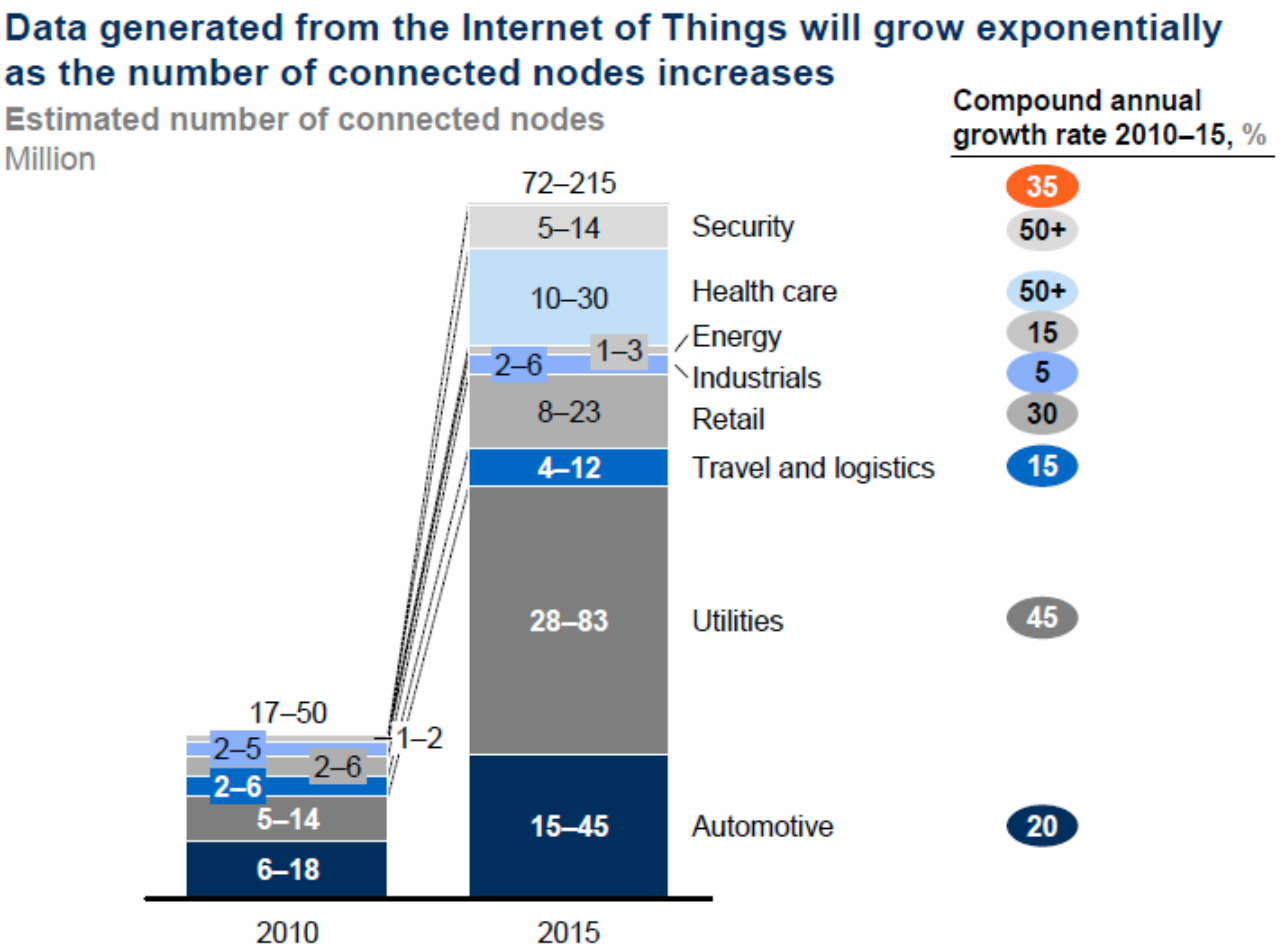

2020 This Is What Happens In An Internet Minute
60 SECONDS
facebook
Google
NETFLIX
YouTube
tinder
Twitter
Snapchat
imgur
amazon echo
Tik Tok
twitch
Created By:
@LoriLewis
@OfficiallyChadd


2021 This Is What Happens In An Internet Minute
60 SECONDS
facebook
Linked in
NETFLIX
YouTube
tinder
Twitter
Snapchat
imgur
amazon echo
Tik Tok
twitch
Created By:
@LoriLewis
@OfficiallyChadd

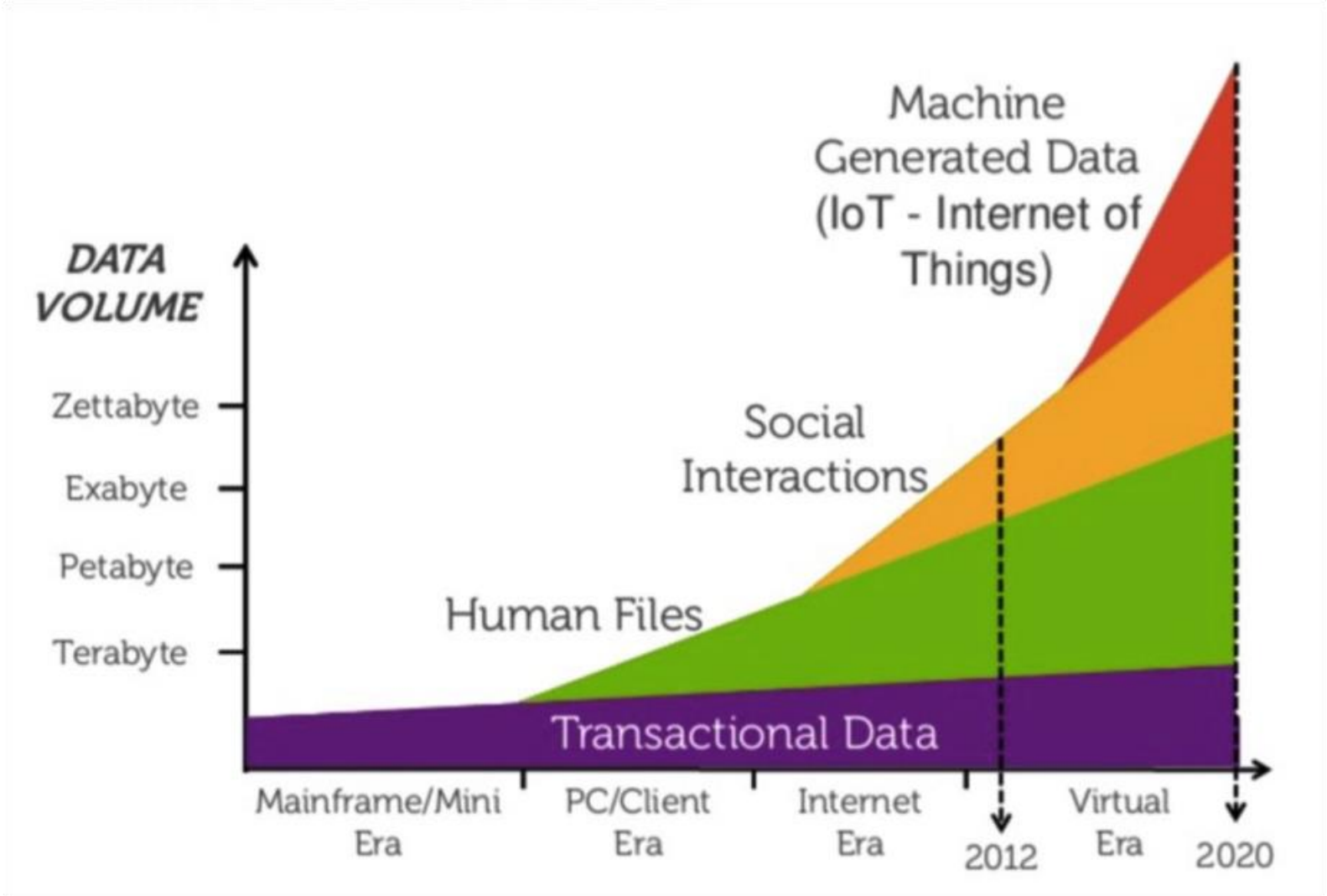
Why Big Data?


THE INTERNET IN 2023 EVERY MINUTE
60 SECONDS
241.2M emails sent
18.8M text messages sent
2.4M Google searches
694,000 video hours viewed
347,222 tweets
3.47M snaps created
10.4M viewing minutes
11,035 fake accounts removed
6.3M total Zoom meeting minutes
34,247 Slack messages
11,834 chats on Microsoft Teams
6.94M emoji sent
3.02M photos created with smartphones
271,309 IOS & Android app downloads
22,831 visits to ChatGPT
Created by: eDiscovery Today & LTMG

Why Big Data?


Data generated from the Internet of Things will grow exponentially as the number of connected nodes increases
Estimated number of connected nodes
Million
Compound annual growth rate 2010–15, %
72–215
5–14
Security
35
50+
10–30
Health care
50+
Energy
15
1–3
2–6
Industrials
5
8–23
Retail
30
4–12
Travel and logistics
15
28–83
Utilities
45
17–50
2–5
1–2
2–6
2–6
5–14
15–45
Automotive
20
6–18
2010
2015
NOTE: Numbers may not sum due to rounding.
SOURCE: Analyst interviews; McKinsey Global Institute analysis

Why Big Data?


DATA VOLUME
Zettabyte
Exabyte
Petabyte
Terabyte
Machine Generated Data (IoT - Internet of Things)
Social Interactions
Human Files
Transactional Data
Mainframe/Mini Era
PC/Client Era
Internet Era
2012
Virtual Era
2020

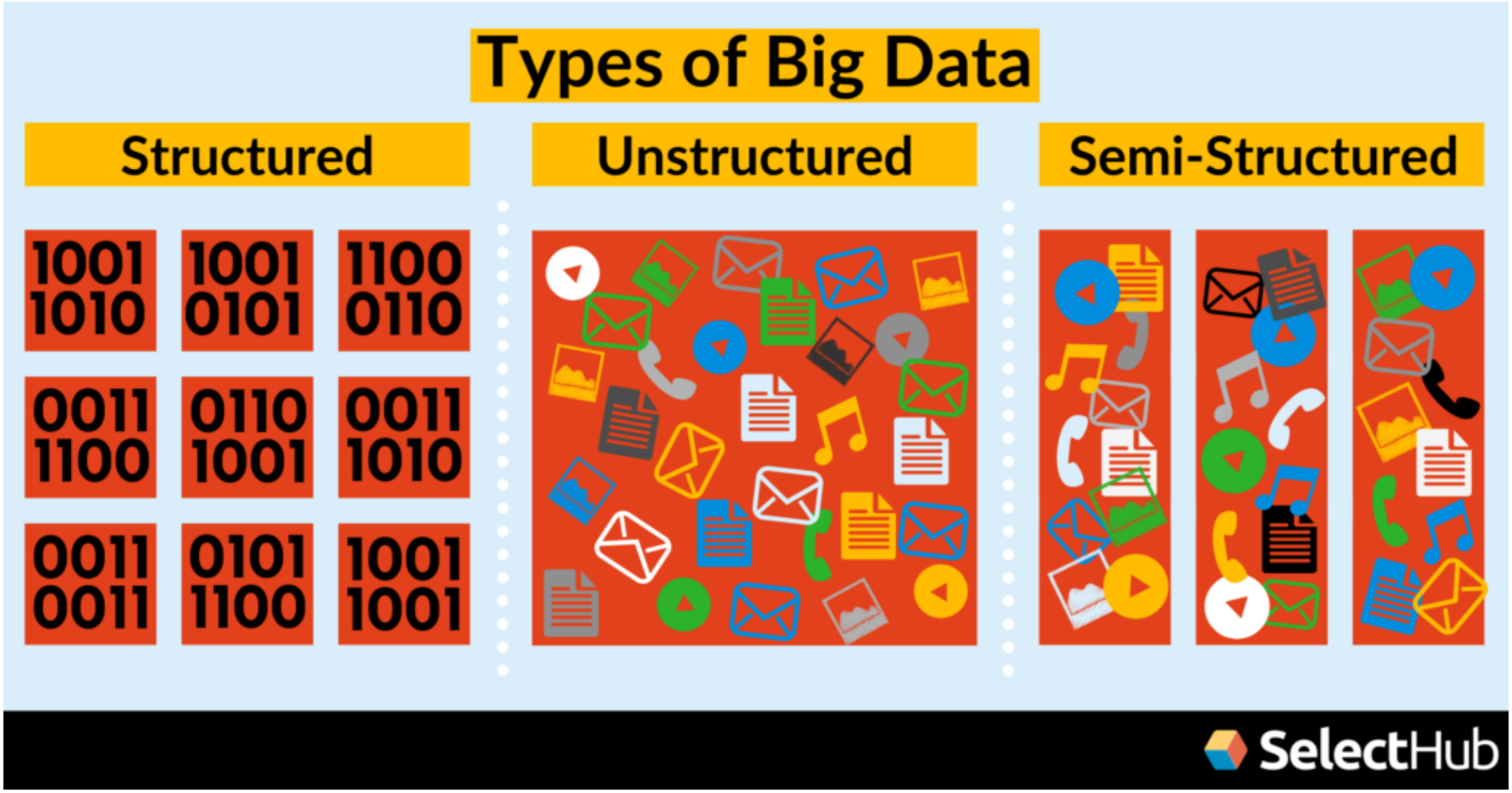
Overview

- What is Big Data?
- Why Big Data?
- Types of Big Data
- Techniques
- Distributed architecture
- Cloud Computing
- Storage and tools

Types of Big Data

- Structured Big Data
- Unstructured Big Data
- Hybrid (Multi-Structured/Semi-Structured) Big data

Types of Big Data



Types of Big Data

- Structured Big Data
  - Refers to data that have a predefined data model
  - Has a strict schema which generally adheres to the Relational Algebra Theory
  - The data set's attributes are predefined and have a strict data type: number, datetime, string, etc.

Types of Big Data

- Structured Big Data sources:
  - Computer or machine generated data is data that generally refers to data that is created by a machine without human intervention.
  - Some examples:
    - Sensor data: generated by sensors, medical devices, GPS, etc.
    - Web log data: data captured by machine's activates, it is generated by servers, applications, networks, etc.
    - Financial data: financial systems use predefined rules that automate their processes, e.g., stock-trading

Types of Big Data

- Structured Big Data sources:
  - Human generated data is data that is generated by the interaction of human with computers
  - Some examples:
    - Input Data: information and data that humans' input in a computer, e.g., when buying an airplane ticket a human will input his name, age, gender, passport ID, etc.
    - Click-stream data: data that is generated when humans click a link on a website. This data is used to analyze and determine customer behavior and buying patterns
    - Gaming related data: used to record the player's moves in a game to understand and develop new wining strategies

Types of Big Data

- Structured Big Data technologies:
  - Relational databases: Oracle, PostgreSQL, MySQL, Microsoft SQL server, Teradata
  - Data Warehouses and Data Marts
  - Enterprise Resource Planning software (ERP)
  - Customer Relationship Management software (CRM)
  - Mainframes

Types of Big Data

- Unstructured Big Data
  - Refers to data that does not have a predefined data model
  - Does not have a strict schema, generally it uses a flexible schema-free model
  - The data set's attributes
    - Are not predefined
    - They do not have a strict data type
  - Employs the use of different data structures: sets, lists, maps, etc.

Types of Big Data

- Unstructured Big Data sources:
  - As in the case of Structured Big Data, they are machine or human generated
  - Some examples of machine generated unstructured data:
    - Satellite images includes weather data or the data the governments captures in their surveillance satellites
    - Scientific data includes seismic imagery, atmospheric data, etc.
    - Photographs and videos includes security, surveillance and traffic videos
    - Radar and sonar data includes meteorological and oceanographic seismic profiles.

Types of Big Data

- Unstructured Big Data sources:
  - Some examples of human generated unstructured data:
    - Text internal to a company: word documents, excel spreadsheets, emails, etc.
    - Social media data: Facebook, Twitter, YouTube, LinkedIn, Instagram, etc.
    - Mobile data: text messages, location information, etc.
    - Website content: News sites, YouTube, Instagram, etc.

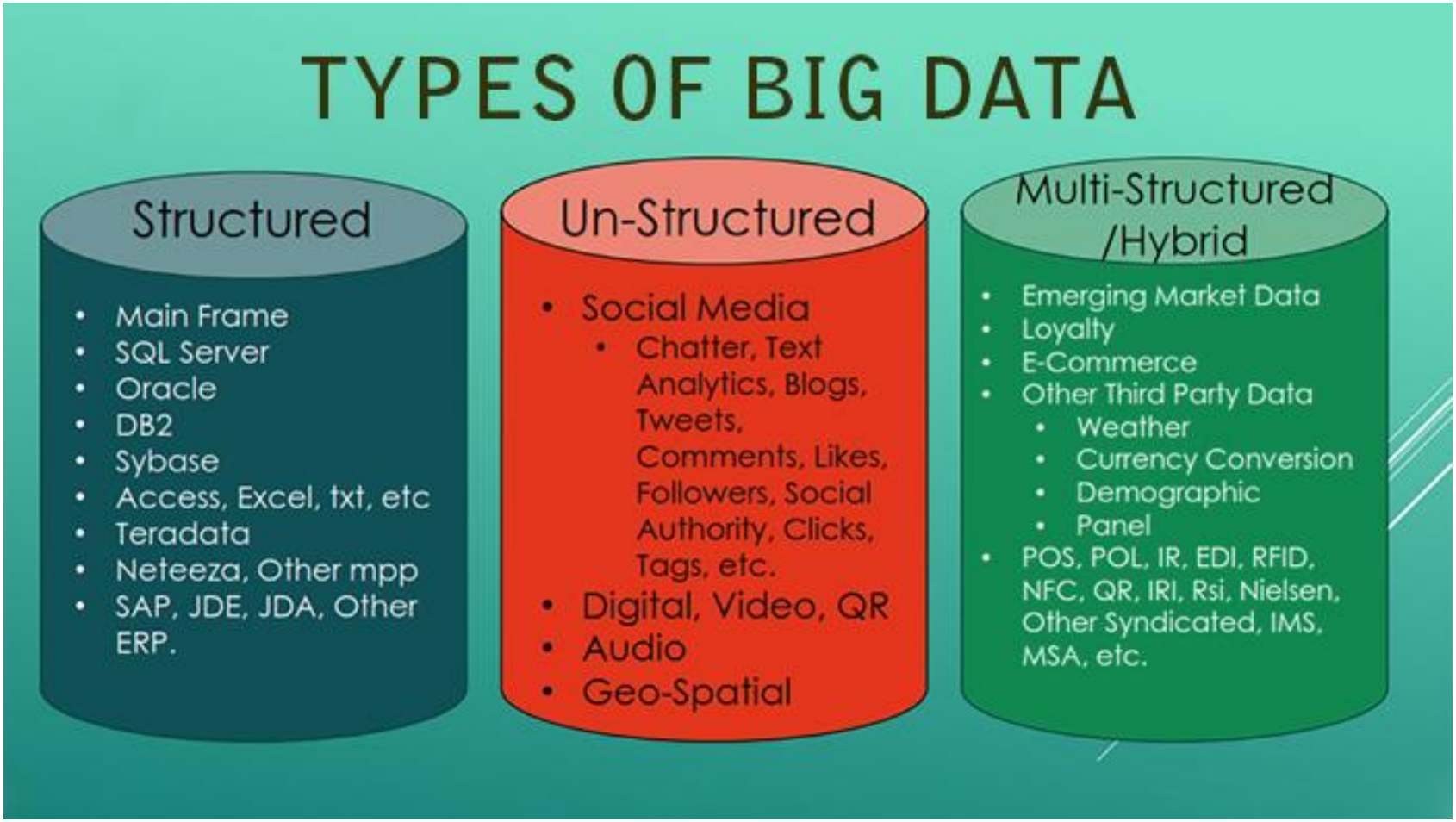
Types of Big Data

- Unstructured Big Data technologies:
  - NoSQL databases: MongoDB, CouchDB, Neo4J, Cassandra, etc.
  - Distributed storage frameworks: Hadoop
  - Data Lakes (a storage repository that stores vast amounts of raw data in its native format).
  - Enterprise Content Management Systems (CMS) that manage the complete life cycle of contend

Types of Big Data

- Hybrid (Multi-structured/Semi-structured) Big Data
  - Combines Structured and Unstructured Big Data
  - Combines the data sources for Structured and Unstructured Big Data and correlates the information
  - Combines the technologies used for structured and unstructured Big Data

Types of Big Data



Types of Big Data

Overview

- What is Big Data?
- Why Big Data?
- Types of Big Data
- Techniques
- Distributed architecture
- Cloud Computing
- Storage and tools

Techniques

- When Big data is a problem?

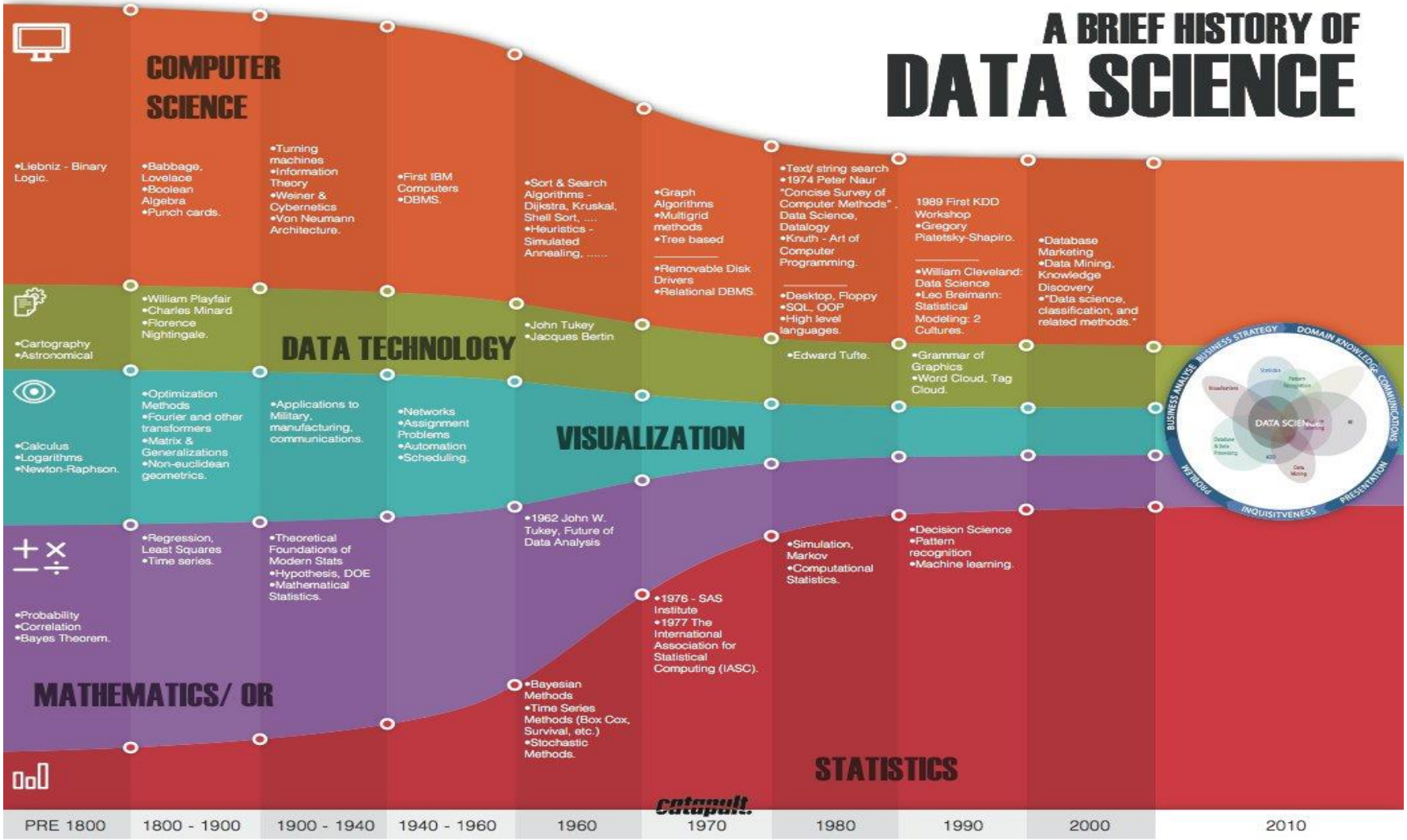
1. When the operations on the data are complex:
   - Modeling and reasoning
   - Knowledge extraction
   - Pattern extraction

2. When usual algorithms cannot handle big amounts of data:
   - Performance and query execution time
   - Parallelism and distribution
   - Resource usage (CPU, RAM)

Techniques

- Big Data combines multiple research fields:
  - Databases
  - Knowledge Discovery in Databases (KDD)
  - Data Mining and Text Mining
  - Parallel and distributed algorithms
  - High Performance Computing (HPC)
  - Information Retrieval (IR) and subdomains: Multi-Lingual Information Retrieval (MLIR)
  - Natural Language Processing (NLP)
  - Semantic Web
  - Probabilities and Statistics
  - Machine Learning
  - Etc.

Techniques


A BRIEF HISTORY OF
DATA SCIENCE
COMPUTER SCIENCE
•Liebniz - Binary Logic.
•Babbage, Lovelace •Boolean Algebra •Punch cards.
•Turning machines •Information Theory •Weiner & Cybernetics •Von Neumann Architecture.
•First IBM Computers •DBMS.
•Sort & Search Algorithms - Dijkstra, Kruskal, Shell Sort, ... •Heuristics - Simulated Annealing, ...
•Graph Algorithms •Multigrid methods •Tree based
•Removable Disk Drivers •Relational DBMS.
•Text/ string search •1974 Peter Naur "Concise Survey of Computer Methods", Data Science, Datalogy •Knuth - Art of Computer Programming.
•Desktop, Floppy •SQL, OOP •High level languages.
1989 First KDD Workshop •Gregory Piatetsky-Shapiro.
•William Cleveland: Data Science •Leo Breimann: Statistical Modeling: 2 Cultures.
•Database Marketing •Data Mining, Knowledge Discovery •"Data science, classification, and related methods."
DATA TECHNOLOGY
•Cartography •Astronomical
•William Playfair •Charles Minard •Florence Nightingale.
•John Tukey •Jacques Bertin
•Edward Tufte.
•Grammar of Graphics •Word Cloud, Tag Cloud.
VISUALIZATION
•Calculus •Logarithms •Newton-Raphson.
•Optimization Methods •Fourier and other transformers •Matrix & Generalizations •Non-euclidean geometrics.
•Applications to Military, manufacturing, communications.
•Networks •Assignment Problems •Automation •Scheduling.
MATHEMATICS/ OR
•Probability •Correlation •Bayes Theorem.
•Regression, Least Squares •Time series.
•Theoretical Foundations of Modern Stats •Hypothesis, DOE •Mathematical Statistics.
•1962 John W. Tukey, Future of Data Analysis
•Bayesian Methods •Time Series Methods (Box Cox, Survival, etc.) •Stochastic Methods.
•1976 - SAS Institute •1977 The International Association for Statistical Computing (IASC).
•Simulation, Markov •Computational Statistics.
•Decision Science •Pattern recognition •Machine learning.
STATISTICS
catapult
PRE 1800
1800 - 1900
1900 - 1940
1940 - 1960
1960
1970
1980
1990
2000
2010

Overview

- What is Big Data?
- Why Big Data?
- Types of Big Data
- Techniques
- Distributed architecture
- Cloud Computing
- Storage and tools

Distributed architectures

- A distributed architecture is a model in which resources located on network computers are put together in a resource pool to achieve a common goal.

- A cluster is a set of tightly connected computers that work together so that, in many respect they can be seen as a single system

Distributed architectures

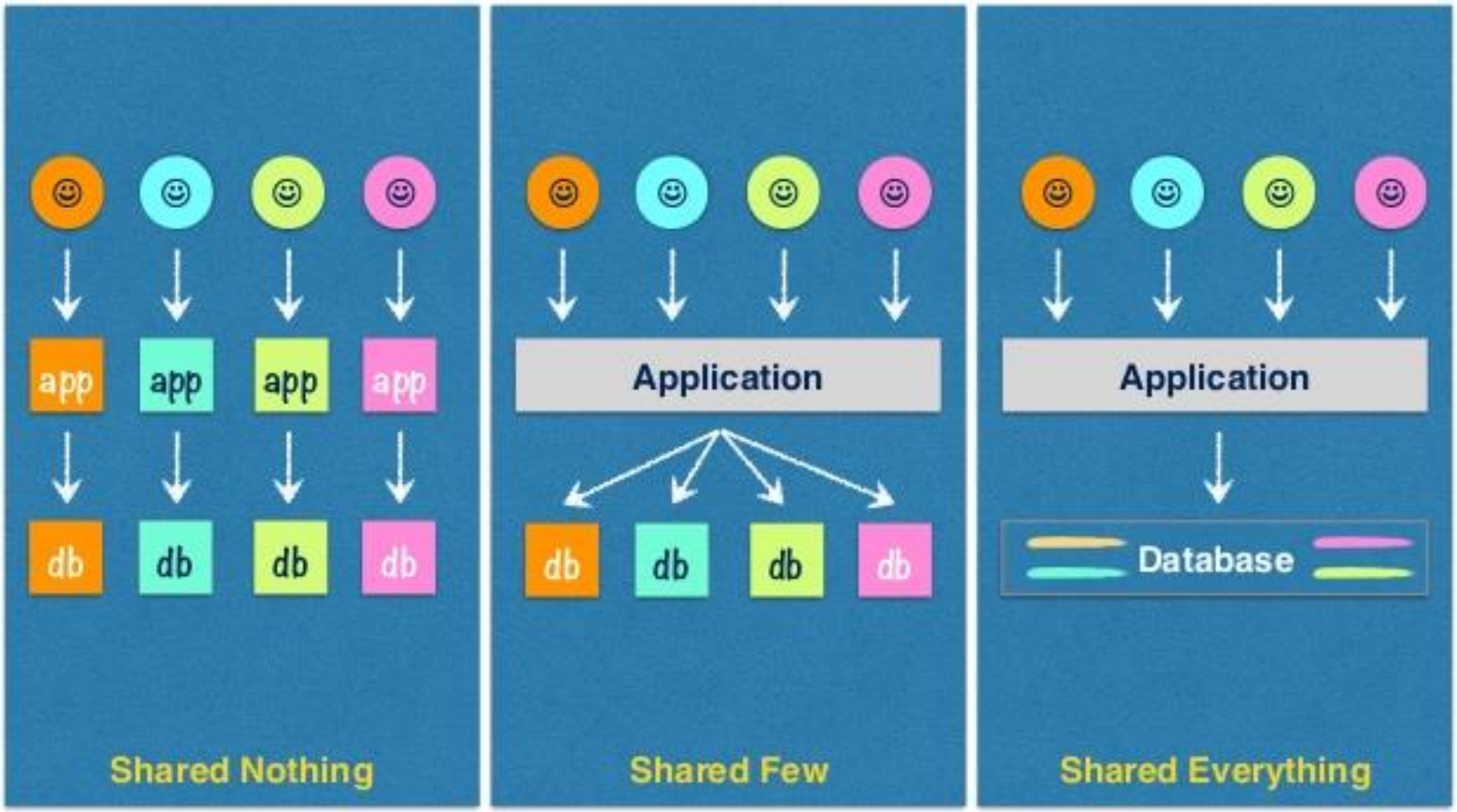
- Scalability is the capability of a system to handle a growing amount of work

- Types of scaling
  - Vertical scaling (scale up/down)
    - Means that you can scale by adding more resources to an existing machine, e.g., CPU, RAM, HDD, etc.
  - Horizontal scaling (scale out/in)
    - Means that you can scale by adding more machines into the resource pool, e.g., add nodes to the cluster.

Distributed architectures

- Resource sharing in a cluster
  - Shared everything architecture
    - Is a distributed computer architecture in which each node is added its resources to the resource pool
    - More specifically, the user sees the total amount of memory, CPU or disk storage and not individual amounts
  - Shared nothing architecture
    - Is a distributed computer architecture in which each node is independent and self sufficient
    - More specifically, none of the nodes share memory, CPU or disk storage

Distributed architectures

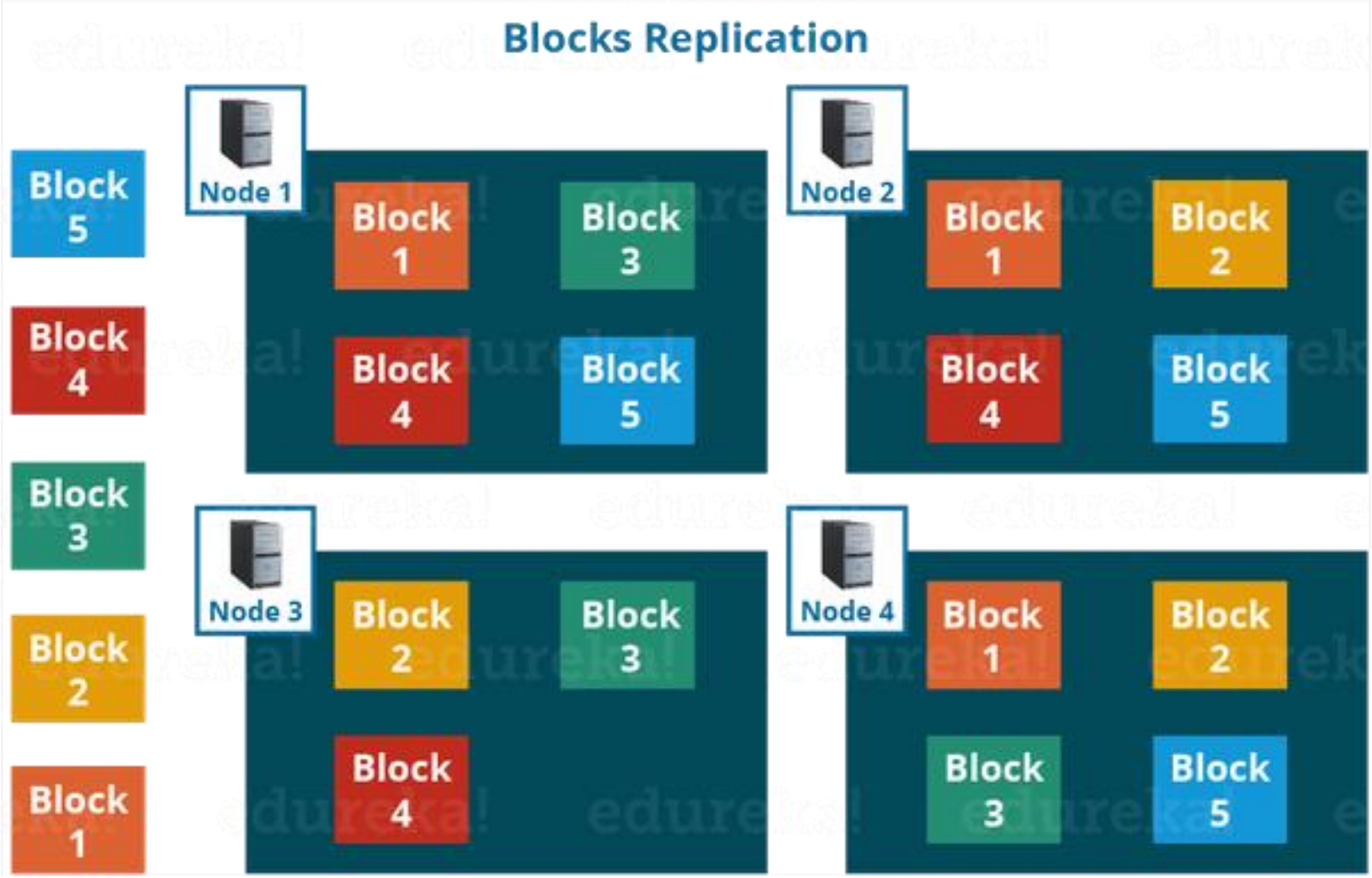


Distributed architectures

- Replication involves sharing information to
  - Ensure consistency between redundant resources
  - Improve reliability, fault-tolerance, or accessibility

- Replication is archived when the same data is stored on multiple storage devices

Distributed architectures



Distributed architectures

- Replication from the architecture perspective
  - Master-Slave (Primary-Secondary)
    - The Master node supports all the operations, i.e., create, read, update, delete (CRUD)
    - Only the Master node can start a transaction
    - Only the read operation is supported on the Slave node
  - Multi-Master (Multi-Primary)
    - All the nodes support all the CRUD operations
    - All the nodes can start a transaction

Distributed architectures

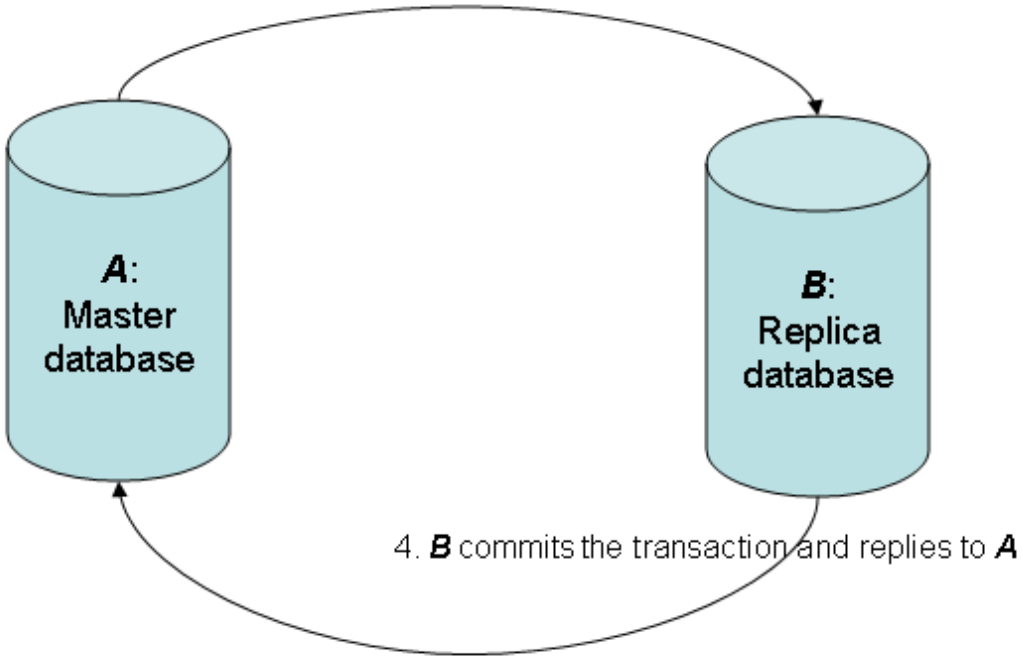
- Types of replication from the transaction perspective
  - Synchronous replication
    - Guarantees 0 data loss, i.e., a transaction either is completed on all the nodes or not at all
    - A transaction is not considered complete without acknowledgment from all the nodes
    - Applications wait for a write transaction to complete before proceeding with further work, hence overall performance decreases considerably.

Distributed architectures

- Types of replication from the transaction perspective
  - Asynchronous replication
    - Transaction is considered complete as soon as the local storage acknowledges it
    - Remote nodes are updated but with (a small) lag
    - Performance is greatly increased, but in case of losing a local storage, the remote storage is not guaranteed to have the current copy of data and most recent data may be lost.

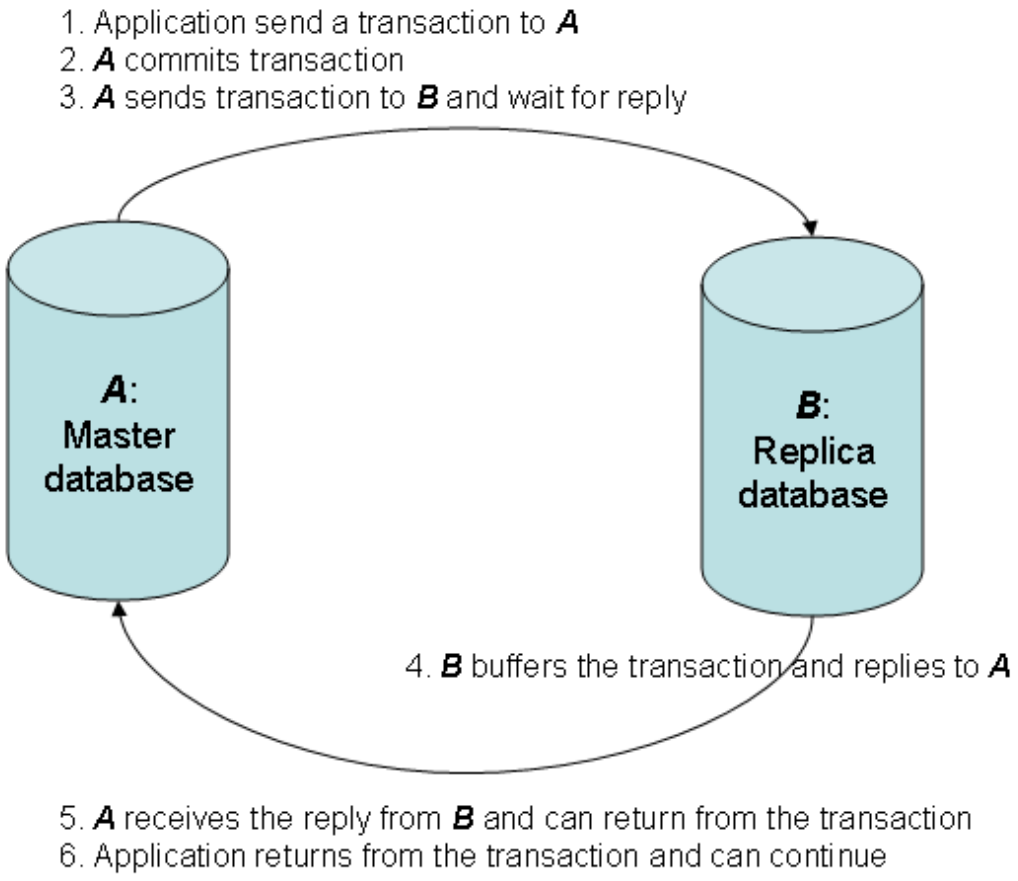
Synchronous replication

1. Application send a transaction to ***A***
2. ***A*** commits transaction
3. ***A*** sends transaction to ***B*** and wait for reply

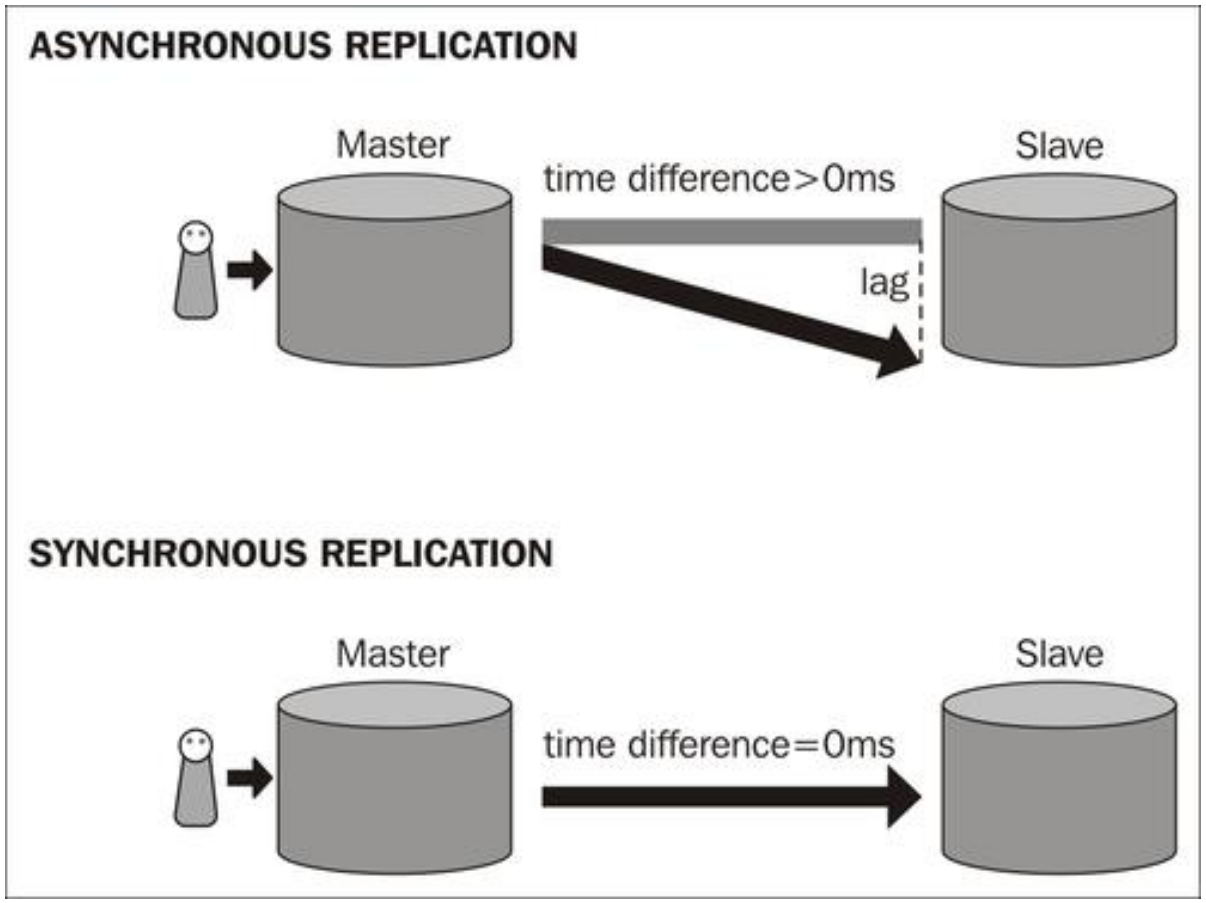


5. ***A*** receives the reply from ***B*** and can return from the transaction
6. Application returns from the transaction and can continue

Asynchronous replication



Distributed architectures



Overview

- What is Big Data?
- Why Big Data?
- Types of Big Data
- Techniques
- Distributed architecture
- Cloud Computing
- Storage and tools

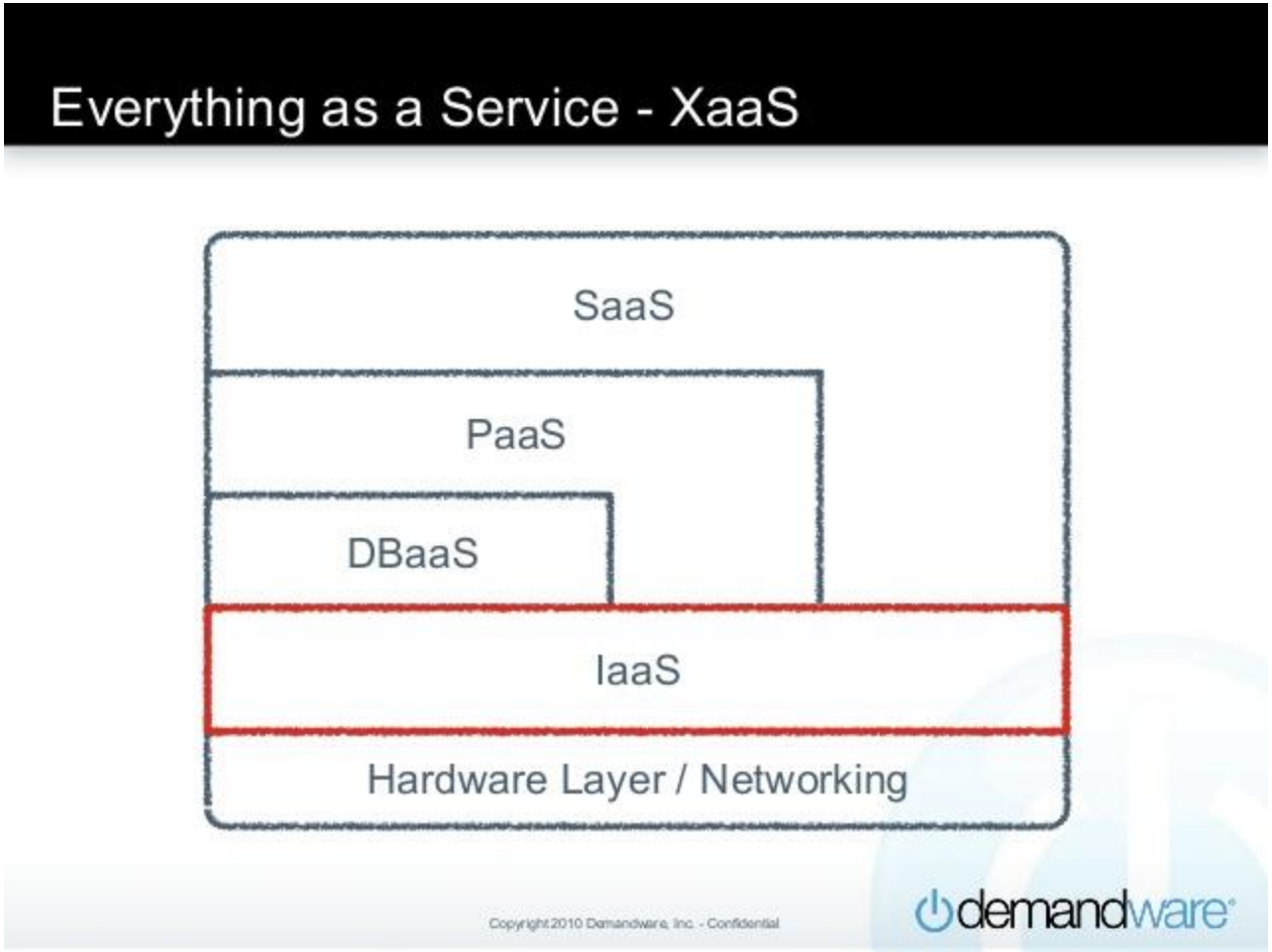
Cloud Computing

- Cloud Computing
  - Is the delivery of on-demand computing resources over the internet on a pay-for-use basis
  - The resources available include everything from data centers to applications
  - Often referred as simply “the cloud”

Cloud Computing

- Cloud Computing
  - Infrastructure as a Service (IaaS)
  - Database as a Service (DBaaS)
  - Platform as a Service (PaaS)
  - Software as a Service (SaaS)

Cloud Computing



Cloud Computing

- IaaS (Cloud Computing)
  - Is an instant computing infrastructure provisioned and managed over the internet.
  - It provides virtual computing resources over the internet

Cloud Computing

- IaaS features:
  - Provision fundamental computing resources in the form of Virtual Machines (VM)
  - Resources are distributed and support dynamic scaling
  - Deploy and run arbitrary software, meaning middleware, and operating systems;
  - Control over operating systems, storage, and deployed applications
  - User does not manage or control the underlying Cloud infrastructure

Cloud Computing

- DBaaS (Cloud Computing)
  - Is an architectural and operational approach enabling DBAs to deliver database functionality as a service to internal and/or external users
  - Some IT specialists consider it as a part of PaaS, others don’t

Cloud Computing

- DBaaS features:
  - Pre-configured, automatically installed database services for users
  - Management of database instances using on demand, self-service mechanisms to the users
  - Maximum availability

Cloud Computing

- PaaS (Cloud Computing):
  - Provides a platform allowing users to develop, run, and manage applications without the complexity of building and maintaining the infrastructure typically associated with developing and launching an application.
  - Is a complete development and deployment environment in the cloud, with resources that enable you to deliver everything from simple cloud-based apps to sophisticated, cloud-enabled enterprise applications

Cloud Computing

- PaaS features:
  - Deploy applications onto the Cloud infrastructure
  - Integrated development framework: programming languages, tools and libraries are supported by the provider
  - Analytics and or business intelligence tools are provided as a service with PaaS to allow users to analyze and mine their data
  - Control over the deployed applications

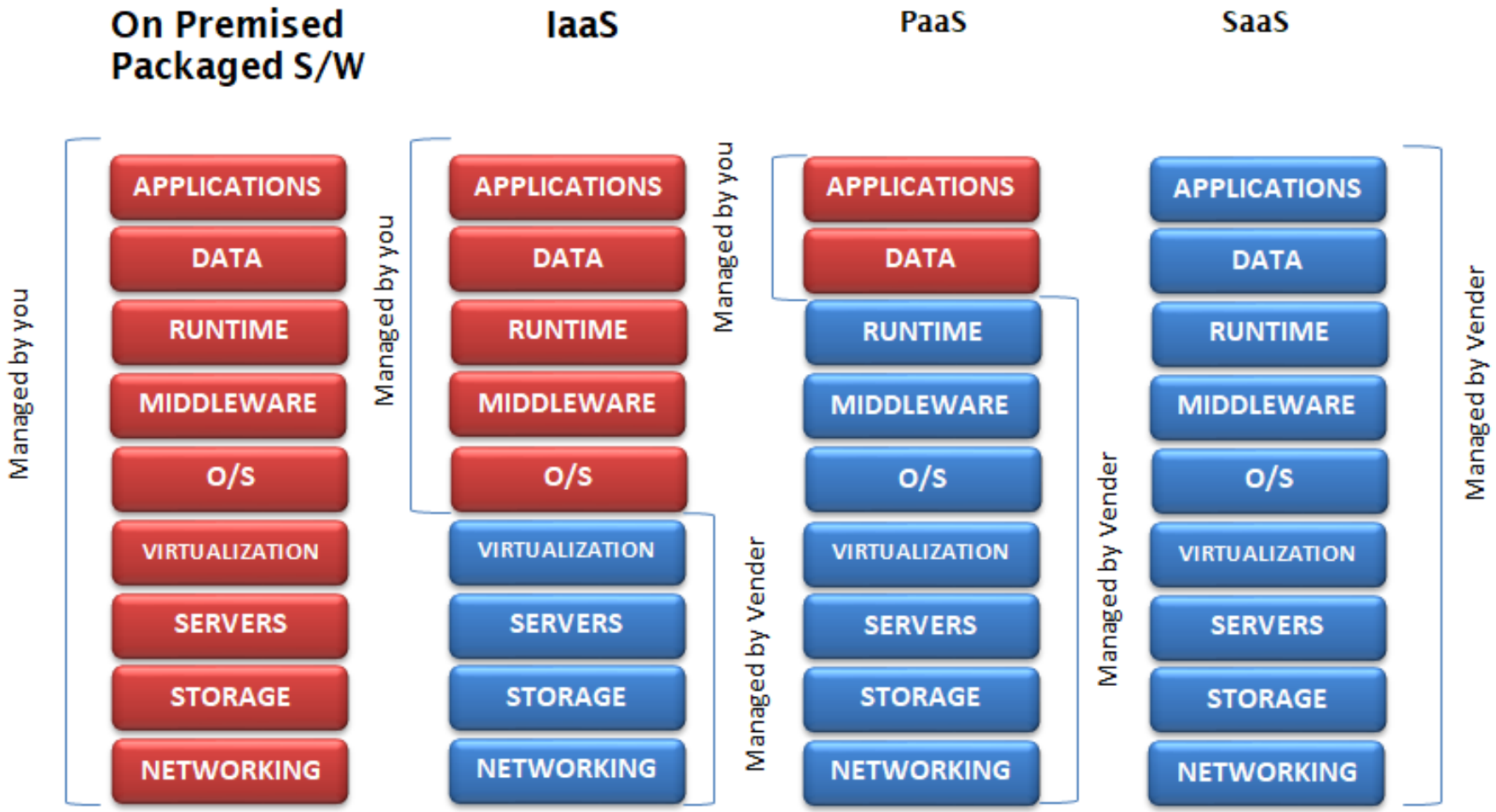
Cloud Computing

- SaaS (Cloud Computing):
  - Allows users to connect to and use cloud-based applications over the Internet
  - It is sometimes referred to as “on-demand software”

Cloud Computing

- SaaS features:
  - Applications are supplied by the cloud provider
  - Applications are accessible from various client devices (e.g. web browsers, APIs);
  - Examples: GMAIL, Dropbox, Office 365

Cloud Computing



Cloud Computing

- Deployment models for Cloud Computing:
  - Private cloud
  - Public cloud
  - Hybrid cloud

Cloud Computing

- Deployment models for Cloud Computing :
  - Private cloud
    - Single-tenant architecture
    - On-premise hardware
    - Direct control of infrastructure
    - E.g., OpenStrack, VMware ESXi, Kubernetes

Cloud Computing

- Deployment models for Cloud Computing :
  - Public cloud
    - Multi-tenant architecture
    - Pay-as-you-go pricing model
    - E.g., Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform

Cloud Computing

- Deployment models for Cloud Computing :
  - Hybrid cloud
    - Cloud bursting capabilities
    - Benefits both public and private environments
    - Combination of both public and private providers

Overview

- What is Big Data?
- Why Big Data?
- Types of Big Data
- Techniques
- Distributed architecture
- Cloud Computing
- Storage and tools

Storage and tools

- Relational and NoSQL databases
- MapReduce
- Hadoop Ecosystem
- Spark Environment

Storage and tools

- Database (DB) is an organized collection of data. The collection of data is organized in schemas, tables, reports, views and other object.

- Database Management Systems (DBMS) is a computer software application that interacts with the user, other applications, and the database itself to capture and analyze data.

- A general-purpose DBMS is designed to allow the definition, creation, querying, update, and administration of databases.

Storage and tools

- There are multiple types of DBMS, usually classified by the way they store data:
  - Relational database management systems (RDBMSs): Oracle, Microsoft SQL Server, MySQL, PostgreSQL, IBM DB2, Hive
  - XML Databases: BaseX, eXist-db
  - Object-oriented DBMSs: ObjectDB, Caché
  - NoSQL DBMSs:
    - Key-Value DBMSs: **Riak**, Redis
    - Document-oriented DBMSs: **MongoDB**, Apache CouchDB
    - Column-oriented DBMSs: Apache HBase, **Cassandra**
    - Graph DBMSs: Neo4J

Storage and tools

- XML Databases
  - Stores the information in XML format
  - This data can be queried, transformed, exported and returned to a calling system
  - To manipulate the data, different technologies are used:
    - XQuery (XML Query) is a language used to query and transform collations of structured and unstructured data in XML format
    - XPath (XML Path Language) is a query language used for selecting nodes from a XML document
    - XSLT (Extensible Stylesheet Language Transformations) is a language for transforming XML documents into other XML documents or formats (e.g. HTML)

Storage and tools

- Object Oriented Databases:
  - Stores the information in the form of objects
  - Supports Object Oriented Programming principles: Polymorphism, Inheritance and Encapsulation
  - Are integrated to work with different frameworks, e.g., ObjectDB is integrated in Java EE (J2EE) and Spring web application and can be deployed on servlet containers (Tomcat, Jetty) as well as J2EE application servers (GlassFish, JBoss).
  - Support of different query languages, e.g., JDOQL (Java Data Objects Query language – based on the Java syntax), JPSQL (JPA Query Language), etc.

Storage and tools

- NoSQL Databases

Type	Example
Key-Value Store	
Wide Column Store	
Document Store	
Graph Store	

Storage and tools

- NoSQL Databases Motives
  - Avoidance of unneeded complexity
    - Avoid strict data consistency
    - ACID properties for transactions are too restrictive
  - High throughput
    - NoSQL database provide a significantly higher data throughput than traditional RDBMSs
  - Easy horizontal scaling
  - Avoidance of expensive object-relational mapping
  - Decrease the complexity and cost of setting up database clusters
  - Compromising reliability for better performance
  - Moving to the cloud

Storage and tools

- NoSQL: Key-Value Databases
  - Stores the information as a map/dictionary
  - Allows clients to put and request values per key
  - Favor high scalability over consistency
  - Omit rich ad-hoc querying and analytics features (join and aggregate operations are set aside)

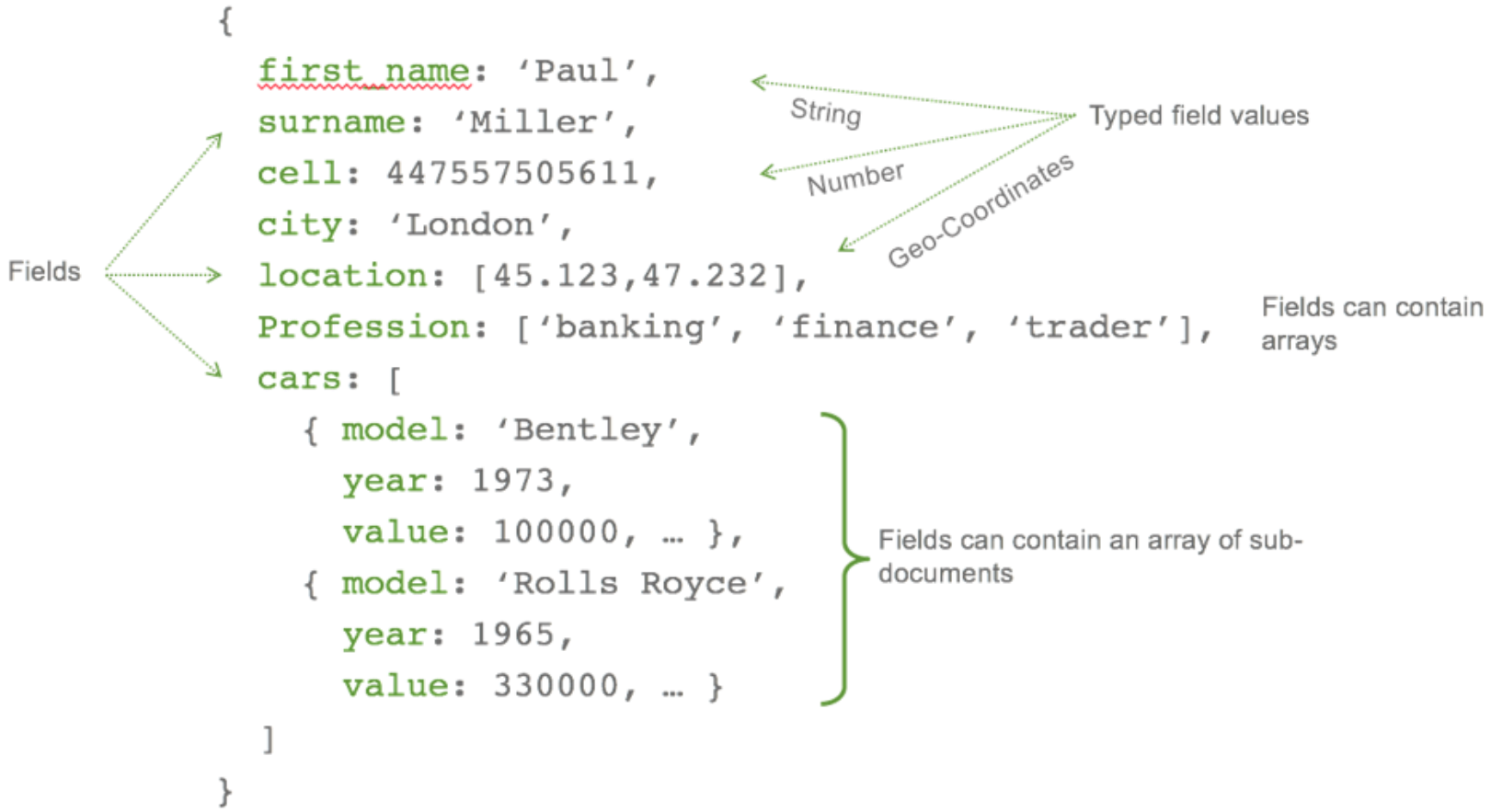
Key	Value
K1	AAA,BBB,CCC
K2	AAA,BBB
K3	AAA,DDD
K4	AAA,2,01/01/2015
K5	3,ZZZ,5623

Storage and tools

- NoSQL: Document-Oriented Databases
  - The next logical step from Key-Value Databases
  - Are schema-less semi-structured databases
  - Support datatypes and collections type (integers, floats, strings, datetime, array, objects, etc.)
  - Supports nested documents
  - MongoDB supports atomic update.

Storage and tools

- NoSQL: Document-Oriented Databases



Storage and tools

- NoSQL: Column-Oriented Databases
  - Are used to store and process data by column instead of row
  - Its origin is in analytics and business intelligence
  - Column-stores operate in a shared-nothing massively parallel processing architecture
  - Can be used to build high-performance applications

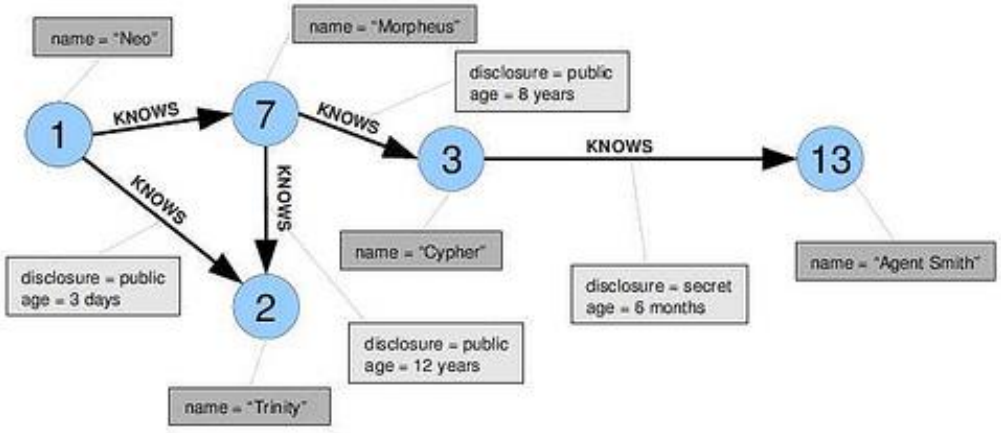
Storage and tools

- NoSQL: Column-Oriented Databases

KEY	COLUMN FAMILIES	
ID	CUSTOMERINFO	ADDRESSINFO
1001	FirstName: Tom MiddleName: T LastName: Tester	Address1: 2001 Bayfront Dr. Address2: Suite#813 City: Tampa State: FL Zip: 34637 Country: US
1002	FirstName: Bob MiddleName: B LastName: Builder	Address1: 1234 Sunny Circle City: Beverly Hills State: CA Zip: 90210

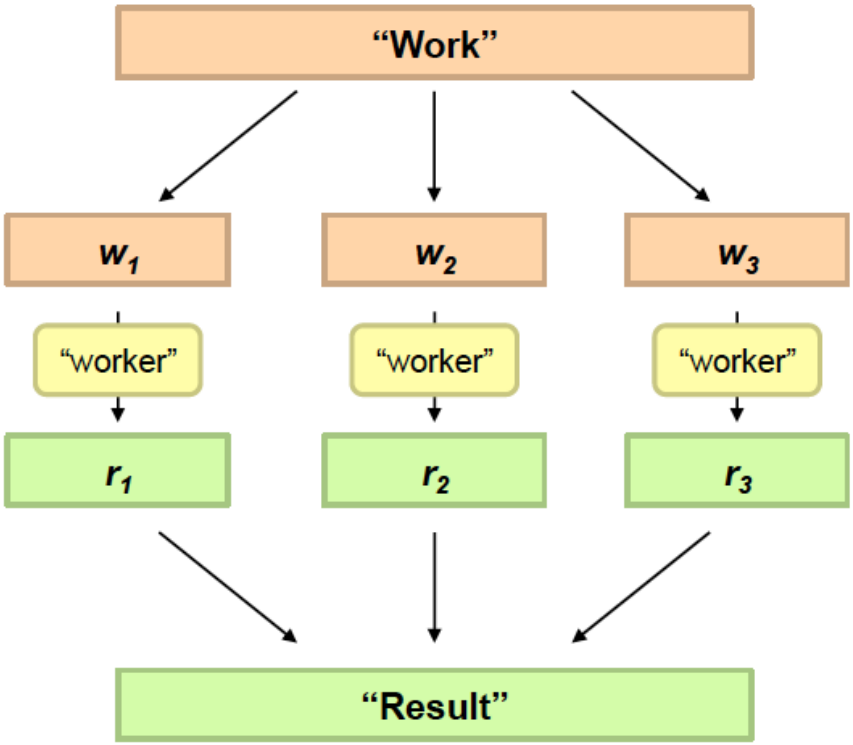
Storage and tools

- NoSQL: Graph Databases:
  - Uses graph structures for semantic queries with nodes, edges and properties to represent and store data
  - A key concept is the graph (vertices and edges) which directly relates data items in the store
    - The vertices are the items (data, things)
    - The edges represent the relationship between the data



Storage and tools

- Philosophy to Scale for Big Data

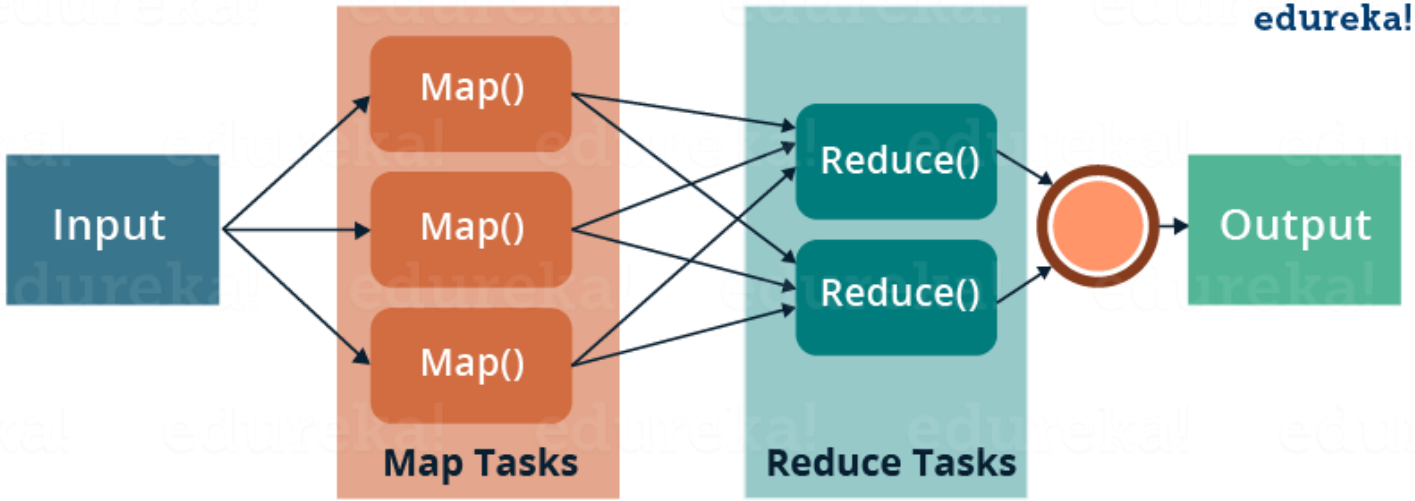


Divide Work

Combine Results

Storage and tools

- MapReduce
  - Is a programming framework (paradigm or model) that allows us to perform distributed and parallel processing on large data sets in a distributed environment.

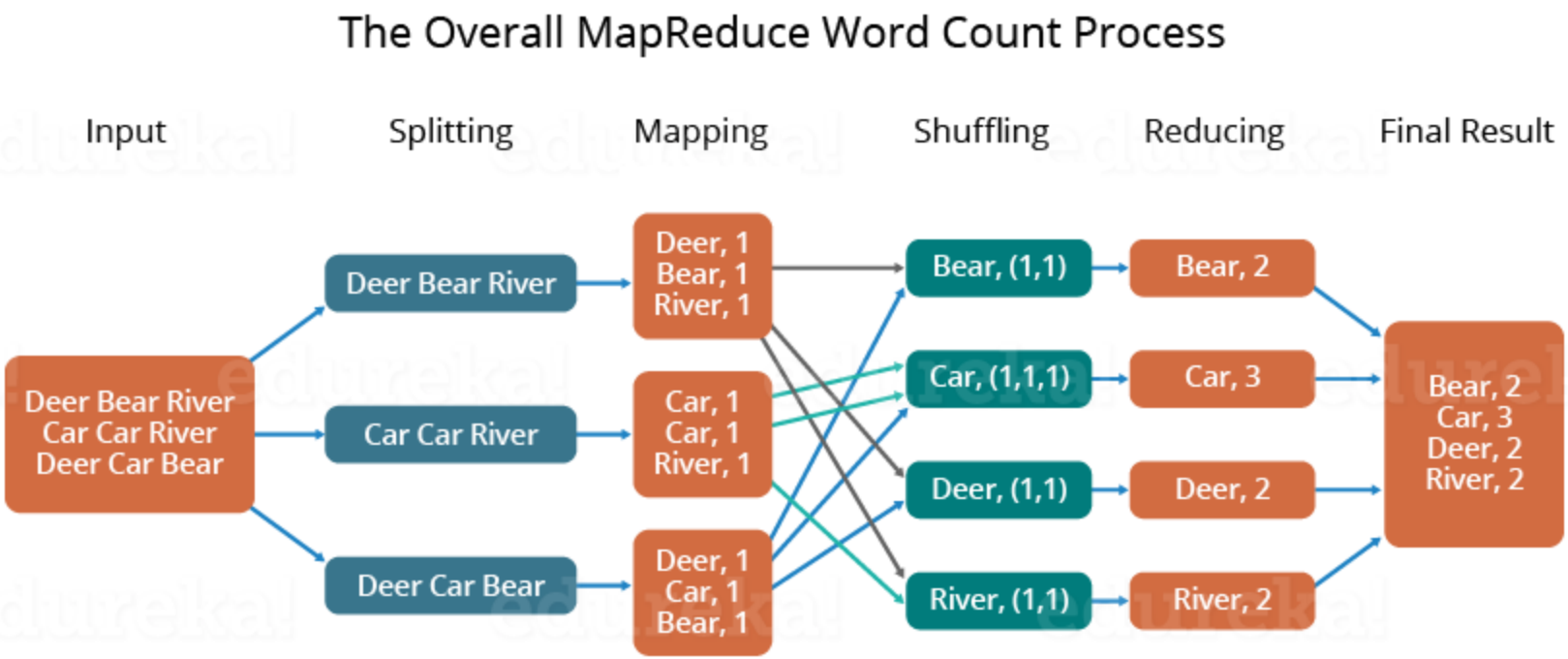


Storage and tools

- MapReduce
  - MapReduce consists of two distinct tasks – Map and Reduce
  - As the name MapReduce suggests, reducer phase takes place after mapper phase has been completed.
  - The output of a Mapper or map job (key-value pairs) is input to the Reducer.
  - The reducer receives the key-value pair from multiple map jobs.
  - Then, the reducer aggregates those intermediate data tuples (intermediate key-value pair) into a smaller set of tuples or key-value pairs which is the final output.

Storage and tools

- MapReduce word count example



Storage and tools

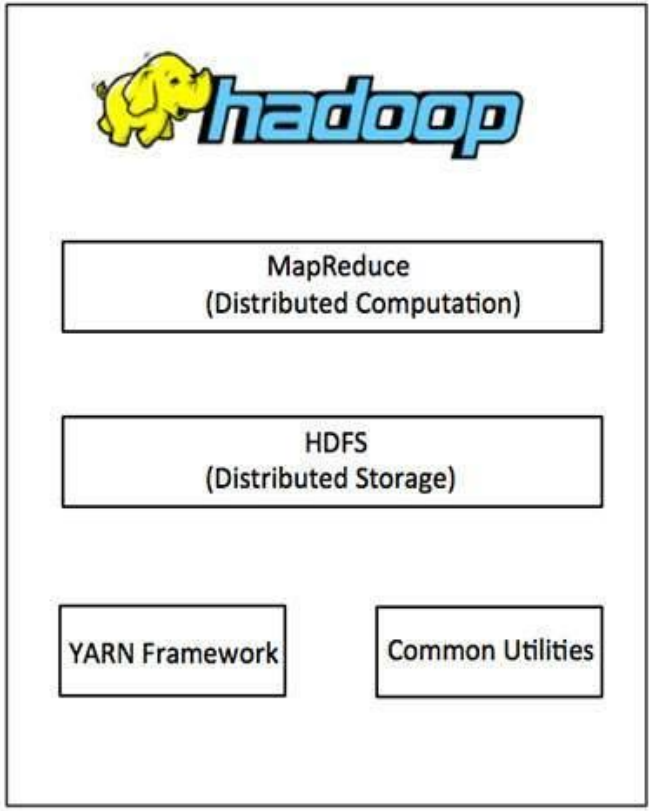
- Hadoop
  - Is a framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models
  - Is designed to scale up from single servers to thousands of machines, each offering local computation and storage.
  - Rather than rely on hardware to deliver high-availability, the framework itself is designed to detect and handle failures at the application layer, so delivering a highly-available service on top of a cluster of computers, each of which may be prone to failures.

Storage and tools

- Hadoop includes these modules
  - Hadoop Common: The common utilities that support the other Hadoop modules
  - Hadoop Distributed File System (HDFS™): A distributed file system that provides high-throughput access to application data
  - Hadoop YARN: A framework for job scheduling and cluster resource management
  - Hadoop MapReduce: A YARN-based system for parallel processing of large data sets

Storage and tools

- Hadoop Ecosystem



Storage and tools

- HDFS
  - Hadoop distributed File System is based on Google File System (GFS)
  - Serves as the distributed file system for most tools in the Hadoop ecosystem
  - Scalability for large data sets
  - Reliability to cope with hardware failures

Storage and tools

- HDFS is good for:
  - Large files
  - Streaming data
- HDFS is not good for:
  - Lots of small files
  - Random access to files
  - Low latency access

Storage and tools

- Spark
  - An open-source and fast engine for large-scale data processing.
  - It supports data streaming and SQL, machine learning and graph processing.
  - Spark uses Hadoop's client libraries for HDFS and YARN

Storage and tools

Storage and tools

- Hadoop Extended Ecosystem

University
Politehnica
of Bucharest

Computer
Science and
Engineering
Department

The NoSQL Movement
Key-Value Databases: Riak

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@upb.ro

Overview

- The NoSQL Movement
- Key-Value Databases: Riak

Overview

- The NoSQL Movement
- Key-Value Databases: Riak

The NoSQL Movement

- The term NoSQL was first used in 1998 for relational database that omitted the use of SQL
- The term NoSQL was picked up again in 2009 by the advocates of non-relational databases
- The main reason for the NoSQL movement: to seek alternatives for solving problems for which relational databases are a bad fit

The NoSQL Movement
Motives

1. Avoidance of unneeded complexity:
   - Relational databases provide a variety of features and strict data consistency
   - Relational databases use the ACID properties for transactions which are for some applications restrictive
   - The features, strict consistency and ACID properties of relational databases might be more than necessary for particular applications and use cases

The NoSQL Movement Motives

2. High throughput:
   - RDBMSs: Strict consistency and ACID properties restrict the throughput
   - NoSQL use looser consistency and different less restrictive transaction properties to improve throughput
   - NoSQL DBMSs use different storage methods than RDBMSs to improve throughput
   - NoSQL DBMSs' processing capabilities are improved using distributed and parallel algorithms, i.e. MapReduce

The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:

   – Due to the huge volume of data, new technologies are needed to store and process this data

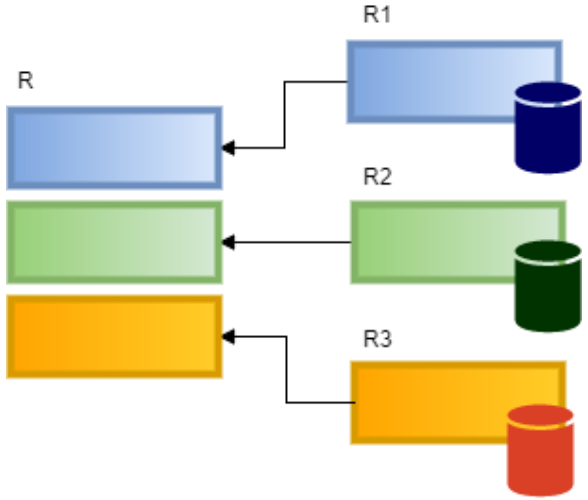
   – NoSQL DBMSs try to address the following current problems that DBMSs have:
     - Problems that appear when scaling out data due to the ACID properties
     - Performance of single servers
     - Processing of data on single servers
     - Rigid schema design
     - Costs of scaling, i.e. for Oracle, one needs at least Oracle Clusterware, Oracle Grid Infrastructure, and Oracle Real Application Server on top of which the database is installed

The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:
   - Fragmentation Horizontal (Sharding)

Fragmentarea pe orizontală.

În fragmentarea pe orizontală (Figura 4.1a), fragmentele R_i, $i = \overline{1,n}$, în care a fost împărțită relația R utilizează operația de reuniune pentru a reconstrui relația inițială, astfel $R = R_1 \cup R_2 \cup \ldots \cup R_n = \bigcup_{i=1}^{n} R_i$.

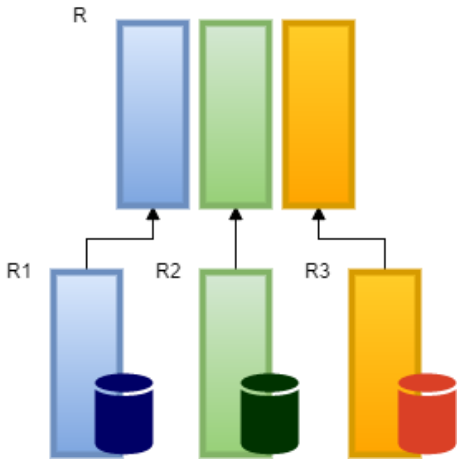


The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:
   - Fragmentation Vertical

Fragmentarea pe verticală.

În fragmentarea pe verticală (Figura 4.1b), fragmentele R_i, $i = \overline{1,n}$, în care a fost împărțită relația R utilizează operația de JOIN pentru a reconstrui relația inițială, astfel $R = R_1 \bowtie R_2 \bowtie \ldots \bowtie R_n$.

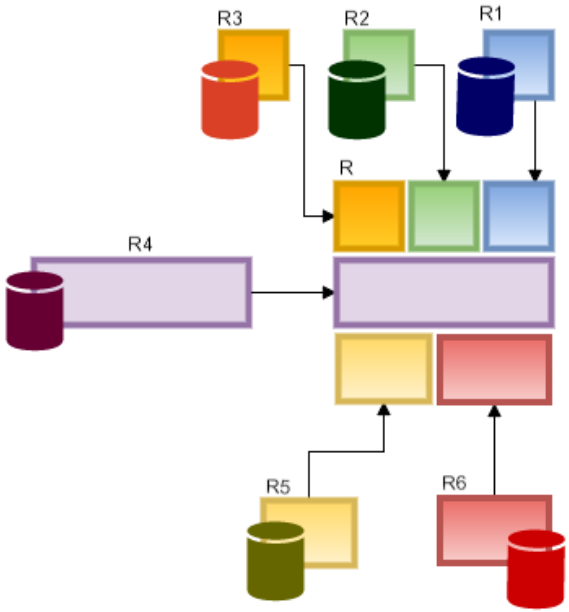


The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:
   - Fragmentation Mixt Horizontal-Vertical

Fragmentarea mixtă orizontală-verticală.

În fragmentare mixtă orizontală-verticală (Figura 4.2b), relația R este mai întâi fragmentată pe orizontală în relațiile R_i, $i = \overline{1,n}$, astfel încât $R = R_1 \cup R_2 \cup \ldots \cup R_n = \bigcup_{i=1}^{n} R_i$, și apoi fiecare relație R_i este la rândul ei fragmentată pe verticală în fragmentele R_{ij}, $j = \overline{1,m_i}$. Pentru a reconstruii fragmentele R_i se va utiliza operația de JOIN astfel: $R_i = R_{i1} \bowtie R_{i2} \bowtie \ldots \bowtie R_{im_i}$. Relația R este reconstruită utilizând mai întâi operația de JOIN și apoi cea de reuniunea: $R = (R_{11} \bowtie R_{12} \bowtie \ldots \bowtie R_{1n_1}) \cup (R_{21} \bowtie R_{22} \bowtie \ldots \bowtie R_{2n_2}) \cup \ldots \cup (R_{n1} \bowtie R_{n2} \bowtie \ldots \bowtie R_{nm_n})$.

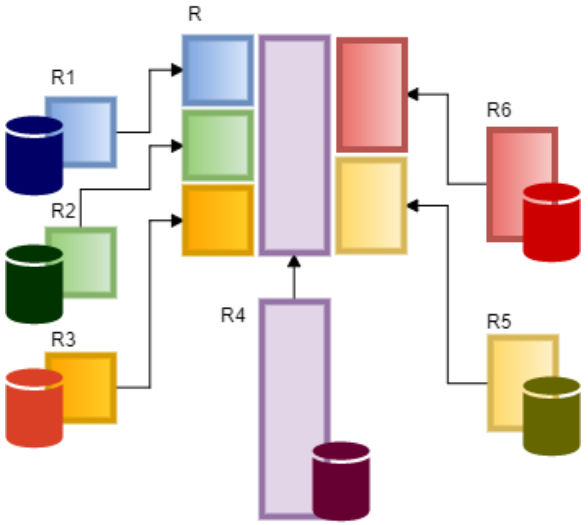


The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:
   - Fragmentation Mixt Vertical-Horizontal

Fragmentarea mixtă verticală-orizontală.

În fragmentare mixtă verticală-orizontală (Figura 4.2a), relația R este mai întâi fragmentată pe verticală în relațiile R_i, $i = \overline{1,n}$, astfel încât $R = R_1 \bowtie R_2 \bowtie \ldots \bowtie R_n$, și apoi fiecare relație R_i este la rândul ei fragmentată pe orizontală în fragmentele R_{ij}, $j = \overline{1,m_i}$. Pentru a reconstrui fragmentele R_i se va utiliza operația de reuniune astfel: $R_i = R_{i1} \cup R_{i2} \cup \ldots \cup R_{im_i}$. Relația R este reconstruită utilizând mai întâi operația de reuniune și apoi cea de JOIN: $R = (R_{11} \cup R_{12} \cup \ldots \cup R_{1n_1}) \bowtie (R_{21} \cup R_{22} \cup \ldots \cup R_{2n_2}) \bowtie \ldots \bowtie (R_{n1} \cup R_{n2} \cup \ldots \cup R_{nm_n})$.



The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:

   – NoSQL DBMSs are designed to:
   - Easily scale out (horizontal scale)
     – Machines can be added and remove without causing the same operational efforts to perform sharding in RDBMS cluster-solutions
   - Not rely on highly available hardware
   - Flexible schema-less design
   - Lower costs of scaling

The NoSQL Movement
Motives

4. Avoidance of Expensive object-relational mapping:

   – Most NoSQL DBMSs are designed to store data that are either simple or more like the ones of object-oriented programing languages compared to relational data structures

   – There is no need for expensive object-relational mapping

   – This is particularly important for applications with data structures of low complexity that can hardly benefit from the features of a relational database

The NoSQL Movement Motives

5. Decrease the Complexity and the Cost of Setting up Database Clusters:

   – NoSQL databases are designed in a way that PC clusters can be easily and cheaply expanded without the complexity and cost of sharding

   – Sharding involves cutting up databases into multiple tables to run on large clusters or grids

The NoSQL Movement
Motives

6. Compromising Reliability for Better Performance

   – There are different scenarios for which applications can compromise reliability for better performance.

   – E.g. HTTP session data which needs to be shared between various web servers but since the data is transient in nature (it goes away when the user logs off) there is no need to store it in persistent storage.

The NoSQL Movement Motives

7. The Current "One size fit's it all" Databases Thinking Was and Is Wrong

   – A growing number of application scenarios cannot be addressed with a traditional database approach, e.g. Facebook, Twitter, Instagram, etc.

   – The search for alternatives towards traditional RDBMSs can be explained by two major trends:
     - The continuous growth of data volumes (to be stored)
     - The growing need to process larger amounts of data in shorter time

The NoSQL Movement Motives

8. Movements in Programming Languages and Development Frameworks

- Multiple programming languages and development frameworks provide abstractions for database access trying to hide the use of SQL and relational databases

- The NoSQL databases react on this trend and try to provide data structures in their APIs that are closer to the ones of programming languages (e. g. key/value-structures, documents, graphs)

- Examples for this trend in the last couple of years include:
  - Object-relational mappers in the Java and .NET world like the Java Persistence API (JPA, part of the EJB 3 specification), implemented by e.g., Hibernate, or the LINQ Framework with the ADO.NET Entity Framework since .NET version 4
  - The popularity of Python and Ruby frameworks which hide the usage of relational databases

The NoSQL Movement Motives

9. Requirements of Cloud Computing
   - There are two major requirements of datastores in cloud computing environments:
     - High until almost ultimate scalability (especially in the horizontal direction)
     - Low administration overhead
   - The following classes of databases work well in the cloud:
     - Data warehousing with specific features for batch data processing and map/reduce operations
     - Simple, scalable and fast key/value-stores
     - Databases containing a richer feature set than key/value-stores to fit the gap with traditional RDBMSs while offering good performance and scalability properties, e.g. document-oriented databases

NoSQL Databases Categories

- By scalability
- By data and query model
- By persistence design

NoSQL Databases Categories Scalability

- Through replication, it is easy to scale:
  - Read operations
  - Load distribution
- The write operations are important for the scalability due to data distribution and partitioning
- For distributed systems that provide auto sharding there are 2 important features:
  - Support of multiple datacenters
  - Possibility to add machines live to an existing cluster, transparent to the applications and the users using the cluster

NoSQL Databases Categories
Data and query model

- Column-Oriented databases
- Key-Value databases
- Document-Oriented databases
- Graph databases

NoSQL Databases Categories Persistence design

1. In-Memory Databases

   - Are very fast
   - The data size is inherently limited by the size of RAM
   - Durability may become a problem as the amount of data which can get lost between subsequent disk flushes is potentially large
     - E.g. due to server crashes or by losing power supply

NoSQL Databases Categories Persistence design

2. Memtables and SSTables:

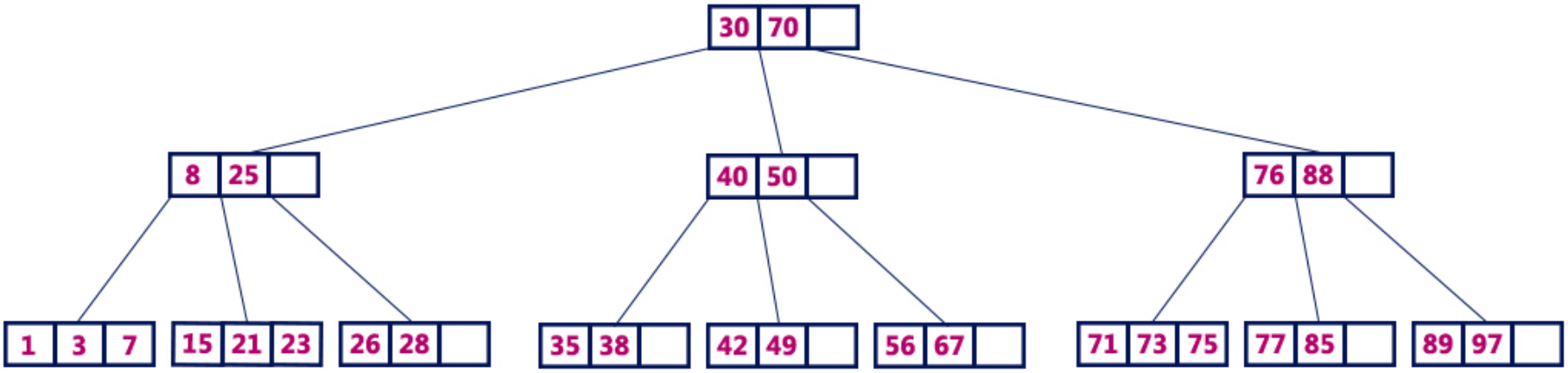
- Write operations are buffered in memory (in a Memtable) after they have been written to an append-only commit log to ensure durability
- After a certain amount of writes the Memtable gets flushed to disk (as a whole) to a SSTable (Idea taken from Google's Bigtable paper)
- SSTable persistence strategy has performance characteristics comparable to those of in-memory-databases but avoids the durability difficulties of pure in-memory databases
- Compared to disk-based strategies, disk seeks are reduced due to the append-only log and the flushing of whole Memtables to disk

NoSQL Databases Categories Persistence design

3. B-trees

- Have been used in databases since their beginning to provide a robust indexing-support
- Their performance characteristics on rotating disks are not very positive due to the amount of disk seeks needed for read and write operations
- The document-oriented database CouchDB uses B-trees internally but tries to avoid the overhead of disk seeks by only appending to B-trees (append-only B-trees):
  - Only one write operation at a time can happen
  - Concurrent reads to different sections of a B-tree are not allowed in this case

NoSQL Databases Categories
Persistence design

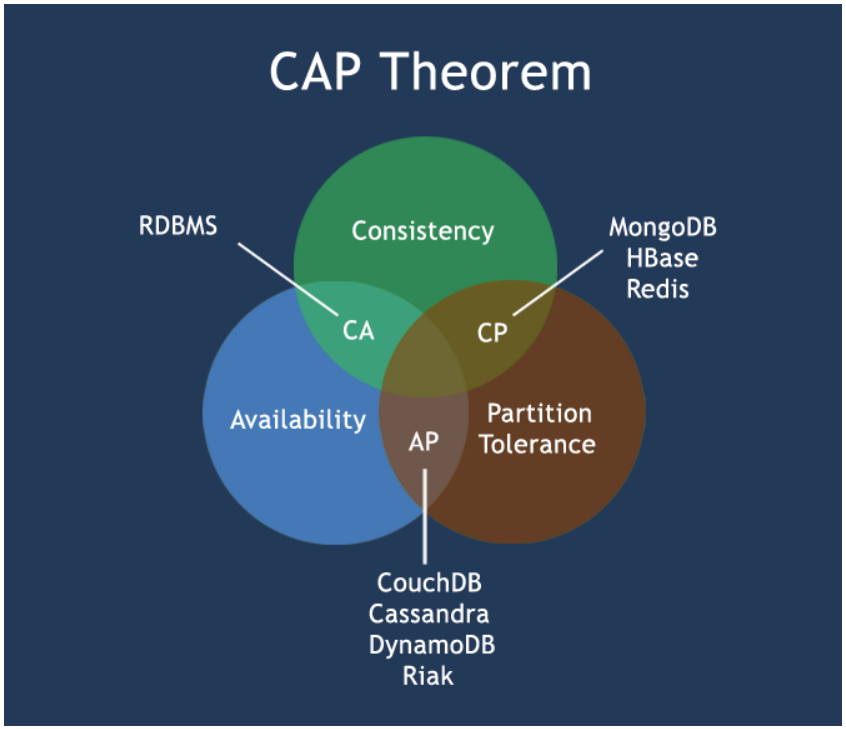


NoSQL Databases Categories

Type	Performance	Scalability	Flexibility	Complexity	Functionality
Key-Value	High	High	High	None	Variable
Column	High	High	Moderate	Low	Minimal
Document	High	Variable (high)	High	Low	Variable
Graph	Variable	Variable	High	High	Graph theory
Relational	Variable	Variable	Low	Moderate	Relational algebra

CAP Theorem

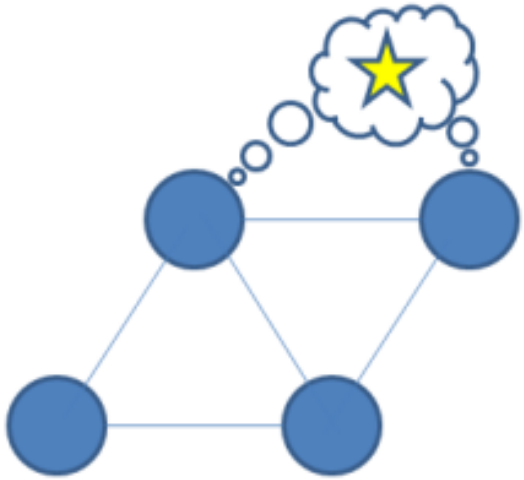
- Consistency
- Availability
- Partition Tolerance



CAP Theorem
Consistency

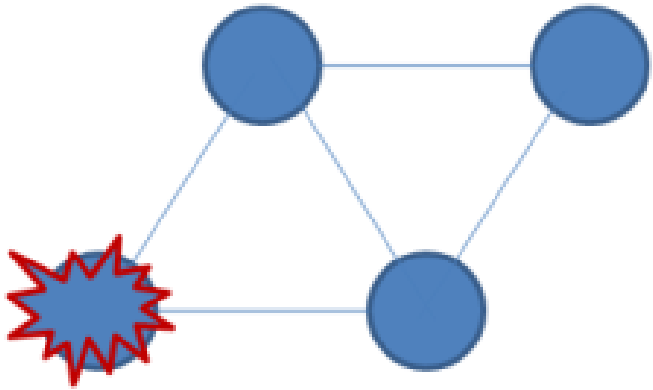
- Consistency meaning if and how a system is in a consistent state after the execution of an operation

- A distributed system is typically considered to be consistent if after an update operation of some writer all readers see his updates in some shared data source



CAP Theorem
Availability

- Availability and especially high availability meaning that a system is designed and implemented in a way that allows it to continue operation (i.e. allowing read and write operations) even if nodes in a cluster crash or some hardware or software parts are down due to upgrades.

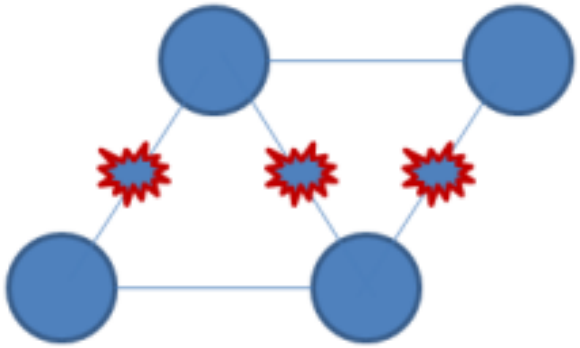


CAP Theorem
Partition Tolerance

- Partition Tolerance is understood as the ability of the system to continue operation in the presence of network partitions

- These occur if two or more "islands" of network nodes arise which (temporarily or permanently) cannot connect to each other.

- Some people also understand partition tolerance as the ability of a system to cope with the dynamic addition and removal of nodes (e. g. for maintenance purposes; removed and again added nodes are considered an own network partition in this notion).



CAP Theorem

- Cap Theorem (Pick 2)

	Traits	Examples
Consistence + Availability	• 2-phase commit (2PC) • Cache-validation protocols	RDBMS: Oracle, MySQL, MS SQL Server, PostgreSQL
Consistence + Partition Tolerance	• Pessimistic locking • Make minority partitions unavailable	MongoDB, HBase, Redis
Availability + Partition Tolerance	• Conflict resolution • Optimistic locking	CouchDB, Cassandra, Riak

CAP Theorem
Consistence + Availability

- 2-phase commit
  - A type of atomic commitment protocol
  - A distributed algorithm that coordinates all the processes that participate in a distributed atomic transaction on whether to commit or abort (roll back) the transaction

- Cache validation protocols:
  - Ensures cache coherence
  - Cache coherence is the uniformity of shared resource data that ends up stored in multiple local caches

CAP Theorem

- Minority partition (Consistence + Partition Tolerance)
  - A partition of nodes fewer or equal than half of the total number of nodes
  - Partions (islands) appear during communition failure between the nodes in a cluster
- Conflict resolution (Availability + Partition Tolerance)
  - Conflicts can occur in an environment that permits concurrent updates to the same data at multiple sites
  - Solutions: Multiversion concurrency control

CAP Theorem

- Pessimistic locking (Consistence + Partition Tolerance)
  - A resource
    - Is locked from the time it is first accessed in a transaction until the transaction is finished
    - Is inaccessible to other transactions during that time.
  - If most transactions simply look at the resource and never change it
    - An exclusive lock may be overkill as it may cause lock contention
    - Optimistic locking may be a better approach
  - Locks are applied in a fail-safe way
  - The lock exists until the transaction has either been committed or rolled back

CAP Theorem

- Optimistic locking (Availability + Partition Tolerance)
  - A resource is not actually locked when it is first accessed by a transaction
    - Other transactions can concurrently access the resource
    - Conflicting changes are possible
  - At commit time, when the resource is about to be updated in persistent storage the state of the resource
    - Is read from storage again
    - Is compared to the state that was saved when the resource was first accessed in the transaction
    - If the two states differ, a conflicting update was made, and the transaction will be rolled back

Transaction properties
ACID

- ACID (**A**tomicity, **C**onsistency, **I**solation, **D**urability)
  - **Atomicity**:
    - Requires that each transaction be "all or nothing"
    - If one part of the transaction fails, then the entire transaction fails, and the database state is left unchanged.
  - **Consistency** ensures that any transaction will bring the database from one valid state to another.
  - **Isolation** ensures that the concurrent execution of transactions results in a system state that would be obtained if transactions were executed sequentially, i.e., one after the other.
  - **Durability** ensures that once a transaction has been committed, it will remain so, even in the event of power loss, crashes, or errors.

Transaction properties
BASE

- BASE (**B**asically **A**vailable, **S**oft-sate, **E**ventual consistency)
- **Basically Available:**
  - This constraint states that the system does guarantee the availability of the data as defined by the CAP Theorem.
  - There will be a response to any request
  - The response could still be 'failure' to obtain the requested data, or the data may be in an inconsistent or changing state
- **Soft-state:**
  - The state of the system could change over time, so even during times without input there may be changes going on due to 'eventual consistency', thus the state of the system is always 'soft'
- **Eventual consistency:**
  - The system will eventually become consistent once it stops receiving input.
  - The data will propagate to everywhere it should sooner or later, but the system will continue to receive input and is not checking the consistency of every transaction before it moves onto the next one

ACID vs BASE

ACID	BASE
Strong consistency	Weak consistency
Isolation	Availability first
Focus on “commit”	Best effort
Real answers	Approximate answers
Availability	Aggressive Availability
Difficulty in modeling	Easier modeling

Overview

- The NoSQL Movement
- Key-Value Databases: Riak

Key-Value Databases

- Have a simple data model: map/dictionary
- Allows clients to request values by key
- Favor high availability over consistency
- Simple queries, they omit rich ad-hoc querying and analytics features – joins and aggregations are set aside
- The length of the keys are limited by a certain number of bytes while there is less limitation in values
- A great influence on the architecture is Amazon Dynamo Database

Riak

- Is a distributed key-value store
- Values can be anything from pain text, JSON, or XML to Images of video clips
- All the values are accessible through a simple HTTP interface
- Is fault tolerant – nodes can go up and down at any moment without any single point of failure
- A Riak cluster works continuously as nodes are added, removed or (ideally not) crush

Riak

- Riak trade-offs:
  - Lacks robust support for ad-hoc queries
  - Trouble linking values together (no foreign keys)

Riak

- Loves the web:
  - Querying is done via URLs, headers, and verbs
  - Riak returns assets and standard HTTP response codes
- Riak supplies an HTTP REST interface
- The interaction is done via the URL tool cURL
- cURL allows the interaction with the REST API without resorting to a particular driver or programming language

Riak

- Riak is a great choice for datacenters that must serve many requests with low latency
- It’s easy to manage, easy to set up, and can grow with your needs
- It is built in Erlang
- Can be installed on different distributions of Linux and Mac OS

Riak

- Riak's default mode of operation is to work as a cluster consisting of multiple nodes, i.e. multiple well-connected data hosts
- Each host in the cluster runs a single instance of Riak, referred to as a Riak node
- Each Riak node manages a set of virtual nodes (vnodes) that are responsible for storing a separate portion of the keys stored in the cluster

Riak

- Riak nodes (in contrast to some high-availability systems)
  - Are not clones of one another
  - Do not all participate in fulfilling every request
- At runtime or at request time, you can configure:
  - The number of nodes on which data is to be replicated
  - When replication occurs
  - Which merge strategy and failure model are to be followed.

Riak

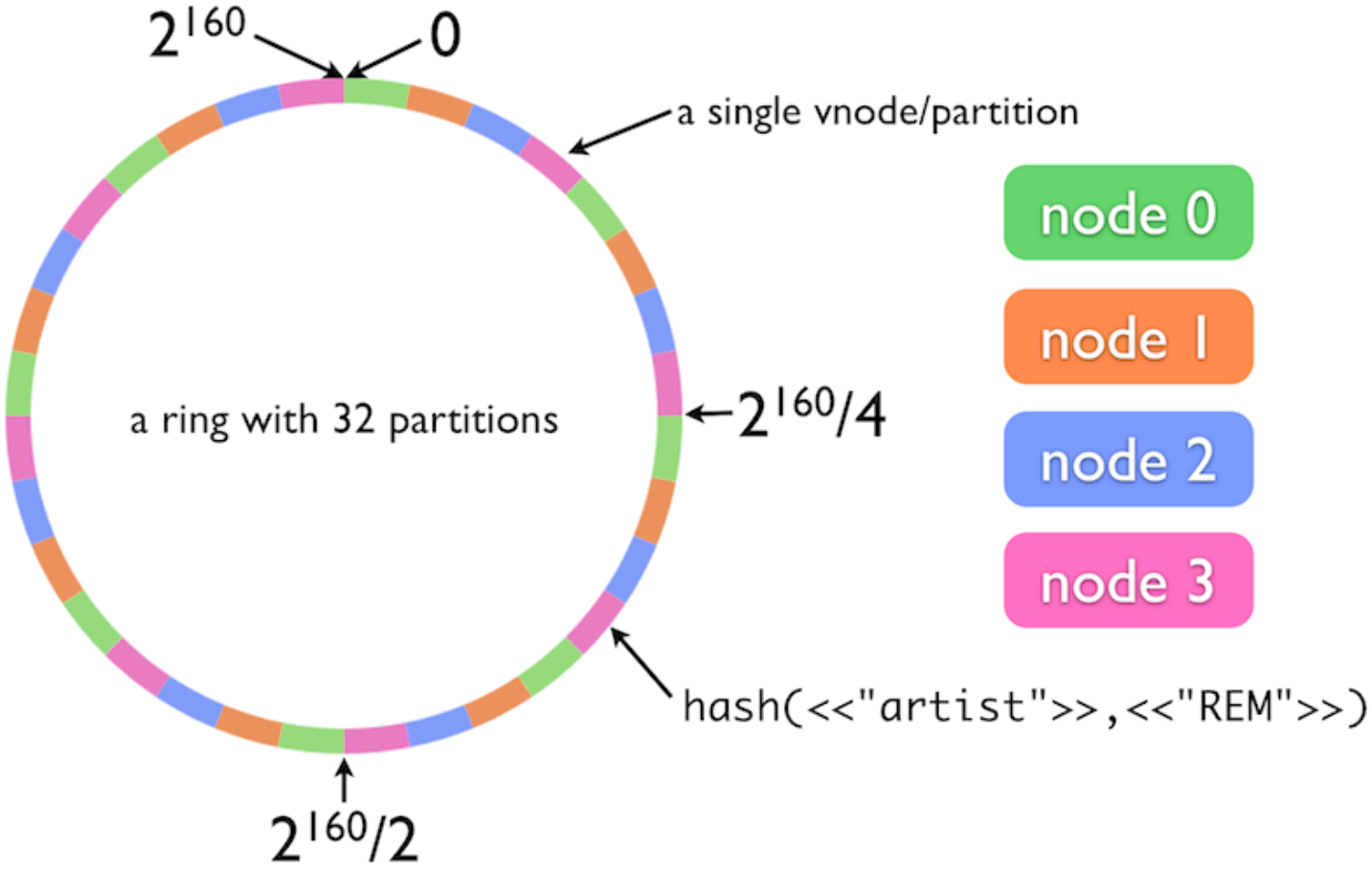
- Any client interface to Riak interacts with objects in terms of the bucket and key in which a value is stored (as well as the bucket type that is used to set the bucket's properties)

- Internally,
  - Riak computes a 160-bit binary hash of each bucket/key pair
  - Maps the hash value to a position on an ordered ring of all such values

- This ring is divided into partitions

- Each Riak **vnode** responsible for one of these partitions (meaning that each **vnode** claims that partition)

Riak


2^{160}
0
a single vnode/partition
node 0
node 1
node 2
node 3
a ring with 32 partitions
$2^{160}/4$
hash(<<"artist">>,<<"REM">>)
$2^{160}/2$

Riak

- The nodes of a Riak cluster each attempt to run a roughly equal number of **vnodes** at any given time
- In the general case, this means that each node in the cluster is responsible for
  - 1/(number of nodes) of the ring
  - (number of partitions)/(number of nodes) vnodes
- E.g., If two nodes define a 16-partition cluster then each node will run 8 vnodes
- Nodes attempt to claim their partitions at intervals around the ring such that
  - There is an even distribution amongst the member nodes
  - No node is responsible for more than one replica of a key

Riak
Intelligent Replication

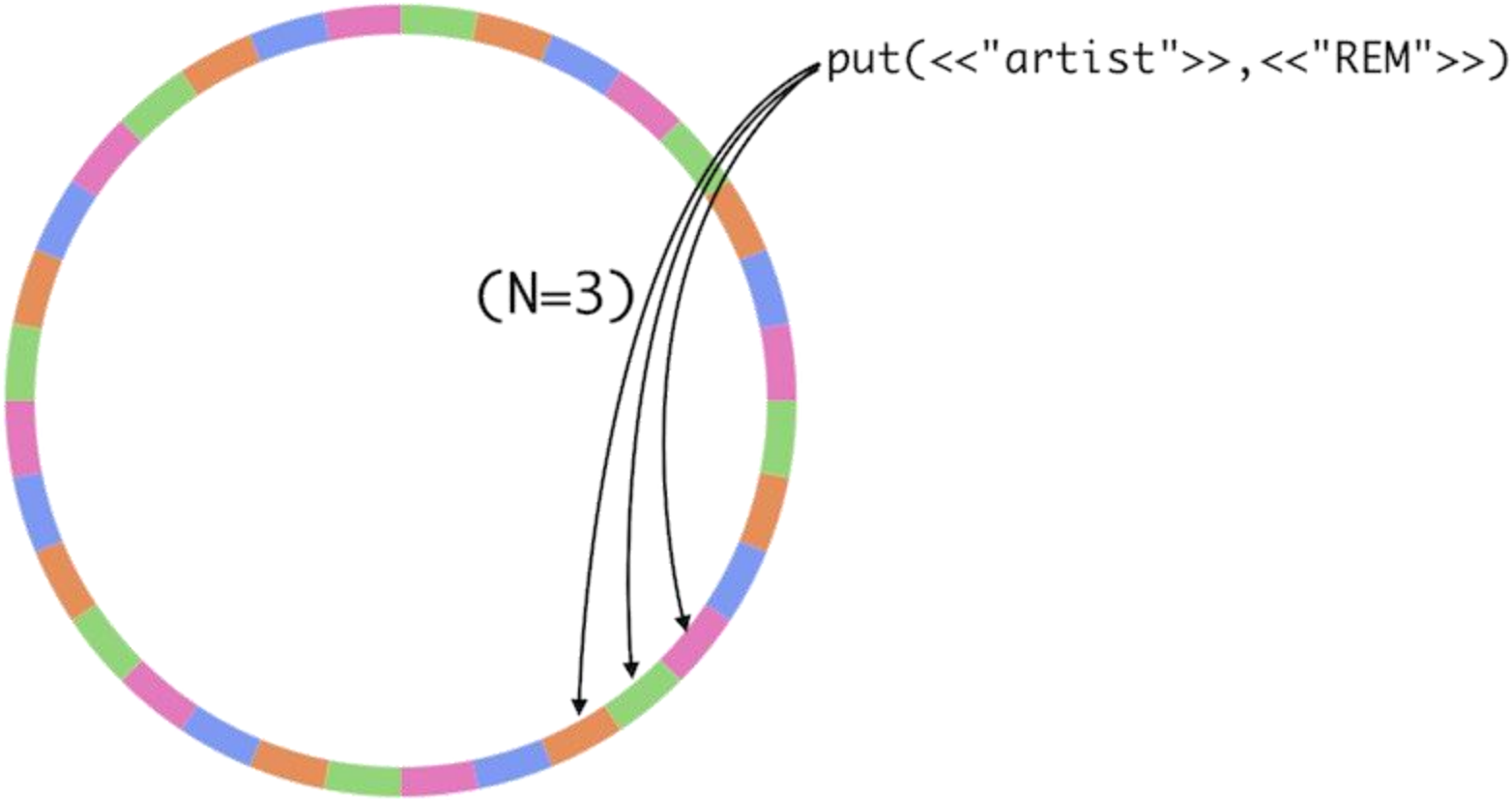
- When an object is being stored in the cluster, any node may participate as the coordinating node for the request

- The coordinating node
  - Consults the ring state to determine which vnode owns the partition in which the value's key belongs
  - Sends the write request to that vnode as well as to the vnodes responsible for the next N-1 partitions in the ring (where N is a configurable parameter that describes how many copies of the value to store)

Riak
Intelligent Replication

- The write request may also specify that
  - At least W (=< N) of those vnodes reply with success
  - DW (=< W) nodes reply with success only after durably storing the value
- A read, or GET, request operates similarly, sending requests
  - To the vnode that "claims" the partition in which the key resides
  - As well as to the next N-1 partitions
- The request also specifies R (=< N), the number of vnodes that must reply before a response is returned

Riak
Intelligent Replication



Riak
Intelligent Replication

- When N is set to 3, the value REM is stored in the key artist
- That key is assigned to 3 partitions out of 32 available partitions
- When a read request is made to Riak, the ring state will be used to determine which partitions are responsible
- From there, a variety of configurable parameters determine how Riak will behave in case the value is not immediately found

Intelligent Replication
Gossiping

- The ring state is shared around the cluster by means of a "gossip protocol"
- Whenever a node changes its claim on the ring, it announces, i.e. "gossips", this change to other nodes so that the other nodes can respond appropriately
- Nodes also periodically re-announce what they know about ring in case any nodes happened to miss previous updates

Installing Riak

- Installing Riak
  - Go to the official web site http://basho.com/products/ and download the package for your distribution.
  - You can also compile the source code but you need to install Erlang and additional packages.

Installing Riak

- Installing Riak on Ubuntu 22.04:
  - Download the latest version:

    wget https://files.tiot.jp/riak/kv/3.2/3.2.0/ubuntu/jammy64/riak_3.2.0-OTP25_amd64.deb

  - Install the package

    sudo dpkg -i riak_3.2.0-OTP25_amd64.deb

  - Start a node on localhost (wither with sudo privileges or as root)

    sudo riak daemon

Installing Riak on a cluster

- Download and install the package on all the nodes - do not start Riak
- On each node modify riak.conf file (the default location is /etc/riak/riak.conf)
- Give a name to the Erlang node (nodename)
  nodename = riak@ip_host
- Set the IP Address and TCP port to bind Riak's HTTP interface
  listener.http.internal = ip_host:port
- Set the IP Address and TCP port to bind Riak's Protocol Buffers interface (Protocol Buffers is a method to serialize data)
  listener.protobuf.internal = ip_host:port

Installing Riak on a cluster

- Enable VM Tunnings

  erlang.schedulers.force_wakeup_interval = 500

  erlang.schedulers.compaction_of_load = false

- Set the ring size

  ring_size = 64

- Start the node

  $ sudo riak daemon

Installing Riak on a cluster

- If the master node is now properly set up proceed to adding other nodes to the cluster by doing the same configurations as for the master

- The new nodes must join the cluster, on each node run the following command

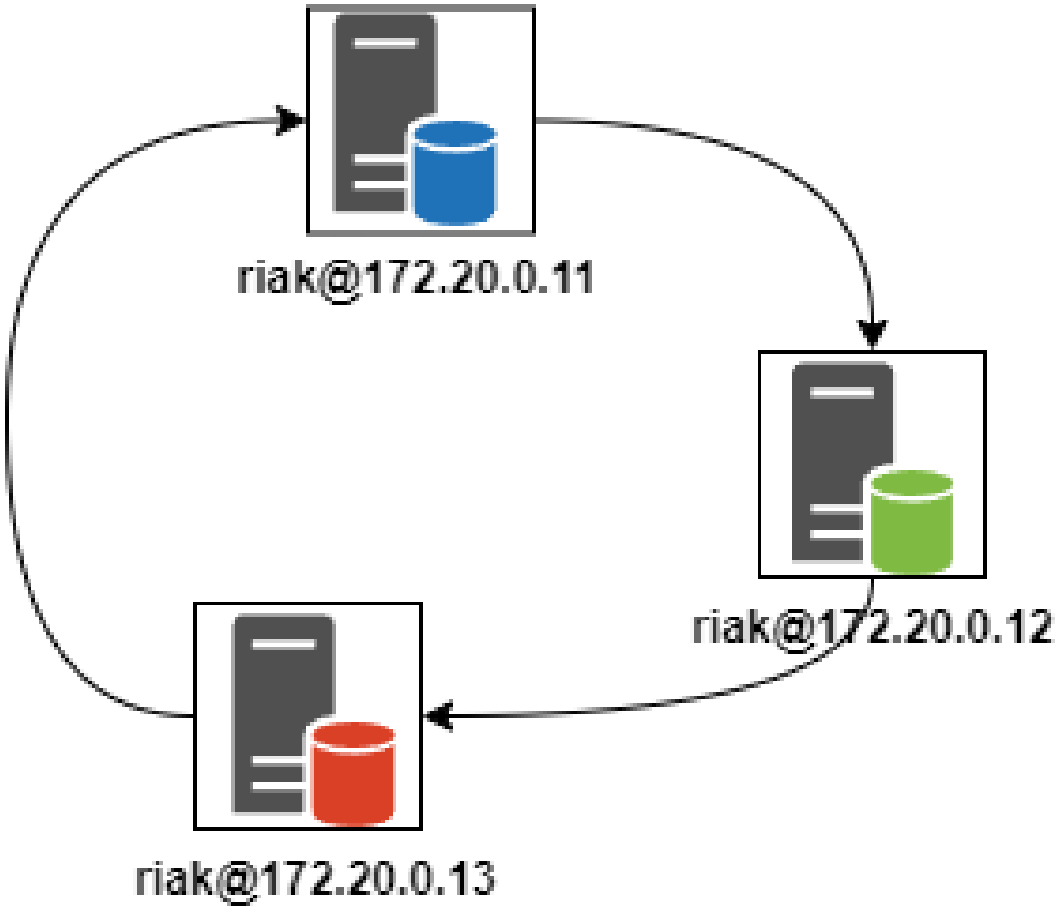
  $ sudo riak admin cluster join riak@ip_master

- Next, plan and commit the changes on the master node:

  $ sudo riak admin cluster plan

  $ sudo riak admin cluster commit

Riak Cluster Example



Riak Cluster Example

- On node riak@172.20.0.11 edit riak.conf as follows:

```
nodename = riak@172.20.0.11
listener.http.internal = 172.20.0.11:8098
listener.protobuf.internal = 172.20.0.11:8097
ring_size  = 64
erlang.schedulers.force_wakeup_interval = 500
erlang.schedulers.compaction_of_load = false
```

- Start the node (as root):

```
$ riak daemon
```

Riak Cluster Example

- On node riak@172.20.0.12 edit riak.conf as follows:

```
nodename = riak@172.20.0.12
listener.http.internal = 172.20.0.12:8098
listener.protobuf.internal = 172.20.0.12:8097
ring_size  = 64
erlang.schedulers.force_wakeup_interval = 500
erlang.schedulers.compaction_of_load = false
```

- Start the node (as root):

```
$ riak daemon
```

Riak Cluster Example

- On node riak@172.20.0.13 edit riak.conf and add as follows:

```
nodename = riak@172.20.0.13
listener.http.internal = 172.20.0.13:8098
listener.protobuf.internal = 172.20.0.13:8097
ring_size  = 64
erlang.schedulers.force_wakeup_interval = 500
erlang.schedulers.compaction_of_load = false
```

- Start the node (as root):

```
$ riak daemon
```

Installing Riak on a cluster

- Run on nodes riak@172.20.0.12 and riak@172.20.0.13 the following command (as root):

  $ riak admin cluster join riak@172.20.0.11

- Run on riak@172.20.0.11 the following command (as root):

  $ riak admin cluster plan

  $ riak admin cluster commit

Installing Riak on a cluster

- Full installation guide: https://www.tiot.jp/riak-docs/riak/kv/3.2.0/configuring/basic/

Installing Riak on a cluster

- The command line administrator tool is: riak admin

- To check the cluster status

  $ sudo riak admin cluster status

```
root@770d7a85290e:/# sudo riak admin cluster status
---- Cluster Status ----
Ring ready: true

+----------------------+------+-------+-----+-------+
|         node         |status| avail |ring |pending|
+----------------------+------+-------+-----+-------+
| (C) riak@172.20.0.11 |valid |  up   | 34.4|  --   |
|     riak@172.20.0.12 |valid |  up   | 32.8|  --   |
|     riak@172.20.0.13 |valid |  up   | 32.8|  --   |
+----------------------+------+-------+-----+-------+

Key: (C) = Claimant; availability marked with '!' is unexpected
ok
```

Installing Riak on a cluster

- To see partitions assigned for each node

  $ sudo riak admin cluster partitions

```
root@770d7a85290e:/# sudo riak admin cluster partitions
Partitions owned by 'riak@172.20.0.11':
+---------+-----------------------------------------------------+--+
|  type   |                        index                        |id|
+---------+-----------------------------------------------------+--+
| primary |                          0                          |0 |
| primary | 91343852333181432387730302044767688728495783936    |4 |
| primary |1826877046663628647754606040895353774569915678 72   |8 |
| primary |2740315569995442971631909061343030661854873518 08   |12|
| primary |3653754093327257295509212081790707549139831357 44   |16|
| primary |4567192616659071619386515102238384436424789196 80   |20|
| primary |5480631139990885943263818122686061323709747036 16   |24|
| primary |6394069663322700267141121143133738210994704875 52   |28|
| primary |7307508186654514591018424163581415098279662714 88   |32|
| primary |8220946709986328914895727184029091985564620554 24   |36|
| primary |9134385233318143238773030204476768872849578393 60   |40|
| primary |9819464125817003981681007469812526538313296773 12   |43|
| primary |1004782375664995756265033322492444576013453623296|44|
| primary |1073290264914881830555831049026020342559825461248|47|
| primary |1096126227998177188652763624537212264741949407232|48|
| primary |1164634117248063262943561351070788031288321245184|51|
| primary |1187470080331358621040493926581979953470445191168|52|
| primary |1255977969581244695331291653115555720016817029120|55|
| primary |1278813932664540053428224228626747642198940975104|56|
| primary |1347321821914426127719021955160323408745312813056|59|
| primary |1370157784997721485815954530671515330927436759040|60|
| primary |1438665674247607560106752257205091097473808596992|63|
|secondary|                         --                          |--|
| stopped |                         --                          |--|
+---------+-----------------------------------------------------+--+

ok
```

Riak Statistics

- To see the statistics of the server

$ sudo riak admin status

- cURL can be used to get the statistics:

$ curl -GET http://172.20.0.11:8098/stats

- To check Riak configuration

$ sudo riak chkconfig

- Ping the cluster to see if it’s up and running:

$ sudo riak chkconfig

$ curl -GET http://172.20.0.11:8098/ping

Riak and cURL

- REST stands for REpresentational State Transfer
- REST is a guideline for mapping resources to URLs and interacting with them using CRUD verbs:
  - POST (**C**reate)
  - GET (**R**ead)
  - PUT (**U**pdate)
  - DELETE (**D**elete).
- cURL is used to directly connect to Riak server's HTTP REST interface without the need for an interactive console

Riak and cURL

- If we want only the header response use –I

$ curl -I http://172.20.0.11:8098/riak/animals

```
root@770d7a85290e:/# curl -I http://172.20.0.11:8098/riak/animals
HTTP/1.1 200 OK
Vary: Accept-Encoding
Server: MochiWeb/3.0.0 WebMachine/1.11.1 (greased slide to failure)
Date: Mon, 27 Nov 2023 17:57:47 GMT
Content-Type: application/json
Content-Length: 520
```

Riak and cURL
PUT

- Adding a new key-value element
  - Use curl with -X PUT or -XPUT
  - The -H attribute sets the following text as HTTP header information. In the example, we set the MIME content type to HTML.
  - Everything passed to -d (also known as the body data) is what Riak will add as a new value.
  - The -v attribute stands for verbose
  - The bucket is favs (equivalent to a schema)
  - The key is db

```
$ curl -v -X PUT http://172.20.0.11:8098/riak/favs/db \
-H "Content-Type: text/html" \
-d "<html><body><h1>My new favorite DB is RIAK</h1></body></html>“
```

Riak and cURL
PUT

- Adding a new key-value element
  - Use curl with -X PUT or -XPUT
  - The -H attribute sets the following text as HTTP header information. In the example, we set the MIME content type to HTML.
  - Everything passed to -d (also known as the body data) is what Riak will add as a new value.
  - The -v attribute stands for verbose
  - The bucket is favs (equivalent to a schema)
  - The key is db

```
$ curl -v -X PUT http://172.20.0.11:8098/riak/favs/db \
-H "Content-Type: text/html" \
-d "<html><body><h1>My new favorite DB is RIAK</h1></body></html>“
```

Riak and cURL
POST

- The attribute POST is used if we want the system to generate a new key by itself

```
curl -v -X POST http://172.20.0.11:8098/riak/favs \
-H "Content-Type: text/html" \
-d "<html><body><h1>Auto generated key</h1></body></html>"
```

Riak and cURL
POST

```
root@770d7a85290e:/# curl -v -X POST http://172.20.0.11:8098/riak/favs \
-H "Content-Type: text/html" \
-d "<html><body><h1>Auto generated key</h1></body></html>"
Note: Unnecessary use of -X or --request, POST is already inferred.
*   Trying 172.20.0.11:8098...
* Connected to 172.20.0.11 (172.20.0.11) port 8098 (#0)
> POST /riak/favs HTTP/1.1
> Host: 172.20.0.11:8098
> User-Agent: curl/7.81.0
> Accept: */*
> Content-Type: text/html
> Content-Length: 53
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 201 Created
< Vary: Accept-Encoding
< Server: MochiWeb/3.0.0 WebMachine/1.11.1 (greased slide to failure)
< Location: /riak/favs/MllwGkO6FvVSefrED1asOeq40TW
< Date: Mon, 27 Nov 2023 18:03:59 GMT
< Content-Type: text/html
< Content-Length: 0
<
* Connection #0 to host 172.20.0.11 left intact
```

New Key

Riak and cURL
DELETE

- The DELETE attribute is used to delete a record

$ curl -v -X DELETE http://172.20.0.11:8098/riak/favs/MllwGkO6FvVSefrED1asOeq40TW

```
root@770d7a85290e:/# curl -v -X DELETE http://172.20.0.11:8098/riak/favs/MllwGkO6FvVSefrED1asOeq40TW
*   Trying 172.20.0.11:8098...
* Connected to 172.20.0.11 (172.20.0.11) port 8098 (#0)
> DELETE /riak/favs/MllwGkO6FvVSefrED1asOeq40TW HTTP/1.1
> Host: 172.20.0.11:8098
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 204 No Content
< Vary: Accept-Encoding
< Server: MochiWeb/3.0.0 WebMachine/1.11.1 (greased slide to failure)
< Date: Mon, 27 Nov 2023 18:06:03 GMT
< Content-Type: text/html
< Content-Length: 0
<
* Connection #0 to host 172.20.0.11 left intact
```

Riak and cURL
GET

- The GET attribute is used to get the information for a record

$ curl -v -X GET http://172.20.0.11:8098/riak/favs/db

```
root@770d7a85290e:/# curl -v -X GET http://172.20.0.11:8098/riak/favs/db
Note: Unnecessary use of -X or --request, GET is already inferred.
*   Trying 172.20.0.11:8098...
* Connected to 172.20.0.11 (172.20.0.11) port 8098 (#0)
> GET /riak/favs/db HTTP/1.1
> Host: 172.20.0.11:8098
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< X-Riak-Vclock: a85hYGBgzGDKBVI8qUcPb1ytZ6IDEUpkzGNleB1y8x5fFgA=
< Vary: Accept-Encoding
< Server: MochiWeb/3.0.0 WebMachine/1.11.1 (greased slide to failure)
< Link: </riak/favs>; rel="up"
< Last-Modified: Mon, 27 Nov 2023 17:59:07 GMT
< ETag: "2ucL3c2ubdgXI1enKHu0X4"
< Date: Mon, 27 Nov 2023 18:07:00 GMT
< Content-Type: text/html
< Content-Length: 61
<
* Connection #0 to host 172.20.0.11 left intact
<html><body><h1>My new favorite DB is RIAK</h1></body></html>
```

Riak Example

- Example: Animal shelter
- Let’s create a shelter for animals and add 2 dogs

```
curl -v -X PUT http://172.20.0.11:8098/riak/animals/ace \
-H "Content-Type: application/json" \
-d '{"nickname" : "The Wonder Dog", "breed" : "German Shepherd"}'

curl -v -X PUT http://172.20.0.11:8098/riak/animals/polly?returnbody=true \
-H "Content-Type: application/json" \
-d '{"nickname" : "Sweet Polly Purebred", "breed" : "Purebred"}'
```

Riak links

- Links are metadata that associate one key to other keys

- The basic structure is this:

Link: </riak/bucket/key>; riaktag=\"whatever\“

- The key to where this value links is in pointy brackets (<…>), followed by a semicolon and then a tag describing how the link relates to this value (it can be whatever string we like).

- There can be many metadata links as necessary separated by commas

- Links are one-directional

Riak Example

- Let’s add 2 rooms for the dogs with links

```
curl -X PUT http://172.20.0.11:8098/riak/cages/1 \
-H "Content-Type: application/json" \
-H "Link: </riak/animals/polly>; riaktag=\"contains\"" \
-d '{"room" : 101}'

curl -X PUT http://172.20.0.11:8098/riak/cages/2 \
-H "Content-Type: application/json" \
-H "Link:</riak/animals/ace>;riaktag=\"contains\", </riak/cages/1>;riaktag=\"next_to\"" \
-d '{"room" : 102}'
```

Riak Example

- The links are seen only on the room records, not the animal records

curl -i http://172.20.0.11:8098/riak/animals/polly

curl -i http://172.20.0.11:8098/riak/animals/ace

curl -i http://172.20.0.11:8098/riak/cages/1

curl -i http://172.20.0.11:8098/riak/cages/2

Riak Example

```
ciprian@hadoop-master:~$ curl -i -w '\n' http://172.19.3.36:8098/riak/animals/ace
HTTP/1.1 200 OK
X-Riak-Vclock: a85hYGBgymDKBVI8wQtdOxbpPrRlYFAXymBKZMpjZXgiOXaZDyp9aoWIObKdGjxAaUGo9EtVoHQWAA==
Vary: Accept-Encoding
Server: MochiWeb/1.1 WebMachine/1.10.9 (cafe not found)
Link: </riak/animals>; rel="up"
Last-Modified: Sun, 19 Nov 2017 15:57:29 GMT
ETag: "4J49QrCprFeCtfU2DlNwZb"
Date: Sun, 19 Nov 2017 16:05:02 GMT
Content-Type: application/json
Content-Length: 60

{"nickname" : "The Wonder Dog", "breed" : "German Shepherd"}
ciprian@hadoop-master:~$ curl -i -w '\n' http://172.19.3.36:8098/riak/cages/2
HTTP/1.1 200 OK
X-Riak-Vclock: a85hYGBgzGDKBVI8Z49e0C65smo/A4OfcgZTInMeK8NX1bbLfFkA
Vary: Accept-Encoding
Server: MochiWeb/1.1 WebMachine/1.10.9 (cafe not found)
Link: </riak/animals/ace>; riaktag="contains", </riak/cages/1>; riaktag="next_to", </riak/cages>; rel="up"
Last-Modified: Sun, 19 Nov 2017 15:57:41 GMT
ETag: "4VPZpyDp4OFOUwpULVTW8A"
Date: Sun, 19 Nov 2017 16:05:08 GMT
Content-Type: application/json
Content-Length: 14

{"room" : 102}
```

Riak Link Walking

- Getting the linked data is achieved by appending a link spec to the URL that is structured like this: /_,_,_
- The underscores (_) in the URL represent wildcards to each of the link criteria:
  - Bucket
  - Tag (name of link)
  - Keep
- Keep (1 or 0) is used to keep the link when traversing multiple links
- It returns a multipart/mixed dump of headers plus bodies of all linked keys/values.
- Replace the underscores in the link spec to filter only values we want

Riak Example

curl http://172.20.0.11:8098/riak/cages/1/_,_,_

curl http://172.20.0.11:8098/riak/cages/1/_,contains,_

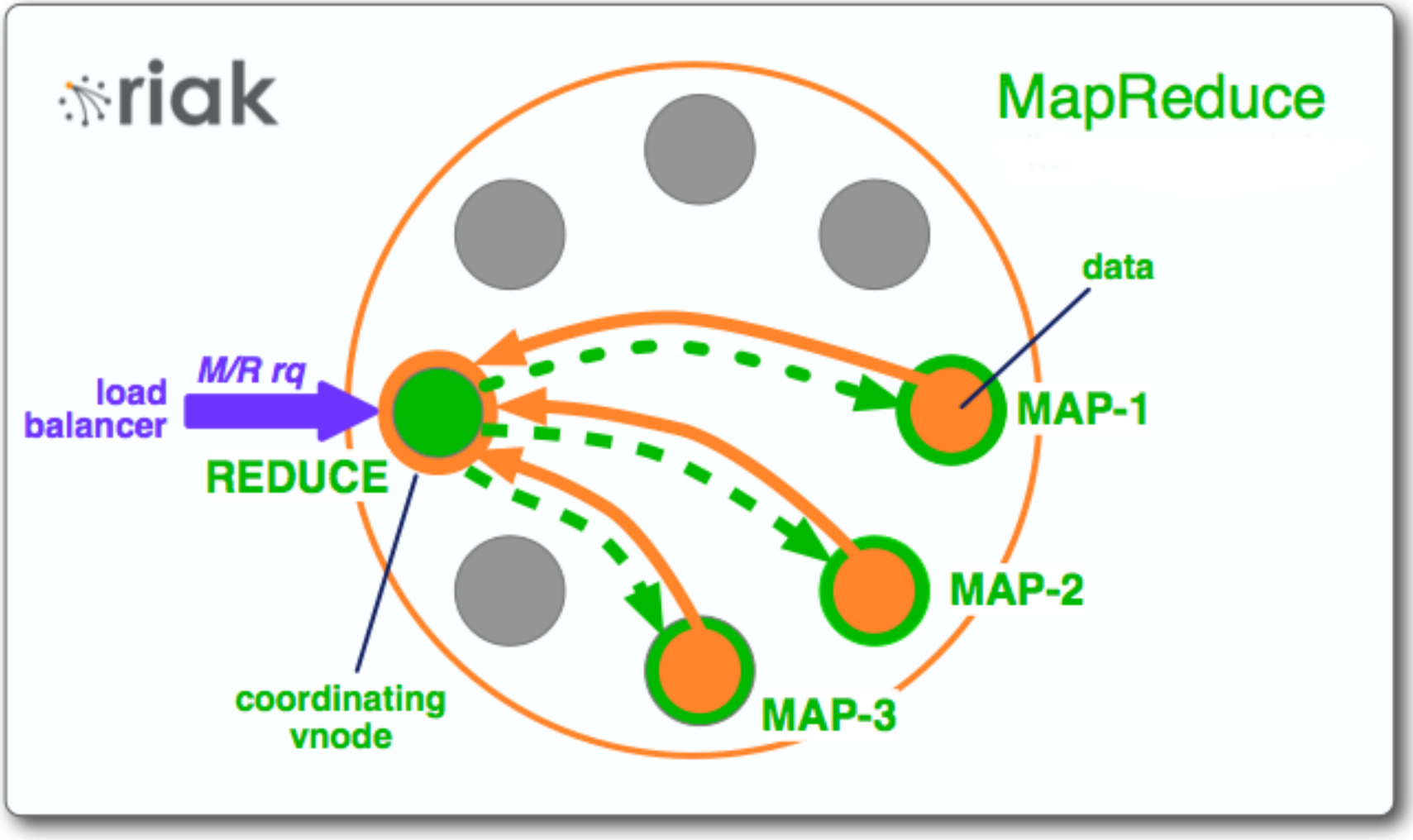
curl http://172.20.0.11:8098/riak/cages/2/_,contains,_

curl http://172.20.0.11:8098/riak/cages/2/_,next_to,_

curl http://172.20.0.11:8098/riak/cages/2/cages,next_to,1/animals,_,_

curl http://172.20.0.11:8098/riak/cages/2/cages,next_to,0/animals,_,_

Riak MapReduce



Bibliography

- Christof Strauch. "NoSQL databases." *Lecture Notes, Stuttgart Media University* 20.24 (2011): 79.

- Eric Redmond, Jim Wilson. Seven databases in seven weeks: a guide to modern databases and the NoSQL movement. 1nd Edition 2012

- Luc Perkins, Jim Wilson, and Eric Redmond. Seven databases in seven weeks: a guide to modern databases and the NoSQL movement. 2nd Edition 2018

- Riak 3.2.0 Documentation https://www.tiot.jp/riak-docs/index.html

University
Politehnica
of Bucharest

Computer
Science and
Engineering
Department

Document-Oriented Databases
MongoDB (I)

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@upb.ro

Overview

- Document-Oriented Databases
- MongoDB

Overview

- Document-Oriented Databases
- MongoDB

Document-Oriented Databases

- Document-Oriented Databases Management Systems (DODBMS) have evolved from Key-Value Databases
- DODBMS are used for storing, retrieving and managing semi-structured data
- DODBMS propose a schema-less flexible data representation for data providing more flexibility for data modeling

Document-Oriented Databases

- DODBMS use documents for storing data
- The term document may usually refer a Word or PDF documents, but in this case refers to JSON or XML files
- The flexibility provided by XML and JSON make it easier to modify documents than it is for tables in a RDBMS
- Documents are stored in collections

Document-Oriented Databases

- In DODBMS information is stored using fields (labels, tags, descriptions) to describe the data and values for that description

- New information (fields) can be added directly to a record without the need to modify the entire schema as it is for RDBMS

RDBMS vs DODBMS Comparison

RDBMS	DODBMS
Database (Schema)	Database
Table	Collection
Attribute	Field (label, tag, description)
Attribute values	Field (label, tag, description) value

RDBMS vs DODBMS Comparison

```
{     customer_id : 1,
      first_name : "Mark",
      last_name : "Smith",
      city : "San Francisco",
      phones: [ {
            type : “work”,
            number: “1-800-555-1212”
      },
      {     type : “home”,
            number: “1-800-555-1313”,
            DNC: true
      },
      {     type : “home”,
            number: “1-800-555-1414”,
            DNC: true
      }
   ]
}
```

Benefits of a DODBMS

- Flexible data modeling
- Fast write performance
- Fast query performance

Benefits of a DODBMS
Flexible data modeling

- Data from the web, mobile, social and IoT applications change the nature of the application's data model

- In a RDBMS, this changes impose the modification of the schema by altering tables and adding or removing new columns

- The Flexibility of DODBMS eliminate the need to force-fit the data into predefined attributes and tables

Benefits of a DODBMS
Fast write performance

- Some DODBMS prioritize write availability over strict data consistency
- This ensures that writes will always be executed even if there is a hardware or network failure
- In case of failure, the replication and eventual consistency mechanisms ensures that the environment will function

Benefits of a DODBMS
Fast query performance

- Most DODBMS
  - Provide powerful query engines for CRUD operations
  - Use indexes and secondary indexes to improve data retrieval
  - Provide aggregation frameworks for Data Analysis and Business Intelligence
    - Aggregation is either native or done through MapReduce

Overview

- Document-Oriented Databases
- MongoDB
  - Overview
  - Storage engine: MMAPv1
  - Storage engine: WiredTiger
  - In-Memory Storage Engine
  - GridFS

MongoDB – Overview

- MongoDB (from humongous) is focused on combining the critical capabilities of RDBMS with the innovations of NoSQL DBMS
- MongoDB is an open-source Document-Oriented Database Management System (DODBMS)
- MongoDB is written in C++, C and JavaScript
- Run on different OS: Linux (most distributions), Windows, MacOS
- Can be easily deployed in the Cloud as a DaaS (MongoDB Atlas, AWS, and MongoDB containers)

MongoDB – Overview

- MongoDB uses a flexible, dynamic schema to store data
- A MongoDB instance is started using mongod
- A record is stored in a document
- Multiple documents are stored in a collection

MongoDB – Overview

- Documents in a collection do not necessarily have the same structure:
  - The number of attributes can differ from one record to another
  - The fields' data type can differ from one record to another

- Although the database allows documents with a different number of attributes and for the same fields different data types, in the same collection records have almost the same structure

- In practice, documents usually model objects from a high-level programming language

MongoDB
Data representation

- MongoDB stores the data in BSON documents
- BSON is a binary-encoded serialization of JSON-like documents
  - Note: Serialization is the process of translating data structures or object state into a format that can be stored (e.g., in a file, memory buffer, etc.) or transmitted (e.g., across a network connection link, etc.) and reconstructed later (possibly in a different computer environment)
- A BSON document is limited in size to 16MB
- Solution to the size limitation MongoDB GridFS

```
{"hello": "world"} →
\x16\x00\x00\x00           // total document size
\x02                       // 0x02 = type String
hello\x00                  // field name
\x06\x00\x00\x00world\x00  // field value
\x00                       // 0x00 = type EOO ('end of object')
```

MongoDB
Data representation

- The main features of BSON are:
  - Lightweight – for fast retrieval of information over a computer network it is necessary to keep the overhead needed to store data to a minimum
  - Traversable – it is import to store data in a format that is easily parsed
  - Efficient – using C language data types, data is easily encoded and decoded

MongoDB
Key Features

- Support of multiple storage engines
- High availability through replication
- Automatic horizontal scaling though sharding
- High performance
- Rich query language

MongoDB
Key Features

- MongoDB supports multiple storage engines:
  - MMAPv1 Storage Engine (deprecated starting MongoDB v4.0, removed in MongoDB v4.2)
  - WiredTiger Storage Engine
  - In-Memory Storage

MongoDB
Key Features

- High performance data persistence
  - Support for embedded data models reduces I/O activity on database systems:
    - Supports nested documents
    - References can also be used
  - Indexes support faster queries
  - Indexes can include keys from embedded documents and arrays
  - Indexes can be added on text and geospatial data
  - MongoDB support compound indexes

MongoDB
Key Features

- Note on Indexes:
  - Index prefix is the label that defines the index, e.g.:

  {idx_1: 1}
  - Compound indexes use multiple fields in their structure, e.g.:

  {idx_1: 1, idx_2: 1, idx_3: 1}
  - Queries that use efficiently a compound index use the labels in order, e.g.
    - idx_1,
    - idx_1 and idx_2
    - idx_1 and idx_2 and idx_3

MongoDB
Key Features

- Note on Indexes:
  - Also works for our example:
    - idx_1 and idx_3
  - Not as efficient as creating a compound index only on idx_1 and idx_3
  - Queries that don't use the first label(s) (i.e., idx_1), don't use the compound index
    - idx_2
    - idx_3
    - idx_2 and idx_3
    - idx_3 and idx_2

MongoDB
Key Features

- Rich query language
  - Supports all the CRUD Operations
  - Supports data aggregation through MapReduce and Aggregation Pipeline
  - Supports Text Search and regular expressions
  - Supports Geospatial queries

MongoDB
Key Features

- High availability
  - Supports replication, called Replica Set, which provide:
    - Automatic failover
    - Data redundancy
    - Increased data availability
  - A Replica Set is a group of MongoDB servers that maintain the same data set

MongoDB
Key Features

- Horizontal Scalability
  - MongoDB provides horizontal scalability as part of its core functionality
  - Scalability is achieved through Sharding
  - Sharding distributes data across a cluster of machines
  - Form MongoDB 3.4 the support of creating zones of data on the shard key was added
  - Using zones in a balanced cluster, MongoDB directs reads and writes covered by a zone only to those shards inside the zone

Overview

- Document-Oriented Databases
- MongoDB
  - Overview
  - Storage engine: MMAPv1
  - Storage engine: WiredTiger
  - In-Memory Storage Engine
  - GridFS

MongoDB – MMAPv1

- MMAPv1 Storage Engine
  - The original storage engine
  - It excels at workloads with high volume of inserts, reads and in-place updates
  - Deprecated in v4.0, removed in v4.2
  - Features:
    - Journal
    - Record storage characteristics
    - Record allocation strategies
    - Memory use

MongoDB – MMAPv1 Journal

- To ensure data persistence, all modifications are registered to an on-disk Journal
- Journal writes happen more frequently than data files writes
- In the default configuration of the MMAPv1 there are Journal writes roughly every 100ms and data file writes every 60s
- To change the write intervals, change the values for the following parameters:
  - Journal file : storage.journal.commitIntervalMs
  - Data file: storage.syncPeriodSecs

MongoDB – MMAPv1 Journal

- The interval values represent the maximum amount of time between
  - The completion of a write operation and
  - When MongoDB writes to the data files or to the journal files

- In many cases MongoDB and the operating system flush data to disk more frequently (the above values represents a theoretical maximum)

- The journal allows MongoDB to successfully recover data from data files after a mongod instance exits without flushing all changes

MongoDB – MMAPv1
Record allocation strategies

- MongoDB supports multiple record allocation strategies that determine how mongod adds padding to a document when creating a record

- Because documents in MongoDB may grow after insertion and all records are contiguous on disk, the padding can reduce the need to relocate documents on disk following updates

- Relocations
  - Are less efficient than in-place updates
  - Can lead to storage fragmentation

- As a result, all padding strategies trade additional space for
  - Increased efficiency
  - Decreased fragmentation

MongoDB – MMAPv1
Record allocation strategies

- There are two allocations strategies
  - Power of 2 allocations
  - No padding allocation strategy (or exact fit allocations)

- The two different allocation strategies support different kinds of workloads:
  - The power of 2 allocations are more efficient for insert/update/delete workloads
  - Exact fit allocations is ideal for collections without update and delete workloads

MongoDB – MMAPv1
Record storage characteristics

- MongoDB uses Power of 2 Sized Allocations by default
- Every document in MongoDB is stored in a record which contains:
  - The document itself
  - Extra space, or padding
- Padding
  - Allows the document to grow as the result of updates
  - Minimizing the likelihood of reallocations

MongoDB – MMAPv1
Power of 2 Sized Allocations

- Appeared with MongoDB 3.0
- Uses the power of 2 sizes allocation as the default record allocation strategy for MMAPv1
- With this strategy, each record has a size in bytes that is a power of 2 (e.g., 32, 64, 128, 256, 512 … 2 MB)
- For documents larger than 2 MB, the allocation is rounded up to the nearest multiple of 2 MB

MongoDB – MMAPv1
Power of 2 Sized Allocations

- Properties
  - Can efficiently reuse freed records to reduce fragmentation:
    - Quantizing record allocation sizes into a fixed set of sizes increases the probability that an insert will fit into the free space created by an earlier document deletion or relocation.
  - Can reduce moves:
    - The added padding space gives a document room to grow without requiring a move
    - In addition to saving the cost of moving, this results in less updates to indexes
    - Although the power of 2 sizes strategy can minimize moves, it does not eliminate them entirely

MongoDB – MMAPv1

No Padding Allocation Strategy

- Is used for collections whose workloads do not change the document sizes
- Workloads consist of
  - Insert-only operations
  - Update operations that do not increase document size
- Prior MongoDB v3.0.0, the allocation strategy included a dynamically calculated padding as a factor of the document size

MongoDB – MMAPv1
Memory use

- MongoDB automatically uses all free memory on the machine as its cache
- System resource monitors show that MongoDB uses a lot of memory (its usage is dynamic)
- If another process suddenly needs half the server's RAM, MongoDB will yield cached memory to the other process

MongoDB – MMAPv1
Memory use

- The operating system's virtual memory subsystem manages MongoDB's memory
- This means that MongoDB will use as much free memory as it can, swapping to disk as needed
- Deployments with enough memory to fit the application's working data set in RAM will achieve the best performance

Overview

- Document-Oriented Databases
- MongoDB
  - Overview
  - Storage engine: MMAPv1
  - Storage engine: WiredTiger
  - In-Memory Storage Engine
  - GridFS

MongoDB – WiredTiger

- The default storage for the latest versions
- Introduced from MongoDB v3.0, main Storage Engine from v4.2
- Features:
  - Document level concurrency
  - Snapshots and checkpoints
  - Journal
  - Compression
  - Memory use

MongoDB – WiredTiger
Document level concurrency

- WiredTiger uses document-level concurrency control for write operations
- Multiple clients can modify different documents of a collection at the same time
- WiredTiger uses optimistic concurrency control (optimistic locking) for most read and write operations

MongoDB – WiredTiger
Document level concurrency

- MongoDB uses reader-writer locks that allow concurrent readers shared access to a resource, such as a database or collection

- Locking modes create a lock on the resource

MongoDB – WiredTiger
Document level concurrency

- Shared Locking mode for reads
  - Allow multiple transactions to read the same data, but prevent any transaction from modifying it

MongoDB – WiredTiger
Document level concurrency

- Exclusive Locking mode for write
  - Protect updates to file resources, both recoverable and non-recoverable
  - Can be owned by only one transaction at a time
  - Any transaction that requires an exclusive lock must wait if another task currently owns an exclusive lock or a shared lock against the requested resource.

MongoDB – WiredTiger
Document level concurrency

- When locking at a certain granularity, all higher levels are locked using an intent lock
- Intent locks
  - Are used by a transaction to inform another transaction about its intention to acquire a lock
  - Ensure that data modification are executed properly by preventing another transaction to acquire a lock on the next in hierarchy object
- WiredTiger uses intent locks only at the Global, Database, and Collection levels
- When the storage engine detects conflicts between two operations, one will incur a write conflict causing MongoDB to transparently retry that operation

MongoDB – WiredTiger
Document level concurrency

- Intent Shared Locking mode to indicate an intent to read using a finer granularity lock
  - Indicates that the transaction has the intention to read some lower hierarchy resources by acquiring Shared Locks individually on those resources lower in the hierarchy
  - Example: When a transaction locks a document using a Shared Lock mode (to read the date), the database server places an Intent Shared lock on the collection to provide an instant check that no other transaction holds an Exclusive Lock on the collection

MongoDB – WiredTiger
Document level concurrency

- Intent Exclusive Locking mode to indicate an intent write using a finer granularity lock
  - Indicates the intention of a transaction to modify some (but not all) resources lower in the hierarchy by placing Exclusive locks on those individual resources
  - Example:
    - When a transaction tries to modify a record using an Exclusive Lock, an Intent Exclusive Lock is put in place on the collection
    - The lock cannot be put in place until all other locks have been released on this record (or collection)
    - While the exclusive lock is in place, no other intent locks can be applied on the record (or collection)

MongoDB – WiredTiger
Document level concurrency

- A single database can simultaneously be locked using an Intent Shared Lock and Intent Exclusive Lock
- An Exclusive Lock cannot coexist with any other locking modes
- A Shared Lock can only coexist with Intent Shared Locks
- For example, when locking a collection for writing (using an Exclusive Lock), both the corresponding database lock and the global lock must be locked using Intent Exclusive Locking

MongoDB – WiredTiger
Document level concurrency

- Global "instance-wide" locks are required in general for some global operations
  - E.g., short-lived operations involving multiple databases
- Exclusive database locks are required for other operations
  - E.g., dropping a collection

MongoDB – WiredTiger
Snapshots and checkpoints

- WiredTiger uses MultiVersion Concurrency Control (MVCC)
  - MVCC is a database optimization technique that creates duplicate copies of records so that data can be safely read and updated at the same time
  - Also available for other databases, e.g., Couchbase

- WiredTiger provides a point-in-time snapshot of the data to the transaction at the start of an operation

- A snapshot presents a consistent view of the in-memory data

MongoDB – WiredTiger
Snapshots and checkpoints

- When writing to disk, WiredTiger writes all the data in a snapshot to disk in a consistent way across all data files
- The now-durable data act as a checkpoint in the data files
- The checkpoint ensures that the data files are consistent up to and including the last checkpoint (i.e., checkpoints can act as recovery points)
- MongoDB configures WiredTiger to create checkpoints (i.e., write the snapshot data to disk) at intervals of 60 seconds or 2 gigabytes of journal data

MongoDB – WiredTiger
Snapshots and checkpoints

- During the write of a new checkpoint, the previous checkpoint is still valid
- As such, even if MongoDB terminates or encounters an error while writing a new checkpoint, upon restart, MongoDB can recover from the last valid checkpoint
- The new checkpoint becomes accessible and permanent when WiredTiger's metadata table is atomically updated to reference the new checkpoint
- Once the new checkpoint is accessible, WiredTiger frees pages from the old checkpoints

MongoDB – WiredTiger
Snapshots and checkpoints

- MongoDB can recover from the last checkpoint when using WiredTiger, even without journaling
- However, to recover changes made after the last checkpoint, MongoDB should be run with journaling
- From v4.2, the following options are not available anymore for replica set members :
  - --nojournal parameter for the mongod demon
  - storage.journal.enabled: false in the configuration file

MongoDB – WiredTiger Snapshot History Retention

- Starting in MongoDB 5.0, you can use the minSnapshotHistoryWindowInSeconds parameter to specify how long WiredTiger keeps the snapshot history.
- Increasing the value of minSnapshotHistoryWindowInSeconds increases disk usage because the server must maintain the history of older modified values within the specified time window.
- The amount of disk space used depends on your workload, with higher volume workloads requiring more disk space.
- MongoDB maintains the snapshot history in the WiredTigerHS.wt file, located in your specified dbPath.

MongoDB – WiredTiger Journal

- WiredTiger uses checkpoints to
  - Provide a consistent view of data on disk
  - Allow MongoDB to recover from the last checkpoint

- However, if MongoDB exits unexpectedly in between checkpoints, journaling is required to recover information that occurred after the last checkpoint

MongoDB – WiredTiger Journal

- The recovery process with journaling:
  - Looks in the data files to find the identifier of the last checkpoint
  - Searches in the journal files for the record that matches the identifier of the last checkpoint
  - Apply the operations in the journal files since the last checkpoint

MongoDB – WiredTiger Journaling Process

- WiredTiger creates one journal record for each client who initiated write operations
  - The journal record includes any internal write operations caused by the initial write
- WiredTiger creates a single journal record that includes both the update operation and its associated index modifications
- MongoDB configures WiredTiger to use in-memory buffering for storing the journal records
- Threads coordinate to allocate and copy into their portion of the buffer
- All journal records up to 128 kB are buffered

MongoDB – WiredTiger Journaling Process

- WiredTiger syncs the buffered journal records to disk according to the following intervals or conditions:
  - Every 50 milliseconds from MongoDB v3.2
  - MongoDB sets checkpoints to occur in WiredTiger on user data at an interval of 60 seconds or when 2 GB of journal data has been written, whichever occurs first.
  - WiredTiger forces a sync of the WiredTiger journal files if the write operation includes a write concern that requests the acknowledgement from MongoDB that the write operation has been written to the journal

MongoDB – WiredTiger Journaling Process

- WiredTiger syncs the buffered journal records to disk according to the following intervals or conditions:
  - When WiredTiger creates a new journal file, WiredTiger syncs the previous journal file
  - WiredTiger creates a new journal file approximately every 100 MB of data
    - This happens because MongoDB uses a journal file size limit of 100 MB

MongoDB – WiredTiger Journal Files

- For the journal files, MongoDB creates a subdirectory named journal under the database directory (in this directory the data is stored)
- WiredTiger journal files have names with the following format WiredTigerLog.<sequence> where <sequence> is a zero-padded number starting from 0000000001
- Journal files contain a record per each write operation
- Each record has a unique identifier
- MongoDB configures WiredTiger to use snappy compression for the journaling data
  - Snappy is a fast data compression and decompression library written in C++ by Google based on ideas from LZ77 and open-sourced in 2011.

MongoDB – WiredTiger Journal Files

- Minimum log record size for WiredTiger is 128 bytes
- If a log record is 128 bytes or smaller, WiredTiger does not compress that record
- WiredTiger journal files have a maximum size limit of approximately 100 MB
- WiredTiger creates a new journal file once the file exceeds that limit
- WiredTiger automatically removes old journal files to maintain only the files needed to recover from last checkpoint
- WiredTiger will pre-allocate journal files.

MongoDB – WiredTiger Compression

- WiredTiger supports compression for all collections and indexes
- Compression minimizes storage use at the expense of additional CPU
- By default, WiredTiger uses
  - Block compression with the snappy compression library for all collections
  - Prefix compression for all indexes
- For collections, block compression with zlib is also available

MongoDB – WiredTiger Compression

- To specify an alternate compression algorithm or no compression, use the following setting:

storage.wiredTiger.collectionConfig.blockCompressor

- For indexes, to disable prefix compression, use the following setting:

storage.wiredTiger.indexConfig.prefixCompression

- Compression settings are also configurable on a per-collection and per-index basis during collection and index creation

MongoDB – WiredTiger Compression

- The default compression settings balance storage efficiency and processing requirements for most workloads

- As already mentioned, the WiredTiger journal is also compressed by default

MongoDB – WiredTiger Memory use

- MongoDB utilizes both the WiredTiger internal cache and the filesystem cache

- Starting with MongoDB v3.4, the WiredTiger internal cache, by default, will use the larger of either:
  - 50% of RAM minus 1 GB, or
  - 256 MB

MongoDB – WiredTiger Memory use

- Different representations are used for data in the WiredTiger internal cache versus the on-disk format
  1. Data in the filesystem cache is the same as the on-disk format
     - This method benefits of any compression for data files
     - The filesystem cache is used by the operating system to reduce disk I/O
  2. Indexes loaded in the WiredTiger internal cache have a different data representation to the on-disk format
     - This method can still take advantage of index prefix compression to reduce RAM usage
     - Index prefix compression de-duplicates common prefixes from indexed fields

MongoDB – WiredTiger
Memory use

3. Collection data in the WiredTiger internal cache is uncompressed and uses a different representation from the on-disk format
   - Block compression can provide significant on-disk storage savings
   - The data must be uncompressed to be manipulated by the server

Overview

- Document-Oriented Databases
- MongoDB
  - Overview
  - Storage engine: MMAPv1
  - Storage engine: WiredTiger
  - In-Memory Storage Engine
  - GridFS

In-Memory Storage Engine

- First release in MongoDB Enterprise version 3.2.6
- The in-memory storage engine is part of general availability (GA) in the 64-bit builds.
- Other than some metadata and diagnostic data, the in-memory storage engine does not maintain any on-disk data, including configuration data, indexes, user credentials, etc.
- By avoiding disk I/O, the in-memory storage engine allows for more predictable latency of database operations

In-Memory Storage Engine

- Concurrency
  - The in-memory storage engine uses document-level concurrency control for write operations
  - Multiple clients can modify different documents of a collection at the same time

- Memory Use
  - In-memory storage engine requires that all its data (including indexes, oplog if mongod instance is part of a replica set, etc.) must fit into the specified amount of memory set when starting the instance
  - By default, the in-memory storage engine uses 50% of physical RAM minus 1 GB
  - If a write operation would cause the data to exceed the specified memory size, MongoDB returns an error

In-Memory Storage Engine

- Durability
  - The in-memory storage engine is non-persistent and does not write data to a persistent storage
  - Non-persisted data includes application data and system data, such as users, permissions, indexes, replica set configuration, sharded cluster configuration, etc.
  - The concept of journal or waiting for data to become durable does not apply to the in-memory storage engine
  - The recovery of in-memory data is impossible

- Transactions
  - Starting in MongoDB 4.2, transactions are supported on replica sets and sharded clusters where:
    - The primary uses the WiredTiger storage engine
    - The secondary members use either the WiredTiger storage engine or the in-memory storage engines

In-Memory Storage Engine Deployment Architectures

- Standalone

- MongoDB instances that use in-memory storage engine can run as part of
  - A replica set
  - A sharded cluster

In-Memory Storage Engine Deployment Architectures – Replica Set

- Create multiple in-memory instances that can become primary
- Create at least 1 hidden WireTiger instance as secondary to avoid losing data
- Set the priority of the hidden instance(s) to 0 so it will never become a primary
- Only the instances running with the in-memory storage engine can become the primary
- Clients can connect only to the in-memory storage engine instances
- Even if all instances running in-memory storage engine crash and restart, they can sync from the member running WiredTiger
- The instance running with WiredTiger persists the data to disk, including the user data, indexes, and replication configuration information.

In-Memory Storage Engine Deployment Architectures – Sharded Cluster

- Create at least 1 replica set that you will use as a shard
  - Create multiple in-memory instances
  - Create at least 1 hidden WireTiger instance
  - Set the priority of the hidden instance(s) to 0 so it will never become a primary
- Create normal replica sets without in-memory instances
- The shard(s) with in-memory instances are called the in-memory shards
- The other shards (that do not use in-memory instances) become persistent shards
- For each sharded collection that should reside on the in-memory shard(s), assign to the entire chunk range
- For each sharded collection that should reside across the persisted shards, assign to the entire chunk range

Overview

- Document-Oriented Databases
- MongoDB
  - Overview
  - Storage engine: MMAPv1
  - Storage engine: WiredTiger
  - In-Memory Storage Engine
  - GridFS

MongoDB – GridFS

- GridFS is a specification for storing and retrieving files that exceed the BSON-document size limit of 16 MB
- Instead of storing a file in a single document, GridFS divides the file into parts, or chunks
- Each chunk is stored as a separate document (in two separate collections)
- GridFS is useful for:
  - Storing files that exceed 16 MB
  - Storing any files for which you want access without having to load the entire file into memory.

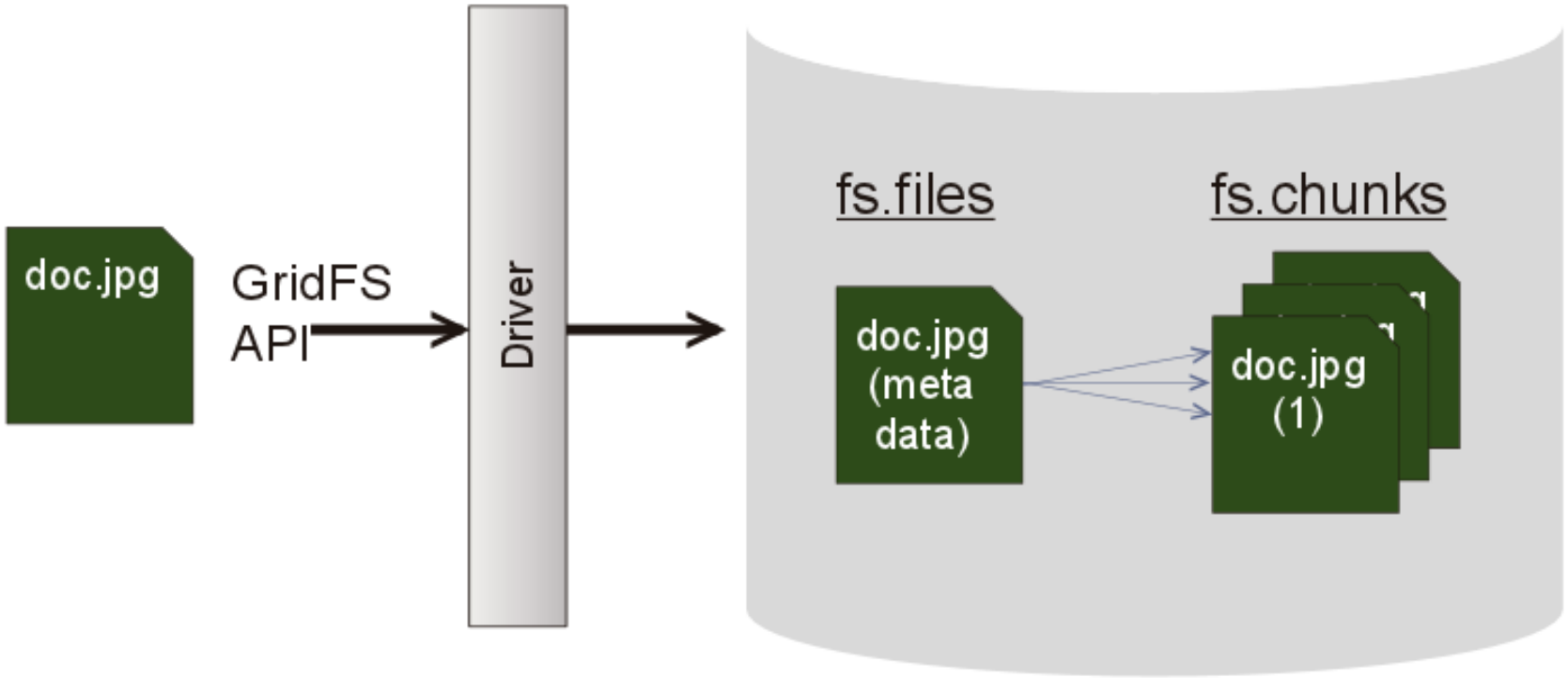
MongoDB – GridFS

- GridFS uses a chunk size of 255 kB by default
  - GridFS divides a file into chunks of 255 kB with the exception of the last chunk
  - The last chunk is only as large as necessary
  - Similarly, files that are no larger than the chunk size
    - Only have a final chunk
    - Use only as much space as needed plus some additional metadata

MongoDB – GridFS

- GridFS uses two collections to store files.
  - One collection stores the file chunks
  - The other stores file metadata
- The driver will reassemble the chunks as needed when querying a file stored on GridFS
- Range queries can be performed on files stored on GridFS
- Information from arbitrary sections of files can also be accessed:
  - E.g., If a video or audio file is stored then you can skip through them

MongoDB – GridFS



MongoDB – GridFS
When to use MongoDB GridFS

- In some situation is more efficient to store large files in a MongoDB database than on a System-Level Filesystem:
  - If the filesystem limits the number of files in a directory
  - To access information from portions of large files without having to load whole files into memory
  - To keep files and their metadata automatically synced and deployed across several systems and facilities

MongoDB – GridFS
When NOT to use MongoDB GridFS

- If the content of the entire file needs to be updated atomically

- As an alternative:
  - Store multiple versions of each file and specify the current version of the file in the metadata
  - Update the metadata field that indicates the "latest" status in an atomic update after uploading the new version of the file
  - The previous versions of the file can be removed if needed

MongoDB – GridFS

- If your files are all smaller than the 16 MB BSON Document Size limit:
  - Consider storing each file in a single document instead of using GridFS
  - The BinData data type can be used to store the binary data
  - The driver documentation presents all the details on how to use BinData

- MongoDB Driver:
  - An application communicates with MongoDB by way of a client library called a driver
  - Handles all interaction with the database in a language appropriate to the application (e.g., Python, C, C++, Java, etc.)

Bibliography

- MongoDB Documentation: https://www.mongodb.com/docs/

University
Politehnica
of Bucharest

Computer
Science and
Engineering
Department

MongoDB (II)

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@cs.pub.ro

Overview

- MongoDB
  - Replication
  - Sharding
  - Using MongoDB

Overview

- MongoDB
  - Replication
  - Sharding
  - Using MongoDB

MongoDB – Replication

- mongod is the instance of a MongoDB server
- A replica set in MongoDB is a group of mongod processes that maintain the same data set.
- Replica sets provide
  - Redundancy
  - High availability
- Replica sets are the basis for all production deployments

MongoDB – Replication

- Replication provides
  - Redundancy
    - With multiple copies of data on different database servers, replication provides a level of fault tolerance against the loss of a single database server
  - High availability:
    - In some cases, replication can provide increased read capacity as clients can send read operations to different servers

MongoDB – Replication

- Maintaining copies of data in different data centers can increase
  - Data locality
  - Availability for distributed applications
- Additional copies can be maintained for dedicated purposes, such as:
  - Disaster recovery
  - Reporting
  - Backup

MongoDB – Replication

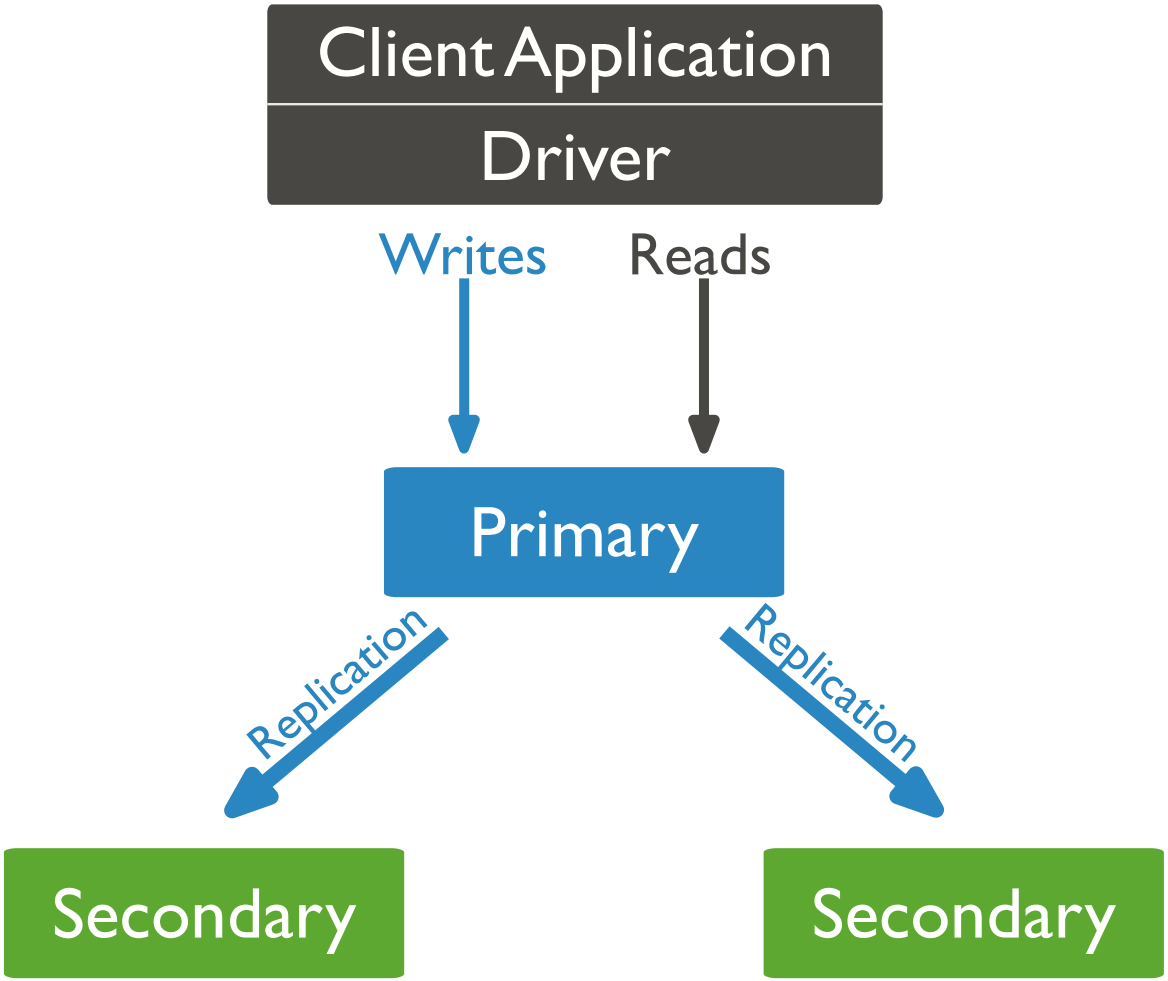
- A replica set is a group of mongod instances that maintain the same data set

- A replica set contains
  - Several data bearing nodes
  - Optionally one arbiter node

- The data bearing nodes
  - One and only one member is deemed the primary node
  - The other nodes are deemed secondary nodes

- The primary node
  - Receives all write operations
  - Records all changes to its data sets in its operation log

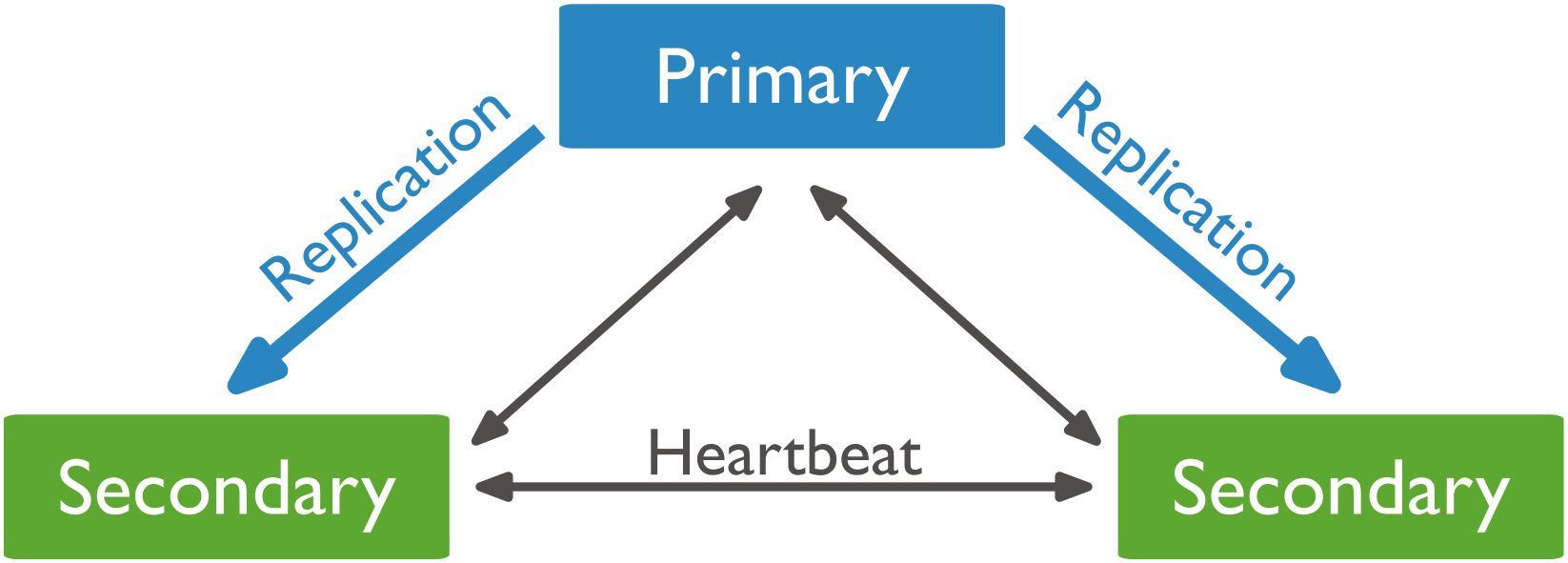
MongoDB – Replication



MongoDB – Replication

- The secondaries
  - Replicate the primary's oplog (operations log)
    - oplog: A special capped collection that keeps a rolling record of all operations that modify the data stored in the databases
    - capped collection: A fixed-sized collection that automatically overwrites its oldest entries when it reaches its maximum size
  - Apply the operations to their data sets such that the secondaries' data sets reflect the primary's data set
- If the primary is unavailable, an eligible secondary will hold an election to elect itself the new primary

MongoDB – Replication

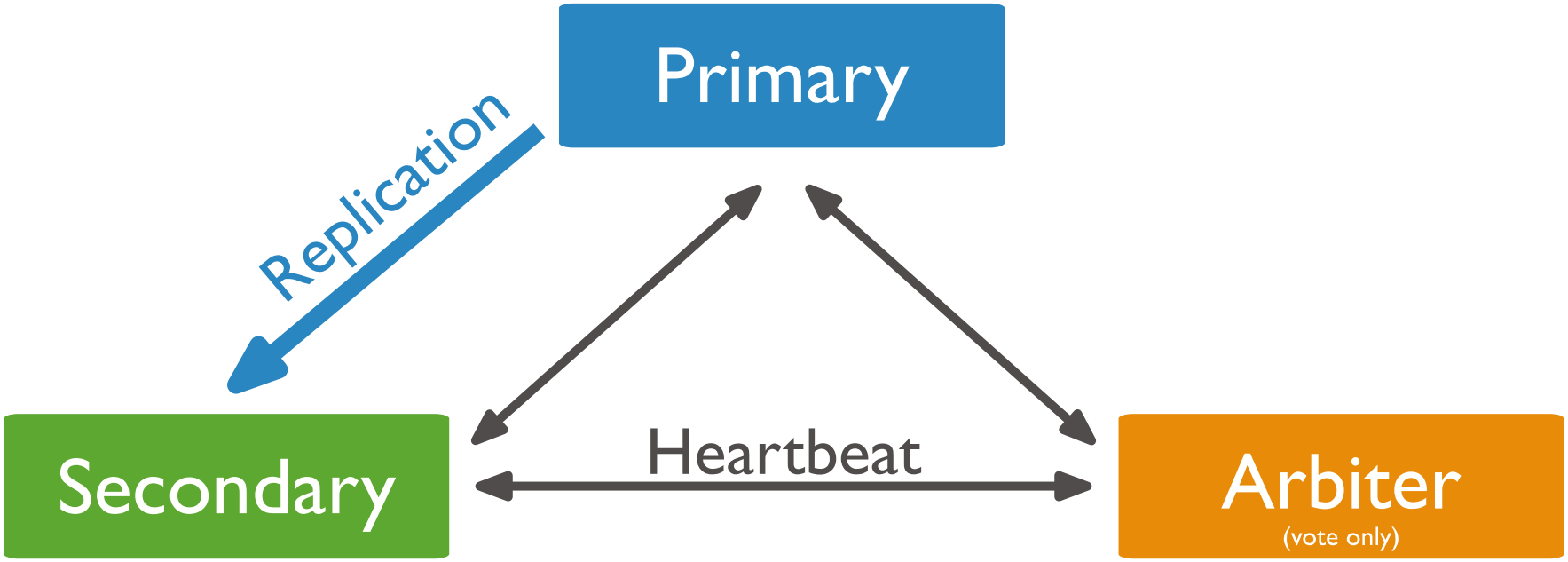


MongoDB – Replication

- To maintain the quorum of votes in a replica set an Arbiter can be added

- Arbiters
  - Do not maintain a data set
  - Respond to heartbeat and election requests by other replica set members
  - Can be a good way to provide replica set quorum functionality with a cheaper resource cost than a fully functional replica set member with a data set

MongoDB – Replication



MongoDB – Replication

MongoDB – Replication

- If your replica set has an even number of members, add an arbiter to obtain most votes in an election for primary
- Arbiters do not require dedicated hardware
- An arbiter will always be an arbiter whereas
  - A primary may step down and become a secondary during an election
  - A secondary may become the primary during an election

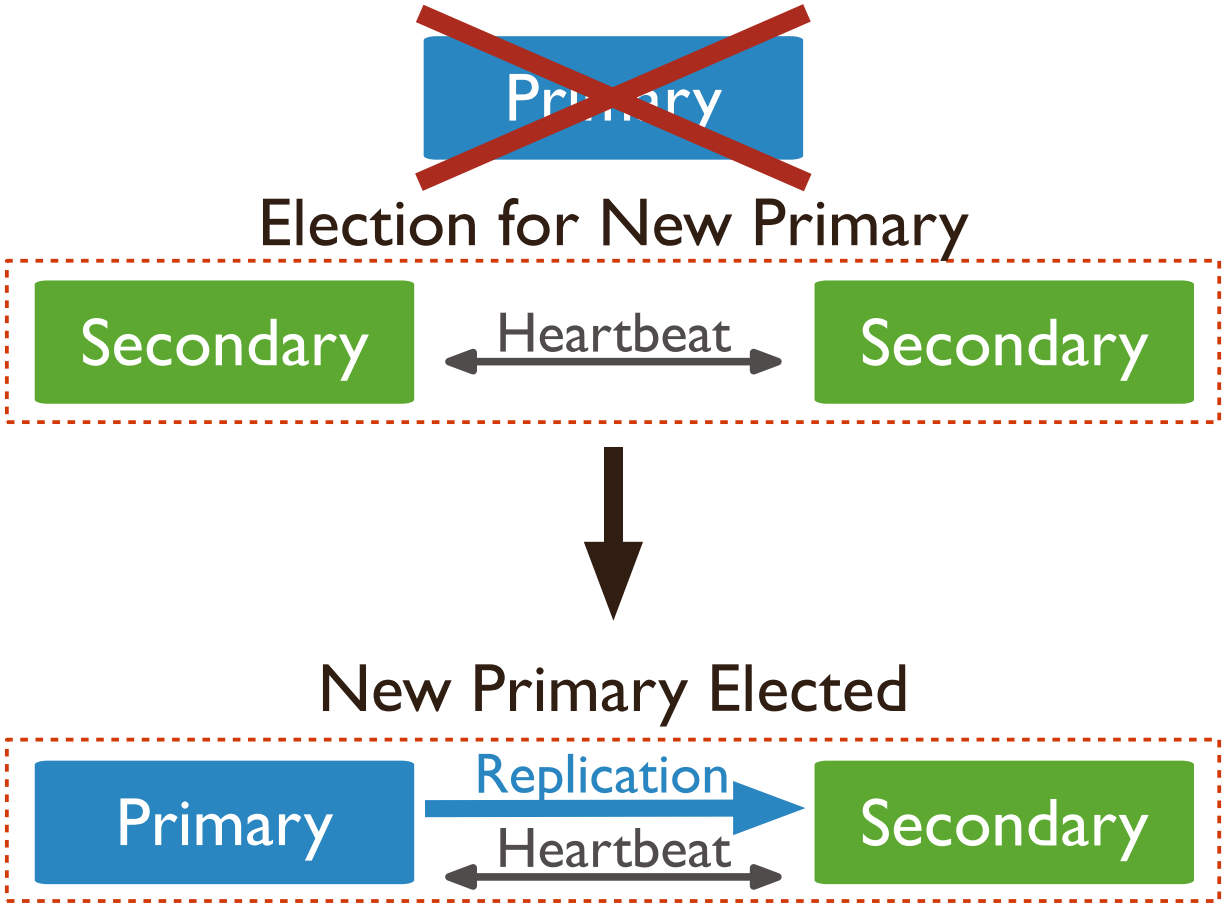
MongoDB – Replication

- Asynchronous Replication
  - Secondaries are updated by applying operations from the primary asynchronously
  - By applying operations after the primary, sets can continue to function despite the failure of one or more members

MongoDB – Replication

- Automatic Failover
  - If the primary node does not respond to a heartbeat for more than 10 seconds an eligible secondary holds an election to elect itself the new primary
  - The first secondary to hold an election and receive most of the members' votes becomes primary

MongoDB – Replication



MongoDB – Replication

- Replication Read Operations
  - By default, clients read from the primary
  - However, clients can specify to read from Secondaries
  - Because of asynchronous replication
    - There is no guaranty that secondaries have the latest information
    - The latest information is only on the primary node
  - Clients can see the results of writes before the write becomes durable

Overview

- MongoDB
  - Replication
  - Sharding
  - Using MongoDB

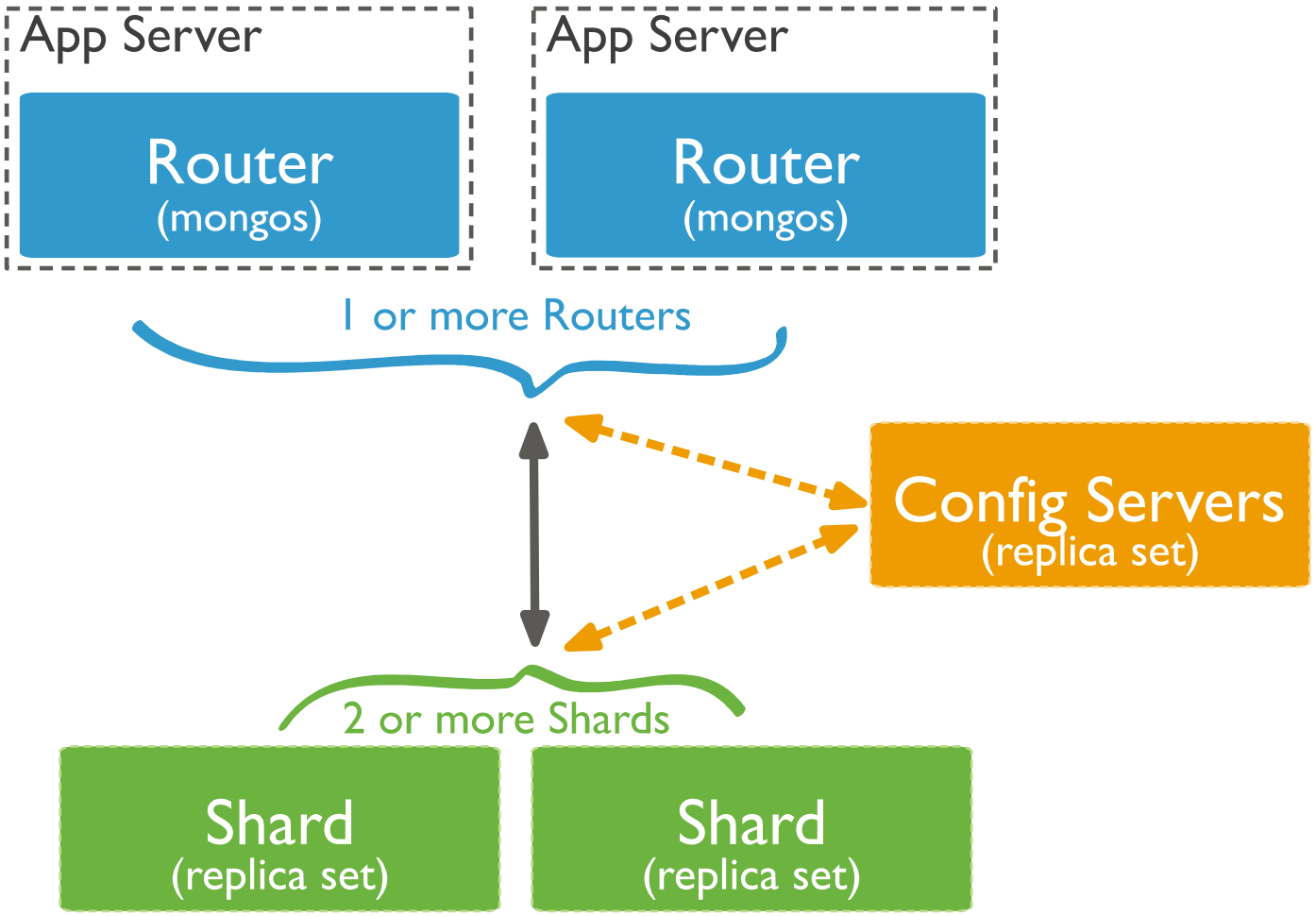
MongoDB – Sharding

- Sharding is a method for distributing data across multiple machines
- MongoDB uses sharding to support deployments with very large data sets and high throughput operations
- Database systems with large data sets or high throughput applications can challenge the capacity of a single server
- High query rates can exhaust the CPU capacity of the server
- Working set sizes larger than the system's RAM stress the I/O capacity of disk drives
- MongoDB supports horizontal scaling through sharding
- MongoDB shards data at the collection level, distributing the collection data across the shards in the cluster

MongoDB – Sharding

- A MongoDB sharded cluster consists of the following components:
  - shard:
    - Each shard contains a subset of the sharded data
    - Each shard must be deployed as a replica set
  - mongos:
    - The mongos acts as a query router
    - Provides an interface between client applications and the sharded cluster
  - config servers:
    - Config servers store metadata and configuration settings for the cluster
    - As of MongoDB 3.4, config servers must be deployed as a replica set (Configuration Servers Replica Set - CSRS)

MongoDB – Sharding



MongoDB – Sharding

- Shard Keys
  - To distribute the documents in a collection, MongoDB partitions the collection using the shard key
  - The shard key consists of an immutable field or fields that exist in every document in the target collection
  - The shard key is chosen when sharding the collection
  - The choice of shard key cannot be changed after sharding
  - The shard key can be a composite key
  - A sharded collection can have only one shard key

MongoDB – Sharding

- Shard Keys
  - To shard a non-empty collection:
    - The collection must have an index that starts with the shard key
  - To shard empty collections:
    - MongoDB creates the index if the collection does not already have an appropriate index for the specified shard key

MongoDB – Sharding

- Shard Keys
  - For a sharded cluster, the choice of shard key affects the:
    - Performance
    - Efficiency
    - Scalability
  - A cluster with the best possible hardware and infrastructure can be bottlenecked by the choice of shard key
  - The choice of shard key and its backing index can also affect the sharding strategy that your cluster can use

MongoDB – Sharding

- Chunks
  - MongoDB partitions sharded data into chunks
  - Each chunk has an inclusive lower and exclusive upper range based on the shard key
  - MongoDB migrates chunks across the shards in the sharded cluster using the sharded cluster balancer
  - The balancer attempts to achieve an even balance of chunks across all shards in the cluster

MongoDB
Advantages of sharding

- Read/Write:
  - MongoDB distributes the read and write workload across the shards in the sharded cluster
  - Each shard processes a subset of cluster operations
- Storage Capacity:
  - Sharding distributes data across the shards in the cluster
  - Each shard contains a subset of the total cluster data
- High Availability:
  - A sharded cluster can continue to perform partial read / write operations even if one or more shards are unavailable

Overview

- MongoDB
  - Replication
  - Sharding
  - Using MongoDB

Installing MongoDB

- To install MongoDB:
  - Download the package for your OS
  - For Ubuntu 24.04 x64:

1. Get the tgz package from https://www.mongodb.com/try/download/community
2. Unzip: $ tar -zxvf mongodb-linux-x86_64-ubuntu2404-8.0.3.tgz
3. Create a directory for the data: $ mkdir mongo_data
4. Start the server: $ mongod --dbpath mongo_data

Starting a MongoDB instance

- When mongo is started the default host is localhost (127.0.0.1) and the default port is 27017

- If you want to change the host, when you start mongod use --bind_ip and give an IP

```
$ mongod --dbpath mongo_data --bind_ip IP
```

- If there is a binding between the host name and an IP (in /etc/hosts) you can use the hostname instead of the IP to connect to MongoDB

- If you want to change the default port, when you start mongod use --port and give a new port

```
$ mongod --dbpath mongo_data --port 27110
```

Connecting to MongoDB

- To connect to MongoDB use mongosh

$ mongosh [--host {IP|hostname}] [--port {port_no}] [DB_name]

$ mongosh [{IP|hostname}:{port_no}/][DB_name]

- If no database name is specified, then mongo will connect directly to the admin database

Replication in MongoDB

- Start multiple instances using mongod with the --replSet ReplicaName parameter

- Local replication example

1. Create a data directory for each node (make sure to have the privileges):

```
$ mkdir -p node1_data node2 _data node3_data
```

2. Start the instances:

```
$ nohup mongod --replSet rs1 --dbpath node1_data --port 27021 > log_n1 2>&1 &
$ nohup mongod --replSet rs1 --dbpath node2_data --port 27022 > log_n2 2>&1 &
$ nohup mongod --replSet rs1 --dbpath node3_data --port 27023 > log_n3 2>&1 &
```

Replication in MongoDB

- Initialize the replication

- Connect to one of the nodes:

```
$ mongosh localhost:27021
```

- Initialize the replica set

```
> rs.initiate( {
   _id: "rs1",
   members: [  { _id : 0, host : "localhost:27021" },  { _id : 1, host
:  "localhost:27022" }, { _id : 2, host : "localhost:27023" }   ]  } )
```

- Verify the status of the replica set

```
> rs.status()
```

Replication in MongoDB

- To read from a secondary:
  - Connect to the secondary node
  - Issue the following command

```
> rs.secondaryOk() // depricated
> db.getMongo().setReadPref("primaryPreferred")
```

  - Disable reading from secondary

```
> db.getMongo().setReadPref("primary")
```

Sharding in MongoDB

- Create the directories for the config and shard nodes

```
$ mkdir -p conf1_data conf2_data conf3_data
$ mkdir -p rs1node1_data rs1node2_data rs1node3_data
$ mkdir -p rs2node1_data rs2node2_data rs2node3_data
$ mkdir -p rs3node1_data rs3node2_data rs3node3_data
```

- Create the configuration servers

```
$ nohup bin/mongod --configsvr --replSet ConfRepl --dbpath conf1_data --port 27121 > log_conf1 2>&1 &
$ nohup bin/mongod --configsvr --replSet ConfRepl --dbpath conf2_data --port 27122 > log_conf2 2>&1 &
$ nohup bin/mongod --configsvr --replSet ConfRepl --dbpath conf3_data --port 27123 > log_conf3 2>&1 &
```

Sharding in MongoDB

- Initialize replication for the config servers

```
$ mongosh localhost:27121
> rs.initiate(  {
   _id: "ConfRepl",
   configsvr: true,
   members: [
                        { _id : 0, host : "localhost:27121" },
                        { _id : 1, host : "localhost:27122" } ,
                        { _id : 2, host : "localhost:27123" }
            ]
} )
```

Sharding in MongoDB

- Start the sharding replica set nodes

```
$ nohup bin/mongod --shardsvr --replSet RS1 --dbpath rs1node1_data --port 27011  > log_rs1node1 2>&1 &
$ nohup bin/mongod --shardsvr --replSet RS1 --dbpath rs1node2_data --port 27012  > log_rs1node2 2>&1 &
$ nohup bin/mongod --shardsvr --replSet RS1 --dbpath rs1node3_data --port 27013  > log_rs1node3 2>&1 &
```

- And initialize replication

```
$ bin/mongosh localhost:27011
> rs.initiate({
   _id: "RS1",
   members: [
      { _id : 0, host : "localhost:27011" },
      { _id : 1, host : "localhost:27012" },
      { _id : 2, host : "localhost:27013" }
   ]
})
```

Note: Do this 2 more times for the other 2 shards – change the configuration accordingly for each shard

Sharding in MongoDB

- Start a mongos instance

```
$ nohup bin/mongos --configdb
ConfRepl/localhost:27121,localhost:27122,localhost:27123 --port 27130 >
/home/log_mongos 2>&1 &
```

- Connect to the mongos instance and add the shards

```
$ bin/mongosh localhost:27130

> sh.addShard("RS1/localhost:27011")
> sh.addShard("RS1/localhost:27012")
> sh.addShard("RS1/localhost:27013")
> sh.addShard("RS2/localhost:27021")
> sh.addShard("RS2/localhost:27022")
> sh.addShard("RS2/localhost:27023")
> sh.addShard("RS3/localhost:27031")
> sh.addShard("RS3/localhost:27032")
> sh.addShard("RS3/localhost:27033")
```

Sharding in MongoDB

- Verify the sharding status

```
> sh.status()
```

- Create a collection

```
> use BD2
> var vec = [ {x: 0 }]
> vec.length
> for (var i=1; i<100000000; i++){ vec.push({x: i})}
> vec.length
> db.documents.insertMany(vec);
> db.documents.findOne()
```

Sharding in MongoDB

- Enable sharding using on a database **sh.enableSharding("db_name")**

```
sh.enableSharding(“BD2")
```

- Shard the collection using **sh.shardCollection("db_name.collection_name", {key: 1})**

```
sh.shardCollection(“BD2.documents", {_id: 1})
```

- Verify the sharding status

```
sh.status({verbose: true})
```

Sharding in MongoDB
Conclusion

- Start the config servers
- Initialize replication for the config servers
- Create shard replicas
- Initialize replication for each shard replica set
- Start mongos
- Add the replica shards

CRUD Operations in MongoDB

- Insert operation:
  - db.collection_name.insertOne(doc)
  - db.collection_name.insert(doc, {ordered: <boolean>}))
  - db.collection_name.insert([docs])
  - db.collection_name.insertMany([docs], {ordered: <boolean>})

CRUD Operations in MongoDB

- Insert Examples

```
> doc = { name: {firstname: "Ciprian", lastname: "Truica"}, age: 32, courses: [
{name: "BD1", year: 3}, {name: "BD2", year: 4}, {name: "DM", year: 5}],
offices: ["EF208", "PR305"]}

> db.teachers.insert(doc)

> doc = { name: {firstname: "Alexandru", lastname: "Boicea"}, age: 59,
courses: [{ name: "BD1", year: 3}, { name: "BD2", year: 4}, { name: "SCAD",
year: 4}, { name: "ICASE", year: 5}], offices: ["EF309"]}

> db.teachers.insertOne(doc)
```

CRUD Operations in MongoDB

- Insert Examples

```
> docs = [
        { name: {firstname: "Florin", lastname: "Radulescu"}, age: 58,
courses: [ {name: "BD1", year: 3}, {name: "BD2", year: 4}, , {name: "UBD",
year: 4}, {name: "DMDW", year: 5}], offices: ["Vice Dean Office", "EF309"]},
        { name: {firstname: "Sorin", lastname: "Ciolofan"}, age: 37, courses:
[{name: "PAJ", year: 5}, {name: "MN", year: 1}]},
        {name: {firstname: "Dan", lastname: "Schrager"}, courses: [ {name:
"PC", year: 1}, {name: "ABD", year: 5}], offices: ["PR304"]}
]

> db.teachers.insertMany(docs, {ordered: true})
```

CRUD Operations in MongoDB

- Query Documents:
  - db.collection.find(query, projection)
  - db.collection.findOne(query, projection)

CRUD Operations in MongoDB

- Find all:

```
> db.teachers.find()
```

- Find one:

```
> db.teachers.findOne()
```

- Find all under 40 years:

```
> db.teachers.find({age: {$lte: 40}}, {name: 1})
```

- Find all over 35:

```
> db.teachers.find({age: {$gte: 35}}, {name: 1})
```

- Find all with age between 40 and 60 years:

```
> db.teachers.find({age: {$gte: 40, $lte: 60}}, {name: 1})
```

CRUD Operations in MongoDB

- Find all who have the office EF309:

```
> db.teachers.find({offices: "EF309"}, {name: 1})
```

- Find all who do not have an office:

```
> db.teachers.find({offices: {$exists: false}}, {name: 1})
```

- Find all who teach BD1:

```
> db.teachers.find({"courses.name": "BD1"}, {name: 1})
```

- Find all teachers whose courses starting with DM (regex):

```
> db.teachers.find({"courses.name": /^DM/}, {name: 1, "courses.name": 1})
```

- Find all who have courses at the 1 year or his age is over 50:

```
> db.teachers.find({$or: [{"courses.year": 1}, {age: {$gt: 50}}]}, {name: 1})
```

CRUD Operations in MongoDB

- Find all who have courses at the 5 year and his age is under 50:

```
> db.teachers.find({$and: [{"courses.year": 5}, {age: {$lt: 50}}]}, {name: 1})
```

- Find all that have 2 offices:

```
db.teachers.find({offices: {$size: 2}}, {name: 1})
```

- Find all that has as his first office EF309:

```
db.teachers.find({"offices.0": "EF309"}, {name: 1})
```

- Find all that have offices in both EF and PR:

```
db.teachers.find({offices: {$all: [/^EF/, /^PR/]}}, {name: 1})
```

CRUD Operations in MongoDB

- Update:
  - db.collection.update(<filter>, <update>, <options>)
  - db.collection.updateOne(<filter>, <update>, <options>)
  - db.collection.updateMany(<filter>, <update>, <options>)
  - db.collection.replaceOne(<filter>, <replacement>, <options>)

CRUD Operations in MongoDB

- Update courses for teachers who are 32 years old:

```
> db.teachers.updateOne({age: 32}, {$set: {courses: {name: 'UBD', year: 4}}})
```

- Update multiple:

```
> db.teachers.update({age: 32}, {$set: {courses: {name: 'UBD', year: 4}}},
{multi: true})
```

- Update courses for all the teachers over 50 years:

```
> db.teachers.updateMany({age: {$gte: 50}}, {$set: {courses: {name: 'UBD',
year: 4}}})
```

```
> db.teachers.update({age: {$gte: 50}}, {$set: {courses: {name: 'UBD', year:
4}}}, {multi: true})
```

CRUD Operations in MongoDB

- Update a nested document, change the name of course BD1 to UBD for Ciprian

```
> db.teachers.update({"name.firstname": "Ciprian", "courses.name": "BD1"},
{$set: {"courses.$.name": 'UBD'}})

> db.teachers.update({"name.firstname": "Ciprian", "courses.name": "UBD"},
{$set: {"courses.$.name": 'BD1'}})

> db.teachers.find({"name.firstname": "Ciprian"}, {_id: 0, "courses.name": 1})
```

CRUD Operations in MongoDB

- Add a new course for Dan

```
> db.teachers.update({"name.firstname": "Dan"}, {$push: {courses: {name: 'BD1', year: 3}}})
```

```
> db.teachers.find({"name.firstname": "Dan"}, { courses: 1})
```

- Add middle name to Ciprian

```
> db.teachers.update({"name.firstname": "Ciprian"}, {$set: {"name.middlename": 'Octavian'}})
```

```
> db.teachers.find({"name.firstname": "Ciprian"}, { name: 1})
```

- Remove middlename from Ciprian

```
> db.teachers.update({"name.firstname": "Ciprian"}, {$unset: { "name.middlename": ""}})
```

CRUD Operations in MongoDB

- Delete:
  - db.collection.remove(<query>, <justOne>)
  - db.collection.deleteOne(<filter>)
  - db.collection.deleteMany(<filter>)
- Drop a collection
  - db.collection.drop()
- Drop a database
  - db.dropDatabase()

CRUD Operations in MongoDB

- Delete one

```
> db.teachers.remove({"name.firstname": "Ciprian"})

> db.teachers.deleteOne({"name.firstname": "Sorin"})
```

- Delete many

```
> db.teachers.deleteMany({age: {$gte: 50}})

> db.teachers.remove({age: {$gte: 50}}, {justOne: false})
```

- Remove everything

```
> db.teachers.remove({})

> db.teachers.deleteMany({})
```

CRUD Operations in MongoDB

- Drop collection

```
> db.teachers.drop()
```

- Drop database

```
> db.dropDatabase()
```

Indexes in MongoDB

- The _id field acts as a primary key and, when build, an index is automatically created
- Secondary indexes can be added on any field
- Secondary indexes can be:
  - Simple indexes on one field
  - Compound indexes on multiple fields
  - Multikey indexes for array
  - Geospatial indexes for geospatial coordinates
  - Text indexes
  - Hashed indexes to support hash based sharding

Indexes in MongoDB

- Simple indexes on one field:

```
> db.collection.createIndex({"label_1": 1})
```

- Compound indexes on multiple fields

```
> db.collection.createIndex({"label_1": 1, "label_2": -1, ...})
```

- Multikey indexes for array

```
> db.collection.createIndex({"label_1.nested_label_1": 1})
```

Indexes in MongoDB

- Geospatial indexes for geospatial coordinates

```
> db.collection.createIndex({"label": "2d"})
> db.collection.createIndex({"label": "2dsphere"})
```

- Text indexes (one per collection, but can be compound)

```
> db.collection.createIndex({"flabel": "text"})

> db.collection.createIndex({"label_1": "text", "label_2": "text",…})
```

- Hashed indexes to support hash based sharding

```
db.collection.createIndex({"label_1": "hashed"})
```

Indexes in MongoDB

- Simple index

```
> db.documents.createIndex({author: 1})
```

- Compound index

```
> db.documents.createIndex({gender: 1, age: 1})
```

- Multikey index

```
> db.documents.createIndex({"words.word": 1})
```

- Geospatial index

```
> db.documents.createIndex({"geoLocation": "2d"})
```

Indexes in MongoDB

- Text index

```
> db.documents.createIndex({"lemmaText": "text"})
```

- Hashed index

```
> db.documents.createIndex({"date": "hashed"})
```

- Get all indexes for a collection

```
> db.documents.getIndexes()
```

- Drop an Index

```
> db.documents.dropIndex("lemmaText_text")
```

- Drop all indexes except for primary key

```
> db.documents.dropIndexes()
```

Indexes in MongoDB

- Searching with Geospatial indexes
  - To find all the documents near a point (± 1)

```
> db.documents.find( { geoLocation: { $near: [25, 25], $minDistance: 0, $maxDistance: 5 } }, {geoLocation: 1} )
```

  - To find all the documents in the circle with the center [25, 10], and radius 3

```
> db.documents.find( { geoLocation: { $within: { $center: [[25, 10], 3] } } }, {geoLocation: 1} )
```

  - To find all the documents in the rectangle defined by [25, 40], [30, 45]

```
> db.documents.find( { geoLocation: { $within: { $box: [[25, 40], [30, 45]] } } }, { geoLocation: 1 } )
```

  - To find all the documents in the polygon defined by a polygon with 3 points

```
> db.documents.find( { geoLocation: { $within: { $polygon: [[20, 40], [40, 50], [30, 30]] } } },  { geoLocation: 1 } )
```

Indexes in MongoDB

- Searching with Text indexes

```
{
  $text:
    {
      $search: <string>,
      $language: <string>,
      $caseSensitive: <boolean>,
      $diacriticSensitive: <boolean>
    }
}
```

Indexes in MongoDB

- Searching with Text indexes

- Search for a word

```
> db.documents.find( { $text: { $search: "cup", $language: "english", $caseSensitive: false, $diacriticSensitive: false } } ,  { lemmaText: 1 } )
```

- Search for a multiple word (with an or between them)

```
> db.documents.find( { $text: { $search: "cup winner", $language: "english", $caseSensitive: false, $diacriticSensitive: false } } ,  { lemmaText: 1 } )
```

Indexes in MongoDB

- Searching with Text indexes

- Search for a Phrase

```
db.documents.find( { $text: { $search: "\"cup winner\"", $language: "english", $caseSensitive: false, $diacriticSensitive: false } } ,  { lemmaText: 1 } )
```

- Search with Text Search Score

```
db.documents.find( { $text: { $search: "cup winner", $language: "english", $caseSensitive: false, $diacriticSensitive: false } } ,  { score: { $meta: "textScore" }, lemmaText: 1, _id: 0})
```

Aggregation in MongoDB

- Aggregation in MongoDB can be done using:
  - Single Purpose Aggregation methods
  - Aggregation Pipeline
  - Map-Reduce

Single purpose aggregation methods

- Count: db.collection.count(query)

```
> db.documents.countDocuments({lemmaTextLength: 10})
```

- Distinct: db.collection.distinct(field, query)

```
> db.documents.distinct("age")

> db.documents.distinct("age", {gender: "female"}).sort()

> db.documents.distinct("age", {gender: "female"}).reverse()
```

Aggregation Pipeline (I)

- MongoDB's aggregation framework is modeled on the concept of data processing pipelines
- Documents enter a multi-stage pipeline that transforms the documents into an aggregated result
- The most basic pipeline stages provide filters that operate like queries and document transformations that modify the form of the output document
- The operation's order matters

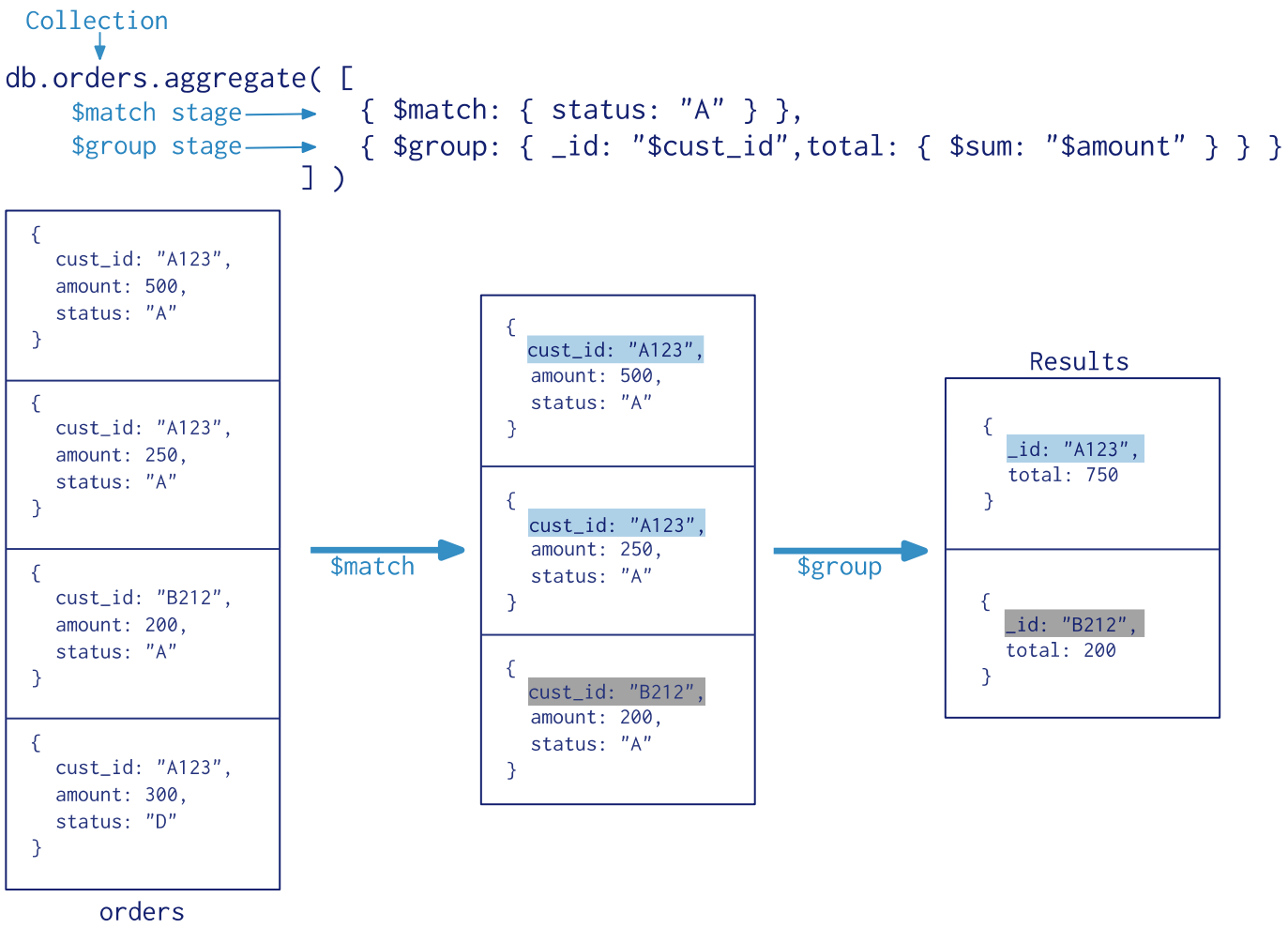
Aggregation Pipeline (I)

- Other pipeline operations provide tools for grouping and sorting documents by specific field or fields as well as tools for aggregating the contents of arrays, including arrays of documents
- In addition, pipeline stages can use operators for tasks such as calculating the average or concatenating a string
- The pipeline provides efficient data aggregation using native operations within MongoDB
- It is the preferred method for data aggregation in MongoDB
- The aggregation pipeline can operate on a sharded collection

Aggregation Pipeline

- The aggregation pipeline can use indexes to improve its performance during some of its stages
- Aggregation pipeline operations have an optimization phase which attempts to reshape the pipeline for improved performance
- The aggregation pipeline can determine if it requires only a subset of the fields in the documents to obtain the results
- If so, the pipeline will only use those required fields, reducing the amount of data passing through the pipeline

Aggregation Pipeline



Aggregation Pipeline

- Aggregation Pipeline (minimal) syntax – the operations' order matters

```
db.collection.aggregate(
   { $match: <query> },
   { $unwind: <array> },
   { $project: <projection>},
   { $group: <aggregation_group, aggregation_functions> },
   { $sort: <sorting_fields> },
   { $limit: <number>}
)
```

Aggregation Pipeline

- Extract Top-K Keywords

```
TFIDF_PA_TopKWords = function(q, k){
    var docs = Array();
    noDocs = db.documents.count(q);
    // aggregate function that computes the number of times the word appears in the corpus and the TF sum
    db.documents.aggregate([
      { $match: q },
      { $unwind: "$words" },
      { $project: { word: '$words.word', wtf: '$words.tf' } },
      { $group: { _id: "$word", sum_tf: { $sum: "$wtf" }, count: { $sum: 1 } } }
    ]).forEach(function(elem){
      var tfidf = elem.sum_tf * (1 + Math.log(noDocs/elem.count));
      var doc = {word: elem._id, TFIDF: tfidf};
      docs.push(doc);
    });
    db.vocabulary.insert(docs);
    var items = db.vocabulary.find().sort({TFIDF: -1}).limit(k);
    return items;
  }
```

Map Reduce

- MongoDB also provides map-reduce operations to perform aggregation (deprecated)

- In general, map-reduce operations have two phases:
  - A map stage that processes each document and emits one or more objects for each input document
  - A reduce phase that combines the output of the map operation

- Optionally, map-reduce can have a finalize stage to make final modifications to the result

- Like other aggregation operations, map-reduce can specify a query condition to select the input documents as well as sort and limit the results.

Map Reduce

- Map-reduce uses custom JavaScript functions to perform the
  - Map operation
  - Reduce operations
  - As well as the optional finalize operation

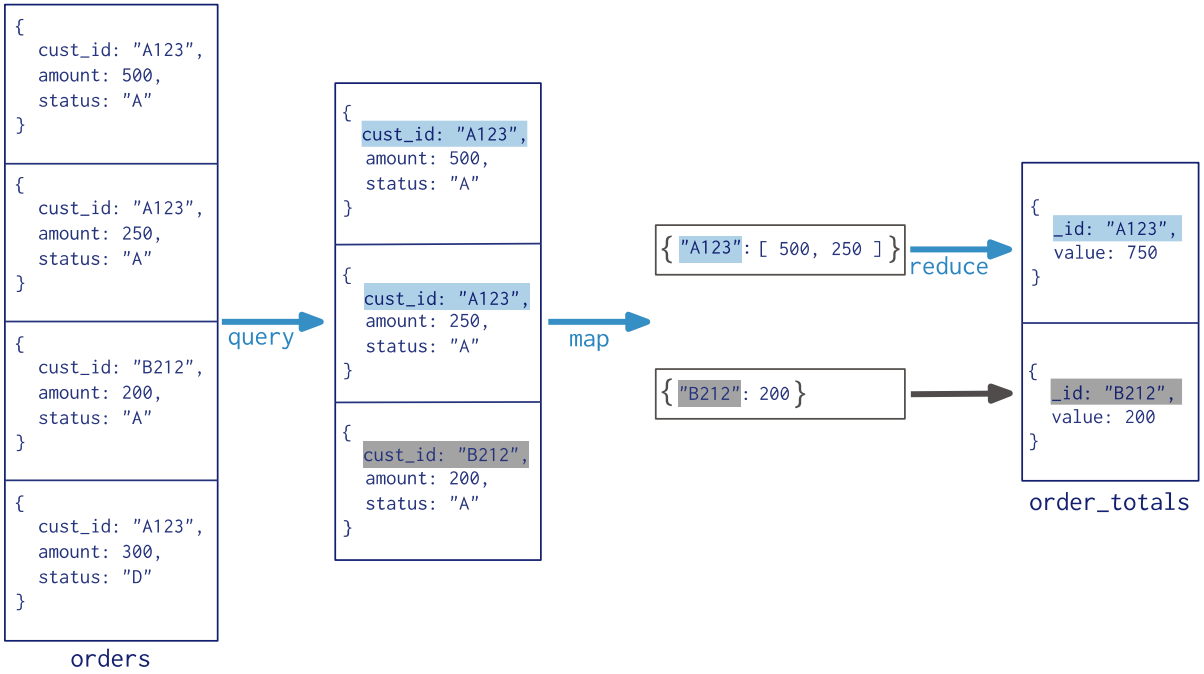
- While the custom JavaScript provide great flexibility compared to the aggregation pipeline, in general, map-reduce is less efficient and more complex than the aggregation pipeline

- Map-reduce can operate on a sharded collection

- Map-reduce operations can also output to a sharded collection

Map Reduce

```
Collection
    ↓
db.orders.mapReduce(
        map      →   function() { emit( this.cust_id, this.amount ); },
        reduce   →   function(key, values) { return Array.sum( values ) },
                     {
        query    →      query: { status: "A" },
        output   →      out: "order_totals"
                     }
                 )
```



Map Reduce

- Map Reduce (minimal) syntax

```
db.collection.mapReduce(
   <mapFunction>,
   <reduceFunction>,
   {
      query: <filter_query>,
      out: <output_collection>,
      scope: <external variable>,
      finalize: <finalize_function,
      sort: <document>,
      limit: <number>,
   }
);
```

Map Reduce

- Extract Top-K Keywords

```
TFIDF_MR_TopKWords = function(q, k){
    // compute the number of times the word apear in the corpus and the TF sum
    mapFunction = function() {
            for (var idx=0; idx<this.words.length; idx++){
              emit(this.words[idx].word, { count: 1, stf: this.words[idx].tf });
            } }
    reduceFunction = function(key, values) {
              reducedVal = { count: 0, stf: 0 };
              for (var idx = 0; idx < values.length; idx++) {
                reducedVal.count += values[idx].count;
                reducedVal.stf += values[idx].stf;
              }
              return reducedVal;};
    // The function  that computes the TFIDF
    finalizeFunction = function(key, reducedVal){
      reducedVal.tfidf = reducedVal.stf * (1 + Math.log(noDocs/reducedVal.count));
      return reducedVal; }
    var noDocs = db.documents.count(q);
    db.documents.mapReduce(mapFunction, reduceFunction, {query: q, out: "vocabulary", scope: {noDocs: noDocs}, finalize:
finalizeFunction});
    // print the values
    var items = db.vocabulary.find().sort({"value.tfidf": -1}).limit(k).addOption(DBQuery.Option.noTimeout);
    return items; }
```

Bibliography

- MongoDB Documentation: https://www.mongodb.com/docs/

University
Politehnica
of Bucharest

Column-Oriented Databases
Cassandra

Ciprian-Octavian Truică
ciprian.truica@upb.ro

Overview

- Column-Oriented Databases
- Cassandra

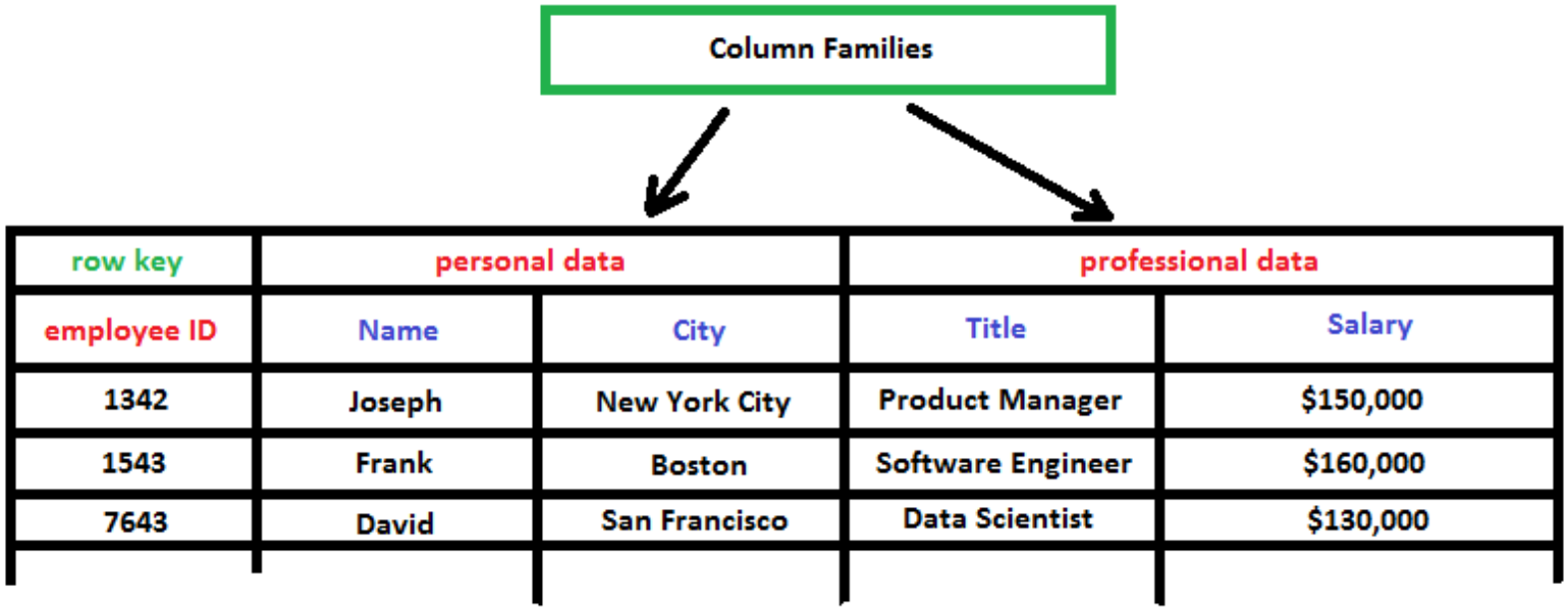
Overview

- Column-Oriented Databases
- Cassandra

Column Stores

- Column-Oriented Databases Management Systems (CODBMS) stores data using tables, rows, and columns.
- Unlike a relational database, the names and format of the columns can vary from row to row in the same table.
- Columns are grouped in Column Families.
- CODBMSs are also called Wide Column stores or Columnar Database.

Column Stores



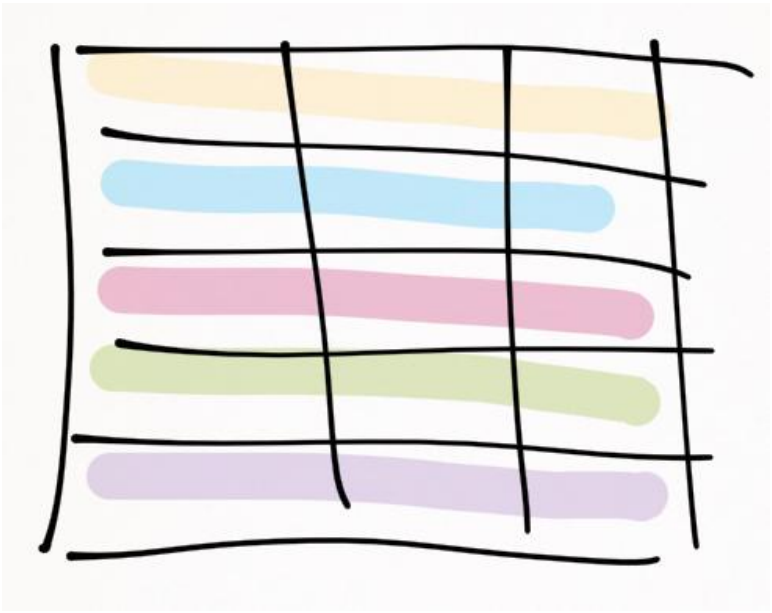
row key	personal data		professional data	
employee ID	Name	City	Title	Salary
1342	Joseph	New York City	Product Manager	$150,000
1543	Frank	Boston	Software Engineer	$160,000
7643	David	San Francisco	Data Scientist	$130,000

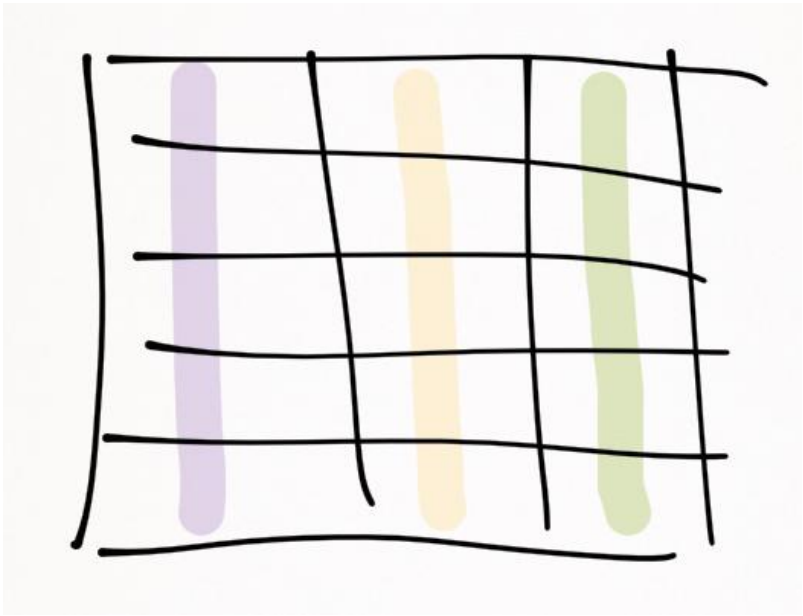
Column Stores

- CODBMS Databases uses one of two approaches to partitioning data on disk:
  - Row-Oriented: partition tables by rows
  - Column-Oriented: partition tables by columns/column families

Column-Oriented Databases

- Partitioning: Row-Oriented vs. Column-Oriented





Column Stores

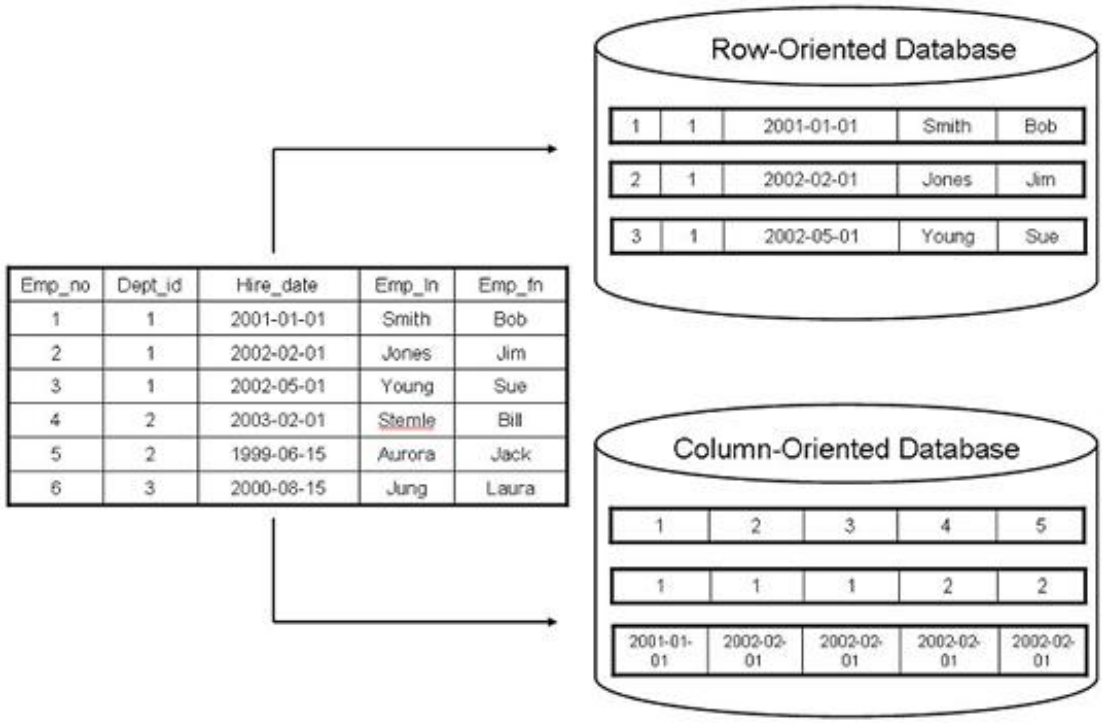
- CODBMS disk storage approach: Row-Oriented partitioning
  - Predictability:
    - Tuples address can be predicted without looking at the data
  - Locality:
    - Reduce the probability to read the data from memory cache or the same block on disk
  - Read operations:
    - To read one attribute the entire tuple must be read

Column Stores

- CODBMS disk storage approach: Column-Oriented partitioning
  - Predictability:
    - We can predict exactly where we find the data if the data uses fixed length types
    - We cannot predict exactly where we find the data in the case of tuples with variable length
  - Locality:
    - The probability to hit the cache (or block) is very high during iteration
  - Read operations:
    - Read only the data needed

Column Stores

- Partitioning: Row-Oriented vs. Column-Oriented



Emp_no	Dept_id	Hire_date	Emp_ln	Emp_fn
1	1	2001-01-01	Smith	Bob
2	1	2002-02-01	Jones	Jim
3	1	2002-05-01	Young	Sue
4	2	2003-02-01	Stemle	Bill
5	2	1999-06-15	Aurora	Jack
6	3	2000-08-15	Jung	Laura

1	1	2001-01-01	Smith	Bob
2	1	2002-02-01	Jones	Jim
3	1	2002-05-01	Young	Sue

1	2	3	4	5
1	1	1	2	2
2001-01-01	2002-02-01	2002-02-01	2002-02-01	2002-02-01

Overview

- Column-Oriented Databases
- Cassandra
  - Overview
  - Data Model

Cassandra

- Cassandra
  - First developed by Facebook to solve the inbox search problem
  - The team wanted to address the following problems:
    - Handle huge volumes of data in the form of message copies
    - Reverse indices of messages
    - Many random reads
    - Many simultaneous random writes

Cassandra

- Cassandra
  - Is open source
  - Is Distributed and Decentralized
  - Is Elastically Scalable
  - Is Highly Available and Fault-Tolerant
  - Uses Tunable Consistency
  - High Performance
  - Row-Oriented Partitioning

Cassandra

- Cassandra is distributed
  - Cassandra can run on multiple machines while appearing to users as a unified whole
  - Cassandra design and code base is specifically engineered to:
    - Work across many different machines
    - Optimize performance across multiple data center racks
    - Run a single Cassandra clusters across geographically dispersed data centers

Cassandra

- Cassandra is decentralized
  - There is no single point of failure
  - All the nodes in a Cassandra cluster function exactly the same
    - This is sometimes referred to as “server symmetry”
  - Because they are all doing the same thing there can’t be a special host that is coordinating activities, as with the master/slave setup

Cassandra

- Cassandra is Elastic Scalable
  - Elastic scalability refers to a special property of horizontal scalability
  - It means that your cluster can seamlessly
    - Scale up
    - Scale (back) down
  - The cluster must be able:
    - To seamlessly accept new nodes
    - New nodes can begin participating by getting a copy of some or all the data
    - New nodes start serving new user requests
      - Without major disruption
      - Reconfiguration of the entire cluster.

Cassandra

- With Elastic Scalability, there is no need to:
  - Restart any process
  - Change any application queries
  - Manually rebalance the data

- With Elastic Scalability, Cassandra will automatically find, start and send work to any new machine added to the cluster

Cassandra

- Cassandra is Highly Available and Fault Tolerance
  - Failed nodes can be replaced in the cluster with no downtime
  - Data can be replicated to multiple data centers to offer improved local performance
  - Replication helps to prevent downtime if one data center experiences a catastrophe

Cassandra

- Cassandra uses Tunable Consistency
  - Cassandra allows the administrator to choose the level of consistency
  - Cassandra's consistency also uses the replication factor
  - The replication factor is a trade-off between how much you want to pay in performance to gain more consistency
  - The replication factor value is equal to the number of nodes in the cluster where a modification (insert, update, delete) will propagate to

Cassandra

- Cassandra consistency level
  - Is a setting that clients must specify on every operation
  - The consistency level allows clients to decide how many replicas in the cluster
    - Must acknowledge a write operation in order to be considered consistent and durable
    - Respond to a read operation in order to be considered successful
  - Can be equal to the replication factor:
    - The gain is stronger consistency
    - The cost is synchronous blocking operations that wait for all nodes to be updated and declare success before returning

Cassandra

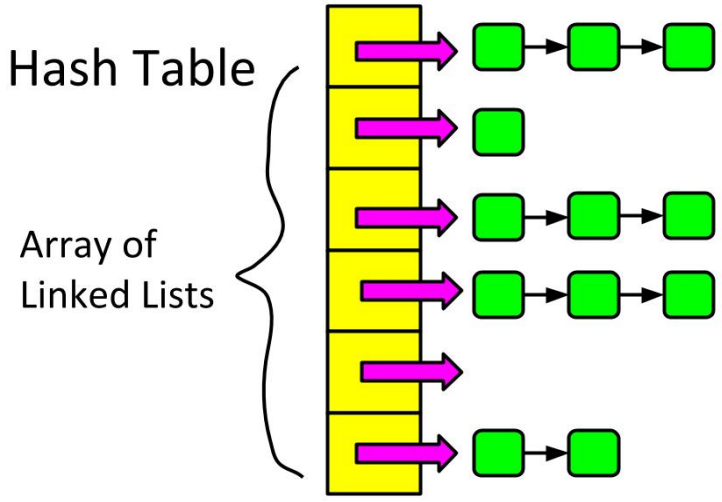
- Cassandra is a High-Performance database
  - Cassandra was designed specifically from the ground up to take full advantage of multiprocessor/multi-core machines
  - It can run across many dozens of these machines housed in multiple data centers
  - It scales consistently and seamlessly to hundreds of terabytes
  - Cassandra has been shown to perform exceptionally well under heavy load

Cassandra

- Cassandra is a High-Performance database
  - It consistently can show very fast throughput for writes per second on basic commodity computers
  - The commodity computers can run on
    - Physical hardware
    - Virtual machines
    - Containers
  - All of Cassandra's desirable properties can be maintained without sacrificing performance as more servers are added

Cassandra

- Cassandra's Row-Oriented partitioning
  - Cassandra's data model can be described as a partitioned row store
  - Data is stored in sparse multidimensional hashtables.



Cassandra

- Cassandra's Row-Oriented partitioning
  - Sparse means that
    - For any given row you can have one or more columns
    - Each row does not need to have all the same columns as other rows like it (as in a relational model)
  - Partitioned means that
    - Each row has a unique key which makes its data accessible
    - The keys are used to distribute the rows across multiple data stores.

Cassandra

row key	columns...			
123	1339707619:"value"	1339817619:"value"	1369707619:"value"	
	1.1	1.11	0.87	
456	1339707619:"value"	1339817619:"value"	1369707619:"value"	1369707888:"value"
	40	111	144	222
789	1339707619:"value"			
	21.345			

Cassandra

- Cassandra is a good fit for
  - Large Deployments
  - Lots of Writes, Statistics, and Analytics
  - Geographical Distribution
  - Evolving Applications

Cassandra

- Large Deployments:
  - If the application is expected to require dozens of nodes
  - For applications that needs high availability
  - For applications that are geographical distributed

Cassandra

- Lots of Writes, Statistics, and Analytics
  - Before choosing a database, consider your application from the perspective of the ratio of reads to writes
  - Cassandra is optimized for excellent throughput on writes
  - One of Cassandra's primary feature is the ability to handle application workloads that require high performance at significant write volumes with many concurrent client threads

Cassandra

- Lots of Writes, Statistics, and Analytics
  - Cassandra is a good choice for storing
    - User activity updates
    - Social network usage
    - Recommendations
    - Reviews
    - Application statistics
  - Cassandra is also a good choice when the application involve lots of writing with less predictable read operations
  - When updates can occur unevenly with sudden spikes

Cassandra

- Lots of Writes, Statistics, and Analytics
  - Cassandra has been used to create a variety of applications:
    - Windowed time-series store
    - An inverted index for document searching
    - A distributed job priority queue

Cassandra

- Geographical Distribution
  - Cassandra has out-of-the-box support for geographical distribution of data
  - Cassandra can be easily configured to replicate data across multiple data centers
  - Cassandra could be a great fit if you have a globally deployed application that could see a performance benefit from putting the data near the user

Cassandra

- Evolving Applications
  - Cassandra might be a good fit if the application is evolving rapidly given its support for flexible schemas
  - Cassandra makes it easy to keep your database in step with application changes as you rapidly deploy

Overview

- Column-Oriented Databases
- Cassandra
  - Overview
  - Data Model

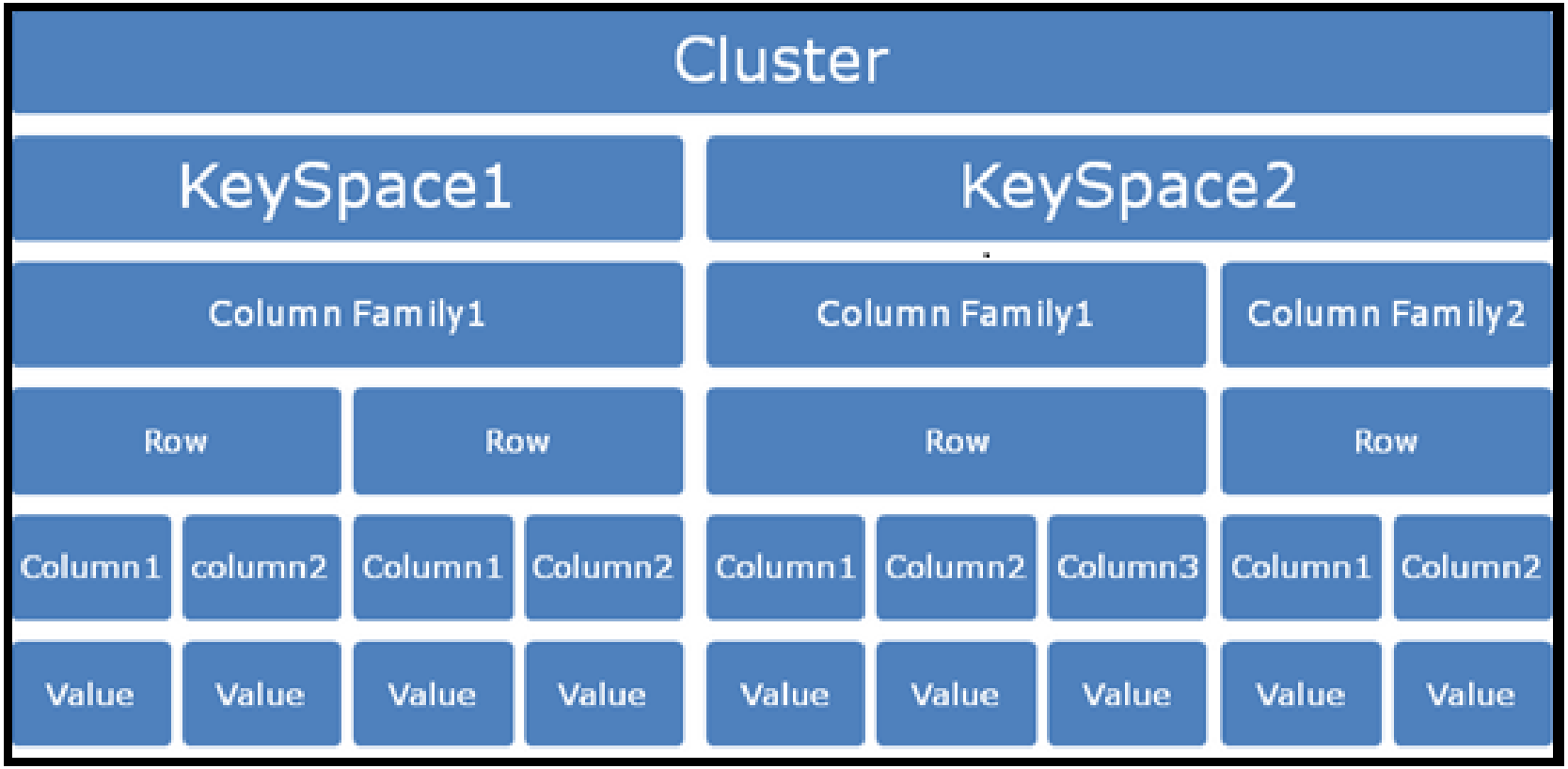
Cassandra – Data Model

- Cassandra data model contains the following structures:
  - Cluster
  - Keyspace
  - Table

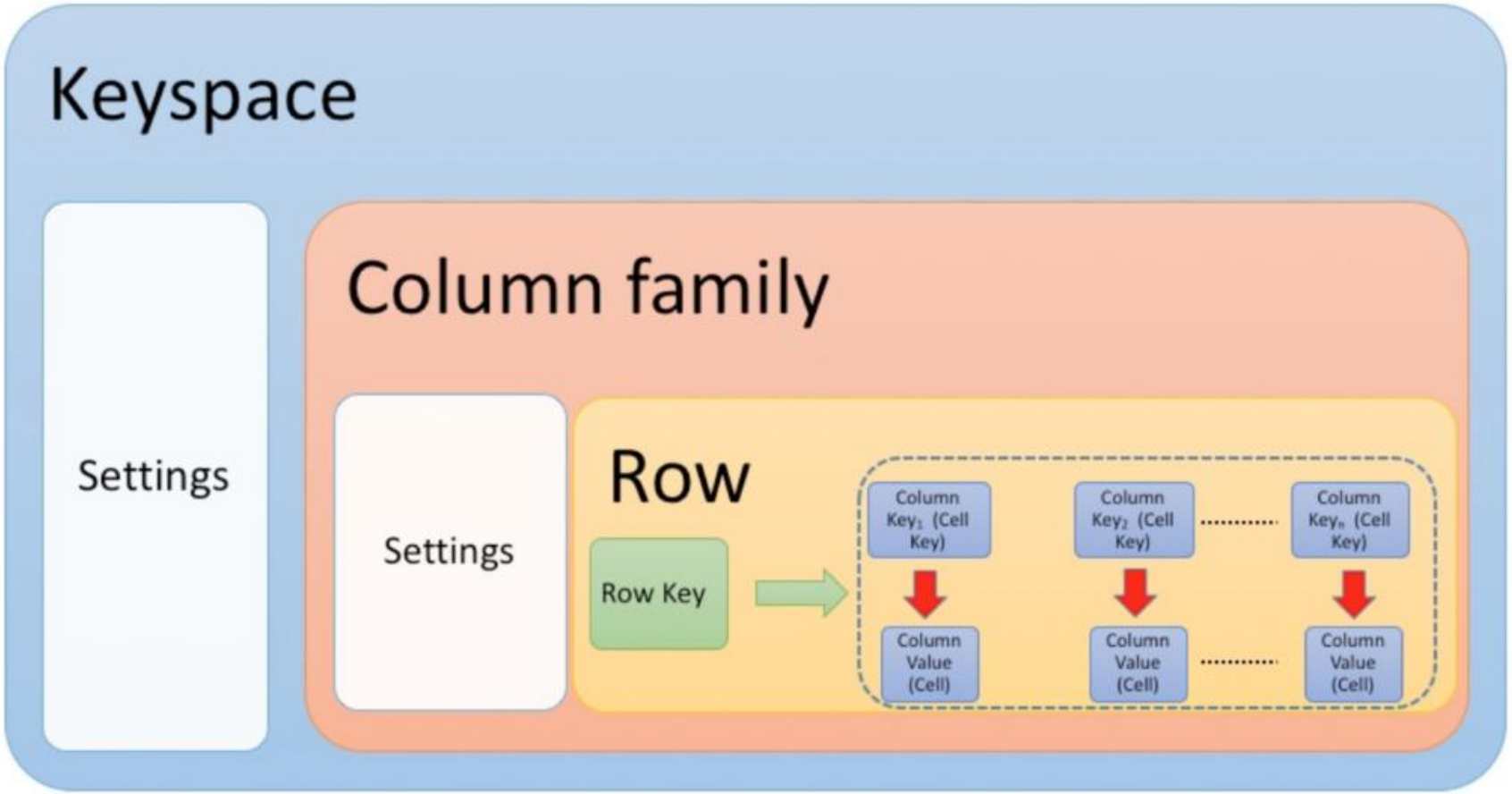
Cassandra – Data Model

- An instance of Cassandra typically consists of only one table
- A table represents a distributed multidimensional map indexed by a key.
- A table is structured by the following dimensions (bottom-up view)
  - Rows
  - Row Key (Partition Key)
  - Column
  - Super Column
  - Column Family
  - Super Column Family

Cassandra – Data Model



Cassandra – Data Model



Cassandra – Data Model

- Rows
  - A row is the smallest unit that stores related data in Cassandra
  - Rows are identified by a string-key of arbitrary length
  - Operations on rows are atomic per replica no matter how many columns are being read or written

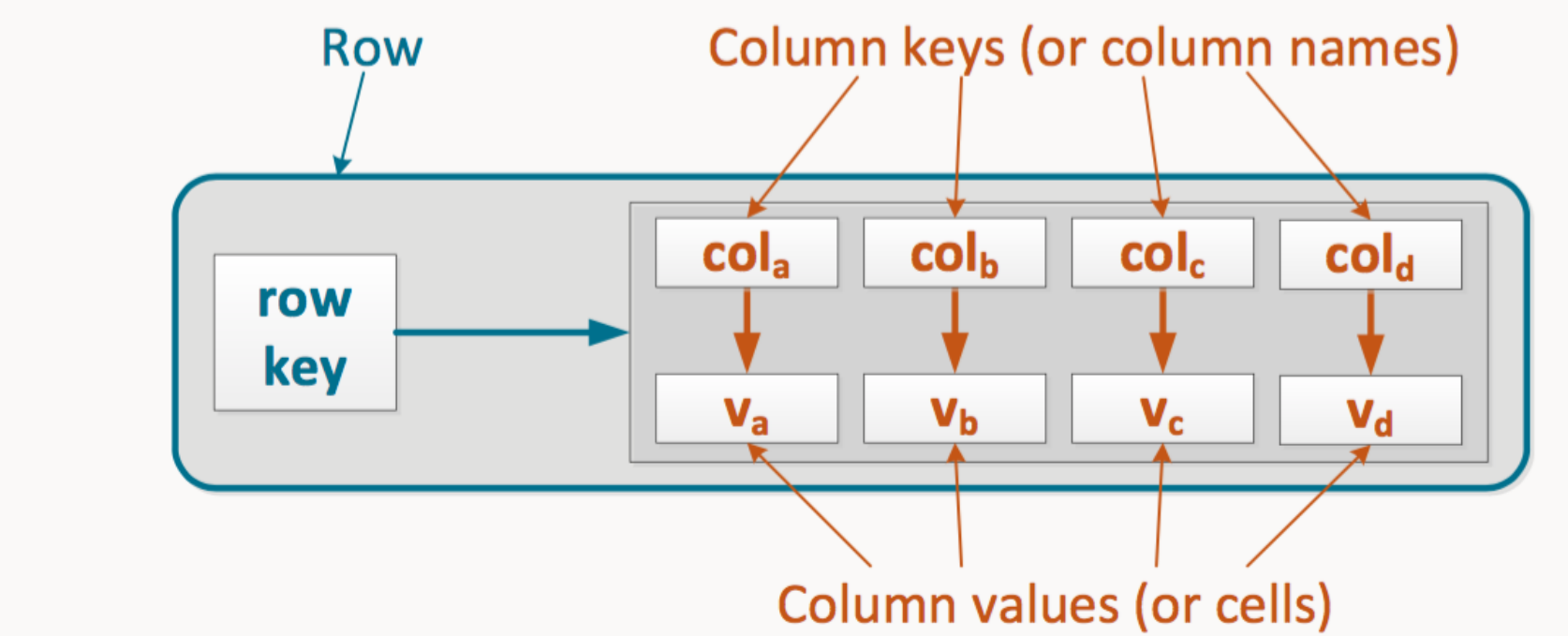
Cassandra – Data Model

- Rows
  - A row key uniquely identifies a row in a column family
  - A row stores pairs of column keys and column values
  - A column key uniquely identifies a column value in a row
  - A column value stores one value or a collection of values

Cassandra – Data Model

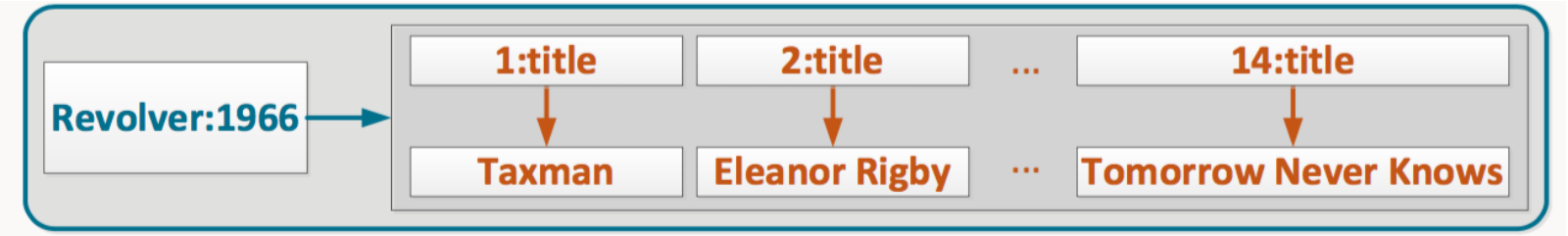
- Rows may be described as skinny or wide
  - Skinny row:
    - Has a fixed number of column keys
    - Relatively small number of column keys.
  - Wide row:
    - Has a relatively large number of column keys (hundreds or thousands)
    - The number of column keys may increase as new data values are inserted

Cassandra – Data Model



Cassandra – Data Model

- Keys
  - Composite column key:
    - Multiple components can form a column key
    - The components are separated by colon



Cassandra – Data Model

- Column
  - Stores a values
  - Is identified by a timestamp and a name
  - The name acts as the column key
  - Each row in a table can have a different number of columns:
    - A table can be thought of as a rectangle.

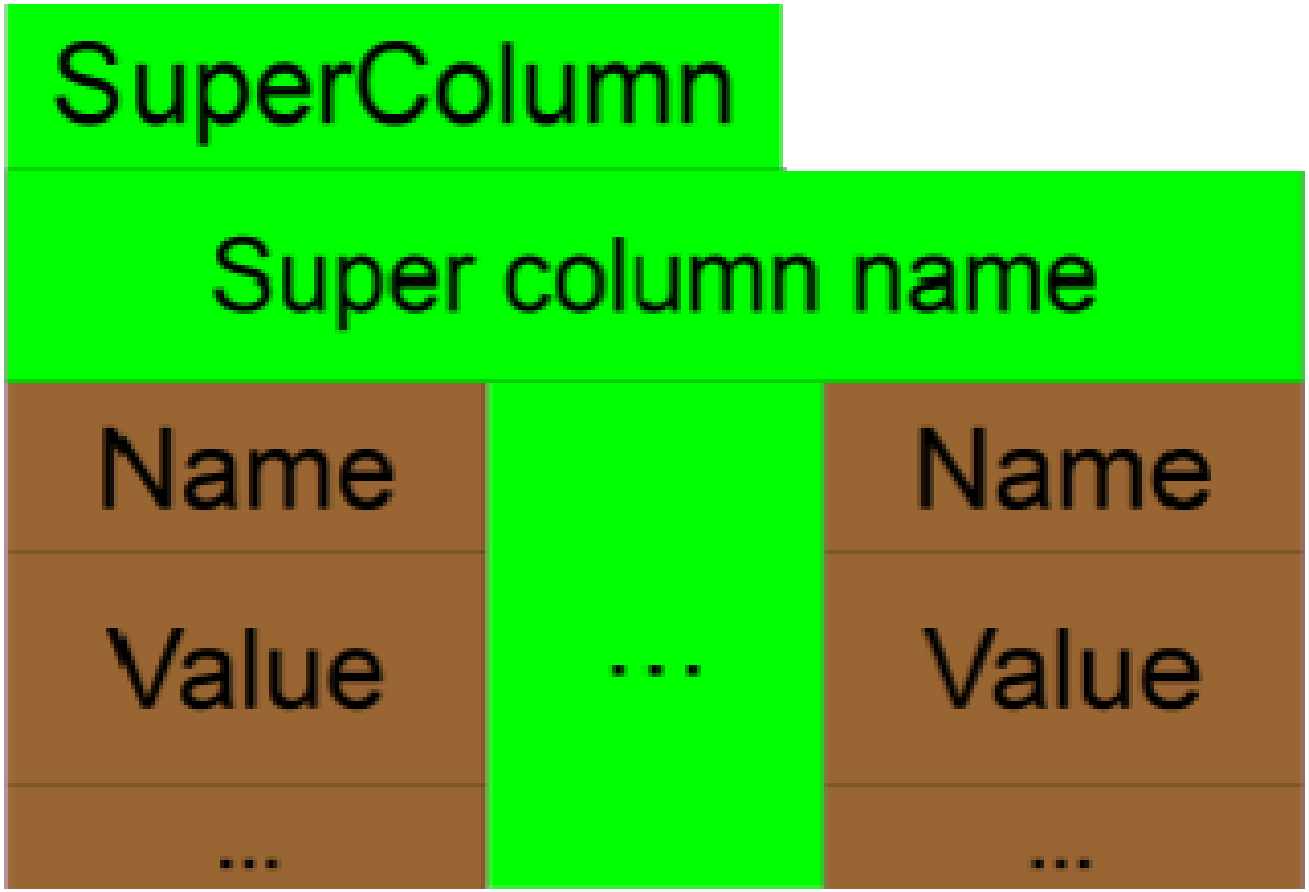
Cassandra – Data Model

- Columns
  - Client applications may specify the ordering of columns within
    - Super column
    - Column Family
  - The ordering can either be by name or by timestamp

Cassandra – Data Model

- Super columns
  - Are column containers sorted by their names
  - Are tuples (pairs) that consist of a key-value pair
    - The key is mapped to a value that is a set of columns
    - Can have an arbitrary number of columns associated with them
  - The number of columns per super column may differ per row
  - Must have a name
  - The name acts as a unique identifier
  - Can be referenced and sorted by their row key

Cassandra – Data Model



Cassandra – Data Model

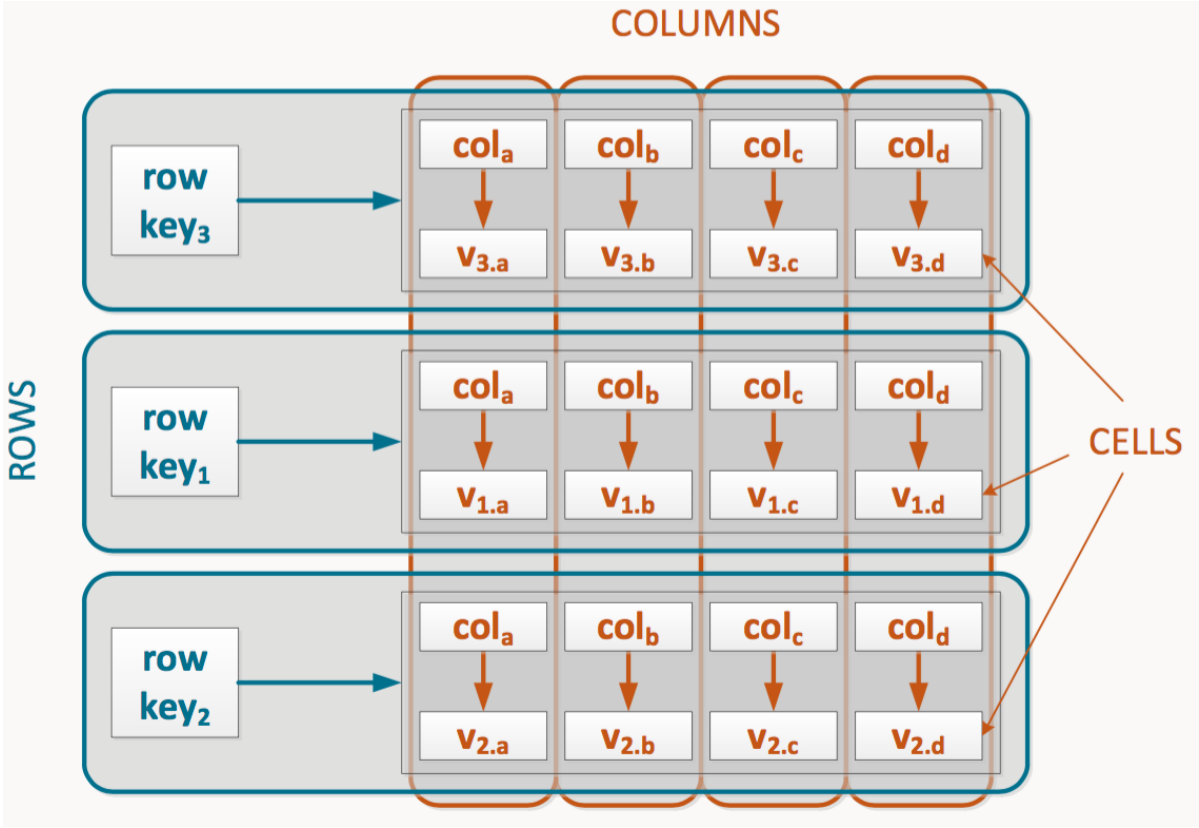
- Column families
  - Are a set of rows with a similar structure
  - Can occur in an arbitrary number per row
  - The number of column families per table is not limited
  - It is expected that only a few of them are specified at the table creation time:
    - These must be defined in advance before a cluster of servers comprising a Cassandra instance is launched

Cassandra – Data Model

- Column families
  - Consists of columns which can be added dynamically (i.e., at runtime) to column families and are not restricted in number.
  - In analogy with relational databases
    - A standard column family is as a table
    - Each key-value pair being a row

Cassandra – Data Model

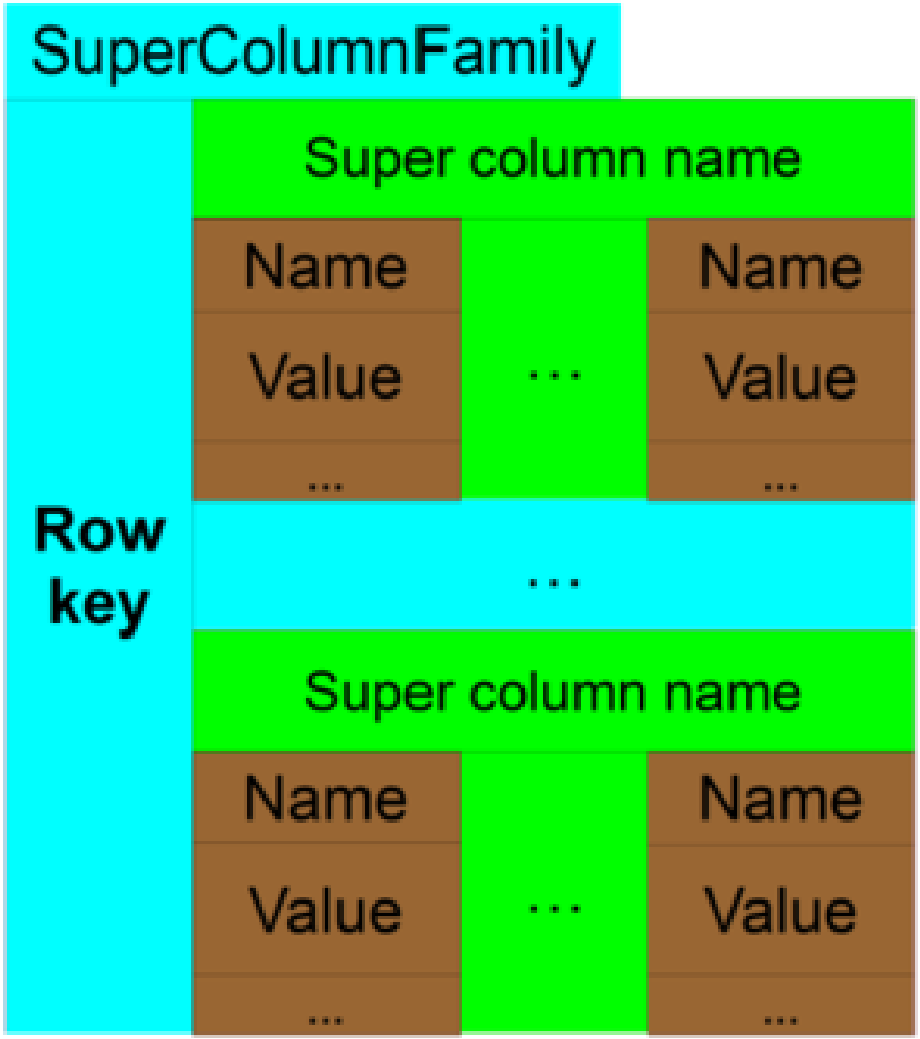
- Column Family



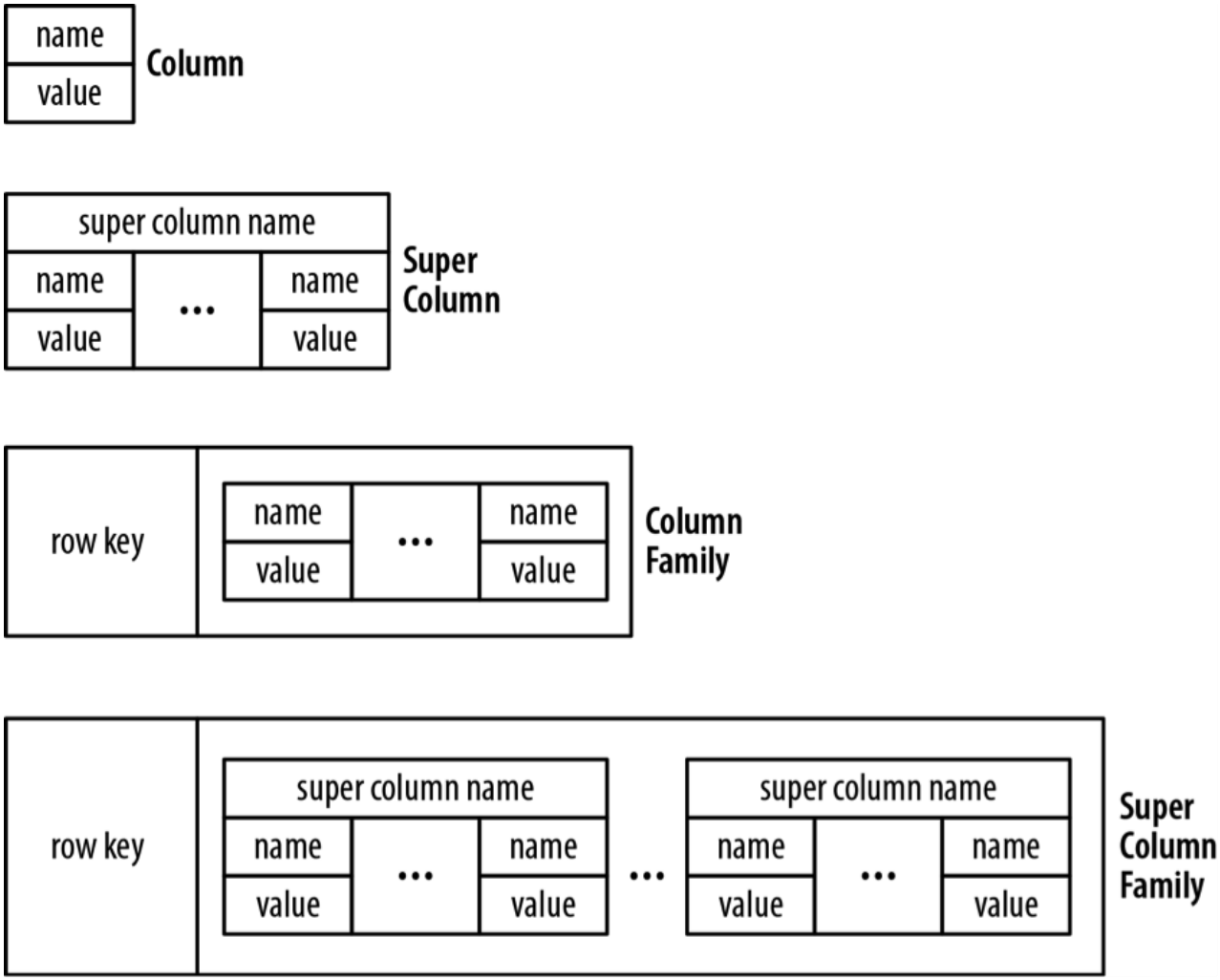
Cassandra – Data Model

- Super column families
  - When rows contain super columns, then the collection is called a super column family
  - It is a tuple (pair) that consists of a key-value pair
  - The key is mapped to a value that are column families
  - In analogy with relational databases, a super column family is something like a view on several tables
  - It can also be a map of tables

Cassandra – Data Model



Cassandra – Data Model



Cassandra – Data Model

- Clusters
  - Cassandra database is specifically designed to be distributed over several machines operating together
  - These machines appear as a single instance to the end user
  - The outermost structure in Cassandra is the cluster
  - The cluster is built on these machines
  - The cluster is sometimes called the ring
    - Cassandra assigns data to nodes in the cluster by arranging them in a ring.

Cassandra – Data Model

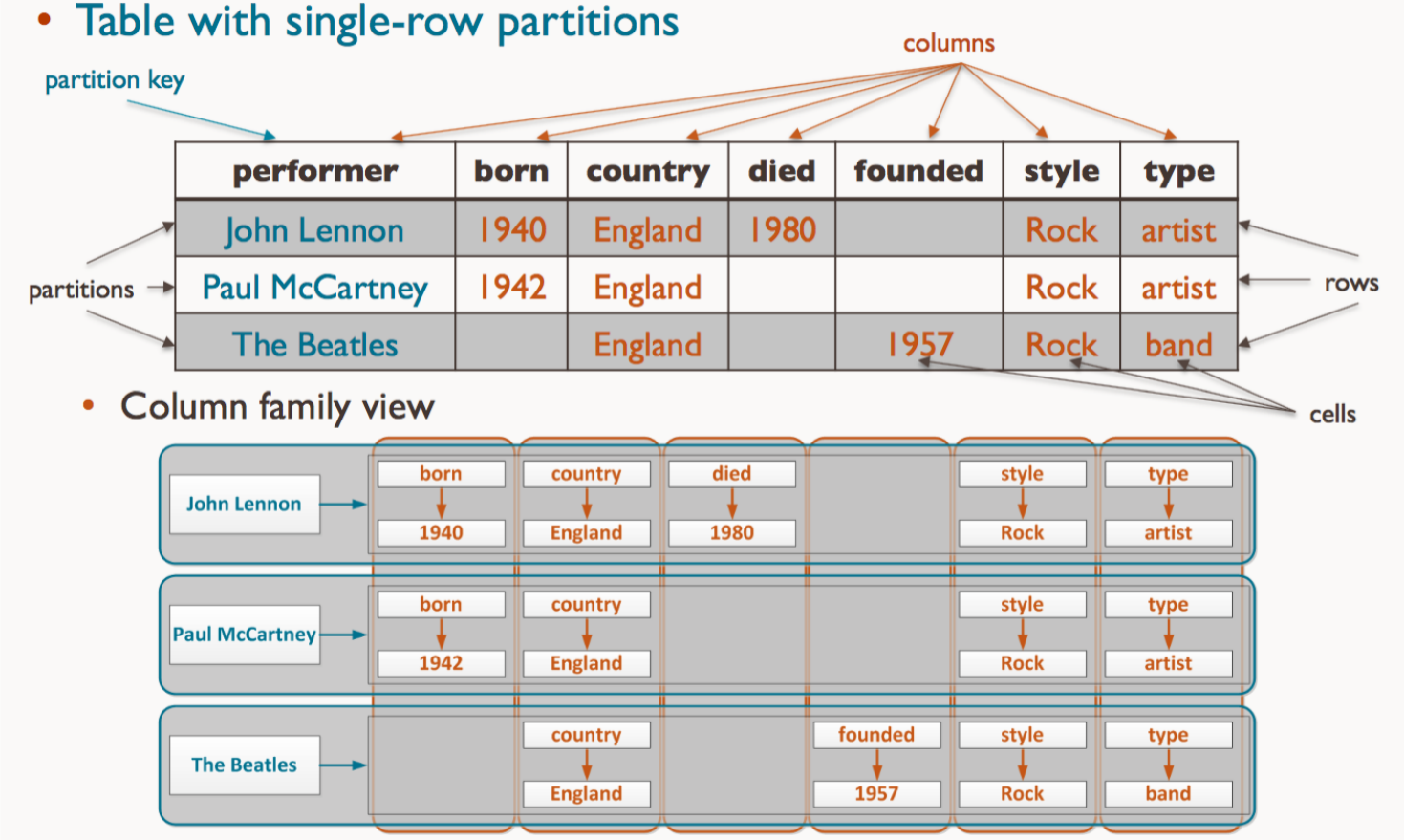
- Keyspaces
  - A cluster is a container for keyspaces.
  - A keyspace is the outermost container for data in Cassandra
  - In analogy with relational databases:
    - The keyspace corresponds closely to a relational database (schema)
    - In the same way that a database (schema) is a container for tables in the relational model, a keyspace is a container for tables in the Cassandra data model.
  - A keyspace has a name and a set of attributes that define keyspace-wide behavior.

Cassandra – Data Model

- Tables
  - A table is a container for an ordered collection of rows (records)
  - Each row (record) is itself an ordered collection of columns values
  - The ordering is determined by the columns
    - When you write data to a table in Cassandra, you specify values for one or more columns.
    - The column names are identified as keys
    - At least one of the values you specify must be a row key that serves as the unique identifier for that record.

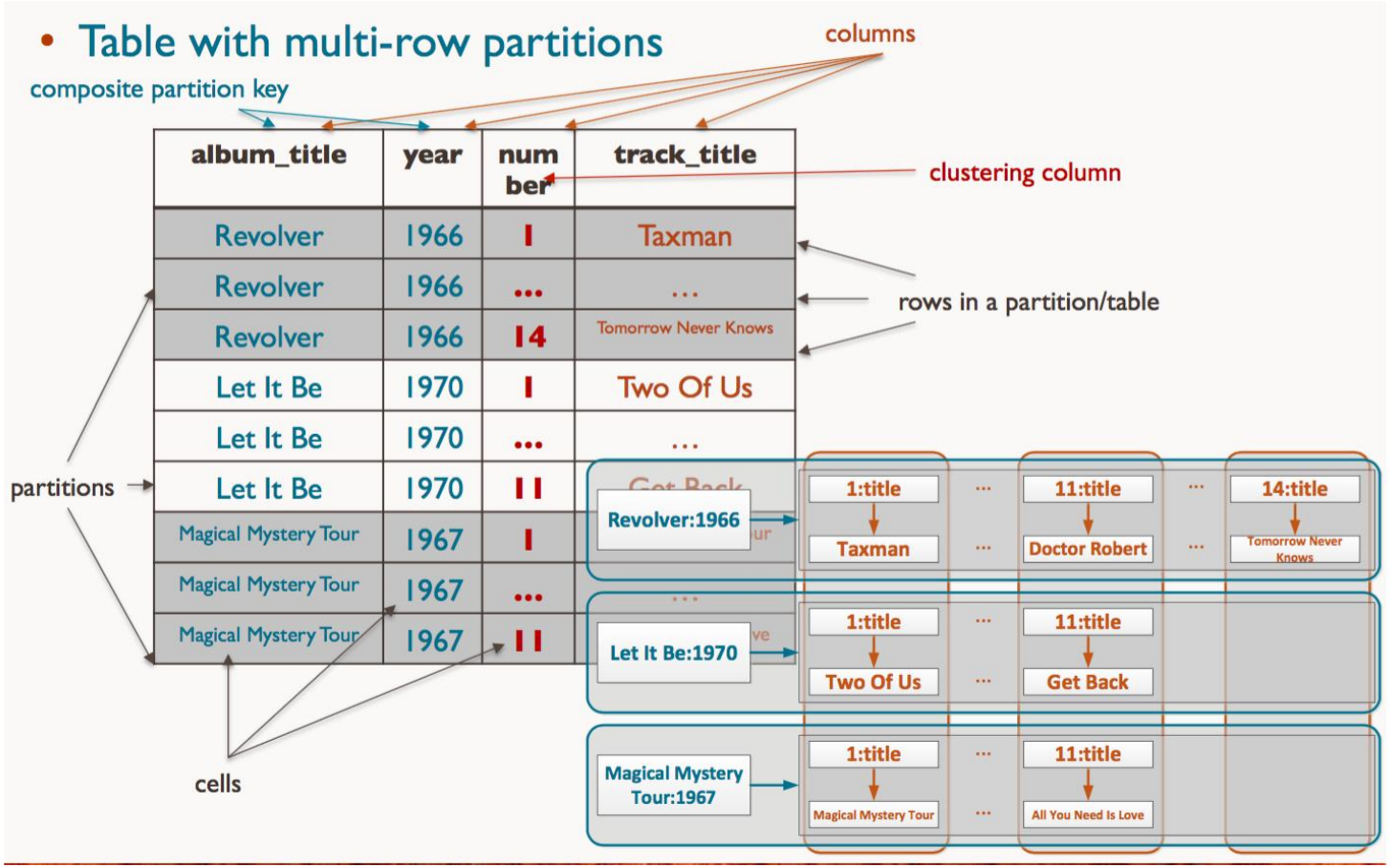
Cassandra – Data Model

- Table with single-row partitions



Cassandra – Data Model

- Table with multi-row partitions



Bibliography

- Apache Cassandra Documentation https://cassandra.apache.org/doc/latest/

- Jeff Carpenter, Eben Hewitt. Cassandra: The Definitive Guide: Distributed Data at Web Scale. O'Reilly Media, Inc., 2016.

- Christof Strauch. "NoSQL databases." *Lecture Notes, Stuttgart Media University* 20.24 (2011): 79.

- Eric Redmond, Jim Wilson. Seven databases in seven weeks: a guide to modern databases and the NoSQL movement. 1nd Edition 2012

- Luc Perkins, Jim Wilson, and Eric Redmond. Seven databases in seven weeks: a guide to modern databases and the NoSQL movement. 2nd Edition 2018

University
Politehnica
of Bucharest

Cassandra (II)

Ciprian-Octavian Truică
ciprian.truica@upb.ro

Overview

- Cassandra Architecture
  - Data Centers and Racks
  - Gossip and Failure Detection
  - Rings and Tokens
  - Vnodes (virtual nodes)
  - Partitioners
  - Replication Strategies
  - Consistency Levels
  - Queries and Coordinator Nodes
  - Commit Logs, Memtables, and SSTables
  - Caching
  - Hinted Handoff

Overview

- Cassandra Architecture
  - Data Centers and Racks
  - Gossip and Failure Detection
  - Rings and Tokens
  - Vnodes (virtual nodes)
  - Partitioners
  - Replication Strategies
  - Consistency Levels
  - Queries and Coordinator Nodes
  - Commit Logs, Memtables, and SSTables
  - Caching
  - Hinted Handoff

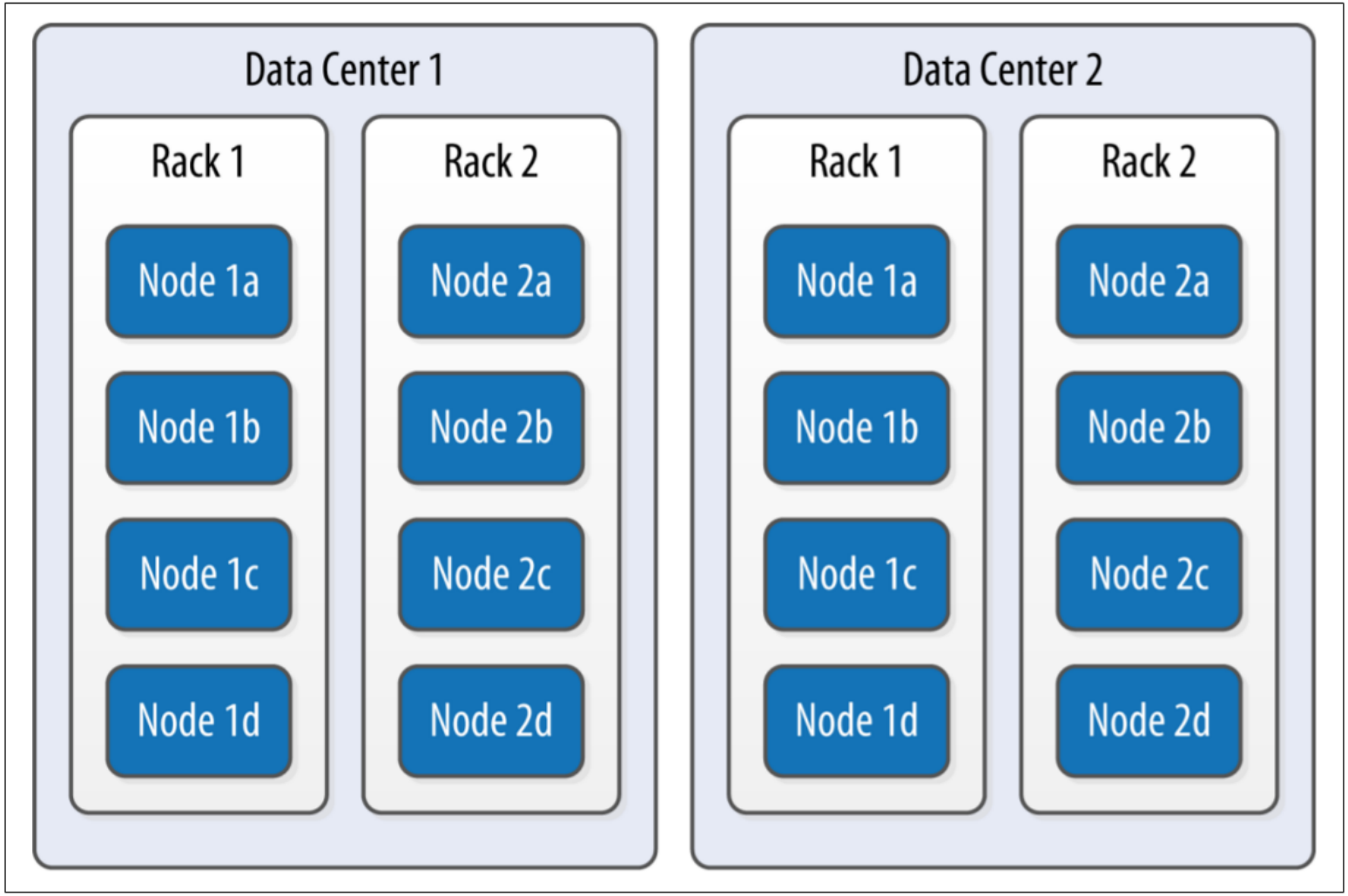
Data Centers and Racks

- Cassandra is frequently used in systems spanning physically separate locations
- Cassandra provides two levels of grouping that are used to describe the topology of a cluster:
  - Data center
  - Rack

Data Centers and Racks

- A rack
  - Is a logical set of nodes in close proximity to each other
  - Perhaps on physical machines in a single rack of equipment
- A data center
  - Is a logical set of racks
  - Is located
    - In the same building
    - At different locations
  - The racks are connected by reliable network

Data Centers and Racks



Data Centers and Racks

- Cassandra leverages the information you provide about your cluster's topology to determine
  - Where to store data
  - How to route queries efficiently
- Cassandra tries to store copies of your data in multiple data centers
  - To maximize availability
  - For partition tolerance
- Cassandra prefers to route queries to nodes in the local data center to maximize performance

Data Centers and Racks

- There are several key use cases for deploying Cassandra across multiple data centers:
  - Live backup
  - Failover
  - Load balancing
  - Geographic distribution
  - Online analysis

Data Centers and Racks
- Live backups -

- Traditional database backups involve:
  - Taking periodic snapshots of the data
  - Storing them offsite in case the system fails
- In the case of failure there will be downtime as a new system is brought up and the data is restored
- This strategy also inevitably leads to data loss for the time period between the last backup and the point of failure
- Snapshot backups are still useful to protect against:
  - Data corruption
  - Accidental updates

Data Centers and Racks
- Live backups -

- Cassandra's data center support can be used to provide a current backup for cases such as hardware failures
- The basic idea involves setting up a second data center that maintains a current set of replicas
- These replicas can be used to rebuild the primary cluster, should a catastrophic event cause the loss of an entire data center
- It is typically enough to maintain a smaller cluster with a replication factor of 1
- This system will never be used to accept live reads or writes
- The primary consideration in this case is storage capacity to handle the same quantity of data as the live data center

Data Centers and Racks
- Failover -

- The failover scenario is very similar to the backup use case
- In this case, the backup data center is generally allocated similar resources as the primary cluster:
  - For a simple backup center, a single replica may be enough
  - For a failover data center, all nodes should be configured with the same replication factor as the primary
- With this strategy, any node can take over responsibility for the full application load in the event of a failure

Data Centers and Racks
- Failover -

- When you build a failover data center, consider whether you expect your failover data center to handle a full production load
- In this case, you need to ensure that the cluster has adequate capacity to handle the entire production load
- Consider that having a hot failover data center protects you from a common single point of failure: the power supply to your hosts

Data Centers and Racks
- Failover -

- Failover considerations:
  - It would be ill-advised to assume you can maintain a small failover data center, and then simply add multiple nodes in the case of failure
  - The additional overhead of bootstrapping the new nodes would reduce the capacity at a critical point when the capacity is needed most

- Bootstrapping refers to the process of loading the basic software into the memory of a computer after power-on or general reset, especially the operating system which will then take care of loading other software as needed.

Data Centers and Racks
- Load balancing -

- Applications may be configured to route traffic to any node in the cluster without considering a specific data center
- This has the effect of load balancing the requests across multiple data centers
- Can be useful in cases where the data centers share a high-bandwidth connection
- The objective is to provide redundancy:
  - Each data center must be able to handle the entire application load
  - Similar to the failover scenario

Data Centers and Racks
- Load balancing -

- Load balancing considerations:
  - Absolute consistency is expensive to guarantee for load balancing:
    - Consistency requires replicating the data across the entire cluster
    - For strong consistency, employing a geographic distribution model is a good strategy
  - Load balancing is appropriate for use cases where eventual consistency is acceptable:
    - The primary read case involves offline data analysis rather than real-time queries
    - Application examples: event capture, time-series data, logging, etc.

Data Centers and Racks
- Geographic distribution -

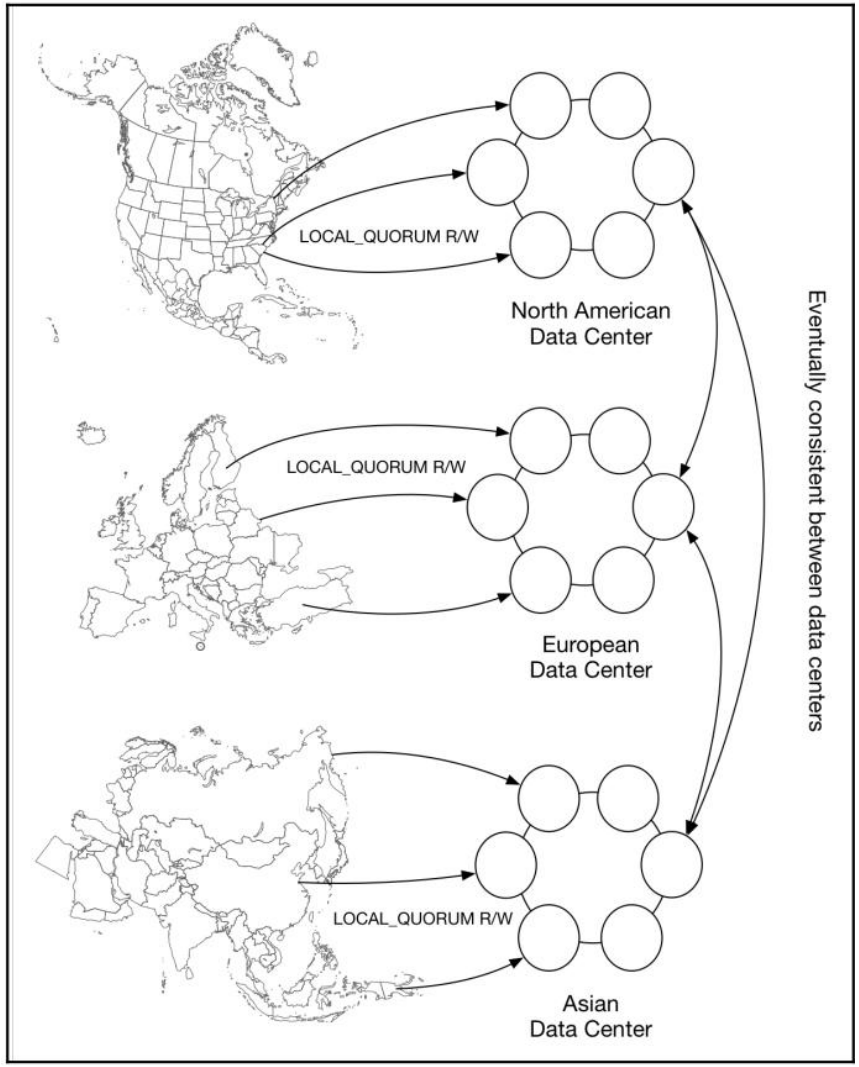
- Some application find it necessary to send requests to a data center located near the originator
- Geographic distribution is useful for systems that span the globe
- Routing all requests to a central location is impractical
- Reasons for geographic distribution
  - To reduce the latency between the originator and data centers
  - To mitigate the potential impact of natural disasters

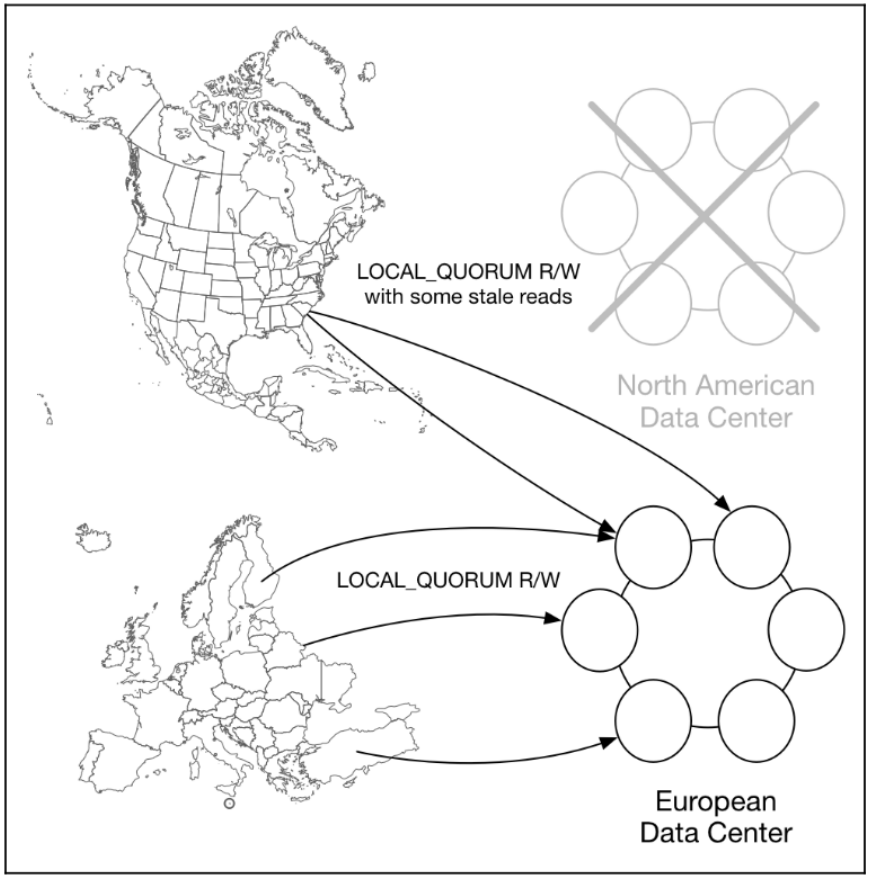
Data Centers and Racks - Geographic distribution -

- Features of geographic distribution
  - The ability to locate data centers in strategic global locations around the world
  - Improve performance
  - Support strong consistency

- To take advantage of these features:
  - Enable the use of the LOCAL_QUORUM consistency level to remove the performance penalty introduced by waiting for a remote data center to acknowledge the write

Data Centers and Racks
- Geographic distribution -





Data Centers and Racks
- Data analysis -

- Data analysis (Online Analytical Processing – OLAP) involves:
  - Taking normalized data from the transactional relational database
  - Moving it into a denormalized form for faster analysis
- This process involves significant Extract Transform and Load (ETL) overhead
- ETL inherently results in a delay in analyzing the data

Data Centers and Racks
- Data analysis -

- Cassandra offers for data analysis:
  - A denormalized data model as default
  - Support of multiple data centers
  - Support replication for load balancing
  - Robust integrations with the Hadoop
  - Robust integrations with the Spark

Overview

- Cassandra Architecture
  - Data Centers and Racks
  - Gossip and Failure Detection
  - Rings and Tokens
  - Vnodes (virtual nodes)
  - Partitioners
  - Replication Strategies
  - Consistency Levels
  - Queries and Coordinator Nodes
  - Commit Logs, Memtables, and SSTables
  - Caching
  - Hinted Handoff

Gossip and Failure Detection

- Cassandra uses a gossip protocol to support decentralization and partition tolerance
- The gossip protocol allows each node to keep track of state information about the other nodes in the cluster
- The gossiper runs every second on a timer
- The gossip protocols take their name from the concept of human gossip
  - A form of communication in which peers can choose with whom they want to exchange information

Gossip and Failure Detection

- The gossip protocols
  - Generally, assume a faulty network
  - Are commonly employed in very large, decentralized network systems
  - Are often used as an automatic mechanism for replication in distributed databases.
  - Cassandra is primarily implemented by the org.apache.cassandra.gms.Gossiper class, which is responsible for managing gossip for the local node

Gossip and Failure Detection

- Cassandra gossip is used for failure detection
- The Gossiper class maintains a list of nodes that are alive and dead
- When a server node is started:
  - It registers itself with the gossiper
  - It will receive from the gossiper the endpoint state information

Gossip and Failure Detection

- The gossiper works as follows:

1. Once per second, the gossiper:
   I. Will choose a random node in the cluster (friend)
   II. Will initialize a gossip session with the chosen friend

2. Each round of gossip requires three messages:
   I. The gossip initiator sends its chosen friend a GossipDigestSynMessage
   II. When the friend receives this message, it returns a GossipDigestAckMessage
   III. When the initiator receives the ack message from the friend, it sends the friend a GossipDigestAck2Message to complete the round of gossip.

Gossip and Failure Detection

- When the gossiper determines that another endpoint is dead:
  - "Convicts" that endpoint
  - Marks it as dead in its local list
  - Logs the modification

- Cassandra has robust support for failure detection

- Failure detections uses the algorithm for distributed computing called Phi Accrual Failure Detection.

Gossip and Failure Detection

- Phi Accrual Failure Detectors
  - Accrual Failure Detectors output a value associated with each process (or node)
  - This value is called Phi
  - The value is output in a manner that is designed from the ground up to be adaptive in the face of volatile network conditions
  - Phi is not a binary condition that simply checks whether a server is up or down
  - The Phi convict threshold in the configuration adjusts the sensitivity of the failure detector

Gossip and Failure Detection

- Phi Accrual Failure Detectors
  - Lower values increase the sensitivity to failure
  - Higher values decrease the sensitivity to failure
  - This process is not done in a linear fashion.
  - The Phi value refers to a level of suspicion that a server might be down.
  - Applications such as Cassandra that employ an AFD can specify variable conditions for the Phi value they emit.
  - Cassandra can generally detect a failed node in about 10 seconds using this mechanism

Gossip and Failure Detection

- Cassandra's accrual failure detection is based on two primary ideas:

1. The first general idea is that failure detection should be flexible
    - This is achieved by decoupling it from the application being monitored

2. The second and more novel idea challenges the notion of traditional failure detectors
    - Traditional failure detectors
        - Are implemented by simple "heartbeats"
        - Decide whether a node is dead or not dead based on whether a heartbeat is received or not
    - Accrual failure detection considers this approach as naive and finds a place in between the extremes of dead and alive - a suspicion level

Gossip and Failure Detection

- Cassandra's accrual failure
  - The failure monitoring system outputs a continuous level of "suspicion" regarding how confident it is that a node has failed.
  - This is desirable because it can consider fluctuations in the network environment.
  - For example, just because one connection gets caught up doesn't necessarily mean that the whole node is dead.
  - So, suspicion offers a more fluid and proactive indication of the weaker or stronger possibility of failure based on interpretation (the sampling of heartbeats), as opposed to a simple binary assessment

Overview

- Cassandra Architecture
  - Data Centers and Racks
  - Gossip and Failure Detection
  - Rings and Tokens
  - Vnodes (virtual nodes)
  - Partitioners
  - Replication Strategies
  - Consistency Levels
  - Queries and Coordinator Nodes
  - Commit Logs, Memtables, and SSTables
  - Caching
  - Hinted Handoff

Rings and Tokens

- Cassandra represents the data managed by a cluster as a ring
- Each node in the ring is assigned one or more ranges of data described by a tokens
- The token
  - Is **the hashed value of the primary key (row/partition key)**
  - Determines the nodes position in the ring
  - Is a 64-bit integer ID used to identify each partition
- Because the token is a 64-bit integer ID, it gives a possible range for tokens in the range $[-2^{63}, 2^{63} - 1]$

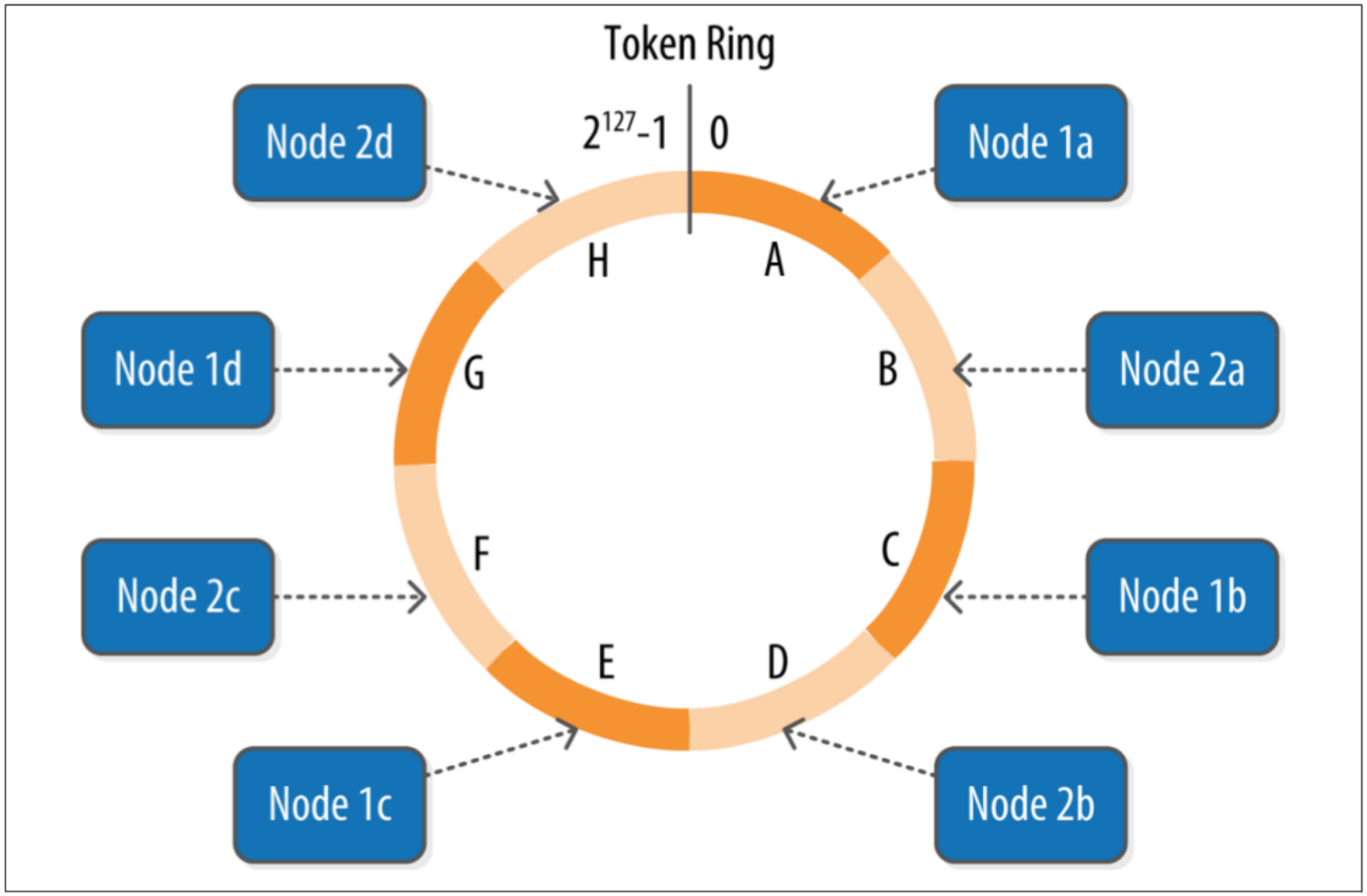
Rings and Tokens

- A node claims ownership of the range of values
  - Less than or equal to each token
  - Greater than the token of the previous node
- The node with lowest token owns the range
  - Less than or equal to its token
  - Greater than the highest token
- This range is also known as the "wrapping range“
- In this way, the tokens specify a complete ring

Rings and Tokens

- The ring is structured such that consecutive token ranges are spread across nodes in different racks
- Data is assigned to nodes by using a hash function to calculate a token for the partition key
- This partition key token is compared to the token values for the various nodes to identify the range, and therefore the node, that owns the data

Rings and Tokens



Overview

- Cassandra Architecture
  - Data Centers and Racks
  - Gossip and Failure Detection
  - Rings and Tokens
  - Vnodes (virtual nodes)
  - Partitioners
  - Replication Strategies
  - Consistency Levels
  - Queries and Coordinator Nodes
  - Commit Logs, Memtables, and SSTables
  - Caching
  - Hinted Handoff

Virtual Nodes

- Cassandra's 1.2 release introduced the concept of virtual nodes, also called **vnodes** for short
- Instead of assigning a single token to a node, the token range is broken up into multiple smaller ranges
- Each physical node is then assigned multiple tokens
- By default, each node will be assigned 256 of these tokens, meaning that it contains 256 virtual nodes

Virtual Nodes

- Virtual nodes have been enabled by default since 2.0.
- Vnodes make it easier to maintain a cluster containing heterogeneous machines
- For nodes in your cluster that have more computing resources available to them, you can increase the number of **vnodes** by setting the **num_tokens** property in the **cassandra.yaml** file
- Conversely, you might **set num_tokens** lower to decrease the number of **vnodes** for less capable machines.

Virtual Nodes

- Cassandra automatically handles the calculation of token ranges for each **vnode (can in consequence for each node)** in the cluster in proportion to their **num_tokens** value

- A further advantage of virtual nodes is that they speed up some of the more heavy-weight Cassandra operations such as
  - Bootstrapping a new node
  - Decommissioning a node
  - Repairing a node

- This is because the load associated with operations on multiple smaller ranges is spread more evenly across the nodes in the cluster.

Overview

- Cassandra Architecture
  - Data Centers and Racks
  - Gossip and Failure Detection
  - Rings and Tokens
  - Vnodes (virtual nodes)
  - Partitioners
  - Replication Strategies
  - Consistency Levels
  - Queries and Coordinator Nodes
  - Commit Logs, Memtables, and SSTables
  - Caching
  - Hinted Handoff

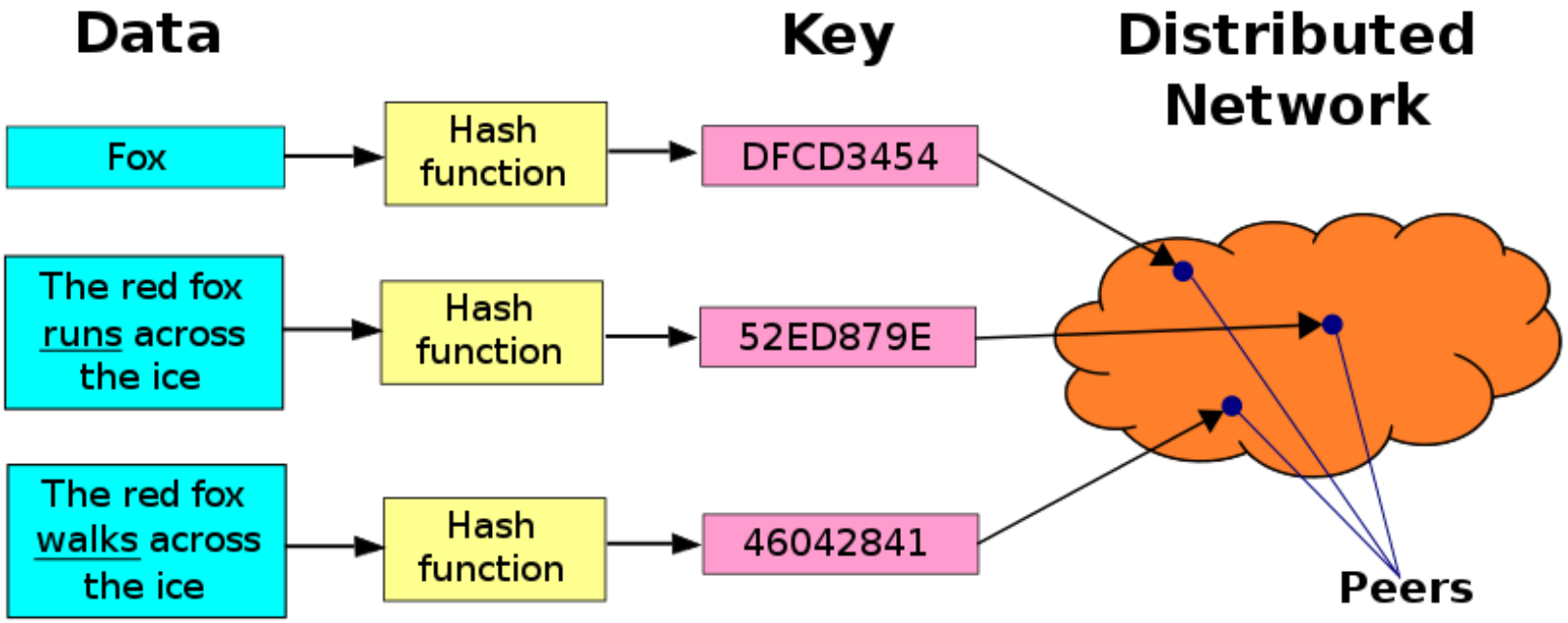
Partitioners

- A **partitioner** determines how data is distributed across the nodes in the cluster
- Cassandra stores data in wide rows, or "partitions“
- Each row has a **partition key** that is used to identify the partition
- A **partitioner** is a hash function for computing the token of a partition key
- Each row of data is distributed within the ring according to the value of the **partition key token**

Partitioners

- Cassandra provides several different **partitioners** that use a distributed hash table (DHT)
- A DHT is a class of a decentralized distributed system that provides a lookup service similar to a hash table
- For hashing it uses a Java implementation of the murmur algorithm that generates 64-bit hashes
- MurmurHash is a non-cryptographic hash function suitable for general hash-based lookup
- Because of Cassandra's generally pluggable design, you can also create your own **partitioner**

Partitioners



Overview

- Cassandra Architecture
  - Data Centers and Racks
  - Gossip and Failure Detection
  - Rings and Tokens
  - Vnodes (virtual nodes)
  - Partitioners
  - Replication Strategies
  - Consistency Levels
  - Queries and Coordinator Nodes
  - Commit Logs, Memtables, and SSTables
  - Caching
  - Hinted Handoff

Replication Strategies

- A node serves as a replica for different ranges of data
- If one node goes down, other replicas can respond to queries for that range of data
- Cassandra replicates data across nodes in a manner transparent to the user
- The replication factor is the number of nodes in your cluster that will receive copies (replicas) of the same data

Replication Strategies

- If the replication factor is 3, then three nodes in the ring will have copies of each row
- The first replica will always be the node that claims the range in which the token falls
- The remainder of the replicas are placed according to the replication strategy (or replica placement strategy)

Replication Strategies

- Cassandra provides two primary replication strategies (we will discuss others in the next course):
  - The SimpleStrategy
    - Places replicas at consecutive nodes around the ring
    - It starts with the node indicated by the partitioner
  - The NetworkTopologyStrategy
    - That allows the administrator to specify a different replication factor for each data center
    - Within a data center, it allocates replicas to different racks in order to maximize availability

Overview

- Cassandra Architecture
  - Data Centers and Racks
  - Gossip and Failure Detection
  - Rings and Tokens
  - Vnodes (virtual nodes)
  - Partitioners
  - Replication Strategies
  - Consistency Levels
  - Queries and Coordinator Nodes
  - Commit Logs, Memtables, and SSTables
  - Caching
  - Hinted Handoff

Consistency Levels

- Cassandra provides tuneable consistency levels that allow you to make trade-offs between consistency, availability, and partition tolerance at a fine-grained level
- Cassandra lets the client specify a consistency level on each read or write query that indicates how much consistency you require
- A higher consistency level means that more nodes need to respond to a read or write query, ensuring that the values present on each replica are the same

Consistency Levels

- For read queries, the consistency level specifies how many replica nodes must respond to a read request before returning the data
- For write operations, the consistency level specifies how many replica nodes must respond for the write to be reported as successful to the client
- Because Cassandra is eventually consistent, updates to other replica nodes may continue in the background

Consistency Levels

- The available consistency levels include ANY, ONE, TWO, and THREE, each of which specify an absolute number of replica nodes that must respond to a request
- The QUORUM consistency level requires a response from a majority of the replica nodes (N is the replication factor): N/2 + 1
- The ALL consistency level requires a response from all of the replicas
- For both reads and writes, the consistency levels of
  - ANY, ONE, TWO , and THREE are considered weak
  - QUORUM and ALL are considered strong

Consistency Levels

- Consistency is tuneable in Cassandra because clients can specify the desired consistency level on both reads and writes
- There is an equation that is popularly used to represent the way to achieve strong consistency in Cassandra: R + W > N = strong consistency
- In this equation
  - R the read replica count
  - W the write replica count
  - N the replication factor
- All client reads will see the most recent write in this scenario, and you will have strong consistency

Overview

- Cassandra Architecture
  - Data Centers and Racks
  - Gossip and Failure Detection
  - Rings and Tokens
  - Vnodes (virtual nodes)
  - Partitioners
  - Replication Strategies
  - Consistency Levels
  - Queries and Coordinator Nodes
  - Commit Logs, Memtables, and SSTables
  - Caching
  - Hinted Handoff

Queries and Coordinator Nodes

- A client may connect to any node in the cluster to initiate a read or write query
- This node is known as the coordinator node
- The coordinator identifies which nodes are replicas for the data that is being written or read and forwards the queries to them

Queries and Coordinator Nodes

- Write:

1. The coordinator node contacts all replicas, as determined by the consistency level and replication factor

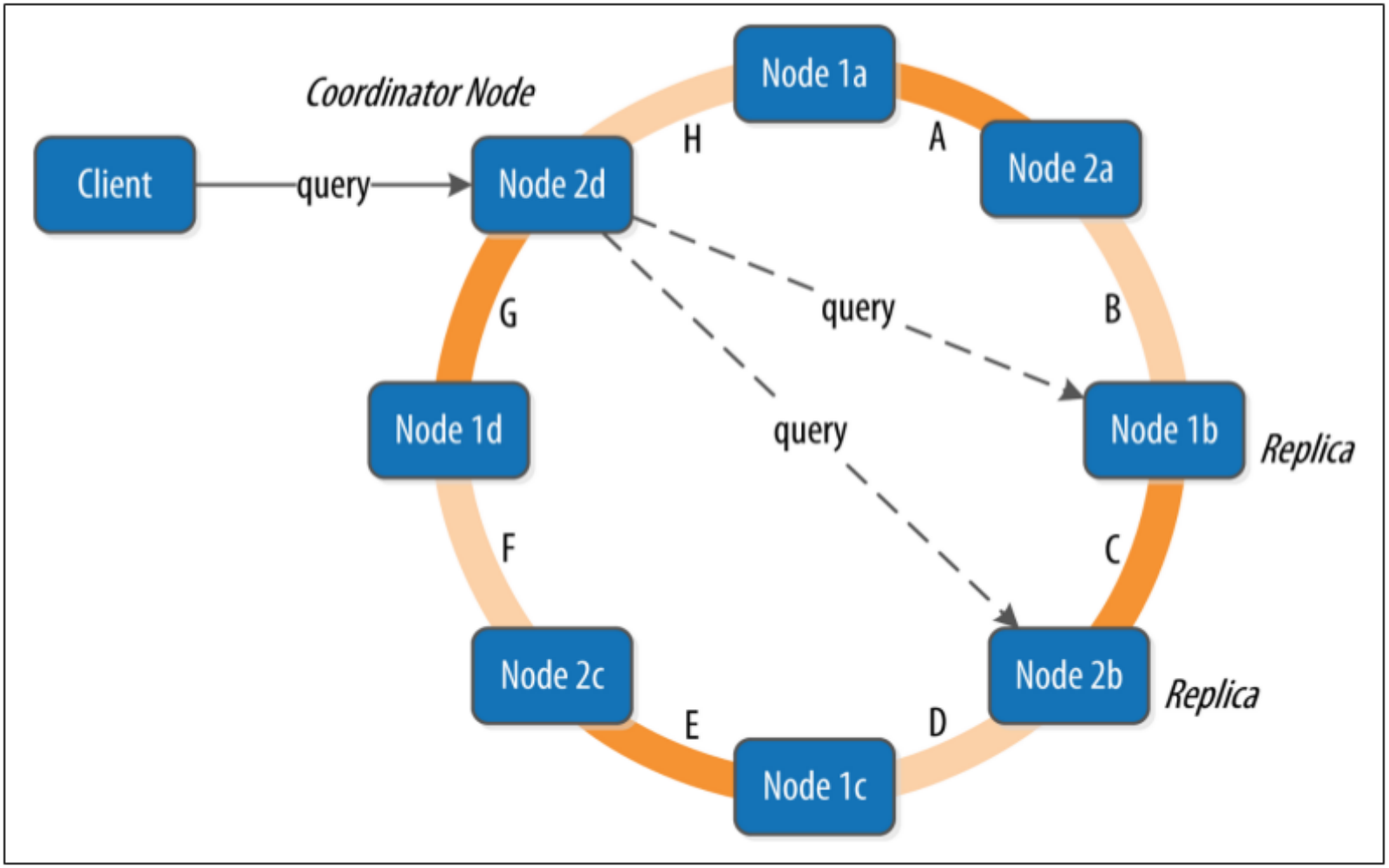
2. Considers the write successful when a number of replicas commensurate with the consistency level acknowledge the write

- Read:

1. The coordinator contacts enough replicas to ensure the required consistency level is met

2. Returns the data to the client

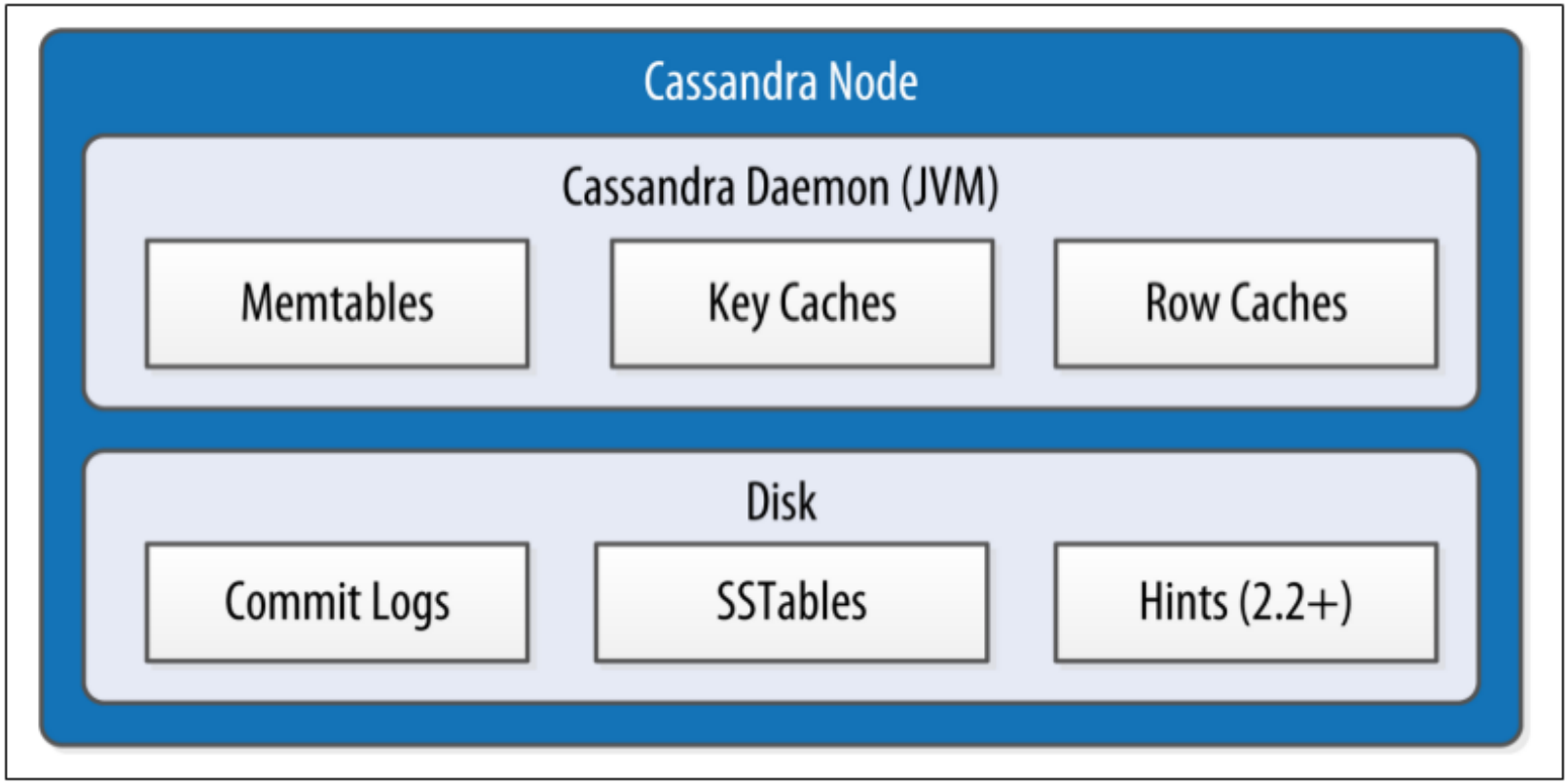
Queries and Coordinator Nodes



Overview

- Cassandra Architecture
  - Data Centers and Racks
  - Gossip and Failure Detection
  - Rings and Tokens
  - Vnodes (virtual nodes)
  - Partitioners
  - Replication Strategies
  - Consistency Levels
  - Queries and Coordinator Nodes
  - Commit Logs, Memtables, and SSTables
  - Caching
  - Hinted Handoff

Commit Logs, Memtables and SSTables



Commit Logs, Memtables and SSTables

- Cassandra stores data both in memory and on disk to provide both high performance and durability

- To support the writing and reading of data from tables, Cassandra uses
  1. Commit logs
  2. Memtables
  3. SSTables
  4. Key Caches
  5. Row Caches
  6. Hints

Commit Logs, Memtables and SSTables

- Commit Logs
  - When a write operation is performed, it is immediately written to a commit log
  - The commit log is a crash-recovery mechanism that supports Cassandra's durability goals
  - A write will not count as successful until it's written to the commit log
  - This ensures that if a write operation does not make it to the in-memory store (memtable) on on disk store (SSTable), it will still be possible to recover the data
    - The commit log can ensure that data is not lost if the database is shut down or it crashes unexpectedly
    - That's because the next time you start the node, the commit log gets replayed
    - In fact, that's the only time the commit log is read – clients never read from it

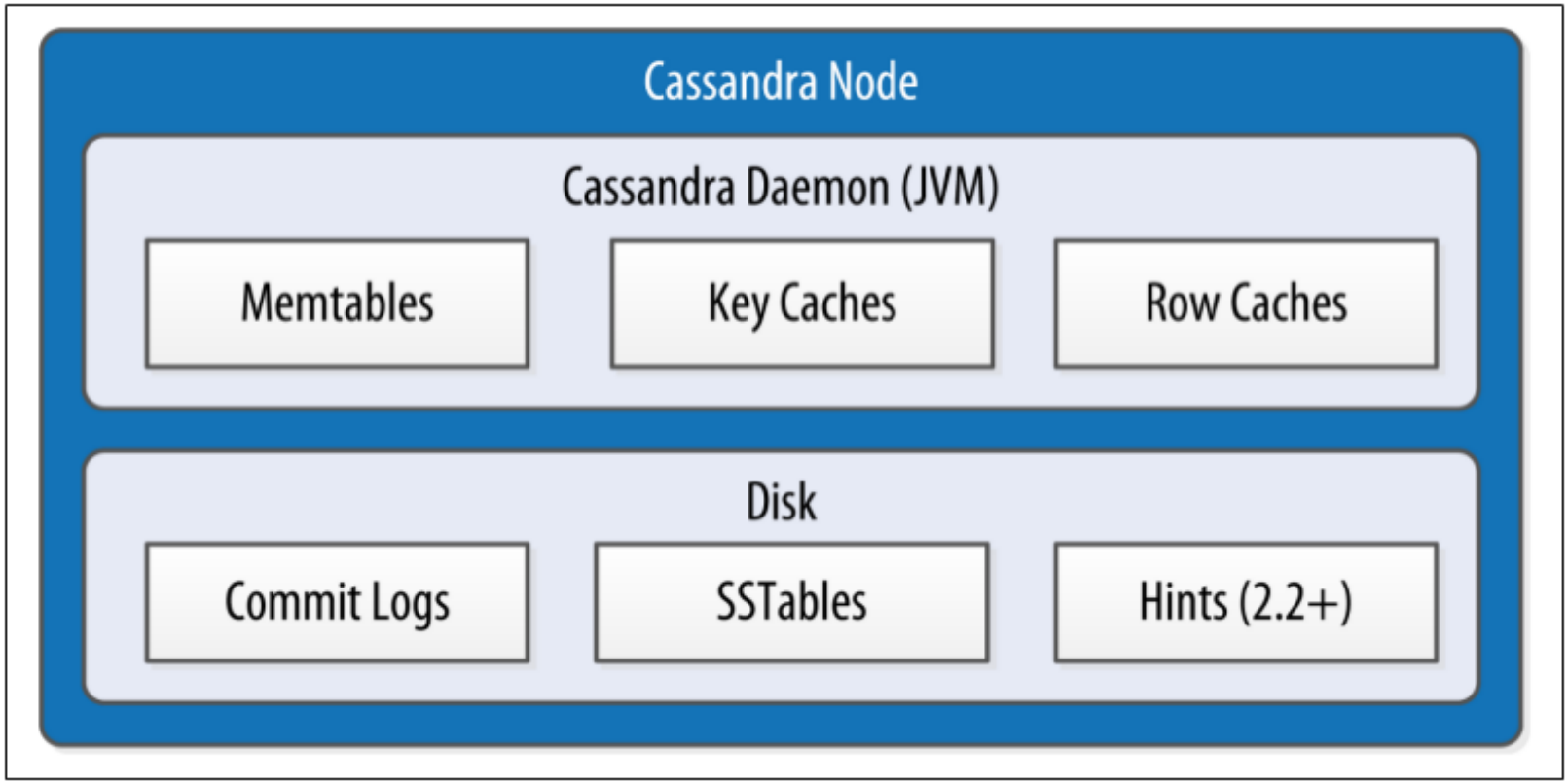
Commit Logs, Memtables and SSTables

- Commit Logs
  - Each commit log maintains an internal bit flag to indicate whether it needs flushing
  - When a write operation is first received, it is written to the commit log and its bit flag is set to 1
  - There is only one bit flag per table, because only one commit log is ever being written to across the entire server
  - All writes to all tables will go into the same commit log, so the bit flag indicates whether a particular commit log contains anything that hasn't been flushed for a particular table
  - After it's written to the commit log, the value is written to a memory-resident data structure called the memtable

Commit Logs, Memtables and SSTables

- Commit Logs
  - Once the memtable has been properly flushed to disk, the corresponding commit log's bit flag is set to 0, indicating that the commit log no longer has to maintain that data for durability purposes
  - Like regular logfiles, commit logs have a configurable rollover threshold, and once this file size threshold is reached, the log will roll over, carrying with it any extant dirty bit flags

Commit Logs, Memtables and SSTables



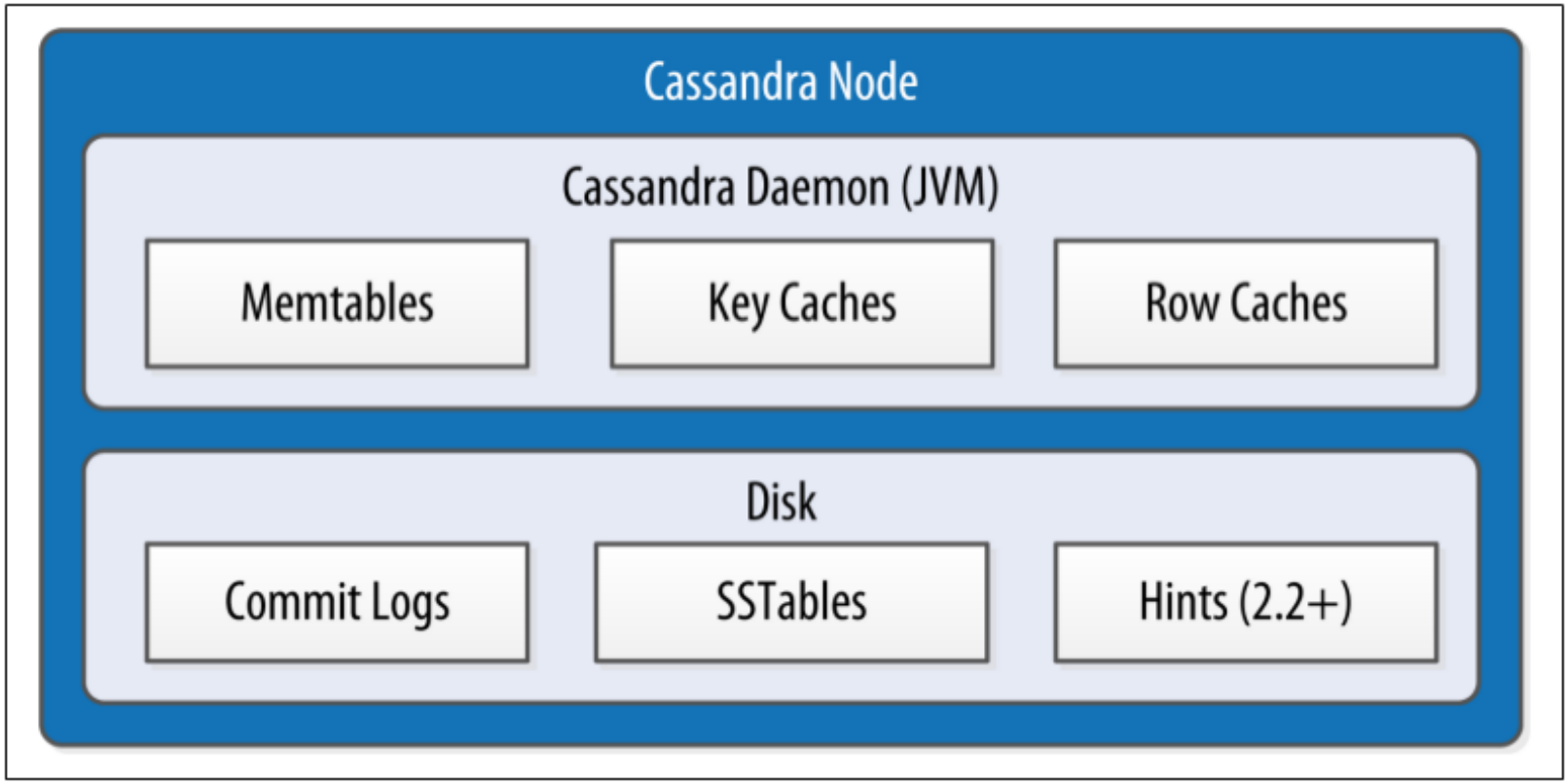
Commit Logs, Memtables and SSTables

- Memtables
  - Each memtable contains data for a specific table
  - In early implementations of Cassandra, memtables were stored on the JVM heap, but now the majority of memtable data was moved to native memory
  - This makes Cassandra less susceptible to fluctuations in performance due to Java garbage collection

Commit Logs, Memtables and SSTables

- Memtables
  - When the number of objects stored in the memtable reaches a threshold, the contents of the memtable are flushed to disk in a file called an SSTable.
  - A new memtable is then created.
  - This flushing is a non-blocking operation; multiple memtables may exist for a single table, one current and the rest waiting to be flushed.
  - They typically should not have to wait very long, as the node should flush them very quickly unless it is overloaded

Commit Logs, Memtables and SSTables



Commit Logs, Memtables and SSTables

- SSTables
  - Once a memtable is flushed to disk as an SSTable, it is immutable and cannot be changed by the application.
  - Even though SSTables are compacted, this compaction changes only their on-disk representation.
  - It essentially performs the "merge" step of a mergesort into new files and removes the old files on success.
  - Cassandra supports the compression of SSTables in order to maximize use of the available storage.
  - This compression is configurable per table.
  - Each SSTable also has an associated Bloom filter, which is used as an additional performance enhancer

Commit Logs, Memtables and SSTables

- SSTables
  - All writes are sequential, which is the primary reason that writes perform so well in Cassandra
  - No reads or seeks of any kind are required for writing a value to Cassandra because all writes are append-only operations
  - This makes one key limitation on performance the speed of your disk
  - Compaction is intended to minimize the reorganization of data: uses sequential I/O

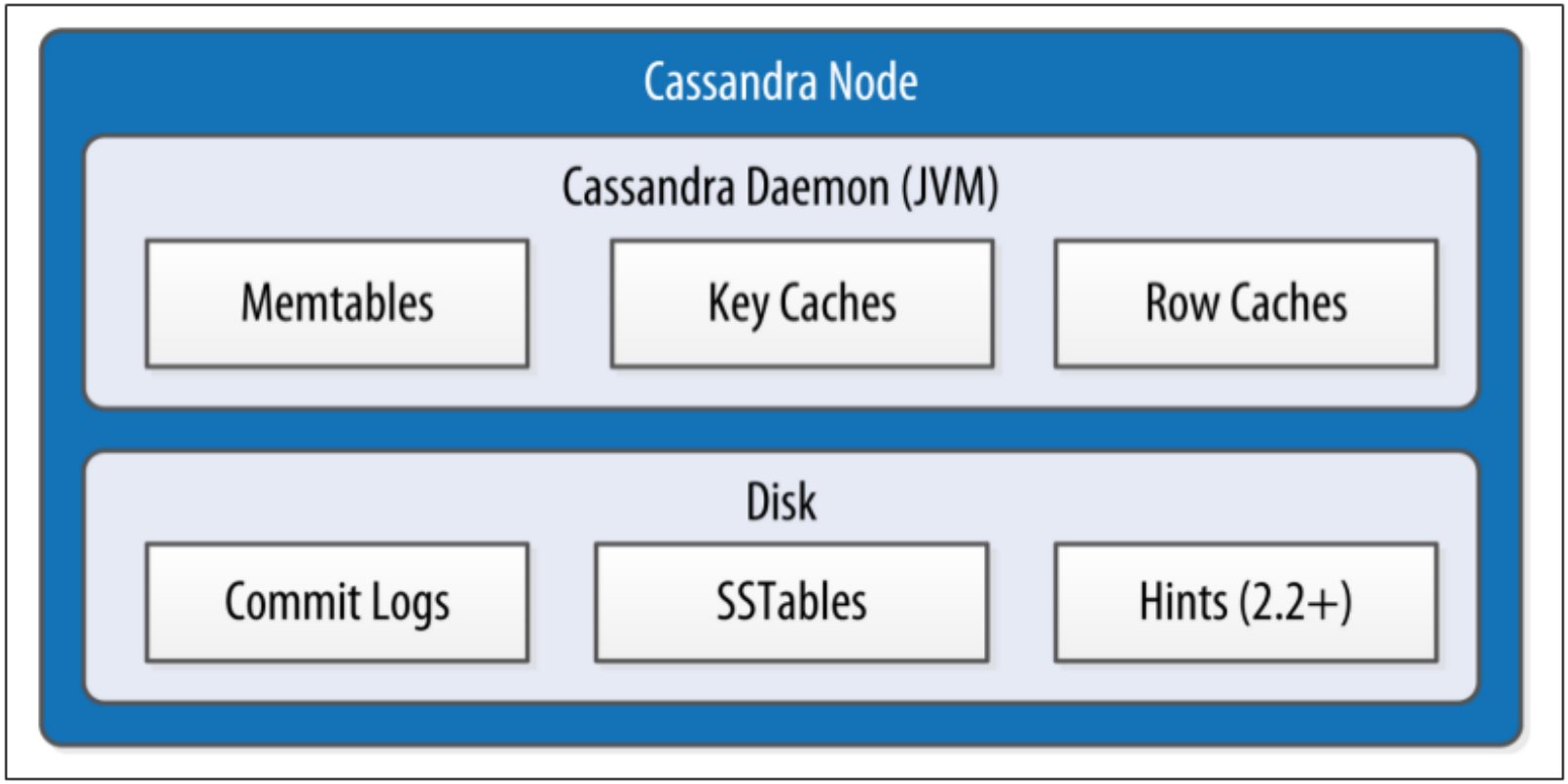
Commit Logs, Memtables and SSTables

- SSTables
  - The write operation is just an immediate append
  - Compaction helps to organize for better future read performance
  - If Cassandra naively inserted values where they ultimately belonged, writing clients would pay for seeks up front
  - On reads, Cassandra will read both SSTables and memtables to find data values, as the memtable may contain values that have not yet been flushed to disk.

Overview

- Cassandra Architecture
  - Data Centers and Racks
  - Gossip and Failure Detection
  - Rings and Tokens
  - Vnodes (virtual nodes)
  - Partitioners
  - Replication Strategies
  - Consistency Levels
  - Queries and Coordinator Nodes
  - Commit Logs, Memtables, and SSTables
  - Caching
  - Hinted Handoff

Caching


Cassandra Node
Cassandra Daemon (JVM)
Memtables
Key Caches
Row Caches
Disk
Commit Logs
SSTables
Hints (2.2+)

Caching

- Cassandra provides three forms of caching:

1. Key caches
   - Stores a map of partition keys to row index entries
   - Facilitating faster read access into SSTables stored on disk
   - The key cache is stored on the JVM heap

2. Row caches
   - Uses entire rows
   - Can greatly speed up read access for frequently accessed rows, at the cost of more memory usage
   - The row cache is stored in off-heap memory

3. The counter cache
   - Developed to improve counter performance by reducing lock contention for the most frequently accessed counters

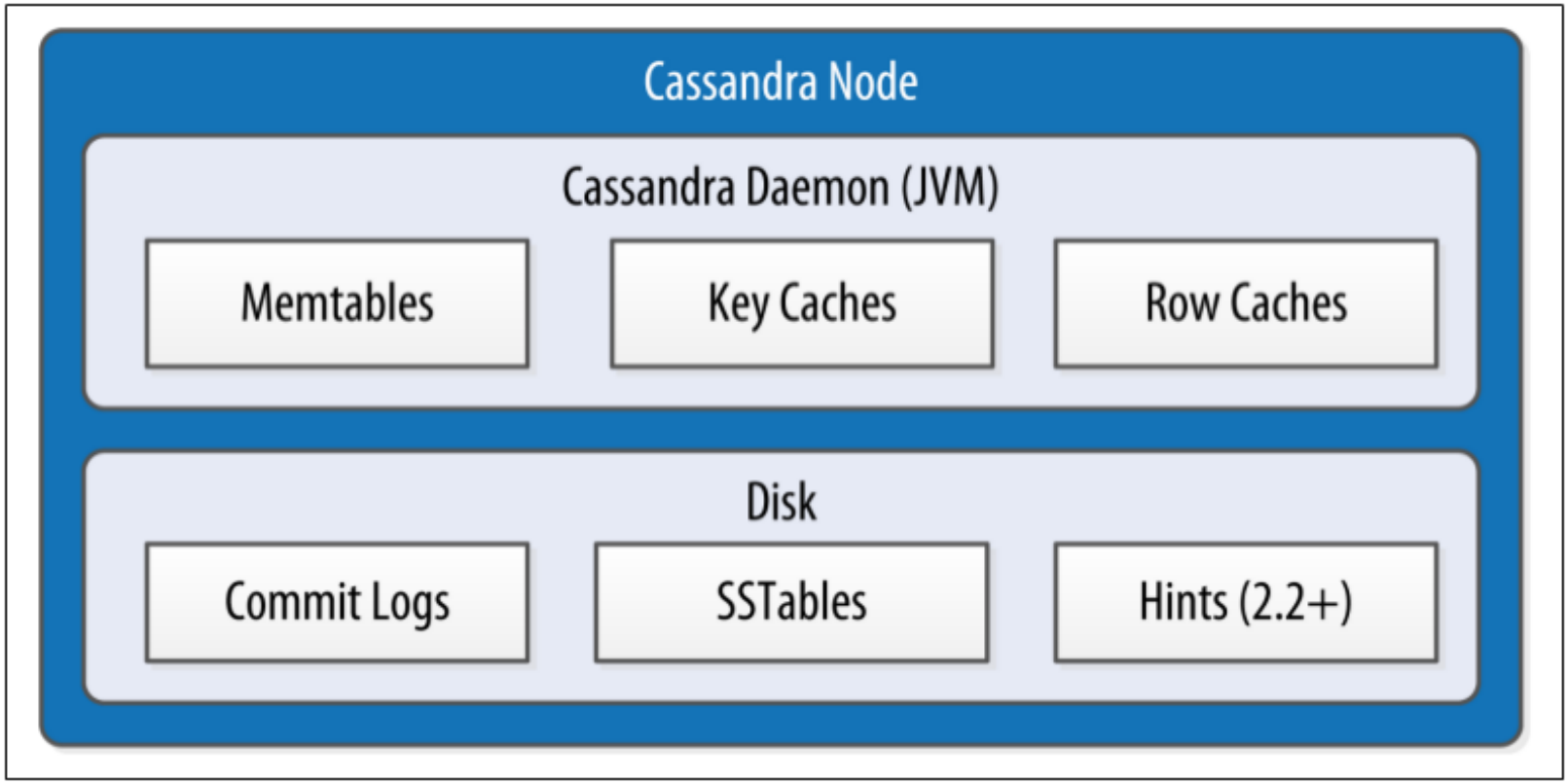
Caching

- By default, key and counter caching are enabled
- Row caching is disabled by default because it requires more memory
- Cassandra saves its caches to disk periodically in order to warm them up more quickly on a node restart

Overview

- Cassandra Architecture
  - Data Centers and Racks
  - Gossip and Failure Detection
  - Rings and Tokens
  - Vnodes (virtual nodes)
  - Partitioners
  - Replication Strategies
  - Consistency Levels
  - Queries and Coordinator Nodes
  - Commit Logs, Memtables, and SSTables
  - Caching
  - Hinted Handoff

Hinted Handoff



Hinted Handoff

- A write request is sent to Cassandra, but a replica node where the write properly belongs is not available (due to network partition, hardware failure, or some other reason)
- In this case, the coordinator will create a hint
- This is used by Cassandra to ensure general availability in such cases
- Cassandra holds a separate hint for each partition that is to be written
- This allows Cassandra to
  - Be always available for writes
  - Enables a cluster to sustain the same write load even when some of the nodes are down
- It also reduces the time that a failed node will be inconsistent after it does come back online

Bibliography

- Apache Cassandra Documentation https://cassandra.apache.org/doc/latest/

- Jeff Carpenter, Eben Hewitt. Cassandra: The Definitive Guide: Distributed Data at Web Scale. O'Reilly Media, Inc., 2016.

- Christof Strauch. "NoSQL databases." *Lecture Notes, Stuttgart Media University* 20.24 (2011): 79.

- Eric Redmond, Jim Wilson. Seven databases in seven weeks: a guide to modern databases and the NoSQL movement. 1nd Edition 2012

- Luc Perkins, Jim Wilson, and Eric Redmond. Seven databases in seven weeks: a guide to modern databases and the NoSQL movement. 2nd Edition 2018

University
Politehnica
of Bucharest

Cassandra (III)

Ciprian-Octavian Truică
ciprian.truica@upb.ro

Overview

- Installation
- Cassandra Query Language (CQL)
- CQL Data Types
- CQL Secondary Indexes
- CQL Aggregation
- CQL User Defined Functions (UDFs)

Installation

- To install Cassandra on a single node:
  - Download the archive

$ wget https://dlcdn.apache.org/cassandra/5.0.2/apache-cassandra-5.0.2-bin.tar.gz

  - Unzip the archive

$ tar -zxvf apache-cassandra-5.0.2-bin.tar.gz

  - Run Cassandra

$ cd ./apache-cassandra/bin
$ ./cassandra

Cluster Installation

- Installing Cassandra on a cluster (follow all the steps on each node)
  - Download the archive
  - Unzip the archive
  - Edit the cassandra.yaml ($CASSANDRA_HOME/conf/cassandra.yaml)
  - Start Cassandra on each node

Cluster Installation

- Edit the cassandra.yaml
  - cluster_name:
    - This is the name of your cluster.
  - - seeds:
    - This is a comma-delimited list of the IP address of each node in the cluster.
  - listen_address:
    - This is IP address that other nodes in the cluster will use to connect to this one.
    - It defaults to localhost and needs changed to the IP address of the node.

Cluster Installation

- Edit the cassandra.yaml
  - rpc_address:
    - This is the IP address for remote procedure calls.
    - It defaults to localhost.
    - If the server's hostname is properly configured, leave this as is.
    - Otherwise, change to server's IP address or the loopback address (127.0.0.1).
  - endpoint_snitch:
    - Name of the snitch, which is what tells Cassandra about what its network looks like.
    - This defaults to SimpleSnitch, which is used for networks in one datacenter.

Cluster Installation

- Edit the cassandra.yaml
  - auto_bootstrap:
    - This directive is not in the configuration file, so it has to be added and set to false.
    - This makes new nodes automatically use the right data.
    - It is optional if you're adding nodes to an existing cluster, but required when you're initializing a fresh cluster, that is, one with no data.

Cluster Installation

- Out of the box, Cassandra provides for endpoint_snitch:
  - SimpleSnitch:
    - Treats Strategy order as proximity.
    - This can improve cache locality when disabling read repair.
    - Only appropriate for single-datacenter deployments.
  - GossipingPropertyFileSnitch
    - This should be your go-to snitch for production use.
    - The rack and datacenter for the local node are defined in cassandra-rackdc.properties and propagated to other nodes via gossip.
    - If cassandra-topology.properties exists, it is used as a fallback, allowing migration from the PropertyFileSnitch.

Cluster Installation

- Out of the box, Cassandra provides for endpoint_snitch:
  - PropertyFileSnitch:
    - Proximity is determined by rack and data center, which are explicitly configured in cassandra-topology.properties.
  - Ec2Snitch:
    - Appropriate for EC2 deployments in a single Region.
    - Loads Region and Availability Zone information from the EC2 API.
    - The Region is treated as the datacenter, and the Availability Zone as the rack.
    - Only private IPs are used, so this will not work across multiple Regions.

Cluster Installation

- Out of the box, Cassandra provides for endpoint_snitch:
  - Ec2MultiRegionSnitch:
    - Uses public IPs as broadcast_address to allow cross-region connectivity.
    - Thus, you should set seed addresses to the public IP as well.
    - You will need to open the storage_port or ssl_storage_port on the public IP firewall.
    - For intra-Region traffic, Cassandra will switch to the private IP after establishing a connection.

Cluster Installation

- Out of the box, Cassandra provides for endpoint_snitch:
  - RackInferringSnitch:
    - Proximity is determined by rack and data center, which are assumed to correspond to the 3rd and 2nd octet of each node's IP address, respectively.
    - Unless this happens to match your deployment conventions, this is best used as an example of writing a custom Snitch class and is provided in that spirit.

Cluster Installation

```
# The name of the cluster. This is mainly used to prevent machines in
# one logical cluster from joining another.
cluster_name: 'DBCassandraCluster'

...

# any class that implements the SeedProvider interface and has a
# constructor that takes a Map<String, String> of parameters will do.
seed_provider:
    # Addresses of hosts that are deemed contact points.
    # Cassandra nodes use this list of hosts to find each other and learn
    # the topology of the ring.  You must change this if you are running
    # multiple nodes!
    - class_name: org.apache.cassandra.locator.SimpleSeedProvider
      parameters:
          # seeds is actually a comma-delimited list of addresses.
          # Ex: "<ip1>,<ip2>,<ip3>"
          - seeds: "172.19.3.31,172.19.3.32,172.193.33"
```

Cluster Installation

```
# Address or interface to bind to and tell other Cassandra nodes to connect to.
# You _must_ change this if you want multiple nodes to be able to communicate!
#
# Set listen_address OR listen_interface, not both.
#
# Leaving it blank leaves it up to InetAddress.getLocalHost(). This
# will always do the Right Thing _if_ the node is properly configured
# (hostname, name resolution, etc), and the Right Thing is to use the
# address associated with the hostname (it might not be).
#
# Setting listen_address to 0.0.0.0 is always wrong.
#
listen_address: 172.19.3.31

...

# The address or interface to bind the Thrift RPC service and native transport
# server to.
#
# Set rpc_address OR rpc_interface, not both.
#
# Leaving rpc_address blank has the same effect as on listen_address
# (i.e. it will be based on the configured hostname of the node).
#
# Note that unlike listen_address, you can specify 0.0.0.0, but you must also
# set broadcast_rpc_address to a value other than 0.0.0.0.
#
# For security reasons, you should not expose this port to the internet.  Firewall it if needed.
rpc_address: 172.19.3.31
```

Cluster Installation

```
# You can use a custom Snitch by setting this to the full class name
# of the snitch, which will be assumed to be on your classpath.
endpoint_snitch: GossipingPropertyFileSnitch


auto_bootstrap: false
```

Cluster Installation

- To verify the status of you cluster use nodetool utility.

$ nodetool status

```
Datacenter: dc1
===============
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
--  Address       Load       Tokens       Owns (effective)  Host ID                               Rack
UN  172.19.3.33   321.07 KiB  256          100.0%            26e4ec87-b562-448e-af70-7e62dba50abe  rack1
UN  172.19.3.32   348.49 KiB  256          100.0%            90a71c7a-15ac-41b8-8437-78984d89b7e4  rack1
UN  172.19.3.31   322.63 KiB  256          100.0%            89cec698-d686-4d97-9024-088babd9ebc3  rack1
```

CQL

- Cassandra Query Language (CQL)
- The syntax of CQL is similar in many ways to SQL, but with some important differences.
- To start the CLI use cqlsh ($CASSANDRA_HOME/bin/cqlsh)
- The default port for cqlsh is 9042
- If the server is listening on localhost then issue the following command:

```
$ cqlsh
```

- If the server is listening on a different IP/PORT (used by one of the nodes in the cluster), use

```
$ cqlsh IP PORT
```

CQL

- To see all the commands use HELP
- See the cluster details

```
cqlsh> DESCRIBE CLUSTER;
```

- To see the keyspaces available:

```
cqlsh> DESCRIBE KEYSPACES;
```

- Initially this list will consist of several system keyspaces.
- Once you have created your own keyspaces, they will be shown as well.
- The system keyspaces are managed internally by Cassandra.
- To see the client, server, and protocol versions:

```
cqlsh> SHOW VERSION;
```

CQL
Creating a Keyspace

- When creating a keyspace, there are different strategies to choose from:

1. SimpleStrategy
   - Places replicas at consecutive nodes around the ring, starting with the node indicated by the partitioner.
   - The replication factor must be specified

2. NetworkTopologyStrategy
   - Allows you to specify a different replication factor for each data center

3. OldNetworkTopologyStrategy - deprecated staring with v4
   - Is similar to the NetworkTopologyStrategy in that it places replicas in multiple data centers, but its algorithm is less sophisticated.
   - This is depricate, it was keept just for backwards compatibility.

CQL

- The following command creats a Keyspace

```
cqlsh> CREATE KEYSPACE faculty WITH replication = {'class': 'SimpleStrategy',
'replication_factor': 3 };
```

- To bypass writing to the commit log for the keyspace then you can use durable_writes property.

```
cqlsh> CREATE KEYSPACE faculty WITH replication = {'class': 'SimpleStrategy',
'replication_factor': 3 } AND durable_writes = true;
```

- To delete a Keyspace use the DROP command

```
cqlsh> DROP KEYSPACE faculty;
```

- To use a keyspace and start creating tables:

```
cqlsh> USE faculty
```

CQL

- To creating a table, use the CREATE TABLE command

```
cqlsh:faculty> CREATE TABLE TEACHERS(
                    id int PRIMARY KEY,
                    firstname VARCHAR,
                    lastname VARCHAR,
                    age INT );
```

- Tables must have a primary key upon creation
- To modify a table, use ALTER TABLE command
- To drop a table, use DROP TABLE command

```
cqlsh:faculty> DROP TABLE teachers;
```

CQL

- Adding data to a table use the INSERT command
- The columns need to be specified

```
cqlsh:faculty> INSERT INTO teachers(id, firstname, lastname, age) VALUES(1, 'Ciprian-Octavian', 'Truică', 32);

cqlsh:faculty> INSERT INTO teachers(id, firstname, lastname, age) VALUES(2, 'Florin', 'Rădulescu', 58);

cqlsh:faculty> INSERT INTO teachers(id, firstname, lastname, age) VALUES(3, 'Alexandru', 'Boicea', 59);

cqlsh:faculty> INSERT INTO teachers(id, firstname, lastname, age) VALUES(4, 'Sorin', 'Ciolofan', 37);

cqlsh:faculty> INSERT INTO teachers(id, firstname, lastname, age) VALUES(5, 'Dan', 'Schrager', 59);
```

CQL

- Selection and projection is done using the SELECT command

```
cqlsh:faculty> SELECT * FROM teachers;
```

```
cqlsh:faculty> SELECT firstname, lastname FROM teachers
WHERE id IN (1, 3);
```

- When filtering we need to specify that we are filtering the data using ALLOW FILTERING property

```
cqlsh:faculty> SELECT firstname, lastname FROM teachers
WHERE age <= 40 ALLOW FILTERING;
```

CQL

- Some limitations on SELECT command:
  - There are no nested queries
  - Some operators don't work, e.g., between ... and ... ,
  - Some operators work only on primary keys, e.g., IN.
  - There are no JOIN operations

CQL

- Update a value

```
cqlsh:faculty> UPDATE teachers SET lastname = 'Truică' WHERE id = 1;

cqlsh:faculty> UPDATE teachers SET lastname = 'Truică', firstname = 'Ciprian'
WHERE id = 1;
```

- Delete a value

```
cqlsh:faculty> DELETE FROM teachers WHERE id = 5;
```

- Delete all values

```
cqlsh:faculty> TRUNCATE teachers;
```

CQL Data Types

- CQL Data Types
  - Numeric Data Types
  - Textual Data Types
  - Time and Identity Data Types
  - Other Simple Data Types
  - Collections
  - User-Defined Types

CQL Data Types
Numeric Data Types

- Integer and floating-point numbers.

- These types are similar to standard types in Java:
  - int - A 32-bit signed integer (as in Java)
  - bigint - A 64-bit signed long integer (equivalent to a Java long)
  - smallint - A 16-bit signed integer (equivalent to a Java short)
  - tinyint - An 8-bit signed integer (as in Java)
  - varint - A variable precision signed integer (equivalent to java.math.BigInteger)
  - float - A 32-bit IEEE-754 floating point (as in Java)
  - double - A 64-bit IEEE-754 floating point (as in Java)
  - decimal - A variable precision decimal (equivalent to java.math.BigDecimal)

CQL Data Types
Textual Data Types

- CQL provides two data types for representing text:
  - text, varchar - Synonyms for a UTF-8 character string
  - ascii - An ASCII character string

- The ascii type is most useful if you are dealing with legacy data that is in ASCII format.

CQL Data Types
Time and Identity Data Types

- The identity of data elements such as rows and partitions is important in any data model in order to be able to access the data.

- Cassandra provides several types which prove quite useful in defining unique partition keys.

- Time Data Types:
  - timestamp
  - date and time
  - uuid
  - timeuuid

CQL Data Types
Time and Identity Data Types

- timestamp
  - Each column has a timestamp indicating when it was last modified
  - A timestamp can also be used as the value of a column itself
  - The time can be encoded as a 64-bit signed integer
  - It is typically much more useful to input a timestamp using one of several supported ISO 8601 date formats
  - The best practice is to always provide time zones rather than relying on the operating system time zone configuration

CQL Data Types
Time and Identity Data Types

- date, time
  - Releases through Cassandra 2.1 only had the timestamp type to represent times, which included both a date and a time of day.

  - The 2.2 release introduced date and time types that allowed these to be represented independently, that is:
    - A date without a time
    - A time of day without reference to a specific date.

CQL Data Types
Time and Identity Data Types

- uuid
  - Universally unique identifier (UUID) is a 128-bit value in which the bits conform to one of several types, of which the most commonly used are known as Type 1 and Type 4.
  - The CQL uuid type is a Type 4 UUID, which is based entirely on random numbers.
  - UUIDs are typically represented as dash-separated sequences of hex digits.
  - The uuid type is often used as a surrogate key, either by itself or in combination with other values.
  - You can obtain a Type 4 UUID value via the uuid() function and use this value in an INSERT or UPDATE.

CQL Data Types
Time and Identity Data Types

- timeuuid
  - This is a Type 1 UUID, which is based on the following 3 components:
    - The MAC address of the computer
    - The system time
    - A sequence number used to prevent duplicates
  - This type is frequently used as a conflict-free timestamp
  - cqlsh provides several convenience functions for interacting with the timeuuid type:
    - now()
    - dateOf()
    - unixTimestampOf().

CQL Data Types
Other Simple Data Types

- boolean
  - This is a simple true/false value.
  - The cqlsh is case insensitive in accepting these values but outputs True or False

- inet
  - This type represents IPv4 or IPv6 Internet addresses.
  - cqlsh accepts any legal format for defining IPv4 addresses, including:
    - Dotted or non-dotted representations containing:
      - Decimal values.
      - Octal values.
      - Hexadecimal values.
  - IPv6 addresses are represented as eight groups of four hexadecimal digits, separated by colons

CQL Data Types
Other Simple Data Types

- counter
  - The counter data type provides 64-bit signed integer
  - Its value cannot be set directly, but only incremented or decremented.
  - Cassandra is one of the few databases that provides race-free increments across data centers
  - Counters are frequently used for tracking statistics such as numbers of page views, tweets, log messages, and so on.
  - The counter type has some special restrictions: It cannot be used as part of a primary key.

CQL Data Types
Other Simple Data Types

- counter
  - The increment and decrement operators are not idempotent.
  - There is no operation to reset a counter directly, but you can approximate a reset by reading the counter value and decrementing by that value.
  - Unfortunately, this is not guaranteed to work perfectly, as the counter may have been changed elsewhere in between reading and writing.

CQL Data Types

- CQL Collections
  - Set
  - List
  - Map

CQL Data Types
Set

- The set data type stores a collection of elements
- The elements are unordered, but cqlsh returns the elements in sorted order
- Sets can contain:
    - Simple types
    - User-defined types
    - Other collections
- One advantage of using set is the ability to insert additional items without having to read the contents first

CQL Data Types
Set

```
cqlsh:faculty> ALTER TABLE teachers ADD offices SET<TEXT>;

cqlsh:faculty> UPDATE teachers SET offices= {'EF208', 'PR305', 'EG105'} WHERE id=1;

cqlsh:faculty> UPDATE teachers SET offices= {'Vice Dean Office'} WHERE id=2;

cqlsh:faculty> UPDATE teachers SET offices= {'EF309'} WHERE id=3;

cqlsh:faculty> UPDATE teachers SET offices= {'EF209'} WHERE id=4;

cqlsh:faculty> UPDATE teachers SET offices= {'PR304'} WHERE id=5;
```

CQL Data Types
Set operations

- Adding a new element using +

```
cqlsh:faculty> UPDATE teachers SET offices= offices + {'EF309'} WHERE id =2;
```

- Removing an element using -

```
cqlsh:faculty> UPDATE teachers SET offices= offices - {'EG105'} WHERE id =1;
```

- Empty the set use {}

```
cqlsh:faculty> UPDATE teachers SET offices= { } WHERE id =4;
```

- Select

```
cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE offices
CONTAINS 'EF309' ALLOW FILTERING;
```

CQL Data Types
List

- The list data type contains an unordered list of elements.
- By default, the values are stored in order of insertion
- Uses the same operations as sets to add and remove an element
- The empty a list is []
- Indexes can be used to access elements (in the WHERE clause of SELECT and SET clause of UPDATE).
- An element can be deleted using the index.

CQL Data Types List

```
cqlsh:faculty> ALTER TABLE teachers ADD telephone list<text>;

cqlsh:faculty> UPDATE teachers SET telephone = ['+40 21 323.222.21', '+40
721.222.111'] WHERE id = 1;

cqlsh:faculty> UPDATE teachers SET telephone = ['+40 21 323.222.22', '+40
721.222.112'] WHERE id = 2;

cqlsh:faculty> UPDATE teachers SET telephone = ['+40 21 323.222.23', '+40
721.222.113'] WHERE id = 3;

cqlsh:faculty> UPDATE teachers SET telephone = ['+40 21 323.222.23', '+40
721.222.114'] WHERE id = 4;

cqlsh:faculty> UPDATE teachers SET telephone = ['+40 21 323.222.25', '+40
721.222.115'] WHERE id = 5;
```

CQL Data Types List

```
cqlsh:faculty> UPDATE teachers SET telephone = telephone - ['+40 721.222.111']
WHERE id = 1;

cqlsh:faculty> UPDATE teachers SET telephone = telephone + ['+40 21 323.222.30']
WHERE id = 3;

cqlsh:faculty> UPDATE teachers SET telephone = [ ] WHERE id = 5;

cqlsh:faculty> UPDATE teachers SET telephone[0] = '+40 21 323.222.90' WHERE id = 2;

cqlsh:faculty> DELETE telephone[2] FROM teachers  WHERE id = 3;

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE telephone
CONTAINS '+40 21 323.222.23' ALLOW FILTERING;
```

CQL Data Types
Map

- The map data type contains a collection of key/value pairs
- The keys and the values can be of any type except counter
- Collection types are very useful in cases where we need to store a variable number of elements within a single column

CQL Data Types
Map

```
cqlsh:faculty> ALTER TABLE teachers ADD title map<text,text>;

cqlsh:faculty> UPDATE teachers SET title = {'academic': 'teaching assistant',
'profession': 'engineer'} WHERE id = 1;

cqlsh:faculty> UPDATE teachers SET title = {'academic': 'professor',
'profession': 'engineer'} WHERE id = 2;

cqlsh:faculty> UPDATE teachers SET title = {'academic': 'associate professor',
'profession': 'engineer'} WHERE id = 3;

cqlsh:faculty> UPDATE teachers SET title = {'academic': 'lecturer', 'profession':
'engineer'} WHERE id = 4;
```

CQL Data Types
Map

```
cqlsh:faculty> UPDATE teachers SET title = {'academic': 'lecturer',
'profession': 'engineer'} WHERE id = 5;

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE
title['academic'] = 'teaching assistant' ALLOW FILTERING;

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE title
contains 'lecturer' ALLOW FILTERING;

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE title
CONTAINS KEY 'academic' ALLOW FILTERING;
```

CQL Data Types
User Defined Types (UDT)

- Cassandra supports UDTs
- They can be created using CREATE TYPE command
- They can be used on their own or using a collection, where they need to be frozen

```
cqlsh:faculty> CREATE TYPE course(
                    name VARCHAR,
                    year INT
                );
```

CQL Data Types
UDT

```
cqlsh:faculty> ALTER TABLE teachers ADD courses list<frozen<course>>;

cqlsh:faculty> UPDATE teachers SET courses = [{name: 'BD1', year: 3}, {name: 'BD2',
year: 4}, {name: 'DM', year: 5}] WHERE id = 1;

cqlsh:faculty> UPDATE teachers SET courses = [{name: 'BD1', year: 3}, {name: 'BD2',
year: 4}, {name: 'UBD', year: 4}, {name: 'DMDW', year: 5}] WHERE id = 2;

cqlsh:faculty> UPDATE teachers SET courses = [{name: 'BD1', year: 3}, {name: 'BD2',
year: 4}, {name: 'SCAD', year: 4}, {name: 'ICASE', year: 5}] WHERE id = 3;

cqlsh:faculty> UPDATE teachers SET courses = [{name: 'PC', year: 1}, {name: 'ABD',
year: 5}] WHERE id = 4;

cqlsh:faculty> UPDATE teachers SET courses = [{name: 'PAJ', year: 5}, {name: 'MN',
year: 1}] WHERE id = 5;
```

CQL Secondary Indexes

- To create a secondary index use the CREATE INDEX command
- It’s also possible to create indexes that are based on values in collections.
- For optimal read performance: denormalized table designs or materialized views are generally preferred to using secondary indexes.
- Note that for maps in particular, we have the option of indexing either the keys (via the syntax KEYS(addresses)) or the values (which is the default), or both (in Cassandra 2.2 or later).

CQL Secondary Indexes

```
cqlsh:faculty> CREATE INDEX idx_offices ON teachers(offices);

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE offices
CONTAINS 'EF309';

cqlsh:faculty> DROP INDEX idx_offices;



cqlsh:faculty> CREATE INDEX idx_telephone ON teachers(telephone);

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE
telephone CONTAINS '+40 21 323.222.23';

cqlsh:faculty> DROP INDEX idx_telephone;
```

CQL Secondary Indexes

```
cqlsh:faculty> CREATE INDEX idx_title ON teachers(title);

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE title CONTAINS
'lecturer';

cqlsh:faculty> DROP INDEX idx_title;



cqlsh:faculty> CREATE INDEX idx_title_key ON teachers(keys(title));

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE title CONTAINS KEY
'academic' ALLOW FILTERING;

cqlsh:faculty> DROP INDEX idx_title_key;
```

CQL Secondary Indexes

- SASI: A New Secondary Index Implementation
  - The Cassandra 2.4 release included an alternative implementation of secondary indexes known as the SSTable Attached Secondary Index (SASI).
  - SASI was developed by Apple and released as an open source implementation of Cassandra’s secondary index API.
  - As the name implies, SASI indexes are calculated and stored as part of each SSTable file, differing from the original Cassandra implementation, which stores indexes in separate, "hidden" tables.
  - Enable it in Cassandra.yaml: sasi_indexes_enabled: false

CQL Secondary Indexes

- SASI: A New Secondary Index Implementation
  - The SASI implementation exists alongside traditional secondary indexes.
  - SASI indexes do offer functionality beyond the traditional secondary index implementation, such as the ability to do inequality (greater than or less than) searches on indexed columns.
  - You can also use the new CQL LIKE keyword to do text searches against indexed columns.
  - While SASI indexes do perform better than traditional indexes by eliminating the need to read from additional tables, they still require reads from a greater number of nodes than a denormalized design.

CQL Secondary Indexes

- SASI: A New Secondary Index Implementation

```
cqlsh:faculty> CREATE CUSTOM index idx_lastname_sasi ON
teachers(lastname) USING 'org.apache.cassandra.index.sasi.SASIIndex';
```

```
cqlsh:faculty> select firstname, lastname from teachers where lastname like
'Tr%';
```

```
cqlsh:faculty> drop index idx_lastname_sasi;
```

CQL Aggregation

- Built-in functions and aggregates
  - COUNT
    - The COUNT function is used to count the number of rows returned by a query.
    - This command can also be used to count the number of non-null values of a specified column.
  - MIN and MAX
    - The MIN and MAX functions can be used to compute the minimum and maximum value returned by a query for a given column.

CQL Aggregation

- Built-in functions and aggregates
  - SUM
    - The sum function can be used to sum up all the values returned by a query for a given column.
  - AVG
    - The avg function can be used to compute the average of all the values returned by a query for a given column.

CQL Aggregation

- Built-in functions and aggregates

```
cqlsh:faculty> SELECT COUNT(firstname) FROM teachers;

cqlsh:faculty> SELECT MIN(age), MAX(age) FROM teachers;

cqlsh:faculty> SELECT SUM(age) FROM teachers;

cqlsh:faculty> SELECT AVG(age) FROM teachers;
```

CQL UDFs

- User defined functions
  - UDFs are functions that are applied on Cassandra nodes to stored data as part of query processing.
  - Before using UDFs in your cluster, they must be enabled in the cassandra.yaml file on each node:
    - Before v4: enable_user_defined_functions: true
    - Starting with v4: user_defined_functions_enabled: true
  - The 3.0 release added a security feature to run UDF code in a separate sandbox to limit the ability for a malicious function to gain unauthorized access to a node's Java runtime.

CQL UDFs

- UDF Restrictions
  - The code you can write inside a UDF is quite limited
  - It has to be a pure function so no IO, logging, external access of any kind.
  - Can't start threads
  - No dependencies

CQL UDFs

- User defined functions

```
cqlsh:faculty> CREATE FUNCTION age_to_range(input int)
           ... RETURNS NULL ON NULL INPUT
           ... RETURNS varchar
           ... LANGUAGE java AS '
           ...    if (input <= 35) return "young";
           ...    else if (input> 55) return "old";
           ...    else return "mature";
           ... ';

cqlsh:faculty> SELECT firstname, lastname, age_to_range(age) FROM
teachers;
```

CQL UDFs

- Custom aggregates allow you to reduce a query that returns many rows into a single value
- These are called user defined aggregation (UDA)
- For UDAs we need to:
  - Define a function that will be executed, sequentially, for every row, passing some state along
  - Initial state for the first row
  - An optional function that will take in the final state and produce a value

CQL UDFs

- Custom aggregates State function – average:

```
cqlsh:faculty> CREATE OR REPLACE FUNCTION avgState ( state
tuple<int,bigint>, val int ) CALLED
        ... ON NULL INPUT RETURNS tuple<int,bigint> LANGUAGE java AS '
        ...   if (val != null) {
        ...     state.setInt(0, state.getInt(0)+1);
        ...     state.setLong(1,state.getLong(1)+val.intValue());
        ...   }
        ...   return state;
        ... ';
```

CQL UDFs

- Custom aggregates Final function – average:

```
cqlsh:faculty> CREATE OR REPLACE FUNCTION avgFinal ( state
tuple<int,bigint> ) CALLED ON
        ... NULL INPUT RETURNS double LANGUAGE java AS '
        ...   double r = 0;
        ...   if (state.getInt(0) == 0)
        ...     return null;
        ...   r = state.getLong(1);
        ...   r /= state.getInt(0);
        ...   return Double.valueOf(r);
        ... ';
```

CQL UDFs

- Custom aggregates creating the aggregate – average

```
cqlsh:faculty> CREATE AGGREGATE IF NOT EXISTS average ( int )
           ... SFUNC avgState
           ... STYPE tuple<int,bigint>
           ... FINALFUNC avgFinal
           ... INITCOND (0,0);

cqlsh:faculty> SELECT average(age) FROM teachers;
```

CQL UDFs

- Custom aggregation to simulate – unique

```
cqlsh:faculty> CREATE OR REPLACE FUNCTION uniq(state set<int>, val int)
           ... CALLED ON NULL INPUT RETURNS set<int> LANGUAGE java
           ... AS 'state.add(val); return state;';

cqlsh:faculty> CREATE OR REPLACE AGGREGATE uniq(int)
           ... SFUNC uniq STYPE set<int> INITCOND {};


cqlsh:faculty> SELECT uniq(age) FROM teachers;
```

CQL UDFs

- Custom aggregates simulate group by

```
cqlsh:faculty> CREATE OR REPLACE FUNCTION state_group_and_max(state map<int, int>, type
int, amount int)
           ... CALLED ON NULL INPUT
           ... RETURNS map<int, int>
           ... LANGUAGE java AS '
           ...   Integer val = (Integer) state.get(type);
           ...   if (val == null) val = amount;
           ...   else val = Math.max(val, amount);
           ...   state.put(type, val);
           ...   return state;
           ... ';

cqlsh:faculty> CREATE OR REPLACE AGGREGATE state_group_and_max(int, int)
           ... SFUNC state_group_and_max
           ... STYPE map<int, int>
           ... INITCOND {};

cqlsh:faculty> SELECT state_group_and_max(age, id) FROM teachers;
```

Bibliography

- Apache Cassandra Documentation https://cassandra.apache.org/doc/latest/

- Jeff Carpenter, Eben Hewitt. Cassandra: The Definitive Guide: Distributed Data at Web Scale. O'Reilly Media, Inc., 2016.

- Christof Strauch. "NoSQL databases." *Lecture Notes, Stuttgart Media University* 20.24 (2011): 79.

- Eric Redmond, Jim Wilson. Seven databases in seven weeks: a guide to modern databases and the NoSQL movement. 1nd Edition 2012

- Luc Perkins, Jim Wilson, and Eric Redmond. Seven databases in seven weeks: a guide to modern databases and the NoSQL movement. 2nd Edition 2018

University
Politehnica
of Bucharest

Search Engines (I)
Information Retrieval

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@upb.ro

Overview

- Information Retrieval and Search Engines

Information Retrieval

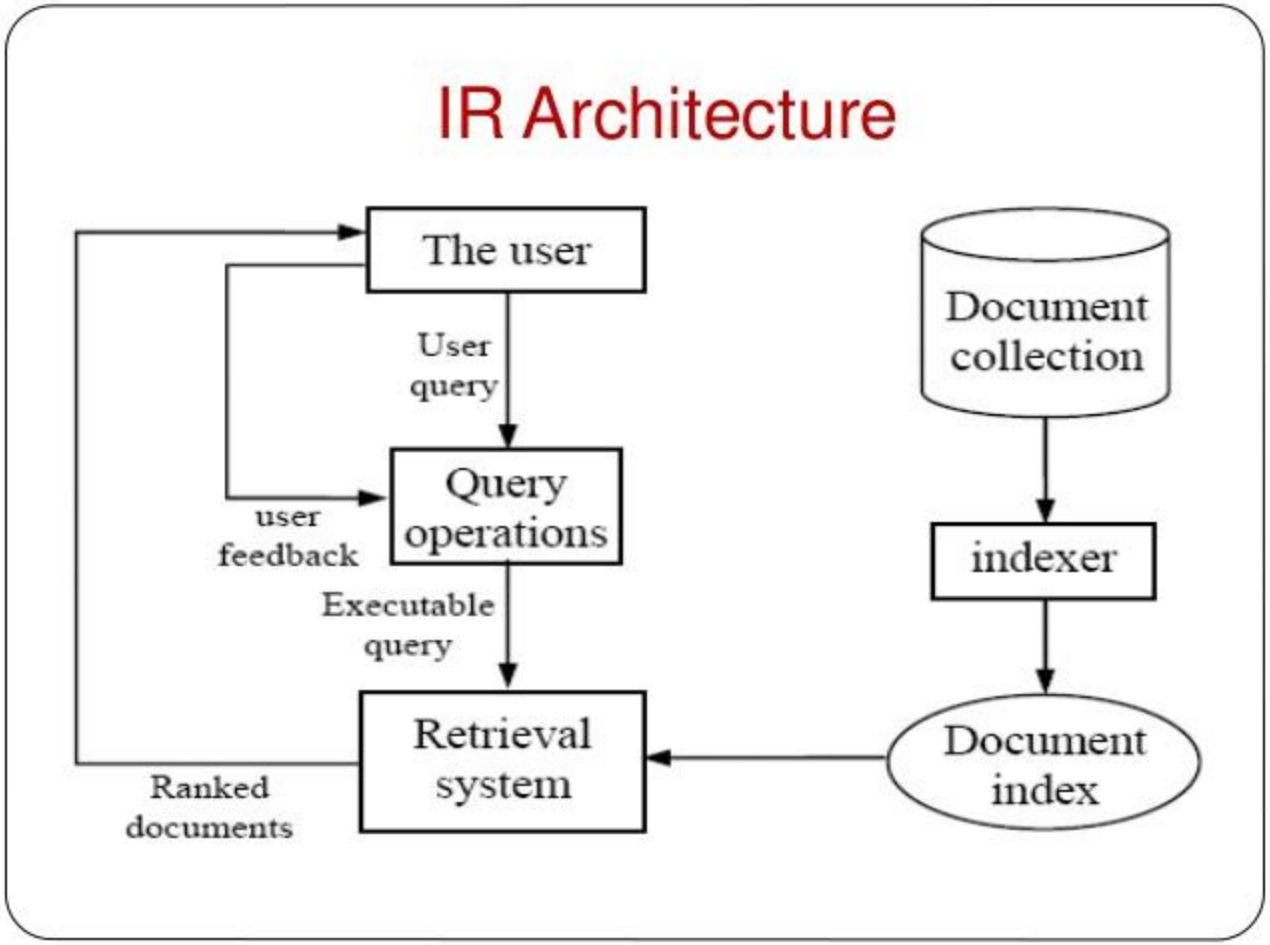
- Information Retrieval (IR) is the field that deals with retrieving relevant information from a large corpus of records given a user search query

- In traditional IR:
  - The records are documents:
  - The search query is a list of words (terms)
  - The output contains the relevant documents for the search query

Information Retrieval and Search Engines
Search Engines

- A **search engine** is an information retrieval system designed to help find information stored on a computer system. (Wikipedia)

Information Retrieval and Search Engines Architecture



Information Retrieval and Search Engines
User Queries

- User Queries types:
  - Keywords queries (search terms – most used): The user uses a list of terms (at list one) with the aim to retrieve documents that contain at least one or all the search terms
    - Soft IR systems – the documents returned contain at least one keyword (logical OR between the terms – added automatically)
    - Hard IR systems – the documents returned contain all the keywords (logical AND between the terms – added automatically)
    - In some IR systems the order of the keywords is also important – returns documents where the list of words appear together
  - Note:
    - Bag of Words (BOW) approach is used when the order is not important
    - N-gram approach is used when the order is important

Information Retrieval and Search Engines
User Queries

- User Queries types:
  - Boolean queries (BOW approach): the user can use Boolean operators in their search queries (AND, OR, and NOT)
    - Example 1: ‘data or web’ or ‘data and web’ – different results
    - Example 2: ‘data or web and not datamining’

Information Retrieval Architecture
User Queries

- User Queries types:
  - Phrase queries (n-gram approach):
    - Such a query consists of a sequence of words that make up a phrase
    - The documents returned must contain at least one instance of this phrase
  - Proximity queries (skipped-gram approach)
    - Is a relaxed version of the phase query and can be a combination of terms and phrases
    - These queries seek the documents that contain the search term in close proximity of each other

Information Retrieval and Search Engines
User Queries

- User Queries types:
  - Full document queries
    - Users search for documents that are similar to the query document
  - Natural language queries
    - This is the most complex case
    - The user asks a question and the IR system returns an answer to the that question
    - Used in question-answering systems (e.g., chatbots)

Information Retrieval and Search Engines Architecture

- Query operation module
  - Can range between
    - Very Simple: just passes the query to the retrieval system
    - Complex: does preprocessing
- Indexer module
  - Used to index the original raw documents in some data structures
  - The data structures enables efficient retrieval
  - Most common is the inverted index

Information Retrieval and Search Engines Architecture

- Retrieval system module:
  - Computes a relevance score (for the user query) for each retrieved documents
  - According the relevance the documents are ranked and presented to the user

- Document collection module:
  - A File System (FS), e.g., OSFS, HDFS, etc.
  - A database (Relational or NoSQL)

Information Retrieval and Search Engines
Retrieval Models

- The way in which terms and documents are represented governs the IR model

- There are three basic document representations:
  - Boolean model
  - Vector Space model
  - Statistical Language model

- Advance document representation:
  - Document Embeddings, e.g., Doc2Vec

Information Retrieval and Search Engines
Document Representations

- A document is represented using the bag of words (terms) model

- Given a set (collection) of n documents:

$$D = \{d_1, d_2, \dots, d_n\}, n = |D|$$

- All the distinct terms in the collection of documents can be modeled as a vocabulary:

$$V = \{t_1, t_2, \dots, t_m\}, m = |V|$$

Information Retrieval and Search Engines
Document Representations

- The entire corpus of documents can be represented as a matrix (document-term matrix) where:
  - The lines represent the document in D
  - The columns represent the terms in V

- Each cell in the matrix is a weight (w_{ij}) associated for the number of occurrences of a term in the document.

- A document $d_i \in D$ is modeled as a vector $d_i = \{w_{i1}, w_{i2}, \ldots, w_{jm}\}$ where each w_{ij} is the weight associated to the term $t_j \in V$

Information Retrieval and Search Engines
Document Representations

- The document-term matrix is:

$$\begin{array}{c} \\ d_1 \\ d_2 \\ \vdots \\ d_n \end{array} \begin{array}{c} \begin{array}{cccc} t_1 & t_2 & \cdots & t_m \end{array} \\ \begin{pmatrix} w_{11} & w_{12} & \cdots & w_{1m} \\ w_{21} & w_{22} & \cdots & w_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n1} & w_{n2} & \dots & w_{nm} \end{pmatrix} \end{array}$$

Information Retrieval and Search Engines
Weighting Schemas

- Boolean model
  - This is one of the simples models used to represent the weights in the document-term matrix
  - This model only considers if a term is present or not in a document, $w_{ij} \in \{0, 1\}$:

$$w_{ij} = \begin{cases} 1 & if\ t_j\ appears\ in\ d_i \\ 0 & otherwise \end{cases}$$

  - The document-term matrix will only contain 1s and 0s

Information Retrieval and Search Engines
Weighting Schemas

- Boolean model
  - This model is used for Boolean queries
  - For example, given 3 terms $x, y\ and\ z$:
    - $(x\ AND\ y)\ AND\ (NOT\ z)$ says that a document must contain both x and y but not the term z
    - $x\ OR\ z$ says that a document must contain at least one of the terms x or z

Information Retrieval and Search Engines
Weighting Schemas

- Vector space model
  - One of the best known and widely used IR models
  - Each weight is computed as a variation of the number of occurrences of the term in the document or collection of documents

Information Retrieval and Search Engines
Weighting Schemas

- Vector space model most widely used representation:
  - $f_{t,d}$ the frequency of the term t in the document d is computed by counting
  - $TF(t,d)$ term frequency, the normalization of $f_{t,d}$
  - $TFIDF(t,d,D)$ term frequency – inverted document frequency
  - $Okapi\ BM25$ (BM stand for Best Match)
    - Okapi is the name of the information system proposed by Robertson, Spärck Jones, et al.

Information Retrieval and Search Engines
Weighting Schemas

- $TF(t,d)$ term frequency is a normalization of the $f_{t,d}$

- Logarithmic normalized $TF(t,d)$:

$$TF(t,d) = \log\left(1 + f_{t,d}\right)$$

- This has a bias towards longer documents because in longer documents terms that are irrelevant can appear multiple times, thus, they have a higher $TF(t,d)$

Information Retrieval and Search Engines
Weighting Schemas

- To removes the bias towards longer documents, the augmented $TF(t,d)$

$$TF(t,d) = K + (1-K) \cdot \frac{f_{t,d}}{\max\limits_{t' \in d} f_{t',d}}$$

- For $K = 0.5$ (determined empirically) we obtained the double normalization form of the $TF(t,d)$:

$$TF(t,d) = 0.5 + 0.5 \cdot \frac{f_{t,d}}{\max\limits_{t' \in d} f_{t',d}}$$

Information Retrieval and Search Engines
Weighting Schemas

- Another way to removes the bias towards longer documents is normalize the frequency of the term with the length of the document

$$TF(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

Information Retrieval and Search Engines
Weighting Schemas

- The $IDF(t, D)$ is a measure of how much information a term provides:
  - Weather a term is common or rare across all documents
  - n is the number of documents in D
  - n_t is the number of documents where term t appears:

$$IDF(t, D) = 1 + \log \frac{n}{n_t}$$

- $TFIDF(t, d, D)$ is used to determine the importance of a term for a document in a corpus of documents:

$$TFIDF(t, d, D) = TF(t, d) \cdot IDF(t, D)$$

Information Retrieval and Search Engines
Weighting Schemas

- Okapi BM25 is another weighting schema used to compute the weights for the document-term matrix

$$OkapiBM25(t,d,D) = \frac{TFIDF(t,d,D)\cdot(k_1+1)}{TF(t,d)+k_1\cdot\left(1-b+b\cdot\frac{|d|}{avg_{d'\in D}(|d'|)}\right)}$$

- Where:
  - The free parameters values are:
    - $k_1 \in [1.2, 2.0]$
    - $b = 0.75$
  - $|d|$ is the length of the document
  - $avg_{d'\in D}(|d'|)$ is the average document length

Information Retrieval and Search Engines
Ranking Functions

- Ranking functions
  - Given a search query $Q = (q_1, q_2, \dots, q_k)$ where q_i are the query terms ($i = \overline{1,k}, k = |Q|$),
  - The objective is to retrieve the documents that contain the terms in the search query Q
  - We need to give a score that measures the importance of each document to the query
  - Then order the list of documents using this score and create a documents ranking

Information Retrieval and Search Engines
Ranking Functions

- The score for a document $d \in D$ is the sums of the weights ($TFIDF$, $OkapiBM25$) computed for each search term in query Q

$$TFIDF_{score}(Q, d, D) = \sum_{i=1}^{k} TFIDF(q_i, d, D)$$

$$OkapiBM25_{score}(Q, d, D) = \sum_{i=1}^{k} OkapiBM25(q_i, d, D)$$

- Using one of these scores, the documents' relevance to the query is determined

- Statistical Language Model
  - Are based on probability and have foundations in statistical theory
  - The input is
    - A set of n documents $D = \{d_1, d_2, \dots, d_n\}$
    - A query $Q = (q_1, q_2, \dots, q_k)$
  - We consider a query Q as being "generated" by a probabilistic model based on a document d_i
  - Using the Bayes rule, the rank of documents are estimated by the posterior probability $p(d_i|q)$:

$$p(d_i|Q) = \frac{p(Q|d_i) \cdot p(d_i)}{p(Q)}$$

- Statistical Language Model
  - For ranking, $p(Q)$ is not needed, as is the same for all the documents
  - The model assumes that each term is independently generated, which is essentially a multinomial distribution over words, and so:

$$p(Q = (q_1, q_2, \dots, q_k)|d_i) = \prod_{j=1}^{k} p(q_j|d_i) = \prod_{j=1}^{m} p(t_j|d_i)^{\delta}$$

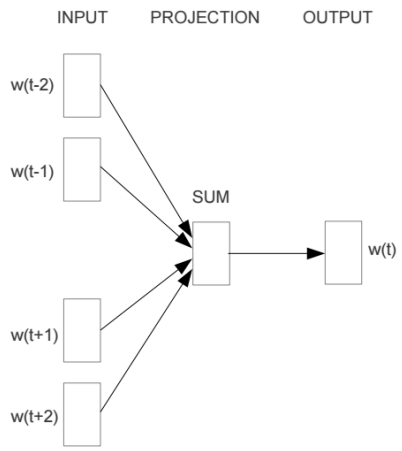
  - Where $\delta = \begin{cases} 1 & if\ q_j\ is\ present\ in\ d_i \\ 0 & otherwise \end{cases}$
  - So, the retrieve problem is reduced to estimating $p(t_j|d_i)$ $(t_j = q \in Q)$:

$$p(t_j|d_i) = \frac{f_{ij}}{|d_i|}$$

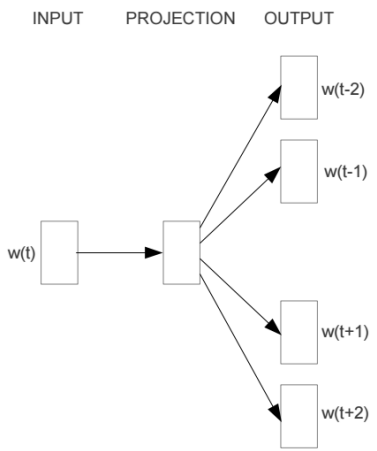
  - Where
    - f_{ij} is actually the frequency of word t_j in document d_i, f_{t_j,d_i}
    - $|d_i|$ is the length of document d_i

Information Retrieval and Search Engines
Other representation models

- Word representation models:
  - Word Embeddings:
    - Word2Vec Skip-Gram (SG)
    - Word2Vec Continuous Bag-of-Words (CBOW)
    - FastText SG
    - FastText CBOW
    - GloVe
  - Transformer Embeddings
    - BERT, RoBERTa, BART, etc.



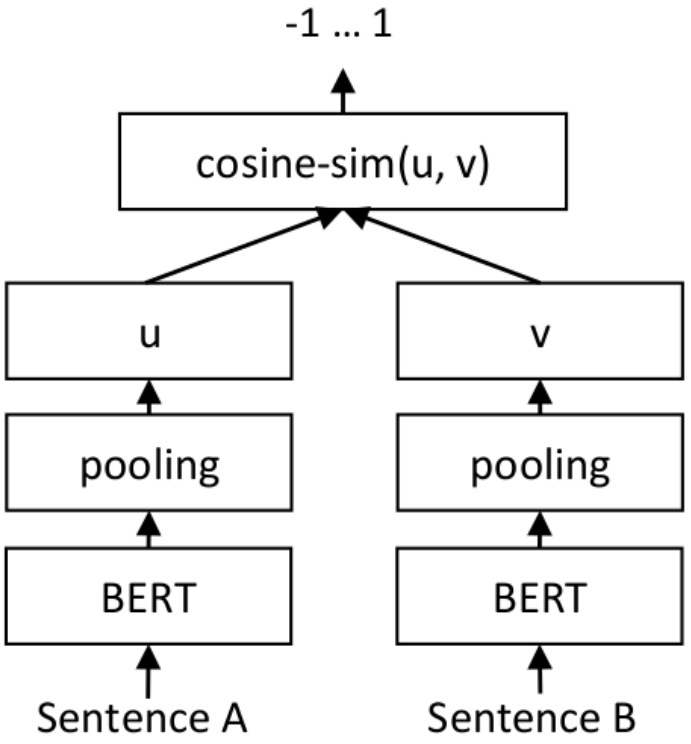
CBOW



Skip-gram

Information Retrieval and Search Engines
Other representation models

- Document representation models:
  - Sentence/Document Embeddings
  - Sentence Transformers



Information Retrieval and Search Engines
Text Preprocessing

- Before the documents in a collection are used for retrieval some preprocessing steps are usually performed

- These steps may include:
  - Expanding contractions
  - Sentence tokenization
  - Extracting terms
  - Removing stop words and punctuation

- Expanding contractions, i.e., shortened versions of the written and spoken forms of a word, syllable, or word group, created by omission of internal letters and sounds

- Sentence tokenization is the process by which the text of a document is split in sentences.
  - This step can be skipped if the terms are not processed further

Information Retrieval and Search Engines
Text Preprocessing

- Extracting terms is the process by which each term is determined
  - Sometimes it is useful to extract the stem or the lemma of a word (not useful for opinion mining and sentiment analysis)
  - To extract the lemma sentence tokenization is used and also the part of speech tagger

- Extracting term – stemming
  - In many languages, a word has various forms depending on the context, e.g., verbs have the gerund form (-ing termination) or nouns can be singular or plural
  - These variations cause low retrieval because a relevant document can contain variation of a word but not the exact word
  - By removing the suffixes and prefixes of a word we can obtain the stem of the word
  - Stemming is the process of extracting the stems
  - The stems sometimes are not accurate, e.g., the words “cops” and “cope” are both reduce to “cop”

- Extracting term – lemmatization
  - A more accurate way of extracting the root word is by extracting the lemma, this process is called lemmatization
  - Lemmatization uses the part of speech of a word, the process that extracts the part of speech is called Part of Speech Tagging (PoS)
  - This is a very costly process, but it increases the accuracy of the retrieval

- Removing stop words and punctuation
  - Stop words are frequently occurring and insignificant words in a language, e.g.. the, a, an, etc.
  - These can be removes together with punctuation as they add no significate information to the process of information retrieval
  - For some tasks are important, e.g., opinion mining and sentiment analysis.

Information Retrieval and Search Engines
Inverted Index

- The main purpose of IR is to retrieve documents given a search query
- One approach is to scan the collection of documents and to return the documents that match the query terms.
  - This method is actually impractical for a large collection of documents
- A second approach is to build a data structure (inverted index) that maps each word to the documents where it appears.

Information Retrieval and Search Engines
Inverted Index

- How to construct an inverted index
  1. Given a collection of documents $D = \{d_1, d_2, \dots, d_n\}$, $n = |D|$
  2. Attach to each document an unique identifier $\{id_1, id_2, \dots, id_n\}$ where id_i is the unique identifier for document d_i
  3. Construct the vocabulary
  4. Attach to each word in the vocabulary the list of documents that contain the word

Information Retrieval and Search Engines
Inverted Index

- In practice, an inverted index is a dictionary with the key the word and the value a list of documents.

$$IV_i = \{id_j | \, t_i \in d_i\}$$

Information Retrieval and Search Engines
Inverted Index

- Sometimes, additional information can be stored in the inverted index, e.g.:
  - The term frequency of the word in the document
  - A list with positions, etc.

$$IV_i = \left\{ < id_j, f_{ij}, \left[o_1, o_2, \ldots, o_{|d_j|} \right] > | t_i \in d_i \right\}$$

- Where:
  - f_{ij} is the frequency of term t_i in document d_i
  - $o_k, k = \overline{1, |d_i|}$ is the position of term t_i in document d_i

Information Retrieval and Search Engines
Inverted Index

- Example:

id_1: I am learning about inverted indexes.

id_2: Inverted indexes are used in information retrieval.

id_3: Applications that retrieve documents use inverted indexes.

- The vocabulary is:
  $\{learning, inverted, indexes, information, retrieval, applications, retrieve, documents\}$

- Notes:
  - the stop words were removed: I, am, about, are, used, in, that, use
  - Stemming or lemmatization can be applied, in this case it wasn't.

Information Retrieval and Search Engines
Inverted Index

- Simple Inverted Index

$$learning: [id_1],$$
$$inverted: [id_1, id_2, id_3],$$
$$indexes: [id_1, id_2, id_3],$$
$$information: [id_2],$$
$$retrieval: [id_2],$$
$$applications: [id_3],$$
$$retrieve: [id_3],$$
$$documents: [id_3]$$

Information Retrieval and Search Engines
Inverted Index

- Complex Inverted Index:

$$learning\colon [< id_1, 1, [3] >],$$
$$inverted\colon [< id_1, 1, [5] >, < id_2, 1, [1] >, < id_3, 1, [6] >],$$
$$indexes\colon [< id_1, 1, [5] >, < id_2, 1, [2] >, < id_3, 1, [6] >],$$
$$information\colon [< id_2, 1, [5] >],$$
$$retrieval\colon [< id_2, 1, [7] >],$$
$$applications\colon [< id_3, 1, [1] >],$$
$$retrieve\colon [< id_3, 1, [3]],$$
$$documents\colon [< id_3, 1, [4] >]$$

Information Retrieval and Search Engines
Inverted Index

- Searching with inverted indexes:
  - Given a search terms query each term is searched in the index and a concatenated list of all the document unique identifiers without duplicates is returned
  - Example:
    - For the search query ‘applications information documents’ the following documents are going to be returned:$\{id_3, id_2\}$
    - If a ranking function is used, then document d_3 will have a bigger weight that document d_2

References

- C.D. Manning, P. Raghavan, H. Schütze. Introduction to Information Retrieval. Cambridge University Press. 2008.

University
Politehnica
of Bucharest

Search Engines (II)
Apache Lucene

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@upb.ro

Overview

- Apache Lucene
- Apache Lucene Architecture

Overview

- Apache Lucene
- Apache Lucene Architecture

Apache Lucene

- Apache Lucene™ is an open-source search software project
- The project releases are
  - Lucene™ core - the core search library written in Java
  - PyLucene - a python binding for Lucene.
- Lucene Core is a Java library providing
  - Powerful indexing
  - Search features
  - Spellchecking
  - Highlighting
  - Advanced analysis/tokenization capabilities
- PyLucene provides Python bindings for Lucene Core.

Apache Lucene

- Applications that can be developed using Apache Lucene:
  - Document analytics: Loading and traversing text documents based on some given criteria, finding the top terms from documents, aggregating on specific fields.
  - Log analytics: Analyzing application logs with high-performance dashboards built directly on top of Lucene.
  - Geospatial search: With the recent advent of geospatial queries and data structures, Lucene is fast becoming a popular choice for indexing latitudinal and longitudinal data and queries such as colocation searches.
  - Vector Database – improvements added by Elastic.co to make Elasticsearch a retrieval platform for Retrieval Augmented Systems (RAG)

- Two major search-based application have spawn from Lucene:
  - Apache Solr
  - Elasticsearch

Apache Lucene Features

- Text/Language Analysis
  - The analysis is used to create appropriate internal representations for
    - Document to be indexed
    - Queries for searching documents
  - During indexing, tokens are created and stored in Lucene's Inverted Index
  - At query time, tokens are created to form appropriate query representations

Apache Lucene Features

- Text/Language Analysis
  - The analysis consists in the following 3 steps:
    - Optional character filtering and normalization (e.g., removing diacritics)
    - Tokenization
    - Token filtering (e.g., stemming, lemmatization, stopword removal, n-gram creation)
  - This feature is used by Lucene for modeling documents

Apache Lucene Features

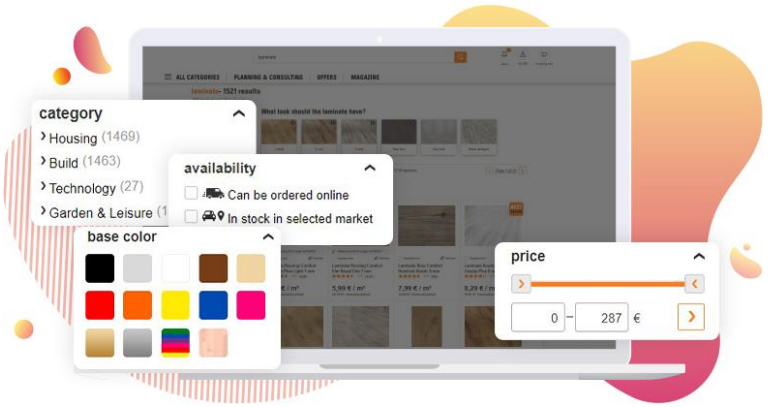
- Indexing and Storage
  - Used to index and store user defined documents
  - High degree of concurrency given by the lock-free indexing
  - Near real time indexing: documents become searchable as soon as they are indexed
  - Transactional support for additions and rollbacks
  - Different scoring models for terms, documents, and data sets

Apache Lucene Features

- Querying
  - Query options include the ability to filter, paginate, sort results, and perform relevance feedback
  - Multiple query representations, query parsers, and a query parsing framework for developer defined query parsers
  - Pluggable scoring model system for developers to implement their own
  - Predefined scoring models: traditional vector-space scoring model, Okapi BM25, Language Modeling, etc.

Apache Lucene Features

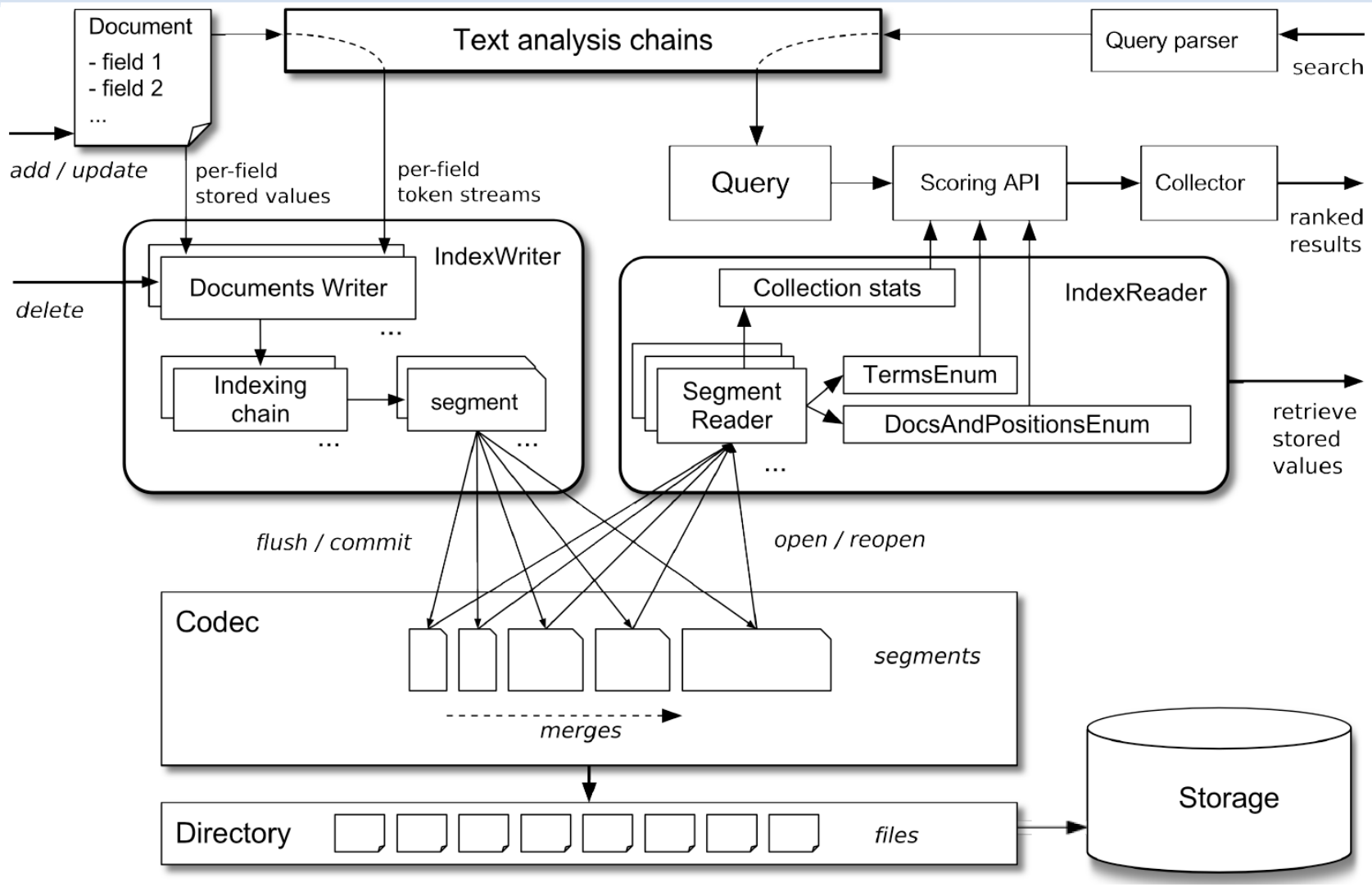
- Developer libraries
  - Used for building search-based applications
  - Example libraries:
    - Result highlighting (snippet generation)
    - Faceting (show part of the analysis result) and Faceted Search (technique that involves augmenting traditional search techniques with a faceted navigation system)
    - Spatial search
    - Document grouping by key (e.g., group all documents with the same base URL together)
    - Document routing (via an optimized, in-memory, single document index)
    - Point-based spatial search
    - Auto-suggest



Overview

- Apache Lucene
- Apache Lucene Architecture

Apache Lucene Architecture



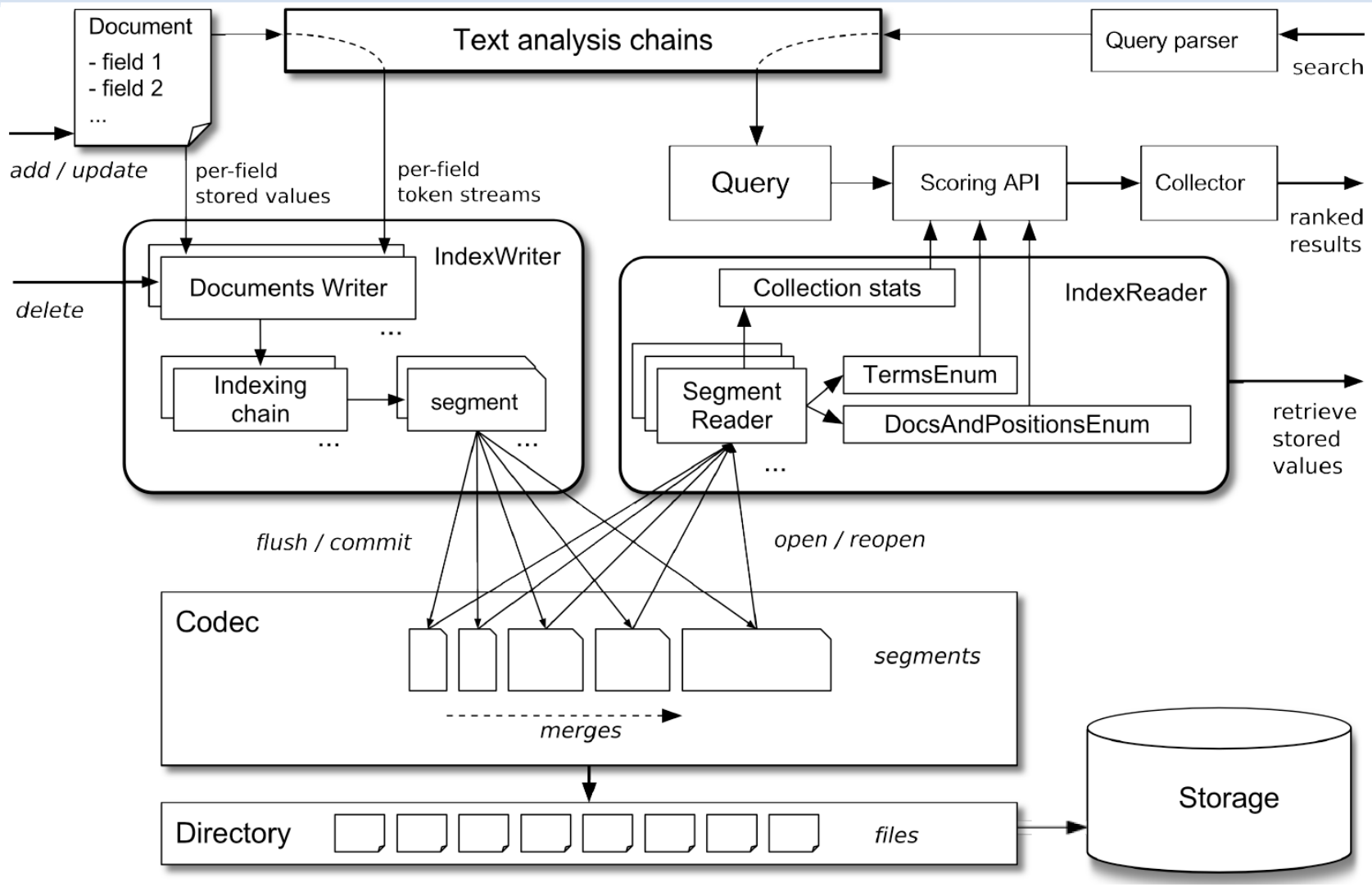
Apache Lucene Architecture

- Text Analysis
  - The text analysis chain produces a stream of tokens from the input data
  - Tokens are represented as a collection of attributes:
    - Term
    - Token position
    - Starting and ending offsets
    - Token type
    - Arbitrary payload data (a byte array to be stored in the index at the current position)
    - Integer flags
    - Other custom application-defined attributes (e.g., part-of-speech tags)

Apache Lucene Architecture

- Text Analysis
  - The text analysis chains consist of
    - Character filters (e.g., diacritics striping)
    - Tokenizers (which are the sources of token streams)
    - Series of token filters that modify the original token stream
  - Token streams perform specific functions such:
    - Tokenization by patterns, rules, and dictionaries (e.g., whitespace, regex, Chinese / Japanese / Korean)
    - Specialized token filters for efficient indexing of numeric values and dates (to support numerical range searching)
    - Language-specific stemming and stop word removal
    - Creation of character or word-level n-grams
    - Tagging, etc.

Apache Lucene Architecture

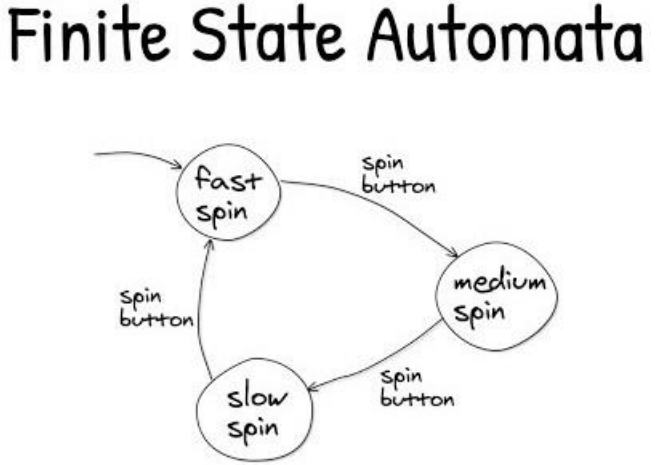


Apache Lucene Indexing Process

- Indexing
  - Lucene uses the well-known inverted index representation, with additional functionality for keeping adjacent non-inverted data on a per-document basis.
  - Both in-use and persistent data uses a variety of encoding schemas that affect the size of the indexed data and the cost of the data compression
  - Lucene uses pluggable mechanisms for data encoding (i.e., Codec API) and for the actual storage of index data (i.e., Directory API).
  - Incremental updates are supported and stored in index extents (i.e., segments) that are periodically merged into larger segments to minimize the total number of index parts

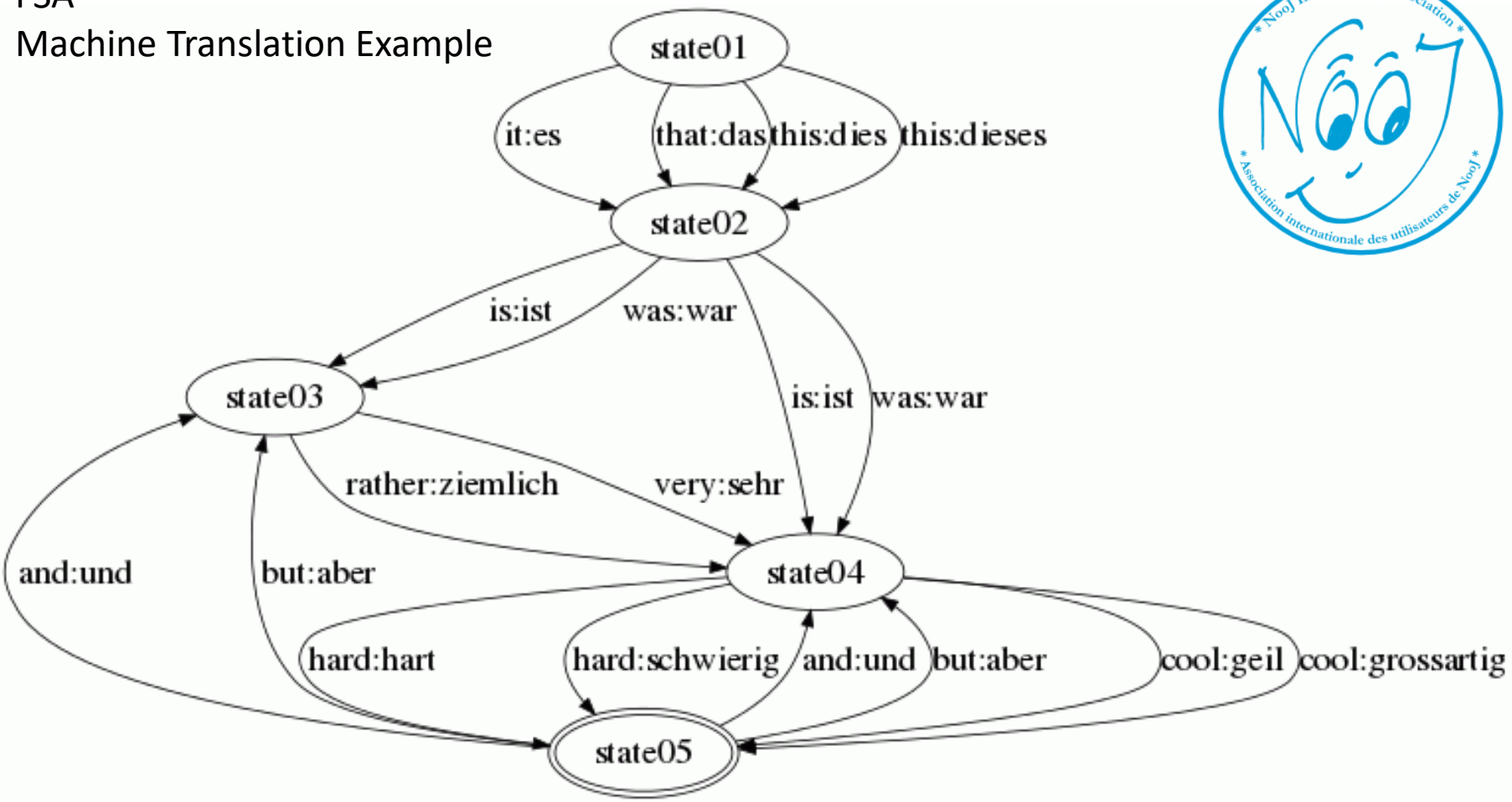
Apache Lucene Indexing Process

- Finite State Automata
  - The in-memory portion of the inverted index is implemented using a Finite State Transducer (FST)
  - An FST is a type of Finite State Automaton (FSA) that maps between two sets of symbols using two memory tapes:
    - Read a set of strings on the input tape
    - Generates a set of relations on the output tape



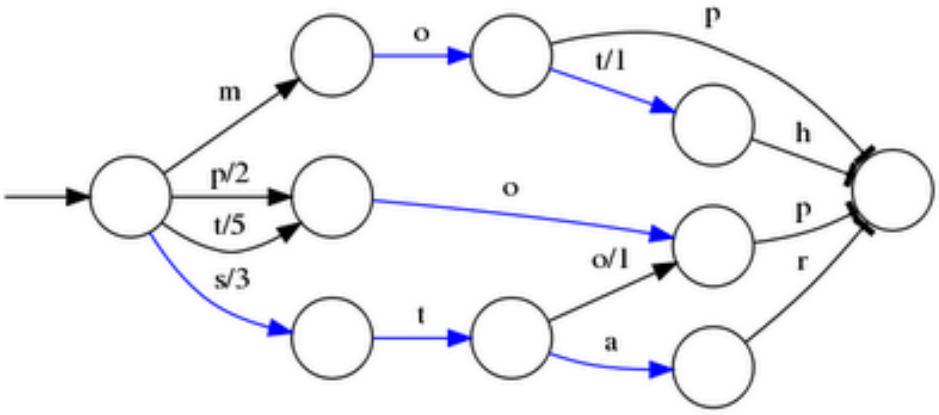
Apache Lucene
Indexing Process

FSA
Machine Translation Example



Apache Lucene Architecture

- The Finite State Transducer (FST) supports
  - Linear time construction of the minimal automaton
  - Compression
  - Reverse lookups
  - Weighted automata (used in NLP to assign weights to words and sentences)
  - Pluggable output algebras
  - Synonym processing
  - Spell correction
  - Auto-suggest/complete
  - Machine translation



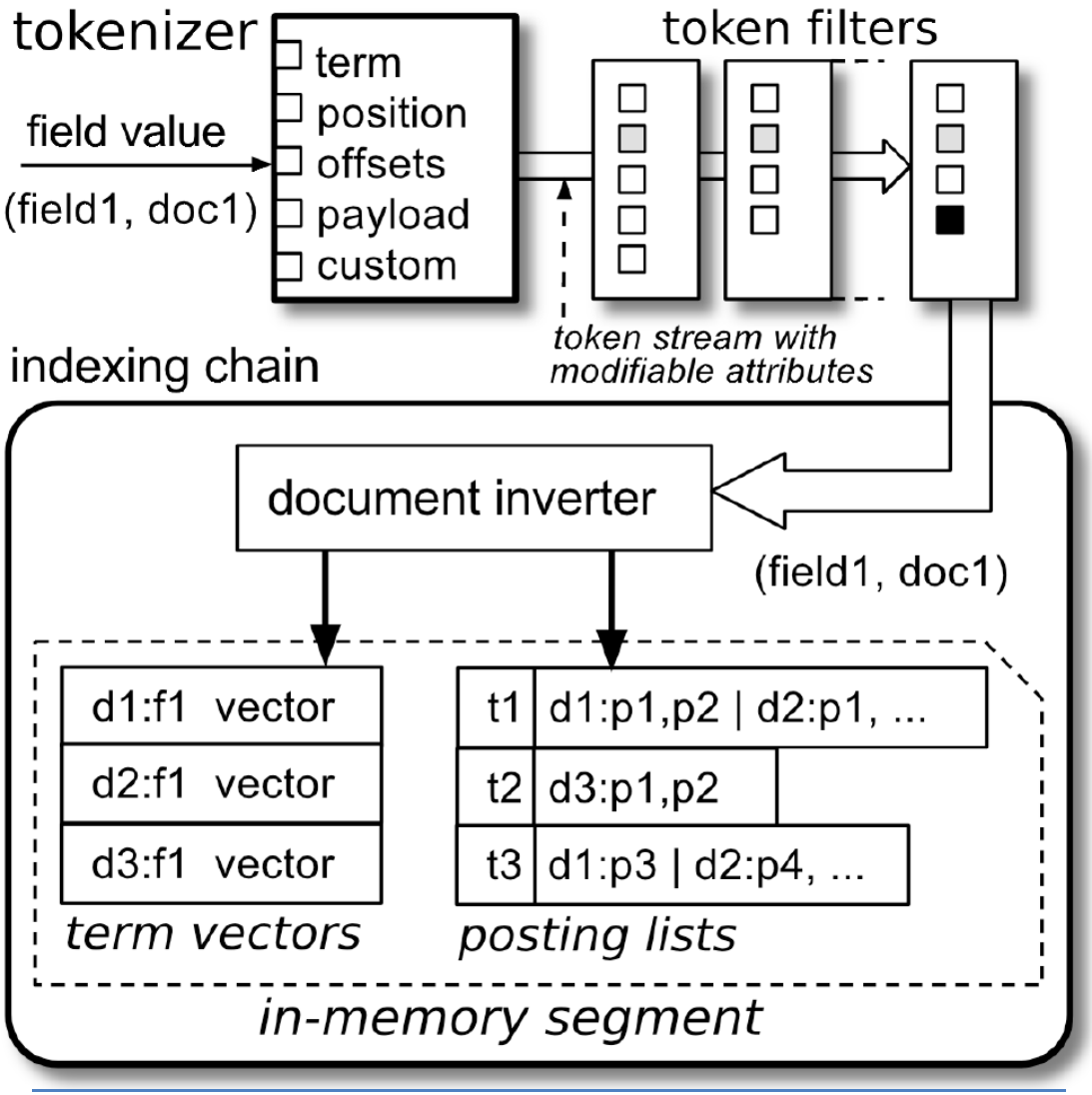
Apache Lucene Indexing Process

- Document Model
  - Documents
    - Are modeled in Lucene as a flat ordered list of fields with content
    - Are not required to have a unique identifier (though they often carry a field with this role for application-level unique key lookup)
    - During the indexing process, documents are assigned internal integer identifiers
  - Fields have a name, content data, a float weight (used later for scoring), and other attributes, depending on their type, which together determine how the content is processed and represented in the index
  - There can be multiple fields with the same name in a document, in which case they will be processed sequentially

Apache Lucene Indexing Process

- Field Types
  - There are two broad categories of fields in Lucene documents
  - Indexed fields contain content to be inverted
  - Stored fields contain content to be stored
- Fields may belong to either or both categories (e.g., with content both to be stored and inverted).
- Both fields can be submitted for storing / indexing
  - Only stored fields can be retrieved
  - Indexed/Inverted data can be accessed and traversed

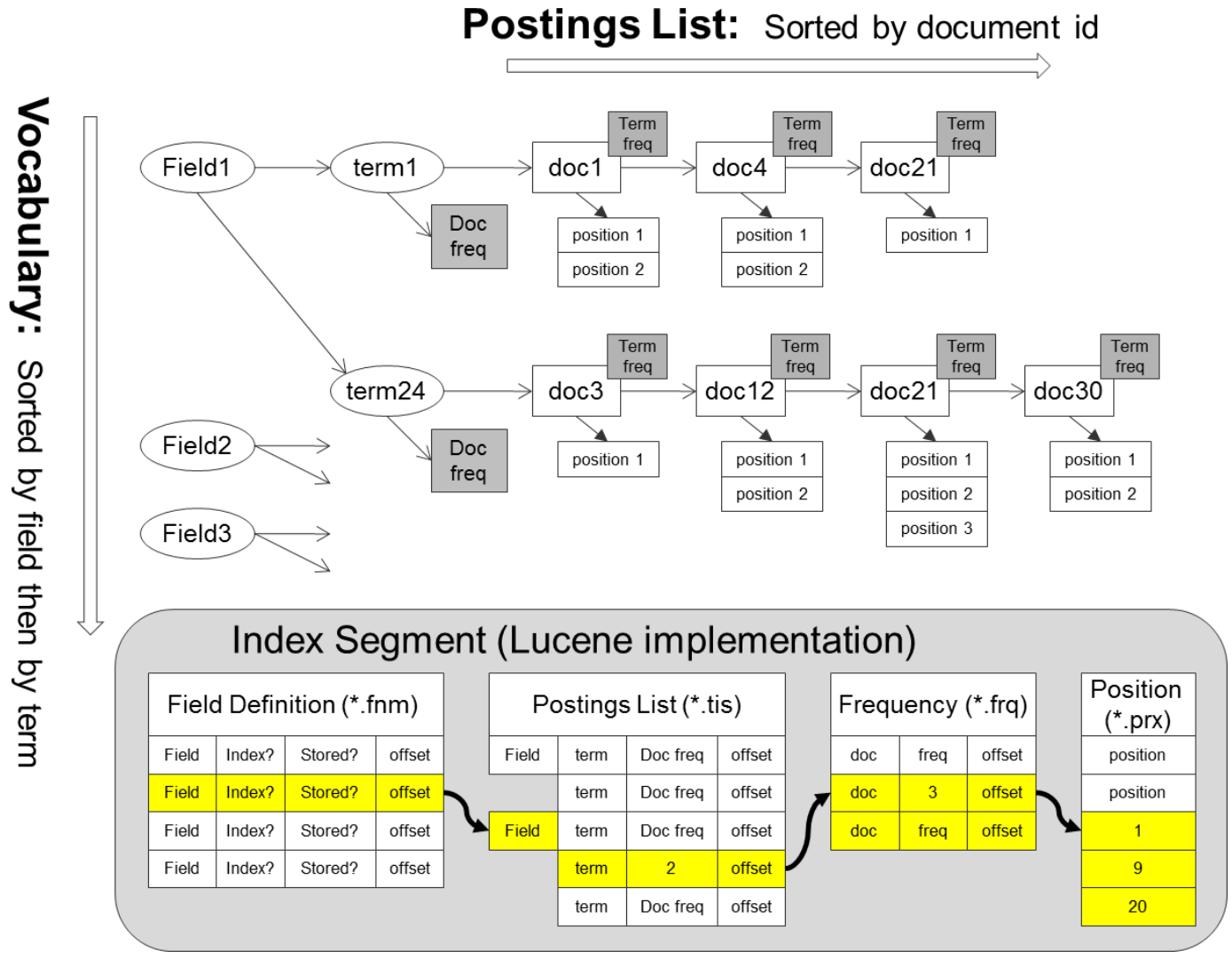
Apache Lucene
Indexing Process



Apache Lucene Indexing Process

- Indexed fields can be
  - Provided in plain text, in which case it will be first passed through the text analysis pipeline to obtain token streams
  - The final form of a sequence of tokens with attributes (i.e., token streams)
- Token streams
  - Are then inverted and added to in-memory segments, which are periodically merged and flushed to disk
  - Depending on the field options, various token attributes (such as positions, starting / ending offsets and per-position payloads) are also stored with the inverted data.
  - It is possible to omit positional information while still storing the in-document term frequencies on a per-field basis
  - Use the Finite State Transducer for their in-memory representation

Apache Lucene Indexing Process



Apache Lucene Indexing Process

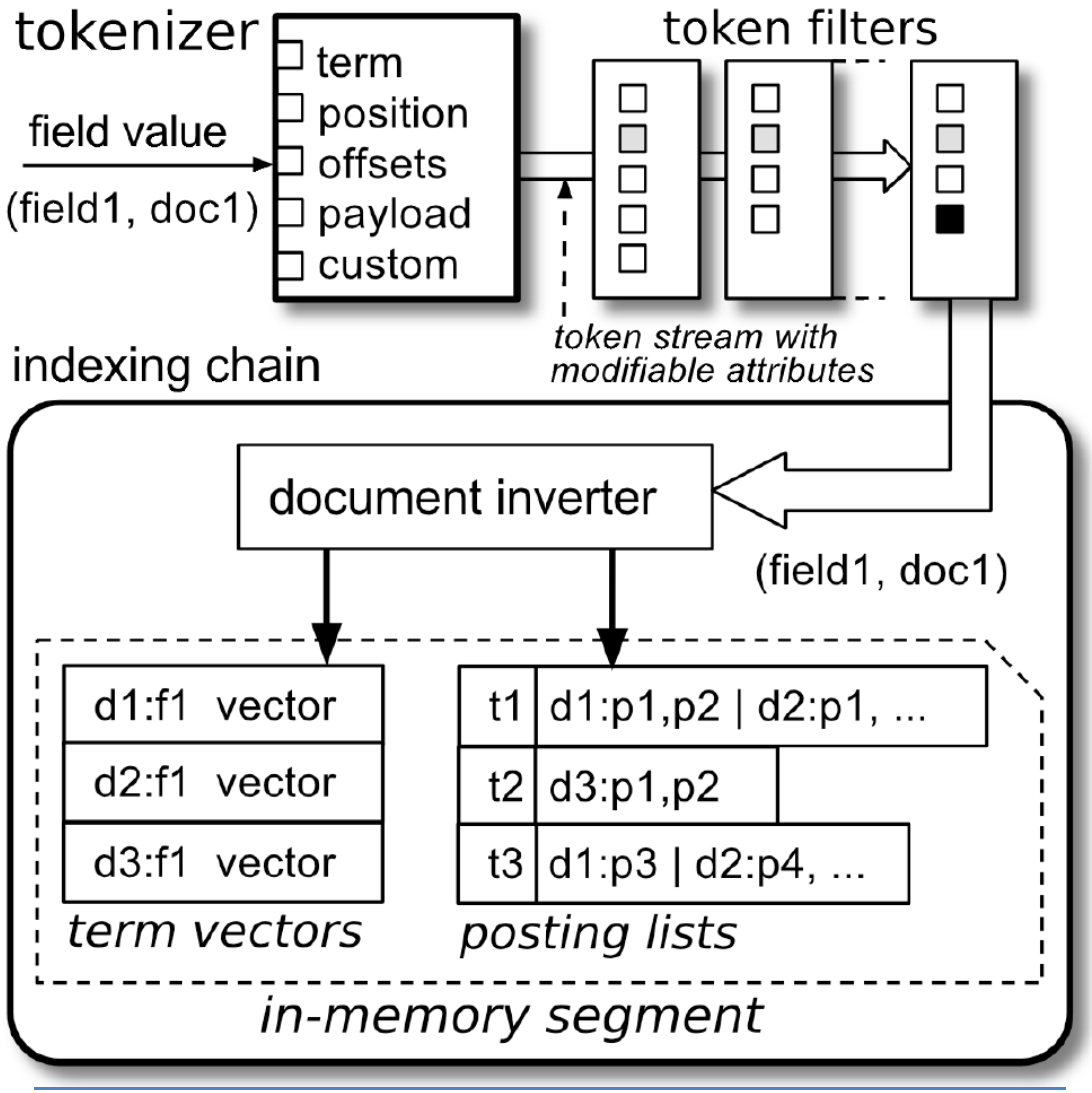
- A variant of an indexed field is a field where the creation and storage of term frequency vectors is requested
  - In this case the token stream is used also for building a small inverted index consisting of data from the current field only
  - This inverted data is then stored on a per-document and per-field basis
  - Term frequency vectors are particularly useful when performing document highlighting, relevance feedback or when generating search result snippets (region of text that best matches the query terms)

Apache Lucene Indexing Process

- Stored fields are typically used for storing auxiliary per-document data that is not searchable but would be cumbersome to obtain otherwise (e.g., it would require retrieval from a separate system)

- The per-document data is stored as byte arrays
  - Can be manipulated to be encoded using UTF-8 strings, numbers, arrays etc.
  - Optionally, it can be stored using strongly typed API (so called “doc values”) that can use a more optimized storage format, i.e., strongly typed storage
  - This kind of strongly typed storage is used for example to store per-document and per-field weights (so called “norms”, as they typically correspond to field length normalization factor that affects scoring)

Apache Lucene
Indexing Process



Apache Lucene Indexing Process

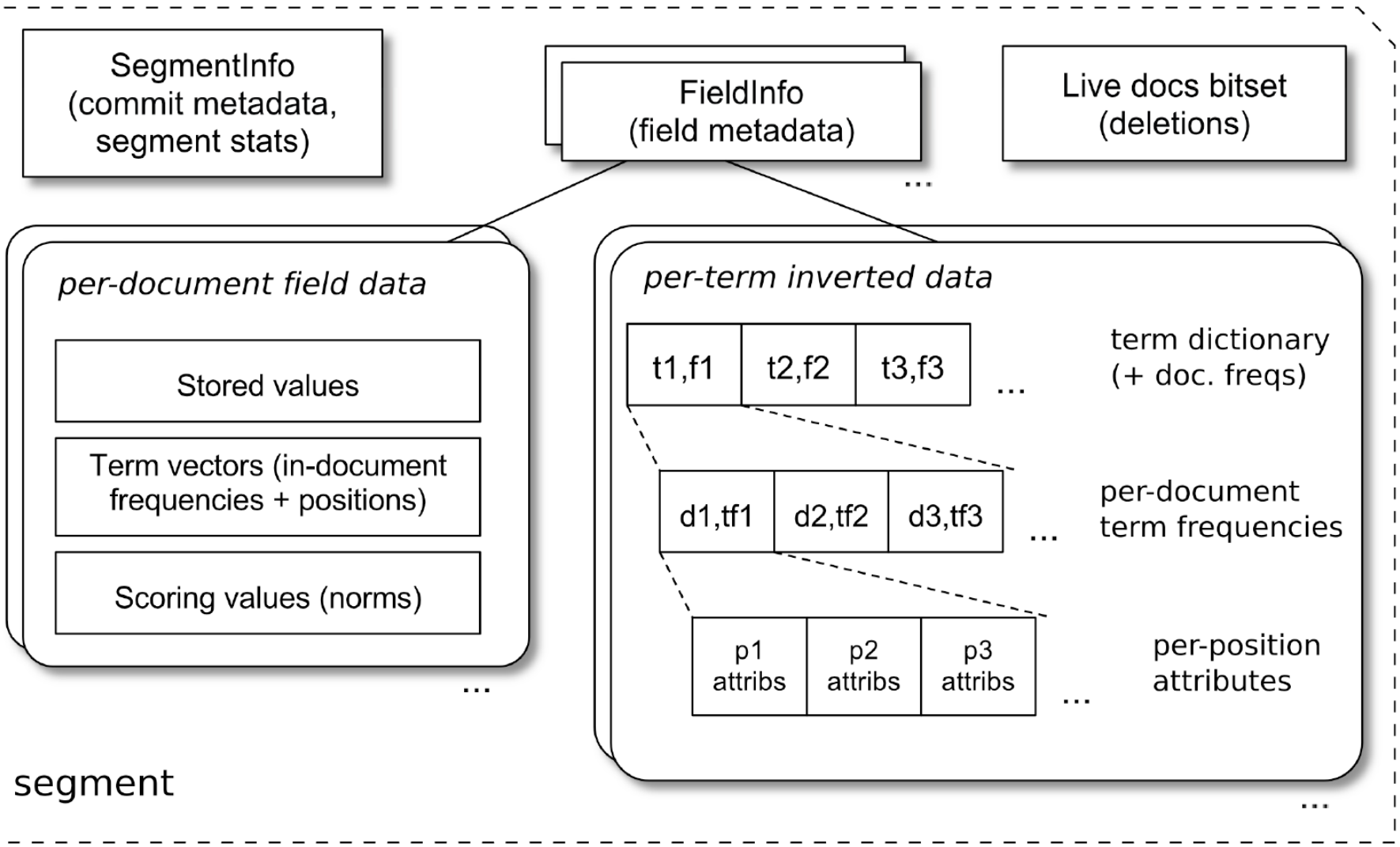
- Indexing Chain
  - The resulting token stream
    - Is finally processed by the indexing chain
    - The supported attributes (term value, position, offsets and payload data) are added to the respective posting lists for each term
    - Posting list - a set of document unique indentifiers (ids) and offsets associated with each term in an inverted index
  - Term values
    - Can be arbitrary byte array values as terms
    - Can use custom comparators to define the sorting order of such terms
  - At this stage documents are assigned their internal document identifiers, i.e., small sequential integers (for efficient delta compression)
  - These identifiers
    - Are used for identifying document data within a particular segment
    - Change after two or more segments are merged (during index compaction)

Apache Lucene Indexing Process

- Incremental Index Updates
  - Indexes can be updated incrementally on-line
    - Simultaneously with searching
    - By adding new documents and/or deleting existing ones (sub-document updates are a work in progress)
  - Index extents
    - Are a common way to implement incremental index updates
    - Do not require modifying the existing parts of the index

Apache Lucene Indexing Process

- When new documents are submitted for indexing
  - It passes through the indexing chain
  - The resulting inverted and non-inverted data is accumulated in new in-memory index extents called segments
  - The segment use a compact in-memory representation (a variant of Codec)
  - Periodically, these in-memory segments are flushed to a persistent storage (using the Codec API and Directory API abstractions)
  - The flushing appears whenever a configurable threshold is reached, e.g., the total number of documents, the size in bytes of the segment, etc.



Apache Lucene Segment

SegmentInfo (commit metadata, segment stats)

FieldInfo (field metadata)

...

Live docs bitset (deletions)

per-document field data

Stored values

Term vectors (in-document frequencies + positions)

Scoring values (norms)

...

per-term inverted data

t1,f1 | t2,f2 | t3,f3 | ...

term dictionary (+ doc. freqs)

d1,tf1 | d2,tf2 | d3,tf3 | ...

per-document term frequencies

p1 attribs | p2 attribs | p3 attribs | ...

per-position attributes

...

segment

Apache Lucene Indexing Process

- The Indexer Writer
  - The Index Writer is a high-level (Java) class responsible for
    - Processing index updates (additions and deletions)
    - Recording them in new segments and creating new commit points
    - Triggering the index compaction (segment merging)
  - It uses a pool of DocumentWriter-s that create new in-memory segments
  - As new documents are being added and in-memory segments are being flushed to storage, periodically an index compaction (merging) is executed in the background that reduces the total number of segments that comprise the whole index

Apache Lucene Indexing Process

- The Indexer Writer
  - Document deletions
    - Are Queries that select the documents to be deleted
    - Are accumulated:
      - Applied to the in-memory segments before flushing (while they are still mutable)
      - Recorded in a commit point so that they can be resolved when reading the already flushed immutable segments

Apache Lucene Indexing Process

- The Indexer Writer
  - The commit point:
    - Is created at each flush operation or index compaction
    - Is recorded in a global index structure using a two-phase commit
    - Is a list of segments and deletions comprising the whole index at the point in time when the commit operation was successfully completed
  - At the commit point, the segment data that is being flushed from in-memory segments is encoded using the configured Codec implementation

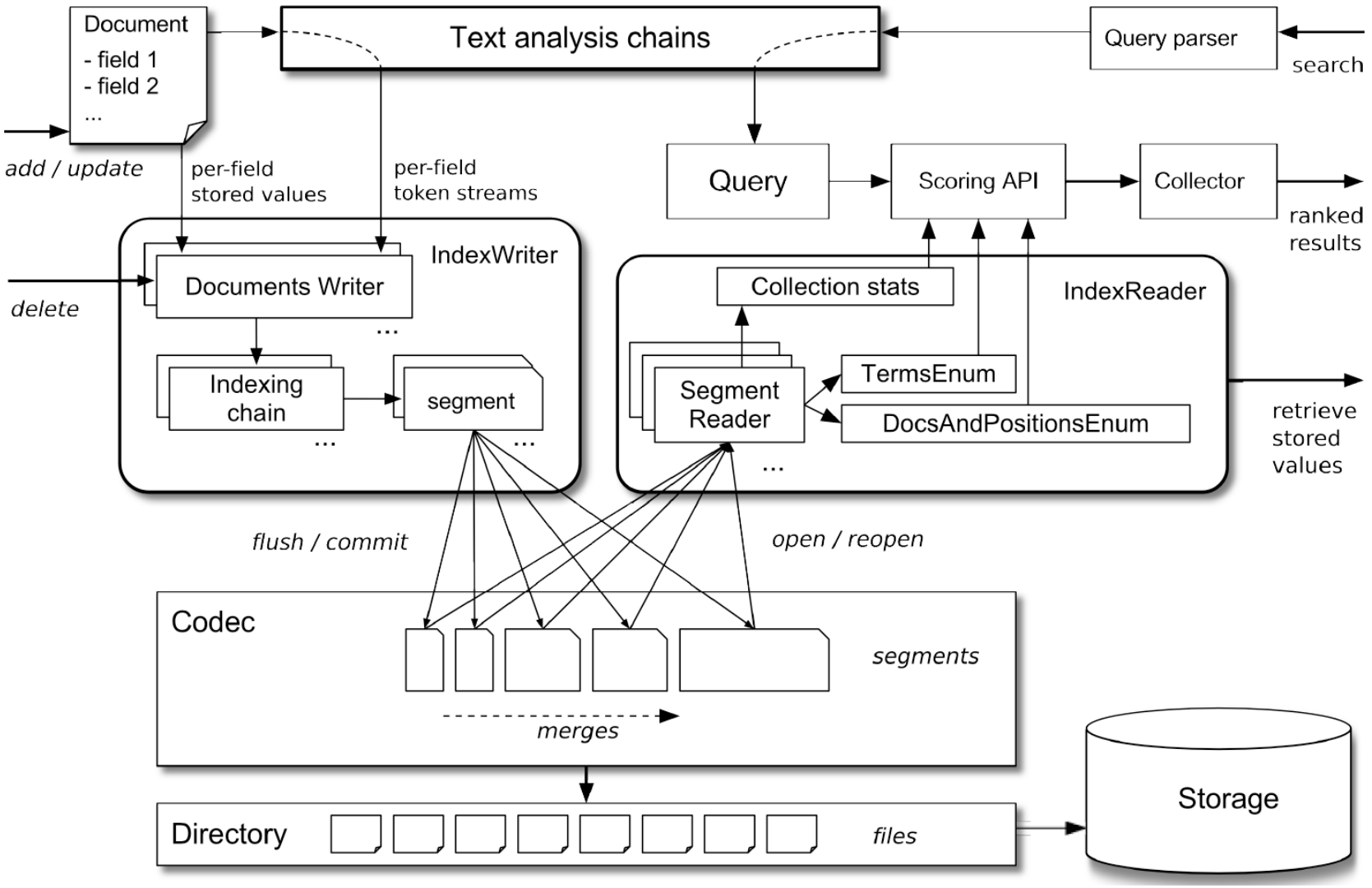
Apache Lucene Indexing Process

- The Indexer Writer
  - The segment data
    - Are fully immutable (write-once)
    - Any changes are expressed either as
      - New segments
      - New lists of deletions
    - Both types of changes create new commit points
  - The updated view of the latest version of the index becomes visible when a commit point is recorded using a two-phase commit
    - This enables lock-free reading operations concurrently with updates
    - Point-in-time travel by opening the index for reading using some existing past commit point

Apache Lucene Indexing Process

- As segments are immutable
  - Deletions do not actually remove data from existing segments
  - Instead, the delete operations are resolved when existing segments are open
  - Thus, the deletions are represented as a bitset of live (not deleted) documents
  - This bitset is then used when enumerating posting lists and stored fields and during search to hide deleted documents.
  - Global index statistics are not recalculated, so they are slightly wrong (still include term statistics from deleted documents)
  - For performance reasons the data of deleted documents is actually removed only during segment merging and the global index statistics are recalculated

Apache Lucene Querying

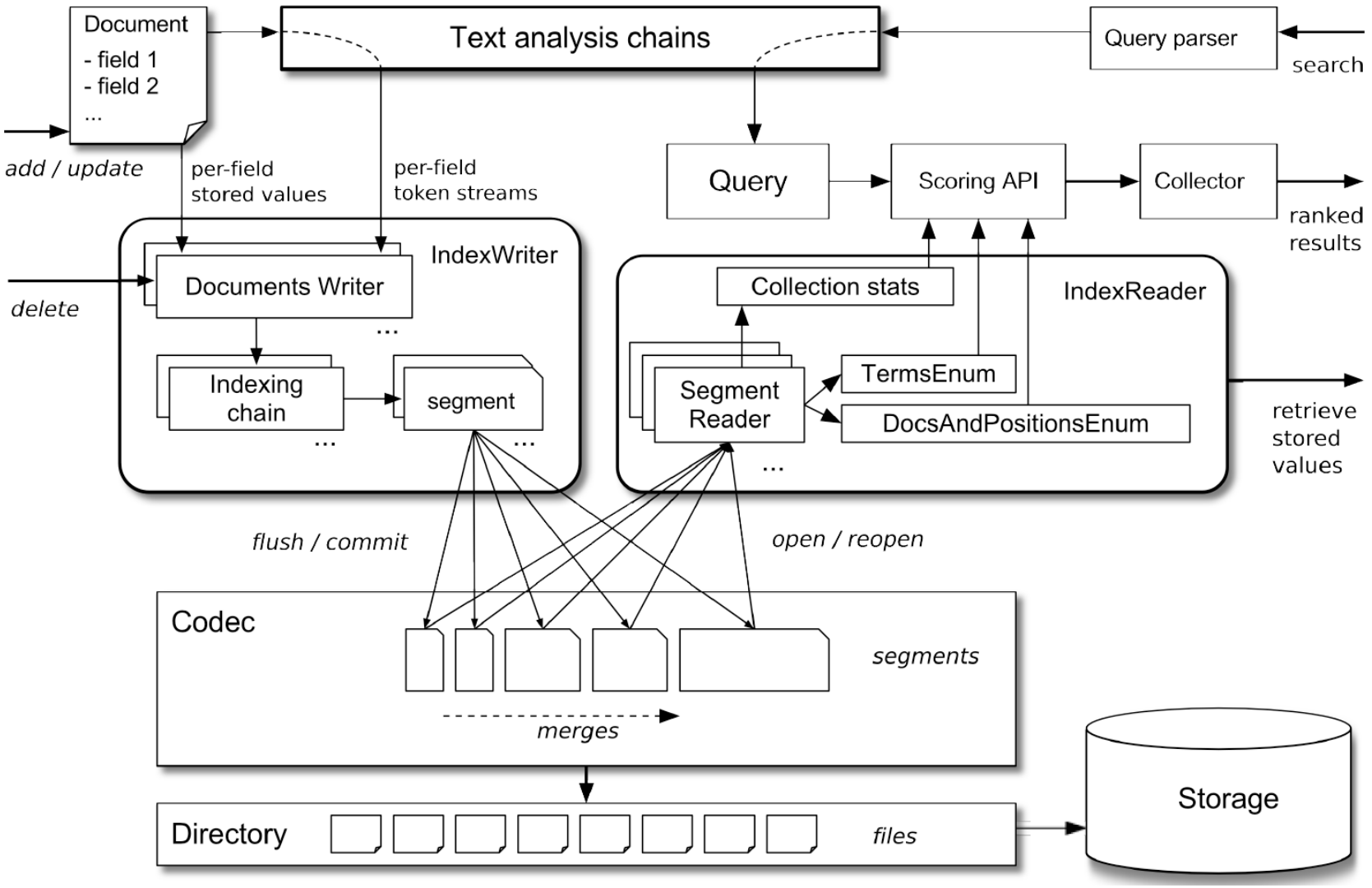


Apache Lucene
Querying

- The Indexer Reader
  - Provides high-level methods to retrieve stored fields, term vectors, and to traverse the inverted index data
  - Uses the Codec API to retrieve and decode the index data
  - Represents the view of an index at a specific point in time

- A user obtains information using an Index Reader from
  - A commit point when all data has been written to disk
  - Directly from Index Writer: a near real-time snapshot that includes both the flushed and the in-memory segments

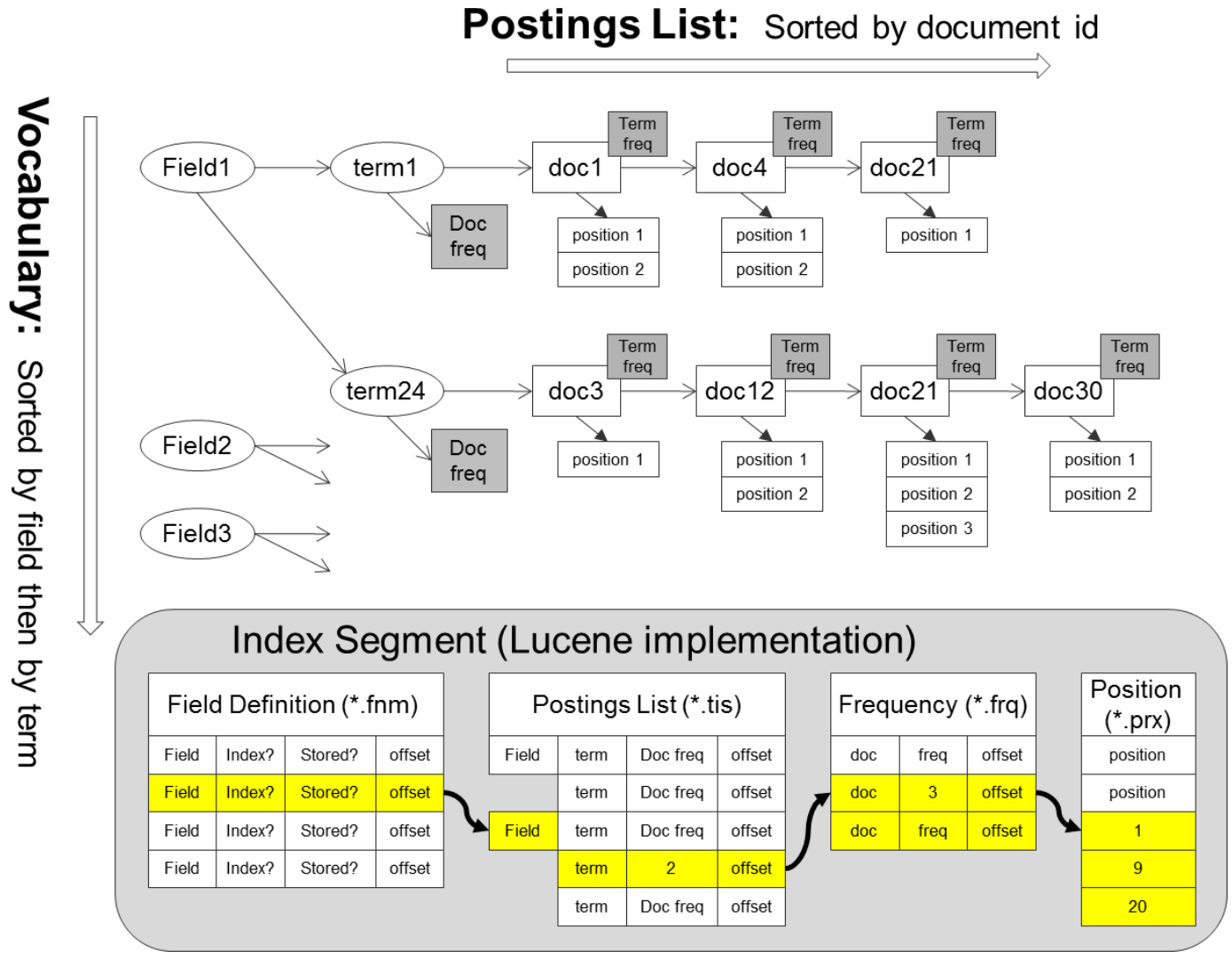
Apache Lucene Codec API



Apache Lucene Codec API

- The Codec API deals with
  - Index data encoding for writing in the storage
  - Index data decoding for reading
  - Segment data merging
- A 4-D View of the Inverted Index
  - The Codec API encapsulates the inverted index data as a logical 4-dimensional table that can be traversed using enumerators
  - The dimensions are: field, term, document, and position
  - Using a cursor, we can advance along rows and columns of this table in each dimension

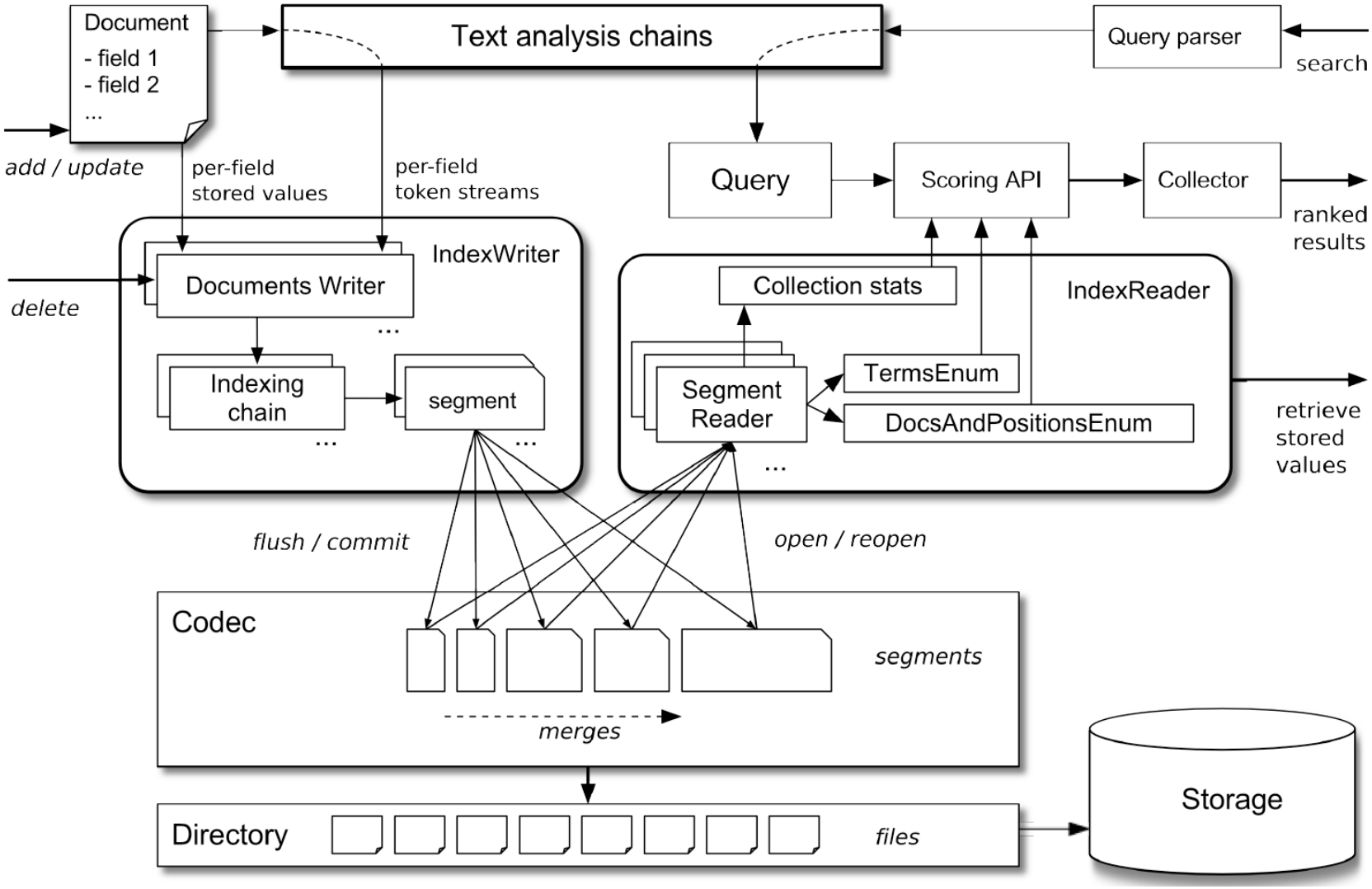
Apache Lucene Indexing Process



Apache Lucene Codec API

- Lucene Codecs API
  - The default codec implementation uses a combination of well-known compression algorithms and strategies selected to provide a good tradeoff between index size (and related costs of I/O seeks) and coding costs
  - The codes offer good balance between high performance indexing and fast execution of queries.

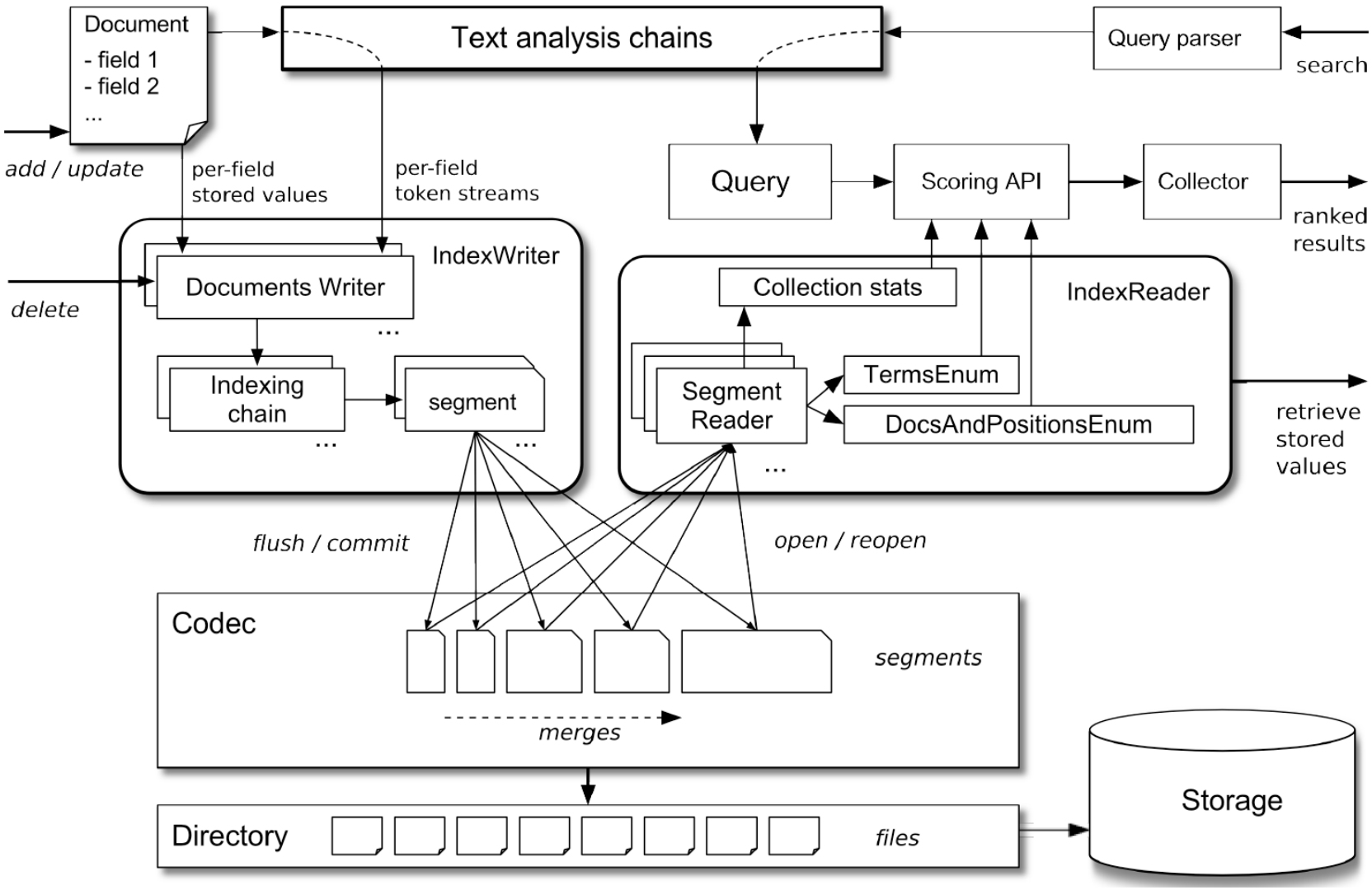
Apache Lucene Directory



Apache Lucene
Directory API

- Directory API
  - The physical I/O access is abstracted using the Directory API
  - The Directory API offers a very simple file system-like views of the persistent storage
  - The Lucene Directory is basically a flat list of files
  - Files are write-once
  - Abstractions are provided for sequential and random access for writing and reading of files

Apache Lucene Searching



Apache Lucene
Searching

- Searching can be broken down as follows
  - Query model
  - Query types
  - Query evaluation
  - Scoring
  - Common search extensions

Apache Lucene
Query Model

- Query model
  - Lucene does not enforce a particular query language
  - Query objects are used to perform searches
  - Several Queries are provided as building blocks to express complex queries
  - Developers can construct their own query language programmatically or via a Query Parser.

Apache Lucene
Query Types

- Query types used in Lucene:
  - Term queries that evaluate a single term in a specific field
  - Boolean queries
  - Proximity queries
  - Position-based queries that allow to express more complex rules for proximity and relative positions of terms
  - Wildcard, fuzzy and regular expression queries that use automata for evaluating matching terms
  - Disjunction-max query that assigns scores based on the best match for a document across several fields
  - Payload query that processes per-position payload data, etc.

Apache Lucene
Query Functions & Search Queries

- Function queries
  - Lucene also supports the incorporation of field values into scoring, i.e., function queries
  - Function queries can be used to add useful scoring factors like time and distance into the scoring model

- Search query are either
  - Parsed by a Query Parser into a Query tree
  - Generated and combined programmatically

Apache Lucene
Query Evaluation

- Query evaluation
  - At Query execution time, the Scoring API
    - Processed each inverted index segment sequentially for efficiency
    - Calculates a score for each document
  - Documents are scored using one of two strategies:
    - Document-at-a-Time (DAAT)
    - TAAT (Term-at-a-Time) only when the number of terms is low.
  - A Collector is responsible for actually consuming these Scorers and doing something with the results, for example populating a priority queue of the top-N documents

Apache Lucene
Scoring

- Scoring is done using a Similarity class that implements a policy for scoring terms and queries

- When computing a score, the Similarity class considers
  - Term and global index statistics
  - Query specifics (e.g., distance between terms of a phrase, Levenshtein edit distance of fuzzy terms, etc.)

- Lucene scoring uses well-known scoring models: TFIDF, BM25, etc.

Apache Lucene
Common Search Extensions

- Common Search Extensions
  - Lucene provides extended query processing capabilities to support easier navigation of search results:
    - Rollup/drilldown capabilities using the Faceting module
    - Result grouping supports folding related documents into a single combined result.
    - Support for nested documents, query expansion, and geospatial search

References

- A. Białecki, R. Muir, G. Ingersoll. Apache Lucene 4. SIGIR 2012 workshop on open source information retrieval, p. 17. 2012.
- M. McCandless, E. Hatcher, O. Gospodnetic. Lucene in Action (2nd Edition). Maning. 2010
- A. Sharma. Practical Apache Lucene 8. Apress. 2020
- Apache Lucene Documentation https://lucene.apache.org/core/10_1_0/index.html